با کاتلین

توسعه اَپ‌های اندرویدی

KOTLIN-BASED ANDROID APPLICATION DEVELOPMENT

در این کتاب
ما برنامه‌های اندرویدی می‌نویسیم و آنچه را که می‌نویسیم درک می‌کنیم.

```
if (resultCode != Activity.RESULT_OK) {
        Log.i(TAG, "Skipping notification.")
        Toast.makeText(context,
            "$TAG",Toast.LENGTH_LONG)
        return
    }
```


- بهرام آملی
- b.amoli1796@gmail.com

مقدمه

به عنوان یک برنامه‌نویس مبتدی اندروید با مسیر یادگیری شیب‌داری روبرو هستید. یادگیری اندروید مانند قدم گذاشتن به یک سرزمین تازه است. جایی که فرهنگ و قواعد خاص خودش را دارد. حتی اگر به زبان این سرزمین مسلط بوده و بتوانید به راحتی صحبت کنید، باز هم خود را یک شهروند بومی نمی‌دانید. همه آن چیزهایی را که از دست داده‌اید درک می‌کنید. تمام اطلاعاتی که قبلا به آن آگاه بودید در زمینه جدید کاملا اشتباه است و کاربردی ندارد.

اندروید فرهنگ مختص به خود را دارد. این فرهنگ به کاتلین یا جاوا (یا کمی از هر دو) صحبت می‌کند، اما دانستن کاتلین یا جاوا کافی نیست. درک شما از اندروید مستلزم یادگیری بسیاری از ایده‌ها و تکنیک‌های جدید است. داشتن یک راهنما در این قلمرو ناآشنا بسیار ضروری به نظر می‌رسد. نقطه ورود ما همین جاست. ما شما را از طریق نوشتن چندین اپلیکیشن اندروید، معرفی مفاهیم و تکنیک‌ها مورد نیاز راهنمایی خواهیم کرد. وقتی به نقاط ناهموار می‌رسید، یا زمانی که برخی موارد مشکل یا مبهم هستند، کنار شما خواهیم بود و تمام تلاش خود را برای روشن کردن محیطی که در آن قرار گرفته‌اید انجام خواهیم داد.

در این کتاب، ما معتقدیم برای اینکه یک برنامه‌نویس اندروید باشید، باید:

- برنامه‌های اندرویدی بنویسید
- آنچه را که می‌نویسید درک کنید

این کتاب به شما کمک می‌کند تا هر دو را انجام دهید. این رویکرد به شما این امکان را می‌دهد که به جای یادگیری تئوری‌های زیاد، آنچه را که آموخته‌اید بلافاصله در یک برنامه کاربردی عملی کنید. شما تجربه و درکی که برای شروع به کار به عنوان یک توسعه دهنده اندروید نیاز دارید، به دست خواهید آورد.

پیش نیازها

برای استفاده از این کتاب باید با زبان برنامه‌نویسی کاتلین از جمله کلاس‌ها و آبجکت‌ها، رابط‌ها، شنونده‌ها، پکیج‌ها، کلاس‌های داخلی و کلاس‌های جنریک آشنا باشید. گرچه این مفاهیم را به طور مختصر و کاربردی ارایه خواهیم داد اما تسلط شما به این موضوع می‌تواند کار را تسریع بخشد.

اگر این مفاهیم برای شما نامفهوم باشد، در میان جزئیات غرق خواهید شد. جای نگرانی نیست ما شما را با این مفاهیم پایه آشنا خواهیم کرد و همانند یه نجات غریق به کمک‌تان خواهیم آمد، البته به شرطی که فصل اول را به خوبی یاد بگیرید.

اما اگر با مفاهیم برنامه‌نویسی شی‌گرا بیگانه نیستید، احتمالا مشکلی نخواهید داشت و می‌توانید فصل اول را به سرعت پشت سر بگذارید. ما در طول کتاب توضیحات مختصری در مورد ویژگی‌های کاتلین ارائه خواهیم داد. اما در صورت نیاز به پشتیبانی بیشتر در حین مرور کتاب، یک مرجع کاتلین را در دسترس داشته باشید.

نحوه استفاده از این کتاب

این کتاب یک کتاب مرجع نیست. هدف آن عبور دادن شما از مراحل اولیه است تا بتوانید از کتاب‌های مرجع و دستورالعمل‌ها بیشترین بهره را ببرید. به همین خاطر قرار است دنباله‌ای مرتبط و فصل‌هایی به هم گره خورده داشته باشیم.

نحوه سازماندهی این کتاب

همانطور که روی این کتاب کار می‌کنید، هفت برنامه اندروید به شرح لیست پایین توسعه خواهیم داد. تعدادی بسیار ساده هستند و فقط یک فصل برای ایجاد آن طول می‌کشد. برخی دیگر پیچیده و طولانی می‌باشند. همه آنها به گونه‌ای طراحی شده‌اند که مفاهیم و تکنیک‌های مهم را به شما آموزش دهند و تجربه مستقیمی در استفاده از آنها در اختیارتان قرار دهند. لیست این برنامه‌ها در ادامه آمده است.

- GeoQuiz: در اولین اپلیکیشن خود، اصول پروژه‌ها، اکتیویتی‌ها، لیوت‌ها و فراخوانی صریح (intents explicit) اندروید را بررسی خواهیم کرد. همچنین یاد خواهیم گرفت که چگونه تغییرات پیکربندی را به طور یکپارچه مدیریت کنیم.

- BookShelf: بزرگترین برنامه کتاب به شما امکان می‌دهد لیست کتاب‌هایی که می‌خوانید و یا خوانده‌اید را ثبت کنید. یاد خواهید گرفت که از فرگمنت‌ها، رابط‌های پشتیبانی شده از لیست، پایگاه داده، منوها، دوربین، فراخوانی ضمنی (explicit intents) و موارد دیگر استفاده کنید.
- BeatBox: با این پروژه در مورد پخش صدا، معماری MVVM، اتصال داده‌ها، تست، تم‌ها و drawableها موارد بسیاری را یاد می‌گیرید.
- BLauncher: ساخت این لانچر سفارشی به شما بینشی در مورد فراخوان سیستم، فرآیندها و وظایف می‌دهد.
- PhotoGallery: یک کلاینت Unsplash است که عکس‌ها را از این سایت دانلود و نمایش می‌دهد، این برنامه در مورد شبکه‌سازی و موارد دیگر راهنمای شماست.
- DragAndDraw: در این اپلیکیشن طراحی ساده، با مدیریت رویدادهای لمس و ایجاد ویوهای سفارشی آشنا خواهید شد.
- Sunset: در این برنامه شبیه به بازی، ضمن یادگیری انیمیشن‌ها، نمایشی زیبا از غروب خورشید بر فراز آب‌های آزاد ایجاد خواهیم کرد.

چالش‌ها

بعضی فصل‌ها دارای یک بخش در انتها با تمریناتی هستند که می‌توانید روی آنها کار کنید. چالش‌ها فرصتی برای شماست تا از آنچه آموخته‌اید استفاده کنید، مستندات را بررسی و به تنهایی برخی از مشکلات را حل کنید.

ما اکیدا توصیه می‌کنیم که چالش‌ها را انجام دهید. خارج شدن از مسیرهای معمول و یافتن راه منحصر به خودتان، یادگیری شما را تقویت می‌کند و به شما اعتماد به نفس در پروژه‌ها را می‌دهد.

بیشتر بدانیم

همچنین بخش‌هایی در انتهای بعضی از فصل‌ها با عنوان "بیشتر بدانیم" وجود دارد. این بخش‌ها توضیحات عمیق‌تر یا اطلاعات بیشتری در مورد موضوعات ارائه شده در فصل ارائه می‌دهند. اطلاعات موجود در این بخش‌ها کاملا ضروری نیست، اما امیدواریم برای شما جالب و مفید باشد.

قراردادهای تایپی

تمام کدها و XMLها در یک لیست با فونت نمونه زیر قرا می‌گیرند. کد یا XML که باید تایپ کنید همیشه پررنگ یا بولد است. کد یا XML که باید حذف شود خط خورده نمایش داده می‌شود. به عنوان مثال، در پیاده سازی تابع زیر، فراخوانی Toast.makeText(…).show() را حذف و تابع checkAnswer(true) را اضافه می‌کنید.

```
trueButton.setOnClickListener { view: View ->
   Toast.makeText(this,
      R.string.correct_toast,
      Toast.LENGTH_SHORT).show()
   checkAnswer(true)
}
```

ابزارهای لازم

برای شروع کار با این کتاب، به اندروید استودیو نیاز دارید. اندروید استودیو یک محیط توسعه یکپارچه است، که برای توسعه اندروید استفاده می‌شود و بر اساس IntelliJ IDEA است.

نصب اندروید استودیو شامل موارد زیر است:

- Android SDK: آخرین نسخه Android SDK
- Android SDK tools و platform tools: ابزارهایی برای دیباگ و تست اپلیکیشن‌ها
- یک ایمیج سیستمی برای شبیه ساز اندروید: ابزاری برای ایجاد و تست اپلیکیشن‌های خود در دستگاه‌ها مجازی

اندروید استودیو دایما در حال توسعه و به روزرسانی است. توجه داشته باشید که ممکن است تفاوت‌هایی بین نسخه اندروید استودیو خودتان و آنچه در این کتاب می‌بینید، پیدا کنید.

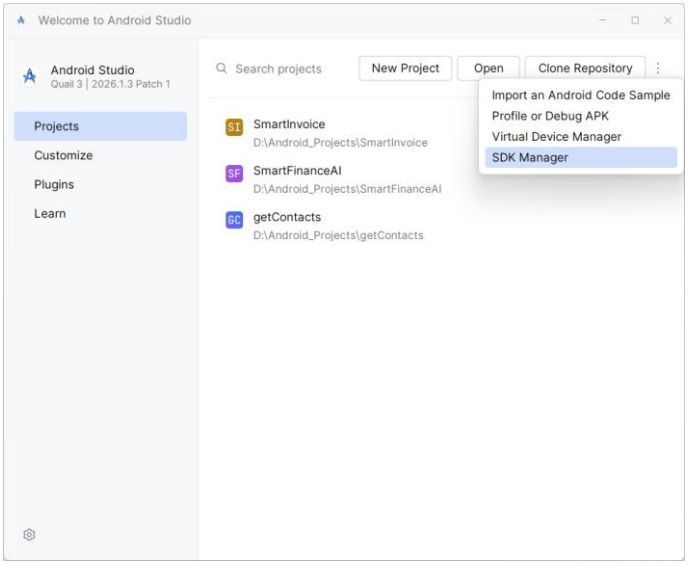
دانلود و نصب اندروید استودیو

اندروید استودیو در آدرس developer.android.com/studio در دسترس است. اگر قبلا آن را نصب نکرده‌اید، فایل نصبی آن را متناسب با سیستم عامل خود دانلود و نسبت به نصب آن اقدام نمایید. توجه داشته باشید این سایت و منابع گوگل برای ایرانی‌ها در دسترس نیست، لذا باید از یک تحریم شکن استفاده نمایید.

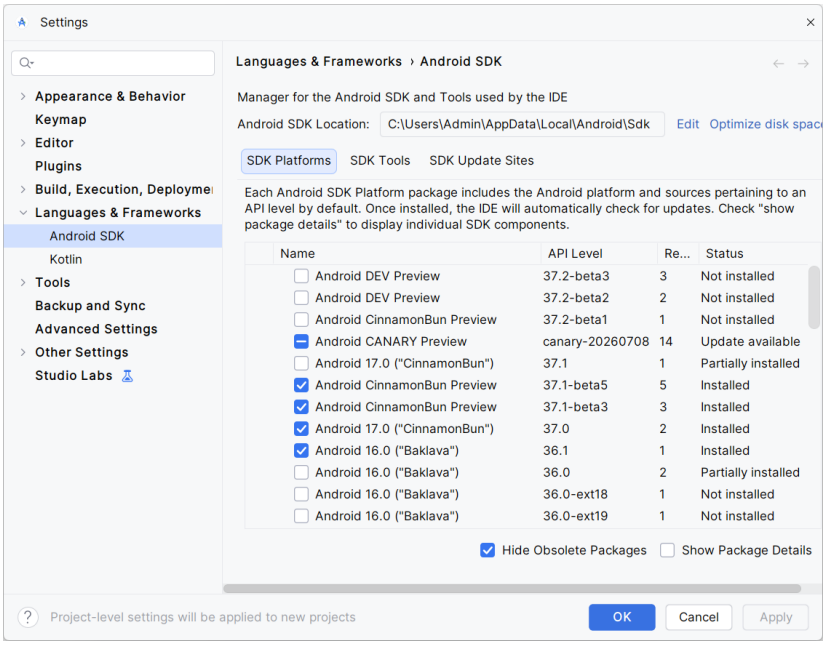
دانلود نسخه‌های قبلی SDK

SDK مخفف Software Development Kit مجموعه کامل ابزارها، کتابخانه‌ها، مستندات و شبیه سازهایی است که برنامه‌نویسان برای ساخت، تست و دیباگ اپلیکیشن‌های اندروید به آن نیاز دارند. اندروید استودیو آخرین نسخه SDK و ایمیج شبیه ساز را با خود به همراه دارد و آن را ارایه می‌دهد. با این حال، ممکن است بخواهیم برنامه‌های خود را در نسخه‌های قبلی اندروید آزمایش کنیم.

با استفاده از Android SDK Manager می‌توان کامپوننت هر پلتفرم را دریافت کرد. در Android Studio منوی Tools → SDK Manager را انتخاب کنید. (تنها در صورتی که پروژه‌ای باز دارید می‌توانید از این منو استفاده کنید در غیر این صورت در پنجره Android Welcome به SDK Manager دسترسی دارید. در شکل زیر مکان این گزینه نمایش داده شده است.)



بعد از انتخاب گزینه SDK Manager شکل زیر نشان داده می شود.



هر نسخه اندروید را که می‌خواهید با آن برنامه‌تان را آزمایش کنید، انتخاب و نصب کنید. توجه داشته باشید که دانلود این مؤلفه‌ها ممکن است مدتی طول بکشد. Android SDK Manager همچنین راهی است که می‌توانید آخرین نسخه‌های اندروید را دریافت کنید.

یک دستگاه سخت افزاری

شبیه‌ساز برای تست برنامه‌ها بسیار مفید است. با این حال، هنگام سنجش عملکرد، جایگزینی برای یک دستگاه اندرویدی واقعی وجود ندارد. توصیه می‌کنیم گاهی اوقات هنگام کار بر روی این کتاب از گوشی اندرویدی خودتان استفاده کنید.

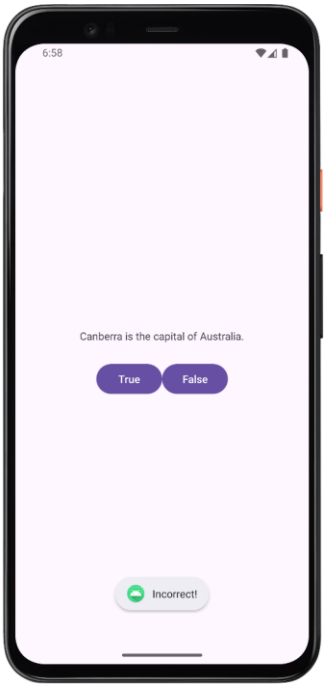
۱

اولین برنامه اندروید شما

این فصل پر از مفاهیم جدید و مورد نیاز برای ساخت یک اپلیکیشن اندروید است. اگر تا پایان این فصل همه چیز را درک نکردید، مشکلی نیست، به خواندن ادامه دهید. با پیشروی در مطالب، این ایده‌ها را با جزئیات بیشتری مرور خواهیم کرد.

اپلیکیشنی که می‌خواهیم ایجاد کنیم GeoQuiz نام دارد. GeoQuiz دانش کاربر را در مورد جغرافیا آزمایش می‌کند. کاربر برای پاسخ به سوال روی صفحه True یا False را انتخاب می‌کند و GeoQuiz بازخورد فوری ارائه می‌دهد. فعلا تمام اپلیکیشن‌ها را به زبان انگلیسی پیاده می‌کنیم. نگران این موضوع نباشید در فصل بومی‌سازی به زبان فارسی نیز خواهیم پرداخت.

تصویر ۱.۱ نتیجه فشار دادن دکمه درست توسط کاربر را نشان می‌دهد.



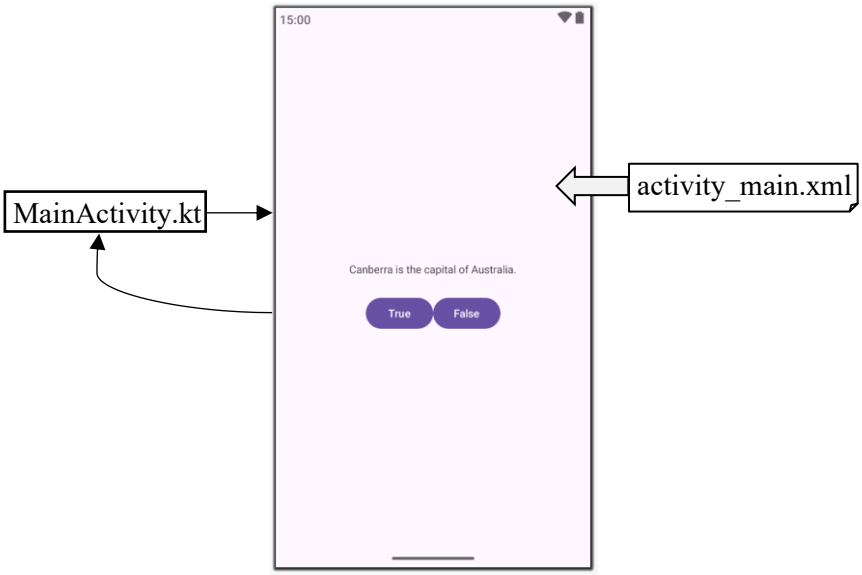
مبانی برنامه

برای شروع برنامه‌نویسی باید به مفهوم اکتیویتی بپردازیم. یکی از مهم‌ترین و پایه‌ای‌ترین اجزای یک اپلیکیشن اندروید، اکتیویتی است. به زبان ساده اکتیویتی یک صفحه از اپلیکیشن ماست که کاربر با آن تعامل دارد. این صفحه می‌تواند صفحه ورود کاربر، صفحه تنظیمات و یا همانند برنامه ما صفحه‌ای برای پرسش سوال جغرافیا باشد. هر اکتیویتی از دو بخش یا دو فایل تشکیل می‌شود.

۱. فایل کلاس جاوا یا کاتلین – البته ما از کاتلین استفاده خواهیم کرد – که منطق برنامه و رویدادها را مدیریت می‌کند. به عنوان مثال ما در این فایل مشخص خواهیم کرد که واکنش برنامه در مقابل لمس یکی از دو دکمه توسط کاربر چگونه باید باشد. یک اکتیویتی نمونه‌ای از کلاس Activity موجود در Android SDK است. ما زیر کلاس‌های Activity را برای پیاده سازی عملکردی که برنامه ما نیاز دارد، توسعه می‌دهیم. یک برنامه ساده ممکن است فقط به یک زیر کلاس نیاز داشته باشد. اما برنامه‌های پیچیدتر به تعداد بیشتری از این زیر کلاس‌ها نیاز دارد. GeoQuiz یک برنامه ساده است و با یک زیر کلاس Activity به نام MainActivity شروع می‌شود. MainActivity رابط کاربری نشان داده شده در تصویر ۱.۱ را مدیریت می‌کند.

۲. فایل لیوت XML که رابط کاربری را در آن طراحی می‌کنیم. یک لیوت مجموعه‌ای از آبجکت‌ها رابط کاربری و موقعیت آنها را روی صفحه تعریف می‌کند. لیوت از تعاریف نوشته شده در فایل XML تشکیل می‌شود. هر تعریف برای ایجاد یک شی که روی صفحه ظاهر می‌شود، مانند یک دکمه یا متن، استفاده می‌شود. GeoQuiz شامل یک فایل لیوت به نام activity_main.xml خواهد بود. XML ما رابط کاربری نشان داده شده در شکل ۱.۱ را تعریف می‌کند. رابطه بین MainActivity و activity_main.xml در شکل ۱.۲ نشان داده شده است.

شکل ۱.۲ MainActivity آنچه را که activity_main.xml تعریف و به نمایش می‌گذارد، مدیریت می‌کند.

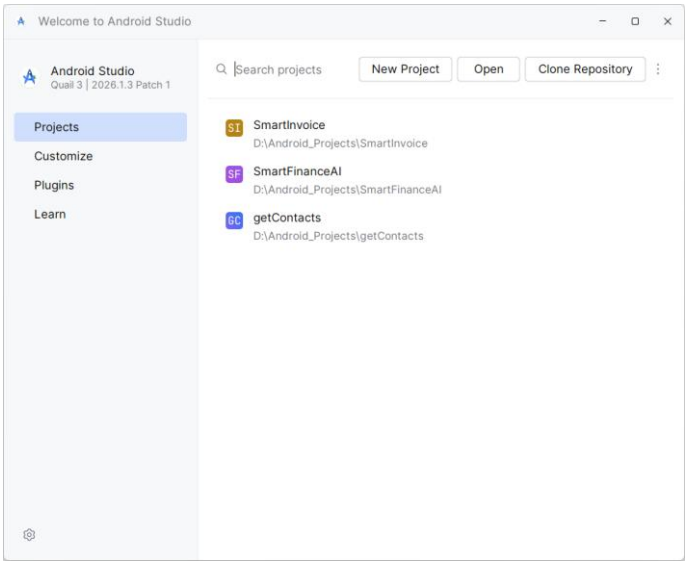


با در نظر گرفتن این ایده، بیایید یک برنامه بسازیم.

ایجاد پروژه اندروید

اولین قدم ایجاد یک پروژه اندروید است. یک پروژه اندروید شامل فایل‌هایی است که یک برنامه را تشکیل می‌دهد. برای ایجاد یک پروژه جدید، ابتدا اندروید استودیو را باز کنید. اگر این اولین بار است که اندروید استودیو را اجرا می‌کنید، کادر خوش آمدگویی را مانند تصویر ۱.۳ مشاهده خواهید کرد.

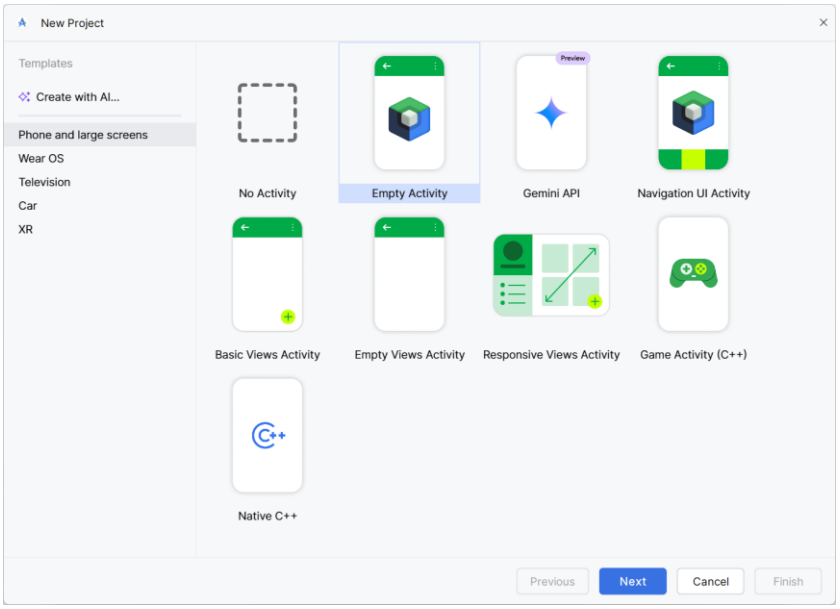
تصویر ۱.۳ به Android Studio خوش آمدید



در کادر خوش آمدگویی، New Project را انتخاب کنید. اگر قبلا پروژه‌ای ایجاد کرده‌اید گزینه File → New → New Project را انتخاب کنید.

به پنجره ایجاد پروژه جدید خوش آمدید (شکل ۱.۴). مطمئن شوید که برگه Phone and Tablet انتخاب شده است، Empty Views Activity را انتخاب کرده و روی Next کلیک کنید. انتخاب این گزینه برنامه را شامل یک اکتیویتی با رابط کاربری خالی می‌کند.

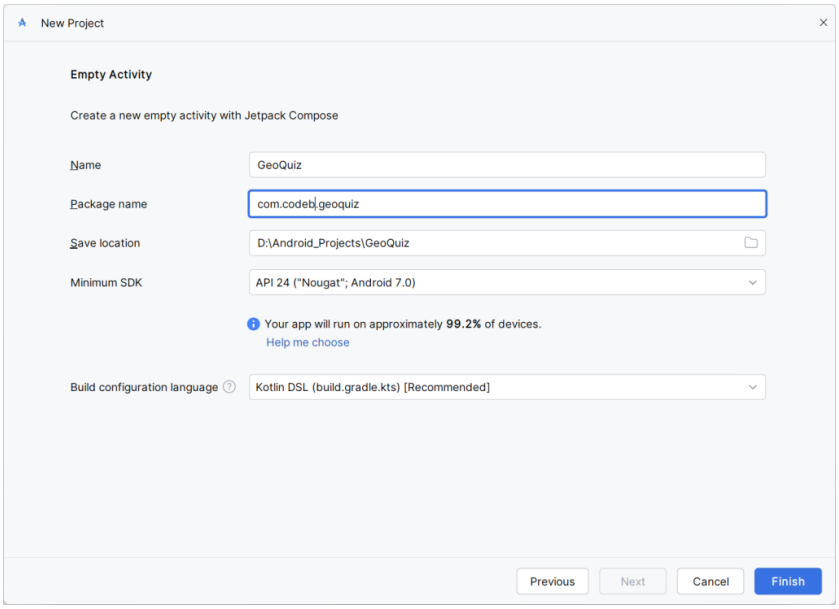
تصویر ۱.۴ انتخاب قالب پروژه



اکنون باید صفحه پیکربندی پروژه خود را ببینید (تصویر ۱.۵). GeoQuiz را به عنوان نام برنامه و برای نام پکیج، com.codeb.geoquiz را وارد کنید. برای Save Location که مربوط به مکان ذخیره‌سازی پروژه است، هر مکانی را که دوست دارید، انتخاب نمایید.

Kotlin را از منوی کشویی Language انتخاب کنید. Minimum SDK را روی API 24 (“Nougat”; Android 7.0) قرار دهید. در مورد این گزینه در آینده مفصلا صحبت خواهیم کرد. در نهایت، با رها کردن گزینه‌های بعدی Finish را کلیک کنید.

تصویر ۱.۵ پیکربندی پروژه جدید شما



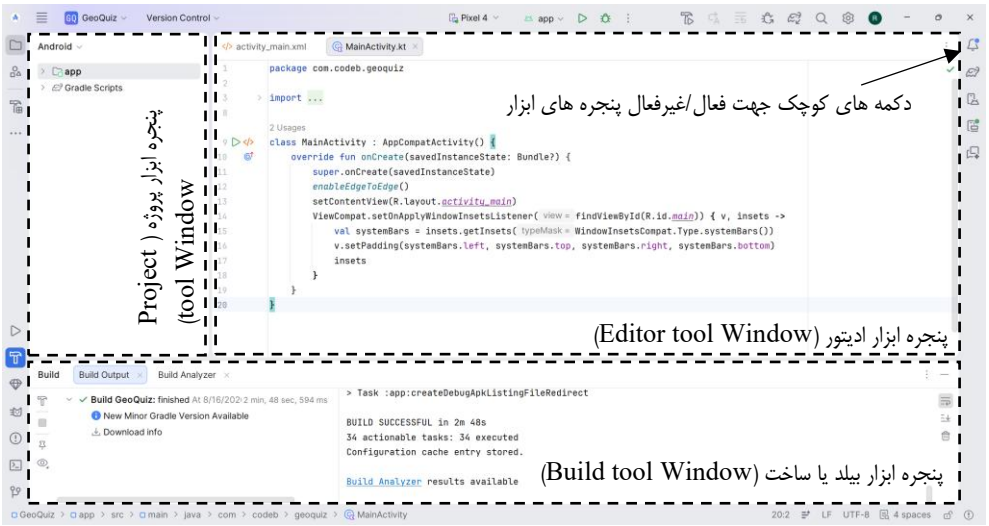
Package Name یک شناسه منحصر به فرد برای اپلیکیشن ماست. اندروید از این نام برای تشخیص اپلیکیشن ما از میلیون‌ها اپلیکیشن دیگر استفاده می‌کند. توجه داشته باشید که نام پکیج از یک قرارداد "DNS معکوس" استفاده می‌کند: نام دامنه سازمان شما معکوس شده و با شناسه‌ی بیشتر همراه می‌شود. انتخاب Kotlin به عنوان زبان به اندروید استودیو اعلام می‌کند که برنامه ما وابستگی‌های (dependencies) لازم مانند ابزارهای ساخت کاتلین برای نوشتن و ساخت کد کاتلین را شامل شود.

با کلیک روی Finish اندروید استودیو پروژه جدید شما را ایجاد و باز می‌کند. البته کمی برای دانلود مولفه‌ها زمان لازم است، صبور باشید.

گشت و گذاری در اندروید استودیو

اندروید استودیو پروژه ما را در پنجره‌ای مانند آنچه در شکل ۱.۷ نشان داده شده است، باز می‌کند. اگر قبلا اندروید استودیو را راه اندازی کرده‌اید، پنجره شما ممکن است کمی متفاوت به نظر برسد.

شکل ۱.۷ پنجره پروژه



به صفحات مختلف پنجره اصلی، پنجره ابزار گفته می‌شود.

پنجره سمت چپ، پنجره Project tool است. از اینجا می‌توان فایل‌های مرتبط با پروژه خود را مشاهده و مدیریت کرد. ویوی پایین پنجره Build tool نام دارد. در اینجا می‌توانید جزئیات مربوط به فرآیند کامپایل و وضعیت ساخت را مشاهده کنید. هنگام ایجاد پروژه، اندروید استودیو به طور خودکار آن را می‌سازد (به اصطلاح build می‌کند). بنابراین باید در پنجره Build tool پیام، فرآیند با موفقیت انجام یافته است، را مشاهده کنید.

اندروید استودیو به طور خودکار فایل‌های activity_main.xml و MainActivity.kt را در ویوی اصلی باز می‌کند. این پنجره editor tool یا فقط edit نامیده می‌شود.

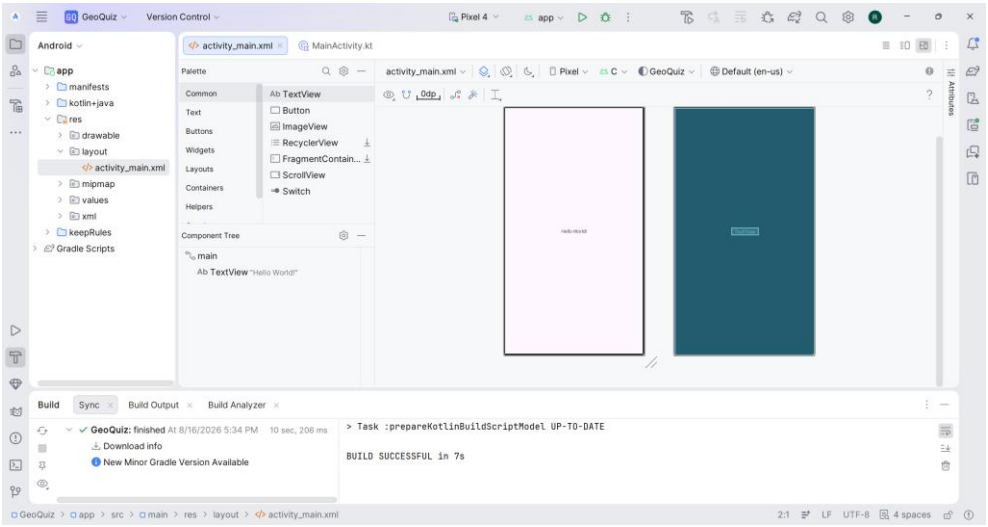
می‌توانید با استفاده از منو View→Tool Windows هر کدام از پنجره‌های ابزار را نمایان یا مخفی نمایید. همچنین آیکون‌های کوچکی همانطور که در شکل نشان داده شده است، در کناره‌ها وجود دارند که هر یک، پنجره خاصی را فعال/غیرفعال می‌کنند. میانبرهای صفحه کلید برای بسیاری از این‌ها نیز وجود دارد.

تعریف رابط کاربری

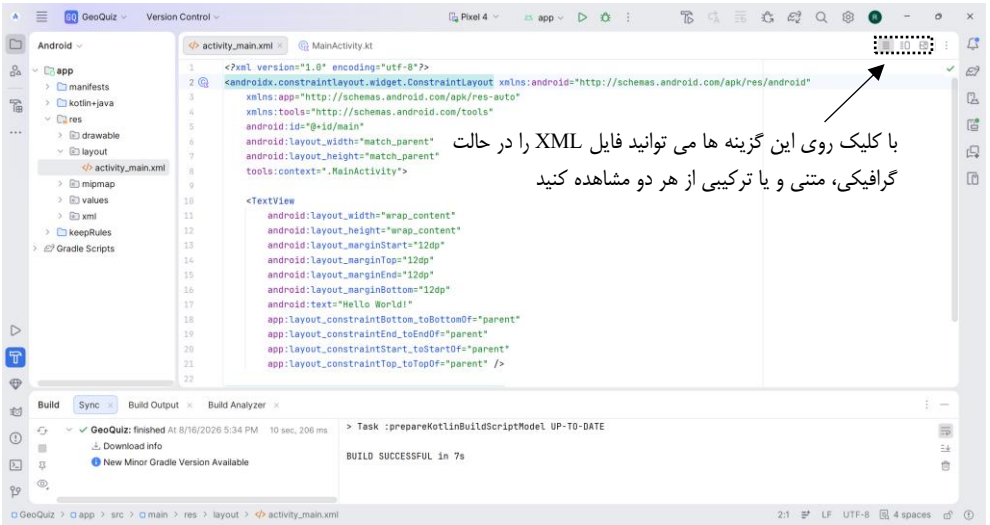
اولین کار ما طراحی رابطه کاربری است.

همانطور که قبلا گفتیم رابط کاربری ما فایل activity_main.xml می‌باشد. روی برگه این فایل لیوت کلیک کنید. با این کار ویرایشگر لیوت در پنجره ابزار ویرایشگر باز می‌شود (تصویر ۱.۸). اگر برگه‌ای برای activity_main.xml نمی‌بینید، نگران نباشید. /app/res/layout را در پنجره ابزار پروژه گسترش دهید. روی activity_main.xml دوبار کلیک کنید تا فایل باز شود. اگر activity_main.xml باز شد اما تعاریف XML را به جای ویرایشگر لیوت نشان می‌دهد، همانند تصویر ۱.۹ روی تب Design در بالا سمت راست پنجره ابزار ویرایشگر کلیک کنید.

شکل ۱.۸ ویرایشگر لیوت



شکل ۱.۹ تب Design



طبق قرارداد، یک فایل لیوت بر اساس اکتیویتی که با آن مرتبط است نامگذاری می‌شود. نام آن با activity_ شروع و بقیه نام اکتیویتی به صورت تمام حروف کوچک با استفاده از زیرخط دنبال می‌شود. بنابراین، به عنوان مثال نام فایل لیوت ما activity_main.xml است و فایل لیوت برای اکتیویتی به نام SplashScreenActivity، activity_splash_screen نامگذاری می‌شود. این استایل نامگذاری برای لیوت‌ها و همچنین سایر منابعی که بعدا با آنها آشنا خواهید شد، توصیه می‌شود.

ویرایشگر لیوت یک پیش نمایش گرافیکی از فایل را نشان می‌دهد. تب Code را انتخاب کنید تا تعاریف XML را ببینید.

در حال حاضر activity_main.xml الگوی لیوت پیش‌فرض را در اختیار دارد. الگو به طور مکرر تغییر می‌کند، اما XML آن شبیه به لیست ۱.۱ خواهد بود.

لیست ۱.۱ لیوت اکتیویتی پیش فرض (res/layout/activity_main.xml)

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">
```

```
    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Hello World!"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />
</androidx.constraintlayout.widget.ConstraintLayout>
```

فایل Layout معمولا از این موارد تشکیل می شود:

- یک تگ ریشه (مثل ConstraintLayout)
- چندین View (المان‌های داخل صفحه)
- ویژگی‌ها (Attributes) برای تنظیم ظاهر و موقعیت هر View
- Namespace برای فعال کردن دستورات اندروید

لیوت پیش‌فرضی که ساختیم، دو View را تعریف می‌کند: یک ConstraintLayout و یک TextView.

Viewها بلوک‌های سازنده‌ای هستند که برای نوشتن یک رابط کاربری استفاده می‌کنیم. هر چیزی که روی صفحه می‌بینید یک ویو است. ویوهایی که کاربر می‌تواند ببیند یا با آنها تعامل داشته باشد، ویجت (widget) نامیده می‌شود. برخی از ویجت‌ها نوشتار را نشان می‌دهند، برخی از ویجت‌ها گرافیک و برخی دیگر، مانند دکمه‌ها، وقتی لمس می‌شوند، کاری را انجام می‌دهند.

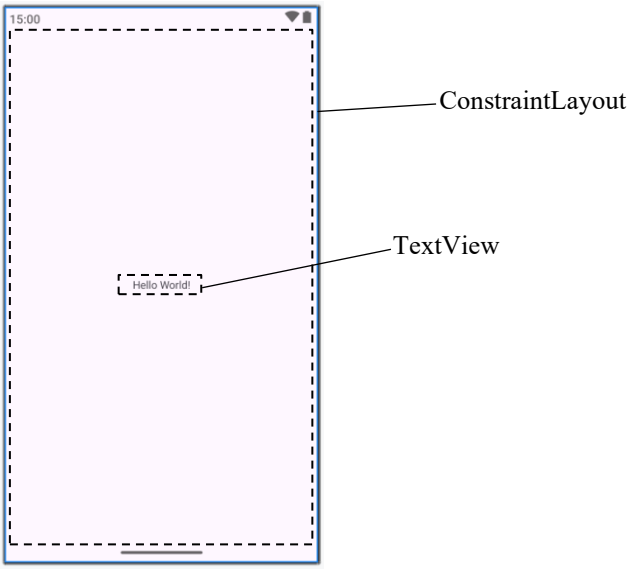
Android SDK شامل ویجت‌های زیادی است که می‌توانیم آنها را پیکربندی کنیم تا ظاهر و رفتار مورد نظر خود را به دست آوریم. هر ویجت نمونه‌ای از کلاس View یا یکی از زیر کلاس‌های آن (مانند TextView یا Button) است.

چیزی باید به ویجت‌ها بگوید که به کجا تعلق دارند و آنها را مرتب نگه دارد. ViewGroup نوعی ویو است که شامل ویوهای دیگر است و آن را مرتب می‌کند. یک ViewGroup خود محتوایی برای نمایش ندارد. در عوض، جایی که محتوای سایر ویوها باید برای نمایش قرار گیرند را تعیین می‌کند. ViewGroupها اغلب به عنوان لیوت شناخته می‌شوند.

لیوت پیش فرض، ConstraintLayout یک ViewGroup است که مسئول چیدمان تنها فرزند خود، ویجت TextView است. در فصل‌های بعد با لیوت‌ها و ویجت‌ها و استفاده از ConstraintLayout بیشتر آشنا خواهید شد.

شکل ۱.۱۰ نشان می‌دهد که چگونه ConstraintLayout و TextView تعریف شده در فهرست ۱.۱ روی صفحه ظاهر می‌شوند.

تصویر ۱.۱۰ ویوهای پیش فرض همانطور که روی صفحه دیده می‌شود.

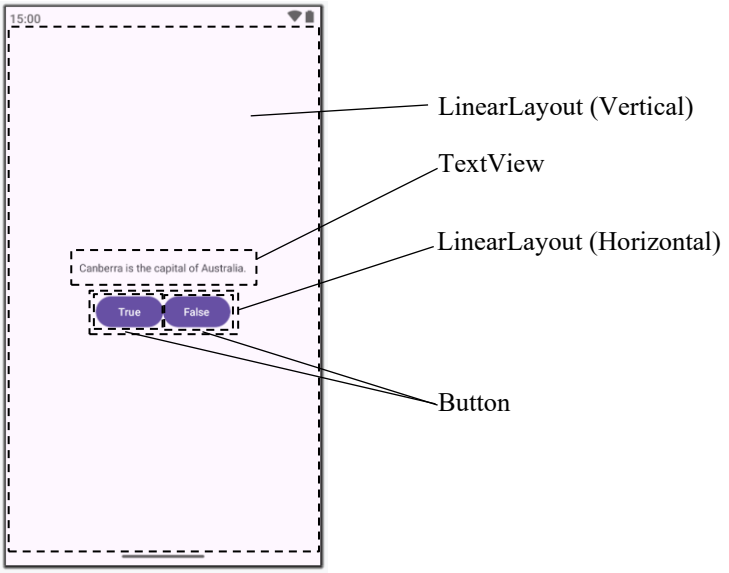


اما اینها ویجت‌هایی نیستند که ما در برنامه خود به دنبال آن هستیم. رابط کاربری MainActivity به پنج ویجت نیاز دارد:

- یک LinearLayout عمودی
- یک TextView
- یک LinearLayout افقی
- دو Button

شکل ۱.۱۱ نشان می‌دهد که چگونه این ویجت‌ها رابط کاربری MainActivity را تشکیل می‌دهند.

شکل ۱.۱۱ ویجت‌ها همانطور که روی صفحه دیده می‌شود



اکنون باید این ویجت‌ها را در XML لیوت خود تعریف کنیم. محتوای متن activity_main.xml را ویرایش کنید تا با فهرست ۱.۲ مطابقت داشته باشد. XML که باید حذف کنید به صورت خط خورده و XML که باید اضافه کنید به صورت بولد است. این الگویی است که ما در طول این کتاب از آن استفاده خواهیم کرد.

نگران درک آنچه تایپ می‌کنید نباشید. در ادامه یاد خواهید گرفت که جریان کار از چه قرار است. با این حال مراقب باشید. XML لیوت در صورت داشتن اشتباهات تایپی تایید نمی‌شود. پس از تکمیل، خطاهایی را در سه خطی که با android:text شروع می‌شوند، مشاهده خواهید کرد. فعلا این خطاها را نادیده بگیرید. به زودی آنها را رفع خواهید کرد.

لیست ۱.۲ تعریف ویجت‌ها در XML (res/layout/activity_main.xml)

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">
```

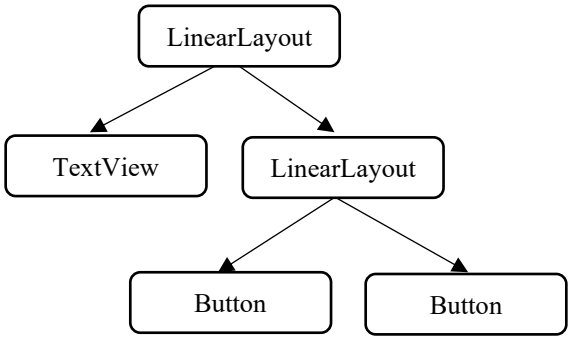
```
    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Hello World!"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />
</androidx.constraintlayout.widget.ConstraintLayout>
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:gravity="center"
    android:orientation="vertical"
    android:id="@+id/main">
    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:padding="24dp"
        android:text="@string/question_text" />
    <LinearLayout
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:orientation="horizontal">
        <Button
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="@string/true_button" />
        <Button
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="@string/false_button" />
    </LinearLayout>
</LinearLayout>
```

XML خود را با رابط کاربری نشان داده شده در شکل ۱.۱۱ مقایسه کنید. هر ویجت دارای یک عنصر XML مربوطه است و نام عنصر نوع ویجت است. هر عنصر دارای مجموعه‌ای از ویژگی‌های XML (XML attributes) است. هر ویژگی دستورالعملی در مورد نحوه پیکربندی ویجت مشخص می‌کند. برای درک نحوه عملکرد عناصر و ویژگی‌ها، نگاه کردن به لیوت از منظر سلسله مراتبی می‌تواند کمک شایانی بکند.

سلسله مراتب ویو

ویجت‌های شما در سلسله مراتبی از آبجکت‌ها View به نام سلسله مراتب View (View hierarchy) قرار می‌گیرند. شکل ۱.۱۲ سلسله مراتب ویو را نشان می‌دهد که با XML فهرست ۱.۲ مطابقت دارد.

شکل ۱.۱۲ چیدمان سلسله مراتبی ویجت‌ها و ویژگی‌ها



عنصر ریشه سلسله مراتب ویو این لیوت یک LinearLayout است. به عنوان عنصر ریشه، LinearLayout باید فضای نام XML منبع اندروید http://schemas.android.com/apk/res/android را در مشخص کند.

LinearLayout از ViewGroup ارث‌بری دارد که قبلا در مورد آن صحبت کردیم. یک زیر کلاس از View که شامل ویوهای دیگر است و کار مرتب کردن آنها را انجام می‌دهد. زمانی که می‌خواهیم ویوها در یک ستون یا ردیف مرتب شوند، از LinearLayout استفاده می‌کنیم. سایر زیر کلاس‌های ViewGroup که بعدا با آنها ملاقات خواهید کرد عبارتند از ConstraintLayout و FrameLayout.

هنگامی که یک ویو داخل یک ViewGroup قرار می‌گیرد، گفته می‌شود که آن ویو فرزند ViewGroup است. LinearLayout دارای دو فرزند است: یک TextView و یک LinearLayout. LinearLayout فرزند دارای دو فرزند Button برای خود است.

ویژگی‌های ویجت

بیایید برخی از ویژگی‌هایی را که برای پیکربندی ویجت‌های خود استفاده کرده‌ایم، مرور کنیم.

android:layout_width و android:layout_height

ویژگی‌های android:layout_width و android:layout_height تقریبا برای هر نوع ویجت مورد نیاز است. آنها ارتفاع و عرض ویجت را مشخص می‌کنند که معمولا روی match_parent یا wrap_content تنظیم می‌شوند:

- match_parent: ویو به اندازه والد خود کش می‌یابد
- wrap_content: ویو به اندازه‌ای که محتویات آن نیاز دارد خواهد بود

برای LinearLayout ریشه، مقدار هر دو ویژگی ارتفاع و عرض match_parent است. LinearLayout عنصر ریشه است، اما هنوز یک والد دارد – ویویی که اندروید برای نگهداری سلسله مراتب ویوی برنامه شما فراهم می‌کند.

ویجت‌های دیگر در لیوت شما عرض و ارتفاع خود را روی wrap_content تنظیم کرده‌اند. در شکل ۱.۱۱ می‌توانید ببینید که چگونه این نسبت‌ها اندازه آنها را تعیین می‌کند.

TextView به دلیل ویژگی "android:padding="24dp کمی بزرگتر از متنی است که در آن وجود دارد. این ویژگی به ویجت می‌گوید که هنگام تعیین اندازه، مقدار فضای مشخص شده را به محتویات آن اضافه کند. استفاده از آن کمی فضا بین سوال و دکمه‌ها ایجاد می‌کند. (با واحد DP در فصل‌های بعد آشنا خواهید شد.)

android:orientation

ویژگی android:orientation در دو ویجت LinearLayout تعیین کننده افقی یا عمودی ردیف شدن فرزندان آنها است. LinearLayout ریشه عمودی است؛ فرزند آن LinearLayout افقی است. ترتیب تعریف فرزندان، ترتیب ظاهر شدن آنها بر روی صفحه را تعیین می‌کند. در یک LinearLayout عمودی، اولین فرزند تعریف شده در بالاترین سطح ظاهر می‌شود. در یک LinearLayout افقی، اولین فرزند تعریف شده سمت چپ خواهد بود. (مگر اینکه دستگاه روی زبانی مانند فارسی تنظیم شده باشد که از راست به چپ اجرا می‌شود. در این صورت، اولین فرزند در راست خواهد بود.)

android:text

ویجت‌های TextView و Button دارای ویژگی‌های android:text هستند. این ویژگی به ویجت می‌گوید که چه متنی را نمایش دهد. توجه داشته باشید که مقادیر این ویژگی‌ها رشته‌های ثابت تایپ شده‌ای نیستند، بلکه ارجاع به منابع رشته‌ای (string resources) هستند که به صورت قرارداد /@string تعریف می‌شوند. منابع رشته‌ای، رشته‌هایی هستند که در یک فایل XML جداگانه به نام strings file قرار دارند. می‌توانیم به یک ویجت یک رشته را مستقیما اختصاص دهید. مثلا در لیست بالا android:text="True" قرار دهید، اما معمولا ایده خوبی نیست. قرار دادن رشته‌ها در یک فایل جداگانه و سپس ارجاع دادن به آنها بهتر است. زیرا بومی‌سازی را آسان می‌کند.

منابع رشته‌ای که در activity_main.xml به آنها ارجاع می‌دهیم هنوز وجود ندارند. بیایید آن را ایجاد کنیم.

ایجاد منابع رشته‌ای

هر پروژه شامل یک فایل رشته‌ای پیش‌فرض به نام strings.xml در مسیر /res/values است. این فایل یکی از مهم‌ترین فایل‌های هر پروژه اندرویدی است که برای ذخیره کردن تمام متن‌های اپلیکیشن استفاده می‌شود.

res/values/strings.xml را باز کنید. الگو پیش‌فرض یک منبع رشته‌ای قبلا به پروژه اضافه کرده است. سه رشته جدید مورد نیاز لیوت خود را اضافه کنید.

لیست ۱.۳ افزودن منابع رشته‌ای (res/values/strings.xml)

```
<resources>
    <string name="app_name">GeoQuiz</string>
    <string name="question_text">
        Canberra is the capital of Australia.
    </string>
    <string name="true_button">True</string>
    <string name="false_button">False</string>
</resources>
```

(بسته به نسخه Android Studio شما، ممکن است رشته‌های اضافی داشته باشید. آنها را حذف نکنید. حذف آنها می‌تواند باعث خطاهایی در کل پروژه شود.)

اکنون، هر زمان که در هر فایل XML در پروژه GeoQuiz به @string/false_button مراجعه کنید، رشته "False" را در زمان اجرا دریافت خواهید کرد. /@string به عنوان اشاره‌ای به فایل strings و false_button نام منبع رشته‌ای است.

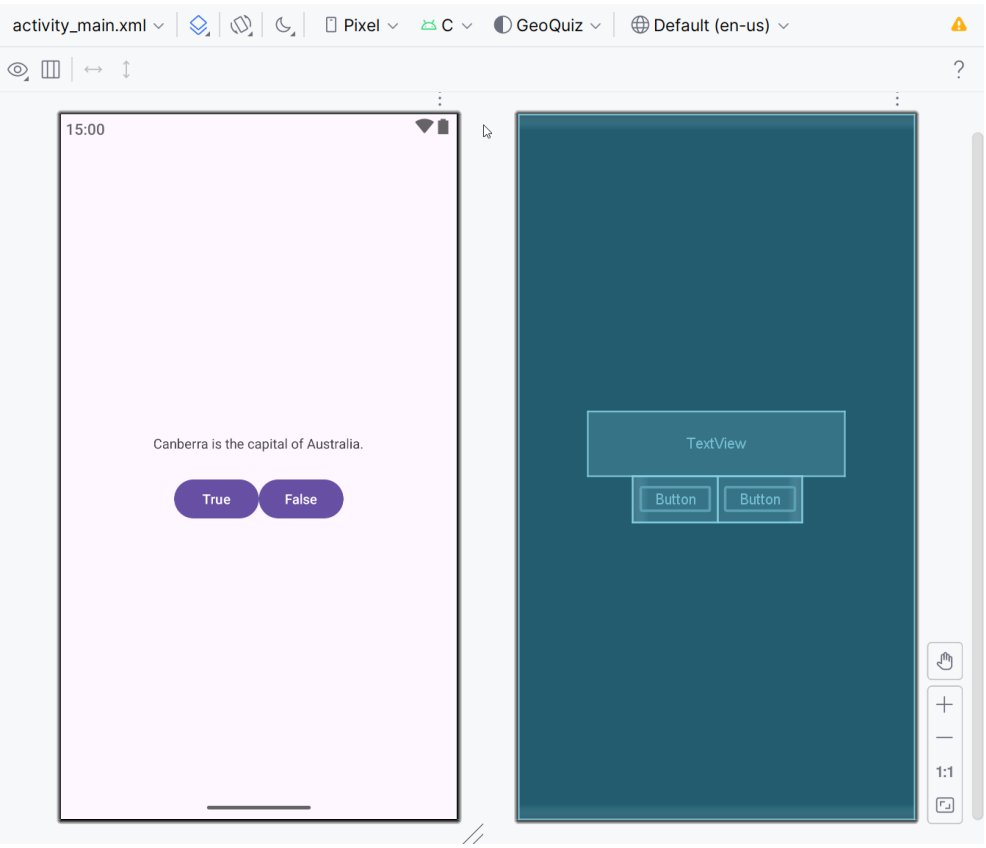
خطاهای موجود در activity_main.xml در مورد منابع رشته‌ای اکنون باید برطرف شده باشد. (اگر همچنان خطا دارید، هر دو فایل را از نظر اشتباه تایپی بررسی کنید.)

اگرچه فایل رشته‌های پیش‌فرض strings.xml نام دارد، اما می‌توانید هر نامی انتخاب کنید. همچنین می‌توانید چندین فایل رشته‌ای در یک پروژه داشته باشید.

پیش نمایش لیوت

لیوت شما اکنون کامل شده است. به activity_main.xml برگردید و پیش نمایش لیوت را با کلیک روی تب Design مشاهده کنید (تصویر ۱.۱۳).

تصویر ۱.۱۳ پیش نمایش activity_main.xml در کادر Design



شکل ۱.۱۳ دو نوع پیش‌نمایش موجود را نشان می‌دهد. پیش‌نمایش سمت چپ پیش‌نمایش Design است. این پیش‌نمایش لیوت را آن طوری که در یک دستگاه واقعی به نمایش در می‌آید، نشان می‌دهد.

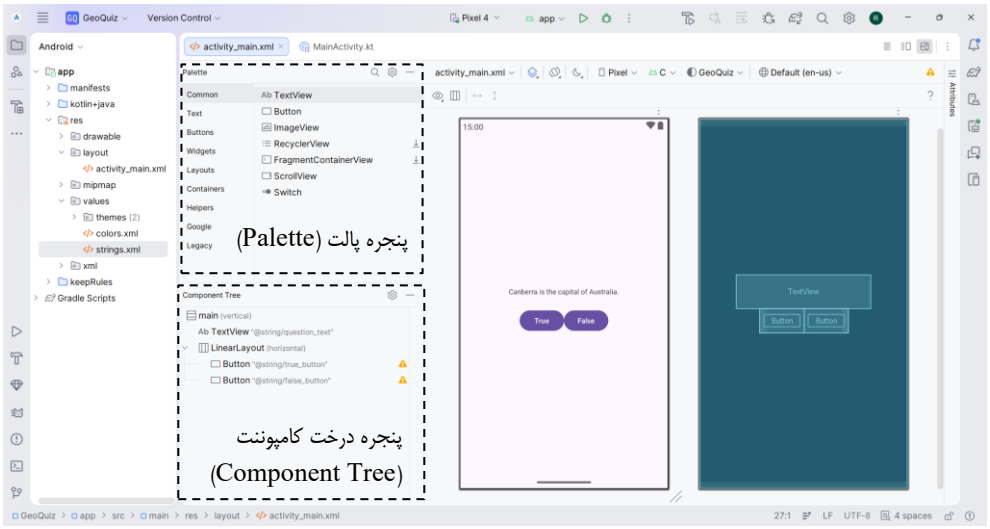
پیش‌نمایش سمت راست پیش‌نمایش Blueprint است. این پیش‌نمایش بر اندازه ویجت‌ها و روابط بین آنها تمرکز دارد.

صفحه Design همچنین به ما امکان مشاهده لیوت در پیکربندی‌های مختلف دستگاه را می‌دهد. در بالای کادر، می‌توانیم نوع دستگاه، نسخه اندروید برای شبیه سازی و تم دستگاه را مشخص کنیم. حتی می‌توانیم وانمود کنیم که از زبان راست به چپ استفاده می‌کنیم.

علاوه بر پیش‌نمایش، می‌توان لیوت‌های خود را با استفاده از ویرایشگر لیوت نیز ساخت. در سمت چپ یک پالت وجود دارد که شامل تمام ویجت‌های داخلی است (شکل ۱.۱۴). می‌توان این ویجت‌ها را از پالت کشیده و روی ویوی خود رها کرد. همچنین می‌توانیم آنها را در درخت کامپوننت در پایین سمت چپ رها کنیم تا کنترل بیشتری بر محل قرار دادن ویجت‌ها داشته باشیم. این پنجره ساختار درختی تمام المان‌های موجود در فایل Layout را نشان می‌دهد.

شکل ۱.۱۴ پیش‌نمایش را با جزئیات لیوت، یعنی به همراه نوار وضعیت دستگاه و نوار پیمایش دستگاه مجازی، نمایش می‌دهد. برای دیدن این جزئیات، روی دکمه با نماد چشم در نوار ابزار درست بالای پیش‌نمایش کلیک و گزینه Show system UI را انتخاب کنید.

شکل ۱.۱۴ ویرایشگر لیوت گرافیکی



این ویرایشگر گرافیکی به ویژه هنگام کار با ConstraintLayout، ارزشمند خواهد بود.

از تعاریف فایل لیوت XML تا آبجکت‌ها ویو

چگونه عناصر XML در activity_main.xml به آبجکت‌ها View تبدیل می‌شوند و در صفحه رابط کاربری ما به نمایش در می‌آیند؟ پاسخ در کلاس MainActivity است.

هنگامی که پروژه GeoQuiz را ایجاد کردیم، یک زیر کلاس از Activity به نام MainActivity برای ما ایجاد شد. فایل کلاس برای MainActivity در دایرکتوری app/kotlin+java پروژه ما قرار دارد.

MainActivity.kt ممکن است از قبل در یک برگه در پنجره ابزار ویرایشگر باز باشد. اگر اینطور نیست، دایرکتوری app/kotlin+java را در پنجره ابزار پروژه پیدا کرده و روی آن کلیک کنید تا محتویات آن نمایان شود. سپس روی پکیج com.codeb.geoquiz کلیک نمایید. فایل MainActivity.kt را باز و به محتویات آن نگاهی بیندازید.

لیست ۱.۴ فایل کلاس پیش‌فرض برای MainActivity (MainActivity.kt)

```
package com.codeb.geoquiz
import android.os.Bundle
import androidx.activity.enableEdgeToEdge
import androidx.appcompat.app.AppCompatActivity
import androidx.core.view.ViewCompat
import androidx.core.view.WindowInsetsCompat
class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContentView(R.layout.activity_main)
        ViewCompat.setOnApplyWindowInsetsListener(
        findViewById(R.id.main)) { v, insets ->
            val systemBars = insets.getInsets(
               WindowInsetsCompat.Type.systemBars())
            v.setPadding(
               systemBars.left, systemBars.top,
               systemBars.right, systemBars.bottom)
            insets
        }
    }
}
```

این کد به صورت پیش‌فرض ساخته می‌شود. به توضیح خط به خط آن می پردازیم. خط اول که با کلمه package شروع می شود، مشخص می‌کند این فایل در کدام پکیج قرار دارد. دستورات import بعدی برای وارد کردن کلاس‌های مورد نیاز است. اگر همه دستورات import را نمی‌بینید، روی نماد < در سمت چپ اولین دستور import کلیک کنید تا مابقی نمایان شود.

خط بعدی تعریف کلاس MainActivity است که از AppCompatActivity ارث‌بری دارد. تابع موجود در کلاس یعنی ()onCreate اولین تابعی است که وقتی اکتیویتی ساخته می‌شود، اجرا می‌شود. یک اکتیویتی ممکن است با اجرای برنامه یا با فرخوانی توسط اکتیویتی دیگر ایجاد شود. سطر بعدی یعنی super.onCreate(savedInstanceState) فراخوانی متد والد را در پی دارد. این خط الزامی است. تابع enableEdgeToEdge() اپلیکیشن را تمام صفحه می‌کند.

و خط مهم، کد setContentView(R.layout.activity_main) است. هنگام ایجاد اکتیویتی رابط کاربری برای نمایش و مدیریت مورد نیاز است. به همین دلیل این تابع را فراخوانی می‌کنیم. این تابع فایل activity_main.xml را به عنوان رابط کاربری این صفحه بارگذاری می‌کند.

حالا مشخص شد که چطور تعریف فایل XML به آبجکت‌ها ویو تبدیل و به نمایش در می‌آید. فراخوانی تابع setContentView(R.layout.activity_main) یک لیوت را بوجود می‌آورد و یا به اصطلاح بارگذاری می‌کند و آن را روی صفحه نمایش قرار می‌دهد. هنگامی که یک لیوت بارگذاری می‌شود، هر ویجت در فایل لیوت همانطور که توسط ویژگی‌های آن تعریف شده است، نمونه‌سازی می‌شود. ما مشخص می‌کنیم که کدام لیوت را با ارسال شناسه منبع لیوت (layout resource ID) بارگذاری کنیم.

خط های بعدی برای محاسبه خودکار فاصله نوارهای سیستم و جلوگیری از تداخل محتوای ما با آنهاست.

(نمی‌دانید AppCompatActivity چیست؟ این یک زیر کلاس از کلاس اکتیویتی اندروید است که برای سازگاری نسخه‌های قدیمی اندروید بوجود آمده است. در فصل‌های بعدی اطلاعات بیشتری در مورد AppCompatActivity خواهید آموخت.)

این فایل دارای یک تابع Activity است: onCreate(Bundle?).

منابع و شناسه‌های آنها

لیوت یک منبع (resourse) است. یک منبع بخشی از برنامه ما است که کد نیست – چیزهایی مانند فایل‌های تصویری، فایل‌های صوتی و فایل‌های XML.

منابع پروژه ما در زیرشاخه‌ای از دایرکتوری app/res قرار دارند. به عنوان مثال در پنجره ابزار پروژه می‌توانید مشاهده کنید که activity_main.xml در /res/layout و فایل strings.xml که حاوی منابع رشته‌ای است، در /res/values قرار دارد.

برای دسترسی به یک منبع در کد، از شناسه منبع آن استفاده می‌کنیم. شناسه منبع برای لیوت شما R.layout.activity_main است.

برای ایجاد یک شناسه منبع برای یک ویجت، ویژگی android:id را در تعریف ویجت قرار می‌دهیم. در activity_main.xml، به هر دکمه یک ویژگی android:id اضافه کنید.

لیست ۱.۵ افزودن شناسه‌ها به دکمه‌ها (res/layout/activity_main.xml)

```
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:gravity="center"
    android:orientation="vertical"
    android:id="@+id/main">
    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:padding="24dp"
        android:text="@string/question_text" />
    <LinearLayout
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:orientation="horizontal">
        <Button
            android:id="@+id/true_button"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="@string/true_button"
            />
        <Button
            android:id="@+id/false_button"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="@string/false_button" />
```

```
    </LinearLayout>
</LinearLayout>
```

توجه داشته باشید که علامت + در مقادیر android:id وجود دارد اما در مقادیر android:text خیر. دلیل به کاربردن این علامت این دلیل است که ما شناسه‌های منبع را ایجاد می‌کنیم اما به رشته‌ها فقط ارجاع می‌دهیم.

اتصال المان‌های رابط کاربری به کد

آماده هستیم تا ویجت‌های دکمه خود را به دست گرفته و آنها را کنترل کنیم. این فرآیند شامل دو مرحله است:

۱. دریافت رفرنس به آبجکت‌ها View
۲. تنظیم شنونده‌ها (Listeners) جهت پاسخ گویی به اقدامات کاربر

دریافت رفرنس به ویجت‌ها

اکنون که دکمه‌ها شناسه منبع دارند، می‌توانیم در MainActivity به آنها دسترسی داشته باشیم. کد زیر (لیست ۱.۶) را در MainActivity.kt تایپ کنید. پس از ذخیره فایل، دو خطا گزارش می‌شود. خطاها را تنها در یک ثانیه برطرف خواهیم کرد.

لیست ۱.۶ دسترسی به آبجکت‌ها view توسط شناسه (MainActivity.kt)

```
class MainActivity : AppCompatActivity() {
    private lateinit var trueButton: Button
    private lateinit var falseButton: Button
    override fun onCreate(savedInstanceState: Bundle?) {
        ...
        trueButton = findViewById(R.id.true_button)
        falseButton = findViewById(R.id.false_button)
    }
}
```

در یک اکتیوتی، می‌توانیم با فراخوانی Activity.findViewById(Int) به یک ویجت ارجاع دهیم. این تابع ویوی مربوطه را برمی‌گرداند. به جای برگرداندن آن به عنوان یک View، ما آن را به زیرگروه مورد انتظار View یعنی نوع Button تبدیل میکنیم.

در کد بالا، از شناسه منابع دکمه‌های خود برای بازیابی آبجکت‌ها و اختصاص دادن آنها به متغییرهای خود استفاده می‌کنیم. از آنجایی که آبجکت‌ها view تا بعد از setContentView(…) به حافظه انتقال نمی‌یابند و در دسترس نیستند، تعریف متغییرها در onCreate(…) قرار می‌گیرد. از lateinit در تعریف متغییرها استفاده می‌کنیم تا به کامپایلر بگوییم متغییر مورد نظر بعدا و درست قبل از استفاده، مقداردهی خواهد شد. در مورد onCreate(…) و چرخه حیات اکتیویتی در فصل مربوطه بیشتر خواهیم آموخت.

حالا بیایید از شر آن خطاهای مزاحم خلاص شویم. ماوس را روی نشانگرهای خطای قرمز قرار دهید. هر دو، مشکل یکسانی را گزارش می‌کنند: Unresolved reference: Button.

این خطاها به ما می‌گویند که باید کلاس android.widget.Button را به MainActivity.kt اضافه نماییم. می‌توانید دستور import زیر را در بالای فایل تایپ کنید:

```
import android.widget.Button
```

همینطور می‌توانید این کار را به روشی آسان انجام دهید و کار را به اندروید استودیو واگذار کنید. با فشردن کلید Alt+Enter جادوی IntelliJ شما را شگفت زده می‌کند. دستور import جدید اکنون با بقیه در بالای فایل ظاهر می‌شود. این میانبر به طور کلی زمانی مفید است که چیزی در کد شما درست نباشد. اغلب آن را امتحان خواهیم کرد!

باید از شر خطاها خلاص شده باشید. (اگر همچنان خطا دارید، اشتباهات تایپی را در کد و XML خود بررسی کنید.) هنگامی که دیگر کد شما گزارش خطایی نداشت، وقت آن است که برنامه را جوری تغییر دهیم که بتوان با آن تعامل کرد.

تنظیم شنونده‌ها (listeners)

برنامه‌های اندروید معمولا رویداد محور هستند. برخلاف برنامه‌ها یا اسکریپت‌های خط فرمان، برنامه‌های رویداد محور پس از اجرا منتظر یک رویداد مانند فشار دادن یک دکمه توسط کاربر و یا ... می‌مانند. (رویدادها همچنین می‌توانند توسط سیستم عامل یا برنامه دیگری آغاز شوند، اما رویدادهای آغاز شده توسط کاربر واضح ترین‌ها هستند.)

هنگامی که برنامه شما منتظر یک رویداد خاص است، می‌گوییم که در حال "گوش دادن" به آن رویداد است. شیئی که برای پاسخ به یک رویداد ایجاد می‌کنیم، شنونده (listener) نامیده می‌شود و شنونده یک رابط (interface) را برای واکنش به آن رویداد پیاده سازی می‌کند.

Android SDK دارای رابط‌های مختلف جهت پاسخ‌گویی به رویدادهای مختلف است، بنابراین لازم نیست خودمان آن‌ها را بنویسیم. در این مورد، رویدادی که می‌خواهیم به آن گوش دهیم، لمس شدن دکمه است (یا "کلیک شدن"). بنابراین شنونده ما رابط View.OnClickListener را پیاده‌سازی می‌کند.

با دکمه TRUE شروع کنید. در MainActivity.kt، کد زیر را به onCreate(Bundle?) درست بعد از تعریف متغیرها اضافه کنید.

لیست ۱.۷ تنظیم شنونده برای دکمه TRUE (MainActivity.kt)

```
override fun onCreate(savedInstanceState: Bundle?) {
    …
    trueButton = findViewById(R.id.true_button)
    falseButton = findViewById(R.id.false_button)
    trueButton.setOnClickListener { view: View ->
        // Do something in response to the click here
    }
}
```

(اگر خطای Unresolved reference دارید: سعی کنید از Alt+Enter برای import کردن کلاس View استفاده کنید.)

در فهرست ۱.۸ شنونده‌ای را تنظیم می‌کنیم که هنگام فشار دادن دکمه معروف به دکمه trueButton ما را آگاه کند تا پاسخگویی مناسبی داشته باشیم. شنونده onClick با استفاده از عبارت لامبدا پیاده‌سازی می‌شود. شنونده مشابهی را برای دکمه FALSE تنظیم کنید.

لیست ۱.۸ تنظیم شنونده برای دکمه FALSE (MainActivity.kt)

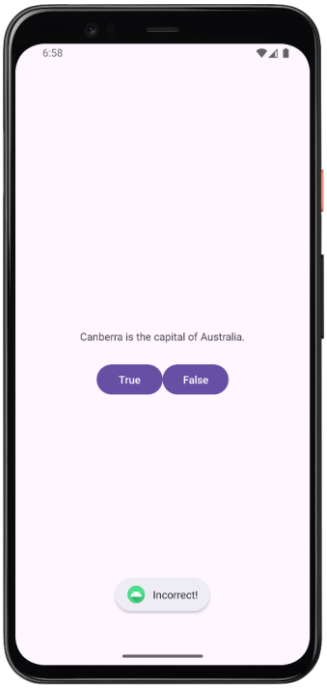
```
override fun onCreate(savedInstanceState: Bundle?) {
    ...
```

```
    trueButton = findViewById(R.id.true_button)
    falseButton = findViewById(R.id.false_button)
    trueButton.setOnClickListener { view: View ->
    }
    falseButton.setOnClickListener { view: View ->
    }
}
```

ایجاد Toast

حالا وقت آن است که دکمه‌ها را کاملا عملیاتی کنیم. می‌خواهیم با فشار دادن هر دکمه یک پیام pop-up به نام Toast ایجاد کنیم. Toast پیام کوتاهی است که کاربر را از چیزی مطلع می‌کند، اما نیازی به ورودی یا اقدامی ندارد. ما می‌خواهیم Toastهایی تهیه کنیم که اعلام می‌کند کاربر پاسخ درست داده یا خیر (شکل ۱.۱۵).

شکل ۱.۱۵ Toast ارائه بازخورد



ابتدا به strings.xml بر گردید و منابع رشته‌ای را که Toastها نمایش می‌دهند را اضافه کنید.

لیست ۱.۹ افزودن رشته‌های Toast (res/values/strings.xml)

```
<resources>
    <string name="app_name">GeoQuiz</string>
    <string name="question_text">
        Canberra is the capital of Australia.
    </string>
    <string name="true_button">True</string>
    <string name="false_button">False</string>
    <string name="correct_toast">Correct!</string>
```

```
    <string name="incorrect_toast">Incorrect!</string>
</resources>
```

در مرحله بعد، شنونده‌های کلیک خود را برای ایجاد و نمایش Toast به روز کنید. از تکمیل کننده کد (code completion) برای کمک استفاده نمایید. تکمیل کننده کد می‌تواند در زمان صرفه‌جویی زیادی ایجاد کند، بنابراین خوب است که زودتر با آن آشنا شوید.

شروع به تایپ کد لیست ۱.۱۰ در MainActivity.kt نمایید. هنگامی که به بعد از Toast می‌رسید، یک پنجره پاپ آپ با لیستی از توابع و ثابت‌های پیشنهادی از کلاس Toast ظاهر می‌شود. برای انتخاب یکی از پیشنهادها، از کلیدهای جهت دار بالا و پایین استفاده کنید. (اگر می‌خواهید تکمیل کننده کد را نادیده بگیرید، می‌توانید به تایپ کردن ادامه دهید. اگر کلید Tab یا کلید Enter را فشار ندهید یا روی پنجره پاپ آپ کلیک نکنید، هیچ کدی کامل نخواهد شد.)

از لیست پیشنهادات، makeText(context: Context, resId: Int, duration: Int) را انتخاب کنید. تکمیل کننده کد، فراخوانی تابع کامل را به برنامه‌مان اضافه می‌کند.

پارامترهای makeText(…) را کامل کنید تا زمانی که کد نشان داده شده در لیست ۱.۱۱ در برنامه کامل شود.

لیست ۱.۱۰ ساخت Toast (MainActivity.kt)

```
override fun onCreate(savedInstanceState: Bundle?) {
    ...
    trueButton.setOnClickListener { view: View ->
        Toast.makeText(this,R.string.correct_toast,
            Toast.LENGTH_SHORT).show()}
    falseButton.setOnClickListener { view: View ->
        Toast.makeText(this,
            R.string.incorrect_toast,
            Toast.LENGTH_SHORT).show()}
}
```

برای ایجاد یک Toast، تابع استاتیک Toast.makeText(Context!, Int, Int) را فراخوانی می‌کنیم. این تابع یک شی Toast ایجاد و پیکربندی می‌کند. پارامتر Context معمولا نمونه‌ای از Activity است (و Activity زیرکلاس Context است). در اینجا نمونه MainActivity را به عنوان آرگومان Context ارسال می‌کنیم.

پارامتر دوم شناسه منبع رشته‌ای است که Toast باید نمایش دهد. Context توسط کلاس Toast مورد نیاز است تا بتواند شناسه منبع رشته را پیدا کرده و از آن استفاده کند. پارامتر سوم یکی از دو ثابت Toast است که مشخص می‌کند Toast چه مدت باید قابل مشاهده باشد.

پس از ایجاد Toast، Toast.show() را روی آن فراخوانی می‌کنیم تا روی صفحه نمایش داده شود. از آنجایی که از تکمیل کننده کد استفاده کردیم، برای import کلاس Toast نیازی به انجام کاری نیست. هنگامی که پیشنهاد تکمیل کننده کد را می‌پذیرید، کلاس‌های لازم به طور خودکار import می‌شوند.

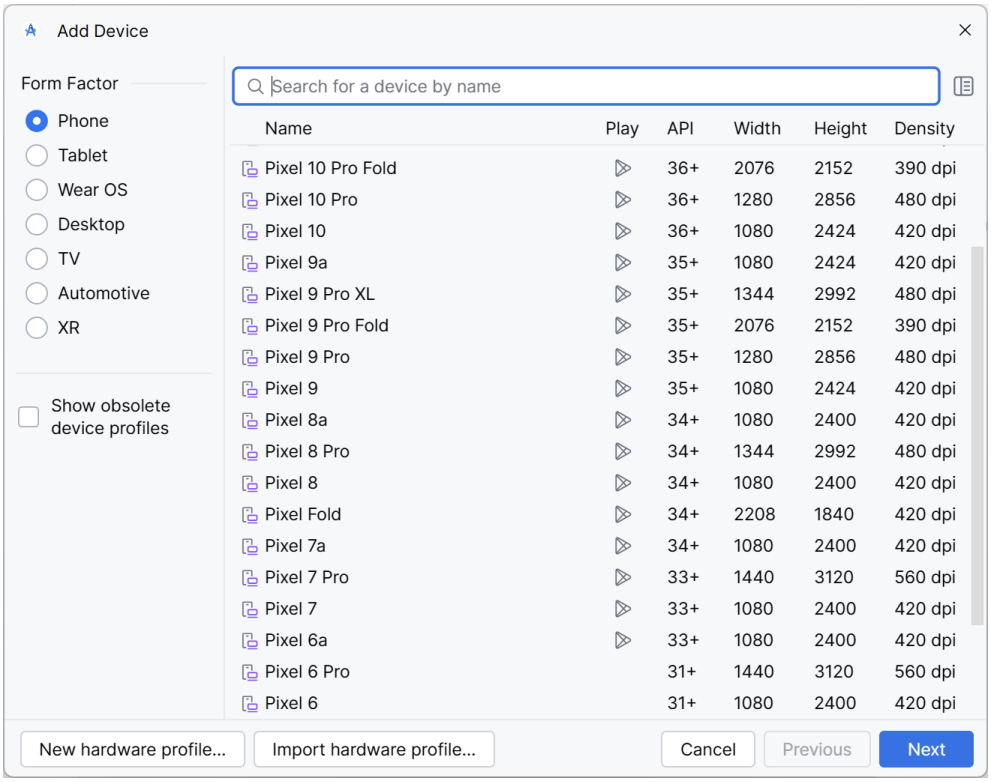
اکنون، بیایید برنامه را در عمل ببینیم.

اجرا بر روی شبیه ساز

برای اجرای یک برنامه اندروید، به یک دستگاه اندرویدی – چه سخت افزای چه یک شبیه‌ساز (مجازی) – نیاز داریم. دستگاه‌های مجازی توسط اندروید استودیو پشتیبانی می‌شوند. برای ایجاد یک دستگاه مجازی Android (یا AVD)، Tools → Device Manager را انتخاب کنید. هنگامی که Device Manager ظاهر شد، روی + در گوشه سمت چپ بالا پنجره کلیک و سپس Create Virtual Device را کلیک نمایید.

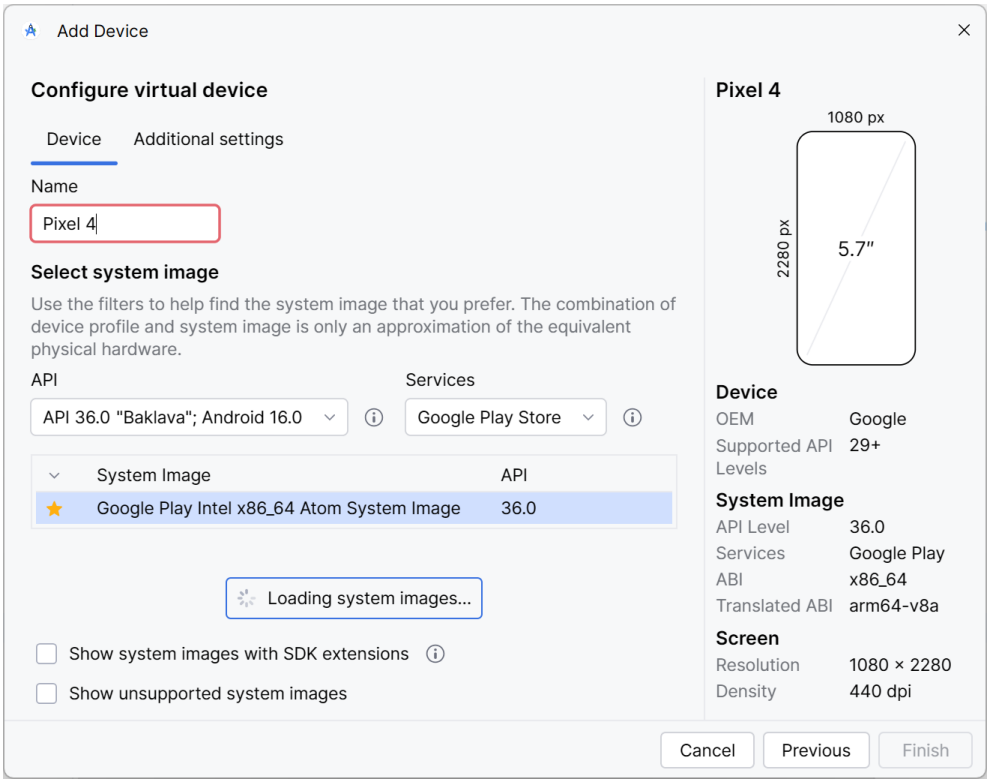
در کادری که ظاهر می‌شود، گزینه‌های زیادی برای پیکربندی یک دستگاه مجازی ارائه شده است. برای ایجاد Device، همانطور که در تصویر ۱.۱۶ نشان داده شده است، گزینه Pixel4 را برای شبیه‌سازی انتخاب و سپس روی Next کلیک کنید.

تصویر ۱.۱۶ انتخاب یک دستگاه مجازی



در صفحه بعدی یک نام، ایمیج سیستمی و نسخه API را برای شبیه‌ساز انتخاب کنید. در نهایت می‌توانید ویژگی‌های شبیه‌ساز را در تب Additional setting بررسی و تغییر دهید. همچنین می‌توان بعدا ویژگی‌های یک شبیه‌ساز موجود را ویرایش کرد. در آخر روی Finish کلیک کنید (شکل ۱.۱۷). (لازم است مراحل دانلود اجزای شبیه‌ساز را قبل از کلیک روی Next دنبال کنید.)

تصویر ۱.۱۷ انتخاب یک ایمیج سیستم



وقتی که یک Device داریم، می‌توانیم GeoQuiz را روی آن اجرا کنیم. از نوار ابزار اندروید استودیو، روی دکمه run کلیک نمایید (نماد سبز رنگ "play" است) یا Shift+F10 را فشار دهید.

راه اندازی شبیه‌ساز و دانلود برخی کامپونت‌ها ممکن است مدتی طول بکشد، از اتصال اینترنت خود مطمئن شوید و صبور بمانید. اما در نهایت برنامه GeoQuiz روی Device که ایجاد کرده‌اید راه اندازی می‌شود. حالا در برنامه‌تان دکمه‌ها را فشار دهید و از دیدن Toastهای خود لذت ببرید.

چالش

چالش‌ها، تمرین‌هایی در پایان فصل هستند که می‌توانید به تنهایی انجام دهید. برخی از آنها آسان و بعضی دیگر سخت و نیاز به حل مسئله دارند. حل آنها آنچه را که آموخته‌اید تقویت می‌کند، اعتماد به نفس تان را بالا می‌برد و شکاف بین آموزش برنامه‌نویسی اندروید و توانایی انجام برنامه‌نویسی اندروید را پر می‌کند.

در اکسپلورر رایانه خود، به دایرکتوری ریشه پروژه خود بروید. پوشه GeoQuiz را کپی کنید و یک نسخه جدید را در کنار نسخه اصلی جایگذاری کنید. نام پوشه جدید را به GeoQuiz Challenge تغییر دهید. به اندروید استودیو برگردید، به File → Open بروید و سپس GeoQuiz Challenge را انتخاب و OK را انتخاب کلیک کنید. پروژه کپی شده در یک پنجره جدید آماده برای کار ظاهر می‌شود.

چالش: سفارشی سازی Toast

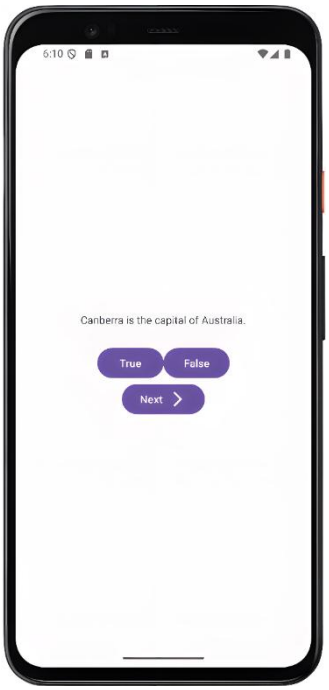
برای این چالش، Toast را سفارشی کنید تا به جای پایین صفحه در بالا نشان داده شود. برای تغییر نحوه نمایش Toast، از تابع setGravity کلاس Toast استفاده کنید. مقدار آن را Gravity.TOP قراردهید.

۲

اندروید و مدل-ویو-کنترلر

در این فصل، همانطور که در شکل ۲.۱ نشان داده شده است، قصد داریم GeoQuiz را برای ارائه بیش از یک سوال ارتقا دهیم.

تصویر ۲.۱ NEXT دکمه جدید

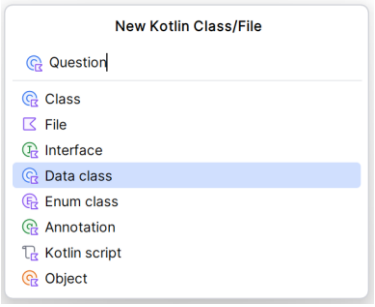


برای تحقق این امر، قصد داریم یک کلاس داده به نام Question را به پروژه GeoQuiz اضافه کنیم. هر نمونه از این کلاس یک سوال با جواب درست یا نادرست را در بر می‌گیرد. سپس، مجموعه‌ای از آبجکت‌های Question را برای مدیریت MainActivity ایجاد خواهیم کرد.

ایجاد یک کلاس جدید

در پنجره ابزار پروژه، روی پکیج com.codeb.geoquiz کلیک راست کرده و New → Kotlin Class/File را انتخاب کنید. Question را برای Name وارد کنید. و در پایین روی گزینه Data class دابل کلیک کنید (تصویر ۲.۲).

تصویر ۲.۲ ایجاد کلاس Question



اندروید استودیو فایلی به نام Question.kt ایجاد و باز می‌کند. در این فایل، دو متغیر عضو در سازنده اضافه کنید.

لیست ۲.۱ افزودن کلاس Question (Question.kt)

```
package com.codeb.geoquiz
import androidx.annotation.StringRes
data class Question(@StringRes val textResId:Int,
                    val answer: Boolean)
```

کلاس Question دو مقدار را در خود دارد: متن سوال و پاسخ سوال (درست یا غلط).

اعلان @StringRes مورد نیاز نیست، اما توصیه می‌کنیم به دو دلیل آن را بنویسید. اول اینکه این انوتیشن‌ها (annotation) به ابزار بررسی کننده کد تعبیه شده در اندروید استودیو (با نام Lint) کمک می‌کند تا در زمان کامپایل تایید کند که سازنده یک شناسه معتبر رشته‌ای ارائه می‌دهد. این کار از خطای زمان اجرا که در آن سازنده با یک شناسه نامعتبر استفاده می‌شود (مانند شناسه‌ای که به منبعی غیر از یک رشته اشاره می‌کند) جلوگیری می‌کند. دوم، انوتیشن کد شما را برای سایر توسعه دهندگان خواناتر می‌کند.

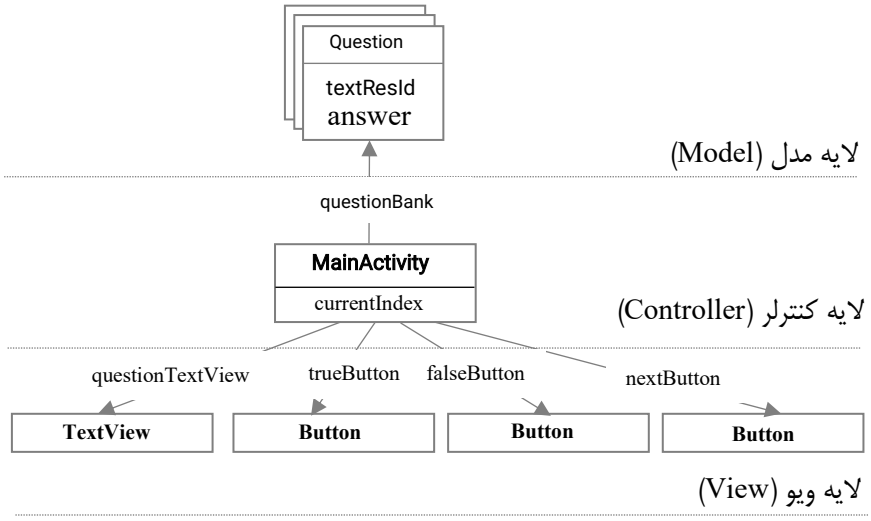
دقت کنید، متغییر textResId یک Int است و یک رشته نیست. زیرا متغیر textResId، شناسه منبع (همیشه یک Int) رشته را برای یک سوال به جای مقدار آن نگه می‌دارد.

ما از کلمه کلیدی data برای تمام کلاس‌های مدل در این کتاب استفاده می‌کنیم. انجام این کار به وضوح نشان دهنده این است که کلاس برای نگهداری داده‌ها است. همچنین، کامپایلر برای کلاس‌های داده می‌تواند کارهای اضافی‌تر انجام دهد که می‌تواند زندگی ما را آسان و دلپذیر کند، مانند تعریف خودکار توابع مفیدی مثل ()equals، ()hashCode و ()toString.

کلاس Question ما اکنون کامل شده است. حال نوبت تغییر MainActivity برای کار با این کلاس است. ابتدا، بیایید نگاهی به نحوه کار قطعات GeoQuiz با هم بیندازیم.

ما از MainActivity انتظار داریم تا لیستی از سوال‌ها ایجاد کند. سپس با TextView آنها را به نمایش بگذارد و در نهایت با سه دکمه و ارائه بازخورد با کاربر تعامل داشته باشد. شکل ۲.۳ این روابط را نشان می‌دهد.

شکل ۲.۳ نمودار آبجکت برای GeoQuiz



اندروید و مدل–ویو–کنترلر (MVC)

توجه داشته باشید که آبجکت‌ها در شکل ۲.۳ به سه بخش با برچسب Model، Controller و View تقسیم شده‌اند. این تقسیم بندی یکی از قدیمی‌ترین و معروف‌ترین الگوهای طراحی نرم‌افزار یعنی مدل–ویو–کنترلر یا MVC است. در اندروید هم از این الگو به خصوص در اَپ‌های ساده‌ای همانند برنامه ما خیلی استفاده می‌شود. پس قصد داریم به توضیح و استفاده این الگو بپردازیم. در معماری MVC، تمام آبجکت‌ها موجود در برنامه‌ی ما باید یکی از این سه نوع باشد: شی مدل، شی ویو یا شی کنترل‌گر.

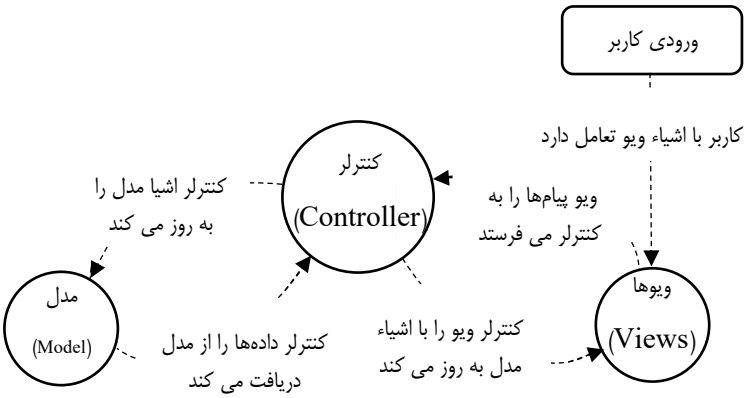
- آبجکت‌های مدل، داده‌های برنامه را در خود جای می‌دهند. کلاس‌های مدل معمولا برای مدل سازی چیزهایی طراحی می‌شوند که برنامه شما به آنها مربوط می‌شود، مانند یک کاربر، یک محصول در یک فروشگاه، یک عکس در سرور، یک برنامه تلویزیونی – یا یک سوال درست/نادرست. آبجکت‌ها مدل هیچ دانشی از رابط کاربری ندارند. تنها هدف آنها نگهداری و مدیریت داده‌ها است. در برنامه‌های اندروید، کلاس‌های مدل به طور کلی کلاس‌های سفارشی هستند که ایجاد می‌کنید. همه آبجکت‌ها مدل در برنامه، لایه مدل آن را می‌سازند. لایه مدل GeoQuiz از کلاس Question تشکیل شده است.

- آبجکت‌ها ویو می‌دانند چگونه خودشان را روی صفحه به نمایش بگذارند و چگونه به ورودی‌های کاربر، مانند لمس‌ها پاسخ دهند. یک قانون ساده این است که اگر چیزی را روی صفحه می‌بینید، پس آن یک View است. اندروید تعداد زیادی کلاس ویوی قابل تنظیم را ارائه می‌دهد. همچنین می‌توانیم کلاس‌های ویوی سفارشی ایجاد کنیم. آبجکت‌ها ویوی یک برنامه لایه ویوی آن را تشکیل می‌دهند. لایه ویوی ویجت‌هایی است که از res/layout/activity_main.xml نمونه سازی می‌شوند.

- آبجکت‌ها کنترلر، ویو و مدل را به هم گره می‌زند. آنها حاوی "منطق برنامه" هستند. کنترلرها برای پاسخگویی به رویدادهای مختلف ایجاد شده توسط آبجکت‌ها ویو و مدیریت جریان داده‌ها به و از آبجکت‌ها مدل و لایه ویو طراحی شده‌اند. در اندروید، یک کنترلر معمولا زیر کلاس Activity یا Fragment است (در مورد فرگمنت‌ها در فصل ۸ بیشتر خواهید شنید). لایه کنترلر GeoQuiz، در حال حاضر، صرفا از MainActivity تشکیل شده است.

شکل ۲.۴ جریان کنترل بین آبجکت‌ها را در پاسخ به یک رویداد کاربر، مانند فشار دادن یک دکمه، نشان می‌دهد. توجه داشته باشید که آبجکت‌ها مدل و ویو مستقیما با یکدیگر صحبت نمی‌کنند بلکه کنترل کننده‌ها در میان قرار دارند و پیام‌هایی را از برخی آبجکت‌ها دریافت و دستورالعمل‌ها را به برخی دیگر ارسال می‌کنند.

شکل ۲.۴ جریان MVC با ورودی کاربر



تصمیم‌گیری برای استفاده از MVC

یک برنامه می‌تواند آن‌قدر ویژگی‌های مختلف را در خود جمع کند که بیش از حد پیچیده شود و درک آن دشوار گردد. تقسیم کردن کد به کلاس‌ها به شما کمک می‌کند تا طراحی و درک کلی برنامه آسان‌تر شود؛ به جای فکر کردن به متغیرها و توابع جداگانه، می‌توانید در قالب کلاس‌ها فکر کنید.

به همین صورت، تقسیم کردن کلاس‌ها به لایه‌های مدل، ویو و کنترلر به شما کمک می‌کند تا طراحی و درک برنامه آسان‌تر شود؛ می‌توانید به جای فکر کردن به کلاس‌های جداگانه، در قالب لایه‌ها فکر کنید.

گرچه GeoQuiz یک برنامه پیچیده نیست، اما همچنان می‌توانید مزایای جدا نگه داشتن لایه‌ها را مشاهده کنید. وقتی این کار را انجام دهید، نیازی نخواهید داشت که حتی یک نکته درباره‌ی کلاس Question که تازه ایجاد کرده‌اید را به خاطر بسپارید. معماری MVC همچنین استفاده‌ی مجدد از کلاس‌ها را آسان‌تر می‌کند. کلاسی که مسئولیت‌های محدودی دارد، قابلیت استفاده‌ی مجدد بیشتری نسبت به کلاسی که در بسیاری موارد دخیل است، دارد.

به عنوان مثال، کلاس مدل ما، Question، چیزی در مورد ویجت‌های مورد استفاده برای نمایش یک سوال درست/نادرست نمی‌داند. این کار استفاده از Question را در سراسر برنامه برای اهداف مختلف آسان می‌کند. به عنوان مثال، اگر می‌خواهید لیستی از همه سوالات را به طور همزمان نمایش دهید، می‌توانید از همان شیی که در اینجا استفاده می‌کنید برای نمایش فقط یک سوال در یک زمان دیگر نیز استفاده کنید.

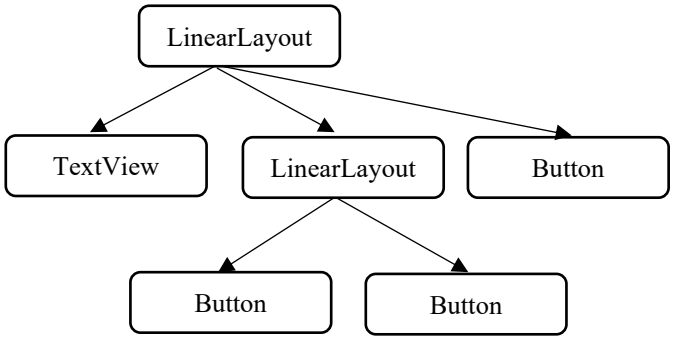
معماری MVC برای اپلیکیشن‌های کوچک و ساده مثل GeoQuiz بسیار خوب عمل می‌کند. اما در اپلیکیشن‌های بزرگ‌تر و پیچیده‌تر، لایه‌ی کنترل می‌تواند بسیار حجیم و پیچیده شود. به‌طور کلی، بهتر است اکتیویتی‌ها و سایر کنترل‌گرها را ساده نگه دارید. اکتیویتی‌های ساده شامل کمترین منطق تجاری و تنظیمات اولیه هستند. زمانی که MVC دیگر امکان ایجاد کنترل‌گرهای کم حجم را در برنامه‌ی شما فراهم نکند، باید به الگوهای جایگزین مانند Model-View-ViewModel فکر کنید (که در فصل ۱۹ با آن آشنا خواهید شد).

بروزرسانی لایه View

اکنون که با MVC آشنا شدید، می‌خواهیم لایه ویوی GeoQuiz را به روز و دکمه NEXT را در آن قرار دهیم.

در اندروید، آبجکت‌ها لایه نمایش معمولا از XML یک فایل لیوت نمونه‌سازی می‌شوند. تنها لیوت GeoQuiz در activity_main.xml تعریف شده است. این لیوت باید همانطور که در شکل ۲.۵ نشان داده شده است به روز شود. (توجه داشته باشید که ویژگی‌های ویجت‌های بدون تغییر را اینجا نمی‌آوریم.)

تصویر ۲.۵ دکمه جدید!



بنابراین تغییراتی که باید در لایه view ایجاد کنید عبارتند از:

- به TextView ویژگی android:id بدهید. این ویجت به یک شناسه منبع نیاز دارد تا بتوانیم متن آن را در کد MainActivity تنظیم کنید. متن TextView را در مرکز ویوی TextView قرار دهید. برای این کار Gravity را روی "Center" تنظیم کنید.
- ویژگی android:text را از TextView حذف کنید. نمی‌خواهیم یک سوال ثابت همیشگی نمایش داده شود. در عوض، متن سوال را به صورت پویا تنظیم خواهیم کرد.
- برای مشخص کردن یک رشته‌ی پیش‌فرض برای TextView به هنگام پیش‌نمایش لیوت، ویژگی tools:text را به یک منبع رشته‌ای که نمایانگر یک سؤال است با استفاده از /string@ ارجاع دهید.

همچنین باید فضای نام tools را به تگ ریشه‌ی فایل layout اضافه کنیم تا Android Studio بتواند ویژگی tools:text را تشخیص دهد. این فضای نام به شما اجازه می‌دهد هر ویژگی‌ای را روی یک ویجت فقط برای نمایش در پیش‌نمایش اندروید استودیو بازنویسی کنید.

ویژگی‌های tools هنگام اجرای برنامه روی دستگاه نادیده گرفته می‌شوند. می‌توانید از android:text استفاده کرده و مقدار آن را در زمان اجرا بازنویسی کنید، اما استفاده از tools:text مشخص می‌کند که مقدار ارائه شده فقط برای پیش‌نمایش است.

- ویجت Button جدید را به عنوان فرزند ریشه LinearLayout اضافه کنید.

به activity_main.xml برگردید و کارهای گفته شده بالا را همانند لیست زیر انجام دهید.

لیست ۲.۲ دکمه جدید و تغییرات TextView (res/layout/activity_main.xml)

```
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    xmlns:tools="http://schemas.android.com/tools"
    android:gravity="center"
    android:orientation="vertical"
    android:id="@+id/main">
    <TextView
        android:id="@+id/question_text_view"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:padding="24dp"
        android:text="@string/question_text"
        tools:text="@string/question_australia"/>
    <LinearLayout
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:orientation="horizontal">
        <Button
            android:id="@+id/true_button"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="@string/true_button"
            />
        <Button
            android:id="@+id/false_button"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="@string/false_button" />
    </LinearLayout>
    <Button
        android:id="@+id/next_button"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="@string/next_button" />
</LinearLayout>
```

خطاهای آشنایی را در مورد منابع رشته‌ای مشاهده خواهید کرد.

به res/values/strings.xml برگردید. نام question_text را تغییر دهید و یک رشته برای دکمه جدید اضافه کنید.

لیست ۲.۳ به روز رسانی رشته‌ها (res/values/strings.xml)

```
<resources>
    <string name="app_name">GeoQuiz</string>
    <string name="question_text">
        Canberra is the capital of Australia.
    </string>
    <string name="question_australia">
        Canberra is the capital of Australia.
    </string>
    <string name="true_button">True</string>
    <string name="false_button">False</string>
    <string name="next_button">Next</string>
    ...
</resources>
```

در حالی که strings.xml باز است، ادامه دهید و رشته‌ها را برای بقیه سوالات جغرافیایی که به کاربر نشان داده می‌شود، اضافه کنید.

لیست ۲.۴ افزودن سوالات بیشتر (res/values/strings.xml)

```
<resources>
    ...
    <string name="question_australia">
    Canberra is the capital of Australia.</string>
    <string name="question_oceans">
        The Pacific Ocean is larger than the Atlantic Ocean.
    </string>
    <string name="question_mideast">
        The Suez Canal connects the Red Sea
        and the Indian Ocean.
    </string>
    <string name="question_africa">
        The source of the Nile River is in Egypt.
    </string>
    <string name="question_americas">
        The Amazon River is the longest river
        in the Americas.
    </string>
    <string name="question_asia">
        Lake Baikal is the world\'s oldest and deepest
        freshwater lake.
```

```
    </string>
    ...
</resources>
```

به activity_main.xml برگردید و تغییرات لیوت خود را مشاهده کنید. تمام این‌ها برای لایه ویوی GeoQuiz بود. وقت آن است که به کلاس کنترلر خود (MainActivity) بپردازیم.

بروزرسانی لایه کنترلر

در فصل قبل در کنترلر GeoQuiz یعنی MainActivity، کار خاصی برای انجام نداشتیم. تنها، لیوت تعریف شده در activity_main.xml را به نمایش گذاشتیم و دو دکمه را برای نمایش Toast برنامه‌نویسی کردیم. اکنون که چندین سوال برای بازیابی و نمایش دارید، MainActivity باید کارهای بیشتری انجام دهد و دو لایه Model و View را به هم متصل کند.

MainActivity.kt را باز کنید. فهرستی از آبجکت‌ها سوال و یک ایندکس برای لیست ایجاد کنید.

لیست ۲.۵ افزودن لیست سوالات (MainActivity.kt)

```
class MainActivity : AppCompatActivity() {
    private lateinit var trueButton: Button
    private lateinit var falseButton: Button
    private val questionBank = listOf(
        Question(R.string.question_australia, true),
        Question(R.string.question_oceans, true),
        Question(R.string.question_mideast, false),
        Question(R.string.question_africa, false),
        Question(R.string.question_americas, true),
        Question(R.string.question_asia, true))
    private var currentIndex = 0
    …
}
```

در اینجا چندین بار سازنده کلاس Question را فراخوانی و لیستی از سوالات ایجاد می‌کنید. (در یک پروژه پیچیده‌تر، این لیست ایجاد و در محل مناسب‌تری ذخیره می‌شود. در برنامه‌های بعدی، گزینه‌های بهتری برای ذخیره داده‌های مدل مشاهده خواهید کرد. در حال حاضر، هدف ساده نگه داشتن لایه کنترلر است.)

قرار است با استفاده از questionBank، currentIndex و خصوصیات کلاس Question رژه‌ای از سوالات روی صفحه ایجاد کنیم.

ابتدا متغییرهایی را برای TextView و دکمه جدید اضافه می‌کنیم. سپس به TextView ارجاع می‌دهیم و متن آن را سوال فعلی در فهرست قرار می‌دهیم. باید ارجاع دیگری نیز برای دکمه جدید دریافت کنیم (بعدا رویداد کلیک را برای دکمه NEXT تنظیم خواهید کرد).

لیست ۲.۶ اتصال ویجت‌ها (MainActivity.kt)

```
class MainActivity : AppCompatActivity() {
    private lateinit var trueButton: Button
    private lateinit var falseButton: Button
    private lateinit var nextButton: Button
    private lateinit var questionTextView: TextView
    …
    override fun onCreate(savedInstanceState: Bundle?) {
        …
        nextButton = findViewById(R.id.next_button)
        questionTextView =
            findViewById(R.id.question_text_view)
        trueButton.setOnClickListener { view: View ->
            …
        }
        falseButton.setOnClickListener { view: View ->
            …
        }
        val questionTextResId =
            questionBank[currentIndex].textResId
        questionTextView.setText(questionTextResId)
    }
}
```

فایل‌های خود را ذخیره و هر گونه خطا را بررسی کنید. سپس GeoQuiz را اجرا کنید. باید اولین سوال در لیست در TextView ظاهر شود. حالا بیایید سراغ دکمه NEXT برویم. یک OnClickListener روی آن تنظیم کنید. این شنونده ایندکس را افزایش می‌دهد و متن TextView را به روز می‌کند. یعنی موقع لمس دکمه NEXT سوال بعدی نمایش داده می‌شود..

لیست ۲.۷ پیوند دکمه جدید (MainActivity.kt)

```
override fun onCreate(savedInstanceState: Bundle?) {
    …
    falseButton.setOnClickListener {
    …
    }
    nextButton.setOnClickListener {
        currentIndex = (currentIndex + 1) % questionBank.size
        val questionTextResId =
```

```
            questionBank[currentIndex].textResId
        questionTextView.setText(questionTextResId)
        }
        val questionTextResId =
            questionBank[currentIndex].textResId
        questionTextView.setText(questionTextResId)
}
```

اکنون کد یکسانی را در دو مکان جداگانه که متن نمایش داده شده در questionTextView را به روز می‌کنند، داریم. لحظه‌ای وقت بگذارید و همانند فهرست ۲.۸ این کد را در یک تابع قرار دهید. سپس آن تابع را در شنونده nextButton و در انتهای onCreate(Bundle?) فراخوانی کنید.

لیست ۲.۸ کپسوله سازی با یک تابع (MainActivity.kt)

```
override fun onCreate(savedInstanceState: Bundle?) {
    …
    falseButton.setOnClickListener {
    …
    }
    nextButton.setOnClickListener {
        currentIndex = (currentIndex + 1) % questionBank.size
        val questionTextResId =
            questionBank[currentIndex].textResId
        questionTextView.setText(questionTextResId)
        updateQuestion()
    }
    val questionTextResId =
        questionBank[currentIndex].textResId
    questionTextView.setText(questionTextResId)
    updateQuestion()
}
private fun updateQuestion() {
    val questionTextResId =
        questionBank[currentIndex].textResId
    questionTextView.setText(questionTextResId)
}
```

GeoQuiz را اجرا و دکمه NEXT خود را تست کنید. اکنون که در نمایش سوالات برنامه عملکرد مناسبی دارد، وقت آن است که به سراغ پاسخ‌ها برویم. به جای نوشتن کد مشابه در دو مکان، یک تابع خصوصی به MainActivity اضافه می‌کنید تا کد را کپسوله کنید:

```
private fun checkAnswer(userAnswer: Boolean)
```

این تابع یک متغیر Boolean را می‌پذیرد که مشخص می‌کند کاربر دکمه TRUE یا FALSE را فشار داده است. سپس پاسخ کاربر را با پاسخ شی Question فعلی بررسی می‌کند. در نهایت، پس از تعیین اینکه آیا کاربر به درستی پاسخ داده است یا خیر، یک Toast با پیام مناسب ایجاد و به کاربر بر می‌گرداند.

در MainActivity.kt، پیاده‌سازی checkAnswer(Boolean) را همانند لیست ۲.۹ اضافه کنید.

لیست ۲.۹ افزودن checkAnswer (Boolean)

```
override fun onCreate(savedInstanceState: Bundle?) {
    …
}
private fun updateQuestion() {
    …
}
private fun checkAnswer(userAnswer: Boolean) {
    val correctAnswer = questionBank[currentIndex].answer
    val messageResId = if (userAnswer == correctAnswer) {
        R.string.correct_toast
    } else {
        R.string.incorrect_toast
    }
    Toast.makeText(this, messageResId, Toast.LENGTH_SHORT)
        .show()
}
```

در شنونده دکمه‌ها همانطور که در لیست ۲.۱۰ نشان داده شده است، checkAnswer(Boolean) را فراخوانی کنید.

لیست ۲.۱۰ فراخوانی checkAnswer(Boolean)

```
override fun onCreate(savedInstanceState: Bundle?) {
    …
    trueButton.setOnClickListener { view: View ->
        Toast.makeText(
            this,R.string.correct_toast,
            Toast.LENGTH_SHORT).show()
        checkAnswer(true)
    }
    falseButton.setOnClickListener { view: View ->
        Toast.makeText(
            this,R.string.incorrect_toast,
            Toast.LENGTH_SHORT).show()
        checkAnswer(false)
    }
    nextButton.setOnClickListener {
        currentIndex = (currentIndex + 1) % questionBank.size
```

```
            updateQuestion()
        }
        updateQuestion()
    }
```

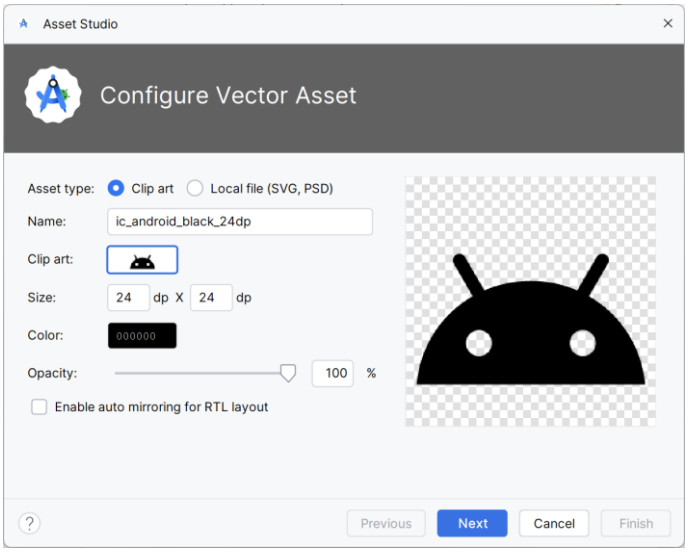
GeoQuiz را اجرا کنید. بررسی کنید که آیا Toastها پیام درستی بر اساس پاسخ سوال فعلی به نمایش می‌گذارند یا خیر.

افزودن آیکون

GeoQuiz در حال اجرا است، تجسم کنید اگر کنار نوشته دکمه NEXT آیکون فلش سمت راست نیز قرار داشت، رابط کاربری چقدر زیباتر به نظر می‌رسید.

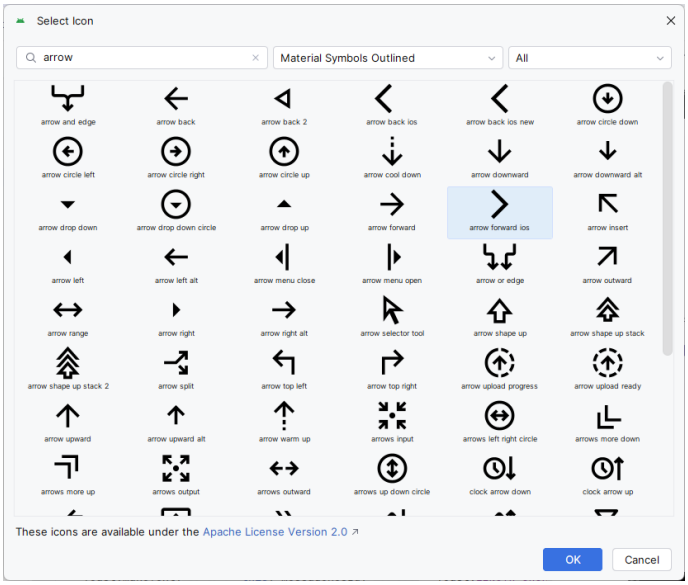
برای اضافه کردن آیکون فلش روی پوشه res راست‌کلیک کرده و گزینه New → Vector Asset را انتخاب کنید. در پنجره ظاهر شده (شکل) سپس روی Clip Art کلیک کنید.

تصویر ۲.۶ آیکون جدید!



در پنجره باز شده (شکل ۲.۷) کلمه arrow را جستجو نمایید و سپس آیکون مورد نظر را انتخاب کنید.

تصویر ۲.۷ آیکون جدید!



نام آن را ic_arrow_right گذاشته، رنگ سفید را برای آیکون انتخاب، Next و سپس Finish را انتخاب نمایید. در نهایت در کد xml مربوط به دکمه Next ویژگیandroid:drawableEnd را مانند لیست زیر اضافه نمایید.

```
...
<Button
    android:id="@+id/next_button"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="@string/next_button"
    android:drawableEnd="@drawable/ic_arrow_right"/>
...
```

حالا دکمه Next در پیش‌نمایش دارای یک آیکون زیباست.

افزودن منابع به یک پروژه

با افزودن یک آیکون به دکمه Next، ما منبعی را به پروژه اضافه کردیم. حالا می‌خواهیم پا را فراتر گذاشته و توضیحات مفصلی در این مورد ارایه دهیم. این کار را با افزودن یک آیکون به عنوان آیکون اصلی برنامه انجام می‌دهیم. آیکون خود برنامه یا آیکون لانچر، آیکونی است که معرف برنامه ما در قسمت برنامه‌های گوشی، همانند شکل زیر است.

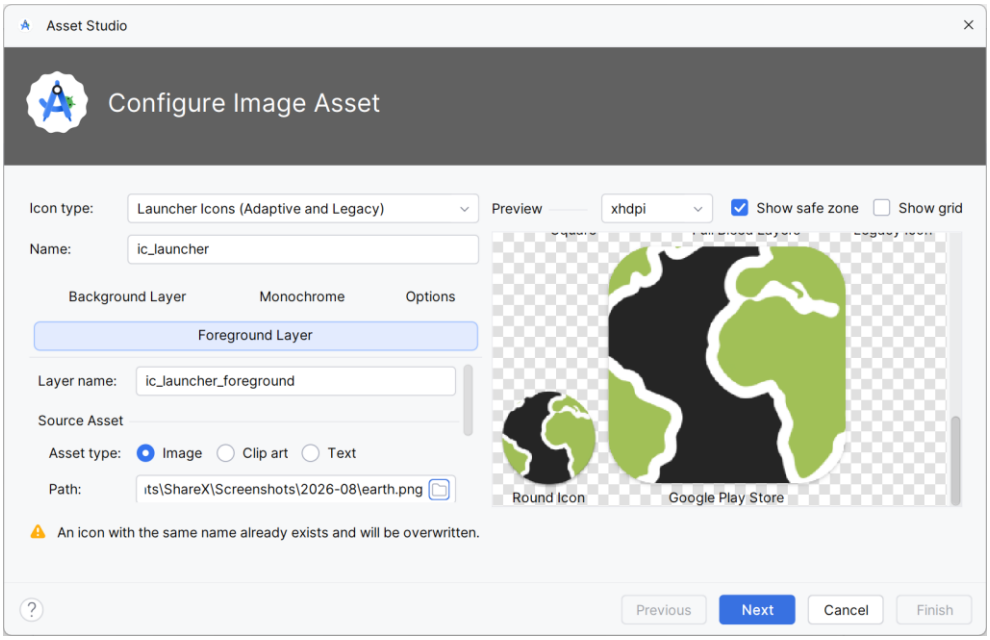
تصویر ۲.۸ آیکون لانچر



اولین مرحله از انجام کار انتخاب یک تصویر مناسب با موضوع برنامه است. ما تصویر زیر را که تصویری از کره زمین است، انتخاب کردیم.

در مرحله بعد روی پوشه res راست کلیک کرده و گزینه New → Image Asset را انتخاب کنید. در پنجره ظاهر شده Icon type را Luncher Icons (Adaptive and Legacy) انتخاب و گزینه Path را مسیر فایل تصویری که دانلود کردیم، انتخاب نمایید. سایر گزینه‌ها را همانطوری که هستند رها کنید و دکمه Next کلیک نمایید.

تصویر ۲.۹ ایجاد آیکون لانچر



تصویر ۲.۱۰ تصویر آیکون



در پنجره بعدی finish را بزنید تا آیکون ساخته شود. حالا یک بار برنامه را اجرا نمایید. سپس به صفحه اصلی برگشته و آیکون تازه GeoQuiz را همانند شکل زیر مشاهده نمایید.

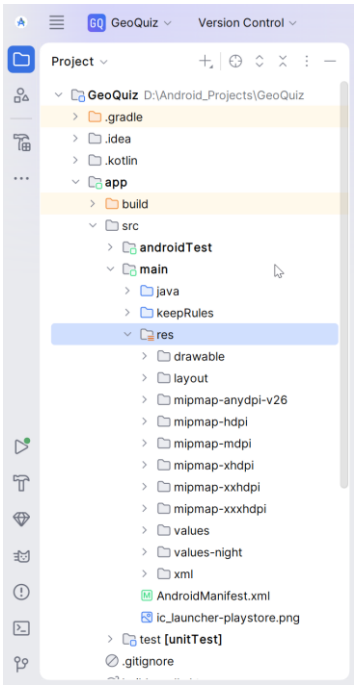
کار راحتی بود، نه؟

تصویر ۲.۱۱ آیکون لانچر



حالا بیایید ببینیم اندروید چگونه با منابع این چنینی رفتار می‌کند. از فعال بودن نوع نمایش project در پنجره ابزار پروژه اطمینان حاصل کنید. همانطور که در شکل ۲.۱۲ نشان داده شده است، محتویات GeoQuiz/app/src/main/res را گسترش دهید.

شکل ۲.۱۲ بررسی منابع در ویوی پروژه



پنج دایرکتوری mipmap-hdpi، mipmap-mdpi، mipmap-xhdpi، mipmap-xxhdpi و mipmap-xxxhdpi را تماشا کنید و فعلا پوشه mipmap-anydpi-v26 را نادیده بگیرید. اگر هر یک از این پوشه‌ها را گسترش دهید فایلی با سه نوع که با نام ic_luncher شروع می شود، خواهید دید. با کلیک روی این فایل‌ها تصویری را مشاهده می‌کنید که برای آیکون برنامه استفاده کردیم. پوشه mipmap برای قرار دادن آیکون اصلی اپلیکیشن استفاده می‌شود. اما چرا پنج نمونه از این فایل‌ها وجود دارد؟

جواب سوال در نحوه استفاده اندروید از منابع تصویری در دستگاه‌های مختلف است. دستگاه‌های مختلف، تراکم صفحه مختلف دارند. این فایل‌ها برای تراکم صفحه نمایش مشخص شده در نام دایرکتوری سفارشی شده‌اند. پسوندهای این دایرکتوری‌ها به تراکم پیکسل صفحه نمایش یک دستگاه اشاره دارد:

- mdpi: صفحه نمایش با چگالی متوسط (~160dpi)
- hdpi: صفحه نمایش با چگالی بالا (~240dpi)
- xhdpi: صفحه نمایش با چگالی خیلی بالا (~320dpi)
- xxhdpi: صفحه نمایش با چگالی خیلی خیلی بالا (~480dpi)
- xxxhdpi: صفحه نمایش با چگالی خیلی خیلی خیلی بالا (~640dpi)

هنگامی که برنامه اجرا می شود، سیستم عامل متناسب با تراکم دستگاهی که برنامه روی آن اجرا می‌شود، فایل تصویری مناسب را انتخاب می‌کند. حالا پوشه drawable را ببینید. این پوشه شامل منابع تصویری برنامه ما می‌باشد. برای ارایه یک منبع تصویری باید همانند mipmap، پنج پوشه با پسوندهای hdpi-، -mdpi، xhdpi-، xxhdpi- و xxxhdpi- بسازیم. یعنی پروژه ما باید دارای پوشه‌های drawable-hdpi، drawable-mdpi، drawable-xhdpi، drawable-xxhdpi و drawable-xxxhdpi باشد. سپس پنج نوع تصویر را برای نمایش در تراکم‌های مختلف در آنها کپی کنیم. توجه داشته باشید که با چندین بار کپی کردن تصاویر، حجم برنامه خود را افزایش می‌دهیم.

اما خبر خوب این است که اگر برنامه‌ای روی دستگاهی اجرا شود که تراکم صفحه نمایش آن در هیچ یک از شرایط تراکم صفحه نمایش برنامه گنجانده نشده باشد، اندروید به طور خودکار تصویر موجود را به اندازه مناسب دستگاه مقیاس می‌کند. به لطف این ویژگی، نیازی به ارائه تصاویر برای همه دسته‌های تراکم پیکسلی نیست. برای کاهش اندازه برنامه خود، می‌توانیم روی یک یا چند دسته با وضوح بالاتر تمرکز کنیم و بقیه کار را به اندروید بسپارید. (در فصل مربوطه جایگزین‌های مناسبی برای مشکل چگالی‌های مختلف ارایه خواهد شد.)

گنجاندن تصاویر در برنامه به همین سادگی است. به هر فایل png، jpg یا gif که به پوشه res/drawable اضافه می‌کنیم، به طور خودکار یک شناسه منبع اختصاص داده می‌شود.

این شناسه‌های منبع با توجه به تراکم صفحه نمایش ایجاد نمی‌شوند، بنابراین نیازی به تعیین تراکم صفحه نمایش دستگاه در زمان اجرا نیست. تنها کاری که باید انجام دهیم این است که از این شناسه منبع در کد خود استفاده کنیم. دقیقا همانطوری که برای استفاده از آیکون فلش برای دکمه Next استفاده کردیم. در یک XML، ما به یک منبع با نوع و نام آن ارجاع می‌دهیم. ارجاع به یک منبع رشته‌ای با /string@ شروع می‌شود. ارجاع به یک منبع تصویری با /drawable@ شروع می‌شود.

هنگامی که برنامه اجرا می‌شود، سیستم عامل تصویر مناسب را برای نمایش در آن دستگاه خاص تعیین می‌کند.

در فصل ۳ با نحوه عملکرد سیستم منابع اندروید (Android resource system) بیشتر آشنا خواهید شد.

تراکم پیکسل صفحه نمایش

در activity_main.xml، مقادیر ویژگی را بر حسب واحدهای dp مشخص کردیم. اکنون وقت آن است که یاد بگیریم مفهوم این کار چیست.

گاهی اوقات باید مقادیری را برای ویژگی‌های view بر حسب اندازه‌های خاص (معمولا بر حسب پیکسل، گاهی پوینت، میلی‌متر یا اینچ) مشخص کنیم. این ویژگی‌ها را معمولا در اندازه متن، margin و padding می‌بینید. اندازه نوشتار، ارتفاع پیکسل متن روی صفحه دستگاه است. margin فاصله بین ویوها و padding فاصله بین لبه‌های بیرونی ویو و محتوای آن را مشخص می‌کند.

همانطور که مشاهده کردید، اندروید به طور خودکار تصاویر را با استفاده از توصیفگرهای تراکم (مانند mipmap-xhdpi) به تراکم پیکسل‌های مختلف صفحه نمایش تبدیل می‌کند. اما چه اتفاقی می‌افتد وقتی تصاویر شما مقیاس شوند اما حاشیه‌ها (margins) نه؟ یا زمانی که کاربر اندازه متن بزرگتر از پیش‌فرض را تنظیم می‌کند؟

برای حل این مشکلات، اندروید واحدهای ابعاد مستقل از تراکم را ارائه می‌دهد که می‌توانیم از آنها برای به دست آوردن اندازه یکسان در تراکم‌های صفحه نمایش مختلف استفاده کنیم. اندروید این واحدها را در زمان اجرا به پیکسل ترجمه می‌کند، بنابراین هیچ عملیات ریاضی دشواری وجود ندارد.

- px: مخفف Pixel است. نشان دهنده دقیقا یک پیکسل روی صفحه بدون در نظر گرفتن تراکم صفحه نمایش است. از آنجایی که پیکسل‌ها با تراکم نمایش دستگاه به طور مناسب مقیاس نمی‌شوند، استفاده از آنها توصیه نمی‌شود.
- dp: مخفف density-independent pixel است. معمولا از این واحدها برای margin، padding یا هر چیز دیگری که با مقدار پیکسل تعیین نمی‌شود، استفاده می‌کنید. یک dp همیشه ۱/۱۶ اینچ روی صفحه نمایش دستگاه است. بدون توجه به تراکم صفحه نمایش، اندازه یکسانی دریافت می‌کنیم. وقتی صفحه نمایش شما تراکم بالاتری دارد، پیکسل‌های dp گسترش می‌یابند تا تعداد بیشتری پیکسل صفحه را پرکند.
- sp: مخفف scale-independent pixel است. پیکسل‌های مستقل از چگالی هستند که اندازه فونت را تقریبا همیشه با آن مشخص می‌کنیم.
- pt, mm, in: این واحدها مانند dp نسبت به تراکم صفحه یا تنظیمات کاربر تغییر می‌کنند و به شما امکان می‌دهند اندازه را برحسب پوینت (۱/۷۲ اینچ)، میلی متر یا اینچ مشخص کنید. با این

حال، استفاده از آنها را توصیه نمی‌کنیم، زیرا همه دستگاه‌ها به درستی تنظیم نیستند تا این واحدها به درستی کار کنند.

در عمل و در این کتاب، تقریبا به طور انحصاری از dp و sp استفاده خواهیم کرد. اندروید این مقادیر را در زمان اجرا به پیکسل ترجمه می‌کند.

اجرا بر روی دستگاه واقعی

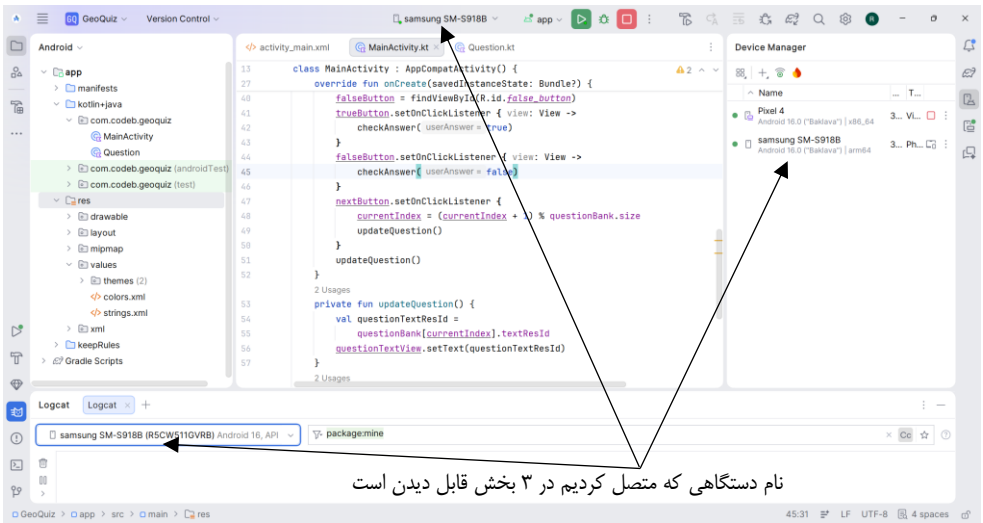
تعامل با برنامه در شبیه‌ساز بسیار جالب بود. اما تعامل در دستگاه فیزیکی اندرویدی حتی از آن هم جالب‌تر است. در این بخش، سیستم و دستگاه را برای اجرای GeoQuiz بر روی دستگاه سخت افزاری خود تنظیم خواهیم کرد.

ابتدا دستگاه را به رایانه خود وصل کنید، سپس USB debugging را در دستگاه اندرویدی خود فعال نمایید. برای انجام این کار، باید به منوی تنظیمات رفته، سپس وارد Developer Options شوید. البته به طور پیش‌فرض گزینه Developer Options قابل مشاهده نیست. برای فعال کردن این گزینه، از تنظیمات به About Tablet/Phone بروید. وارد گزینه Software information شوید و Build number را پیدا کرده و هفت بار پشت سر هم آن را لمس کنید. پس از چندین لمس، پیامی را مشاهده خواهید کرد که به شما می‌گوید چند کلیک دیگر با فعال کردن Developer Options فاصله دارید. بعدا از وارد کردن پین گوشی پیغام "Developer mod has been turned on!" دریافت می‌کنید، حالا در تنظیمات به گزینه Developer options دسترسی خواهید داشت. به تنظیمات برگردید، این گزینه را پیدا کرده و USB debugging را فعال کنید.

البته در دستگاه‌های مختلف مکان این گزینه‌ها بسیار متفاوت است. اگر در فعال کردن دستگاه خود مشکل دارید، به developer.android.com/tools/device.html برای کمک بیشتر رجوع کنید. همچنین در بعضی از دستگاه‌های جدید می‌توان بیسیم وصل شد.

در نهایت، در پنجره ابزار Device Manager با دیدن نام دستگاه‌تان، از اتصال آن مطمئن شوید.

تصویر ۲.۱۳ مشاهده دستگاه‌های متصل



اگر در شناسایی دستگاه خود مشکل دارید، بررسی کنید که دستگاه شما روشن و گزینه USB debugging فعال باشد.اگر همچنان نمی‌توانید دستگاهتان را ببینید، می‌توانید راهنمایی بیشتری در سایت توسعه دهندگان Android پیدا کنید. با developer.android.com/tools/device.html شروع کنید.

توجه داشته باشید که دستگاهتان در منوی کشویی بالای صفحه انتخاب شده باشد. سپس GeoQuiz را مانند قبل اجرا کنید. این بار برنامه در گوشی ما نصب و اجرا می‌شود.

چالش: افزودن یک Listener به TextView

دکمه NEXT شما مناسب است، اما می‌توانید برنامه را طوری تغییر دهید تا کاربر بتواند، TextView را برای دیدن سوال بعدی لمس کند.

چالش: افزودن دکمه قبلی

دکمه‌ای اضافه کنید که کاربر بتواند آن را برای دیدن سوال قبلی لمس کند.

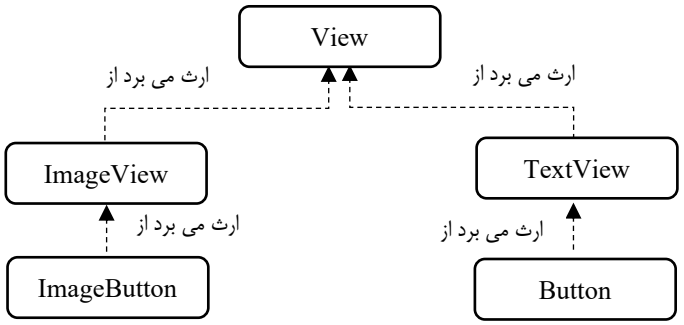
این یک چالش بزرگ است. این امر مستلزم آن است که بسیاری از مراحل این دو فصل را دنبال کنید.

چالش: از Button به ImageButton

شاید اگر دکمه‌های بعدی و قبلی فقط آیکون‌ها را نشان می‌دادند، رابط کاربری حتی بهتر به نظر می‌رسید.

برای انجام این چالش، این دو ویجت باید به ImageButtons تبدیل شوند. ImageButton ویجتی است که از ImageView ارث‌بری دارد. حال آنکه Button ازTextView ارث‌بری دارد. شکل ۲.۱۶ سلسله مراتب مختلف وراثت آنها را نشان می‌دهد.

شکل ۲.۱۶ نمودار وراثت برای ImageButton و Button



می‌توانید ویژگی‌های text و drawable را در دکمه NEXT با تنها یک ویژگی ImageView جایگزین کنید:

```
<~~Button~~ ImageButton
    android:id="@+id/prev_button"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
```

```
android:text="@string/prev_button"
android:drawableLeft="@drawable/ic_arrow_left"
android:drawablePadding="4dp"
android:src="@drawable/ic_arrow_left"/>
```

البته، برای کار با ImageButton باید MainActivity را نیز تغییر دهید. پس از اینکه دکمه‌ها را به ImageButtons تغییر دادید، اندروید استودیو در مورد نبودن ویژگی android:contentDescription هشدار می‌دهد. این ویژگی از دسترسی برای کاربران دارای اختلالات بینایی پشتیبانی می‌کند. مقدار رشته‌ای برای آن تنظیم کنید تا زمانی که کاربر تنظیمات مناسب را اعمال کرده باشند، با صدای بلند خوانده شود. برای تکمیل چالش، یک ویژگی android:contentDescription به هر ImageButton اضافه کنید.

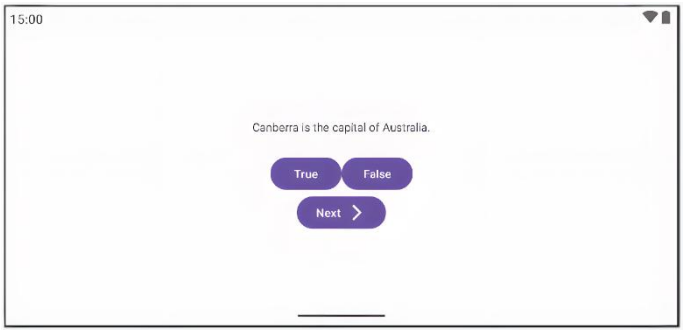
۳

چرخه حیات اکتیویتی

در این فصل علت "مشکل چرخش صفحه" یا "rotation problem" که ترسناک و رایج است را کشف خواهیم کرد. این مشکل یکی از چالش‌های همیشگی توسعه دهندگان است. وقتی گوشی را از حالت عمودی به افقی یا بلعکس چرخش می‌دهیم، اکتیویتی از نوع ساخته می‌شود. به همین خاطر داده‌ها و وضعیت فعلی برنامه از بین می‌رود. مثلا اگر کاربر اطلاعات خود را برای ثبت نام در فرم طراحی شده در برنامه ما وارد کرده باشد و قبلا از ثبت اطلاعات، گوشی را بچرخاند، مقدار TextViewها از بین رفته و داخل کادرها خالی می‌ماند.

همچنین چگونگی استفاده از لیوت جایگزین متناسب با حالت صفحه را در زمانی که دستگاه در حالت افقی است، فرا خواهیم گرفت (شکل ۳.۱).

شکل ۳.۱ GeoQuiz در حالت landscape چرخش GeoQuiz



GeoQuiz تا زمانی که دستگاه را نچرخانید عالی کار می‌کند. در حالی که برنامه در حال اجرا است، دکمه NEXT را فشار دهید تا سوال بعدی به نمایش درآید. سپس دستگاه را بچرخانید. اگر روی شبیه‌ساز کار می‌کنید، روی دکمه چرخش به چپ یا راست در نوار ابزار کلیک کنید.

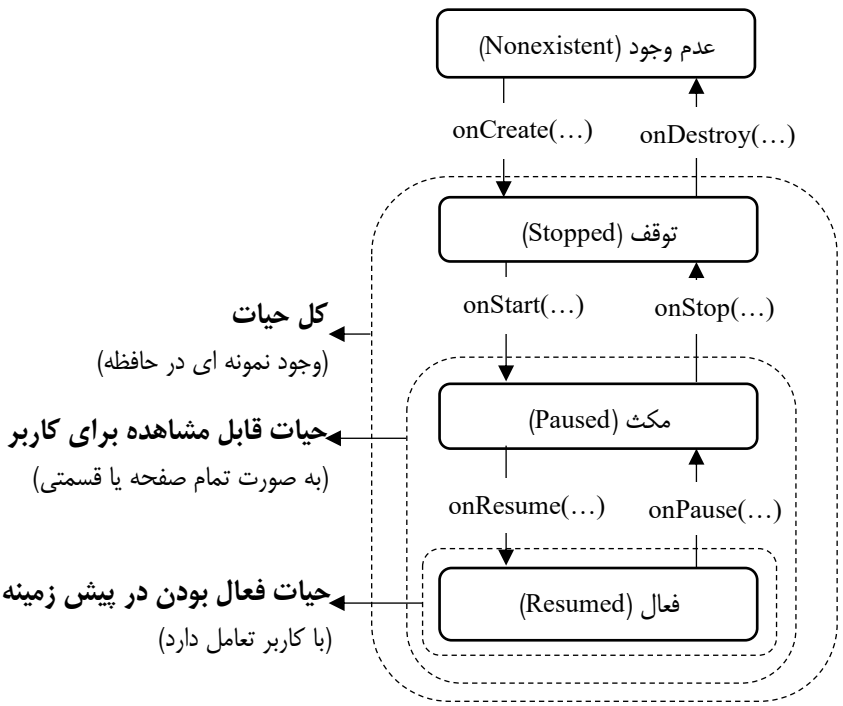
مشاهده می‌کنید که پس از چرخش، دوباره سوال اول بالا می‌آید. چگونه و چرا این اتفاق افتاد؟ پاسخ به این سوالات مربوط به چرخه حیات اکتیویتی است.

در فصل ۴ یاد خواهید گرفت که چگونه این مشکل را برطرف کنید. اما ابتدا، بسیار مهم است که ریشه مشکل را درک کنیم تا بتوانید از باگ‌های مرتبطی که ممکن است بوجود بیایند، جلوگیری نماییم.

حالت‌های اکتیویتی و توابع پاسخ به آن

هر نمونه از اکتیویتی یک چرخه حیات دارد. در طول این چرخه، یک اکتیویتی بین چهار حالت منتقل می‌شود: resumed، paused، stopped و nonexistent. برای هر انتقال، یک تابع وجود دارد که اکتیویتی را از تغییر در حالت خود مطلع می‌کند. شکل ۳.۲ چرخه حیات اکتیویتی، حالت‌ها و توابع را نشان می‌دهد.

شکل ۳.۲ نمودار وضعیت اکتیویتی



شکل ۳.۳ تمام حالات اکتیویتی، وجود نمونه اکتیویتی در حافظه و اینکه آیا برای کاربر قابل مشاهده است یا در پیش زمینه فعال است (با پذیرش ورودی کاربر) را به نمایش می‌گذارد. جدول ۳.۱ این اطلاعات را خلاصه می‌کند.

جدول ۳.۱ حالت‌های اکتیویتی

وضعیت	فعال در پیش زمینه ؟	قابل مشاهده؟	در حافظه؟
nonexistent	نه	نه	نه
stopped	نه	نه	بله
paused	نه	بله/تا حدی*	بله
resumed	بله	بله	بله

(* بسته به شرایط، یک اکتیویتی متوقف شده ممکن است به طور کامل یا جزئی قابل مشاهده باشد.)

- Nonexistent: نشان دهنده اکتیویتی است که هنوز اجرا نشده و یا اکتیویتی که به تازگی از بین رفته است (به عنوان مثال توسط کاربر با فشار دادن دکمه بازگشت). به همین دلیل، گاهی اوقات از این حالت به عنوان حالت "destroyed " نیز یاد می‌شود. هیچ نمونه‌ای در حافظه وجود ندارد و هیچ ویوی مرتبطی قابل مشاهده برای کاربر نیست.
- Stopped: نشان دهنده اکتیویتی است که یک نمونه در حافظه دارد اما روی صفحه قابل مشاهده نیست. این حالت زمانی اتفاق می‌افتد که اکتیویتی به طور کامل مسدود شود (مثلا زمانی که کاربر یک اکتیویتی تمام صفحه دیگر را در پیش زمینه راه اندازی می‌کند، دکمه home را فشار می‌دهد یا از صفحه مدیریت فرآیندها اقدام به سوییچ بین برنامه‌ها می‌کند).
- Paused: نشان دهنده اکتیویتی است که در پیش زمینه فعال نیست اما ویوی آن قابل مشاهده یا تا حدی قابل مشاهده است. مثلا، زمانی که کاربر یک اکتیویتی جدید با مضمون دیالوگ راه اندازی کند. یک اکتیویتی همچنین می‌تواند به طور کامل قابل مشاهده باشد اما در پیش زمینه نباشد. موقعی این اتفاق می افتد که کاربر دو اکتیویتی را در حالت چند پنجره‌ای مشاهده کند (که "split-screen mode " نیز نامیده می‌شود).
- Resumed: نشان دهنده اکتیویتی است که در حافظه وجود دارد، کاملا قابل مشاهده و در پیش زمینه است. این اکتیویتی است که کاربر در حال حاضر با آن در تعامل است. فقط یک اکتیویتی در کل سیستم می‌تواند در هر زمان در حالت resumed باشد. این بدان معناست که اگر یک اکتیویتی به حالت resumed تغییرکند، اکتیویتی دیگری احتمالا از حالت resumed خارج می‌شود.

زیر کلاس‌های Activity می‌توانند از توابع نامبرده شده در شکل ۳.۲ برای مدیریت امور به هنگام انتقال‌ها در چرخه حیات اکتیویتی استفاده کنند. این توابع اغلب توابع پاسخ به چرخه حیات (lifecycle callbacks) نامیده می‌شوند.

شما قبلا با یکی از این توابع چرخه حیات آشنا شده‌اید – onCreate(Bundle?). سیستم عامل این تابع را پس از ایجاد نمونه اکتیویتی و درست قبل از قرار دادن روی صفحه فراخوانی می‌کند.

به طور معمول، اکتیویتی onCreate(Bundle?) را برای انجام امور مربوط به آماده سازی رابط کاربری خود از قبیل موارد زیر پیاده‌سازی می‌کند.

- بارگذاری ویجت‌ها و قرار دادن آنها بر روی صفحه (در فراخوانی setContentView(Int))
- دریافت ارجاع به ویجت‌ها
- تنظیم Listenerها روی ویجت‌ها
- اتصال به مدل داده خارجی فراخوانی می‌کند.

درک این نکته مهم است که هرگز onCreate(Bundle?) یا هر یک از سایر توابع چرخه حیات اکتیویتی را خودمان فراخوانی نمی‌کنیم. بلکه به سادگی این توابع را در زیر کلاس اکتیویتی خود بازنویسی یا override می‌کنیم. سپس اندروید آن‌ها را در زمان مناسب (در رابطه با کاری که کاربر انجام می‌دهد و آنچه در بقیه سیستم اتفاق می‌افتد) فراخوانی می‌کند تا به اکتیویتی اطلاع دهد که وضعیت آن در حال تغییر است.

ثبت وقایع مربوط به چرخه حیات اکتیویتی

در این بخش، قصد داریم توابع چرخه حیات را برای ایجاد گزارش نقل و انتقالات چرخه حیات MainActivity بازنویسی کنیم. هر پیاده‌سازی به سادگی پیامی را که مشخص کننده فراخوانی هر یک از توابع مربوط به چرخه حیات است، ثبت می‌کند. این کار به شما کمک می‌کند تا ببینید چگونه وضعیت MainActivity در زمان اجرا نسبت به کاری که کاربر انجام می‌دهد، تغییر می‌کند.

ساخت پیام‌های گزارش

در اندروید، کلاس android.util.Log پیام‌ها را به عنوان گزارش ارسال می‌کند. Log دارای چندین تابع برای ثبت پیام‌ها است. در اینجا موردی که بیشتر در این کتاب از آن استفاده خواهیم کرد آورده شده است:

```
public static Int d(String tag, String msg)
```

d مخفف "debug" است و به سطح پیام گزارش اشاره دارد. پارامتر اول منبع پیام و پارامتر دوم محتویات پیام است. رشته اول معمولا یک ثابت تگ با نام کلاس به عنوان مقدار آن است. این امر تعیین منبع یک پیام خاص را آسان می‌کند.

MainActivity.kt را باز کنید و یک ثابت TAG به MainActivity اضافه کنید:

لیست ۳.۱ افزودن یک ثابت TAG (MainActivity.kt)

```
import ...
private const val TAG = "MainActivity"
class MainActivity : AppCompatActivity() {
…
}
```

سپس در onCreate(Bundle?) تابع Log.d(…) برای ارسال پیام فرخوانی کنید.

لیست ۳.۲ فراخوانی Log

```
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    enableEdgeToEdge()
    setContentView(R.layout.activity_main)
    ViewCompat.setOnApplyWindowInsetsListener(...)
        insets
    }
    Log.d(TAG, "onCreate(Bundle?) called")
}
```

اکنون پنج تابع چرخه حیات دیگر را در MainActivity با اضافه کردن موارد زیر بازنویسی کنید.

لیست ۳.۳ بازنویسی توابع چرخه حیات (MainActivity.kt)

```
class MainActivity : AppCompatActivity() {
    ...
    override fun onCreate(savedInstanceState: Bundle?) {
        ...
    }
    override fun onStart() {
        super.onStart()
        Log.d(TAG, "onStart() called")
    }
    override fun onResume() {
        super.onResume()
        Log.d(TAG, "onResume() called")
    }
    override fun onPause() {
        super.onPause()
        Log.d(TAG, "onPause() called")
    }
    override fun onStop() {
        super.onStop()
        Log.d(TAG, "onStop() called")
    }
    override fun onDestroy() {
        super.onDestroy()
        Log.d(TAG, "onDestroy() called")
    }
    private fun updateQuestion() {
        ...
    }
    private fun checkAnswer(userAnswer: Boolean) {
        ...
    }
}
```

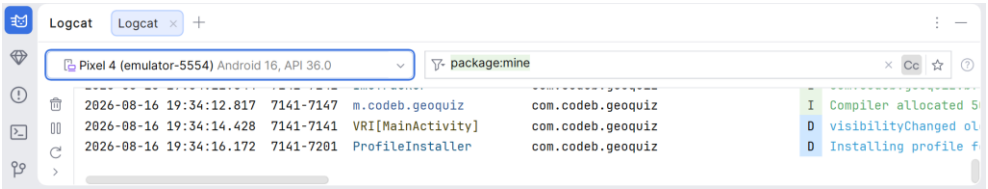
توجه داشته باشید که قبل از ثبت پیام‌ها، سوپرکلاس را فراخوانی می‌کنیم. این کار لازم و ضروری است. فراخوانی سوپرکلاس باید اولین خط هر پیاده‌سازی تابع باشد.

استفاده از Logcat

GeoQuiz را اجرا و پیام‌ها همانطور که در شکل ۳.۵ نشان داده شده است، در پنجره ابزار Logcat در پایین اندروید استودیو شروع به نمایش می‌کنند. اگر Logcat هنگام اجرای GeoQuiz به طور خودکار باز

نشد، می‌توانید با کلیک بر روی دکمه Logcat با آیکون شکل گربه در پایین پنجره اندروید استودیو آن را باز کنید.

تصویر ۳.۳ اندروید استودیو با Logcat



پیام‌های خود را به همراه خروجی سیستم مشاهده خواهید کرد. برای یافتن آسان پیام‌هایتان، می‌توانید خروجی را با استفاده از مقداری که برای ثابت TAG تنظیم کرده‌اید، فیلتر کنید. برای اینکار در قسمت فیلتر مقدار tag:MainActivity را وارد کنید.

تصویر ۳.۴ ایجاد یک فیلتر در Logcat



مشاهده می کنید که سه تابع onCreate(…)، onStart() و onResumed() فراخوانی شده‌اند.

بررسی نحوه پاسخ توابع چرخه حیات به رفتار کاربر

سه تابع چرخه حیات پس از راه اندازی GeoQuiz و ایجاد نمونه اولیه MainActivity فراخوانی شد: onCreate(Bundle?)، onStart() و onResume() (شکل ۳.۴). نمونه MainActivity ما اکنون در حالت onResume (در حافظه، قابل مشاهده و فعال در پیش زمینه) است.

در ادامه مطلب، توابع مختلف چرخه حیات اکتیویتی را برای انجام کارهای دلخواه خود بازنویسی می‌کنیم. بعد از انجام این کار، در مورد کاربردهای هر تابع بیشتر خواهیم آموخت. در حال حاضر، با تعامل با برنامه خود و بررسی گزارش‌ها در Logcat، از آشنایی با نحوه رفتار چرخه حیات در سناریوهای مختلف لذت ببرید.

ترک موقت یک اکتیویتی

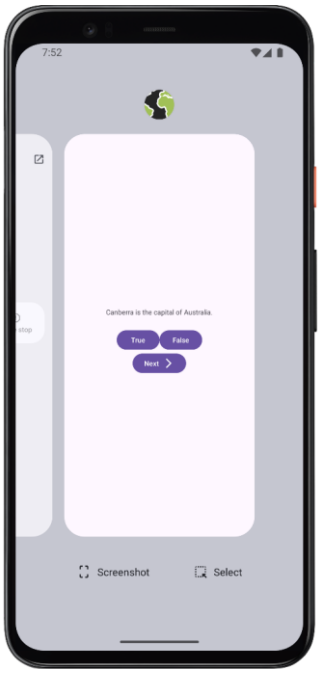
اگر GeoQuiz در حال اجرا نیست، آن را اجرا کنید. حالا دکمه Home را فشار دهید. صفحه اصلی نمایش داده می‌شود و MainActivity کاملا از دید خارج می‌شود. MainActivity در حال حاضر در چه حالتی است؟ Logcat را برای راهنمایی بررسی کنید. اکتیویتی شما onPause() و onStop() را دریافت اما onDestroy() را دریافت نکرده است.

با فشار دادن دکمه Home، کاربر به اندروید می‌گوید: "می‌خواهم به کار دیگری بپردازم، اما ممکن است برگردم. من واقعا هنوز با این صفحه کارم تمام نشده است." اندروید اکتیویتی را pause و در نهایت اکتیویتی را stop می‌کند.

بنابراین پس از فشار دادن دکمه Home، نمونه MainActivity شما در حالت stopped قرار می‌گیرد. اندروید این کار را برای تسریع اجرا برنامه هنگام برگشت به آن، درست از جایی که متوقف شده بود، انجام می‌دهد. (این کل داستان در مورد فشار دادن دکمه Home نیست. برنامه‌های متوقف شده به صلاحدید سیستم عامل از بین می‌روند. برای ادامه داستان به فصل ۴ مراجعه کنید.)

با انتخاب GeoQuiz در لیست صفحه برنامه‌های اخیر به آن برگردید.

تصویر ۳.۵ صفحه برنامه‌های اخیر



هر کارت در صفحه برنامه‌های اخیر نشان دهنده برنامه‌ای است که کاربر در گذشته با آن تعامل داشته است.

روی کارت GeoQuiz در صفحه کلیک کنید. MainActivity صفحه را پر می‌کند. نگاهی گذرا به Logcat نشان می‌دهد که اکتیویتی شما ()onStart و ()onResume را دریافت کرده است. توجه داشته باشید که (…)onCreate فراخوانی نشد. زیرا نمونه اکتیویتی هنوز در حافظه وجود داشت و نیازی به ایجاد دوباره نبود. اکتیویتی فقط باید شروع و از سر گرفته شود.

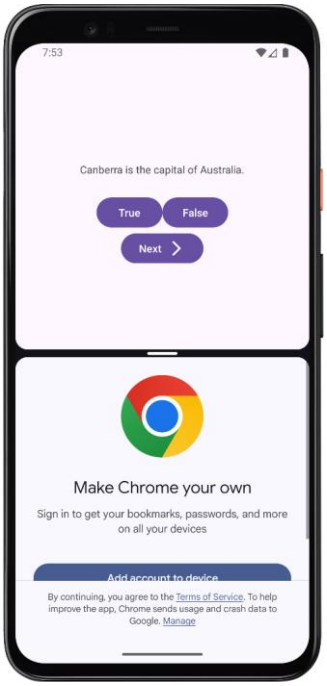
قبلا گفتیم این امکان وجود دارد که یک اکتیویتی در حالت paused قرار گیرد حتی زمانی که تا حدی قابل مشاهده باشد (مانند زمانی که یک اکتیویتی جدید با پس زمینه شفاف یا اندازه کوچکتر از صفحه نمایش در بالای آن راه اندازی شود) و یا زمانی که کاملا قابل مشاهده باشد (در حالت چند پنجره). بیایید حالت چند پنجره‌ای را در عمل ببینیم.

حالت چند پنجره‌ای فقط در Android 7.0 Nougat و بالاتر در دسترس است. دوباره صفحه برنامه‌های اخیر را باز کنید و نماد بالای کارت GeoQuiz را به مدت طولانی فشار دهید سپس Split screen را

انتخاب کنید. یک پنجره جدید در قسمت پایین صفحه ظاهر می‌شود که لیستی از کارت‌های برنامه را نشان می‌دهد. روی یکی از کارت‌ها کلیک کنید تا برنامه مربوطه راه اندازی شود.

این حالت GeoQuiz را در پنجره بالا و برنامه دومی را که انتخاب کرده‌اید در پنجره پایین راه اندازی می‌کند.

تصویر ۳.۶ باز کردن دو برنامه در حالت چند پنجره‌ای



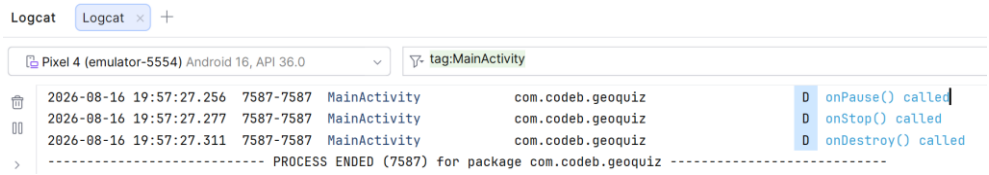
به گزارش‌های Logcat نگاه کنید.

برای خروج از حالت چند پنجره‌ای، جداکننده پنجره را در وسط صفحه به پایین صفحه کشیده و پنجره پایین را رد کنید.

پایان دادن به یک اکتیویتی

دکمه Back را فشار دهید و سپس Logcat را بررسی کنید. در نسخه‌های اندروید قبل ۱۴ اکتیویتی شما توابع onPause()، onStop() و onDestroy() را فراخوانی می‌کند. اما برای پایان دادن به اکتیویتی در نسخه‌های ۱۴ و به بالا کارت آن را از بین برنامه‌های اخیر بیرون بکشید. نمونه MainActivity شما اکنون در حالت nonexistent است (در حافظه وجود ندارد، بنابراین قابل مشاهده نیست و مطمئنا در پیش زمینه فعال نیست).

تصویر ۳.۷ پایان دادن به اکتیویتی



هنگامی که دکمه بازگشت را فشار دادید، شما به عنوان کاربر برنامه اکتیویتی را به پایان رساندید. به عبارت دیگر، شما به اندروید گفتید، "کارم با این اکتیویتی تمام شده است و دیگر به آن نیاز نخواهم داشت." سپس اندروید ویوی اکتیویتی شما را از بین برد و تمام آثار اکتیویتی را از حافظه حذف کرد. این روش اندروید برای صرفه‌جویی در منابع محدود دستگاه شماست.

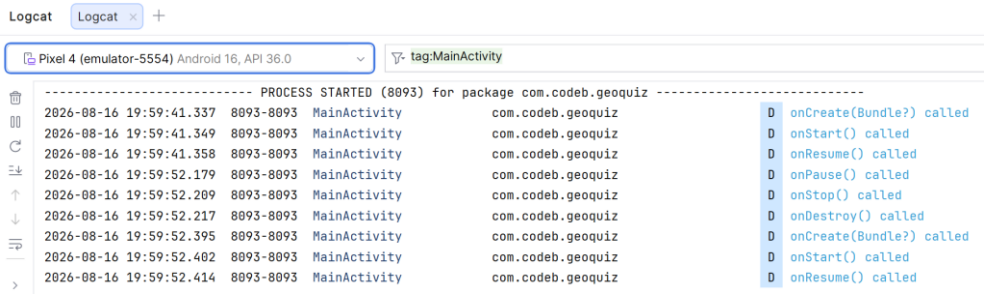
راه دیگری که کاربر می‌تواند یک اکتیویتی را به پایان برساند این است که کارت برنامه را از صفحه برنامه‌های اخیر بیرون بکشد. به عنوان یک توسعه دهنده، می‌توانید یک اکتیویتی را به صورت برنامه‌نویسی با فراخوانی Activity.finish() به پایان برسانید.

چرخاندن یک اکتیویتی

اکنون وقت آن است که به اشکالی که در ابتدای این فصل اشاره کردیم، برگردیم. GeoQuiz را اجرا کنید، دکمه NEXT را فشار دهید تا سوال دوم نمایان شود و سپس دستگاه را بچرخانید.

پس از چرخش، GeoQuiz دوباره سوال اول را نمایش می‌دهد. Logcat را بررسی کنید تا ببینید چه اتفاقی افتاده است. خروجی شما باید مانند شکل ۳.۸ باشد.

شکل ۳.۸ MainActivity از بین رفته و دوباره ساخته شده است



هنگامی که دستگاه را چرخاندید، نمونه MainActivity که جلوی دیدگان شما بود از بین رفت و یک نمونه جدید ایجاد شد. دستگاه را دوباره بچرخانید تا شاهد دور دیگری از تخریب و تولد باشید.

باگ GeoQuiz شما همین مورد است. هر بار که دستگاه را می‌چرخانید، نمونه MainActivity فعلی به طور کامل از بین می‌رود. مقداری که در currentIndex نمونه ذخیره شده است از حافظه پاک می‌شود. این بدان معنی است که وقتی چرخش اتفاق می‌افتد، GeoQuiz فراموش می‌کند که به کدام سوال نگاه می‌کردید. با پایان چرخش، اندروید نمونه جدیدی از MainActivity را از ابتدا ایجاد می‌کند. currentIndex در onCreate(Bundle?) روی صفر مقداردهی اولیه می‌شود و کاربر از اولین سوال شروع می‌کند.

این مشکل را در فصل ۴ برطرف خواهیم کرد. اما قبل از انجام این کار بهتر است نگاهی دقیق‌تر به این بیندازیم که چرا سیستم عامل وقتی کاربر دستگاه را می‌چرخاند، اکتیویتی شما را از بین می‌برد.

تغییرات تنظیمات دستگاه و چرخه حیات اکتیویتی

چرخاندن، تنظیمات دستگاه (device configuration) را تغییر می‌دهد. پیکربندی دستگاه مجموعه‌ای از ویژگی‌ها است که وضعیت فعلی یک دستگاه را توصیف می‌کند. ویژگی‌هایی که پیکربندی را تشکیل می‌دهند شامل جهت صفحه، تراکم صفحه، اندازه صفحه نمایش، نوع صفحه کلید، زبان و موارد دیگر است.

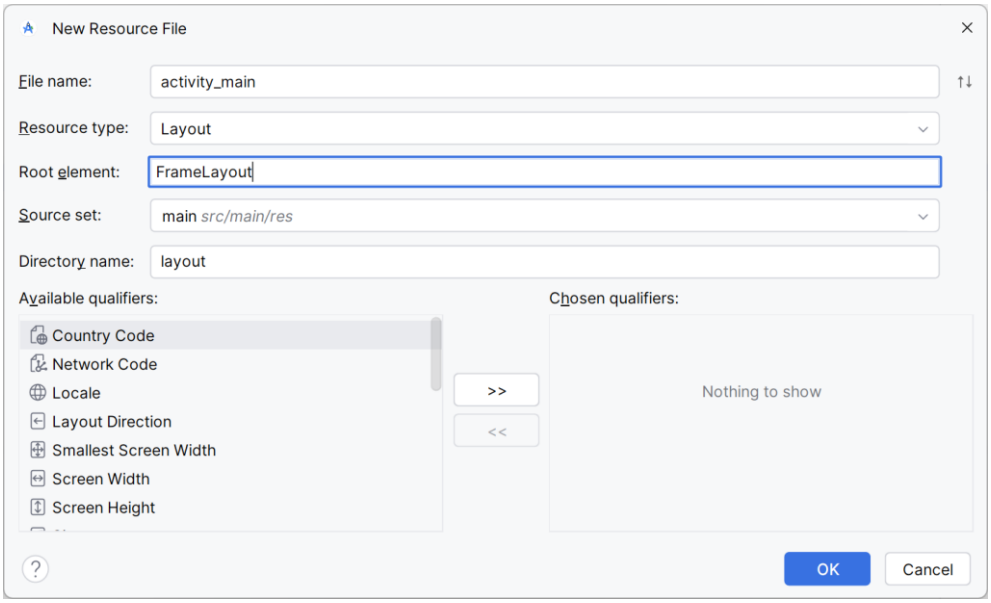
به طور معمول، برنامه‌ها منابع جایگزینی را برای مطابقت با تنظیمات دستگاه ارائه می‌دهند. نمونه‌ای از این کار را زمانی که چندین آیکون برای تراکم‌های مختلف صفحه نمایش به پروژه خود اضافه کردیم، دیدید.

هنگامی که یک تغییر پیکربندی زمان اجرا (runtime configuration change) رخ می‌دهد، ممکن است منابعی وجود داشته باشد که با پیکربندی جدید مطابقت بهتری داشته باشند. بنابراین اندروید اکتیویتی را از بین می‌برد، به دنبال منابعی می‌گردد که برای پیکربندی جدید مناسب‌تر هستند و سپس نمونه جدیدی از اکتیویتی را با آن منابع بازسازی می‌کند. برای دیدن این موضوع در عمل، یک منبع جایگزین برای اندروید ایجاد می‌کنیم تا زمانی که جهت صفحه نمایش دستگاه به افقی تغییر می‌کند، از آن استفاده کند.

ایجاد یک لیوت افقی

در پنجره ابزار پروژه، روی دایرکتوری res کلیک راست کرده و New → Android Resource File را انتخاب کنید. باید پنجره‌ای شبیه به شکل ۳.۱۵ را ببینید که انواع منابع و توصیف‌گرها آن نوع را فهرست می‌کند. activity_main را در قسمت نام فایل تایپ کنید. Layout را در منوی کشویی Resource type انتخاب کنید. FrameLayout را در قسمت عنصر Root تایپ کنید. گزینه Source set را روی main تنظیم کنید.

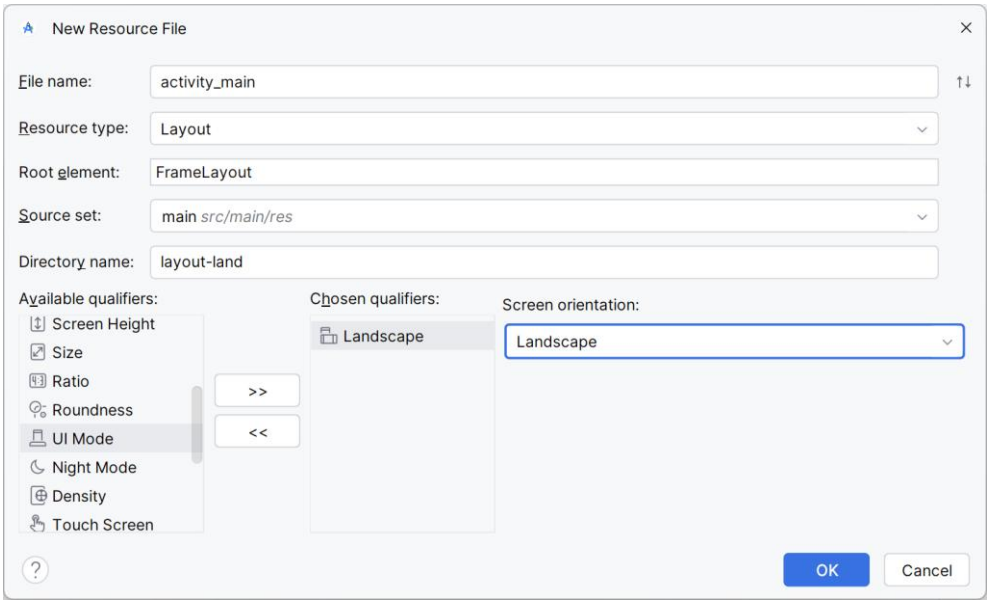
تصویر ۳.۹ ایجاد یک فایل منبع جدید



در مرحله بعد، توصیفگر را انتخاب خواهیم کرد. توصیفگر، عنصر را برای شرایط خاص اجرا تعریف می‌کند. Orientation را در لیست اول انتخاب و روی دکمه >> کلیک کنید تا Orientation به بخش Chosen qulifiers منتقل شود.

در نهایت، مطمئن شوید که Landscape را در منوی کشویی Screen orientation همانطور که در شکل ۳.۹ نشان داده شده است، انتخاب کرده‌اید. بررسی کنید که نام دایرکتوری اکنون layout-land را نشان می‌دهد. در حالی که این پنجره پر زرق و برق به نظر می‌رسد، هدف آن فقط تعیین نام دایرکتوری است که فایل منبع جدید در آن قرار می‌گیرد. روی OK کلیک کنید.

تصویر ۳.۱۰ ایجاد res/layout-land/activity_main.xml



اندروید استودیو پوشه res/layout-land/ را در پروژه شما ایجاد می‌کند و یک فایل لیوت جدید به نام activity_main.xml را در پوشه جدید قرار می‌دهد. پنجره ابزار پروژه را به ویوی Project تغییر دهید تا فایل‌ها و پوشه‌های پروژه خود را همانطور که هستند، ببینید. برای مشاهده خلاصه فایل‌ها به ویوی Android برگردید.

همانطور که دیدید، توصیفگرهای پیکربندی در زیرشاخه‌های res مشخص می‌کنند که چطور اندروید بهترین منبع را با توجه به تنظیمات دستگاه فعلی انتخاب کند. پسوند land- نمونه دیگری از یک توصیفگر پیکربندی است. می‌توانید لیستی از توصیفگرهای پیکربندی را در آدرس زیر بیابید.

developer.android.com/guide/topics/resources/
providing-resources.html

اکنون یک لیوت افقی و یک لیوت پیش‌فرض دارید. هنگامی که دستگاه در جهت افقی قرار دارد، اندروید لیوت را در دایرکتوری /res/layout-land جستجو کرده و از آن استفاده می‌کند. در غیر این صورت لیوت پیش‌فرض را از /res/layout بر می‌دارد. توجه داشته باشید که دو فایل لیوت باید نام فایل یکسانی داشته باشند تا بتوان با شناسه منبع یکسان به آنها ارجاع داد.

در حال حاضر لیوت افقی یک صفحه خالی را نمایش می‌دهد. برای رفع این مشکل، تمام محتویات را به جز تگ‌های باز و بسته LinearLayout ریشه از res/layout/activity_main.xml کپی کنید. کد را در res/layout-land/activity_main.xml بین تگ‌های باز و بسته FrameLayout قرار دهید.

در مرحله بعد، تغییرات نشان داده شده در فهرست ۳.۴ را در لیوت افقی ایجاد کنید تا با لیوت پیش‌فرض متفاوت باشد.

لیست ۳.۴ بهینه سازی لیوت افقی (res/layout-land/activity_main.xml)

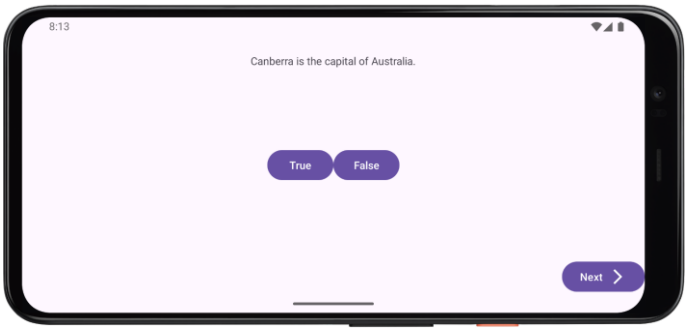
```
<FrameLayout xmlns:android="http://schemas.an-
droid.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    xmlns:tools="http://schemas.android.com/tools"
    android:gravity="center"
    android:id="@+id/main">
    <TextView
        android:id="@+id/question_text_view"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:padding="24dp"
        tools:text="@string/question_australia"
        android:layout_gravity="center_horizontal"/>
    <LinearLayout
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:orientation="horizontal"
        android:layout_gravity=
            "center_vertical|center_horizontal">
        <Button
            android:id="@+id/true_button"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="@string/true_button"
            />
        <Button
            android:id="@+id/false_button"
```

```
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="@string/false_button" />
    </LinearLayout>
    <Button
        android:id="@+id/next_button"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_gravity="bottom|right"
        android:text="@string/next_button"
        android:drawableEnd="@drawable/ic_arrow_right"
        android:drawablePadding="4dp"/>
</FrameLayout>
```

FrameLayout ساده‌ترین ViewGroup است که فرزندان خود را به روش خاصی مرتب نمی‌کند. در این لیوت، ویوهای فرزند بر اساس ویژگی‌های android:layout_gravity خودشان مرتب می‌شوند. به همین دلیل است که android:layout_gravity را به TextView، LinearLayout و Button اضافه کردیم – همه آنها فرزندان FrameLayout هستند. Buttonها، دقیقا در همان جای قبلیشان باقی می‌مانند زیرا آنها فرزندان مستقیم FrameLayout نیستند.

GeoQuiz را دوباره اجرا کنید. دستگاه را به صورت افقی بچرخانید تا لیوت جدید را ببینید (شکل ۳.۱۱). البته، این فقط یک لیوت جدید نیست بلکه یک نمونه جدید MainActivity نیز هست.

شکل ۳.۱۱ اکتیویتی اصلی در جهت گیری افقی



به حالت عمودی برگردید تا لیوت پیش‌فرض و یک MainActivity جدید دیگر را ببینید.

برنامه‌ای که وقتی کاربر دستگاه را می‌چرخاند، خود را بازنشانی می‌کند، چه فایده‌ای دارد، حتی اگر یک لیوت افقی خاص دوست داشتنی داشته باشد؟ در فصل ۴ یاد خواهید گرفت که چگونه باگ GeoQuiz ناشی از مشکل چرخش را برطرف کنید.

بیشتر بدانیم: سطوح Log

هنگامی که از کلاس android.util.log برای ارسال پیام‌های گزارش استفاده می‌کنیم، نه تنها محتوای یک پیام را کنترل می‌کنیم، بلکه سطحی که میزان اهمیت پیام را مشخص می‌کند را نیز کنترل می‌کنیم. اندروید از پنج سطح گزارش پشتیبانی می‌کند که در جدول ۳.۲ نشان داده شده است. ارسال گزارش به خروجی به سادگی فراخوانی تابع Log مربوطه است.

جدول ۳.۲ سطوح و توابع لاگ

سطوح	تابع	استفاده شده برای
ERROR	Log.e(…)	خطاهای
WARNING	Log.w(…)	هشدارها
INFO	Log.i(…)	پیام‌های اطلاعاتی
DEBUG	Log.d(…)	خروجی دیباگ
VERBOSE	Log.v(…)	فقط توسعه

چالش: جلوگیری از پاسخ‌های تکراری

هنگامی که کاربر پاسخی برای یک سوال خاص ارائه داد، دکمه‌های را غیرفعال کنید تا از لمس شدن چندین باره آنها جلوگیری کنید.

چالش: آزمون درجه بندی شده

پس از اینکه کاربر پاسخ تمام سؤالات مسابقه را ارائه داد، یک Toast با درصد امتیاز برای مسابقه نمایش دهید.

موفق باشید!

۴

حفظ وضعیت رابط کاربری

اندروید در ارائه منابع جایگزین در زمان مناسب عملکرد فوق العاده‌ای دارد. با این حال، از بین بردن و ایجاد مجدد اکتیویتی‌ها در چرخش دردسرهای عظیمی مانند آنچه در GeoQuiz دیدید در پی دارد.

برای رفع این اشکال، نمونه MainActivity پس از چرخش باید مقدار قدیمیcurrentIndex را به خاطر بیاورد. تحقق این امر به راهی برای ذخیره این داده در زمان تغییر پیکربندی، مانند چرخش، بستگی دارد.

در این فصل، با ذخیره داده‌های رابط کاربری در یک ViewModel، باگ GeoQuiz را برطرف خواهیم کرد. همچنین با استفاده از مکانیزم saved instance state اندروید یک باگ کمتر قابل کشف اما به همان اندازه مشکل ساز – از دست دادن وضعیت رابط کاربری در هنگام مرگ فرآیند – را نیز حل می‌کنیم.

اضافه کردن ViewModel

کلاس ViewModel از یک کتابخانه Android Jetpack به نام lifecycle-extensions می‌آید، یکی از کتابخانه‌های متعددی که در طول این کتاب از آن استفاده خواهید کرد. برای استفاده از این کلاس، ابتدا باید کتابخانه را در لیست وابستگی‌های (dependencies) پروژه خود وارد کنید.

لیست وابستگی‌های پروژه فهرستی از کتابخانه‌ها، ماژول‌ها و یا ابزارهایی است که پروژه برای اجرا به آنها نیاز دارد. وابستگی‌های پروژه در فایلی به نام build.gradle اعلام می‌شوند (Gradle ابزار ساخت اندروید است). در پنجره ابزار پروژه، با فعال بودن ویوی اندروید، بخش Gradle scripts را گسترش دهید و نگاهی به محتویات بیندازید. پروژه شما در واقع دارای دو فایل build.gradle است: یکی برای پروژه به عنوان یک کل و دیگری برای ماژول برنامه شما. فایل build.gradle واقع در ماژول برنامه خود را باز کنید. باید چیزی شبیه به لیست ۴.۱ ببینید.

فهرست ۴.۱ وابستگی‌ها (app/build.gradle)

```
plugins {
    alias(libs.plugins.android.application)
}
android {
    namespace = "com.codeb.geoquiz"
    compileSdk = 36
    defaultConfig {
        applicationId = "com.codeb.geoquiz"
        minSdk = 24
        targetSdk = 36
        versionCode = 1
        versionName = "1.0"
        testInstrumentationRunner =
            "androidx.test.runner.AndroidJUnitRunner"
    }
    buildTypes {
        release {
            isMinifyEnabled = false
            proguardFiles(
                getDefaultProguardFile
                ("proguard-android-optimize.txt"),
                "proguard-rules.pro"
            )
        }
    }
    compileOptions {
        sourceCompatibility = JavaVersion.VERSION_11
        targetCompatibility = JavaVersion.VERSION_11
```

```
    }
}
dependencies {
    implementation(libs.androidx.core.ktx)
    implementation(libs.androidx.appcompat)
    implementation(libs.material)
    implementation(libs.androidx.activity)
    implementation(libs.androidx.constraintlayout)
    testImplementation(libs.junit)
    androidTestImplementation(libs.androidx.junit)
    androidTestImplementation(libs.androidx.espresso.core)
}
```

Gradle امکان مشخص کردن وابستگی‌های جدید را فراهم می‌کند. هنگامی که برنامه شما کامپایل شد، Gradle وابستگی‌های مربوط به برنامه شما را پیدا و دانلود می‌کند. تنها کاری که باید انجام دهید این است که یک مقدار رشته‌ای دقیق و جادویی مشخص کنید، سپس Gradle بقیه کارها را انجام خواهد داد.

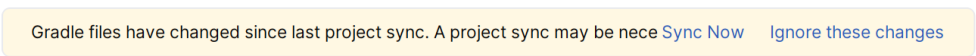
همانطور که در فهرست ۴.۲ نشان داده شده است، lifecycle را به فایل app/build.gradle خود اضافه کنید. برای مرتب نگه داشتن کد خوب است که وابستگی‌های جدید را در آخر لیست قرار دهید.

لیست ۴.۲ افزودن وابستگی lifecycle-extensions (app/build.gradle)

```
dependencies {
    …
    implementation(libs.androidx.constraintlayout)
    implementation
        ("androidx.lifecycle:lifecycle-viewmodel-ktx:2.10.0")
    implementation("androidx.activity:activity-ktx:1.9.0")
    …
}
```

هنگامی که تغییری در build.gradle به وجود آوریم، اندروید استودیو از ما می‌خواهد که فایل را همگام سازی کنیم (شکل ۴.۱).

شکل ۴.۱ اعلان همگام سازی Gradle



این همگام سازی از Gradle می‌خواهد که بیلد را بر اساس تغییرات شما با دانلود یا حذف وابستگی‌ها به روز کند. برای همگام سازی، یا روی Sync Now در اعلان کلیک کنید یا File → Sync Project with Gradle Files را انتخاب کنید.

افزودن یک ViewModel

حالا به ViewModel برگردیم. ViewModel کلاسی است که داده‌ها و منطق مرتبط با رابط کاربری را بدون آن که خودش به آن وابسته باشد، نگهداری می‌کند. هدف اصلی ViewModel مدیریت داده‌ها بین مدل و ویو و در عین حال در امان ماندن از چرخه حیات اکتیویتی یا فرگمنت است.

ViewModel بخشی از پکیج androidx.lifecycle است که شامل APIهای مرتبط با چرخه حیات از جمله کامپوننت‌های lifecycle-aware است. گوگل پکیج androidx.lifecycle را برای کنار آمدن با چرخه حیات اکتیویتی (و سایر چرخه‌های حیات، که بعدا در این کتاب با آنها آشنا خواهید شد) ایجاد کرده است تا این برخورد کمترین دردسر را داشته باشد.

اکنون آماده ایجاد زیر کلاس ViewModel خود یعنی QuizViewModel هستیم. در پنجره ابزار پروژه، روی پکیج com.codeb.geoquiz کلیک راست کرده و New → Kotlin File/Class را انتخاب کنید. QuizViewModel را برای نام وارد و Class را از منوی زیری انتخاب نمایید.

در QuizViewModel.kt، یک بلوک init اضافه کرده و ()onCleared را بازنویسی کنید. همانطور که در لیست ۴.۳ نشان داده شده است، ایجاد و تخریب نمونه QuizViewModel را Log کنید.

لیست ۴.۳ ایجاد یک کلاس ViewModel (QuizViewModel.kt)

```
package com.codeb.geoquiz
import android.util.Log
import androidx.lifecycle.ViewModel
private const val TAG = "QuizViewModel"
class QuizViewModel: ViewModel() {
    init {
    Log.d(TAG, "ViewModel instance created")
    }
    override fun onCleared() {
        super.onCleared()
        Log.d(TAG,
            "ViewModel instance about to be destroyed")
    }
}
```

تابع ()onCleared درست قبل از از بین رفتن ViewModel فراخوانی می‌شود. به همین دلیل بهترین مکان برای انجام هر گونه پاکسازی است. در حال حاضر، این واقعیت را ثبت می‌کنیم که ViewModel در شرف نابودی است تا بتوانیم چرخه حیات آن را بررسی کنیم (به همان روشی که چرخه حیات MainActivity را در فصل ۳ بررسی کردیم).

اکنون، MainActivity.kt را باز کرده و در (…)onCreate، اکتیویتی را با نمونه‌ای از ViewModel مرتبط کنید. حتما دستور ایمپورت مربوطه را نیز وارد کنید.

لیست ۴.۴ دسترسی به ViewModel (MainActivity.kt)

```
import androidx.activity.viewModels
...
override fun onCreate(savedInstanceState: Bundle?) {
    …
    ViewCompat.setOnApplyWindowInsetsListener(…)
    val quizViewModel: QuizViewModel by viewModels()
    Log.d(TAG, "Got a QuizViewModel: $quizViewModel")
    Log.d(TAG, "onCreate(Bundle?) called")
    …
}
```

با استفاده از این کد نمونه ای از ViewModel را بدست می‌آوریم. دقت نمایید که در هر بار ایجاد MainActivity ویومدل جدید فقط در صورتی ایجاد می‌شود که قبلا وجود نداشته باشد. در غیر این صورت همان ویومدل موجود برگردانده می‌شود.

چرخه حیات ViewModel

در فصل ۳ یاد گرفتید که اکتیویتی‌ها بین چهار حالت تغییر می‌کنند: resumed، paused، stopped و nonexistent. همچنین در مورد روش‌های مختلفی که می‌شد با آن یک اکتیویتی را از بین برد، آشنا شدید: یا توسط کاربر اکتیویتی به پایان می‌رسد یا توسط سیستم و در زمان تغییر پیکربندی.

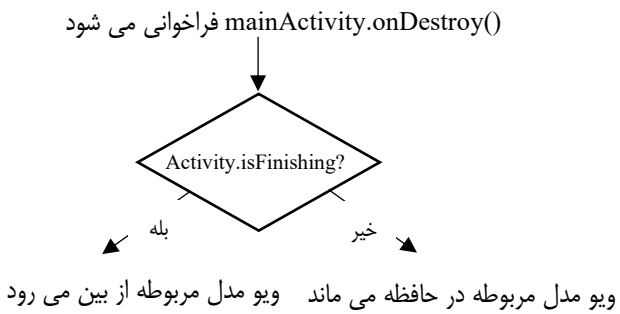
هنگامی که کاربر اکتیویتی را به پایان می‌رساند، انتظار دارد که وضعیت رابط کاربری به حالت اولیه برگردد. همچنین هنگامی که کاربر اکتیویتی را می‌چرخاند، انتظار دارد که وضعیت رابط کاربری او قبل و بعد از چرخش یکسان باشد.

می‌توان با بررسی ویژگی isFinishing اکتیویتی مشخص کرد که کدام یک از این دو سناریو در حال وقوع است. اگر isFinishing، True باشد، اکتیویتی از بین می‌رود زیرا کاربر اکتیویتی را به پایان رسانده است (مثلا با فشار دادن دکمه بازگشت یا با پاک کردن کارت برنامه از صفحه برنامه‌های اخیر). اگر isFinishing، False باشد اکتیویتی به دلیل تغییر پیکربندی توسط سیستم از بین می‌رود.

خوشبختانه نیازی به دنبال کردن isFinishing برای حفظ وضعیت رابط کاربری نداریم. در عوض، میتوانیم از یک ViewModel برای نگه داشتن داده‌های وضعیت رابط کاربری یک اکتیویتی در حافظه به هنگام تغییرات پیکربندی استفاده کنیم. چرخه حیات ViewModel انتظارات کاربر را منعکس می‌کند: از تغییرات پیکربندی جان سالم به در می‌برد و تنها زمانی از بین می‌رود که اکتیویتی مرتبط با آن به پایان برسد.

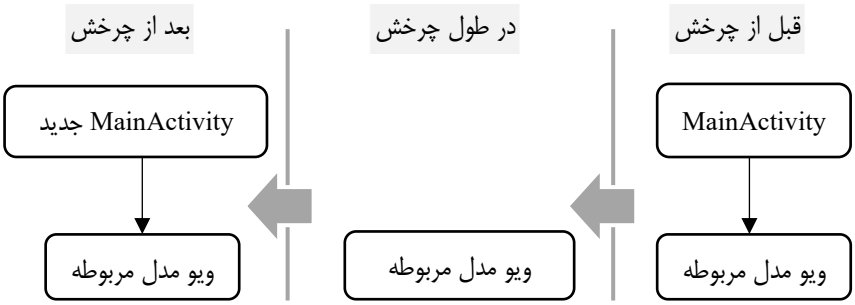
وقتی یک نمونه ViewModel را با چرخه حیات یک اکتیویتی مرتبط می‌کنیم، همانطور که در لیست ۴.۴ انجام دادیم، گفته می‌شود که ViewModel به چرخه حیات آن اکتیویتی محدود شده است. این بدان معناست که ViewModel بدون توجه به وضعیت اکتیویتی، تا زمانی که اکتیویتی به پایان برسد، در حافظه باقی می‌ماند. پس از اتمام اکتیویتی نمونه ViewModel نیز از بین می‌رود (تصویر ۴.۲).

تصویر ۴.۲ QuizViewModel محدود به MainActivity



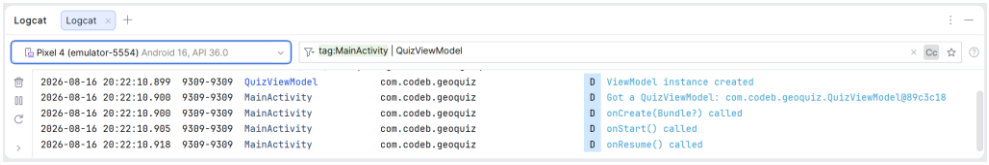
این بدان معنی است که ViewModel در طول تغییر پیکربندی، مانند چرخش، در حافظه باقی می‌ماند. در طول تغییر پیکربندی، نمونه اکتیویتی از بین می‌رود و دوباره ایجاد می‌شود، اما ViewModel محدود به اکتیویتی در حافظه باقی می‌مانند. در شکل ۴.۳ این مورد با استفاده از MainActivity و QuizViewModel نشان داده شده است.

شکل ۴.۳ – MainActivity و QuizViewModel در سراسر چرخش



برای مشاهده در عمل، GeoQuiz را اجرا کنید. پیام‌های Logcat را با استفاده از متن tag:MainActivity | QuizViewModel فیلتر کنید و به بررسی پیام‌ها بپردازید .

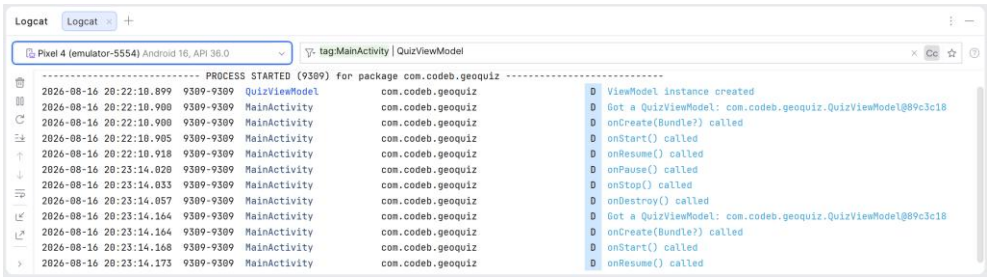
شکل ۴.۴ فیلتر پیام‌ها



حالا به گزارش مربوط نگاه کنید. هنگامی که MainActivity برای اولین بار راه اندازی می‌شود، برای ViewModel در onCreate(...)، یک نمونه جدید با نام QuizViewModel ایجاد می‌کند. با ایجاد نمونه پیامی در گزارش به ثبت می‌رسد.

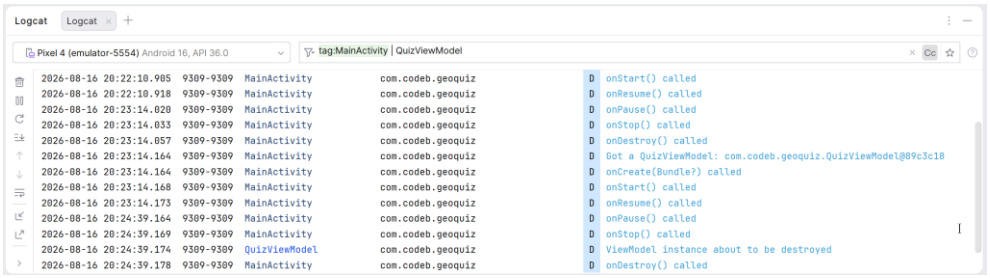
دستگاه را بچرخانید. گزارش‌ها مرگ اکتیویتی را تایید می‌کنند (شکل ۴.۵) اما QuizViewModel را نه. هنگامی که نمونه جدید MainActivity پس از چرخش ایجاد می‌شود، یک QuizViewModel درخواست می‌کند. از آنجایی که QuizViewModel اصلی هنوز در حافظه است به جای ایجاد یک نمونه جدید برگردانده می شود.

شکل ۴.۵ MainActivity از بین رفته و دوباره ایجاد می‌شود اما QuizViewModel خیر!



در نهایت به اکتیوتی پایان دهید. QuizViewModel.onCleared() فراخوانی می‌شود که نشان می‌دهد نمونه QuizViewModel همانطور که گزارش‌ها نشان می‌دهد، در شرف نابودی است (شکل ۴.۶). QuizViewModel به همراه نمونه MainActivity از بین می‌رود.

شکل ۴.۶ MainActivity و QuizViewModel نابود شدند



رابطه بین MainActivity و QuizViewModel یک طرفه است. اکتیویتی به ViewModel ارجاع می‌دهد، اما ViewModel به اکتیویتی دسترسی ندارد. ViewModel شما هرگز نباید ارجاعی به یک اکتیویتی یا ویوی داشته باشد، در غیر این صورت نشت حافظه (memory leak) رخ خواهد داد.

نشت حافظه زمانی اتفاق می‌افتد که یک شی دارای ارجاع قوی به شی دیگری داشته باشد که باید از بین برود. نگه داشتن مرجع قوی مانع از پاک کردن شی از حافظه توسط مدیر پاکسازی حافظه (garbage collector) می‌شود. نشت حافظه به دلیل تغییر پیکربندی از اشکالات رایج است. جزئیات مرجع قوی و garbage collector خارج از محدوده این کتاب است. اگر علاقه‌مند به این موضوع هستید توصیه می‌کنیم به سراغ کتاب‌های مرجع کاتلین یا جاوا بروید.

نمونه ViewModel در طول چرخش در حافظه باقی می‌ماند، در حالی که نمونه اکتیویتی اصلی شما از بین می‌رود. اگر ViewModel ارجاع قوی به نمونه اکتیویتی اصلی داشته باشد، دو مشکل رخ می‌دهد: اول، نمونه اکتیویتی اصلی از حافظه حذف نمی‌شود و بنابراین نشت حافظه اتفاق می‌افتد. دوم، ViewModel ارجاعی به یک اکتیویتی کهنه دارد. اگر ViewModel سعی کند ویوی اکتیویتی کهنه را به روز کند، یک خطای IllegalStateException را در پی دارد.

افزودن داده به ViewModel

بالاخره زمان رفع اشکال چرخش فرا رسیده است. QuizViewModel در چرخش مانند MainActivity از بین نمی‌رود، بنابراین می‌توانیم داده‌های وضعیت رابط کاربری اکتیویتی را در نمونه QuizViewModel که از چرخش جان سالم به در می‌برد، ذخیره کنیم.

می‌خواهیم سوال و داده‌های currentIndex را از اکتیویتی خود به همراه تمام منطق مربوط به آنها را در ViewModel خود کپی کنیم. با حذف currentIndex و questionBank از MainActivity (فهرست ۴.۵) شروع کنید.

لیست کردن ۴.۵ انتقال داده‌های مدل از اکتیویتی (MainActivity.kt)

```
class MainActivity : AppCompatActivity() {
    …
    private val questionBank = listOf(
        Question(R.string.question_australia, true),
        Question(R.string.question_oceans, true),
        Question(R.string.question_mideast, false),
        Question(R.string.question_africa, false),
        Question(R.string.question_americas, true),
        Question(R.string.question_asia, true))
    private var currentIndex = 0
...
}
```

اکنون، ویژگی‌های currentIndex و questionBank را همانطور که در لیست ۴.۶ نشان داده شده است، در QuizViewModel قرار دهید.

دسترسی private را از currentIndex حذف کنید تا توسط کلاس‌های خارجی مانند MainActivity قابل دسترسی باشد. دسترسی را در questionBank بگذارید – MainActivity مستقیما با questionBank تعامل نخواهد داشت. در حالی که در حال ویرایش QuizViewModel هستید، init و ثبت پیام ()onCleared را حذف کنید، زیرا دیگر از آنها استفاده نخواهیم کرد.

لیست کردن ۴.۶ – جایگذاری در QuizViewModel (QuizViewModel.kt)

```
class QuizViewModel: ViewModel() {
    init {
    Log.d(TAG, "ViewModel instance created")
    }
    override fun onCleared() {
        super.onCleared()
        Log.d(TAG,
            "ViewModel instance about to be destroyed")
    }
    private val questionBank = listOf(
        Question(R.string.question_australia, true),
        Question(R.string.question_oceans, true),
        Question(R.string.question_mideast, false),
        Question(R.string.question_africa, false),
        Question(R.string.question_americas, true),
        Question(R.string.question_asia, true))
    private var currentIndex = 0
}
```

در مرحله بعد، یک تابع برای انتقال به سوال بعدی به QuizViewModel اضافه کنید.

لیست ۴.۷ افزودن منطق برنامه به QuizViewModel (QuizViewModel.kt)

```
class QuizViewModel: ViewModel() {
    …
    var currentIndex = 0
    val currentQuestionAnswer: Boolean
        get() = questionBank[currentIndex].answer
    val currentQuestionText: Int
        get() = questionBank[currentIndex].textResId
    fun moveToNext() {
        currentIndex = (currentIndex + 1) % questionBank.size
    }
}
```

قبلا گفتیم که یک ViewModel تمام داده‌هایی را که صفحه نمایش مرتبط با آن نیاز دارد ذخیره، قالب بندی و دسترسی به آن را آسان می‌کند. این کار به شما امکان می‌دهد منطق ارائه را از اکتیویتی حذف کنید، که به نوبه خود اکتیویتی شما را ساده نگه می‌دارد. ساده نگه داشتن اکتیویتی‌ها تا حد امکان کار بسیار خوبی است.

با این حال، شما توابع updateQuestion() و checkAnswer(Boolean) را در MainActivity به جا خواهید گذاشت تا بعدا این توابع را به روز کنید. در ضمن باید تعریف ویومدل را نیز به خارج تابع onCreate(…) انتقال دهیم.

در نهایت، MainActivity را برای نمایش محتوا و تعامل با QuizViewModel به روز کنید

لیست ۴.۸ بروزرسانی سوال از طریق QuizViewModel (MainActivity.kt)

```
class MainActivity : AppCompatActivity() {
    ...
    private lateinit var questionTextView: TextView
    private val quizViewModel: QuizViewModel by viewModels()
    override fun onCreate(savedInstanceState: Bundle?) {
        ...
        val quizViewModel: QuizViewModel by viewModels()
        ...
        falseButton.setOnClickListener { view: View ->
            checkAnswer(false)
        }
        nextButton.setOnClickListener {
            currentIndex =
                (currentIndex + 1) % questionBank.size
            quizViewModel.moveToNext()
            updateQuestion()
        }
        updateQuestion()
    }
    ...
    private fun updateQuestion() {
        val questionTextResId =
            questionBank[currentIndex].textResId
        val questionTextResId =
            quizViewModel.currentQuestionText
        questionTextView.setText(questionTextResId)
    }
    private fun checkAnswer(userAnswer: Boolean) {
        val correctAnswer = questionBank[currentIndex].answer
        val correctAnswer =
            quizViewModel.currentQuestionAnswer
        val messageResId = if (userAnswer == correctAnswer) {
```

```
            R.string.correct_toast
        } else {
            R.string.incorrect_toast
        }
        Toast.makeText(this,
            messageResId, Toast.LENGTH_SHORT)
            .show()
    }
}
```

GeoQuiz را اجرا و NEXT را فشار دهید. سپس دستگاه یا شبیه‌ساز را بچرخانید. مهم نیست که چند بار بچرخانید، MainActivity به یاد می‌آورد که در چه سوالی بودید. جشن مفصلی به مناسبت حل باگ برنامه به راه بیاندازید. اما زیاد خوشحال نباشید یک اشکال دیگر وجود دارد که به راحتی قابل کشف نیست.

ذخیره داده در فرآیند مرگ برنامه

تغییرات پیکربندی تنها زمانی رخ نمی‌دهد که سیستم عامل یک اکتیویتی را از بین می‌برد. هر برنامه، فرآیند (process) خاص خود را دارد که شامل یک نخ (thread) برای اجرای کار مرتبط با رابط کاربری و یک تکه حافظه برای ذخیره آبجکت‌ها در آن است.

اگر کاربر برای مدتی از برنامه استفاده نکند و اندروید نیاز به بازیابی حافظه داشته باشد، فرآیند یک برنامه می‌تواند توسط سیستم عامل از بین برود. هنگامی که فرآیند یک برنامه از بین می‌رود، تمام آبجکت‌ها ذخیره شده در حافظه آن فرآیند نیز از بین می‌رود. (در مورد فرآیندهای اپلیکیشن اندروید در فصل‌های آتی بیشتر خواهید آموخت.)

فرآیندهای حاوی اکتیویتی‌های resumed یا paused اولویت بالاتری نسبت به سایر فرآیندها دارند. هنگامی که سیستم عامل نیاز به آزاد کردن منابع دارد، ابتدا فرآیندهای با اولویت پایین را انتخاب می‌کند. از نظر عملی، فرآیندی که حاوی یک اکتیویتی قابل مشاهده باشد، توسط سیستم عامل از بین نخواهد رفت. اگر یک فرآیند پیش زمینه توسط سیستم از بین برود، به این معنی است که مشکلی در دستگاه وجود دارد (و از بین رفتن برنامه احتمالا می‌تواند کمترین نگرانی کاربر باشد).

اما اکتیویتی‌های stopped گزینه منصفانه برای از بین رفتن هستند. بنابراین، به عنوان مثال، اگر کاربر دکمه Home را فشار داده و سراغ تماشای ویدیویی برود و یا بازی اجرا کند، ممکن است فرآیند برنامه شما از بین برود.

هنگامی که سیستم عامل فرآیند برنامه را از بین می‌برد، هر یک از اکتیویتی‌های برنامه و ViewModel ذخیره شده در حافظه پاک می‌شود. سیستم عامل در هنگام تخریب در مورد نگرانی‌های شما بی‌تفاوت خواهد بود و هیچ یک از توابع پاسخ به چرخه حیات اکتیویتی یا ViewModel را فراخوانی نمی‌کند.

بنابراین چگونه می‌توان داده‌های وضعیت رابط کاربری را ذخیره کرد و از آن برای بازسازی اکتیویتی استفاده کرد تا کاربر هرگز متوجه نشود که اکتیویتی از بین رفته است؟

یکی از راه‌های انجام این کار، ذخیره داده‌ها در saved instance state است. saved instance state داده‌هایی است که سیستم عامل به طور موقت خارج از اکتیویتی ذخیره می‌کند. می‌توانیم با بازنویسی Activity.onSaveInstanceState(Bundle) مقادیری را به saved instance state اضافه کنید.

هر زمان اکتیویتی که تمام نشده است به حالت stopped منتقل می‌شود (مانند زمانی که کاربر دکمه Home را فشار می‌دهد) سیستم عامل Activity.onSaveInstanceState(Bundle) را فراخوانی می‌کند. این زمان بندی مهم است، زیرا اکتیویتی‌های stopped به عنوان killable (قابل حذف) علامت

گذاری می‌شوند. اگر فرآیند برنامه ما به دلیل اینکه یک برنامه پس زمینه با اولویت پایین است، از بین برود، می‌توان مطمئن بود که Activity.onSaveInstanceState(Bundle) قبلا فراخوانی شده است.

پیاده‌سازی پیش‌فرض onSaveInstanceState(Bundle) همه ویوهای اکتیویتی را برای ذخیره وضعیت خود در شی Bundle هدایت می‌کند. Bundle ساختاری است که کلیدهای رشته‌ای را به مقادیر نگاشت می‌کند.

شما قبلا Bundle را دیده‌اید که به onCreate(Bundle?) پاس می‌شود:

```
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    ...
}
```

وقتی onCreate(Bundle?) را بازنویسی می‌کنیم، onCreate(Bundle?) مربوط به سوپرکلاس اکتیویتی را نیز فراخوانی می‌کنیم و Bundle را که به تازگی دریافت کرده‌ایم به آن می‌فرستیم. در پیاده‌سازی سوپرکلاس، حالت ذخیره شده ویوها بازیابی می‌شود و برای ایجاد مجدد سلسله مراتب ویوی اکتیویتی استفاده می‌شود.

بازنویسی متد onSaveInstanceState(Bundle)

می‌توان onSaveInstanceState(Bundle) را برای ذخیره داده‌ها در Bundle بازنویسی کرد. به این ترتیب امکان خواندن آن در onCreate(Bundle?) فراهم می‌شود. در برنامه خودتان مقدار currentIndex را در زمان مرگ فرآیند ذخیره کنید.

ابتدا در MainActivity.kt، یک ثابت اضافه کنید که کلید جفت کلید–مقدار است که در Bundle ذخیره می‌شود.

لیست ۴.۹ افزودن کلید برای مقدار (MainActivity.kt)

```
import ...
private const val TAG = "MainActivity"
private const val KEY_INDEX = "index"
class MainActivity : AppCompatActivity() {
...
}
```

در مرحله بعد onSaveInstanceState(Bundle) را بازنویسی کنید تا مقدار currentIndex را با ثابت INDEX_KEY به عنوان کلید آن در Bundle بنویسید.

لیست ۴.۱۰ onSaveInstanceState(...) (MainActivity.kt)

```
...
override fun onDestroy() {
    ...
}
override fun onSaveInstanceState(savedInstanceState: Bundle)
{
    super.onSaveInstanceState(savedInstanceState)
    Log.i(TAG, "onSaveInstanceState")
    savedInstanceState.putInt(KEY_INDEX,
        quizViewModel.currentIndex)
}
...
```

در نهایت در onCreate(Bundle?) این مقدار را بررسی می‌کنیم تا در صورت وجود آن را به currentIndex اختصاص دهیم. اگر مقداری با کلید "index" در Bundle وجود نداشت، یا اگر شی Bundle مربوطه تهی بود، مقدار را روی صفر تنظیم می‌کنیم.

لیست ۴.۱۱ بررسی Bundle در onCreate(Bundle?) (MainActivity.kt)

```
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
```

```
        …
        ViewCompat.setOnApplyWindowInsetsListener(…) {
            …
        }
        val currentIndex =
            savedInstanceState?.getInt(KEY_INDEX, 0) ?: 0
        quizViewModel.currentIndex = currentIndex
...
}
```

onCreate یک Bundle تهی‌پذیر (nullable) را به عنوان ورودی می‌پذیرد. به این دلیل است که وقتی نمونه جدیدی از اکتیویتی توسط کاربر برای اولین بار راه اندازی می‌شود Bundle تهی (null) خواهد بود. اما هنگامی که اکتیویتی پس از چرخش یا مرگ فرآیند دوباره ایجاد می‌شود، شی Bundle غیر تهی خواهد بود. Bundle غیر تهی شامل هر جفت کلید–مقداری است که در onSaveInstanceState(Bundle) اضافه می‌کنیم. این Bundle ممکن است حاوی اطلاعات اضافه شده توسط framework باشد، مانند محتویات EditText یا سایر وضعیت ویجت‌های رابط کاربری اولیه.

آزمایش چرخش آسان بود و خوشبختانه وضعیت حافظه کم (low-memory) نیز همینطور است. اکنون آن را امتحان می‌کنیم.

در دستگاه یا شبیه‌ساز خود، نماد تنظیمات را در لیست برنامه‌ها پیدا کرده و روی آن کلیک کنید. وارد گزینه developer options شوید. به بخش Apps بروید و تنظیمات با عنوان Don’t keep activities را فعال کنید. این کار باعث می‌شود که اندروید پس از هر خروج از اپلیکیشن فورا اکتیویتی آن را از حافظه پاک کند.

اکنون GeoQuiz را اجرا و NEXT را فشار دهید تا به سوال دیگری بروید و دکمه Home را فشار دهید. همانطور که می‌دانید با فشار دادن دکمه Home باعث paused و stopped اکتیویتی می‌شوید. گزارش‌ها به شما می‌گویند که اکتیویتی stopped از بین هم رفته است، درست مانند اینکه سیستم عامل اندروید آن را برای خالی کردن حافظه از بین برده باشد است. با این حال، همانطور که گزارش‌ها نیز نشان می‌دهند، onSaveInstanceState(Bundle) فراخوانی شده است، بنابراین امیدی وجود دارد.

برنامه را بالا بیاورید (با استفاده از لیست برنامه‌های اخیر در دستگاه یا شبیه‌ساز) تا ببینید آیا وضعیت شما آنطور که انتظار داشتید ذخیره شده است یا خیر. وقتی GeoQuiz با سوالی که آخرین بار دیدید باز شد، به خودتان افتخار کنید.

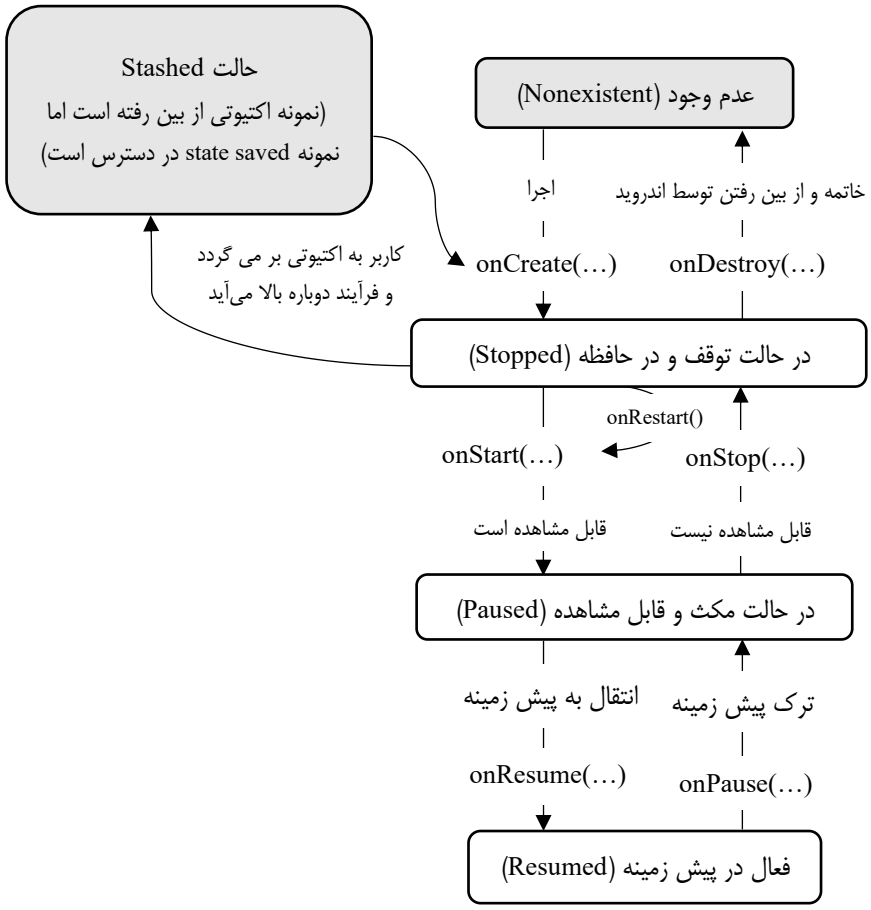
پس از اتمام آزمایش، Don’t keep activities را غیرفعال کنید، زیرا باعث کاهش عملکرد می‌شود.

Saved instance state و activity records

چگونه داده‌هایی که در onSaveInstanceState(Bundle) ذخیره می‌کنید از مرگ اکتیویتی (و فرآیند) جان سالم به در می‌برد؟ هنگامی که onSaveInstanceState(Bundle) فراخوانی می‌شود، داده‌ها در شی Bundle ذخیره می‌شوند. سپس آن شی Bundle توسط سیستم عامل در activity record اکتیویتی شما قرار می‌گیرد.

برای درک رکورد اکتیویتی، بیایید یک حالت stashed به چرخه حیات اکتیویتی اضافه کنیم (شکل ۴.۷).

شکل ۴.۷ چرخه حیات کامل اکتیویتی



هنگامی که اکتیویتی شما stashed می‌شود، شی Activity وجود ندارد، اما شی رکورد اکتیویتی در سیستم عامل زنده است. سیستم عامل می‌تواند در صورت نیاز با استفاده از رکورد اکتیویتی، اکتیویتی را دوباره برگرداند.

توجه داشته باشید که اکتیویتی شما می‌تواند بدون فراخوانی ()onDestroy به حالت stashed منتقل شود. می‌توانید به فراخوانی ()onStop و (onSaveInstanceState(Bundle تکیه کنید (مگر اینکه مشکلی در دستگاه پیش آمده باشد). به طور معمول (onSaveInstanceState(Bundle را بازنویسی می‌کنیم تا داده‌های کوچک و گذرا را که متعلق به اکتیویتی فعلی است در Bundle ذخیره کند. ()onStop را برای ذخیره داده‌های دائمی، مانند چیزهایی که کاربر در حال ویرایش است، بازنویسی کنید، زیرا ممکن است اکتیویتی شما در هر زمانی از بین برود.

بنابراین چه زمانی رکورد اکتیویتی از بین می‌رود؟ وقتی اکتیویتی به پایان می‌رسد، واقعا یک بار برای همیشه از بین می‌رود. در آن مرحله، activity record شما دورانداخته می‌شود. همچنین activity record در راه اندازی مجدد نیز دور انداخته می‌شود.

ViewModel در مقابل Saved Instance State

در حالی که Saved Instance State یک رکورد اکتیویتی را در مرگ فرآیند ذخیره می‌کند، یک رکورد اکتیویتی را در یک تغییر پیکربندی نیز ذخیره می‌کند. هنگامی که برای اولین بار اکتیویتی را راه اندازی می‌کنید، bundle مربوط به saved instance state تهی است. هنگامی که دستگاه را می‌چرخانید، سیستم عامل (onSaveInstanceState(Bundle را در اکتیویتی شما فراخوانی می‌کند. سپس سیستم عامل داده‌هایی را که در Bundle ذخیره کرده‌اید به (onCreate(Bundle? منتقل می‌کند.

اگر Saved Instance State هم از تغییرات پیکربندی و هم از مرگ فرآیند برنامه تان را محافظت می‌کند، اصلا چرا با ViewModel کار می‌کنیم و این همه به خودمان زحمت می‌دهیم؟ اگر منصف باشیم، GeoQuiz آنقدر ساده است که می‌توانستیم فقط با Saved Instance State کار را تمام کنیم.

با این حال، اکثر برنامه‌ها مانند GeoQuiz به مجموعه داده‌های بانک سؤال کوچک متکی نیستند. این روزها اکثر برنامه‌ها داده‌های پویا را از یک پایگاه داده، از اینترنت یا ترکیبی از هر دو بیرون می‌کشند. این عملیات ناهمزمان هستند، می‌توانند کند باشند و از باتری و منابع شبکه استفاده می‌کنند. گره زدن این عملیات به چرخه حیات اکتیویتی می‌تواند بسیار خسته کننده و مستعد خطا باشد.

ViewModel زمانی که از آن برای هماهنگ کردن داده‌های پویا برای اکتیویتی استفاده می‌کنیم واقعا می‌درخشد، همانطور که در فصل‌های آینده خواهید دید. به عنوان مثال ViewModel ادامه عملیات دانلود

را در یک تغییر پیکربندی ساده می‌کند. همچنین بارگیری داده‌ها از حافظه که هزینه بالایی دارد را به هنگام تغییر پیکربندی آسان می‌کند. همانطور که مشاهده کردید ViewModel پس از اتمام اکتیویتی توسط کاربر به طور خودکار پاک می‌شود.

ViewModel در سناریوی مرگ فرآیند جلوه خوب خود را از دست می‌دهد. زیرا همراه با فرآیند از حافظه پاک می‌شود. اینجاست که onSaveInstanceState(Bundle) در مرکز صحنه قرار می‌گیرد. اما onSaveInstanceState(Bundle) محدودیت‌های خاص خود را دارد.

از آنجایی که onSaveInstanceState(Bundle) روی دیسک ذخیره می‌شود، باید از ذخیره آبجکت‌ها بزرگ یا پیچیده خودداری گردد.

مسئله این نیست که کدام بهتر است، ViewModel یا Saved Instance State ؟ توسعه دهندگان باهوش از Saved Instance State و ViewModel به صورت هماهنگ استفاده می‌کنند.

از حالت Saved Instance State برای ذخیره حداقل مقدار اطلاعات لازم برای ایجاد مجدد وضعیت رابط کاربری (به عنوان مثال، فهرست سوال فعلی) استفاده کنید و از ViewModel برای ذخیره مجموعه غنی از داده‌های مورد نیاز برای پر کردن رابط کاربری در حافظه در تغییرات پیکربندی برای دسترسی سریع و آسان استفاده کنید. هنگامی که اکتیویتی پس از مرگ فرآیند دوباره ایجاد می‌شود، از اطلاعات Saved Instance State برای راه اندازی ViewModel استفاده کنید که گویی ViewModel و اکتیویتی هرگز از بین نرفته‌اند.

نه ViewModel و نه Saved Instance State راه حلی برای ذخیره‌سازی طولانی مدت نیستند. اگر برنامه شما نیاز به ذخیره داده‌هایی دارد که باید تا زمانی که برنامه روی دستگاه نصب شده است، صرف نظر از وضعیت اکتیویتی شما، از یک جایگزین ذخیره‌سازی دائمی استفاده کنید. در این کتاب با دو گزینه ذخیره‌سازی پایدار محلی آشنا خواهید شد: پایگاه داده‌ها و shared preferences که در فصل‌های آتی به آنها خواهیم پرداخت. تنظیمات shared preferences برای داده‌های بسیار کوچک و بسیار ساده خوب است. یک پایگاه داده محلی گزینه بهتری برای داده‌های بزرگتر و پیچیده‌تر است.

علاوه بر ذخیره‌سازی محلی، می‌توانیم داده‌ها را در یک سرور راه دور در جایی ذخیره کنیم. در فصل مربوطه نحوه دسترسی به داده از یک وب سرور را یاد خواهید گرفت.

اگر GeoQuiz سوالات بیشتری داشت، منطقی بود که سوالات را در یک پایگاه داده یا یک وب سرور ذخیره کنیم. از آنجایی که سؤالات ثابت هستند، منطقی است که آنها را مستقل از حالت چرخه حیات اکتیویتی حفظ کنیم. اما دسترسی به پایگاه‌های داده در مقایسه با دسترسی به مقادیر موجود در حافظه یک عملیات نسبتا کند است. بنابراین منطقی است که آنچه را که برای نمایش رابط کاربری خود نیاز دارید

بارگیری کنیم و در حالی که رابط کاربری با استفاده از ViewModel نشان داده می‌شود، آن را در حافظه نگه داریم.

در این فصل، با محاسبه صحیح تغییرات پیکربندی و مرگ فرآیند، باگ‌های GeoQuiz را از بین بردید. در فصل بعدی، یاد خواهید گرفت که چگونه از debugging اندروید استودیو برای عیب‌یابی سایر باگ‌های خاص برنامه که ممکن است ایجاد شوند، استفاده کنید.

بیشتر بدانیم: اجتناب از راه حل نیمه کاره

برخی از افراد سعی می‌کنند با غیرفعال کردن چرخش، از دست دادن وضعیت رابط کاربری را برطرف کنند. اگر کاربر نتواند برنامه را بچرخاند، هرگز وضعیت رابط کاربری خود را از دست نمی‌دهد، درست است؟ این درست است – اما متأسفانه این رویکرد برنامه شما را مستعد اشکالات دیگر می‌کند.

اول، تغییرات پیکربندی دیگری مانند تغییر اندازه پنجره و تغییر به حالت شب نیز وجود دارد که می‌تواند در زمان اجرا رخ دهد. می‌توانید این تغییرات را نیز نادیده بگیرید. اما این روش خوبی نیست – یکی از ویژگی‌های خوب سیستم را غیرفعال می‌کند – یعنی به طور خودکار استفاده از منابع مناسب را بر اساس تغییرات پیکربندی زمان اجرا را از دست می‌دهید.

دوم، مدیریت تغییرات پیکربندی یا غیرفعال کردن چرخش، مشکل مرگ فرآیند را حل نمی‌کند.

اگر می‌خواهید برنامه خود را در حالت عمودی یا افقی قفل کنید همچنان باید در برابر تغییرات پیکربندی و مرگ فرآیند به صورت دفاعی برنامه ریزی کنید. شما با دانش جدید خود در مورد ViewModel و saved instance state برای انجام این کار آماده هستید.

به طور خلاصه، مقابله با از دست دادن وضعیت رابط کاربری با مسدود کردن تغییرات پیکربندی قابل حل نیست.

۵

اشکال زدایی (Debugging)

در این فصل متوجه خواهیم شد که وقتی برنامه‌ها به مشکل می‌خورند و رفتار نامناسبی دارند و یا خطایی به همراه دارند، چه کاری باید انجام دهیم. یاد خواهیم گرفت که چگونه از Logcat، Android Lint و debugger همراه با اندروید استودیو استفاده کنیم.

برای تمرین دیباگ، قرار است مشکلی ساختگی در برنامه خود به وجود آوریم. در MainActivity.kt و در متد onCreate(Bundle?) کدی را که با آن questionTextView را با شناسه آن پیدا می‌کنید، به توضیح یا کامنت تبدیل کنید.

لیست ۵.۱ کامنت کردن یک خط مهم (MainActivity.kt)

```
override fun onCreate(savedInstanceState: Bundle?) {
    ...
    nextButton = findViewById(R.id.next_button)
    //questionTextView =
        //findViewById(R.id.question_text_view)
    ...
}
```

GeoQuiz را اجرا و ببینید چه اتفاقی می‌افتد. برنامه تقریبا بلافاصله از کار می‌افتد.

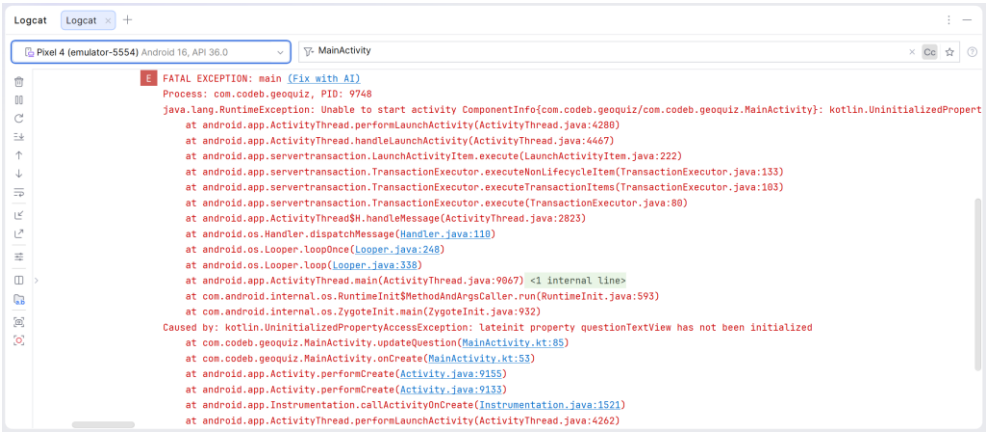
اگر صفحه را تماشا کنید، ممکن است برای لحظه‌ای کوتاه قبل از ناپدید شدن برنامه را ببینید. البته دقیقا می‌دانید که چه مشکلی در برنامه شما وجود دارد.

استثناها (Exceptions) و دنباله پشته (Stack Traces)

پنجره ابزار Logcat تمام پیام‌های لاگ که از دستگاه اندرویدی یا شبیه‌ساز می‌آید را نمایش می‌دهد. آن را باز کنید تا ببینید چه اتفاقی افتاده است. اگر بالا و پایین بروید، در نهایت باید خطوط قرمزی را همانطور که در شکل ۵.۲ نشان داده شده است، پیدا کنید. این یک گزارش استاندارد استثنای AndroidRuntime است.

اگر چیز زیادی در Logcat نمی‌بینید و نمی‌توانید استثنا را پیدا کنید، فیلتر را به MainActivity و یا AndroidRuntime تغییر دهید.

شکل ۵.۲ ردیابی استثنا و دنباله پشته در Logcat

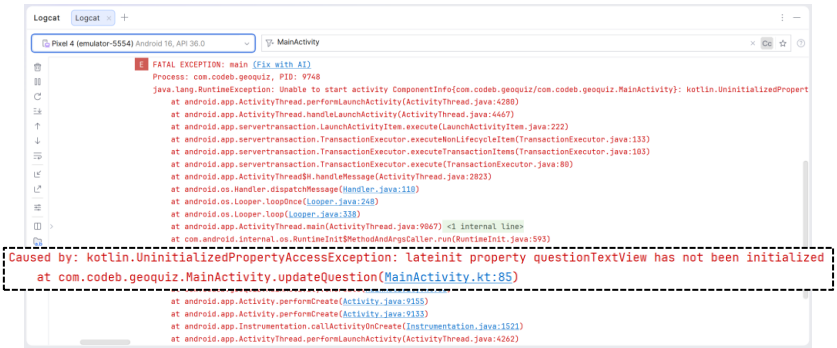


این گزارش به ما استثنای سطح بالا (top-level) و دنباله پشته آن، سپس جزئیات استثناء و گزارش آن و ... را تا زمانی که استثنایی بدون دلیل پیدا کند، ارایه می‌دهد. در اندروید Stack Trace گزارشی از زنجیره فراخوانی متدهاست که منجر به یک خطا یا استثنا شده؛ این ابزار اصلی برای پیدا کردن محل دقیق کرش یا Exception در کد است.

ممکن است دیدن یک استثنا java.lang در گزارش پشته عجیب به نظر برسد، زیرا شما در حال نوشتن کد کاتلین هستید. هنگام build، کد کاتلین به همان نوع بایت کد سطح پایینی که کد جاوا به آن کامپایل می‌شود، تبدیل می‌گردد. در طول این فرآیند بسیاری از استثناهای کاتلین به java.lang نگاشت می‌شوند. kotlin.RuntimeException سوپرکلاس kotlin.UninitializedPropertyAccessException است و هنگام اجرا در اندروید با نام java.lang.RuntimeException جایگزین می‌شود.

در بیشتر کدهایی که می‌نویسیم، آخرین استثنا بدون دلیل، جالبترین است. در کد ما این استثنا بدون علت یک kotlin.UninitializedPropertyAccessException است. خطی که درست زیر این استثنا قرار دارد، اولین خط در گزارش پشته آن است. این خط به شما کلاس و تابعی را نشان می‌دهد که استثنا در آن رخ داده است. همچنین استثنا در چه فایل و شماره خطی رخ داده است را نیز نمایش می‌دهد. روی لینک آبی کلیک کنید و اندروید استودیو شما را به آن خط در کد می‌برد.

شکل ۵.۳ ردیابی استثنا و دنباله پشته در Logcat



خطی که به آن هدایت می‌شوید اولین استفاده از questionTextView در داخل updateQuestion() است. عبارت UninitializedPropertyAccessException اشاره‌ای به خطای "این متغیر مقداردهی اولیه نشده است" را دارد.

برای رفع اشکال، خط مقداردهی اولیه questionTextView را از حالت کامنت خارج کنید.

هنگامی که با استثنائات زمان اجرا مواجه می‌شوید، به یاد داشته باشید که به دنبال آخرین استثنا در Logcat و اولین خط در گزارش پشته آن بگردید تا شما را به کدی که نوشته‌اید باز گرداند. اینجا مکانی است که مشکل رخ داده است و بهترین نقطه برای شروع جستجوی پاسخ است.

تشخیص رفتارهای نادرست

مشکلات مربوط به برنامه‌های شما همیشه خرابی نخواهد بود. در برخی موارد، آنها رفتار نادرست خواهند داشت. به عنوان مثال فرض کنید هر بار که دکمه NEXT را فشار می‌دهید، هیچ اتفاقی نیفتد.

در MainActivity.kt، تغییری در شنونده nextButton ایجاد کنید. کدی را که سؤال بعدی را نمایش می‌دهد، به حالت کامنت درآورید.

لیست ۵.۲ فراموش کردن یک خط مهم (MainActivity.kt)

```
override fun onCreate(savedInstanceState: Bundle?) {
...
nextButton.setOnClickListener {
    //quizViewModel.moveToNext()
    updateQuestion()
}
```

GeoQuiz را اجرا کرده و دکمه NEXT را فشار دهید. نباید هیچ تاثیری ببینید.

این اشکال پیچیده‌تر از خطایی است که در اول فصل ایجاد و حل کردیم. این اشکال برنامه را متوقف و ایجاد استثنا نمی‌کند، بنابراین رفع اشکال، آسان نخواهد بود. علاوه بر آن، این رفتار نادرست می‌تواند به دو روش مختلف ایجاد شود: ممکن است ایندکس تغییر نکند، یا ممکن است ()updateQuestion فراخوانی نشود.

می‌دانید که چه چیزی باعث این اشکال شده است. اما اگر این نوع اشکال به خودی خود ظاهر شد، باید آن را ردیابی کنیم. در چند بخش بعدی، دو راه برای انجام این کار را مشاهده خواهید کرد: ثبت تشخیصی (Log) گزارش پشته و استفاده از دیباگ برای تنظیم یک breakpoint.

لاگ کردن گزارش پشته

در MainActivity، یک دستور Log به ()updateQuestion اضافه کنید.

لیست ۵.۳ استثنا برای سرگرمی (MainActivity.kt)

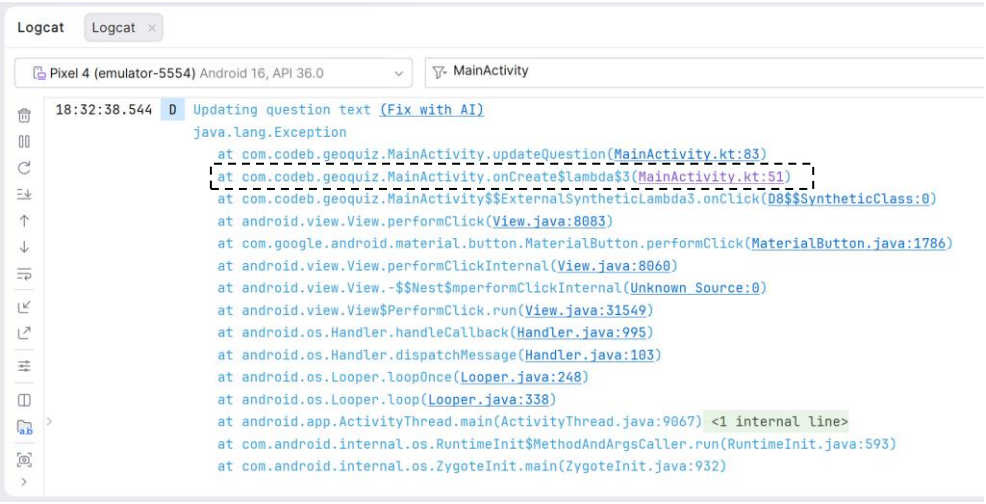
```
private fun updateQuestion() {
    Log.d(TAG, "Updating question text", Exception())
    val questionTextResId =quizViewModel.currentQuestionText
    questionTextView.setText(questionTextResId)
}
```

فرمت Log.d(String, String, Throwable) کل گزارش پشته را ثبت می‌کند. گزارش پشته به شما می‌گوید که فراخوانی به ()updateQuestion کجا انجام شده است.

استثنایی که به Log.d (String, String, Throwable) منتقل می‌کنیم لزوما استثنایی نیست که برنامه ما را متوقف کند. می‌توانیم یک استثنای کاملا جدید ایجاد و آن را به تابع بفرستیم تا گزارشی از محل ایجاد استثنا دریافت کنیم.

GeoQuiz را اجرا و دکمه NEXT را فشار دهید، سپس خروجی Logcat را با "MainActivity" فیلتر و آن را بررسی کنید (شکل ۵.۴).

شکل ۵.۴ نتایج



خط بالایی در گزارش پشته، خطی است که در آن استثنا را Log کرده‌ایم. یک خط پس از آن می‌توانید ببینید که updateQuestion() از داخل لمبدا در MainActivity فراخوانی شده است. روی لینک این خط کلیک کنید، شما به خطی که کامنت کرده بودید می‌رسید. اشکال را برطرف نکنید، چون می‌خواهیم از debugger برای پیدا کردن دوباره اشکال استفاده کنیم.

خروجی گرفتن از لاگ‌ها می‌تواند بسیار مفید باشد، اما کار پر زحمت و طولانی به نظر می‌رسد. از سوی دیگر، گاهی اوقات گزارش پشته دقیقا همان چیزی نیست که ما نیاز داریم.

قبل از ادامه، دستور Log را در MainActivity.kt حذف کنید.

تنظیم Breakpoint

اکنون از دیباگر ارائه شده با اندروید استودیو برای ردیابی همان باگ استفاده خواهیم کرد. یک breakpoint در updateQuestion() تنظیم می‌کنیم تا ببینیم که آیا تابع مورد نظر فراخوانی می‌شود یا خیر.

یک breakpoint در ابتدای یک خط، اجرا را متوقف می‌کند و به شما امکان می‌دهد خط به خط آنچه را که در آینده اتفاق می‌افتد بررسی کنید.

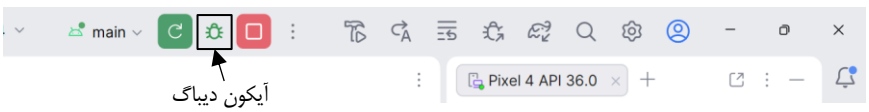
در MainActivity.kt، به تابع updateQuestion() برگردید. در کنار خط اول این تابع، روی ناحیه خاکستری در حاشیه سمت چپ کلیک کنید. اکنون باید یک دایره قرمز را در سمت چپ مانند آنچه در شکل ۵.۵ نشان داده شده است مشاهده کنید. این یک breakpoint است. می‌توانید با کلیک دوباره و یا با فشردن Ctrl+F8 یک breakpoint را روشن و خاموش کنید.

شکل ۵.۵ breakpoint

```
82      private fun updateQuestion() {
            val questionTextResId =
84              quizViewModel.currentQuestionText
85          questionTextView.setText(questionTextResId)
86      }
```

برای درگیر کردن دیباگر و راه اندازی breakpoint خود، باید برنامه خود را به جای اجرای آن دیباگ کنیم. برای دیباگ برنامه، روی آیکون دیباگ بالا صفحه همانند شکل ۵.۵ کلیک و یا Run → Debug app در نوار منو انتخاب کنید.

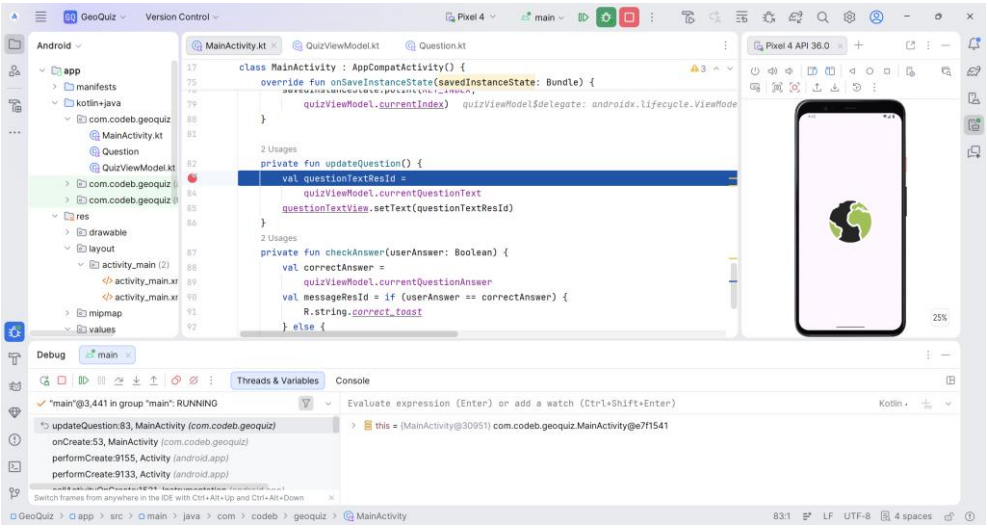
تصویر ۵.۶ دکمه‌های برنامه دیباگ



پس از اجرا، برنامه توسط دیباگر متوقف می‌شود. برنامه دقیقا تا ابتدای خطی که breakpoint دارد پیش می‌رود و با رسیدن به آن مکث می‌کند.

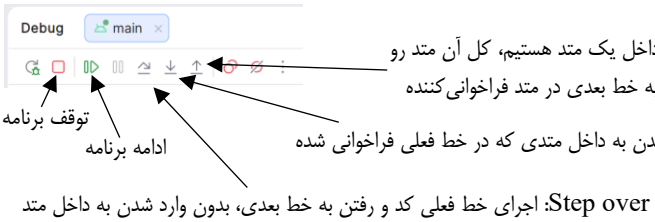
در تصویر ۵.۷ مشاهده می‌کنید که MainActivity.kt در پنجره ابزار ویرایشگر باز و خطی که breakpoint در آن تنظیم شده، های‌لایت شده است. پنجره ابزار دیباگ در پایین صفحه اکنون قابل مشاهده است.

تصویر ۵.۷ همانجا توقف کنید!



می‌توانیم از دکمه‌های فلشی شکل در بالای پنجره ابزار دیباگ (تصویر ۵.۸) برای اجرای ادامه برنامه خود استفاده کنیم. همچنین می‌توان از دکمه Assessment Expression برای اجرای دستورات ساده کاتلین در صورت نیاز در حین دیباگ استفاده کنید، که یک ابزار قدرتمند است.

تصویر ۵.۸ کنترل‌های پنجره ابزار دیباگ

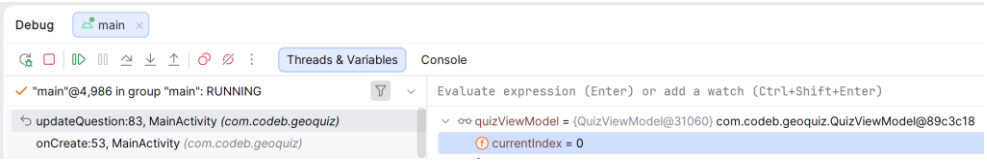


با استفاده از گزارش پشته به فراخوانی updateQuestion() از داخل onCreate(Bundle?) پی می‌بریم. هدف ما بررسی رفتار دکمه NEXT است. بنابراین روی دکمه Resume Program کلیک کنید تا اجرا ادامه یابد. سپس دکمه NEXT را در GeoQuiz فشار دهید تا ببینیم breakpoint اجرای برنامه را متوقف می‌کند یا خیر. (باید اینگونه باشد.)

اکنون که در یک نقطه جالب از اجرا متوقف شده‌ایم، می‌توان نگاهی به اطراف انداخت. ویوی Threads & Variables به ما امکان می‌دهد مقادیر آبجکت‌ها (objects) را در برنامه خود بررسی کنیم. در بالا، باید مقدار this (خود نمونه MainActivity) را ببینید. این متغیر را گسترش دهید تا تمام متغیرهای اعلام شده در MainActivity، در سوپرکلاس MainActivity (Activity)، در سوپرکلاس Activity، در super-superclass آن و غیره را ببینید. در حال حاضر، روی متغیرهایی که ایجاد کرده‌اید، تمرکز کنید.

فقط باید به مقدار یه متغییر توجه داشته باشیم: quizViewModel.currentIndex. در ویوی متغیرها راست کلیک کرده و گزینه New Watch را بزنید. برای مقدار ورودی quizViewModel را وارد و Enter را بزنید. حالا گزینه جدید با نام ویومدل ما به وجود می‌آید روی آن کلیک کرده و مقدار متغییر currentIndex را مشاهده کنید (شکل ۵.۹).

شکل ۵.۹ بررسی مقادیر متغیر در زمان اجرا



انتظار داشتیم currentIndex دارای مقدار ۱ باشد. زیرا پس از فشردن دکمه NEXT، currentIndex باید از ۰ به ۱ تغییر می‌کرد. با این حال، همانطور که در شکل ۵.۹ نشان داده شده است، currentIndex هنوز مقدار ۰ دارد.

کد را در پنجره ابزار ویرایشگر بررسی کنید. کد موجود در MainActivity.updateQuestion() به سادگی متن سوال را بر اساس محتویات QuizViewModel به روز می‌کند. هیچ مشکلی در آنجا وجود ندارد. پس این اشکال از کجا سرچشمه می‌گیرد؟

برای ادامه بررسی خود، باید مشخص کنید چه کدی درست قبل از MainActivity.updateQuestion() اجرا شده است. برای انجام این کار، روی دکمه Step Out کلیک کنید.

پنجره ابزار ویرایشگر را بررسی کنید. اکنون به زیبایی به OnClickListener دکمه nextButton خود درست پس از فراخوانی updateQuestion() که منبع رفتار غیرمعقول است، منتقل شده‌اید.

همانطور که می‌دانستید، رفتار مشکل ساز نتیجه فراخوانی نشده quizViewModel.moveToNext() می‌باشد (زیرا آن را کامنت کرده‌اید). می‌خواهیم مشکل را برطرف کنیم – اما قبل از ایجاد هر گونه تغییر در کد، باید دیباگ را متوقف کنید. اگر کد خود را در حین دیباگ ویرایش کنید، کدی که با دیباگر پیوست

شده است در مقایسه با آنچه در پنجره ابزار ویرایشگر است مغایر خواهد بود، بنابراین دیباگ می‌تواند اطلاعات گمراه کننده را در مقایسه با کد به روز شده نشان دهد.

می‌توانید دیباگ را به دو روش متوقف کنید: برنامه را متوقف کنید، یا به سادگی دیباگ را متوقف کنید. برای متوقف کردن برنامه، روی دکمه Stop که در شکل ۵.۸ نشان داده شده است کلیک کنید.

اکنون OnClickListener را به شکوه سابق برگردانید.

دو روش مختلفی که برای عیب‌یابی کد نادرست امتحان کردیم: لاگ گزارش پشته و تنظیم breakpoint در دیباگ. کدام بهتر است؟ هر کدام کاربردهای خود را دارند و احتمالا یکی از آنها مورد علاقه شما خواهد بود.

ثبت گزارش پشته امکان ردیابی پشته را از چندین مکان ممکن می‌سازد. نکته منفی این روش نوشتن دستورات جدید اضافی است. دیباگ راحت‌تر است. اگر برنامه خود را با دیباگر اجرا کنیم، می‌توانیم در حین اجرای برنامه یک breakpoint تنظیم و اطلاعات بسیاری از چندین منبع به دست بیاورید.

دیباگ مختص اندروید (Android-Specific)

بیشتر دیباگ کردن‌های اندروید درست مانند دیباگ کردن‌های کاتلین است. با این حال، گاهی اوقات با بخش‌های خاص اندروید، مانند منابع، که کامپایلر کاتلین چیزی در مورد آنها نمی‌داند، با مشکلاتی مواجه خواهید شد. در این بخش با دو نوع مشکل مختص اندروید آشنا می‌شویم: مشکلات Android Lint و مشکلات کلاس R.

استفاده از Android Lint

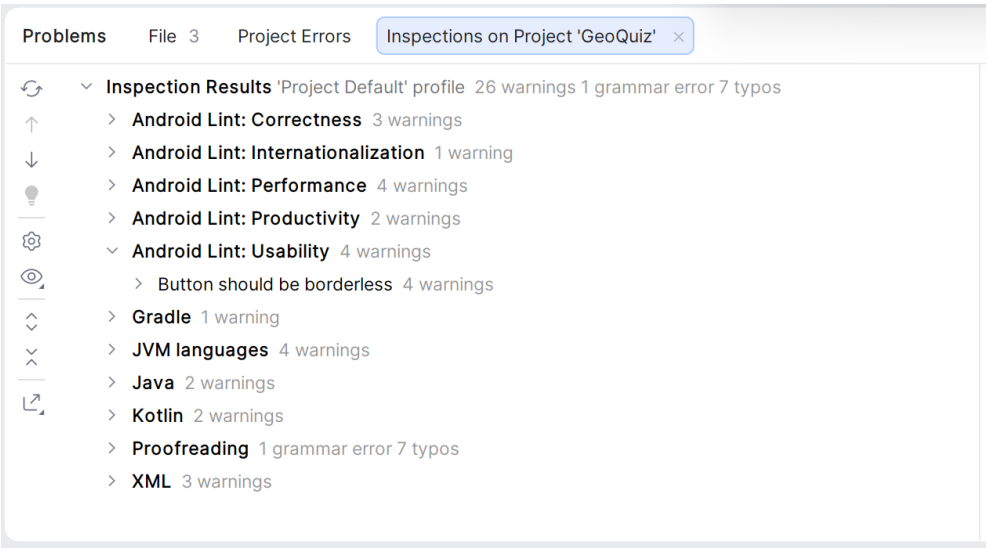
Android Lint (یا فقط "Lint") یک تحلیلگر استاتیک (static analyzer) برای کد اندروید است. تحلیلگر استاتیک برنامه‌ای است که کد را بدون اجرای آن بررسی می‌کند تا ایرادها را پیدا کند. ابزار Lint با شناختی که از فریم‌ورک‌های Android دارد می‌تواند تحلیل‌های عمیقتری روی کد ما انجام دهد. در بسیاری از موارد، توصیه‌های Lint ارزش توجه و پیگیری را دارد.

در فصل ۷، خواهید دید که لینت در مورد مشکلات سازگاری (compatibility problems) گزارش هشدار ارایه می‌دهد. Lint همچنین می‌تواند بررسی نوع (type-checking) را برای اشیایی که در XML تعریف شده‌اند، انجام دهد.

می‌توانیم به صورت دستی Lint را اجرا کنیم تا تمام مشکلات احتمالی، از جمله موارد کم اهمیت، در پروژه خود را ببینیم. Code → Inspect Code را از نوار منو انتخاب کنید. در جواب سوال "می‌خواهید کدام بخش از پروژه خود را بررسی کنید"، کل پروژه را انتخاب کرده و روی OK کلیک کنید. اندروید استودیو، Lint و همچنین چند تحلیلگر استاتیک دیگر را مانند تحلیلگر املای کلمات و کاتلین را روی کد شما اجرا می‌کند.

پس از اتمام اسکن، دسته‌بندی‌هایی از مشکلات احتمالی را در پنجره ابزار inspection مشاهده خواهید کرد (شکل ۵.۱۰).

شکل ۵.۱۰ هشدارهای Lint



(اگر گزارش شما با تصویر ارایه شده متفاوت است، نگران نباشید. ابزار اندروید به طور مداوم در حال تکامل است.)

گزینه Internationalization را گسترش دهید تا اطلاعات دقیق‌تری در مورد این موضوع در پروژه خود مشاهده کنید. روی Using left/right instead of start/end attributes کلیک کنید تا اطلاعات مربوط به این مورد خاص به نمایش درآید (شکل ۵.۱۱).

شکل ۵.۱۱ توضیحات هشدار Lint



Lint هشدار می‌دهد که اگر برنامه شما در دستگاهی با زبان راست چین اجرا شود، می‌تواند مشکلاتی در پی داشته باشد. (در مورد آماده‌سازی اپلیکیشن خود برای استفاده بین المللی در فصل بومی سازی بیشتر یاد خواهید گرفت.)

قطعه کد مشکل ساز در سمت راست نشان داده شده است.

روی توضیحات هشداری که در زیر نام فایل ظاهر می‌شود دوبار کلیک کنید. با این کار res/layout/land/activity_main.xml در پنجره ابزار ویرایشگر باز می‌شود و روی خطی که باعث ایجاد هشدار می‌شود، قرار می‌گیرد:

```
<Button
    android:id="@+id/next_button"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_gravity="bottom|right"
    android:text="@string/next_button"
    android:drawableEnd="@drawable/ic_arrow_right"
    android:drawablePadding="4dp"/>
```

خطی که Lint نگران آن است gravity دکمه NEXT است، که باعث نمایش دکمه NEXT در گوشه سمت راست پایین لیوت افقی می‌شود. برای رفع هشدار، مقدار bottom|right به bottom|end را تغییر دهید. به این ترتیب، دکمه در پایین سمت چپ دستگاهی که روی زبان راست‌چین تنظیم شده است، قرار می‌گیرد.

Lint را دوباره اجرا کنید. مشاهده خواهید کرد که مشکل متن دو طرفه‌ای که به تازگی برطرف کرده‌اید دیگر در نتایج Lint فهرست نمی‌شود.

در بیشتر موارد، حتی اگر به هشدارهای Lint بی‌توجهی کنید، باز هم برنامه شما به خوبی اجرا می‌شود. با این حال اغلب رسیدگی به هشدارهای Lint می‌تواند به جلوگیری از مشکلات آتی کمک کند یا تجربه کاربران برنامه شما را بهتر کند. توصیه می‌کنیم همه هشدارهای Lint را جدی بگیرید، حتی اگر در نهایت تصمیم شما صرف نظر از مدیریت آنها باشد. در غیر این صورت ممکن است به نادیده گرفتن Lint عادت کنید و یک مشکل جدی را از دست بدهید.

ابزار Lint اطلاعات دقیقی در مورد هر ایراد و نحوه رفع آن ارائه می‌دهد. به عنوان تمرین، بررسی GeoQuiz با Lint را به شما واگذار می‌کنیم. می‌توانید مشکلات را نادیده بگیرید، یا آنها را همانطور که Lint توصیه می‌کند، برطرف کنید. برای بقیه فصل‌ها فرض می‌کنیم که مسائل باقیمانده Lint را بدون رسیدگی رها کرده‌اید.

مسائل مربوط به کلاس R

با خطاهای Build که هنگام ارجاع به منابع، قبل از افزودن آنها یا حذف منابعی که فایل‌های دیگر به آنها رجوع می‌کنند رخ می‌دهد، آشنا هستید. معمولا ذخیره مجدد فایل‌ها پس از اضافه شدن یک منبع یا حذف مراجع باعث می‌شود اندروید استودیو بدون هیچ سر و صدایی پروژه را دوباره Build کند.

با این حال، گاهی اوقات، خطاهای Build ادامه می‌یابد یا ظاهرا از ناکجا آباد ظاهر می‌شود. اگر این اتفاق برای شما افتاد، موارد زیر را می‌توانید امتحان کنید:

- معتبر بودن XML فایل‌های منبع خود را بررسی کنید: اگر فایل R.java شما برای آخرین build تولید نشده باشد، هر کجا که به منبعی ارجاع دهید، خطاهایی را در پروژه خود مشاهده خواهید کرد. اغلب، این امر به دلیل اشتباه تایپی در یکی از فایل‌های XML شما ایجاد می‌شود. لیوت XML لحظه‌ای اعتبار سنجی نمی‌شود، بنابراین اشتباهات تایپی در این فایل‌ها ممکن است به طور مستقیم مورد توجه شما قرار نگیرد. یافتن اشتباه تایپی و ذخیره مجدد فایل باید باعث بازسازی R.java شود.
- پروژه خود را Clean کنید: یعنی فایل‌های موقت، کش شده و ساخته شده قبلی را حذف کنید تا دوباره ساخته شوند. Build → Clean Project را انتخاب کنید. اندروید استودیو پروژه را از ابتدا می‌سازد، که اغلب خطاها را برطرف می‌کند.
- پروژه خود را با Gradle همگام سازی کنید: اگر تغییراتی در فایل build.gradle خود ایجاد کنید، باید آن را همگام سازی کنید تا تنظیمات ساخت پروژه خود را به روز کنید. File → Sync Project with Gradle Files را انتخاب کنید. اندروید استودیو پروژه را از ابتدا با تنظیمات صحیح پروژه می‌سازد، که می‌تواند به حل مشکلات پس از تغییر پیکربندی Gradle کمک کند.
- Android Lint را اجرا کنید: به هشدارهای لینت توجه زیادی داشته باشید. با استفاده از این ابزار، اغلب مشکلات غیرمنتظره‌ای را کشف خواهید کرد.

اگر هنوز با منابع مشکل دارید (یا مشکلات متفاوتی دارید)، هر گونه خطا و هشدار Lint را نیز بررسی کنید. بازنگری دقیق پیام‌های خطا ممکن است یک اشکال یا اشتباه تایپی را نشان دهد.

چالش: بررسی Layout Inspector

برای دیباگ مشکلات فایل لیوت، Layout Inspector را می‌توان به کار برد. برای شروع، مطمئن شوید که GeoQuiz در شبیه‌ساز اجرا می‌شود و از نوار منو Tools → Layout Inspector را انتخاب کنید. پس از فعال شدن Inspector، می‌توانید با کلیک بر روی عناصر موجود در ویوی Layout Inspector، ویژگی‌های لیوت خود را بررسی کنید.

چالش: بررسی Profiler

پنجره ابزار Profiler گزارش‌های دقیقی از نحوه استفاده برنامه شما از منابع دستگاه اندرویدی‌تان، مانند CPU و حافظه ایجاد می‌کند که هنگام ارزیابی و تنظیم عملکرد برنامه بسیار مفید است.

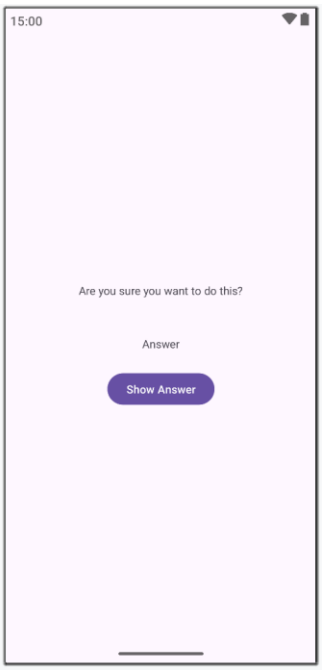
برای مشاهده پنجره ابزار Profiler، برنامه خود را روی یک دستگاه یا شبیه‌ساز اندرویدی متصل اجرا و View → Tool Windows → Profiler را از نوار منو انتخاب کنید. هنگامی که Profiler باز شد، می‌توانید بخش‌هایی برای CPU، حافظه، شبکه و energy مشاهده کنید.

۶

اکتیویتی دوم شما

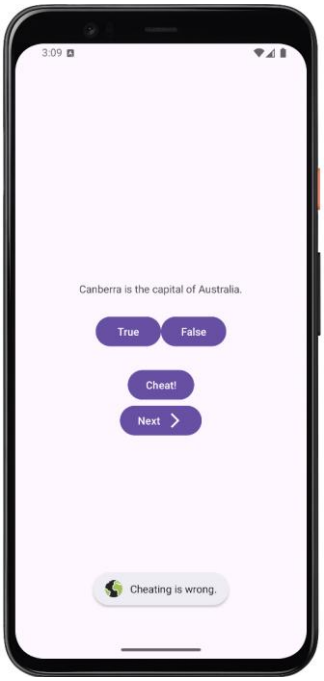
در این فصل، اکتیویتی دوم را به GeoQuiz اضافه خواهیم کرد. یک اکتیویتی صفحه‌ای از اطلاعات را کنترل می‌کند. اکتیویتی دوم صفحه‌ای را اضافه می‌کند که به کاربران فرصت تقلب را با نشان دادن پاسخ سوال فعلی، می‌دهد. شکل ۶.۱ اکتیویتی جدید را نشان می‌دهد.

شکل ۶.۱ CheatActivity فرصتی برای یافتن پاسخ



اگر کاربران به اکتیویتی دوم بروند و از فرصت تقلب استفاده کرده و دوباره به MainActivity بازگردند، پیامی همانند پیام نشاده داده شده در شکل ۶.۲ دریافت خواهند کرد.

شکل ۶.۲ MainActivity می‌داند که آیا تقلب کرده‌اید یا خیر



چرا این یک تمرین برنامه‌نویسی خوب است؟ زیرا شما یاد خواهید گرفت که چگونه:

- یک اکتیویتی جدید و یک لیوت جدید برای آن ایجاد کنید.
- اکتیویتی را از اکتیویتی دیگری راه اندازی کنید. راه اندازی یک اکتیویتی به این معنی است که از سیستم عامل بخواهید یک نمونه اکتیویتی ایجاد و onCreate(Bundle?) آن را فراخوانی کند.
- داده‌ها را بین اکتیویتی والد (راه اندازی کننده) و اکتیویتی فرزند (راه اندازی شونده) منتقل کنید.

تنظیم پیش‌نیازهای اکتیویتی دوم

در این فصل کارهای زیادی برای انجام دادن پیش‌رو داریم. خوشبختانه، برخی از کارهای سخت را می‌توان به New Android Activity اندروید استودیو واگذار کرد.

قبل از این کار، res/values/strings.xml را باز و تمام منابع رشته‌ای را که برای این فصل نیاز داریم، اضافه کنید.

لیست ۶.۱ افزودن رشته‌ها (res/values/strings.xml)

```
<resources>
    ...
    <string name="incorrect_toast">Incorrect!</string>
    <string name="warning_text">
        Are you sure you want to do this?
    </string>
    <string name="show_answer_button">Show Answer</string>
    <string name="cheat_button">Cheat!</string>
    <string name="judgment_toast">Cheating is wrong.</string>
</resources>
```

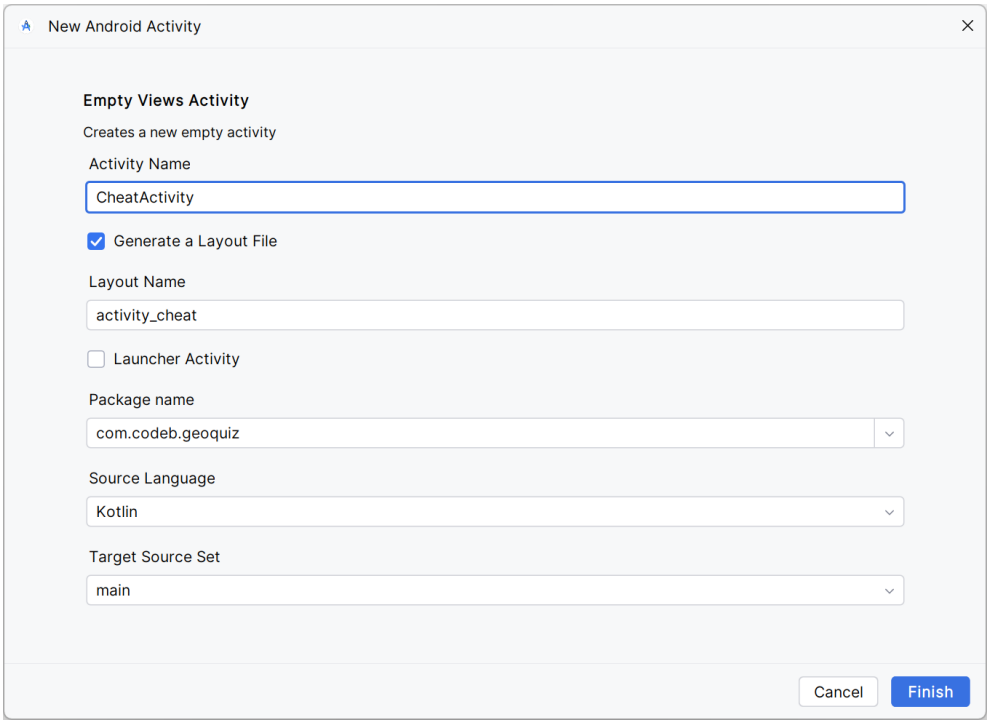
ایجاد اکتیویتی جدید

برای ایجاد یک اکتیویتی حداقل باید با سه فایل مختلف کار کرد: فایل کلاس کاتلین، یک فایل لیوت XML و فایل مانیفست برنامه. اگر این فایل‌ها را به درستی ایجاد و یا تغییر ندهیم، دچار مشکل بزرگی خواهیم شد. برای اطمینان از درستی انجام کار، می‌توان از New Android Activity اندروید استودیو استفاده کرد.

New Android Activity را با کلیک راست بر روی پوشه app در پنجره ابزار پروژه راه اندازی و New → Activity → Empty View Activity را از منو انتخاب کنید.

سپس باید کادری مانند شکل ۶.۳ را مشاهده کنید. CheatActivity را به عنوان نام اکتیویتی وارد کنید. این نام زیر کلاس اکتیویتی شماست. نام لیوت به طور خودکار روی activity_cheat تنظیم می‌شود. این نام فایل لیوت خواهد بود که ویزارد ایجاد می‌کند.

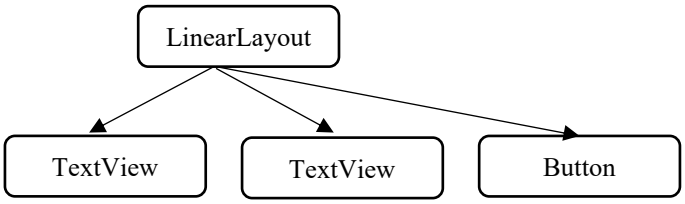
تصویر ۶.۳ ویزارد اکتیویتی جدید



بقیه فیلدها را با مقادیر پیش‌فرض رها کنید، اما مراقب باشید که نام پکیج همان چیزی باشد که انتظار دارید. نام پکیج تعیین می‌کند که CheatActivity.kt در کجای سیستم فایل قرار گیرد. روی دکمه Finish کلیک کنید تا جادو اتفاق بیفتد.

اکنون وقت پرداختن به رابط کاربری اکتیوتی است. اسکرین‌شات ابتدای فصل نشان دهنده چگونگی ویوی CheatActivity است. شکل ۶.۴ تعاریف ویجت را نشان می‌دهد.

شکل ۶.۵ نمودار چیدمان برای CheatActivity



res/layout/activity_cheat.xml را باز و به ویوی Code بروید.

با استفاده از شکل ۶.۴ به عنوان راهنما، سعی کنید XML را برای لیوت ایجاد کنید. لیوت نمونه را با یک لیوت جدید همانند لیست ۶.۲ جایگزین کنید.

لیست ۶.۲ پر کردن لیوت اکتیویتی دوم (res/layout/activity_cheat.xml)

```
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:gravity="center"
    android:orientation="vertical"
    android:id="@+id/main">
    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:padding="24dp"
        android:text="@string/warning_text"/>
    <TextView
        android:id="@+id/answer_text_view"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:padding="24dp"
        tools:text="Answer"/>
    <Button
        android:id="@+id/show_answer_button"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="@string/show_answer_button"/>
</LinearLayout>
```

جایگزین افقی برای activity_cheat.xml ایجاد نخواهیم کرد.

زیر کلاس اکتیویتی جدید

CheatActivity.kt ممکن است به طور خودکار در پنجره ابزار ویرایشگر باز شده باشد. اگر اینطور نبود، آن را از پنجره ابزار پروژه باز کنید.

کلاس CheatActivity در حال حاضر شامل یک پیاده‌سازی اولیه (onCreate(Bundle? است که شناسه منبع لیوت تعریف شده در activity_cheat.xml را به (…)setContentView ارسال می‌کند.

CheatActivity در نهایت کارهای بسیاری در تابع (onCreate(Bundle? خود انجام خواهد داد. فعلا آن را کنار بگذارید تا نگاهی به کار دیگری که New Android Activity برای شما انجام داد، بیاندازیم: اعلام CheatActivity در مانیفست برنامه.

اعلام اکتیویتی‌ها در مانیفست

مانیفست یک فایل XML حاوی داده‌های توصیفی (metadata) است که برنامه شما را در سیستم عامل اندروید توصیف می‌کند. این فایل همیشه AndroidManifest.xml نام دارد و در app/manifests پروژه قرار دارد. در پنجره ابزار پروژه، manifests/AndroidManifest.xml را پیدا کرده و باز کنید.

هر اکتیویتی در برنامه باید در مانیفست اعلام شود تا سیستم عامل بتواند به آن دسترسی داشته باشد.

هنگامی که از New Project wizard برای ایجاد MainActivity استفاده کردید، ویزارد اکتیویتی را برای شما در مانیفست با افزودن XML های‌لایت در لیست ۶.۳، معرفی کرد.

لیست ۶.۳ اعلام CheatActivity در مانیفست (manifests/AndroidManifest.xml)

```
<?xml version="1.0" encoding="utf-8"?>
<manifest
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">
    <application
        android:allowBackup="true"
        android:dataExtractionRules=
           "@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.GeoQuiz">
```

```
        <activity
            android:name=".MainActivity"
            android:exported="true">
            <intent-filter>
                <action android:
                    name="android.intent.action.MAIN" />
                <category android:
                    name="android.intent.category.LAUNCHER"/>
            </intent-filter>
        </activity>
        <activity
            android:name=".CheatActivity"
            android:exported="false"
            android:windowSoftInputMode="adjustResize" />
    </application>
</manifest>
```

ویژگی android:name مورد نیاز است و نقطه در ابتدای مقدار این ویژگی به سیستم عامل می‌گوید که کلاس این اکتیویتی در پکیج مشخص شده در ویژگی package در عنصر مانیفست در بالای فایل قرار دارد.

چیزهای جالب زیادی در مانیفست وجود دارد، اما در حال حاضر، بیایید روی راه اندازی و اجرای CheatActivity متمرکز بمانیم. در فصل‌های بعدی با بخش‌های مختلف مانیفست آشنا خواهید شد.

افزودن دکمه CHEAT به MainActivity

برنامه این است که کاربر دکمه‌ای را در MainActivity فشار دهد تا نمونه‌ای از CheatActivity را روی صفحه نمایش ببیند. بنابراین شما به دکمه‌های جدید در res/layout/activity_main.xml و res/layout-land/avtivity_main.xml نیاز دارید.

در شکل ۶.۲ می‌توانید دکمه CHEAT جدید که در بالای دکمه NEXT قرار دارد را مشاهده کنید. در لیوت پیش‌فرض دکمه جدید را به عنوان فرزند مستقیم LinearLayout ریشه درست قبل از تعریف دکمه NEXT تعریف کنید.

لیست ۶.۴ افزودن دکمه (res/layout/activity_main.xml)

```
...
</LinearLayout>
    <Button
        android:id="@+id/cheat_button"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="@string/cheat_button"
        android:layout_marginTop="24dp"/>
    <Button
        android:id="@+id/next_button"
        ...
        "/>
...
```

در لیوت افقی، دکمه جدید را در پایین و مرکز ریشه FrameLayout قرار دهید.

لیست ۶.۵ اضافه کردن دکمه به لیوت افقی (res/layout-land/activity_main.xml)

```
...
</LinearLayout>
<Button
    android:id="@+id/cheat_button"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_gravity="bottom|center_horizontal"
    android:text="@string/cheat_button" />

<Button
    android:id="@+id/next_button"
    ...
/>
...
```

اکنون در MainActivity.kt یک متغیر اضافه کنید، به دکمه ارجاع دهید و View.OnClickListener برای دکمه CHEAT را تنظیم کنید.

لیست ۶۶ اتصال دکمه CHEAT (MainActivity.kt)

```
class MainActivity : AppCompatActivity() {
    ...
    private lateinit var questionTextView: TextView
    private lateinit var cheatButton: Button
    override fun onCreate(savedInstanceState: Bundle?) {
    ...
        questionTextView =
            findViewById(R.id.question_text_view)
        cheatButton = findViewById(R.id.cheat_button)
        ...
        nextButton.setOnClickListener {
            quizViewModel.moveToNext()
            updateQuestion()
        }
        cheatButton.setOnClickListener {
        }
        updateQuestion()
        }
    }
```

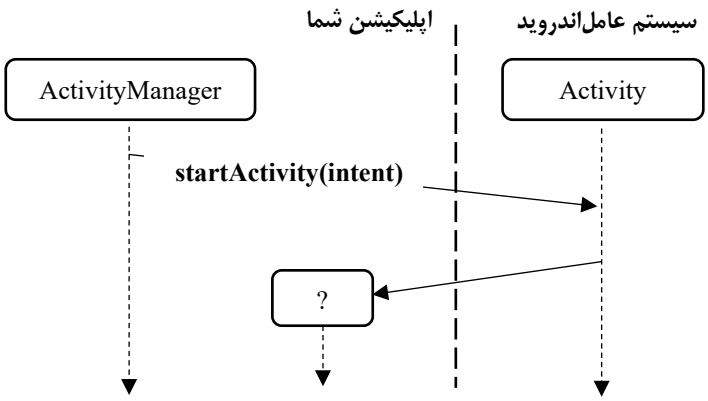
اکنون می‌توانیم به مسئله راه اندازی CheatActivity برسیم.

راه اندازی یک اکتیویتی

startActivity(Intent) ساده ترین راهی که یک اکتیویتی می‌تواند اکتیویتی دیگری را شروع کند، استفاده از تابع است.

شاید فکر کنید که startActivity(Intent) یک تابع استاتیک[1] است که در زیر کلاس Activity فراخوانی می‌کنیم. اما اینطور نیست. هنگامی که یک اکتیویتی startActivity(Intent) را فراخوانی می‌کند، این فراخوانی به سیستم عامل ارسال می‌شود. البته به بخشی از سیستم عامل به نام ActivityManager ارسال می‌شود. سپس ActivityManager نمونه Activity را ایجاد و تابع onCreate(Bundle?) آن را همانطور که در شکل ۶.۶ نشان داده شده فراخوانی می‌کند.

تصویر ۶.۶ شروع یک اکتیویتی



ActivityManager چگونه می‌داند که کدام اکتیویتی را شروع کند؟ این اطلاعات در پارامتر Intent قرار داده می‌شود.

برقراری ارتباط با Intent

Intent شیئی است که یک مؤلفه (component) می‌تواند از آن برای برقراری ارتباط با سیستم عامل استفاده کند. شما تنها مؤلفه‌هایی که تاکنون دیده‌اید اکتیویتی‌ها هستند.

[1] تابع استاتیک به تابعی گفته می‌شود که بدون نیاز به ساخت آبجکت از کلاس قابل فراخوانی است.

Intentها ابزارهای ارتباطی چند منظوره هستند و کلاس Intent، سازنده‌های مختلفی را بسته به کاری که قصد انجام آن را دارید، ارایه می‌دهد.

در مورد هدف ما – راه اندازی اکتیویتی دیگر – از یک Intent استفاده می‌کنیم تا به ActivityManager بگوییم کدام اکتیویتی را شروع کند. بنابراین از این سازنده استفاده خواهید کرد:

```
Intent(packageContext: Context, class: Class<?>)
```

در شنونده cheatButton، یک Intent ایجاد کنید که شامل کلاس CheatActivity باشد. سپس Intent را به تابع startActivity(Intent) منتقل کنید.

لیست ۶.۷ شروع CheatActivity (MainActivity.kt)

```
cheatButton.setOnClickListener {
    // Start CheatActivity
    val intent = Intent(this, CheatActivity::class.java)
    startActivity(intent)
}
```

آرگومان Class، کلاس اکتیویتی را مشخص می‌کند که ActivityManager باید شروع کند. آرگومان Context به ActivityManager می‌گوید که کلاس اکتیویتی را می‌توان در کدام پکیج برنامه یافت.

قبل از شروع اکتیویتی ActivityManager مانیفست پکیج را برای اعلامیه‌ای با همان نام کلاس مشخص شده بررسی می‌کند. اگر اعلامیه‌ای پیدا کند اکتیویتی را شروع می‌کند و همه چیز خوب پیش می‌رود. اگر اینطور نباشد یک ActivityNotFoundException تند و زننده دریافت می‌کنیم که برنامه ما را از کار می‌اندازد. به همین دلیل است که تمام اکتیویتی‌های شما باید در مانیفست اعلام شود.

GeoQuiz را اجرا کنید. CHEAT را فشار دهید تا نمونه‌ای از اکتیویتی جدید روی صفحه به نمایش درآید. حالا دکمه Back را فشار دهید. این کار شما را به MainActivity باز می‌گرداند.

Intent صریح (explicit) و ضمنی (implicit)

هنگامی که یک Intent با یک Context و یک شی Class ایجاد می‌کنیم، در واقع ما یک Intent صریح ایجاد کرده‌ایم. از Intent صریح برای شروع اکتیویتی‌ها در برنامه خود استفاده می‌کنیم.

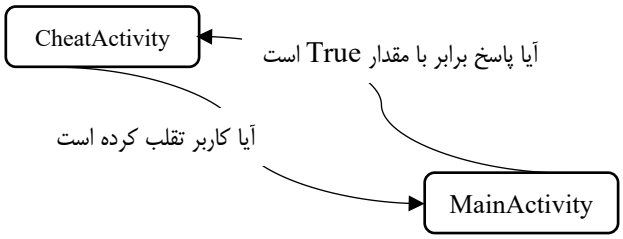
ممکن است عجیب به نظر برسد که دو اکتیویتی در برنامه ما باید از طریق ActivityManager که خارج از برنامه است ارتباط برقرار کنند. با این حال، این الگو کار با یک اکتیویتی در یک برنامه را آسان می‌کند.

هنگامی که اکتیویتی در برنامه شما می‌خواهد اکتیویتی را در برنامه دیگری شروع کند، از Intent ضمنی استفاده خواهد کرد.

انتقال داده بین اکتیویتی‌ها

اکنون که یک MainActivity و یک CheatActivity داریم، می‌توان به انتقال داده‌ها بین آنها فکر کرد. شکل ۶.۷ نشان می‌دهد که چه داده‌هایی را بین دو اکتیویتی منتقل خواهیم کرد.

شکل ۶.۷ مکالمه بین MainActivity و CheatActivity



وقتی MainActivity، CheatActivity را شروع می‌کند پاسخ سوال فعلی را به او اطلاع می‌دهد. هنگامی که کاربر دکمه بازگشت را برای برگشتن به MainActivity لمس می‌کند CheatActivity از بین می‌رود. اما در آخرین نفس خود داده‌هایی را در مورد اینکه آیا کاربر تقلب کرده است یا خیر به MainActivity ارسال می‌کند.

با انتقال داده از MainActivity به CheatActivity شروع خواهیم کرد.

استفاده از extraها

برای اطلاع رسانی پاسخ سوال فعلی، مقدار questionBank[currentIndex].answer را به آن منتقل خواهیم کرد. این مقدار را به عنوان یک extra در Intent ارسال خواهیم کرد.

Extraها داده‌های دلخواهی هستند که اکتیویتی فراخوانی کننده می‌تواند داخل intent منتقل کند. سیستم عامل intent را به همراه extraهای آن به اکتیویتی گیرنده ارسال می‌کند.

یک extra یک جفت کلید–مقدار است. مانند جفتی که برای ذخیره مقدار currentIndex در MainActivity.onSaveInstanceState(Bundle) استفاده کردیم.

برای افزودن extra به یک intent از Intent.putExtra(…) استفاده می‌کنیم. در مورد برنامه ما ساختار زیر به کار می‌رود.

```
putExtra(name: String, value: Boolean)
```

Intent.putExtra(…) انواع مختلفی دارد اما همیشه دو آرگومان می‌گیرد. آرگومان اول همیشه یک کلید رشته‌ای است و آرگومان دوم مقدار آن کلید با انواع متفاوت خواهد بود. در CheatActivity.kt یک کلید برای extra اضافه کنید.

لیست ۶.۸ اضافه کردن یک ثابت اضافی (CheatActivity.kt)

```
…
private const val EXTRA_ANSWER_IS_TRUE =
    "com.codeb.geoquiz.answer_is_true"
class CheatActivity: AppCompatActivity() {
…
}
```

یک اکتیویتی ممکن است از چندین مکان مختلف راه اندازی شود. بنابراین باید کلیدهایی را برای extraها در اکتیویتی‌هایی که آنها را استفاده می‌کنند، تعریف کنیم. استفاده از نام پکیج برای extraها همانطور که در فهرست ۶.۸ نشان داده شده است، گزینه مناسبی به نظر می‌رسد. زیرا از تداخل با extraها سایر برنامه‌ها جلوگیری می‌کند.

اکنون می‌توانیم به MainActivity برگشته و extra را روی intent قرار دهیم. اما رویکرد بهتری وجود دارد. هیچ دلیلی وجود ندارد که MainActivity یا هر کد دیگری در برنامه شما جزئیات پیاده‌سازی آنچه CheatActivity به عنوان extra در intent خود انتظار دارد، بداند. در عوض می‌توان آن کار را در تابع newIntent(…) کپسوله کنید.

اکنون این تابع را در CheatActivity ایجاد کنید. تابع را در داخل یک companion object قرار دهید. در کاتلین companion object بخشی از کلاس است که مثل عضوهای استاتیک عمل می‌کند. یعنی بدون ساختن نمونه‌ای از کلاس می‌توان به متدها و متغییرهای داخلش دسترسی داشت.

لیست ۶.۹ یک تابع newIntent(…) برای CheatActivity (CheatActivity.kt)

```
override fun onCreate(savedInstanceState: Bundle?) {
    ...
}
companion object {
    fun newIntent(packageContext: Context,
        answerIsTrue: Boolean): Intent {
        return Intent(packageContext,
        CheatActivity::class.java).apply {
            putExtra(EXTRA_ANSWER_IS_TRUE, answerIsTrue)
        }
    }
}
```

این تابع به شما امکان می‌دهد یک Intent ایجاد کنید که به درستی با extraها مورد نیاز CheatActivity تنظیم شده است. آرگومان answerIsTrue یک مقدار بولی بصورت private با استفاده از ثابت EXTRA_ANSWER_IS_TRUE در intent قرار می‌گیرد که بعدا آن بازیابی می‌کنید.

companion object به شما امکان می‌دهد بدون داشتن نمونه‌ای از کلاس، شبیه به توابع استاتیک در جاوا به توابع دسترسی داشته باشید.

اکنون از این تابع جدید در شنونده دکمه CHEAT استفاده کنید.

لیست ۶.۱۰ راه اندازی CheatActivity با extra (MainActivity.kt)

```
cheatButton.setOnClickListener {
    // Start CheatActivity
    val intent = Intent(this, CheatActivity::class.java)
    val answerIsTrue = quizViewModel.currentQuestionAnswer
    val intent = CheatActivity.newIntent(this@MainActivity
        , answerIsTrue)
    startActivity(intent)
}
```

در این مورد فقط به یک extra نیاز داریم. اما در صورت نیاز می‌توانید چندین extra را در یک Intent قرار دهید. برای بازیابی مقدار extra از Intent.getBooleanExtra(String, Boolean) استفاده خواهیم کرد. اولین آرگومان نام extra است. آرگومان دوم getBooleanExtra(…)یک پاسخ پیش‌فرض است که اگر کلید پیدا نشود، برگشت داده می‌شود.

در CheatActivity، extra را در onCreate(Bundle?) بازیابی و آن را در یک متغیر ذخیره کنید.

لیست ۶.۱۱ استفاده از extra (CheatActivity.kt)

```
class CheatActivity : AppCompatActivity() {
    private var answerIsTrue = false
    override fun onCreate(savedInstanceState: Bundle?) {
        ...
        answerIsTrue =
        intent.getBooleanExtra (EXTRA_ANSWER_IS_TRUE, false)
    }
}
```

توجه داشته باشید که Activity.getIntent() همیشه Intentی را که اکتیویتی را شروع کرده است برمی‌گرداند. این همان چیزی است که هنگام فراخوانی startActivity(Intent) ارسال کردید.

در نهایت TextView پاسخ و دکمه SHOW ANSWER را متصل کنید تا از مقدار بازیابی شده استفاده کنید.

لیست ۶.۱۲ فعال کردن CHEAT (CheatActivity.kt)

```
class CheatActivity : AppCompatActivity() {
    private var answerIsTrue = false
    private lateinit var answerTextView: TextView
    private lateinit var showAnswerButton: Button
    override fun onCreate(savedInstanceState: Bundle?) {
        ...
        answerIsTrue =
          intent.getBooleanExtra(EXTRA_ANSWER_IS_TRUE, false)
        answerTextView = findViewById(R.id.answer_text_view)
        showAnswerButton =
            findViewById(R.id.show_answer_button)
        showAnswerButton.setOnClickListener {
            val answerText = when {
                answerIsTrue -> R.string.true_button
                else -> R.string.false_button
            }
            answerTextView.setText(answerText)
        }
    }
}
```

این کد بسیار ساده است. متن TextView را با استفاده از TextView.setText(Int) تنظیم می‌کنیم. TextView.setText(…) پیاده‌سازی‌های مختلفی دارد. در اینجا از پیاده‌سازی استفاده می‌کنیم که شناسه منبع یک رشته را می‌پذیرد.

GeoQuiz را اجرا و CHEAT را برای راه اندازی CheatActivity لمس کنید. سپس SHOW ANSWER را فشار دهید تا پاسخ سوال فعلی نمایش داده شود.

دریافت نتیجه از اکتیویتی فرزند

در این مرحله کاربر می‌تواند بدون ترس از عواقب تقلب کند. بیایید این مشکل را با گزارش CheatActivity به MainActivity که آیا کاربر پاسخ را مشاهده کرده است یا خیر حل کنیم.

برای دریافت پاسخ از اکتیویتی فرزند از Luncher استفاده کنیم. لانچر (Launcher) در Activity Result API در واقع یک آبجکت از نوع ActivityResultLauncher<Intent> هست که مسئول شروع اکتیویتی دوم و دریافت نتیجه‌ی آن به صورت callback می‌باشد. callback تابعی که مستقیما فراخوانی نمی‌شود بلکه سیستم با توجه به رویدادی آن را اجرا می‌کند.

پس ابتدا یک لانچر در MainActivity تعریف و سپس با آن Intent را لانچ می‌کنیم. تغییرات زیر را در MainActivity بوجود بیاورید.

لیست ۶.۱۳ تغییرات (MainActivity.kt)

```
…
class MainActivity : AppCompatActivity() {
    private lateinit var trueButton: Button
    …
    private lateinit var launcher:
        ActivityResultLauncher<Intent>
    …
    override fun onCreate(savedInstanceState: Bundle?) {
        …
        nextButton.setOnClickListener {
            quizViewModel.moveToNext()
            updateQuestion()
        }
      launcher = registerForActivityResult(
            ActivityResultContracts.StartActivityForResult())
            { result ->
                if (result.resultCode == REQUEST_CODE_CHEAT){
                    val data: Intent? = result.data
                    quizViewModel.isCheater =
                        data?.getBooleanExtra
(EXTRA_ANSWER_SHOWN, false) ?: false
                }
            }
       cheatButton.setOnClickListener {
```

```
            // Start CheatActivity
            val answerIsTrue =
            quizViewModel.currentQuestionAnswer
            val intent =
                CheatActivity.newIntent(this@MainActivity
                , answerIsTrue)
            startActivity(intent)
            launcher.launch(intent)
        }
        updateQuestion()
    }
```

registerForActivityResult در واقع متدی است که اجازه می‌دهد یک لانچر ساخته و مشخص کنیم وقتی نتیجه برگشت چه کاری انجام شود. این متد جایگزین onActivityResult قدیمی شده است. ما اطلاعات را بیرون کشیده و در متغییر جدیدی به نام isCheater در quizViewModel ذخیره می‌کنیم، تا وضعیت تقلب کاربر همیشه حفظ شود. پس آن را به ویومدل اضافه نمایید.

```
class QuizViewModel: ViewModel() {
    ...
    var currentIndex = 0
    var isCheater = false
    ...
}
```

result.data حاوی اطلاعاتی است که اکتیویتی CheatActivity برا ما برگشت داده است. با استفاده از getBooleanExtra(…) نتیجه برگشتی که وضعیت تقلب کاربر است را بدست می‌آوریم.

برگشت نتیجه

با استفاده از دو تابع در اکتیویتی فرزند می‌توان نتیجه را به والد اعلام کرد:

```
setResult(resultCode: Int)
setResult(resultCode: Int, data: Intent)
```

ما علاقه‌مند به ارسال برخی از داده‌های خاص به MainActivity هستیم. بنابراین یک Intent با extra ایجاد و سپس Activity.setResult(Int, Intent) را فراخوانی می‌کنیم تا داده‌ها را به دست MainActivity برساند.

در CheatActivity یک ثابت برای کلید extra، یک ثابت برای کد درخواست و یک تابع private که این کار را انجام می‌دهد، اضافه کنید. سپس این تابع را در شنونده دکمه SHOW ANSWER فراخوانی کنید.

لیست ۶.۱۴ تنظیم نتیجه (CheatActivity.kt)

```
...
const val EXTRA_ANSWER_SHOWN =
    "com.codeb.geoquiz.answer_shown"
private const val REQUEST_CODE_CHEAT = 123
class CheatActivity : AppCompatActivity() {
    ...
    override fun onCreate(savedInstanceState: Bundle?) {
        ...
         setAnswerShownResult(true)
         }
     }
     private fun setAnswerShownResult
          (isAnswerShown: Boolean)
    {
        val data = Intent().apply {
            putExtra(EXTRA_ANSWER_SHOWN, isAnswerShown)
        }
        setResult(REQUEST_CODE_CHEAT, data)
    }
    ...
 }
```

هنگامی که کاربر دکمه SHOW ANSWER را فشار می‌دهد CheatActivity کد نتیجه و intent را در فراخوانی setResult(Int, Intent) قرار می‌دهد.

در نهایت تابع checkAnswer(Boolean) را در MainActivity برای بررسی این که آیا کاربر تقلب کرده است یا خیر و دادن پاسخ مناسب، تغییر دهید.

لیست ۶.۱۵ تغییر پیام Toast بر اساس مقدار isCheater (MainActivity.kt)

```
private fun checkAnswer(userAnswer: Boolean) {
    val correctAnswer = quizViewModel.currentQuestionAnswer
    val messageResId = if (userAnswer == correctAnswer) {
        R.string.correct_toast
    } else {
        R.string.incorrect_toast
    }
    val messageResId = when {
        quizViewModel.isCheater -> R.string.judgment_toast
        userAnswer == correctAnswer -> R.string.correct_toast
        else -> R.string.incorrect_toast
    }
    Toast.makeText(this, messageResId, Toast.LENGTH_SHORT)
```

```
        .show()
}
```

GeoQuiz را اجرا و CHEAT را لمس کنید. سپس SHOW ANSWER را در صفحه CHEAT فشار دهید. پس از تقلب دکمه Back را فشار دهید. به سوال فعلی پاسخ دهید. باید Toast تغییر داده شده ظاهر شود.

اگر به سوال بعدی بروید چه اتفاقی می‌افتد؟ هنوز برنامه شما را یک متقلب می‌بیند. اگر می‌خواهید قوانین خود را در مورد تقلب آسان بگیرید چالش مشخص شده در بخش چالش **ردیابی وضعیت تقلب بر اساس سوال** را امتحان کنید.

در این مرحله GeoQuiz از نظر فنی کامل شده است.

اندروید اکتیویتی‌های شما را چگونه می‌بیند

بیایید ببینیم در سطح سیستم‌عامل هنگام جابجایی بین اکتیویتی‌ها چه اتفاقی می‌افتد. ابتدا وقتی روی اپلیکیشن GeoQuiz در لانچر کلیک می‌کنید سیستم‌عامل خودِ برنامه را اجرا نمی‌کند بلکه یک اکتیویتی از برنامه را اجرا می‌کند. به‌طور دقیق‌تر اکتیویتی لانچر (launcher activity) برنامه را اجرا می‌کند. برای GeoQuiz، اکتیویتی لانچر همان MainActivity است.

وقتی ابزار ایجاد پروژه جدید اپلیکیشن GeoQuiz و اکتیویتی MainActivity را ساخت، به‌صورت پیش‌فرض MainActivity را به عنوان اکتیویتی لانچر تعیین کرد. وضعیت لانچر بودن اکتیویتی در فایل مانیفست مشخص می‌شود و این کار از طریق عنصر intent-filter در تعریف MainActivity انجام می‌گیرد (مطابق با لیست ۶.۱۶).

لیست ۶.۱۶ MainActivity لانچر (manifests/AndroidManifest.xml)

```
<?xml version="1.0" encoding="utf-8"?>
<manifest
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">
    <application
        android:allowBackup="true"
        android:dataExtractionRules=
            "@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
```

```
        android:theme="@style/Theme.GeoQuiz">
        <activity
            android:name=".MainActivity"
            android:exported="true">
            <intent-filter>
                <action android:name=
                    "android.intent.action.MAIN" />

                <category android:name=
                    "android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
        <activity
            android:name=".CheatActivity"
            android:exported="false"
            android:windowSoftInputMode="adjustResize" />
    </application>
</manifest>
```

پس از آن‌که نمونه‌ای از MainActivity روی صفحه ظاهر شد، کاربر می‌تواند دکمه‌ی CHEAT را فشار دهد. با این کار نمونه‌ای از CheatActivity در بالای MainActivity اجرا می‌شود. این اکتیویتی‌ها در یک پشته (stack) قرار می‌گیرند.

فشردن دکمه‌ی بازگشت در CheatActivity باعث می‌شود این نمونه از پشته خارج و MainActivity دوباره در بالای پشته همان‌طور قرار گیرد. فراخوانی متد ()Activity.finish در CheatActivity نیز باعث می‌شود CheatActivity از پشته خارج شود. اگر برنامه‌ی GeoQuiz را اجرا و از MainActivity دکمه‌ی بازگشت (Back) را فشار دهید MainActivity نیز از پشته خارج می‌شود و شما به آخرین صفحه‌ای که قبل از اجرای GeoQuiz مشاهده می‌کردید، باز می‌گردید.

اگر GeoQuiz را از برنامه لانچر شروع کرده باشید با لمس دکمه Back از MainActivity به لانچر باز خواهید گشت.

آنچه در اینجا مشاهده می‌کنید این است که ActivityManager یک پشته‌ی بازگشتی (back stack) را مدیریت می‌کند و این پشته فقط مختص اکتیویتی‌های برنامه‌ی شما نیست. اکتیویتی‌های همه‌ی برنامه‌ها این پشته را به صورت مشترک استفاده می‌کنند که یکی از دلایلی است که ActivityManager مسئول اجرای اکتیویتی‌هاست و در سطح سیستم‌عامل قرار دارد نه در خود برنامه. این پشته نمایانگر استفاده از سیستم‌عامل و دستگاه به‌صورت کلی و نه فقط استفاده از یک برنامه خاص است.

چالش: بستن راه‌های سوء استفاده برای متقلبان

متقلبان هرگز برنده نمی‌شوند. مگر اینکه با پشتکار، تدابیر ضد تقلب شما را دور بزنند – که احتمالاً همین کار را خواهند کرد. چون متقلب‌اند.

برنامه‌ی GeoQuiz یک خلأ بزرگ دارد: کاربران می‌توانند پس از تقلب CheatActivity را بچرخانند تا نتیجه‌ی تقلب پاک شود. وقتی به MainActivity برمی‌گردند انگار اصلاً تقلبی صورت نگرفته است.

برای رفع این باگ باید وضعیت رابط کاربری CheatActivity را در برابر چرخش صفحه و مرگ پردازش با استفاده از تکنیک‌هایی که در فصل ۴ یاد گرفتید، حفظ کنید.

چالش: پیگیری وضعیت تقلب به‌صورت سؤال‌به‌سؤال

در حال حاضر اگر کاربر فقط در یک سؤال تقلب کند به عنوان متقلب در تمام سؤال‌ها در نظر گرفته می‌شود. برنامه‌ی GeoQuiz را به‌گونه‌ای به‌روزرسانی کنید که وضعیت تقلب کاربر را برای هر سؤال به‌صورت جداگانه پیگیری کند. وقتی کاربر در یک سؤال خاص تقلب می‌کند، هر بار که تلاش کند به آن سؤال پاسخ دهد، پیامی‌همانند آنچه که ساختیم نمایش داده شود.

اگر کاربر به سؤالی پاسخ دهد که هنوز در آن تقلب نکرده است، پیام درست یا نادرست بودن پاسخ (correct/incorrect toast) را مطابق با پاسخ نمایش دهید.

۷

نسخه‌های Android SDK و سازگاری

اکنون که با تکمیل برنامه GeoQuiz قدم به دنیای توسعه برنامه‌های اندرویدی با کاتلین گذاشتید وقت آن است که مروری بر اطلاعات پایه‌ای مربوط به نسخه‌های مختلف اندروید داشته باشیم. اطلاعات این فصل برای ادامه‌ی مطالعه کتاب و توسعه اپلیکیشن‌های پیچیده و واقعی بسیار مهم و ضروری است.

نسخه‌های Android SDK

جدول ۷.۱ نسخه‌های SDK، نسخه‌های مرتبط سیستم عامل Android و درصد دستگاه‌هایی که آنها را اجرا می‌کنند را نشان می‌دهد (بروزرسانی ژوئن ۲۰۲۶).

جدول ۷.۱ نسخه‌های اندروید

نسخه اندروید	API	نام کد (Codename)	سهم استفاده
Android 16	36	Baklava	۲۰.۶٪
Android 15	35	VanillaIceCream	۱۷.۵٪
Android 14	34	UpsideDownCake	۱۴.۲٪
Android 13	33	Tiramisu	۱۳.۶٪
Android 12	31-32	SnowCone	۱۰.۹٪
Android 11	30	RedVelvetCake	۱۰.۰٪
Android 10	29	QuinceTart (Q)	۵.۳٪
Android 9	28	Pie (P)	۳.۸٪
Android 8	26-27	Oreo (O)	۱.۸٪
Android 7	24-25	Nougat (N)	۱.۶٪

درصد دستگاه‌هایی که از هر نسخه استفاده می‌کنند دائماً در حال تغییر است. اما این آمار یک روند مهم را نشان می‌دهد: دستگاه‌های اندرویدی که نسخه‌های قدیمی‌تری دارند، بلافاصله با عرضه نسخه جدید به‌روزرسانی یا جایگزین نمی‌شوند. تا سال ۲۰۲۶ تقریباً ۱۰ درصد از دستگاه‌ها هنوز از RedVelvetCake یا نسخه‌های قدیمی‌تر استفاده می‌کنند. (می‌توانید این داده‌ها را در سایت گوگل به صورت به روز مشاهده نمایید.)

چرا هنوز تعداد زیادی از دستگاه‌ها از نسخه‌های قدیمی اندروید استفاده می‌کنند؟ دلیل اصلی اینکه هنوز تعداد زیادی دستگاه روی نسخه‌های قدیمی اندروید هستند، ترکیبی از محدودیت سخت‌افزاری، سیاست‌های تجاری سازندگان و ساختار پراکنده اندروید است. نتیجه این می‌شود که بیش از ۴۰٪ گوشی‌ها روی Android 12 یا پایین‌تر مانده‌اند و دیگر آپدیت امنیتی کامل دریافت نمی‌کنند.

دلایل فنی و تجاری بسیاری پشت این موضوع قرار دارد، اما بیشتر این موضوع به رقابت شدید میان تولیدکنندگان دستگاه‌های اندرویدی مربوط می‌شود. تولیدکنندگان اغلب ترجیح می‌دهند منابع خود را صرف دستگاه‌های جدیدتر کنند تا اینکه دستگاه‌های قدیمی را به‌روز نگه دارند.

سازگاری و برنامه‌نویسی اندروید

تاخیر در ارتقاء همراه با ارایه نسخه‌های جدید سازگاری را به یک مسئله مهم در برنامه‌نویسی اندروید تبدیل می‌کند. برای دستیابی به یک بازار گسترده، توسعه دهندگان اندروید باید برنامه‌هایی ایجاد کنند که در دستگاه‌هایی که جدیدترین نسخه اندروید به علاوه سه تا چهار نسخه قبلی اندروید را اجرا می‌کنند و همچنین روی دستگاه‌های شرکت‌های مختلف عملکرد خوبی داشته باشند.

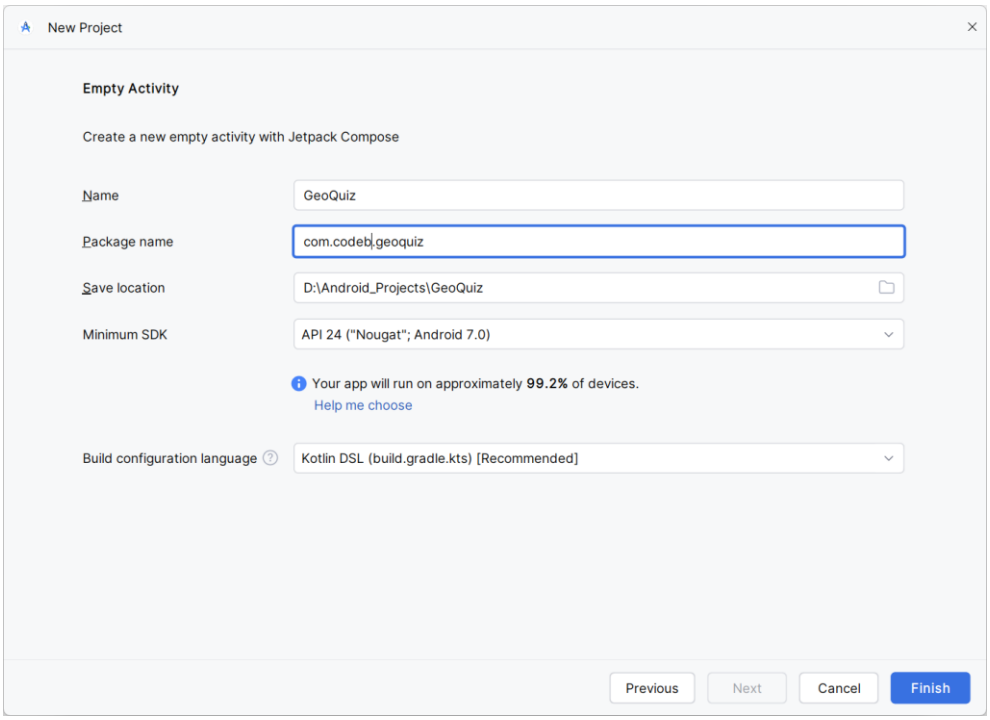
هدف قرار دادن اندازه‌های مختلف دستگاه‌ها آسان‌تر از آن چیزی است که فکر می‌کنید. صفحه نمایش گوشی‌ها اندازه‌های مختلفی دارند. اما سیستم لیوت اندروید در سازگاری با آن به خوبی عمل می‌کند. اگر نیاز به ارائه منابع سفارشی دارید یا لیوت بر اساس اندازه صفحه نمایش می‌خواهید، می‌توانید از توصیفگرهای پیکربندی برای انجام کار استفاده کنید. با این حال برای دستگاه‌های Android TV و Wear OS by Google (که هر دو نیز Android را اجرا می‌کنند) تفاوت در رابط کاربری به اندازه کافی محسوس است که باید در الگوهای تعامل کاربر و طراحی برنامه خود تجدید نظر کنید.

حداقل معقول

قدیمی‌ترین نسخه اندروید که تمرینات این کتاب از آن پشتیبانی می‌کند سطح API 24 (Nougat) است. ارجاعاتی به نسخه‌های قدیمی اندروید وجود دارد اما تمرکز بر روی نسخه‌های مدرن (سطح API 24+) است. با توجه به از دست دادن سهم دستگاه‌ها توسط نسخه‌های قدیمی‌تر پرداختن و پشتیبانی از آنها چندان قابل توجیه نیست.

هنگام ایجاد پروژه GeoQuiz حداقل نسخه SDK را در New Project wizard همانطور که در شکل ۷.۱ نشان داده شده است، تنظیم کردیم. (توجه داشته باشید که اندروید از اصطلاحات "نسخه SDK" و "سطح API" به جای یکدیگر استفاده می‌کند.)

شکل ۷.۱ مرا به خاطر دارید؟



علاوه بر حداقل نسخه پشتیبانی شده می‌توان نسخه هدف (target) و کامپایل را نیز تنظیم کرد. بیایید گزینه‌های پیش‌فرض را توضیح و چگونگی تغییر آنها را ببینیم.

همه این ویژگی‌ها در فایل build.gradle در ماژول برنامه شما تنظیم شده است. نسخه کامپایل منحصرا در این فایل قرار دارد. حداقل نسخه SDK و نسخه SDK هدف در فایل build.gradle تنظیم می‌شوند.

فایل build.gradle را در ماژول برنامه خود باز کنید. به مقادیر compileSdk، minSdk و targetSdk توجه کنید:

```
android {
    namespace = "com.codeb.geoquiz"
    compileSdk {
        version = 36
```

```
    }

    defaultConfig {
        applicationId = "com.codeb.geoquiz"
        minSdk = 24
        targetSdk = 36
        versionCode = 1
        versionName = "1.0"
    }
```

minSdk

مقدار minSdk حد پایین غیرقابل عبوری است که سیستم عامل اجازه نصب برنامه را به دستگاه‌هایی که نسخه‌ای پایین‌تر از آن را اجرا می‌کنند، نمی‌دهد.

با تنظیم این نسخه روی سطح API 24 به اندروید اجازه می‌دهیم GeoQuiz را روی دستگاه‌های دارای Nougat یا بالاتر نصب کند. اندروید از نصب GeoQuiz بر روی دستگاهی که نسخه کمتر از Nougat را دارد خودداری می‌کند.

با نگاهی مجدد به جدول ۷.۱ می‌توانید ببینید که چرا سطح API 24 انتخاب خوبی برای حداقل نسخه SDK است: این انتخاب اجازه اجرا شدن برنامه شما را تقریبا روی ۹۹ درصد از دستگاه‌ها می‌دهد.

targetSdk

مقدار targetSdk به اندروید می‌گوید که برنامه شما برای اجرا در کدام سطح API طراحی شده است. اغلب این آخرین نسخه اندروید خواهد بود.

چه زمانی SDK هدف را کاهش می‌دهیم؟ نسخه‌های جدید SDK می‌توانند نحوه نمایش برنامه شما در دستگاه یا حتی نحوه رفتار سیستم عامل در پشت صحنه را تغییر دهند. اگر قبلا برنامه‌ای را طراحی کرده‌اید، باید تأیید کنید که طبق انتظار در نسخه‌های جدید کار می‌کند. مستندات زیر را برای یافتن مشکلات احتمالی بررسی نمایید. می‌توانید برنامه خود را تغییر دهید تا با نسخه جدید کار کند یا SDK مورد نظر را کاهش دهد.

developer.android.com/reference/android/os/Build.VERSION_CODES.html

افزایش ندادن SDK هدف هنگام انتشار نسخه جدید اندروید تضمین می‌کند که برنامه شما همچنان با ظاهر و رفتار نسخه مورد نظر که روی آن به خوبی کار کرده است، اجرا می‌شود. این گزینه برای سازگاری با نسخه‌های جدیدتر اندروید وجود دارد. زیرا تغییرات در نسخه‌های بعدی تا زمانی که targetSdk افزایش

نیابد، نادیده گرفته می‌شود. اگر targetSdk پایین باشد، اَپ روی نسخه‌های جدید هم اجرا می‌شد، اما رفتارهای امنیتی و محدودیت‌های جدید اندروید بر روی آن اعمال نمی‌شود.

گوگل محدودیت‌هایی در مورد ارسال برنامه در فروشگاه Play دارد. یکی از این محدودیت‌ها نسخه SDK هدف است. این کار تضمین می‌کند که کاربران می‌توانند از بهبود عملکرد و امنیت در نسخه‌های اخیر اندروید بهره‌مند شوند. با انتشار نسخه‌های جدید اندروید این نسخه‌ها نیز باید فزایش یابند، بنابراین مستندات را زیر نظر داشته باشید تا بدانید چه زمانی باید نسخه مورد نظر خود را به روز کنید.

compileSdk

آخرین گزینه compileSdk است. compileSdk اطلاعات خصوصی بین شما و کامپایلر است.

ویژگی‌های اندروید از طریق کلاس‌ها و توابع موجود در SDK به ظهور می‌رسند. نسخه کامپایل SDK مشخص می‌کند که هنگام ساخت برنامه از کدام نسخه استفاده شود. هنگامی که اندروید استودیو به دنبال یافتن کلاس‌ها و توابعی است که در importهای خود به آنها مراجعه می‌کنید، نسخه کامپایل SDK تعیین می‌کند که کدام نسخه SDK مورد بررسی قرار خواهد گرفت.

بهترین انتخاب برای نسخه کامپایل SDK آخرین سطح API موجود است. با این حال در صورت نیاز می‌توانید نسخه کامپایل SDK یک برنامه موجود را تغییر دهید. به عنوان مثال ممکن است بخواهید نسخه کامپایل SDK را هنگام انتشار نسخه جدید اندروید به روز کنید تا بتوانید از توابع و کلاس‌های جدیدی که معرفی می‌کند، استفاده کنید.

می‌توانید حداقل نسخه SDK، نسخه SDK هدف و نسخه SDK کامپایل را در فایل build.gradle خود تغییر دهید. به یاد داشته باشید باید پروژه خود را با Gradle همگام سازی کنید.

افزودن ایمن کد از APIهای بعدی

تفاوت بین حداقل نسخه SDK و نسخه کامپایل در برنامه GeoQuiz شما را با یک شکاف سازگاری روبرو می‌کند. به عنوان مثال اگر کدی را از نسخه SDK فراخوانی کنید که بعد از SDK Nougat (سطح API 24) باشد چه اتفاقی می‌افتد؟ هنگامی که برنامه شما روی دستگاه Nougat نصب و اجرا می‌شود، از کار می‌افتد.

به لطف پیشرفت در Android Lint، مشکلات احتمالی ناشی از فراخوانی کدهای جدیدتر در دستگاه‌های قدیمی‌تر را می‌توان در زمان کامپایل تشخیص داد. اگر از کد نسخه‌ای بالاتر از حداقل SDK خود استفاده می‌کنید Lint خطاهای بیلد را گزارش می‌کند.

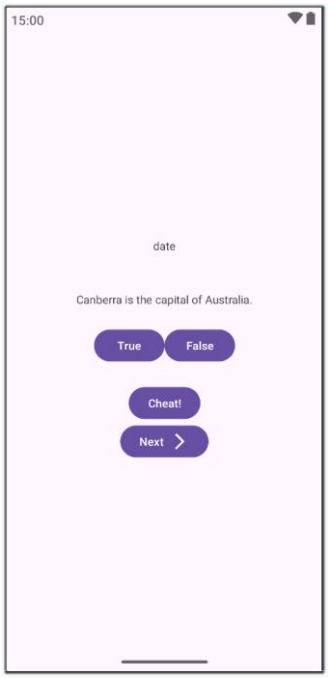
در حال حاضر تمام کدهای ساده GeoQuiz در سطح API 24 یا قبل از آن معرفی شده است. بیایید چند کد جدید بعد از API سطح ۲۴ اضافه کنیم و ببینیم چه اتفاقی می‌افتد. می‌خواهیم تاریخ جاری را در اول صفحه MainActivity به نمایش بگذاریم. برای اینکار یک TextView به activity_main.xml همانند لیست زیر اضافه نمایید.

لیست ۷.۱ اضافه کردن TextView

```
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"…>
    <TextView
        android:id="@+id/date_text_view"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:padding="24dp"
        tools:text="date"/>
    <TextView
        android:id="@+id/question_text_view"
        …
</LinearLayout>
```

لیوت شما باید همانند شکل زیر باشد.

شکل ۷.۲ لیوت شما



MainActivity.kt را باز کنید. در onCreate(…) برای ارجاع به TextView و نمایش تاریخ کد زیر را اضافه نمایید.

لیست ۷.۲ نمایش تاریخ (MainActivity.kt)

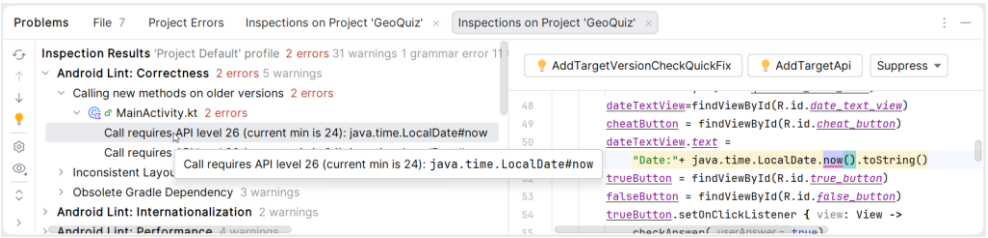
```
class MainActivity : AppCompatActivity() {
    …
    private lateinit var questionTextView: TextView
    private lateinit var dateTextView: TextView
    …
    override fun onCreate(savedInstanceState: Bundle?) {
        …
        questionTextView =
            findViewById(R.id.question_text_view)
        dateTextView=findViewById(R.id.date_text_view)
        cheatButton = findViewById(R.id.cheat_button)
        dateTextView.text =
            "Date:"+ java.time.LocalDate.now().toString()

    }
}
```

استفاده از کلاس java.time.LocalDate.now() برای بدست آوردن تاریخ خطایی در پی دارد.

اگر Lint را فراخوانی کنیم همانند شکل زیر خطایی نمایش می‌دهد.

شکل ۷.۳ لینت



این خطا می‌گویید که این تابع در سطح API 26 به Android SDK اضافه شده است. بنابراین برنامه اگر در دستگاهی که API 24 یا پایین تر را دارد اجرا شود، از کار می‌افتد.

از آنجایی که نسخه کامپایل SDK ما سطح API 36 است خود کامپایلر مشکلی با این کد ندارد. اما Lint در مورد حداقل نسخه SDK شما می‌داند. به همین خاطر گزارش می‌دهد. هنوز هم می‌توانیم کد را با این هشدار اجرا کنید (آن را امتحان کنید و ببینید) اما Lint می‌داند که اجرا به این روش اصلا ایمن نیست.

چگونه از شر این خطا خلاص شویم؟ یکی از گزینه‌ها این است که حداقل نسخه SDK را به ۲۶ افزایش دهیم. با این حال افزایش حداقل نسخه SDK واقعا به اندازه رفع آن با این مشکل سازگاری مقابله نمی‌کند. زیرا برنامه را نمی‌توان روی دستگاه‌های سطح API 24 و قدیمی‌تر نصب کرد. به همین خاطر کاربران زیادی را از دست می‌دهیم.

گزینه بهتر قرار دادن کد API بالاتر در یک عبارت شرطی که نسخه اندروید را بررسی می‌کند، است.

لیست ۷.۳ ابتدا نسخه اندروید دستگاه را بررسی کنید (MainActivity.kt)

```
override fun onCreate(savedInstanceState: Bundle?) {
    ...
    cheatButton = findViewById(R.id.cheat_button)
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
        dateTextView.text =
            "Date:"+ java.time.LocalDate.now().toString()
    }
    trueButton.setOnClickListener { view: View ->
        checkAnswer(true)
    }
...
}
```

ثابت Build.VERSION.SDK_INT نسخه دستگاه اندروید است. ما آن نسخه را با ثابتی مقایسه می‌کنیم که مخفف نسخه Oreo است.

اکنون کد ما تنها زمانی فراخوانی می‌شود که برنامه روی دستگاهی با سطح API 26 یا بالاتر اجرا شود. به این ترتیب کد خود را برای سطح API 24 ایمن کردیم و Lint اکنون باید راضی باشد. این تنها یک مثال ساده برای فهم موضوع بود. در دنیای واقعی این مفهوم به کمک شما خواهد آمد.

کتابخانه‌های Jetpack

محافظت از سطوح مختلف API در مثال بالا انجام شد. اما به چند دلیل بهینه به نظر نمی‌آید. اول اینکه شما به عنوان توسعه دهنده کار بیشتری برای انجام دارید. دلیل دوم این است که کاربران برنامه شما بر اساس نسخه‌ای که دستگاهشان آن را اجرا می‌کند، تجربه متفاوتی خواهند داشت.

کتابخانه‌های Jetpack مجموعه‌ای از ابزارها و کتابخانه‌های اندروید هستند که توسط گوگل ارائه می‌شوند تا توسعه اپلیکیشن‌ها ساده‌تر، سازگارتر و پایدارتر شود. این کتابخانه‌ها مستقل از نسخه سیستم‌عامل منتشر می‌شوند و به شما امکان می‌دهند بدون نگرانی از تفاوت نسخه‌های اندروید، از قابلیت‌های مدرن استفاده کنید. علاوه بر ارائه ویژگی‌های جدید (مانند ViewModel) کتابخانه‌های Jetpack سازگاری با گذشته را برای ویژگی‌های جدید در دستگاه‌های قدیمی و در صورت امکان در نسخه‌های اندروید رفتار ثابتی را ارائه می‌دهند (یا تلاش می‌کنند تا ارائه دهند). استفاده از کتابخانه‌ها به شما این امکان را می‌دهد که تا حد امکان شرط‌های کمتری برای بررسی‌های API بنویسید.

متأسفانه کتابخانه‌های Jetpack یک راه حل کامل برای عدم سازگاری نیستند. زیرا تمام ویژگی‌هایی که می‌خواهید در برنامه تان استفاده کنید در آنها وجود ندارند. تیم اندروید با گذشت زمان در افزودن APIهای جدید به کتابخانه‌های Jetpack بسیار خوب عمل کرده اما همچنان مواردی را خواهید یافت که در دسترس نیست. در این حالت باید از بررسی نسخه استفاده کنید.

استفاده از Android Developer Documentation

خطاهای Android Lint به شما می‌گوید که کد ناسازگار شما از چه سطح API است. اما همچنین می‌توانید در Android Developer Documentation دریابید که کلاس‌ها و توابع خاص به کدام سطح API تعلق دارند.

استفاده از این مستندات ایده خوبی است. در SDKهای اندروید موارد زیادی وجود دارد که نمی‌توانید در ذهن خود نگه دارید و با ظاهر شدن منظم نسخه‌های جدید باید گوش به زنگ باشید تا بدانید موارد جدید چیست و چگونه می‌توانید از آن استفاده کنید.

Android Developer Documentation یک منبع عالی و حجیم از اطلاعات است. صفحه اصلی مستندات developer.android.com است. این صفحه به شش بخش تقسیم می‌شود: Platform، Android Studio،Google Play، Android Jetpack، اسناد و اخبار. اگر فرصتی پیدا کردید، ارزش مطالعه را دارد. هر بخش جنبه‌های مختلف توسعه اندروید از ابتدا تا انتشار برنامه خود در فروشگاه Play را تشریح می‌کند.

- Platform: اطلاعات مربوط به پلت فرم اصلی
- اندروید استودیو: مقالاتی در مورد IDE برای کمک به یادگیری ابزارها
- گوگل پلی: نکات و ترفندهایی برای انتشار اپلیکیشن‌ها و همچنین بدست آوردن کاربران بیشتر، ارایه می‌دهد.
- Android Jetpack: اطلاعاتی در مورد کتابخانه‌های Jetpack و نحوه تلاش تیم اندروید برای بهبود تجربه توسعه برنامه
- اسناد: در اینجا گنجینه‌ای از آموزش‌ها و کدها خواهید یافت که می‌توانید برای بهبود و تقویت مهارت‌های خود از آنها بهره ببرید.
- اخبار: مقالات و پست‌هایی که شما را در جریان آخرین اتفاقات اندروید قرار می‌دهند.

همچنین می‌توانید اسناد را دانلود کنید تا به صورت آفلاین به آن دسترسی داشته باشید.

چالش: گزارش نسخه اندروید دستگاه

یک ویجت TextView به لیوت GeoQuiz اضافه کنید که به کاربر سطح API دستگاهی که برنامه را روی آن اجرا می‌کند، گزارش دهد. نمی‌توان متن این TextView را در لیوت تنظیم کرد زیرا نسخه اندروید دستگاه را تا زمان اجرا نمی‌دانیم.

از سایر ویژگی‌های XML فهرست شده در مرجع TextView برای تنظیم اندازه یا حروف متن استفاده کنید.

چالش: تقلب محدود

به کاربر اجازه دهید حداکثر سه بار تقلب کند. موارد تقلب کاربر را پیگیری کنید و تعداد تقلب‌های باقی مانده را در زیر دکمه CHEAT نمایش دهید. اگر هیچ فرصت تقلبی باقی نماند، دکمه CHEAT را غیرفعال کنید.

۸

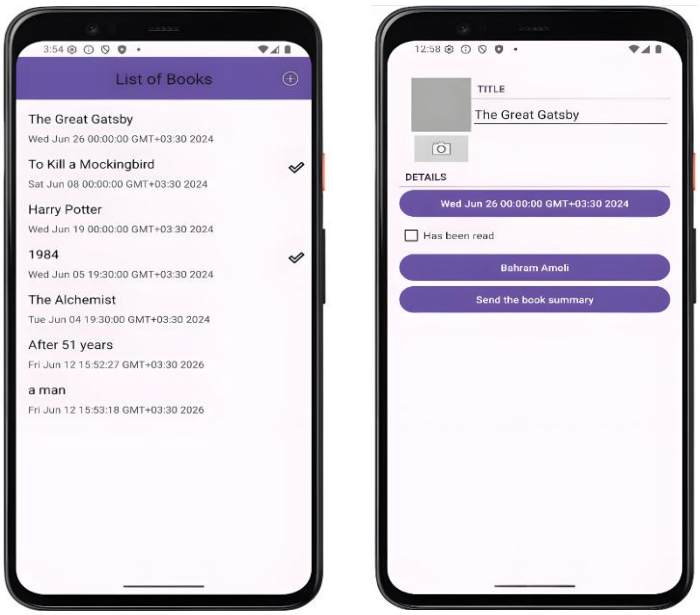
فرگمنت‌های رابط کاربری و مدیر فرگمنت

در این فصل شروع به ساخت اپلیکیشنی به نام BookShelf خواهیم کرد. BookShelf جزئیات "کتاب‌هایی که می‌خوانیم" را ثبت می‌کند – مواردی مانند نام، نویسنده، تاریخ شروع و ... .

با BookShelf می‌توان یک کتاب شامل عنوان، تاریخ و عکس ثبت کرد. همچنین می‌توانیم کتابی را به یکی از دوستان که نامش در لیست مخاطبین ما قرار دارد از طریق ایمیل، پیامک یا برنامه دیگری پیشنهاد دهیم.

BookShelf یک برنامه پیچیده است که تکمیل آن چندین فصل را در بر می‌گیرد. این برنامه یک رابط لیست–جزئیات (list-detail) خواهد داشت: صفحه اصلی فهرستی از کتاب‌ها را نمایش می‌دهد و کاربران می‌توانند کتاب جدیدی را اضافه کنند یا یک کتاب موجود را برای مشاهده و ویرایش جزئیات آن انتخاب کنند (تصویر ۸.۱).

تصویر ۸.۱ BookShelf



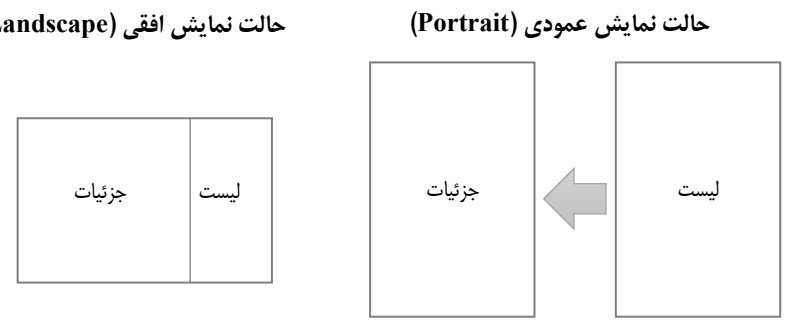
نیاز به انعطاف‌پذیری رابط کاربری

ممکن است تصور کنید که یک برنامه لیست–جزئیات از دو اکتیویتی تشکیل شده است: یکی مدیریت لیست و دیگری مدیریت ویوی جزئیات. لمس یک کتاب در لیست، نمونه‌ای از اکتیویتی جزئیات را آغاز می‌کند. فشار دادن دکمه بازگشت اکتیویتی جزئیات را از بین می‌برد و شما را به لیست باز می‌گرداند، جایی که می‌توانید کتاب دیگری را انتخاب کنید.

ایده درستی است و کار می‌کند. اما اگر بخواهیم ارائه و انتقال مفیدی بین صفحه نمایش‌ها داشته باشید، آن وقت چه؟

امکان اجرای BookShelf بر روی یک دستگاه بزرگ را در نظر بگیرید. برخی از دستگاه‌ها دارای صفحه نمایش به اندازه کافی بزرگ هستند که لیست و جزئیات را به طور همزمان نشان می‌دهند – حداقل در جهت افقی (شکل ۸.۲).

تصویر ۸.۲ رابط کاربری ایده‌آل برای عرض‌های مختلف صفحه نمایش



تصور کنید کاربری در حال مشاهده یک کتاب است و می‌خواهد کتاب بعدی را در لیست ببیند. بهتر است آنها بتوانند کتاب‌های متفاوتی را از لیست انتخاب کنند بدون اینکه ابتدا به صفحه قبلی برگردند.

وجه مشترک این سناریوها انعطاف‌پذیری رابط‌کاربری است: توانایی نوشتن و ترکیب مجدد ویوی یک اکتیویتی در زمان اجرا بر اساس آنچه کاربر یا دستگاه نیاز دارد.

اکتیویتی‌ها برای ارائه این مقدار از حجم انعطاف پذیری ساخته نشده‌اند. ویوهای یک اکتیویتی ممکن است در زمان اجرا تغییر کند اما کد کنترل ویوها باید در داخل اکتیویتی قرار گیرد. در نتیجه اکتیویتی‌ها به شدت با صفحه نمایش خاصی که استفاده می‌شود، مرتبط می‌شوند.

معرفی فرگمنت‌ها

می‌توان با انتقال مدیریت رابط کاربری برنامه از اکتیویتی به یک یا چند فرگمنت از انعطاف‌پذیری برنامه خود اطمینان حاصل کرد.

فرگمنت یک شی کنترل کننده است که یک اکتیویتی می‌تواند انجام قسمتی از وظایف را به آن محول کند. معمولا این کار مدیریت یک رابط کاربری است. رابط کاربری می‌تواند یک صفحه نمایش کامل یا فقط یک قسمت از صفحه باشد.

فرگمنتی که یک رابط کاربری را مدیریت می‌کند به عنوان فرگمنت رابط کاربری (UI fragment) شناخته می‌شود. یک فرگمنت رابط کاربری خاص خود را دارد که از یک فایل لیوت بارگذاری شده است.

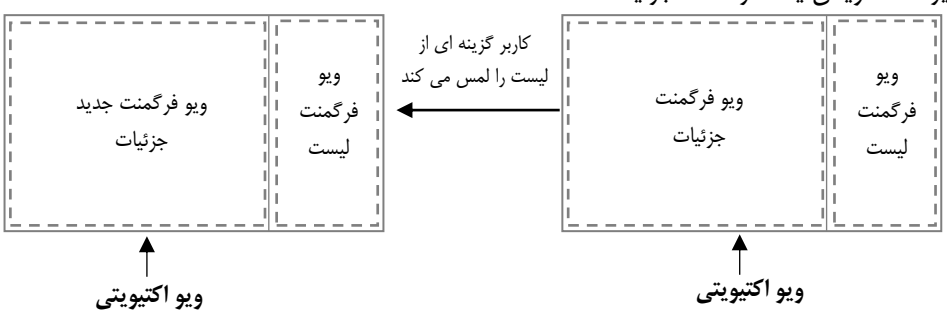
به جای رابط کاربری، ویوی اکتیویتی می‌تواند یک ظرف برای نگهداری فرگمنت‌ها باشد. ویوی فرگمنت پس از بارگذاری در این ظرف قرار می‌گیرد. در این فصل اکتیویتی میزبان یک فرگمنت واحد خواهد بود. اما یک اکتیویتی می‌تواند چندین محل در ویوی خود برای فرگمنت‌های مختلف داشته باشد.

می‌توانیم از فرگمنت (یا فرگمنت‌ها) مرتبط با اکتیویتی برای نوشتن و ترکیب مجدد صفحه نمایش طبق نیاز برنامه و کاربران استفاده کنیم. ویوی این اکتیویتی از نظر فنی در طول حیات خود ثابت می‌ماند و هیچ قانون اندرویدی نقض نمی‌شود.

بیایید ببینیم که در یک برنامه لیست–جزئیات این روش چگونه کار می‌کند. ما ویوی اکتیویتی را از یک فرگمنت لیست و یک فرگمنت جزئیات ترکیب می‌کنیم. ویوی جزئیات، شرح گزینه انتخاب شده را نشان می‌دهد.

انتخاب مورد دیگری باید ویوی جزئیات جدیدی را نمایش دهد. این کار با فرگمنت‌ها آسان است. اکتیویتی فرگمنت جزئیات را با فرگمنت جزئیات دیگری جایگزین می‌کند (شکل ۸.۳). برای این تغییر عمده در نمایش نیازی به مرگ اکتیویتی نیست.

تصویر ۸.۳ تعویض یک فرگمنت جزئیات

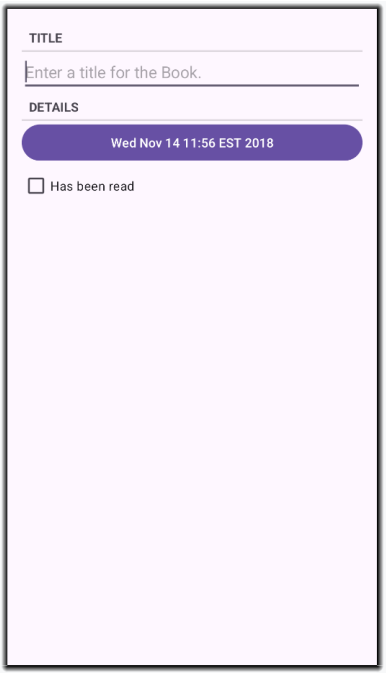


استفاده از فرگمنت‌های رابط کاربری، رابط کاربری برنامه را به بلوک‌ها تقسیم می‌کند که برای بسیاری از برنامه‌های کاربردی، فارغ از برنامه جزئیات–لیست مفید است. کار با این بلوک‌ها ساخت تب‌ها، سایدبارهای متحرک و موارد دیگر را آسان می‌کند. علاوه بر این برخی از APIهای جدید Android Jetpack مانند کنترلر پیمایش (navigationcontroller) با فرگمنت‌ها بهتر کار می‌کنند.

شروع BookShelf

در این فصل قصد داریم بخش جزئیات BookShelf را شروع کنیم. شکل ۸.۴ به شما نشان می‌دهد که BookShelf در پایان این فصل چگونه خواهد بود.

شکل ۸.۴ BookShelf در پایان این فصل



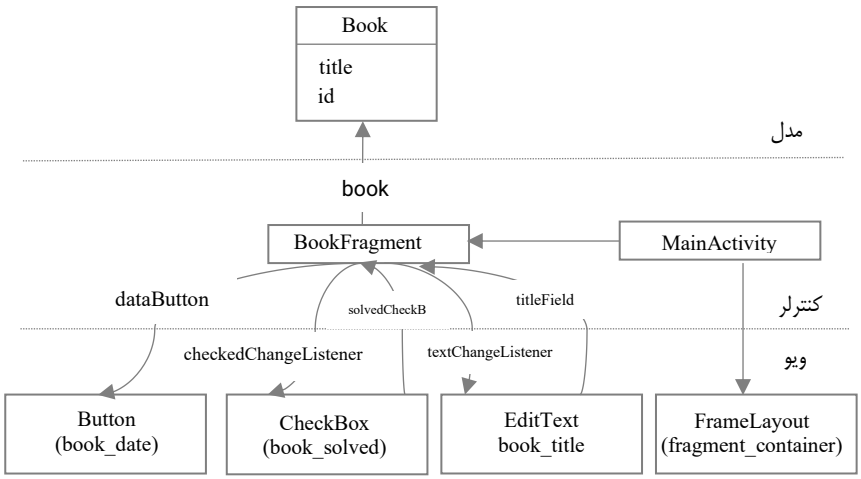
صفحه نمایش داده شده در شکل ۸.۴ توسط یک فرگمنت رابط کاربری به نام BookFragment مدیریت می‌شود. نمونه‌ای از BookFragment توسط اکتیویتی به نام MainActivity میزبانی می‌شود.

در حال حاضر اکتیویتی در نظر بگیرید که نقطه‌ای را در سلسله مراتب ویو خود فراهم می‌کند تا فرگمنت و ویوی آن را در خود جای دهد. یک فرگمنت به تنهایی قادر به دریافت ویو روی صفحه نمایش نیست. تنها زمانی که در سلسله مراتب یک اکتیویتی قرار گیرد می‌تواند ویو خود را به نمایش بگذارد.

BookShelf یک پروژه بزرگ خواهد بود و یکی از راه‌های نگه‌داری پروژه‌های پیچیده در ذهن، استفاده از نمودار است. شکل ۸.۵ تصویری از BookShelf را به شما نشان می‌دهد. لازم نیست این اشیا و روابط آنها را به خاطر بسپارید. اما خوب است قبل از شروع ایده‌ای از اینکه به کجا خواهید رسید، داشته باشید.

می‌توانید ببینید که BookFragment کاری را انجام می‌دهد که اکتیویتی شما در GeoQuiz انجام می‌داد: ایجاد و مدیریت رابط کاربری و تعامل با آبجکت‌های مدل.

شکل ۸.۵ نمودار شی برای BookShelf



سه کلاس نشان داده شده در شکل ۸.۵ کلاس‌هایی هستند که می‌نویسیم: Book، BookFragment و MainActivity.

نمونه‌ای از Book نشان دهنده یک کتاب واحد خواهد بود. در این فصل یک Book شامل عنوان، شناسه، تاریخ و یک مقدار از نوع Boolean که نشان دهنده وضعیت خوانده شدن کتاب است، خواهد بود. عنوان یک نام توصیفی است و شناسه به طور منحصر به فرد نمونه‌ای از کتاب را شناسایی می‌کند. برای این فصل همه چیز را بسیار ساده نگه می‌داریم و از یک نمونه Book استفاده می‌کنیم. BookFragment تنها دارای یک Book برای نمایش خواهد بود. ویوی MainActivity از یک FrameLayout تشکیل شده است که نقطه‌ای را که ویوی BookFragment در آن ظاهر می‌شود، مشخص می‌کند.

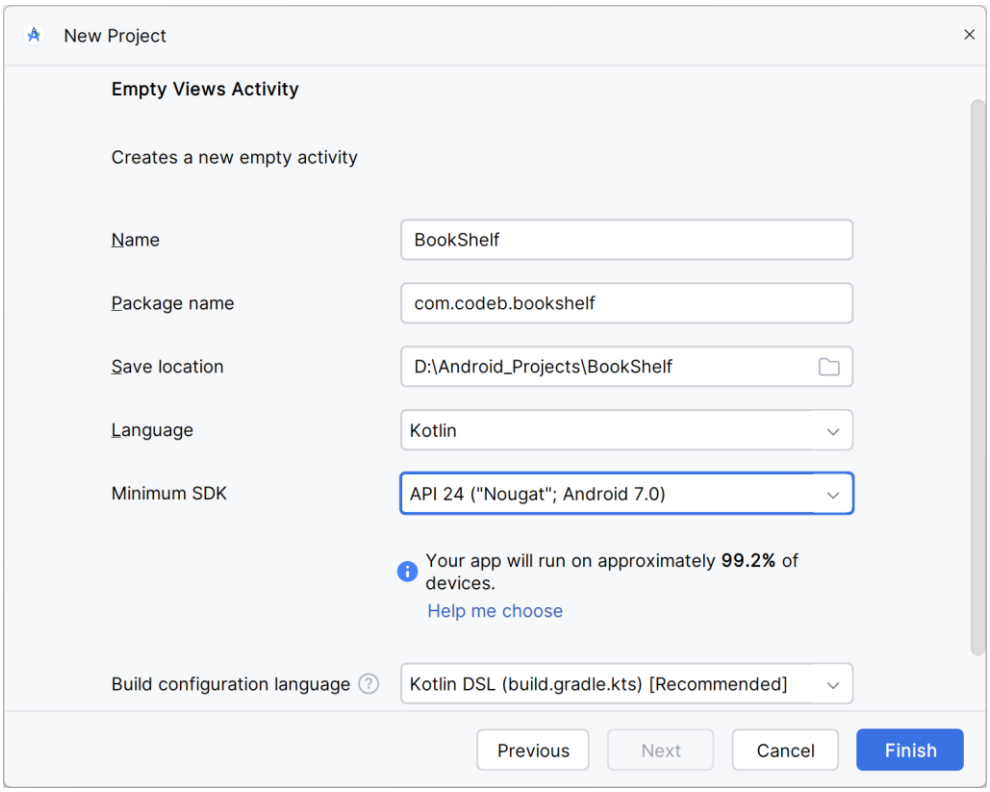
ویوی BookFragment شامل یک LinearLayout با چند ویوی فرزند در داخل آن از جمله EditText، Button و CheckBox خواهد بود. BookFragment برای هر یک از این ویوها یک متغییر خواهد داشت و شنوندها را روی آنها تنظیم می‌کند تا در صورت بروز شدن تغییرات، لایه مدل را به روز کنند.

ایجاد پروژه

صحبت کافی است؛ زمان ساخت یک اپلیکیشن جدید است. یک اپلیکیشن اندروید جدید ایجاد کنید. الگوی Empty View Activity را انتخاب و روی Next کلیک کنید.

پروژه خود را همانطور که در شکل ۸.۶ نشان داده شده است، پیکربندی کنید. برنامه را BookShelf نام گذاری و مطمئن شوید که نام پکیج com.codeb.bookshelf و زبان Kotlin است. API 24: Android 7.0 (Nougat) را از منوی کشویی حداقل سطح API انتخاب کنید و در نهایت Finish را بزنید.

تصویر ۸.۶ پیکربندی پروژه BookShelf



قبل از ادامه کار با MainActivity، لایه مدل را برای BookShelf با نوشتن کلاس Book ایجاد کنید.

ایجاد یک کلاس داده

در پنجره ابزار پروژه روی پکیج com.codeb.bookshelf کلیک راست کرده و New → Kotlin File/Class را انتخاب و فایلی با نام Book.kt و از نوع Dtat class ایجاد کنید.

در Book.kt فیلدهایی را برای نشان دادن شناسه، عنوان، تاریخ و وضعیت کتاب و سازنده‌ای که فیلدهای شناسه و تاریخ را مقداردهی اولیه می‌کند، اضافه کنید.

لیست ۸.۱ افزودن کلاس داده Book (Book.kt)

```
package com.codeb.bookshelf
import java.util.Date
import java.util.UUID
data class Book(val id: UUID = UUID.randomUUID,()
                var title: String,"" =
                var date: Date = Date,()
                var isSolved: Boolean = false)
```

هنگام import کردن Date چندین گزینه به شما ارائه می‌شود. مطمئن شوید که java.util.Date را import می‌کنید.

UUID یک کلاس کمکی گنجانده شده در فریم‌ورک اندروید است که امکان تولید ساده مقادیر شناسه منحصر به فرد را فراهم می‌کند. در سازنده با فراخوانی UUID.randomUUID() یک شناسه منحصر به فرد تصادفی ایجاد می‌کنیم.

مقداردهی اولیه متغیر Date با استفاده از سازنده پیش‌فرض Date، تاریخ را روی تاریخ فعلی تنظیم می‌کند. این تاریخ پیش‌فرض برای یک کتاب خواهد بود.

تمام چیزی که برای کلاس Book و لایه مدل BookShelf در این فصل نیاز دارید، تامین شد. در این مرحله شما لایه مدل و اکتیویتی را که قادر به میزبانی یک فرگمنت پشتیبانی است، ایجاد کرده‌اید. اکنون به جزئیات نحوه انجام وظایف اکتیویتی به عنوان یک میزبان می‌پردازیم.

ایجاد یک فرگمنت رابط کاربری

مراحل ایجاد یک فرگمنت رابط کاربری شامل مراحل زیر است:

- با تعریف ویجت‌ها در یک فایل لیوت یک رابط کاربری بسازید
- کلاس فرگمنت را ایجاد و view آن را لیوت تعریف شده در مرحله قبلی قرار دهید
- ویجت‌های لیوت را به کد متصل کنید

تعریف لیوت BookFragment

ویوی BookFragment اطلاعات موجود در یک نمونه از کتاب را نشان می‌دهد. ابتدا منابع رشته‌ای را در res/values/strings.xml تعریف کنید.

لیست ۸.۲ افزودن منابع رشته‌ای (Strings.xml)

```
<resources>
    <string name="app_name">BookShelf</string>
    <string name="book_title_hint">
        Enter a title for the Book.</string>
    <string name="book_title_label">Title</string>
    <string name="book_details_label">Details</string>
    <string name="book_solved_label">Has been read</string>
</resources>
```

در مرحله بعد رابط کاربری را تعریف خواهیم کرد. لیوت BookFragment از یک LinearLayout عمودی تشکیل شده است که شامل دو TextView، یک EditText، یک Button و یک CheckBox است.

برای ایجاد یک فایل لیوت روی پوشه res/layout در پنجره ابزار پروژه کلیک راست کرده New→Layout resource file را انتخاب کنید. نام این فایل را fragment_book.xml بگذارید و LinearLayout را به عنوان عنصر ریشه وارد کنید.

اندروید استودیو فایل را ایجاد و LinearLayout را برای شما اضافه می‌کند. ویجت‌هایی که لیوت فرگمنت را تشکیل می‌دهند به res/layout/fragment_book.xml اضافه کنید.

لیست ۸.۳ فایل لیوت برای ویوی فرگمنت (res/layout/fragment_book.xml)

```
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
```

```
    android:orientation="vertical"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:layout_margin="16dp"
    android:id="@+id/fragment_book">
    <TextView
        style="?android:listSeparatorTextViewStyle"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="@string/book_title_label"/>
    <EditText
        android:id="@+id/book_title"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="@string/book_title_hint"/>
    <TextView
        style="?android:listSeparatorTextViewStyle"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="@string/book_details_label"/>
    <Button
        android:id="@+id/book_date"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        tools:text="Wed Nov 14 11:56 EST 2018"/>
    <CheckBox
        android:id="@+id/book_solved"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="@string/book_solved_label"/>
</LinearLayout>
```

(برخی سینتکس‌های جدید مثل style: style=”? android:listSeparatorTextViewStyle” در تعریف وجود دارد. نترسید، معنای پشت این سینتکس را در بخشی به نام استایل‌ها، تم‌ها و ویژگی‌های تم در فصل مربوطه خواهید آموخت.)

به یاد بیاورید که فضای نام Tools به شما امکان ارائه اطلاعاتی را که پیش‌نمایش می‌تواند آن را به نمایش بگذارد، می‌دهد. متنی را به دکمه تاریخ اضافه می‌کنیم تا در پیش‌نمایش خالی دیده نشود. برای مشاهده پیش‌نمایش ویوی فرگمنت خود تب Design را بررسی کنید.

ایجاد کلاس BookFragment

یک فایل کاتلین دیگر برای کلاس BookFragment از نوع Class ایجاد کنید. تا اندروید استودیو تعریف کلاس را برای شما انجام دهد. کلاس را با ارث‌بری از Fragment به یک فرگمنت تبدیل کنید.

فهرست ۸.۴ ارث برای از Fragment (BookFragment.kt)

```
package com.codeb.bookshelf
import androidx.fragment.app.Fragment
class BookFragment : Fragment(){
}
```

با انتخاب کلاس Fragment به عنوان کلاس والد، اندروید استودیو دو کلاس با نام Fragment پیدا می‌کند: android.app.Fragment و androidx.fragment.app.Fragment. ما از نسخه Jetpack استفاده خواهیم کرد، بنابراین حتما androidx.fragment.app.Fragment را انتخاب کنید. (کتابخانه‌های Jetpack در پکیج‌هایی هستند که با androidx شروع می‌شوند.)

اگر این کادر را نمی‌بینید، روی نام کلاس Fragment کلیک کنید. اگر همچنان این کادر ظاهر نمی‌شود، می‌توانید به صورت دستی کلاس صحیح را import کنید: خط androidx.fragment.app.Fragment را در بالای فایل import کنید و اگر یک import برای android.app.Fragment دارید آن خط کد را حذف کنید.

پیاده‌سازی توابع چرخه حیات فرگمنت

BookFragment یک کنترل کننده است که با مدل تعامل دارد و آبجکت‌ها را مشاهده می‌کند. وظیفه آن ارائه جزئیات یک کتاب خاص و به روز رسانی آن جزئیات با توجه به انتخاب کاربر است.

در GeoQuiz اکتیویتی‌های شما بیشتر کار کنترلر خود را در توابع چرخه حیات اکتیویتی انجام می‌دادند. در BookShelf این کار توسط فرگمنت‌ها در توابع چرخه حیات فرگمنت انجام می‌شود. بسیاری از این توابع مانند onCreate(Bundle?) مربوط به توابع Activity هستند که قبلا با آن آشنا شده‌اید. (در مورد چرخه حیات فرگمنت در بخشی به نام FragmentManager و چرخه حیات فرگمنت در ادامه این فصل بیشتر خواهید آموخت.)

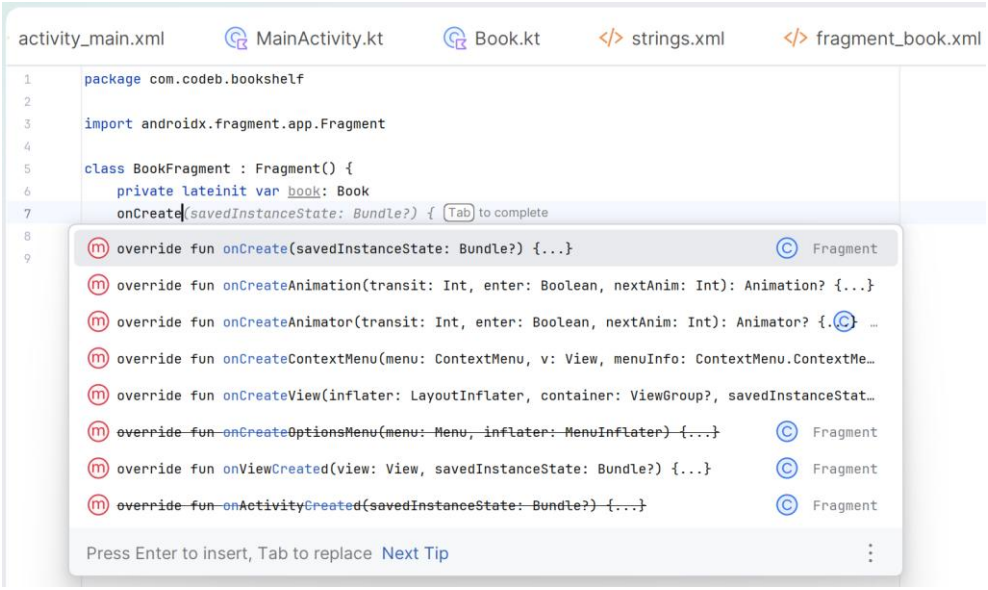
در BookFragment.kt یک عضو برای نمونه Book اضافه و Fragment.onCreate(Bundle?) را پیاده‌سازی کنید.

```
class BookFragment : Fragment() {
    private lateinit var book: Book
{
```

اندروید استودیو می‌تواند هنگام پیاده‌سازی توابع کمک کند.

وقتی شروع به تایپ نام onCreate(Bundle?) می‌کنید همانطور که در شکل ۸.۷ نشان داده شده است، اندروید استودیو فهرستی از پیشنهادات را ارائه می‌دهد. اگر آن را نمی‌بینید کلید Ctrl+Space را بزنید.

شکل ۸.۷ پیاده‌سازی تابع onCreate(Bundle?)



Enter را فشار دهید تا گزینه پیاده‌سازی تابع onCreate(Bundle?) انتخاب شود. سپس اندروید استودیو تابع را برای شما پیاده‌سازی می‌کند. کد خود را برای ایجاد یک Book مطابق با فهرست ۸.۵ به روز کنید.

لیست ۸.۵ پیاده‌سازی Fragment.onCreate(Bundle?) (BookFragment.kt)

```
class BookFragment : Fragment() {
    private lateinit var book: Book
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        book= Book()
    }
}
```

در این پیاده‌سازی باید به چند نکته توجه کرد. اول، Fragment.onCreate(Bundle?) یک تابع عمومی است. بر خلاف Activity.onCreate(Bundle?) که محافظت شده بود. توابع کاتلین اگر بدون توصیفگر دسترسی تعریف شوند، حالت دسترسی پیش‌فرض یعنی دسترسی عمومی برای آنها لحاظ می‌شود.

Fragment.onCreate(Bundle?) و سایر توابع چرخه حیات Fragment باید عمومی باشند زیرا توسط اکتیویتی که میزبان فرگمنت است، فراخوانی می‌شوند.

دوم، مشابه یک اکتیوتی، یک فرگمنت دارای یک bundle است که وضعیت خود را در آن ذخیره و بازیابی می‌کند. می‌توان Fragment.onSaveInstanceState(Bundle) را همانند برای اهداف خود بازنویسی کرد.

همچنین، توجه داشته باشید که چه اتفاقی در Fragment.onCreate(Bundle?) نمی‌افتد: ویوی فرگمنت را بارگذاری نمی‌کنیم. تنها نمونه فرگمنت را در Fragment.onCreate(Bundle?) پیکربندی می‌کنیم اما ویوی فرگمنت را در تابع چرخه حیات دیگری ایجاد و پیکربندی می‌شود:

```
onCreateView(LayoutInflater,ViewGroup?, Bundle?)
```

این تابع جایی است که لیوت فرگمنت را بارگذاری می‌کنیم و ویو را به اکتیویتی میزبانی بر می‌گردانیم. پارامترهای LayoutInflater و ViewGroup برای بارگذاری کردن لیوت ضروری هستند. Bundle حاوی داده‌هایی است که این تابع می‌تواند از آن برای ایجاد مجدد ویو از یک حالت ذخیره شده استفاده کند.

در BookFragment.kt پیاده‌سازی onCreateView(…) را اضافه کنید تا fragment_book.xml را بارگذاری کند. می‌توانیم از همان ترفند شکل ۸.۱۱ برای اعلان تابع استفاده کنیم.

لیست ۸.۶ onCreateView(...) (BookFragment.kt)

```
class BookFragment : Fragment() {
    private lateinit var book: Book
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        book= Book()
    }
    override fun onCreateView(
        inflater: LayoutInflater,
        container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View? {
        val view = inflater.inflate(R.layout.fragment_book,
            container, false)
        return view
```

```
        }
}
```

در onCreateView(…)، با ارسال شناسه منبع لیوت در LayoutInflater.inflate(…) ویوی فرگمنت را بارگذاری می‌کنیم. پارامتر دوم والد ویوی شما است که معمولا برای پیکربندی صحیح ویجت‌ها مورد نیاز است. پارامتر سوم به لیوت می‌گوید که آیا بلافاصله ویو را به ویو والد اضافه کند یا خیر. مقدار false را برای آن انتخاب کنید. زیرا ویوی فرگمنت در ویوی اکتیویتی میزبانی می‌شود. نیازی نیست که ویوی فرگمنت بلافاصله به ویوی والد اضافه شود. اکتیویتی بعدا افزودن ویو را انجام می‌دهد.

اتصال ویجت‌ها در یک فرگمنت

تابع (…)onCreateView مکانی برای اتصال ویجت‌ها است. با EditText شروع کنید. پس از بارگذاری ویو، با استفاده از findViewById به EditText ارجاع دهید.

لیست ۸.۷ سیم کشی ویجت EditText (BookFragment.kt)

```
class BookFragment : Fragment() {
    private lateinit var book: Book
    private lateinit var titleField: EditText
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        book= Book()
    }
    override fun onCreateView(
        inflater: LayoutInflater,
        container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View? {
        val view = inflater.inflate(R.layout.fragment_book
            , container, false)
        titleField = view.findViewById(R.id.book_title)
            as EditText
        return view
    }
}
```

دریافت ارجاع در Fragment.onCreateView(…) تقریبا مشابه Activity.onCreate(Bundle?) است. تنها تفاوت این است که شما View.findViewById(Int) را در ویوی فرگمنت فراخوانی می‌کنید.

پس از ارجاع، یک شنونده در پاسخ به فراخوانی چرخه حیات ()onStart به EditText اضافه کنید.

لیست ۸.۸ افزودن شنونده به ویجت EditText (BookFragment.kt)

```
class BookFragment : Fragment() {
    …
    override fun onCreate(savedInstanceState: Bundle?) {
        …
    }
    override fun onStart() {
        super.onStart()
        val titleWatcher = object : TextWatcher {
            override fun beforeTextChanged(
                sequence: CharSequence?,
                start: Int,
                count: Int,
                after: Int
```

```
            ) {
                // This space intentionally left blank
            }
            override fun onTextChanged(
                sequence: CharSequence?,
                start: Int,
                before: Int,
                count: Int
            ) {
                book.title = sequence.toString()
            }
            override fun afterTextChanged
                (sequence: Editable?) {
                // This one too
            }
        }
        titleField.addTextChangedListener(titleWatcher)
    }
}
```

تنظیم شنونده‌ها در یک فرگمنت دقیقا مانند یک اکتیویتی است. در اینجا ما یک کلاس ناشناس ایجاد می‌کنیم که رابط TextWatcher را پیاده‌سازی می‌کند. TextWatcher سه عملکرد دارد اما ما فقط به یکی اهمیت می‌دهیم: onTextChanged(…).

در onTextChanged() ما toString() را در CharSequence که ورودی کاربر است فراخوانی می‌کنیم. این تابع یک رشته را برمی‌گرداند که از آن برای عنوان کتاب استفاده می‌کنیم.

توجه داشته باشید که شنونده TextWatcher در onStart() قرار داده شده است. برخی از شنوندگان نه تنها زمانی که کاربر با آنها تعامل دارد بلکه زمانی که داده‌ها را دریافت می‌کنند یا زمانی که حالت ویو بازیابی می‌شود، مانند چرخش صفحه، فعال می‌شوند. شنوندگانی که به ورودی داده‌ها واکنش نشان می‌دهند، مانند TextWatcher برای EditText یا OnCheckChangedListener برای یک CheckBox نمونه‌ای از آن هستند.

شنوندگانی که فقط به تعامل کاربر واکنش نشان می‌دهند، مانند OnClickListener، از این نوع نیستند. زیرا تحت تأثیر ورود داده‌ها در یک ویو قرار نمی‌گیرند. به همین دلیل است که شما در GeoQuiz با این موضوع مواجه نشدید. وضعیت View پس از onCreateView(…) و قبل از onStart() بازیابی می‌شود. هنگامی که وضیت بازیابی می‌شود، محتویات EditText روی هر مقداری که در حال حاضر در book.title است تنظیم می‌شود. در این مرحله اگر قبلا یک شنونده در EditText تنظیم کرده‌اید ، توابع beforeTextChanged(...)، onTextChanged(…) و afterTextChanged(…) تکست‌واچر اجرا

خواهند شد. تنظیم شنونده در ()onStart از این رفتار جلوگیری می‌کند زیرا شنونده پس از بازیابی حالت view متصل می‌شود.

در مرحله بعد دکمه را وصل کنید تا تاریخ شروع کتاب نمایش داده شود.

لیست ۸.۹ تنظیم متن دکمه (BookFragment.kt)

```
class BookFragment : Fragment() {
    …
    private lateinit var titleField: EditText
    private lateinit var dateButton: Button
    override fun onCreate(savedInstanceState: Bundle?) {
        …
    }
    override fun onCreateView(
        inflater: LayoutInflater,
        container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View? {
        …
        titleField = view.findViewById(R.id.book_title)
            as EditText
        dateButton = view.findViewById(R.id.book_date)
            as Button
        dateButton.apply {
            text = book.date.toString()
            isEnabled = false
        }
        return view
    }
    override fun onStart() {
        …
    }
}
```

غیرفعال کردن دکمه تضمین می‌کند که به هیچ وجه به فشار دادن کاربر پاسخ نمی‌دهد. دکمه غیرفعال ظاهر خاصی به خود می‌گیرد تا غیرفعال بودنش مشخص شود.

نوبت CheckBox است. در (…)onCreateView به آن ارجاع دهید. سپس یک شنونده را در ()onStart تنظیم کنید که solvedCheckBox مربوط به book را همانطور که در لیست ۸.۱۰ نشان داده شده است، به روز کند.

لیست ۸.۱۰ گوش دادن به تغییرات CheckBox (BookFragment.kt)

```
class BookFragment : Fragment() {
    …
```

```
    private lateinit var dateButton: Button
    private lateinit var solvedCheckBox: CheckBox
    override fun onCreate(savedInstanceState: Bundle?) {
        …
        dateButton = view.findViewById(R.id.book_date)
            as Button
        dateButton.apply {
            text = book.date.toString()
            isEnabled = false
        }
        solvedCheckBox = view.findViewById(R.id.book_solved)
            as CheckBox
        return view
    }
    override fun onStart() {
        …
        titleField.addTextChangedListener(titleWatcher)
        solvedCheckBox.apply {
            setOnCheckedChangeListener { _, isChecked ->
                book.isSolved = isChecked
            }
        }
    }
}
```

اکنون کد شما برای BookFragment کامل شده است. اگر بتوانید BookShelf را اجرا و با کدی که نوشته‌اید کار کنید، عالی خواهد بود. اما این کار ممکن نیست زیرا فرگمنت‌ها نمی‌توانند ویوهای خود را به تنهایی روی صفحه نمایش قرار دهند. برای تحقق تلاش‌های خود ابتدا باید یک BookFragment به MainActivity اضافه کنید.

میزبانی یک فرگمنت رابط کاربری

برای میزبانی یک فرگمنت رابط کاربری یک اکتیویتی باید:

- نقطه‌ای در لیوت خود برای ویوی فرگمنت تعریف کند
- چرخه حیات نمونه فرگمنت را مدیریت کند

می‌توانیم با کد فرگمنت‌ها را به اکتیویتی‌های خود پیوست کنیم. تعیین کننده این که چه زمانی فرگمنت به اکتیویتی اضافه شود و پس از آن چه اتفاقی برای آن بیافتد، ما هستیم. می‌توانیم فرگمنت را حذف و یا آن را با فرگمنت دیگری جایگزین کنیم.

جزئیات کد بعدا در همین فصل ارائه خواهد شد. ابتدا می‌خواهیم لیوت MainActivity را تعریف کنیم.

تعریف یک ویوی نگهدارنده (container view)

برای اضافه کردن فرگمنت رابط کاربری در اکتیویتی میزبان باید یک محل برای ویوی فرگمنت در سلسله مراتب ویوی اکتیویتی ایجاد کنیم. لیوت MainActivity را در res/layout/activity_main.xml پیدا و باز کنید. سپس لیوت پیش‌فرض را با FrameLayout جایگزین کنید. XML شما باید با لیست ۸.۱۱ مطابقت داشته باشد.

لیست ۸.۱۱ ایجاد لیوت نگهدارنده فرگمنت (res/layout/activity_main.xml)

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout xmlns:an-
droid="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent">
    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Hello World!"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />
</androidx.constraintlayout.widget.ConstraintLayout>
<FrameLayout
xmlns:android="http://schemas.android.com/apk/res/android"
```

```
android:id="@+id/fragment_container"
android:layout_width="match_parent"
android:layout_height="match_parent"/>
```

FrameLayout ویوی نگهدارنده برای یک BookFragment خواهد بود. توجه داشته باشید که ویوی نگهدارنده عمومی است. می‌توانید از همین لیوت برای میزبانی فرگمنت‌ها دیگر استفاده کنید.

در حالی که activity_main.xml صرفا از یک ویوی نگهدارنده برای یک فرگمنت تشکیل شده است، لیوت یک اکتیویتی می‌تواند پیچیده‌تر باشد و چندین ویوی نگهدارنده و همچنین ویجت‌های خاص خود را تعریف کند.

حالا در MainActivity تغییر زیر را ایجاد نمایید.

```
class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContentView(R.layout.activity_main)
        ViewCompat.setOnApplyWindowInsetsListener
        (findViewById(R.id.~~main~~ fragment_container))
        { v, insets ->
            val systemBars = insets.getInsets
            (WindowInsetsCompat.Type.systemBars())
            v.setPadding(systemBars.left, systemBars.top
            , systemBars.right, systemBars.bottom)
            insets
        }
    }
}
```

BookShelf را اجرا و کد خود را بررسی کنید. یک FrameLayout خالی را خواهید دید. (شکل ۸.۸)

تصویر ۸.۸ یک FrameLayout خالی

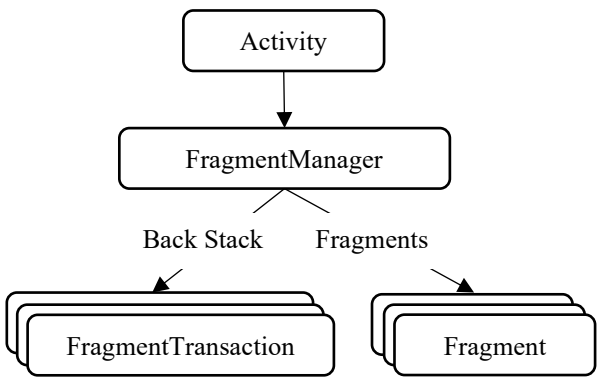


FrameLayout خالی است زیرا MainActivity هنوز یک فرگمنت را میزبانی نمی‌کند. بعدا کدی می‌نویسیم که ویوی یک فرگمنت را در داخل این FrameLayout قرار می‌دهد.

افزودن یک فرگمنت به FragmentManager

هنگامی که کلاس Fragment در Honeycomb معرفی شد، کلاس Activity به گونه‌ای تغییر کرد تا شامل عنصری به نام FragmentManager باشد. FragmentManager دو چیز را مدیریت می‌کند: یک لیست از فرگمنت‌ها و یک پشته از تراکنش‌های فرگمنت (شکل ۸.۹). این عنصر مسئول افزودن ویوهای فرگمنت‌ها به سلسله مراتب ویوی اکتیویتی و هدایت چرخه حیات فرگمنت‌ها است.

شکل ۸.۹ FragmentManager



برای BookShelf نگرانی ما فقط لیست فرگمنت‌ها FragmentManager خواهد بود.

تراکنش‌های فرگمنت

اکنون که FragmentManager را داریم، کد زیر را برای مدیریت فرگمنت به آن می‌دهیم. (بعدا این کد را مرور خواهیم کرد. فعلا فقط آن را وارد کنید.)

لیست ۸.۱۲ افزودن BookFragment (MainActivity)

```
class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        ...
        val currentFragment =
            supportFragmentManager.findFragmentById
(R.id.fragment_container)
        if (currentFragment == null) {
            val fragment = BookFragment()
            supportFragmentManager
                .beginTransaction()
```

```
                .add(R.id.fragment_container, fragment)
                .commit()
        }
    }
}
```

برای افزودن یک فرگمنت به یک اکتیویتی در کد، فراخوانی صریح FragmentManager را انجام می‌دهیم. با استفاده از ویژگی supportFragmentManager می‌توانیم به مدیر فرگمنت اکتیویتی دسترسی داشته باشید. استفاده ما از supportFragmentManager بخاطر استفاده از کتابخانه Jetpack و کلاس AppCompatActivity است.

برای درک بقیه کدی که به تازگی اضافه کرده‌اید (...)add را پیدا کنید و کدهای حوالی آن را بررسی کنید. این کد یک تراکنش فرگمنت ایجاد و انجام می‌دهد.

```
if (currentFragment == null) {
    val fragment = BookFragment()
    supportFragmentManager
        .beginTransaction()
        .add(R.id.fragment_container, fragment)
        .commit()
}
```

تراکنش‌های فرگمنت برای افزودن، حذف، پیوست، جدا کردن یا جایگزینی فرگمنت‌ها در لیست فرگمنت‌ها استفاده می‌شود. به شما امکان‌گروه بندی چندین عملیات مانند افزودن چندین فرگمنت به نگهدارنده‌های مختلف به طور همزمان با هم را می‌دهد. آنها قلب چگونگی استفاده از فرگمنت‌ها برای نوشتن و ترکیب مجدد صفحه نمایش در زمان اجرا هستند.

FragmentManager یک پشته از تراکنش‌های فرگمنت را حفظ می‌کند که می‌توانیم آن را پیمایش کنیم. هنگامی که عملیات فرگمنت خود را در یک تراکنش واحد گروه‌بندی می‌کنیم، کنترل بیشتری بر وضعیت رابط کاربری فراهم می‌شود.

کد های‌لایت شده در بالا می‌گوید: "یک تراکنش فرگمنت جدید ایجاد و یک عملیات add را در آن بگنجان و سپس آن را اعمال کن."

تابع (...)Add هسته اصلی تراکنش، دارای دو پارامتر است: شناسه ویوی نگهدارنده و BookFragment تازه ایجاد شده. شناسه ویوی نگهدارنده برای شما آشنا به نظر می‌رسد. این شناسه منبع FrameLayout در activity_main.xml است.

شناسه ویوی نگهدارنده دو هدف را دنبال می‌کند:

- به FragmentManager محل ویوی فرگمنت در ویوی اکتیویتی را نشان می‌دهد.
- به عنوان یک شناسه منحصر به فرد برای یک فرگمنت در لیست FragmentManager استفاده می‌شود.

هنگامی که نیاز به بازیابی BookFragment از FragmentManager دارید، آن را با شناسه ویوی نگهدارنده درخواست می‌کنید:

```
val currentFragment =
    supportFragmentManager.findFragmentById
        (R.id.fragment_container)
if (currentFragment == null) {
    val fragment = BookFragment()
    supportFragmentManager
        .beginTransaction()
        .add(R.id.fragment_container, fragment)
        .commit()
}
```

ممکن است عجیب به نظر برسد که FragmentManager مربوطه، BookFragment را با استفاده از شناسه یک FrameLayout شناسایی می‌کند. اما شناسایی یک فرگمنت رابط کاربری توسط شناسه منبع ویوی نگهدارنده آن در نحوه عملکرد FragmentManager تعبیه شده است. اگر چندین فرگمنت را به یک اکتیویتی اضافه کنیم، معمولا نگهدارنده‌های جداگانه‌ای با شناسه‌های جداگانه برای هر یک از آن فرگمنت‌ها در نظر می‌گیریم.

اکنون می‌توانیم کدی را که در لیست ۸.۱۲ اضافه کرده‌ایم تماما تشریح کنیم:

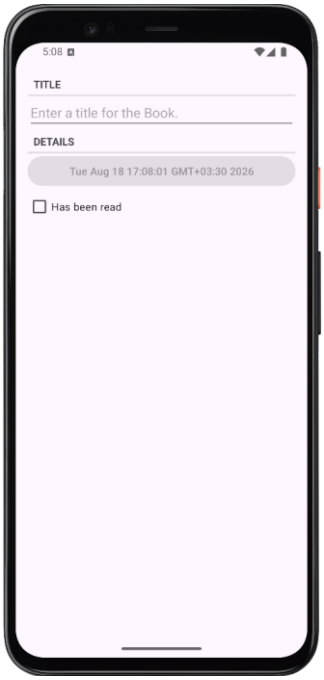
ابتدا از FragmentManager فرگمنت‌ای را با شناسه ویوی نگهدارنده R.id.fragment_container درخواست می‌کنیم. اگر این فرگمنت از قبل در لیست باشد FragmentManager آن را برمی‌گرداند.

چرا یک فرگمنت از قبل در لیست وجود دارد؟ فراخوانی به MainActivity.onCreate(Bundle?) می‌تواند در پاسخ به ایجاد مجدد MainActivity پس از از بین رفتن در چرخش یا بازیابی حافظه باشد. هنگامی که یک اکتیویتی از بین می‌رود FragmentManager آن، لیست فرگمنت‌ها آن را ذخیره می‌کند و هنگامی که اکتیویتی دوباره ایجاد می‌شود FragmentManager جدید لیست را بازیابی می‌کند و فرگمنت‌ها فهرست شده را دوباره ایجاد می‌کند تا همه چیز مانند قبل باشد.

از سوی دیگر اگر فرگمنت‌ای با شناسه ویوی نگهدارنده داده شده وجود نداشته باشد، فرگمنت تهی خواهد بود. در این حالت ما یک BookFragment جدید و یک تراکنش فرگمنت جدید ایجاد می‌کنیم که فرگمنت را به لیست اضافه می‌کند.

MainActivity اکنون میزبان یک BookFragment است. برای اثبات این موضوع BookShelf را اجرا کنید. همانطور که در شکل ۸.۱۰ نشان داده شده است، باید ویوی تعریف شده در fragment_Book.xml را مشاهده کنید.

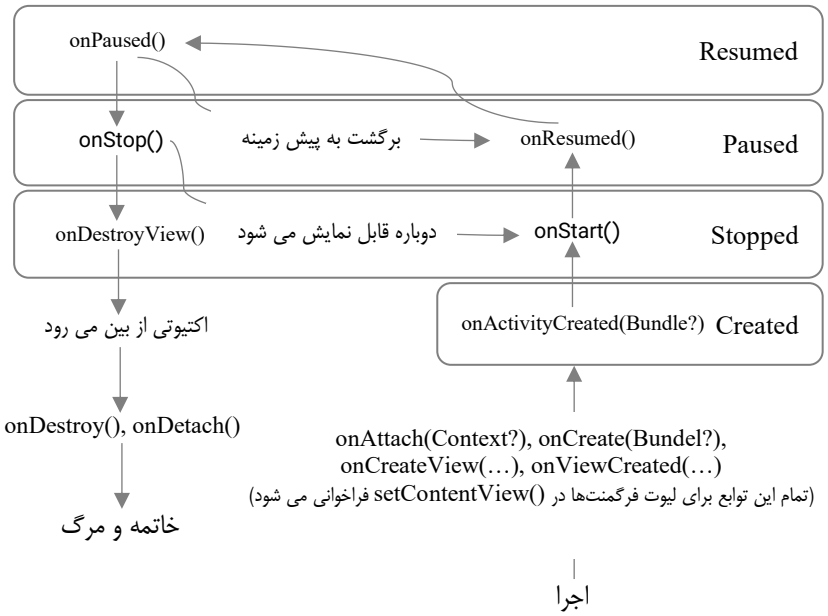
شکل ۸.۱۰ ویوی BookFragment به میزبانی MainActivity



FragmentManager و چرخه حیات فرگمنت

شکل ۸.۱۱ چرخه حیات فرگمنت را نشان می‌دهد. چرخه حیات فرگمنت شبیه به چرخه حیات اکتیویتی است: دارای حالت‌ها stopped، paused و resumed است و دارای توابعی است که می‌توان بازنویسی کرد.

شکل ۸.۱۱ نمودار چرخه حیات فرگمنت



از آنجا که یک فرگمنت به نمایندگی از یک اکتیویتی کار می‌کند، وضعیت آن باید وضعیت اکتیویتی را منعکس کند.

یکی از تفاوت‌های اساسی بین چرخه حیات فرگمنت و چرخه حیات اکتیویتی فراخوانی توابع چرخه حیات فرگمنت توسط FragmentManager اکتیویتی میزبان و نه سیستم عامل است. سیستم عامل چیزی در مورد فرگمنت‌های که یک اکتیویتی برای مدیریت امورش استفاده می‌کند، نمی‌داند.

فرگمنت‌ها مربوط به امور داخلی اکتیویتی هستند. توابع onAttach(Context?)، onCreate(Bundle?)، onCreateView(…) و onViewCreated(…) زمانی فراخوانی می‌شوند که فرگمنت را به FragmentManager اضافه می‌کنید.

تابع onActivityCreated(Bundle?) پس از onCreate(Bundle?) اکتیویتی میزبانی فراخوانی می‌شود. ما در حال اضافه کردن BookFragment در MainActivity.onCreate(Bundle?) هستیم، بنابراین این تابع پس از اضافه شدن فرگمنت فراخوانی می‌شود.

اگر یک فرگمنت را در حالی که اکتیویتی resumed است اضافه کنیم چه اتفاقی می‌افتد؟ در این صورت FragmentManager بلافاصله فرگمنت را از طریق هر مرحله‌ای که لازم است تا به وضعیت اکتیویتی برسد، عبور می‌دهد. به عنوان مثال هنگامی که یک فرگمنت به اکتیویتی که قبلا resumed است اضافه می‌شود، توابع onAttach(…)، onCreate(…)، onCreateView(...)، onViewCreated(...)، onActivityCreated(…)، onStart() و سپس onResume() فراخوانی می‌شوند.

هنگامی که وضعیت فرگمنت به وضعیت اکتیویتی رسید FragmentManager اکتیویتی میزبان تقریبا در همان زمانی که فراخوانی مربوطه را از سیستم عامل دریافت می‌کند، توابع چرخه حیات بیشتری را برای هماهنگ نگه داشتن وضعیت فرگمنت با وضعیت اکتیویتی فراخوانی می‌کند.

معماری اپلیکیشن با فرگمنت‌ها

طراحی برنامه با فرگمنت‌ها به روش صحیح بسیار مهم است. بسیاری از توسعه دهندگان پس از اولین یادگیری در مورد فرگمنت‌ها تلاش می‌کنند برای هر جزء برنامه خود از فرگمنت‌ها استفاده کنند. این روشی اشتباه برای استفاده از فرگمنت‌ها است.

فرگمنت‌ها برای کپسوله کردن اجزای اصلی برنامه به روشی که بتوان از آنها به تکرار استفاده کرد در نظر گرفته شده‌اند. یک جزء اصلی در مورد برنامه ما می‌تواند کل صفحه نمایش باشد. اگر تعداد قابل توجهی فرگمنت را به طور همزمان روی صفحه داشته باشید کد شما مملو از تراکنش‌های فرگمنت و امور کنترل نشده، خواهد بود. یک راه حل بهتر برای استفاده مجدد اجزای کوچکتر ساخت ویوی سفارشی است (کلاسی که از View یا یکی از زیر کلاس‌های آن ارث‌بری دارد).

به درستی از فرگمنت‌ها استفاده کنید. یک قاعده کلی خوب این است که بیش از دو یا سه فرگمنت روی صفحه نمایش در یک زمان نداشته باشید.

تصمیم‌گیری در مورد استفاده از فرگمنت‌ها

فرگمنت‌ها موضوع بحث‌های زیادی در جامعه اندروید هستند. برخی از افراد با پیچیدگی اضافی مربوط به فرگمنت‌ها و چرخه حیات آنها بسیار مخالف هستند. بنابراین هرگز از آنها در پروژه‌های خود استفاده نمی‌کنند. مشکل این رویکرد این است که چندین API در اندروید وجود دارد که به فرگمنت‌ها متکی هستند، مانند ViewPager و کتابخانه Jetpack Navigation. اگر نیاز به استفاده از این گزینه‌ها در برنامه خود دارید، باید از فرگمنت‌ها استفاده کنید تا از مزایای آنها بهره‌مند شوید.

اگر به APIهای وابسته به فرگمنت نیاز ندارید، پیچیدگی ایجاد شده توسط فرگمنت‌ها و چرخه حیات آنها اغلب غیر ضروری است.

نکته‌ای که هنگام شروع یک برنامه جدید باید در نظر بگیرید این است که افزودن فرگمنت‌ها در آینده می‌تواند پیچده و پر ریسک باشد. تغییر یک اکتیویتی به اکتیویتی که میزبان یک فرگمنت رابط کاربری است دشوار نیست، اما انبوهی از نکات آزار دهنده وجود دارد. نگه داشتن مدیریت برخی از رابط‌ها توسط اکتیویتی‌ها و برخی دیگر توسط فرگمنت‌ها فقط اوضاع را بدتر می‌کند. نوشتن کد با استفاده از فرگمنت‌ها از ابتدا بسیار آسان‌تر است.

بنابراین وقتی صحبت از فرگمنت‌ها به میان می‌آید ما اصل متفاوتی داریم: تقریبا همیشه از فرگمنت‌ها استفاده کنید. اما اگر برنامه‌ای که می‌سازید بسیار کوچک باقی می‌ماند، تلاش اضافی برای استفاده از فرگمنت‌ها ممکن است ارزشش را نداشته باشد. بنابراین می‌توان آنها را کنار گذاشت. برای برنامه‌های بزرگتر، پیچیدگی با انعطاف‌پذیری فرگمنت‌ها جبران می‌شود و توجیه داشتن آنها در پروژه را آسان‌تر می‌کند.

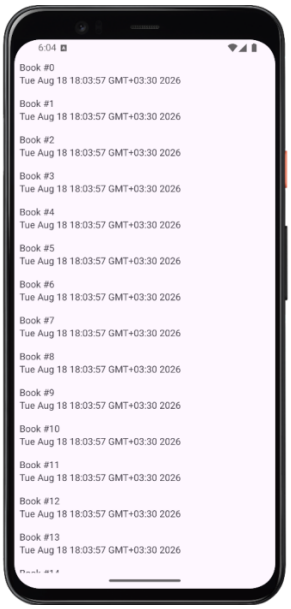
از اینجا به بعد برخی از برنامه‌های این کتاب از فرگمنت‌ها استفاده می‌کنند و برخی دیگر از آن استفاده نمی‌کنند. ما در طول یک فصل چند اپلیکیشن کوچک خواهیم ساخت و با استفاده نکردن از فرگمنت‌ها، ساخت این اپلیکیشن‌ها را آسان‌تر می‌کنیم. با این حال برنامه‌های بزرگتر در این کتاب از فرگمنت‌ها استفاده می‌کنند تا تجربه بیشتری در استفاده از آنها به شما بدهند و گسترش برنامه‌ها را در آینده آسان‌تر کنند.

۹

نمایش لیست‌ها با RecyclerView

لایه مدل BookShelf از یک نمونه واحد از Book تشکیل شده است. در این فصل BookShelf را برای کار با لیستی از کتاب‌ها به روز خواهیم کرد. این لیست عنوان و تاریخ شروع هر کتاب را همانطور که در شکل ۹.۱ نشان داده شده است، نشان می‌دهد.

شکل ۹.۱ فهرستی از Book



در لایه کنترلر یک ViewModel جدید که داده‌های صفحه نمایش جدید را کپسوله می‌کند، خواهیم داشت. BookListViewModel لیستی از آبجکت‌ها Book را ذخیره می‌کند.

برای نمایش لیستی از کتاب‌ها یک فرگمنت جدید در لایه کنترل کننده BookShelf مورد نیاز می‌باشد. این فرگمنت BookListFragment نام دارد.

MainActivity میزبان نمونه‌ای از BookListFragment است که به نوبه خود لیست کتاب‌ها را روی صفحه نمایش می‌دهد.

ویوی اکتیویتی از همان فرگمنت حاوی FrameLayout تشکیل شده است اما ویوی فرگمنت شامل یک RecyclerView خواهد بود. در ادامه فصل با کلاس RecyclerView بیشتر آشنا خواهید شد.

افزودن یک فرگمنت جدید و ViewModel

اولین قدم اضافه کردن یک ViewModel جدید برای ذخیره لیستی از آبجکت‌ها Book برای نمایش خواهد بود. بنابراین با افزودن وابستگی آن به فایل app/build.gradle شروع کنید.

لیست ۹.۱ افزودن dependency (app/build.gradle)

```
dependencies {
    ...
    implementation(libs.material)
    implementation("androidx.lifecycle:lifecycle-viewmodel-
ktx:2.10.0")
    implementation("androidx.activity:activity-ktx:1.9.0")
    implementation("androidx.fragment:fragment-ktx:1.8.9")
    testImplementation(libs.junit)
    ...
}
```

دو مورد دیگر وابستگی برای استفاده از مدل ویو در داخل اکتیویتی یا فرگمنت کاربرد دارد. فراموش نکنید که پس از انجام این تغییر فایل‌های Gradle خود را همگام سازی کنید.

در مرحله بعد یک کلاس کاتلین جدید به نام BookListViewModel ایجاد کنید. کلاس جدید را به روز کنید تا از ViewModel ارث‌بری داشته باشد. یک عضو برای ذخیره لیست Bookها اضافه کنید. در بلوک init لیست را با داده‌های ساختگی پر نمایید.

لیست ۹.۲ ایجاد books (BookListViewModel.kt)

```
package com.codeb.bookshelf
import androidx.lifecycle.ViewModel

class BookListViewModel: ViewModel() {
    val books = mutableListOf<Book>()
    init {
        for (i in 0 until 100) {
            val book = Book()
            book.title = "Book #$i"
            book.isSolved = i % 2 == 0
```

```
            books += book
        }
    }
}
```

BookListViewModel راه حلی برای ذخیره‌سازی طولانی مدت داده‌ها نیست اما تمام داده‌های لازم برای پر کردن ویوی BookListFragment را دارد. در فصل ۱۱ زمانی BookShelf را برای ذخیره لیست Book در یک پایگاه داده به روز و در مورد ذخیره‌سازی طولانی مدت داده‌ها بیشتر خواهیم آموخت.

گام بعدی ساخت BookListFragment و مرتبط کردن آن با BookListViewModel است. کلاس BookListFragment را ایجاد و آن را به زیر کلاس androidx.fragment.app.Fragment تبدیل کنید.

لیست ۹.۳ پیاده‌سازیBookListFragment (BookListFragment.kt)

```
package com.codeb.bookshelf
import android.os.Bundle
import android.util.Log
import androidx.fragment.app.Fragment
import androidx.fragment.app.viewModels
import kotlin.getValue
private const val TAG="BookListFragment"
class BookListFragment: Fragment() {
    private val bookListViewModel:
        BookListViewModel by viewModels()
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        Log.d(TAG
            , "Total Books: ${bookListViewModel.books.size}")
    }
    companion object {
        fun newInstance(): BookListFragment {
            return BookListFragment()
        }
    }
}
```

در حال حاضر BookListFragment یک پوسته خالی از یک فرگمنت است که تعداد کتاب‌ها یافت شده در BookListViewModel را Log می‌کند. در ادامه فصل توضیحات بیشتری در مورد این فرگمنت خواهد آمد.

یکی از تمرین‌های خوب برای پیروی ارائه تابع (…)newInstance است که اکتیویتی‌های شما می‌توانند برای دریافت نمونه‌ای از فرگمنت خود فراخوانی کند. شبیه به تابع ()newIntent است که در GeoQuiz استفاده کردید. در فصل ۱۲ نحوه انتقال داده به فرگمنت‌های خود را خواهید دید.

چرخه حیات ViewModel با فرگمنت‌ها

در فصل ۴ در مورد چرخه حیات ViewModel هنگام استفاده با یک اکتیویتی چیزهای بسیاری آموختید. اما این چرخه حیات زمانی که ViewModel با یک فرگمنت استفاده می‌شود، کمی‌متفاوت است. هنوز هم فقط دو حالت created یا destroyed/nonexistent را دارد. اما اکنون به جای اکتیویتی به چرخه حیات فرگمنت گره خورده است.

ViewModel تا زمانی که ویوی فرگمنت روی صفحه نمایش باشد، فعال خواهد ماند. ViewModel در طول چرخش باقی می‌ماند (حتی اگر نمونه فرگمنت این کار را نکند) و برای نمونه فرگمنت جدید قابل دسترس خواهد بود.

ViewModel با از بین رفتن فرگمنت از بین می‌رود. همچنین اگر اکتیویتی میزبانی فرگمنت را با فرگمنت دیگری جایگزین کند، ممکن است این اتفاق روی دهد. حتی اگر اکتیویتی یکسانی روی صفحه نمایش باشد هم فرگمنت و هم ViewModel مرتبط با آن از بین می‌روند، زیرا دیگر نیازی به آنها نیست.

یک مورد خاص زمانی است که تراکنش فرگمنت را به پشته اضافه می‌کنیم. هنگامی که اکتیویتی فرگمنت فعلی را با فرگمنت دیگری جایگزین می‌کند، اگر تراکنش به پشته اضافه شود، نمونه فرگمنت و ViewModel آن از بین نمی‌رود. در این حالت وضعیت شما حفظ می‌شود: اگر کاربر دکمه Back را فشار دهد، تراکنش فرگمنت معکوس می‌شود. نمونه فرگمنت اصلی دوباره روی صفحه قرار می‌گیرد و تمام داده‌های موجود در ViewModel حفظ می‌شوند. در مرحله بعد MainActivity را به روز کنید تا به جای BookFragment نمونه‌ای از BookListFragment را میزبانی کند.

لیست ۹.۴ افزودن BookListFragment با تراکنش فرگمنت (MainActivity.kt)

```
override fun onCreate(savedInstanceState: Bundle?) {
    …
    val currentFragment =
        supportFragmentManager.findFragmentById
            (R.id.fragment_container)
    if (currentFragment == null) {
        val fragment =~~BookFragment()~~
            BookListFragment.newInstance()
        supportFragmentManager
```

```
                .beginTransaction()
                .add(R.id.fragment_container, fragment)
                .commit()
    }

}
```

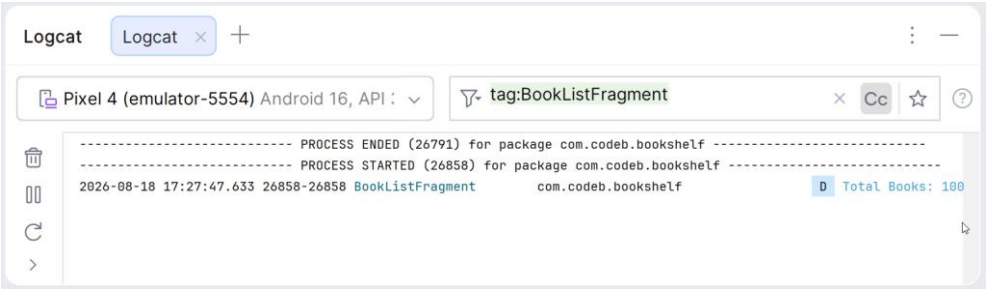
در حال حاضر MainActivity طوری به روز شده است که همیشه BookListFragment نمایش داده شود. در فصل ۱۲ آن را طوری به روز خواهیم کرد تا BookListFragment و BookFragment بسته به پیمایش کاربر جایگزین شود.

BookShelf را اجرا کنید. خواهید دید کهFrameLayout موجود در MainActivity همانطور که در شکل ۹.۲ نشان داده شده است، میزبان یک BookListFragment خالی است.

تصویر ۹.۲ صفحه خالی MainActivity



خروجی Logcat را برای BookListFragment جستجو کنید. گزارشی از تعداد کل کتاب‌ها موجود مشاهده خواهید کرد:



افزودن RecyclerView

اگر می‌خواهید BookListFragment لیستی از کتاب‌ها را به کاربر نمایش دهد باید از RecyclerView استفاده کنید.

کلاس RecyclerView در کتابخانه Jetpack دیگری قرار دارد. اولین قدم برای استفاده از RecyclerView اضافه کردن کتابخانه RecyclerView به عنوان یک dependency است.

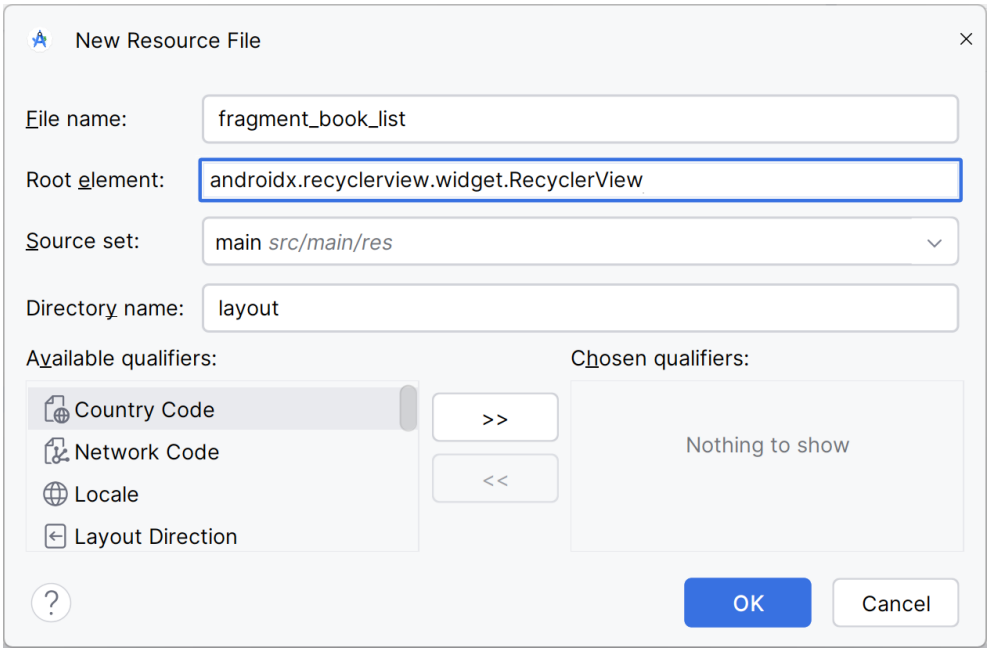
لیست ۹.۵ افزودن وابستگی RecyclerView (app/build.gradle)

```
dependencies {
    …
    implementation("androidx.fragment:fragment-ktx:1.8.9")
    implementation("androidx.recyclerview:recycler-
view:1.4.0")
    …
}
```

باز هم فایل‌های Gradle خود را قبل از ادامه همگام سازی کنید.

RecyclerView را در فایل لیوت BookListFragment قرار خواهیم داد. البته ابتدا باید فایل لیوت را ایجاد کنید. یک فایل لیوت جدید به نام fragment_book_list ایجاد و برای عنصر Root، androidx.recyclerview.widget.RecyclerView را مشخص کنید (شکل ۹.۴).

تصویر ۹.۳ افزودن فایل لیوت BookListFragment



در فایل layout/fragment_book_list.xml جدید یک ویژگی ID به RecyclerView اضافه کنید. تگ بسته را همانند لیست ۹.۶ تغییر دهید. زیرا هیچ فرزندی به RecyclerView اضافه نخواهیم کرد.

لیست ۹.۶ افزودن RecyclerView (layout/fragment_Book_list.xml)

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.recyclerview.widget.RecyclerView
xmlns:android="http://schemas.android.com/apk/res/android"
    android:id="@+id/book_recycler_view"
    android:layout_width="match_parent"
    android:layout_height="match_parent"/>
</androidx.recyclerview.widget.RecyclerView>
```

پس از تکمیل ویوی BookListFragment آن را به فرگمنت وصل کنید. BookListFragment را طوری به روز کنید تا از این فایل لیوت استفاده کند و ارجاعی به RecyclerView همانطور که در لیست ۹.۷ نشان داده شده است، ایجاد کنید.

لیست ۹.۷ تنظیم ویوی برای BookListFragment

```
class BookListFragment: Fragment() {
```

```
    private lateinit var bookRecyclerView: RecyclerView
    …
    override fun onCreate(savedInstanceState: Bundle?) {
        …
    }
    override fun onCreateView(
        inflater: LayoutInflater,
        container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View? {
        val view = inflater.inflate
            (R.layout.fragment_book_list, container, false)
        bookRecyclerView =
            view.findViewById(R.id.book_recycler_view)
            as RecyclerView
        bookRecyclerView.layoutManager =
            LinearLayoutManager(context)
        return view
    }
    …
}
```

توجه داشته باشید که به محض ایجاد RecyclerView باید شی دیگری به نام LayoutManager به آن بدهیم. RecyclerView برای کار به یک LayoutManager نیاز دارد.

RecyclerView آیتم‌ها را خودش روی صفحه قرار نمی‌دهد بلکه انجام این کار را به LayoutManager محول می‌کند. LayoutManager محل قرارگیری هر آیتم و همچنین نحوه پیمایش آنها را تعریف می‌کند. بنابراین اگر LayoutManager وجود نداشته باشد، RecyclerView بلافاصله باعث توقف برنامه می‌شود.

چند LayoutManager داخلی برای انتخاب وجود دارد. در اینجا ما از LinearLayoutManager استفاده می‌کنیم که موارد موجود در لیست را به صورت عمودی ردیف می‌کند. بعدا در این کتاب از GridLayoutManager برای مرتب کردن آیتم‌ها به صورت شبکه‌ای استفاده خواهیم کرد.

برنامه را اجرا کنید. شما باید دوباره یک صفحه خالی ببینید، اما اکنون به یک RecyclerView خالی نگاه می‌کنید.

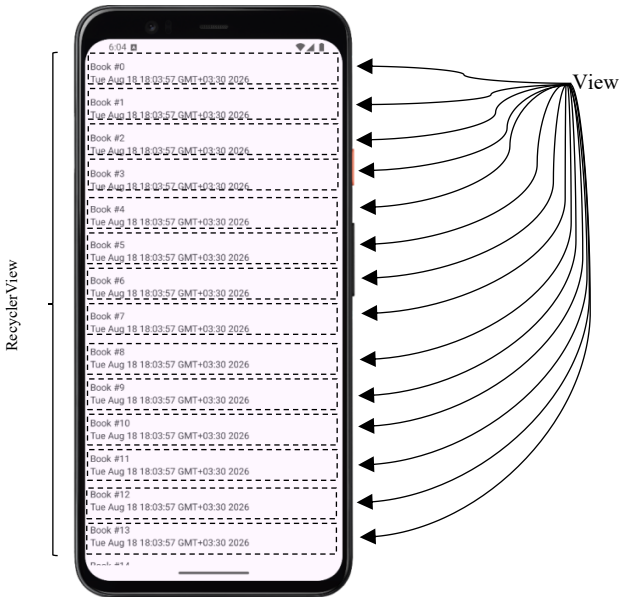
ایجاد لیوت آیتم‌های لیست

RecyclerView زیر کلاس ViewGroup است. لیستی از آبجکت‌ها ویوی فرزند که item views نامیده می‌شود را نمایش می‌دهد. هر ویوآیتم نشان دهنده یک شی واحد از لیست داده‌هایی است که تامین

کننده RecyclerView است (در مورد ما، یک کتاب واحد از لیست کتاب‌ها). بسته به پیچیدگی آنچه باید نمایش دهیم، این ویوهای فرزند می‌توانند پیچیده یا بسیار ساده باشند.

برای اولین اجرا، هر مورد در لیست عنوان و تاریخ یک کتاب را همانطور که در تصویر ۹.۵ نشان داده شده است، به نمایش می‌گذارد.

تصویر ۹.۴ RecyclerView با ویوهای فرزند



هر مورد نمایش داده شده در RecyclerView سلسله مراتب ویوی خاص خود را خواهد داشت. دقیقا به همان روشی که BookFragment سلسله مراتب ویو برای کل صفحه دارد. شی View ما در اینجا در هر ردیف یک LinearLayout شامل دو TextView خواهد داشت.

یک لیوت جدید برای ویوآیتم لیست به همان روشی که برای ویوی یک اکتیویتی یا یک فرگمنت ایجاد می‌کردید، ایجاد نمایید. در پنجره ابزار پروژه روی دایرکتوری res/layout کلیک راست کرده و New → Layout resource file را انتخاب کنید. نام فایل را list_item_book، عنصر ریشه را روی LinearLayout تنظیم و روی OK کلیک کنید.

فایل لیوت خود را همانطور که در لیست ۹.۸ نشان داده شده است به روز کنید تا به LinearLayout، padding و دو TextView را اضافه کنید.

لیست ۹.۸ بروزرسانی فایل لیوت‌آیتم لیست (لیوت و list_item_book.xml)

```
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:layout_height="wrap_content"
    android:padding="8dp">
    <TextView
        android:id="@+id/book_title"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Book Title"/>
    <TextView
        android:id="@+id/book_date"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Book Date"/>
</LinearLayout>
```

به پیش‌نمایش نگاهی بیندازید. خواهید دید که دقیقا یک ردیف از لیست را ایجاد کرده‌اید. RecyclerView تمام لیست را برای شما ایجاد خواهد کرد.

پیاده‌سازی یک ViewHolder

RecyclerView انتظار دارد که یک ویوآیتم در نمونه‌ای از ViewHolder قرار گیرد. ViewHolder ارجاعی به ویوی یک مورد (و گاهی اوقات ارجاع به ویجت‌های خاص در آن ویو) ذخیره می‌کند.

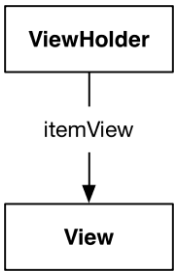
یک ViewHolder با افزودن یک کلاس داخلی که از RecyclerView.ViewHolder ارث‌بری دارد، تعریف کنید.

لیست ۹.۹ آغاز یک ViewHolder (BookListFragment.kt)

```
class BookListFragment: Fragment() {
    ...
    private inner class BookHolder(view: View):
        RecyclerView.ViewHolder(view) {
    }
}
```

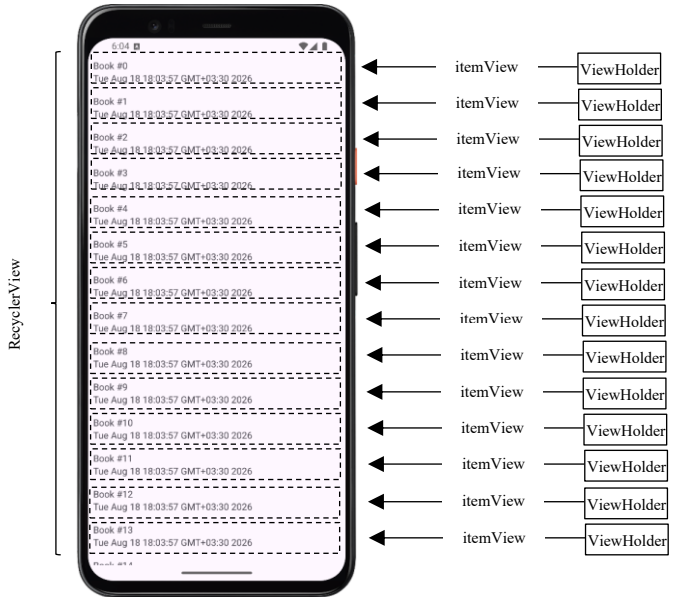
در سازنده BookHolder ویوآیتم را دریافت می‌کنیم تا بعدا از آن استفاده نمایید. سپس بلافاصله آن را به عنوان آرگومان به سازنده RecyclerView.ViewHolder ارسال می‌کنیم. سپس کلاس پایه ViewHolder ویو را در متغییری به نام itemView نگهداری می‌کند (شکل ۹.۵).

تصویر ۹.۵ ViewHolder و itemView



RecyclerView هرگز به تنهایی ویوها را ایجاد نمی‌کند بلکه همیشه ViewHolder را ایجاد می‌کند تا itemViewها خود را نیز به همراه بیاورد (شکل ۹.۶).

تصویر ۹.۶ ViewHolder تجسم شده است



وقتی View هر مورد از لیست طراحی و اطلاعات ساده‌ای در بردارد، ViewHolder مسئولیت کمی خواهد داشت. برای ویوهای پیچیده‌تر ViewHolder اتصال قسمت‌های مختلف itemView را به یک Book ساده‌تر و کارآمدتر می‌کند.

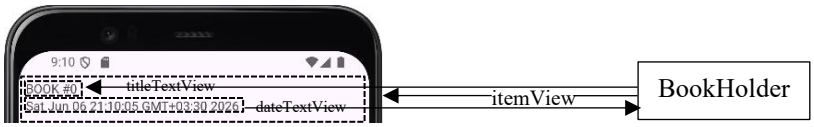
BookHolder را به روز کنید تا ویوهای متن عنوان و تاریخ را در سلسله مراتب itemView هنگام اولین ایجاد یک نمونه پیدا کنید.

لیست ۹.۱۰ ارجاع به ویوها در سازنده (BookListFragment.kt)

```
private inner class BookHolder(view: View):
    RecyclerView.ViewHolder(view) {
    val titleTextView: TextView =
        itemView.findViewById(R.id.book_title)
    val dateTextView: TextView =
        itemView.findViewById(R.id.book_date)
}
```

ViewHolder به روز شده اکنون ارجاعاتی به ویوهای عنوان و تاریخ را دارد (شکل ۹.۸).

تصویر ۹.۷ ViewHolder



توجه داشته باشید که BookHolder فرض می‌کند که ویوی ارسال شده به سازنده آن دارای ویوهای متن فرزند با شناسه‌های R.id.book_title و R.id.book_date است. ممکن است از خود بپرسید "چه کسی (یا چه چیزی) نمونه‌های BookHolder را ایجاد می‌کند و آیا می‌توانم با خیال راحت فرض کنم که سلسله مراتب ویوی منتقل شده به سازنده شامل ویجت‌های فرزندی است که انتظار دارم؟" پاسخ این سوالات را تنها در یک لحظه خواهید آموخت.

پیاده‌سازی یک آداپتور برای پر کردن RecyclerView

شکل ۹.۶ تا حدودی ساده شده است. RecyclerView، ViewHolderها را خودش ایجاد نمی‌کند بلکه از یک آداپتور انجام این کار را درخواست می‌کند. آداپتور یک شی کنترل کننده است که بین RecyclerView و مجموعه داده‌ای که RecyclerView باید نمایش داده شود، قرار می‌گیرد.

آداپتور مسئول موارد زیر است:

- ایجاد ViewHolderهای لازم در صورت درخواست
- اتصال ViewHolderها به داده‌های لایه مدل در صورت درخواست

RecyclerView مسئول موارد زیر است:

- درخواست از آداپتور برای ایجاد یک ViewHolder جدید
- درخواست از آداپتور برای اتصال یک ViewHolder به یک آیتم از داده پشتیبان در یک موقعیت معین

زمان ایجاد آداپتور است. یک کلاس داخلی جدید به نام BookAdapter به BookListFragment اضافه کنید. یک سازنده اولیه اضافه کنید که انتظار دارد لیستی از کتاب‌ها را به عنوان ورودی داشته باشد و لیست کتاب‌های ارسال شده را همانطور که در فهرست ۹.۱۱ نشان داده شده است، در یک متغییر ذخیره کند.

در BookAdapter جدید خود، سه تابع (...)onCreateViewHolder، (…)onBindViewHolder و ()getItemCount را پیاده‌سازی کنید. برای صرفه‌جویی در تایپ (و اشتباهات تایپی)، اندروید استودیو می‌تواند این پیاده‌سازی‌ها را برای شما انجام دهد. هنگامی که خط اولیه کد جدید را تایپ کردید، مکان نشانگر موس خود را روی BookAdapter قرار دهید و Alt+Enter را فشار دهید. پیاده‌سازی اعضا را از پنجره بازشو انتخاب کنید. در کادر Implementing members نام هر سه تابع را انتخاب کرده و روی OK کلیک کنید. در ادامه فقط باید بدنه توابع را پر کنید.

لیست ۹.۱۱ ایجاد BookAdapter (BookListFragment.kt)

```
class BookListFragment: Fragment() {
    …
    private inner class BookHolder(view: View) :
        RecyclerView.ViewHolder(view) {
        …
    }
    private inner class BookAdapter(var books: List<Book>) :
        RecyclerView.Adapter<BookHolder>() {
        override fun onCreateViewHolder(
            parent: ViewGroup,
            viewType: Int
        ): BookHolder {
            val view = layoutInflater.inflate(
                R.layout.list_item_book, parent, false)
            return BookHolder(view)
        }
        override fun onBindViewHolder(
            holder: BookHolder,
```

```
            position: Int
        ) {
            val book = books[position]
            holder.apply {
                titleTextView.text = book.title
                dateTextView.text = book.date.toString()
            }
        }
        override fun getItemCount()=books.size
    }
}
```

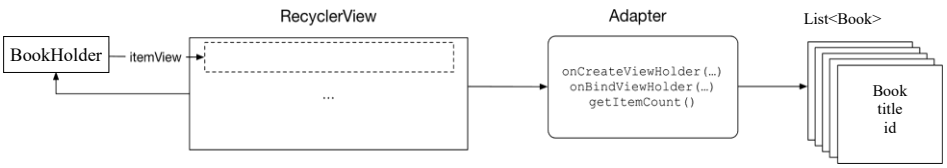
Adapter.onCreateViewHolder(…) مسئول ایجاد یک ویو برای نمایش، قرار دادن ویو در یک ویوهولدر و بازگرداندن نتیجه است. در این مورد list_item_view.xml را بارگذاری و ویو حاصل را به نمونه‌ای از BookHolder می‌فرستید.

Adapter.onBindViewHolder(holder: BookHolder, position: Int) مسئول پر کردن یک holder معین با یک Book از جایی تعیین شده از لیست است. در این حالت book را از لیست books در موقعیت درخواستی دریافت می‌کنیم. سپس از عنوان و داده‌های آن برای تنظیم متن در ویوهای متنی مربوطه استفاده می‌کنیم.

وقتی RecyclerView نیاز به اطلاعاتی در مورد تعداد داده‌هایی که از آن پشتیبانی می‌کند، دارد متد Adapter.getItemCount() از آداپتور را فراخوانی می‌کند. در اینجا getItemCount() تعداد موارد موجود در لیست کتاب‌ها را برمی‌گرداند تا به درخواست RecyclerView پاسخ دهد.

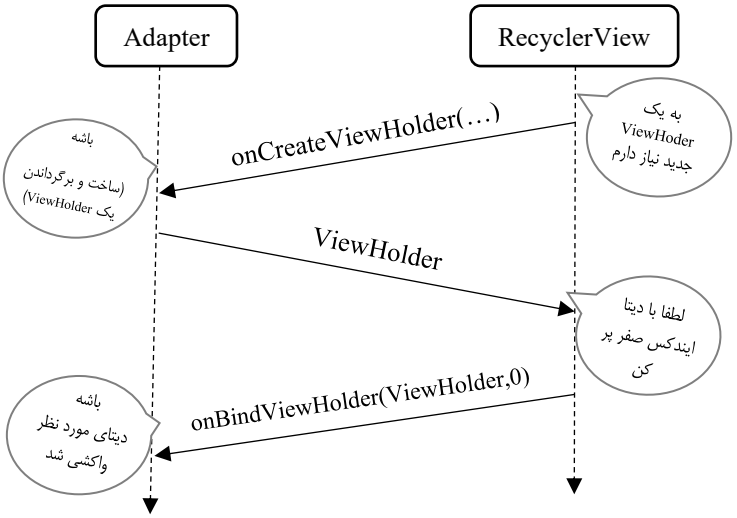
RecyclerView خود چیزی در مورد شی Book یا لیست آبجکت‌ها Book که باید نمایش داده شود نمی‌داند. در عوض BookAdapter تمام جزئیات یک Book را می‌داند. آداپتور همچنین در مورد لیست Bookها که از RecyclerView پشتیبانی می‌کند، اطلاعات کافی دارد (شکل ۹.۸).

شکل ۹.۸ آداپتور بین RecyclerView و مجموعه داده‌ها قرار دارد



هنگامی که RecyclerView برای نمایش به یک شی view نیاز دارد با آداپتور خود همانند شکل ۹.۱۰ مکالمه خواهد داشت.

شکل ۹.۹ یک مکالمه درخشان RecyclerView-Adapter



RecyclerView تابع onCreateViewHolder(ViewGroup, Int) آداپتور را برای ایجاد یک ViewHolder جدید همراه با یک View برای نمایش، فراخوانی می‌کند. ViewHolder (و itemView آن) که آداپتور ایجاد و به RecyclerView تحویل می‌دهد، هنوز با داده‌های خاصی پر نشده است.

در مرحله بعد RecyclerView، onBindViewHolder(ViewHolder, Int) را با پارامترهای ViewHolder و یک عدد که نشانگر موقعیت لیست است، فراخوانی می‌کند. آداپتور داده‌های مدل را بر اساس آن عدد جستجو می‌کند و آن را به ویوی ViewHolder متصل می‌کند. برای اتصال آداپتور View را پر می‌کند تا داده‌ها را در شی مدل منعکس کند.

پس از تکمیل این فرآیند RecyclerView یک مورد لیست را روی صفحه قرار می‌دهد.

تنظیم آداپتور RecyclerView

اکنون که یک آداپتور دارید، آن را به RecyclerView خود وصل کنید. تابعی به نام updateUI را پیاده‌سازی کنید که رابط کاربری BookListFragment را تنظیم می‌کند. در حال حاضر updateUI یک BookAdapter ایجاد و آن را به RecyclerView وصل می‌کند.

لیست ۹.۱۲ تنظیم آداپتور (BookListFragment.kt)

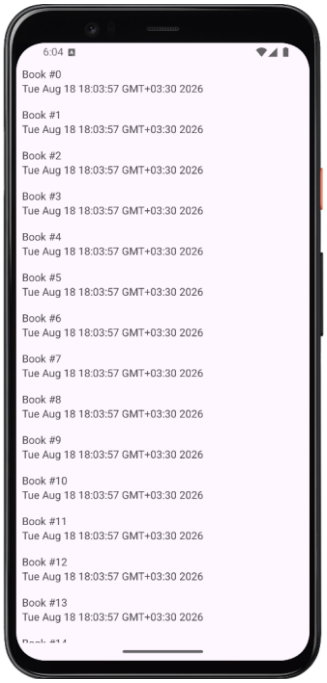
```
class BookListFragment: Fragment() {
    …
    private lateinit var bookRecyclerView: RecyclerView
```

```
    private var adapter: BookAdapter? = null
    override fun onCreate(savedInstanceState: Bundle?) {
        …
    }
    companion object {
        …
    }
    override fun onCreateView(
        …
        updateUI()
        return view
    }
    private fun updateUI() {
        val books= bookListViewModel.books
        adapter = BookAdapter(books)
        bookRecyclerView.adapter = adapter
    }
    …
}
```

در فصل‌های بعدی موارد بیشتری به updateUI() برای مدیریت رابط کاربری پیچیدتر اضافه خواهیم کرد.

BookShelf را اجرا و در RecyclerView جدید خود پیمایش کنید. باید شبیه شکل ۹.۱۰ باشد.

شکل ۹.۱۰ RecyclerView پر از کتاب‌ها



با کشیدن انگشت خود به بالا و پایین ویوهای بیشتری را در سراسر صفحه نمایش خود خواهید دید. هر BookHolder قابل مشاهده باید یک کتاب متمایز را نشان دهد.

هنگامی که ویوی را به سمت بالا می‌کشید، انیمیشن پیمایش باید به اندازه کره گرم روان به نظر برسد. این اثر نتیجه مستقیم و کارآمد onBindViewHolder() است که فقط حداقل کار لازم را انجام می‌دهد. در غیر این صورت انیمیشن اسکرول شما می‌تواند به‌اندازه یک فنر کلفت سخت به نظر برسد.

استفاده مجدد ویوها

شکل ۹.۱۰ چهارده ردیف ویو را نشان می‌دهد. برای پیمایش هر صد Book به پایین اسکرول کنید. آیا این بدان معناست که شما صد شی View در حافظه دارید؟ با لطف RecyclerView نه.

ایجاد یک ویوی برای هر مورد از لیست آن هم یکباره، می‌تواند برنامه را غیر قابل اجرا کند. همانطور که می‌توانید تصور کنید یک لیست می‌تواند بیش از صد مورد و موارد لیست می‌تواند اجزای خیلی بیشتری داشته باشد. همچنین یک کتاب فقط زمانی به یک ویو نیاز دارد که روی صفحه نمایش باشد، بنابراین نیازی به صد ویو آماده نیست. ایجاد آبجکت‌ها view فقط در صورت نیاز بسیار منطقی‌تر خواهد بود.

RecyclerView دقیقا همین کار را انجام می‌دهد. به جای ایجاد صد ویو، فقط به‌اندازه کافی برای پر کردن صفحه ویو ایجاد می‌کند. هنگامی که یک ویو از صفحه نمایش خارج می‌شود RecyclerView به جای دور انداختن آن، از آن استفاده مجدد می‌کند. به طور خلاصه ویوها را بارها و بارها بازیافت می‌کند. به همین خاطر نام این عنصر RecyclerView است.

به همین دلیل onCreateViewHolder(…) بسیار کمتر از onBindViewHolder(…) اتفاق می‌افتد. وقتی ViewHolderهای کافی ایجاد شد، RecyclerView فراخوانی onCreateViewHolder(…) را متوقف می‌کند.

در عوض با بازیافت ViewHolderهای قدیمی و انتقال آنها به onBindViewHolder(…) در زمان و حافظه صرفه‌جویی می‌کند.

تفکیک وظایف

در حال حاضر آداپتور، داده‌های را مستقیما به ویوهای متنی Bookholder در متصل می‌کند. این کار به خوبی انجام می‌شود، اما بهتر است وظایف را بین view holder و آداپتور تفکیک کنید. آداپتور باید تا حد امکان کمتر در مورد عملکرد داخلی و جزئیات view holder بداند.

توصیه می‌کنیم تمام کدهایی که کار واقعی اتصال را انجام می‌دهند در BookHolder قرار دهید. ابتدا یک ویژگی برای نگهداری Book که در حال اتصال است اضافه کنید. تکست ویوی موجود را private کنید. یک تابع bind(Book) به BookHolder اضافه کنید. در این تابع جدید Book که در حال اتصال به رابط کاربری است را در یک ویژگی ذخیره و مقادیر متنی را روی titleTextView و dateTextView تنظیم کنید.

لیست ۹.۱۳ نوشتن یک تابع bind(Book) (BookListFragment.kt)

```
private inner class BookHolder(view: View) :
RecyclerView.ViewHolder(view) {
    private lateinit var book: Book
    private val titleTextView: TextView =
        itemView.findViewById(R.id.book_title)
    private val dateTextView: TextView =
        itemView.findViewById(R.id.book_date)
    fun bind(book: Book){
        this.book=book
        titleTextView.text=this.book.title
        dateTextView.text=this.book.date.toString()
    }
}
```

هنگامی که Bookی برای اتصای به شما داده می‌شود BookHolder اکنون عنوان TextView و تاریخ TextView را به روز می‌کند تا وضعیت Book را منعکس کند.

در مرحله بعد تابع bind(Book) تازه خود را هر بار که RecyclerView درخواست اتصال یک BookHolder به یک کتاب خاص را دارد، فراخوانی کنید.

لیست ۹.۱۴ فراخوانی تابع bind(Book) (BookListFragment.kt)

```
private inner class BookAdapter(var books: List<Book>) :
RecyclerView.Adapter<BookHolder>() {
    override fun onCreateViewHolder(
        parent: ViewGroup,
        viewType: Int
    ): BookHolder {
        val view = layoutInflater.inflate
            (R.layout.list_item_book, parent, false)
        return BookHolder(view)
    }
    override fun onBindViewHolder(
        holder: BookHolder,
        position: Int
    ) {
        val book = books[position]
        holder.apply {
            titleTextView.text = book.title
            dateTextView.text = book.date.toString()
        }
        holder.bind(book)
    }

    override fun getItemCount()=books.size
}
```

BookShelf را یک بار دیگر اجرا کنید. نتیجه باید مانند شکل ۹.۱۰ باشد.

پاسخگویی به لمس آیتم‌های لیست

RecyclerView باید به لمس موارد لیست پاسخ دهد. در فصل‌های بعدی زمانی که کاربر کتابی را در لیست لمس می‌کند ویوی جزئیات یک کتاب را راه اندازی خواهیم کرد. در حال حاضر فقط به نمایش یک Toast اکتفا می‌کنیم.

همانطور که ممکن است متوجه شده باشید RecyclerView در حالی که قدرتمند و توانا به نظر می‌رسد، مسئولیت‌های واقعی کمی دارد. در اینجا هم همینطور است: مدیریت رویدادهای لمسی بیشتر به ما بستگی دارد. اگر به آنها نیاز دارید RecyclerView می‌تواند رویدادهای لمسی خام را هدایت کند. اما بیشتر اوقات این کار ضروری نیست.

در عوض می‌توانیم آنها را مانند معمول مدیریت کنید: با تنظیم OnClickListener. هر ویو دارای یک ViewHolder مرتبط است. می‌توانید OnClickListener مربوط به ViewHolder را به برای ویوی آن به کار ببرید. BookHolder را برای مدیریت لمس کل ردیف‌ها تغییر دهید.

لیست ۹.۱۵ تشخیص پرس در BookHolder (BookListFragment.kt)

```
private inner class BookHolder(view: View) :
RecyclerView.ViewHolder(view) {
    private lateinit var book: Book
    private val titleTextView: TextView =
        itemView.findViewById(R.id.book_title)
    private val dateTextView: TextView =
        itemView.findViewById(R.id.book_date)
    init {
        itemView.setOnClickListener {
            Toast.makeText(context, "${book.title} pressed!"
                , Toast.LENGTH_SHORT)
                .show()
        }
    }
    fun bind(book: Book){
        this.book=book
        titleTextView.text=this.book.title
        dateTextView.text=this.book.date.toString()
    }
}
```

در لیست ۹.۱۵ BookHolder خودش رابط OnClickListener را پیاده‌سازی می‌کند. در itemView، که View برای کل ردیف است، BookHolder به عنوان گیرنده رویدادهای کلیک تنظیم می‌شود.

BookShelf را اجرا کنید و روی یک مورد در لیست فشار دهید. باید یک Toast ببینید.

بیشتر بدانیم: ListView و GridView

یکی دیگر از ویژگی‌های کلیدی RecyclerView انیمیشن آیتم‌های موجود در لیست است. متحرک سازی افزودن یا حذف آیتم‌ها را RecyclerView بسیار آسان انجام می‌دهد. شامل چند انیمیشن داخلی است و امکان سفارشی سازی آسان این انیمیشن‌ها را فراهم می‌کند.

به عنوان مثال اگر متوجه شدید که کتابی در موقعیت ۰ به موقعیت ۵ منتقل شده است، می‌توانید آن تغییر را به این صورت متحرک کنید:

```
recyclerView.adapter.notifyItemMoved(0,5)
```

چالش: ViewTypeهای RecyclerView

برای این چالش پیشرفته دو نوع ردیف در RecyclerView خود ایجاد خواهید کرد: یک ردیف معمولی و یک ردیف برای کتاب های مورد علاقه. برای پیاده‌سازی این کار با ویژگی نوع ویوی موجود در RecyclerView.Adapter کار می‌کنیم. یک ویژگی جدید به نام favorite را به شی Book اضافه و از آن برای تعیین اینکه کدام ویو در BookAdapter بارگذاری شده با پیاده‌سازی تابع getItemViewType(Int) استفاده کنید.

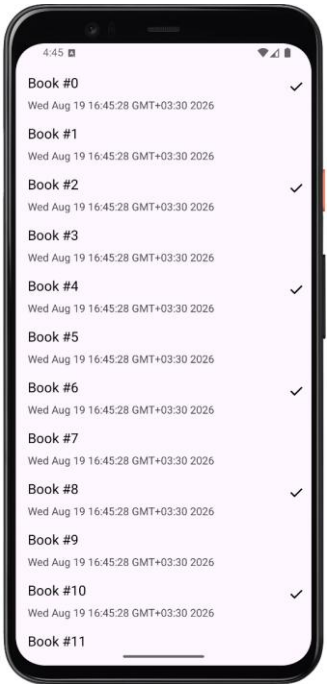
در تابع onCreateViewHolder(ViewGroup, Int) باید منطقی اضافه کنید که ViewHolder متفاوتی را بر اساس مقدار viewType جدید بازگردانده شده توسط getItemViewType(Int) را برمی‌گرداند. از لیوت اصلی برای کتاب‌های معمولی و یک لیوت جدید با یک رابط کاربری ساده حاوی یک آیکون ستاره برای کتاب‌های مورد علاقه استفاده کنید.

۱۰

ایجاد UI با لیوت‌ها و ویجت‌ها

در این فصل در مورد لیوت‌ها و ویجت‌ها بیشتر یاد خواهیم گرفت و در عین حال برخی از استایل‌ها را به آیتم‌های لیست خود در RecyclerView اضافه خواهیم کرد. همچنین چگونگی استفاده از ویرایشگر لیوت گرافیکی برای مرتب کردن ویجت‌ها در یک ConstraintLayout را یاد خواهیم گرفت. شکل ۱۰.۱ نشان می‌دهد که ویوی BookListFragment چگونه خواهد بود.

شکل ۱۰.۱ BookShelf اکنون با تصاویر زیبا



در فصل‌های قبلی از سلسله مراتب لیوت تودرتو برای مرتب کردن ویجت‌های خود استفاده کردیم. به عنوان مثال layout/activity_main.xml که در فصل اول، یک LinearLayout را در یک LinearLayout دیگر قرار می‌داد. خواندن و ویرایش این لیوت‌های تودرتو برای شما به عنوان یک توسعه

دهنده سخت و پیچیده است. بدتر از آن، این ساختار تودرتو می‌تواند عملکرد برنامه شما را کاهش دهد. لیوت‌های تو در تو زمان زیادی از سیستم عامل اندروید برای‌اندازه‌گیری و چیدمان می‌گیرند. به این معنی که کاربران شما ممکن است قبل از دیدن ویوهای شما روی صفحه با تأخیر مواجه شوند.

لیوت‌های غیر تودرتو تاخیر کمی دارند و این یکی از مواردی است که ConstraintLayout اهمیت خود را نشان می‌دهد. ما می‌توانیم لیوت‌های پیچیده زیبایی را بدون استفاده از ساختار تودرتو ایجاد کنیم.

قبل از اینکه به ConstraintLayout بپردازیم، باید کمی اطلاعات پس‌زمینه‌ای ارایه دهیم.

به یک آیکون تیک همانند شکل ۱۰.۱ در پروژه خود نیاز داریم. می‌توانید آن را همانطور که توضیح دادیم با استفاده از Vector asset به پروژه اضافه نمایید.

معرفی ConstraintLayout

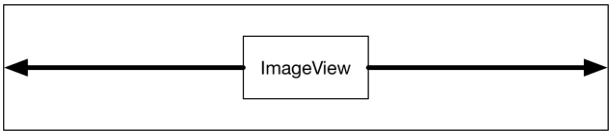
با ConstraintLayout به جای استفاده از لیوت‌های تو در تو مجموعه‌ای از محدودیت‌ها (constraints) را به لیوت خود اضافه می‌کنیم. یک محدودیت مانند یک نوار لاستیکی است: دو چیز را به سمت یکدیگر می‌کشد. بنابراین به عنوان مثال اگر یک ImageView داریم می‌توان همانند شکل ۱۰.۲، یک محدودیت را از لبه سمت راست آن به لبه سمت راست والد آن وصل کنیم. این محدودیت ImageView را در سمت راست نگه می‌دارد.

شکل ۱۰.۲ ImageView با محدودیت در لبه سمت راست



می‌توان از هر چهار لبه ImageView خود (چپ، بالا، راست و پایین) یک محدودیت ایجاد کرد. اگر محدودیت‌های متضاد داشته باشید، همه چیز متعادل می‌شود و ImageView شما درست در مرکز دو محدودیت قرار خواهد گرفت (شکل ۱۰.۳).

شکل ۱۰.۳ ImageView با محدودیت‌های مخالف



بنابراین این خلاصه ماجراست: برای قرار دادن ویوها در یک ConstraintLayout در جایی دلخواه به آنها محدودیت اعمال می‌کنیم.

در مورد اندازه ویجت‌ها چطور؟ برای آن سه گزینه وجود دارد: اجازه دهید ویجت تصمیم بگیرد (دوست قدیمی ما wrap_content)، خودتان تصمیم بگیرید، یا اجازه دهید ویجت شما متناسب با محدودیت‌های شما گسترش یابد.

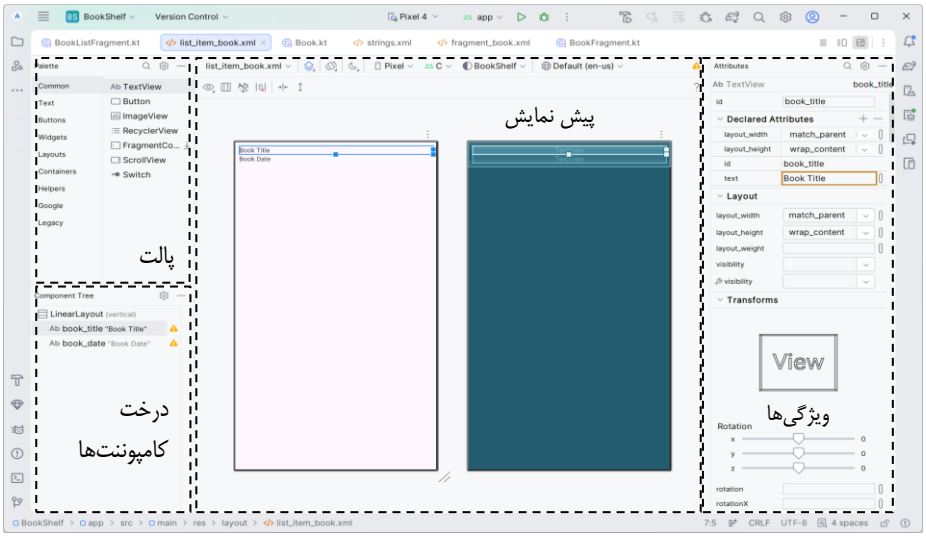
با استفاده از این گزینه‌ها می‌توانیم با یک ConstraintLayout بدون نیاز به ساختار تودرتو به چیدمان‌های زیادی دست پیدا کنیم. همانطور که پیش می‌رویم چگونگی استفاده از محدودیت‌ها برای طراحی فایل لیوت list_item_book.xml خود را خواهید آموخت.

معرفی ویرایشگر گرافیکی لیوت

تا کنون با تایپ XML لیوت ایجاد می‌کردیم اما در این بخش از ابزار گرافیکی لیوت اندروید استودیو استفاده خواهیم کرد.

layout/list_item_book.xml را باز و تب Design را انتخاب کنید (تصویر ۱۰.۴).

تصویر ۱۰.۴ ویوها در ابزار گرافیکی لیوت

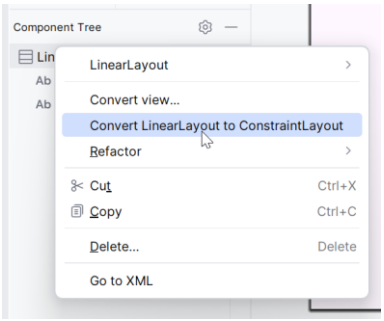


در وسط ویرایشگر گرافیکی لیوت پیش‌نمایشی از لیوت به نمایش در می‌آید. در سمت چپ صفحه palette قرار دارد که شامل تمام ویجت‌هایی است که می‌توان استفاده کرد. ویجت‌ها بر اساس نوع، دسته‌بندی

شده‌اند. درخت کامپوننت (component tree) در پایین سمت چپ قرار دارد. این درخت نشان دهنده چگونگی سازماندهی ویجت‌ها در لیوت است. اگر درخت کامپوننت یا پالت را نمی‌بینید، روی برگه‌های سمت چپ پیش‌نمایش کلیک کنید تا پنجره‌ها باز شوند. در سمت راست صفحه پنجره ویژگی‌ها (attributes) قرار دارد. در اینجا می‌توانید ویژگی‌های ویجت انتخاب شده در درخت کامپوننت‌ها را مشاهده و ویرایش کنید. اولین کاری که باید انجام دهیم به روز رسانی list_item_book.xml برای استفاده از ConstraintLayout است.

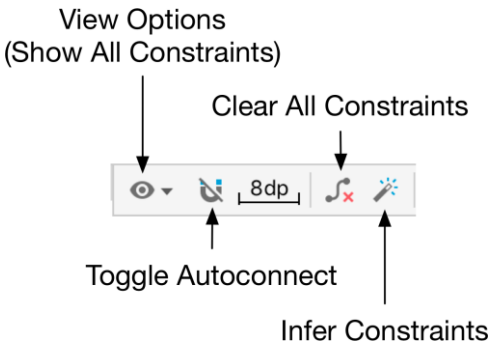
روی LinearLayout ریشه در درخت کامپوننت کلیک راست کرده و Convert LinearLayout to ConstraintLayout را انتخاب کنید (شکل ۱۰.۵).

شکل ۱۰.۵ تبدیل ویوی ریشه به ConstraintLayout



اندروید استودیو سطح تبدیل لیوت را می‌پرسد (شکل ۱۰.۵). از آنجایی که list_item_book.xml یک فایل لیوت ساده است چیز زیادی برای بهینه کردن وجود ندارد. مقادیر پیش‌فرض را علامت بزنید و OK را انتخاب کنید. صبور باشید. فرآیند تبدیل ممکن است کمی‌طول بکشد. هنگامی که کامل شد، یک ConstraintLayout جدید و خوب برای کار خواهیم داشت. نوار ابزار بالای پیش‌نمایش، دارای چند کنترل است (تصویر ۱۰.۶). ممکن است لازم باشد روی پیش‌نمایش کلیک کنید تا همه کنترل‌ها ظاهر شوند.

شکل ۱۰.۶ کنترل محدودیت



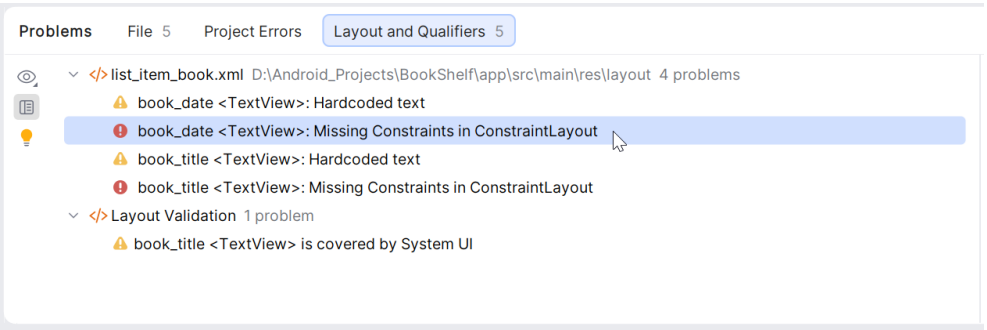
- View Options: View Options → Show All Constraints محدودیت‌هایی را که در پیش‌نمایش و Blueprint تنظیم شده‌اند را نشان می‌دهد. این گزینه در بعضی مواقع مفید خواهد بود. اگر محدودیت‌های زیادی دارید این تنظیم حجم زیادی از اطلاعات را فعال می‌کند. View Options شامل گزینه‌های مفید دیگری مانند Show system UI است. انتخاب این گزینه app bar و همچنین برخی از رابط‌های کاربری سیستم (مانند نوار وضعیت) را که کاربر در زمان اجرا می‌بیند، نمایش می‌دهد. در فصل ۱۴ با app bar بیشتر آشنا خواهید شد.
- Toggle Autoconnect: هنگامی که این گزینه فعال باشد، محدودیت‌ها به طور خودکار در هنگام کشیدن ویوها اعمال می‌شوند. اندروید استودیو محدودیت‌هایی را که می‌خواهید یک ویو داشته باشید حدس می‌زند و آن اتصالات را در صورت تقاضا برقرار می‌کند.
- Clear All Constraints: این دکمه تمام محدودیت‌های موجود در فایل لیوت را حذف می‌کند. به زودی از این گزینه استفاده خواهیم کرد.
- Infer Constraints: این گزینه شبیه به Toggle Autoconnect است. به طور خودکار محدودیت‌هایی را برای شما ایجاد می‌کند. اما تنها زمانی فعال می‌شود که این دکمه را انتخاب کنید.

استفاده از ConstraintLayout

هنگامی که list_item_book.xml را به ConstraintLayout تبدیل کردیم، اندروید استودیو به طور خودکار محدودیت‌هایی را برای حفظ رفتار لیوت قدیمی اضافه می‌کند. با این حال برای یادگیری نحوه عملکرد محدودیت‌ها از ابتدا شروع می‌کنیم.

ویوی سطح بالا در درخت کامپوننت‌ها با برچسب main را انتخاب کنید. در حالی که linearLayout در درخت کامپوننت‌ها انتخاب شده است، روی دکمه Clear All Constraints کلیک کنید (در شکل ۱۰.۸ نشان داده شده است). بلافاصله علامت هشدار قرمز را از جمله در سمت راست بالای صفحه، مشاهده خواهید کرد. برای نمایش علت مشکل روی آن کلیک کنید (شکل ۱۰.۷).

شکل ۱۰.۷ هشدارهای ConstraintLayout



هنگامی که ویوها محدودیت کافی ندارند، ConstraintLayout نمی‌تواند دقیقا بداند که آنها را باید کجا قرار دهد. TextViewها شما اصلا محدودیتی ندارد، بنابراین هر هشدار به شما می‌گوید که در زمان اجرا، این ویجت‌ها در مکان مناسب ظاهر نمی‌شوند.

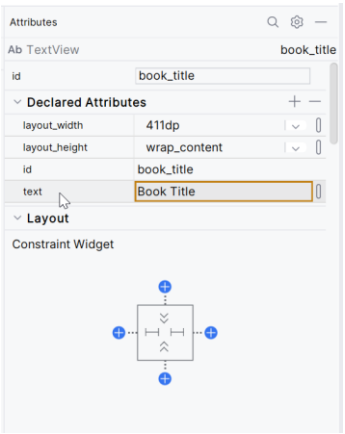
در ادامه محدودیت‌ها را برای رفع مشکلات بوجود آمده اضافه خواهیم کرد. به هشدارها توجه داشته باشید تا از رفتار غیرمنتظره در زمان اجرا جلوگیری کنید.

ایجاد فضا

باید مقداری فضا برای کار در ویرایشگر گرافیکی لیوت ایجاد کنیم. دو TextView کل منطقه را اشغال کرده‌اند که اتصال هر چیز دیگری را دشوار می‌کند. زمان کوچک کردن این دو ویجت است.

book_title را در درخت کامپوننت انتخاب و به پنجره ویژگی‌ها در سمت راست نگاه کنید (شکل ۱۰.۸). اگر این کادر برای شما باز نیست، روی تب Attributes در سمت راست کلیک کنید.

تصویر ۱۰.۸ عنوان ویژگی‌های TextView



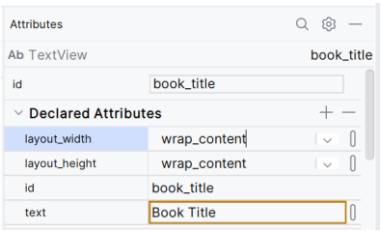
اندازه‌های عمودی و افقی TextView شما به ترتیب توسط تنظیم height و width کنترل می‌شود. می‌توان مقادیر آنها را روی یکی از سه تنظیمات اندازه ویو قرار داد که هر کدام مربوط به مقداری برای layout_width یا layout_height است.

جدول ۱۰.۱ مشاهده انواع تنظیمات‌اندازه

نوع	تنظیم	استفاده
fixed	Xdp	اندازه صریح (که تغییر نخواهد کرد) را برای ویو مشخص می‌کند.اندازه در واحدهای dp مشخص شده است.
Wrap content	wrap_content	اندازه "دلخواه" را به ویو اختصاص می‌دهد. برای TextView این بدان معنی است که به‌اندازه کافی بزرگ خواهد بود تا محتوای آن را در بر گیرد.
Match constraint	match_constraint	اجازه کشش را به ویو می‌دهد تا بتواند محدودیت‌های مشخص شده را برآورده کند.

هر دو TextView عنوان و تاریخ روی یک عرض ثابت تنظیم شده‌اند، به همین دلیل است که کل صفحه را اشغال می‌کنند. عرض و ارتفاع هر دوی این ویجت‌ها را تنظیم کنید. در حالی که book_title هنوز در درخت کامپونت‌ها انتخاب شده است روی تنظیمات عرض کلیک و آن را روی wrap_content قرار دهید. در صورت لزوم ارتفاع را نیز تنظیم کنید (تصویر ۱۰.۹).

شکل ۱۰.۹ تنظیم عرض و ارتفاع عنوان



این فرآیند را با ویجت book_date تکرار کنید تا عرض و ارتفاع آن تنظیم شود.

اکنون این دو ویجت اندازه مناسبی دارند (شکل ۱۰.۱۱). اما زمانی که برنامه اجرا می‌شود، همچنان همپوشانی خواهند داشت، زیرا هیچ محدودیتی روی آنها اعمال نشده است. توجه داشته باشید که موقعیتی که در پیش‌نمایش می‌بینید با آنچه هنگام اجرای برنامه آشکار می‌شود، متفاوت است.

تصویر ۱۰.۱۱ TextView با اندازه صحیح

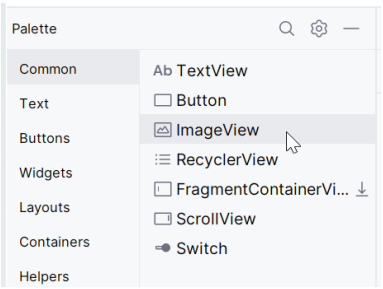


بعدا محدودیت‌هایی را برای موقعیت صحیح TextViewها اضافه خواهیم کرد. ابتدا ویوی سومی را که نیاز داریم به لیوت اضافه می‌کنیم.

افزودن ویجت‌ها

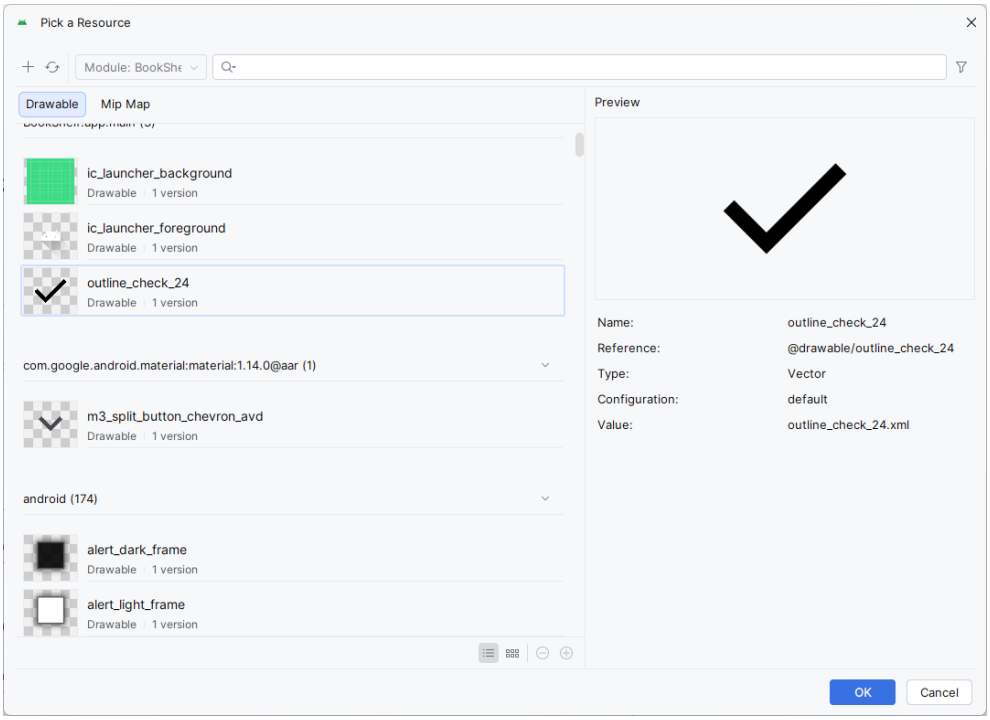
با کنار رفتن ویجت‌های قبلی می‌توانیم تصویر تیک را به لیوت خود اضافه کنیم. یک ImageView به فایل لیوت خود اضافه کنید. در پالت ImageView را در دسته common پیدا کنید (شکل ۱۰.۱۲). آن را به عنوان فرزند ConstraintLayout درست زیر book_date به داخل درخت کامپونت خود بکشید.

تصویر ۱۰.۱۲ پیدا کردن ImageView



در پنجره‌ای که باز می‌شود تب Drawable و سپس بخشی با عنوان پروژه خودمان را گسترش دهید و آیکونی را که ساخته‌اید، به عنوان منبع ImageView انتخاب کنید (شکل ۱۰.۱۳). از این تصویر برای نشان دادن اینکه کدام کتاب‌ها مطالعه‌شان پایان یافته، استفاده خواهیم کرد. روی OK کلیک کنید.

تصویر ۱۰.۱۳ انتخاب منبع ImageView



ImageView اکنون بخشی از لیوت شما است، اما هیچ محدودیتی ندارد. درست است که ویرایشگر گرافیکی لیوت به ImageView موقعیتی برای نمایش می‌دهد اما این موقعیت واقعا هیچ معنایی ندارد.

زمان اضافه کردن برخی محدودیت‌ها است. روی ImageView خود در قسمت پیش‌نمایش کلیک کنید. (ممکن است بخواهید پیش‌نمایش را وسعت ببخشید تا ظاهر بهتری داشته باشید. کنترل‌های بزرگنمایی در نوار ابزار بالای ابزارهای محدودیت قرار دارند.) در هر طرف ImageView نقطه‌هایی را خواهید دید (شکل ۱۰.۱۴). هر یک از این نقاط نشان دهنده یک هندل محدودیت (constraint handle) است.

شکل ۱۰.۱۴ محدودیت‌های ImageView



می‌خواهیم ImageView در سمت راست ویوی قرار گیرد. برای انجام این کار باید محدودیت‌هایی را از لبه‌های بالا، راست و پایین ImageView ایجاد کنید.

قبل از افزودن محدودیت‌ها ImageView را به سمت راست و پایین بکشید تا آن را از TextViewها دور کنید. نگران محل قرار گرفتن ImageView نباشید. پس از اعمال محدودیت‌ها ImageView دقیقا در همان جایی که شما می‌خواهید قرار می‌گیرد.

ابتدا یک محدودیت بین بالای ImageView و بالای ConstraintLayout تنظیم کنید. در پیش‌نمایش، هندل محدودیت بالایی را از ImageView به بالای ConstraintLayout بکشید. یک فلش از هندل بالای ImageView به بالای ConstraintLayout رسم می‌شود.

وقتی هندل آبی شد ماوس را رها کنید تا محدودیت ایجاد شود (شکل ۱۰.۱۵).

شکل ۱۰.۱۵ ایجاد یک محدودیت بالا



مراقب باشید که به طور ناخواسته محدودیت را به یکی از TextViewها متصل نکنید. اگر این کار را کردید، روی هندل محدودیت کلیک کنید تا حذف شود، سپس دوباره امتحان کنید.

هنگامی که محدودیت را اعمال کردید ویو در موقعیت خود قرار می‌گیرد. اینگونه ویوها را با تنظیم و حذف محدودیت‌ها در یک ConstraintLayout جابجا می‌کنیم.

با نگه داشتن ماوس روی ImageView خود مشاهده می‌کنید که دارای یک محدودیت بالایی به ConstraintLayout است.

همین کار را برای هندل محدودیت پایین انجام دهید. آن را از ImageView به پایین ویوی ریشه بکشید (شکل ۱۰.۱۶). دوباره مراقب باشید که از اتصال آن به TextViewها خودداری کنید.

شکل ۱۰.۱۶ ImageView با محدودیت‌های بالا و پایین



در نهایت هندل محدودیت سمت راست را از ImageView به سمت راست ویوی ریشه بکشید. محدودیت‌های شما باید مانند شکل ۱۰.۱۷ باشد.

شکل ۱۰.۱۷ سه محدودیت ImageView



ساز و کار داخلی ConstraintLayout

هر ویرایشی که با ویرایشگر گرافیکی لیوت انجام می‌دهید در XML پشت صحنه منعکس می‌شود. هنوز هم می‌توان XML خام را ویرایش کرد. اما ویرایشگر گرافیکی لیوت اغلب برای افزودن محدودیت‌ها آسان خواهد بود. ConstraintLayout نسبت به سایر ViewGroups جزئیات بیشتری دارد، بنابراین افزودن محدودیت‌های اولیه به صورت دستی می‌تواند کار سختی باشد. کار مستقیم با XML زمانی که نیاز به ایجاد تغییرات کوچک دارید، می‌تواند مفید باشد. (ابزارهای ویرایشگر گرافیکی به خصوص در کار با ConstraintLayout مفید هستند. با این حال همه از آن استفاده نمی‌کنند. انتخاب با شماست. در هر زمان می‌توانید بین ویرایشگر گرافیکی لیوت و ویرایش مستقیم XML جابجا شوید. از هر ابزاری که ترجیح می‌دهید برای ایجاد لیوت‌های این کتاب استفاده کنید.)

به قسمت Code بروید تا هنگام ایجاد محدودیت‌ها برای ImageView تغییرات اعمال شده در XML ببینید:

```
androidx.constraintlayout.widget.ConstraintLayout...>
    <ImageView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_marginTop="8dp"
    android:layout_marginEnd="8dp"
    android:layout_marginBottom="8dp"
    android:src="@drawable/done_outline_24"
    app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintTop_toTopOf="parent" />
    ...
</androidx.constraintlayout.widget.ConstraintLayout>
```

(همچنان خطاهای مربوط به دو TextView را مشاهده خواهید کرد. بعدا آنها را برطرف می‌کنیم.)

همه ویجت‌ها فرزندان مستقیم ConstraintLayout هستند – هیچ لیوت تودرتو وجود ندارد. ایجاد یک لیوت با LinearLayout همانند آنچه که با ConstraintLayout ساختید، بسیار پیچیده‌تر و نیازمند ساختار تودرتو است. همانطور که قبلا گفتیم، عدم استفاده از ساختار تودرتو زمان مورد نیاز برای رندر لیوت را کاهش می‌دهد و منجر به تجربه کاربری سریع و یکپارچه می‌شود.

نگاهی دقیق‌تر به محدودیت بالایی ImageView بیندازید:

```
app:layout_constraintBottom_toBottomOf="parent"
```

این ویژگی با _layout شروع می‌شود. تمام ویژگی‌هایی که با _layout شروع می‌شوند به عنوان پارامترهای لیوت شناخته می‌شوند. برخلاف سایر ویژگی‌ها، پارامترهای لیوت مسیرهایی به والد آن ویجت و نه خود ویجت هستند. آنها به لیوت والد می‌گویند که چگونه عنصر فرزند را در درون خود مرتب کند. تا کنون چند پارامتر لیوت مانند layout_width و layout_height را مشاهده کرده‌اید.

محدودیت constraintTop محدودیت اعمال شده به بالای ImageView ماست.

در نهایت ویژگی با "toTopOf="parent به پایان می‌رسد. این بدان معنی است که این محدودیت به لبه بالایی والد (ConstraintLayout) متصل است.

وقت آن است که XML را پشت سر گذاشته و به ویرایشگر گرافیکی لیوت بازگردید.

ویرایش ویژگی‌ها

ImageView شما اکنون به درستی سر جایش قرار گرفته است. مرحله بعدی: تنظیم موقعیت و سایز TextView عنوان است.

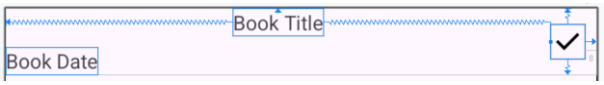
ابتدا Book Date را در پیش‌نمایش انتخاب و در محل دیگری قرار دهید. به یاد داشته باشید هر تغییری در موقعیت ویو در پیش‌نمایش هنگام اجرای نمایش داده نمی‌شود. در زمان اجرا فقط محدودیت‌ها باقی می‌مانند.

اکنون book_title را در درخت کامپوننت انتخاب کنید. می‌خواهیم عنوان کتاب در سمت چپ بالای لیوت و سمت چپ ImageView قرار گیرد. به سه محدودیت نیاز داریم:

- از سمت چپ ویو خود به سمت چپ والد
- از بالای ویو تا بالای والد
- از سمت راست ویو خود به سمت چپ ImageView

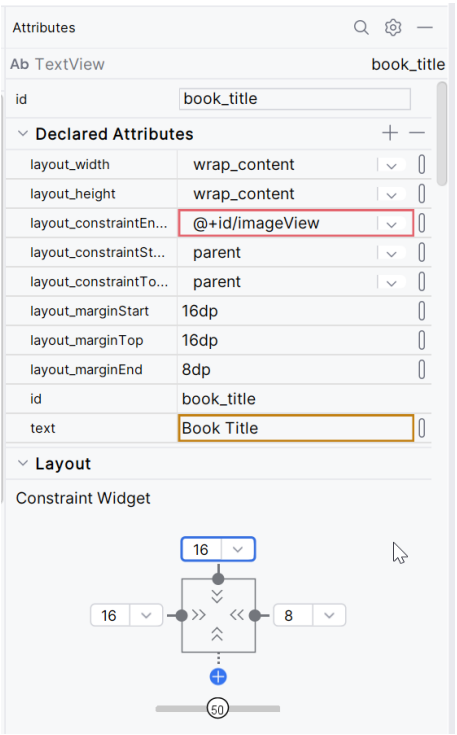
این محدودیت‌ها را اعمال کنید (شکل ۱۰.۱۸). اگر محدودیتی آنطور که انتظار داشتید کار نکرد، می‌توانید برای برگشت به عقب از کلید Ctrl-Z استفاده کنید.

شکل ۱۰.۱۸ محدودیت‌های TextView



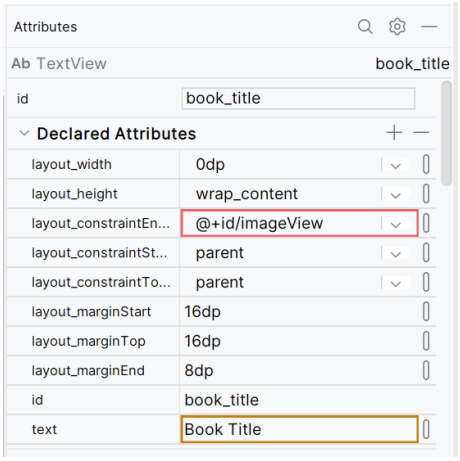
اکنون می‌خواهیم حاشیه‌هایی را به TextView اضافه کنیم. با انتخاب Book Title در قسمت پیش‌نمایش، صفحه ویژگی‌ها را در سمت راست بررسی کنید. از آنجایی که محدودیت‌هایی را به بالا، چپ و راست TextView اضافه کرده‌اید، منوهای کشویی به شما امکان انتخاب حاشیه را می‌دهند (شکل ۱۰.۲۵). dp۱۶ را برای حاشیه‌های چپ و بالا و dp۸ را برای حاشیه سمت راست انتخاب کنید.

تصویر ۱۰.۱۹ افزودن حاشیه به TextView



اکنون که محدودیت‌ها تنظیم شده‌اند، می‌توانیم TextView عنوان را به شکوه کامل خود بازگردانیم. تنظیمات layout_width آن را روی match_constraint تنظیم کنید تا به TextView عنوان اجازه دهید تمام فضای موجود در محدودیت‌های خود را پر کند. تنظیم layout_height را روی wrap_content تنظیم کنید. بررسی کنید که تنظیمات شما با تنظیمات نشان داده شده در تصویر ۱۰.۲۰ مطابقت داشته باشد.

تصویر ۱۰.۲۰ تنظیمات ویو book_title

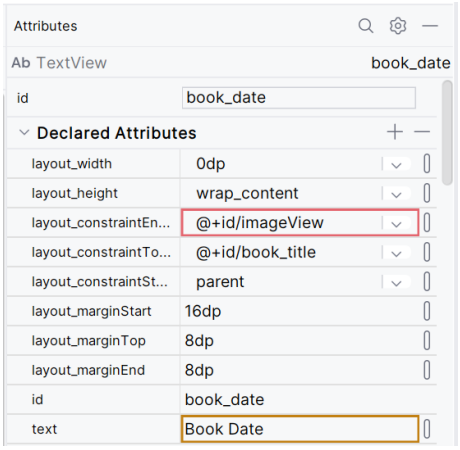


اکنون محدودیت‌هایی را به TextView تاریخ اضافه کنید. book_date را در درخت کامپوننت انتخاب کنید. مراحل TextView عنوان را تکرار کنید تا سه محدودیت زیر اضافه شود:

- از سمت چپ ویو خود به سمت چپ والد با حاشیه dp۱۶
- از بالای ویو شما تا پایین عنوان کتاب با حاشیه dp۸
- از سمت راست ویو خود به سمت چپ ImageView با حاشیه dp۸

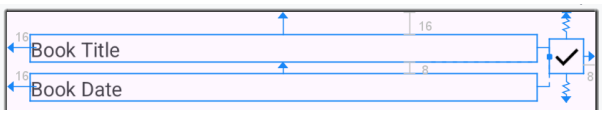
پس از افزودن محدودیت‌ها، ویژگی‌های TextView را تنظیم کنید. می‌خواهیم عرض و طول TextView تاریخ شما درست مانند TextView عنوان match_constraint و wrap_content باشد. بررسی کنید که تنظیمات شما با تنظیمات نشان داده شده در تصویر ۱۰.۲۱ مطابقت داشته باشد.

تصویر ۱۰.۲۱ تنظیمات book_date



لیوت شما در پیش‌نمایش باید شبیه به شکل ۱۰.۱ در ابتدای فصل باشد. از نزدیک، پیش‌نمایش شما باید با شکل ۱۰.۲۲ مطابقت داشته باشد.

شکل ۱۰.۲۲ محدودیت‌های نهایی

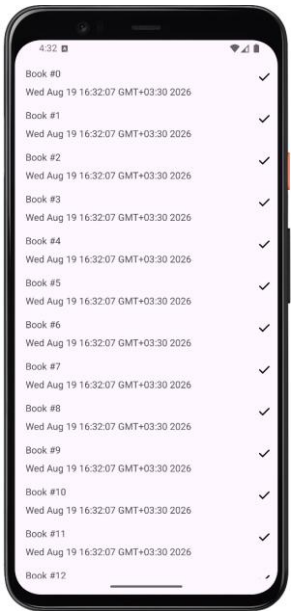


به ویو Code در پنجره ابزار ویرایشگر بروید تا XML حاصل از تغییراتی که در ویرایشگر گرافیکی لیوت ایجاد کرده‌اید را بررسی کنید. خطوط قرمز دیگر در زیر هر یک از برچسب‌های TextView ظاهر نمی‌شود. به این دلیل است که ویجت‌های TextView اکنون به اندازه کافی محدود شده‌اند، بنابراین ConstraintLayout که آنها در بر می‌گیرد می‌تواند بفهمد که ویجت‌ها را در زمان اجرا به درستی در کجا قرار دهد.

دو نشانگر هشدار زرد مرتبط با TextViewها باقی می‌ماند. هشدارها مربوط به رشته‌های استفاده شده است. برای BookShelf می‌توانید آنها را نادیده بگیرید. برای حذف این هشدارها بهتر است از منابع رشته‌ای استفاده کنید.

BookShelf را اجرا و بررسی کنید که هر سه کامپوننت به خوبی در ردیف خود در RecyclerView قرار می‌گیرند (شکل ۱۰.۲۳).

تصویر ۱۰.۲۳ اکنون با سه ویو در هر ردیف



پویا کردن آیتم‌های لیست

اکنون که لیوت شامل محدودیت‌های مناسب است ImageView را به روز کنید تا تیک فقط در کتاب‌های که مطالعه‌شان پایان یافته نمایش داده شود.

ابتدا ID، ImageView را به روز کنید. هنگام اضافه کردن ImageView به ConstraintLayout یک نام پیش‌فرض به آن داده شد. این نام خیلی گویا نیست. ImageView را انتخاب و در کادر ویژگی‌ها، مشخصه ID را به book_solved تغییر دهید. اندروید استودیو از شما می‌پرسد که تمام مواردی که از این ID استفاده کرده‌اید را به روز کند یا خیر. بله را انتخاب کنید.

ممکن است متوجه شده باشید که از شناسه‌های یکسانی در لیوت‌های مختلف استفاده می‌کنیم. شاید فکر کنید استفاده مجدد از شناسه‌ها مشکل ساز خواهد بود اما شناسه‌های لیوت فقط باید در یک لیوت منحصر به فرد باشند. از آنجایی که شناسه‌های شما در فایل‌های لیوت مختلف تعریف شده‌اند، استفاده از شناسه یکسان در هر دو مشکلی ایجاد نمی‌کند.

با وجود یک شناسه مناسب کد خود را به روز می‌کنیم. BookListFragment.kt را باز کرده و در BookHolder یک متغیر نمونه ImageView اضافه کنید و قابلیت مشاهده (visibility) آن را بر اساس وضعیت کتاب تغییر دهید.

لیست ۱۰.۱ به روز رسانی نمایش تیک (BookListFragment.kt)

```
private inner class BookHolder(view: View) :
RecyclerView.ViewHolder(view) {
    ...
    private val dateTextView: TextView =
        itemView.findViewById(R.id.book_date)
    private val solvedImageView: ImageView =
        itemView.findViewById(R.id.book_solved)
    init {
        …
    }
    fun bind(book: Book){
        this.book=book
        titleTextView.text=this.book.title
        dateTextView.text=this.book.date.toString()
        solvedImageView.visibility = if (book.isSolved) {
            View.VISIBLE
        } else {
            View.GONE
        }
    }
}
```

BookShelf را اجرا کنید. این بار تیک‌ها در هر ردیفی ظاهر نمی‌شوند.

اطلاعات بیشتر در مورد خصوصیات لیوت

اکنون چند ترفند دیگر در طراحی list_item_book.xml استفاده خواهیم کرد. در این فرآیند پاسخ برخی از سوالاتتان را در مورد ویجت‌ها و ویژگی‌ها خواهید یافت.

به design لیوت list_item_book.xml برگردید. book_title را انتخاب و برخی از ویژگی‌های آن را تنظیم کنید: بخش textAppearance را گسترش دهید و ویژگی textSize را روی sp۱۸ و ویژگی textColor را روی @android:color/black تنظیم کنید (شکل ۱۰.۲۴).

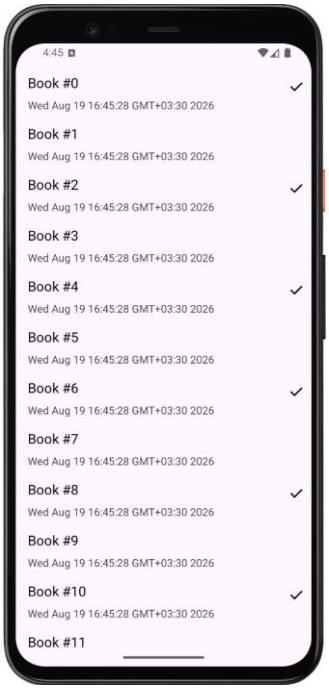
تصویر ۱۰.۲۴ به روز رسانی رنگ و اندازه عنوان



این مقادیر را می‌توانید مستقیما در فیلدهای کادر ویژگی‌ها تایپ کنید. از منوی کشویی (در مورد textSize) می‌توانید سایز را انتخاب و یا روی ... در کنار فیلد برای انتخاب ابعاد استفاده کنید.

BookShelf را اجرا کنید. شگفت زده خواهید شد. زیرا که همه چیز با یک لایه رنگ تازه بهتر به نظر می‌رسد (شکل ۱۰.۲۵).

شکل ۱۰.۲۶ رنگ تازه



استایل‌ها، تم‌ها و ویژگی‌های تم

یک استایل یک منبع XML است که حاوی ویژگی‌هایی است که نحوه ظاهر و رفتار یک ویجت را توصیف می‌کند. به عنوان مثال کدی که در ادامه آمده یک استایل است که ویجتی را با اندازه متن بزرگتر از حد معمول پیکربندی می‌کند:

```
<style name="BigTextStyle">
   <item name="android:textSize">20sp</item>
   <item name="android:padding">3dp</item>
</style>
```

می‌توانیم استایل‌های خود را ایجاد کنیم (و در فصل مربوطه این کار را خواهید کرد). آنها را به عنوان یک فایل styles در /res/values/ قرار داده و در لیوت‌های به آنها ارجاع دهیم: @style/my_own_style.

نگاهی دیگر به ویجت‌های TextView در layout/fragment_book.xml بیندازید (نه فایل list_item_book.xml که برای این فصل روی آن کار کرده‌اید). هر کدام دارای ویژگی استایل هستند که به استایلی که توسط اندروید ایجاد شده است اشاره دارد. این استایل خاص که از تم برنامه می‌آید باعث می‌شود TextViewها مانند جداکننده‌های لیست به نظر برسد. تم مجموعه‌ای از استایل‌ها است. از نظر ساختاری یک تم خود یک منبع استایل است که به صورت کلی بر ظاهر یک اپلیکیشن یا ویجت تاثیر می‌گذارد.

اندروید تم‌هایی را ارائه می‌دهد که برنامه‌های شما می‌توانند از آنها استفاده کنند. هنگامی که BookShelf را ایجاد کردید، ویزارد تمی را برای برنامه تنظیم کرد که در تگ application مانیفست به آن ارجاع داده شده است.

می‌توانیم با استفاده از ارجاع ویژگی استایل، استایلی از تم برنامه را روی ویجت اعمال کنیم. همان کاری است که در fragment_book.xml زمانی که از مقدار ?android:listSeparatorTextViewStyle به کار می‌بردیم، انجام دادیم.

در یک ارجاع ویژگی استایل به مدیر منابع زمان اجرا اندروید می‌گویید: "به تم برنامه بروید و ویژگی به نام listSeparatorTextViewStyle را پیدا کنید. این ویژگی به منبع استایل دیگری اشاره می‌کند. مقدار آن منبع را در اینجا قرار دهید."

هر تم اندروید شامل یک ویژگی به نام listSeparatorTextViewStyle خواهد بود، اما ظاهر آن بسته به ظاهر کلی متفاوت است. استفاده از ارجاع ویژگی استایل تضمین می‌کند که TextViewها ظاهر مناسبی برای برنامه شما داشته باشند.

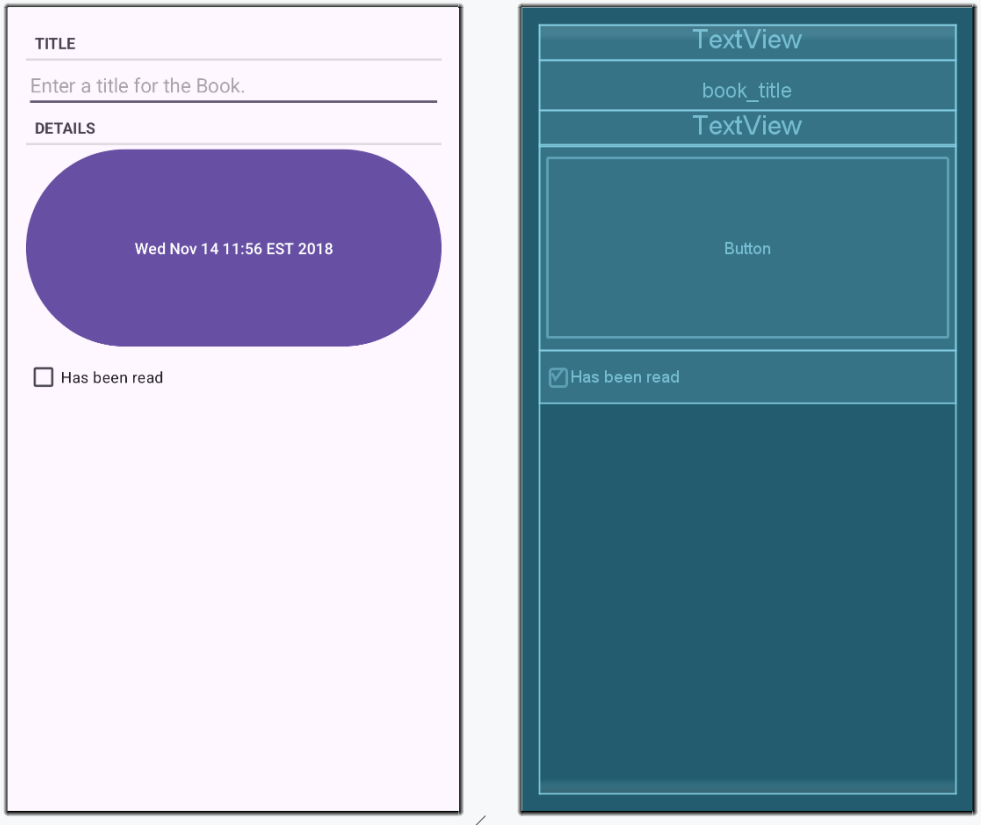
بیشتر بدانیم: Marginها در مقابل Padding

در هر دو برنامه GeoQuiz و BookShelf به ویجت‌ها margin و padding را اختصاص دادیم. توسعه دهندگان مبتدی گاهی اوقات در مورد این دو گیج می‌شوند. اکنون که متوجه شدید پارامتر layout چیست، توضیح تفاوت آنها آسانتر است.

ویژگی حاشیه (margin) پارامترهای لیوت هستند. آنها فاصله بین ویجت‌ها را تعیین می‌کنند. با توجه به اینکه یک ویجت فقط در مورد خودش اطلاعاتی دارد، تنظیم حاشیه‌ها باید بر عهده والد ویجت باشد.

از سوی دیگر Padding یک پارامتر لیوت نیست. ویژگی android:padding به ویجت می‌گوید که چقدر باید بزرگتر از محتویاتش، خودش را ترسیم کند. به عنوان مثال فرض کنید که می‌خواهیم دکمه تاریخ بدون تغییر اندازه متن آن به طرز چشمگیری بزرگ باشد (شکل ۱۰.۲۷).

شکل ۱۰.۲۷ من دکمه‌های بزرگ را دوست دارم و نمی‌توانم دروغ بگویم...



می‌توانید ویژگی زیر را به دکمه اضافه کنید:

```
<Button
    android:id="@+id/book_date"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    tools:text="Wed Nov 14 11:56 EST 2018"
    android:padding="80dp"/>
```

افسوس باید قبل از ادامه این ویژگی را حذف کنید.

بیشتر بدانیم: تحولات جدید در ConstraintLayout

ConstraintLayout دارای قابلیت‌های اضافی برای کمک به مرتب کردن ویوهای فرزند خود است. در این فصل ویوها را با محدود کردن آنها به والدین و همچنین سایر ویوهای خواهر و برادر مرتب کردیم.

ConstraintLayout همچنین شامل ویوهای کمکی مانند Guidelineها است، که تنظیم ویوها روی صفحه را ساده می‌کند.

Guidelineها روی صفحه برنامه نمایش داده نمی‌شوند. آنها فقط ابزاری هستند که به شما کمک می‌کنند ویوها را به روشی که نیاز دارید مرتب کنید. Guidelineهای افقی و عمودی هستند و می‌توان آنها را در یک مکان خاص روی صفحه با استفاده از مقادیر dp یا با تنظیم آنها به عنوان درصدی از صفحه قرار داد. ویوهای دیگر را می‌توان به Guideline محدود کرد تا اطمینان حاصل شود که در یک مکان خاص ظاهر می‌شوند، حتی اگر اندازه صفحه نمایش متفاوت باشد.

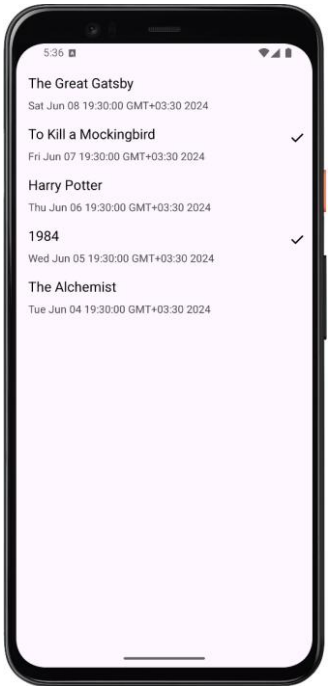
MotionLayout افزونه‌ای از ConstraintLayout است که افزودن انیمیشن‌ها به ویوهای شما را ساده می‌کند. برای استفاده از MotionLayout یک فایل MotionScene ایجاد می‌کنیم که نحوه اجرای انیمیشن‌ها را توضیح می‌دهد. همچنین می‌توانید Keyframeها را که ویوهای واسطه‌ای در انیمیشن ارائه می‌دهند، تنظیم کنید. MotionLayout می‌تواند از ویوی اولیه از طریق کی فریم‌های مختلفی که ارائه می‌دهید به لیوت پایانی متحرک کند.

۱۱

پایگاه داده‌ها و کتابخانه ROOM

تقریبا هر برنامه‌ای به مکانی برای ذخیره داده‌ها برای دراز مدت نیاز دارد. در این فصل یک پایگاه داده برای BookShelf پیاده‌سازی کرده و آن را با داده‌های ساختگی پر می‌کنیم. سپس برنامه را طوری به روز خواهیم کرد که داده‌های را از پایگاه داده بیرون کشیده و در لیست نمایش دهد (شکل ۱۱.۱).

تصویر ۱۱.۱ نمایش کتاب‌ها از پایگاه داده



در فصل ۴ چگونگی حفظ داده‌ها را در رویداد چرخش و همچنین مدیریت مرگ اکتیویتی را با استفاده از ViewModel و saved instance state یاد گرفتید. این دو رویکرد برای تعداد کمی از داده‌های مرتبط با رابط کاربری عالی هستند. با این حال این رویکردها نباید برای ذخیره داده‌های غیر رابط کاربری یا

داده‌هایی که به یک اکتیویتی یا نمونه فرگمنت مرتبط نیستند و بدون توجه به وضعیت رابط کاربری نیاز به تداوم دارند، استفاده شوند.

در عوض داده‌های غیر رابط کاربری خود را به صورت محلی (در سیستم فایل یا در یک پایگاه داده، همانطور که برای BookShelf انجام می‌دهیم) یا در یک وب سرور ذخیره کنید.

کتابخانه Room

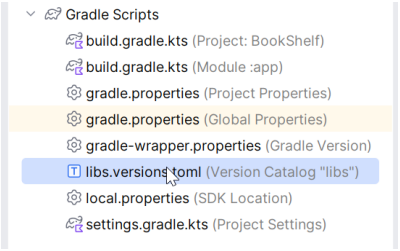
Room یک کتابخانه Jetpack است که راه اندازی و دسترسی به پایگاه داده را ساده می‌کند. امکان ایجاد ساختار و پرس و جوهای (Queries) پایگاه داده را با استفاده از کلاس‌های کاتلین فراهم می‌آورد.

Room از یک API، حاشیه‌نویسی (annotation) یا اعلان و یک کامپایلر تشکیل شده است. API شامل کلاس‌هایی است که برای تعریف پایگاه داده و ساخت نمونه‌ای از آن به کار می‌رود. حاشیه‌نویسی برای نشان دادن مواردی مانند این که کدام کلاس‌ها باید در پایگاه داده ذخیره شوند، کدام کلاس پایگاه داده شما را نشان می‌دهد و کدام کلاس‌ها مختص توابع جانبی (accessor) به جداول پایگاه داده می‌باشند، به کار می‌رود. کامپایلر کلاس‌های حاشیه‌نویسی شده را پردازش و پیاده‌سازی پایگاه داده شما را تولید می‌کند.

برای استفاده از Room ابتدا باید وابستگی‌های مورد نیاز را اضافه کنیم. اما برای اینکار باید سه فایل رو تغییر بدیم:

اول، فایل libs.versions.toml. برای تغییر این فایل به قسمت Gradle Scripts پروژه در پنجره ابزار پروژه بروید (شکل پایین). خط‌های بولد اضافه شده همانند لیست ۱۱.۱ را اضافه نمایید. بعد از هر مرحله حتما پروژه را همگام سازی کنید.

تصویر ۱۱.۲ lib



لیست ۱۱.۱ فایل lib

```
[versions]
...
ksp = "2.3.6"

[plugins]
...
ksp = { id = "com.google.devtools.ksp", version.ref = "ksp" }
```

دوم، فایل build.gradle.kts پروژه. دقت کنید این فایل مربوط به ماژول نیست. تغییرات را همانند لیست ۱۱.۲ اضافه نمایید.

لیست ۱۱.۲ KSP

```
plugins {
    alias(libs.plugins.android.application) apply false
    alias(libs.plugins.ksp) apply false
}
```

سوم، فایل build.gradle.kts ماژول برنامه. همانند لیست ۱۱.۳ خط‌های بولد را اضافه نمایید. لیست ۱۱.۳ افزودن وابستگی‌ها (app/build.gradle)

```
plugins {
    alias(libs.plugins.android.application)
    alias(libs.plugins.ksp)
}

...
    implementation
       ("androidx.recyclerview:recyclerview:1.4.0")
    implementation ("androidx.room:room-runtime:2.8.4")
    ksp("androidx.room:room-compiler:2.6.1")
    testImplementation(libs.junit)
    ...
}
```

بالای فایل یک پلاگین (plug-in) جدید اضافه کردیم. پلاگین‌ها راهی برای افزودن قابلیت به IDE هستند. KSP مخفف "Kotlin Symbole Processor Tool" است. هنگامی که کتابخانه‌ها کد تولید می‌کنند، اغلب می‌خواهیم از آن کلاس‌های تولید شده مستقیما در کد خود استفاده کنیم. اما به طور پیش‌فرض آن کلاس‌های برای اندروید استودیو قابل مشاهده نیست. بنابراین اگر بخواهیم آنها را وارد کنیم، خطایی مشاهده خواهیم کرد. افزودن این پلاگین فایل‌های تولید شده توسط کتابخانه را برای اندروید استودیو قابل مشاهده می‌کند تا بتوانیم آنها را در کلاس‌های خود وارد کنیم. تغییر دو فایل دیگر برای فعال سازی KSP بود.

اولین وابستگی، room-runtime برای Room API است که شامل تمام کلاس‌ها و حاشیه‌نویسی‌هایی است که برای تعریف پایگاه داده خود نیاز داریم. وابستگی دوم، room-compiler برای کامپایلر Room است که پیاده‌سازی پایگاه داده را بر اساس حاشیه‌نویسی‌هایی ما انجام می‌دهد. کامپایلر به جای implementation از کلمه کلیدی ksp استفاده می‌کند، به طوری که کلاس‌های تولید شده از کامپایلر به لطف افزونه ksp برای اندروید استودیو قابل مشاهده هستند.

همگام سازی فایل‌های Gradle خود را فراموش نکنید. با اعمال وابستگی‌ها می‌توانیم به آماده سازی لایه مدل خود برای ذخیره‌سازی در پایگاه اقدام کنیم.

ایجاد یک پایگاه داده

سه مرحله برای ایجاد پایگاه داده با Room وجود دارد:

۱. حاشیه‌نویسی کلاس مدل خود برای تبدیل آن به یک موجودیت پایگاه داده
۲. ایجاد کلاسی که خود پایگاه داده را نشان می‌دهد
۳. ایجاد یک مبدل نوع به طوری که پایگاه داده بتواند داده مدل را مدیریت کند

Room هر یک از این مراحل را ساده می‌کند.

تعریف موجودیت‌ها (entities)

Room جداول پایگاه داده را برای برنامه ما بر اساس موجودیتی‌هایی که تعریف می‌کنیم، می‌سازد. موجودیت‌ها، کلاس‌های مدلی هستند که ما ایجاد و با عنوان Entity@ حاشیه‌نویسی می‌کنیم. Room یک جدول پایگاه داده برای هر کلاس با این حاشیه‌نویسی ایجاد می‌کند.

از آنجایی که می‌خواهیم آبجکت‌ها کتاب را در پایگاه داده خود ذخیره کنیم، Book را به عنوان یک موجودیت Room به روز می‌کنیم.

Book.kt را باز و دو حاشیه‌نویسی را به آن اضافه کنید:

فهرست ۱۱.۴ تبدیل کتاب به یک موجودیت (Book.kt)

```
...
import androidx.room.Entity
import androidx.room.PrimaryKey
@Entity
data class Book(@PrimaryKey val id: UUID = UUID.randomUUID(),
                var title: String = "",
                var date: Date = Date(),
                var isSolved: Boolean = false)
```

اولین حاشیه‌نویسی، Entity@ در سطح کلاس اعمال می‌شود. اگر این حاشه‌نویسی همراه با خطا بود ایمپورت زیر را انجام دهید و یا از Alt+Enter استفاده نمایید.

```
import androidx.room.Entity
import androidx.room.PrimaryKey
```

این حاشیه‌نویسی نشان می‌دهد که کلاس ساختار یک جدول یا مجموعه‌ای از جداول را در پایگاه داده تعریف می‌کند. در این حالت، هر ردیف در جدول نشان دهنده یک کتاب خواهد بود. هر ویژگی تعریف شده در کلاس یک ستون در جدول خواهد بود که نام ویژگی به عنوان نام ستون است. جدولی که کتاب‌ها شما را ذخیره می‌کند دارای چهار ستون خواهد بود: id، title، date و isSolved.

حاشیه‌نویسی دیگر @PrimaryKey است که به ویژگی id اضافه کردیم. این حاشیه‌نویسی مشخص می‌کند که کدام ستون در پایگاه داده شما کلید اصلی است. کلید اصلی در یک پایگاه داده ستونی است که داده‌هایی را در خود جای می‌دهد که برای هر ورودی یا ردیف منحصر به فرد است، تا بتوان از آن برای جستجوی ورودی‌های جداگانه استفاده کرد. ویژگی id برای هر کتاب منحصر به فرد است، بنابراین با افزودن @PrimaryKey به این ویژگی می‌توانید با استفاده از شناسه یک کتاب آن را در پایگاه داده جستجو کنید.

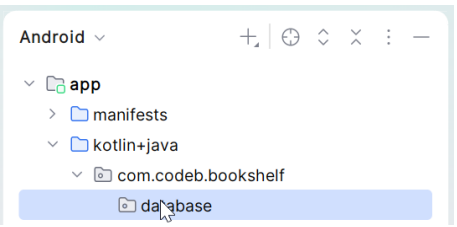
اکنون که کلاس Book حاشیه‌نویسی شده است، می‌توانیم به سراغ ایجاد کلاس پایگاه داده خود برویم.

ایجاد یک کلاس پایگاه داده

کلاس‌های موجودیت، ساختار جداول پایگاه داده را تعریف می‌کنند. اگر برنامه شما بیش از یک پایگاه داده داشته باشد، می‌توان از یک کلاس موجودیت واحد در چندین پایگاه داده استفاده کرد. این مورد رایج نیست، اما امکان پذیر است. به همین دلیل، یک کلاس Entity توسط Room برای ایجاد یک جدول استفاده نمی‌شود، مگر اینکه صریحا آن را با یک پایگاه داده مرتبط کنیم که به زودی انجام خواهیم داد.

ابتدا یک پکیج جدید به نام database برای کدهای مختص پایگاه داده ایجاد کنید. در پنجره ابزار پروژه، روی پوشه com.codeb.bookshelf کلیک راست کرده و New → Package را انتخاب کنید. پکیج جدید خود را database نامگذاری کنید.

تصویر ۱۱.۳ پایگاه داده



اکنون یک کلاس جدید به نام BookDatabase در پکیج database ایجاد و کلاس را مطابق شکل زیر تعریف کنید.

لیست ۱۱.۵ کلاس BookDatabase اولیه (database/BookDatabase.kt)

```
package com.codeb.bookshelf.database
import androidx.room.Database
import androidx.room.RoomDatabase
import com.codeb.bookshelf.Book
@Database(entities = [ Book::class ], version=1)
abstract class BookDatabase : RoomDatabase() {
}
```

حاشیه‌نویسی @Database به Room می‌گوید که این کلاس یک پایگاه داده را در برنامه ما نشان می‌دهد. حاشیه‌نویسی به دو پارامتر نیاز دارد. اولین پارامتر لیستی از کلاس‌های موجودیت است که به Room می‌گوید هنگام ایجاد و مدیریت جداول برای این پایگاه داده از کدام کلاس‌های موجودیت استفاده کند. در این مورد، ما فقط از کلاس Book استفاده می‌کنیم، زیرا تنها موجودیت در برنامه است.

پارامتر دوم نسخه پایگاه داده است. هنگامی که برای اولین بار یک پایگاه داده ایجاد می‌کنیم، نسخه باید ۱ باشد. همانطور که برنامه خود را در آینده توسعه می‌دهیم، می‌توانیم موجودیت‌های جدید و ویژگی‌های جدیدی را به موجودیت‌های موجود اضافه کنیم. هنگامی که این اتفاق می‌افتد، باید لیست موجودیت‌های خود را تغییر دهیم و نسخه پایگاه داده خود را افزایش دهیم تا به Room بگوییم چیزی تغییر کرده است. (این کار را در فصل مربوطه انجام خواهیم داد.)

خود کلاس پایگاه داده در این مرحله خالی است. BookDatabase از RoomDatabase گسترش می‌یابد و به عنوان database علامت گذاری شده است، بنابراین نمی‌توانید مستقیما نمونه‌ای از آن بسازید. یاد خواهید گرفت که چگونه از Room برای دریافت یک نمونه پایگاه داده استفاده کنید.

ایجاد یک مبدل نوع (type converter)

Room از SQLite در پشت صحنه استفاده می‌کند. SQLite یک پایگاه داده رابطه‌ای منبع باز مانند MySQL یا PostgreSQL است. (SQL مخفف Structured Query Language و یک زبان استاندارد است که برای تعامل با پایگاه‌های داده استفاده می‌شود.) برخلاف سایر پایگاه‌های داده SQLite داده‌های خود را در فایل‌های ساده‌ای ذخیره می‌کند که می‌توان با استفاده از کتابخانه SQLite خواند و نوشت. اندروید این کتابخانه SQLite را در کتابخانه استاندارد خود به همراه برخی از کلاس‌های کمکی اضافی قرار می‌دهد.

Room استفاده از SQLite را آسان‌تر و شفاف‌تر می‌کند و به عنوان یک لایه نگاشت شیء رابطه‌ای (یا ORM) بین آبجکت‌های کاتلین و پیاده‌سازی پایگاه داده عمل می‌کند. در بیشتر موارد هنگام استفاده از Room نیازی به دانستن یا اهمیت دادن به SQLite ندارید، اما اگر می‌خواهید بیشتر یاد بگیرید، می‌توانید به www.sqlite.org مراجعه کرده و مستندات کامل SQLite را دریافت کنید.

Room قادر است انواع اولیه را به راحتی در جداول پایگاه داده SQLite ذخیره کند، اما نوع‌های دیگر مشکلاتی ایجاد می‌کنند. کلاس Book ما به آبجکت‌ها Date و UUID متکی است که Room نمی‌داند چگونه به طور پیش‌فرض آنها را ذخیره کند. ما باید به پایگاه داده کمک کنیم تا بداند چگونه این انواع را ذخیره کند و چگونه آنها را به درستی از جدول پایگاه داده خارج کند.

برای اینکه به Room بگوییم چگونه انواع داده‌های خود را تبدیل کند، یک مبدل نوع مشخص می‌کنیم. یک مبدل نوع به Room می‌گوید که چگونه یک نوع خاص را به قالبی که باید در پایگاه داده ذخیره شود، تبدیل کند. ما به دو تابع نیاز داریم که برای هر نوع با @TypeConverter حاشیه‌نویسی خواهیم کرد: یکی به Room می‌گوید که چگونه نوع را تبدیل کند تا در پایگاه داده ذخیره شود و دیگری چگونگی تبدیل داده از پایگاه به نوع اولیه را مشخص می‌کند.

یک کلاس به نام BookTypeConverters در پکیج database ایجاد و در هر کدام دو تابع برای انواع Date و UUID اضافه کنید.

لیست ۱۱.۶ افزودن توابع TypeConverter (database/BookTypeConverters.kt)

```
package com.codeb.bookshelf.database
import androidx.room.TypeConverter
import java.util.Date
import java.util.UUID
class BookTypeConverters {
    @TypeConverter
    fun fromDate(date: Date?): Long? {
        return date?.time
    }
    @TypeConverter
    fun toDate(millisSinceEpoch: Long?): Date? {
        return millisSinceEpoch?.let {
            Date(it)
        }
    }
    @TypeConverter
    fun toUUID(uuid: String?): UUID? {
        return UUID.fromString(uuid)
    }
    @TypeConverter
```

```
    fun fromUUID(uuid: UUID?): String? {
        return uuid?.toString()
    }
}
```

دو تابع اول آبجکت‌ها Date و دو تابع دوم UUID را مدیریت می‌کند. مطمئن شوید که نسخه java.util.Date کلاس Date را وارد کرده‌اید.

اعلام توابع مبدل پایگاه داده ما را قادر به استفاده از آنها نمی‌کند. باید صریحا مبدل‌ها را به کلاس پایگاه داده خود اضافه کنیم.

لیست ۱۱.۷ فعال کردن TypeConverters (database/BookDatabase.kt)

```
…
import androidx.room.TypeConverters
…
@Database(entities = [ Book::class ], version=1)
@TypeConverters(BookTypeConverters::class)
abstract class BookDatabase : RoomDatabase() {
}
```

با افزودن حاشیه‌نویسی @TypeConverters و ارسال BookTypeConverters به پایگاه داده می‌گوییم که هنگام تبدیل نوع از توابع آن کلاس استفاده کند.

با این کار پایگاه داده و تعاریف جدول کامل می‌شود.

تعریف آبجکت دسترسی به داده (Data Access Object)

یک جدول پایگاه داده اگر نتوانیم محتوای آن را ویرایش یا به آن دسترسی داشته باشیم، فایده چندانی ندارد. اولین قدم برای تعامل با جداول پایگاه داده ایجاد یک شی دسترسی به داده یا DAO است. DAO شامل توابعی برای هر عملیات پایگاه داده است. در این فصل DAO برنامه ما به دو تابع پرس و جو نیاز دارد: یکی برای بازگرداندن لیستی از تمام کتاب‌ها موجود در پایگاه داده و دیگری برای بازگرداندن یک کتاب واحد مطابق با یک UUID معین.

فایلی با نام BookDao.kt را به پکیج database اضافه کنید. در آن، یک رابط خالی به نام BookDao با حاشیه‌نویسی Dao@ تعریف کنید.

لیست ۱۱.۸ ایجاد یک DAO خالی (database/BookDao.kt)

```
package com.codeb.bookshelf.database
import androidx.room.Dao
@Dao
interface BookDao {
}
```

حاشیه‌نویسی Dao@ به Room می‌فهماند که BookDao یکی از آبجکت‌ها دسترسی به داده‌های ماست. هنگامی که BookDao را به کلاس پایگاه داده خود متصل می‌کنیم، Room پیاده‌سازی توابعی را که به این رابط اضافه می‌کنیم، تولید می‌کند.

دو تابع پرس و جو را به BookDao اضافه کنید.

لیست ۱۱.۹ افزودن توابع کوئری پایگاه داده (database/BookDao.kt)

```
package com.codeb.bookshelf.database
import androidx.room.Dao
import androidx.room.Query
import com.codeb.bookshelf.Book
import java.util.UUID
@Dao
interface BookDao {
    @Query("SELECT * FROM Book")
    fun getBooks(): List<Book<
    @Query("SELECT * FROM Book WHERE id=(:id)")
    fun getBook(id: UUID): Book?
}
```

حاشیه‌نویسی @Query نشان می‌دهد که getBooks() و getBook(UUID) به جای درج، به روز رسانی یا حذف موارد از پایگاه داده، برای بیرون کشیدن اطلاعات از پایگاه داده مورد استفاده قرار می‌گیرد. نوع بازگشتی هر تابع پرس و جو در رابط DAO نشان دهنده نوع نتیجه‌ای است که پرس و جو برمی‌گرداند.

SELECT * FROM BOOK از Room می‌خواهد که تمام ستون‌ها را برای همه ردیف‌های جدول پایگاه داده Book برگرداند. SELECT* FROM Book WHERE id=(:id) از Room می‌خواهد که همه ستون‌ها را فقط از ردیفی که شناسه آن با مقدار شناسه ارائه شده مطابقت دارد برگرداند.

با این کار BookDao به صورت حداقلی در حال حاضر کامل شده است. در فصل ۱۲ یک تابع برای به روز رسانی یک کتاب موجود و در فصل ۱۴ یک تابع برای درج یک کتاب جدید اضافه خواهیم کرد.

در مرحله بعد باید کلاس DAO را در کلاس پایگاه داده خود ثبت کنیم. از آنجایی که BookDao یک interface است، Room تولید نسخه اصلی کلاس را برای شما انجام می‌دهد. اما برای اینکه این کار انجام شود، باید به کلاس پایگاه داده خود بگوییم که نمونه‌ای از DAO را تولید کند.

برای اتصال DAO، BookDatabase.kt را باز و یک تابع انتزاعی اضافه کنید که BookDao را به عنوان نوع بازگشتی دارد.

لیست ۱۱.۱۰ ثبت DAO در پایگاه داده (database/BookDatabase.kt)

```
@Database(entities = [ Book::class ], version=1)
@TypeConverters(BookTypeConverters::class)
abstract class BookDatabase : RoomDatabase() {
    abstract fun BookDao(): BookDao
}
```

هنگامی که پایگاه داده ایجاد می‌شود، Room یک پیاده‌سازی مشخص از DAO ایجاد می‌کند که می‌توانیم به آن دسترسی داشته باشید. هنگامی که به DAO ارجاع دادیم، می‌توانیم هر یک از توابع تعریف شده روی آن را برای تعامل با پایگاه داده خود فراخوانی کنید.

دسترسی به پایگاه داده با استفاده از الگو Repository

برای دسترسی به پایگاه داده از الگو repository توصیه شده توسط گوگل استفاده خواهیم کرد.

یک کلاس repository دسترسی به داده‌ها از یک منبع واحد یا مجموعه‌ای از منابع را در بر می‌گیرد. چگونگی واکشی و ذخیره‌سازی مجموعه خاصی از داده‌ها را چه به صورت محلی در یک پایگاه داده و چه از یک سرور راه دور تعیین می‌کند. کد رابط کاربری شما تمام داده‌ها را از repository درخواست می‌کند،

زیرا رابط کاربری اهمیتی نمی‌دهد که داده‌ها چگونه ذخیره یا واکشی می‌شوند. اینها جزئیات پیاده‌سازی خود repository است.

از آنجایی که BookShelf یک برنامه ساده است، repository فقط واکشی داده‌ها را از پایگاه داده انجام خواهد داد.

یک کلاس به نام BookRepository در پکیج com.codeb.bookshelf ایجاد وcompanion object را در کلاس تعریف کنید.

لیست ۱۱.۱۱ پیاده‌سازی یک repository (BookRepository.kt)

```
package com.codeb.bookshelf
import android.content.Context
class BookRepository private constructor(context: Context) {
    companion object {
        private var INSTANCE: BookRepository? = null
        fun initialize(context: Context) {
            if (INSTANCE == null) {
                INSTANCE = BookRepository(context)
            }
        }
        fun get(): BookRepository {
            return INSTANCE ?:
            throw IllegalStateException(
                "BookRepository must be initialized")
        }
    }
}
```

BookRepository یک singleton است. این بدان معناست که تنها یک نمونه از آن در فرآیند برنامه شما وجود خواهد داشت.

یک singleton تا زمانی که برنامه در حافظه باقی بماند وجود دارد، بنابراین ذخیره هر گونه ویژگی در singleton باعث می‌شود در طول هر تغییر چرخه حیات در اکتیویتی‌ها و فرگمنت‌ها ویژگی در دسترس باشد. مراقب کلاس‌های singleton باشید زیرا زمانی که اندروید برنامه شما را از حافظه حذف می‌کند، از بین می‌روند.

BookRepository راه حلی برای ذخیره طولانی مدت داده‌ها نیست. در عوض برای برنامه، مالکیتی روی داده‌های کتاب به وجود می‌آورد و راهی برای انتقال آسان آن داده‌ها بین کلاس‌های کنترلر فراهم می‌کند.

برای تبدیل BookRepository به یک singleton، دو تابع به companion object آن اضافه می‌کنیم. یکی نمونه جدیدی از repository را مقداردهی اولیه می‌کند و دیگری به repository دسترسی پیدا

می‌کند. همچنین سازنده را به عنوان private علامت گذاری کنید تا مطمئن شوید که هیچ کامپوننتی نتواند نمونه خودش را ایجاد کند.

تابع getter اگر قبل از آن ()initialize را فراخوانی نشده باشد، یک IllegalStateException را موجب می‌شود. بنابراین باید مطمئن شویم که هنگام شروع برنامه، repository خود را مقداردهی اولیه می‌کنیم.

برای انجام امور به محض آماده شدن برنامه می‌توانیم یک زیر کلاس Application ایجاد کنیم. این کار به ما امکان می‌دهد به اطلاعات چرخه حیات در مورد خود برنامه دسترسی داشته باشید. کلاسی به نام BookShelfApplication که Application را گسترش می‌دهد و Application.onCreate() را برای راه اندازی مقداردهی اولیه repository بازنویسی می‌کند، ایجاد کنید.

لیست ۱۱.۱۲ ایجاد یک زیر کلاس اپلیکیشن (BookShelfApplication.kt)

```
package com.codeb.bookshelf
import android.app.Application
class BookShelfApplication : Application() {
    override fun onCreate() {
        super.onCreate()
        BookRepository.initialize(this)
    }
}
```

مشابه Activity.onCreate(...)، Application.onCreate() نیز زمانی که برنامه شما برای اولین بار در حافظه بارگذاری شد، توسط سیستم فراخوانی می‌شود. به همین دلیل این نقطه مکان خوبی برای انجام هر نوع عملیات اولیه است. نمونه application مانند اکتیویتی یا کلاس‌های فرگمنت به طور مداوم از بین نمی‌رود و دوباره ایجاد نمی‌شود. در هنگام راه اندازی برنامه ایجاد و زمان مرگ فرآیند از بین می‌رود. تنها تابع چرخه حیاتی که در BookShelf بازنویسی خواهیم کرد ()onCreate است.

می‌خواهیم نمونه application را به عنوان یک شی Context به repository خود ارسال کنیم. این شی تا زمانی که فرآیند شما در حافظه باشد معتبر است، بنابراین نگه داشتن ارجاع به آن در کلاس repository ایمن است.

اما برای اینکه کلاس اپلیکیشن شما توسط سیستم استفاده شود، باید آن را در مانیفست خود اعلام کنید. AndroidManifest.xml را باز و ویژگی android:name را برای راه اندازی اپلیکیشن خود مشخص کنید.

لیست ۱۱.۱۳ اتصال به زیر کلاس اپلیکیشن (manifests/AndroidManifest.xml)

```
<manifest
```

```
xmlns:android="http://schemas.android.com/apk/res/android">
    <application
        android:name=".BookShelfApplication"
        ...
    </application>
</manifest>
```

با ثبت کلاس در مانیفست، سیستم عامل هنگام راه اندازی برنامه، نمونه‌ای از BookShelfApplication ایجاد می‌کند. سپس سیستم عامل ()onCreate را در نمونه BookShelfApplication فراخوانی می‌کند. BookRepository ما مقداردهی اولیه می‌شود و می‌توانیم از سایر مؤلفه‌های خود به آن دسترسی داشته باشید.

در مرحله بعد دو ویژگی را در BookRepository خود اضافه کنید تا ارجاعات پایگاه داده و آبجکت‌ها DAO خود را ذخیره کنید.

لیست ۱۱.۱۴ تنظیم ویژگی‌های repository (BookRepository.kt)

```
...
import androidx.room.Room
import com.codeb.bookshelf.database.BookDatabase
private const val DATABASE_NAME = "book-database"
class BookRepository private constructor(context: Context) {
    private val database : BookDatabase =
        Room.databaseBuilder(
            context.applicationContext,
            BookDatabase::class.java,
            DATABASE_NAME
        ).build()
    private val BookDao = database.BookDao()
    companion object {
        ...
        }
    }
}
```

Room.databaseBuilder() با استفاده از سه پارامتر، یک پیاده‌سازی مشخص از BookDatabase انتزاعی ما ایجاد می‌کند. ابتدا به یک شی Context نیاز دارد، زیرا پایگاه داده به سیستم فایل دسترسی دارد. Context برنامه را به آن پاس می‌کنیم زیرا همانطور که در بالا توضیح داده شد، singleton به احتمال زیاد بیشتر از هر یک از کلاس‌های اکتیویتی شما عمر خواهد کرد.

پارامتر دوم کلاس پایگاه داده‌ای است که می‌خواهیم Room ایجاد کند. سومین پارامتر نام فایل پایگاه داده‌ای است که می‌خواهیم Room برای ما ایجاد کند. از یک ثابت رشته خصوصی که در همان فایل تعریف شده است استفاده می‌کنیم، زیرا هیچ کامپوننت دیگری نیازی به دسترسی به آن ندارد.

در مرحله بعد BookRepository خود را کامل می‌کنیم تا سایر کامپوننت‌های بتوانند هر عملیاتی را که در پایگاه داده نیاز دارند، انجام دهند. برای هر تابع در DAO خود یک تابع به repository خود اضافه کنید.

لیست ۱۱.۱۵ افزودن توابع repository (BookRepository.kt)

```
…
import java.util.UUID
…
class BookRepository private constructor(context: Context) {
    ...
    private val BookDao = database.BookDao()
    fun getBooks(): List<Book> = BookDao.getBooks()
    fun getBook(id: UUID): Book? = BookDao.getBook(id)
    companion object {
        ...
        }
    }
}
```

از آنجایی که Room پیاده‌سازی‌های پرس و جو را در DAO فراهم می‌کند از repository خود به پیاده‌سازی آنها فراخوانی می‌کنیم. این کار باعث کوتاهی و قابل درک شدن repository ما می‌شود.

این طور به نظر می‌رسد که کار زیادی برای سود کم انجام دادیم. نگران نباشید، به زودی توابعی را برای کپسوله کردن کارهای اضافی که repository باید انجام دهد، اضافه خواهیم کرد.

تست پرس و جوها (Queries)

در حال حاضر BookListViewModel ما ۱۰۰ کتاب ساختگی برای نمایش در لیست ایجاد می‌کند. آن را حذف و با فراخوانی به تابع ()getBooks در BookRepository جایگزین کنید.

لیست ۱۱.۱۶ دسترسی به repository در ViewModel(BookListViewModel.kt)

```
class BookListViewModel: ViewModel() {
    val books = mutableListOf<Book>()
    init {
        for (i in 0 until 100) {
            val book = Book()
            book.title = "BOOK #$i"
            book.isSolved = i % 2 == 0
            books += book
        }
    }
```

```
    private val bookRepository = BookRepository.get()
    val books = bookRepository.getBooks()
}
```

اکنون برنامه خود را اجرا کنید تا نتیجه را مشاهده نمایید. برنامه از کار می‌افتد.

نگران نباشید، این رفتار مورد انتظار است. (حداقل، چیزی است که ما انتظار داشتیم.) به استثنا در Logcat نگاهی بیندازید تا ببینید چه اتفاقی افتاده است.

```
java.lang.IllegalStateException: Cannot access database on the
main thread since it may potentially lock the UI for a long
period of time.
```

این خطا از کتابخانه Room سرچشمه می‌گیرد. Room از تلاش شما برای دسترسی به پایگاه داده در نخ اصلی (main thread) فرآیند برنامه ناراضی است. در ادامه این فصل با مدل نخ بندی (threading model) که اندروید استفاده می‌کند و همچنین هدف نخ اصلی و ملاحظات مربوط به نوع کاری که روی نخ اصلی اجرا می‌کنید، آشنا خواهید شد. علاوه بر این تعاملات پایگاه داده خود را به یک نخ پس زمینه (background thread) محول خواهیم کرد. این کار Room را راضی و ما را از شر خطا خلاص می‌کند.

نخ‌های برنامه (Application Threads)

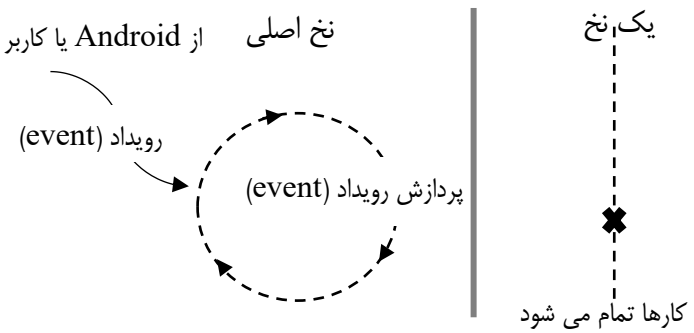
خواندن از پایگاه داده آنی اتفاق نمی‌افتد. از آنجایی که دسترسی ممکن است بسیار طول بکشد Room تمام عملیات پایگاه داده را در نخ اصلی مجاز نمی‌کند.

اگر این قانون را نقض کنید، Room IllegalStateException را که به تازگی دیده‌اید، ایجاد می‌کند.

چرا؟ برای درک آن باید بدانید که نخ چیست، نخ اصلی چیست و نخ اصلی چه کاری انجام می‌دهد.

نخ یک دنباله واحد از اجرا است. کدهای یک نخ، یکی پس از دیگری اجرا می‌شود. هر برنامه اندرویدی حیات خود را با یک نخ اصلی شروع می‌کند. با این حال نخ اصلی یک لیست از پیش تعیین شده از مراحل نیست. به همین دلیل در یک حلقه بی نهایت قرار می‌گیرد و منتظر رویدادهایی است که توسط کاربر یا سیستم آغاز می‌شود. سپس کدها را در پاسخ به آن رویدادها به محض وقوع اجرا می‌کند (شکل ۱۱.۶).

شکل ۱۱.۴ نخ‌های معمولی در مقابل نخ اصلی



تصور کنید که برنامه شما یک فروشگاه بزرگ کفش است و شما فقط یک کارمند با نام احمد دارید. کارهای زیادی برای راضی نگه داشتن مشتریان در یک فروشگاه وجود دارد: مرتب کردن کالا، آوردن کفش برای مشتریان، الصاق قیمت و ... . احمد برای انجام تمام امور به‌اندازه کافی سریع است، حتی اگر فقط یک نفر باشد. اما اگر محموله‌ای از کفش گم شود چه؟ کسی باید زمان زیادی را صرف صحبت با تلفن برای پیدا کردنش بکند. مشتریان شما در حالی که احمد در حالت پیدا کردن محموله گم شده است در انتظار خواهند ماند، سفارش خود را دریافت نخواهند کرد و به تدریج صبر و حوصله‌شان را از دست می‌دهند.

احمد همانند نخ اصلی در برنامه شما است. تمام کدهای لازم برای به روز رسانی رابط کاربری را اجرا می‌کند که شامل کدهایی مربوط به پاسخ رویدادهای مختلف مرتبط با رابط کاربری است – راه اندازی

اکتیویتی، فشار دادن دکمه و غیره. (از آنجایی که رویدادها همگی به نوعی به رابط کاربری مرتبط هستند، گاهی اوقات رشته اصلی را نخ رابط کاربری (UI thread) می‌نامند.)

حلقه رویداد کد رابط کاربری را به ترتیب اجرا می‌کند. در حالی که اطمینان حاصل می‌کند که کدها به موقع اجرا شوند، مطمئن می‌شود که هیچ یک از این عملیات روی یکدیگر نمی‌افتد. تا کنون تمام کدهایی که نوشته‌اید روی نخ اصلی اجرا شده است.

نخ‌های پس زمینه

دسترسی به پایگاه داده بسیار شبیه یک تماس تلفنی با توزیع کننده کفش شما است: در مقایسه با سایر کارها زمان زیادی می‌برد. در این مدت رابط کاربری پاسخگو نخواهد بود که ممکن است منجر به عدم پاسخگویی برنامه یا ANR (application not responding) شود.

ANR زمانی اتفاق می‌افتد که ناظر اندروید تشخیص دهد که رشته اصلی نتوانسته است به یک رویداد مهم پاسخ دهد، همانند فشار دادن دکمه بازگشت.

در فروشگاه خود مشکل را با (طبیعتا) استخدام یک کارمند دوم برای تماس با توزیع کننده کفش حل می‌کنید. در برنامه اندروید کاری مشابه انجام می‌دهیم – یک رشته پس زمینه ایجاد و از آنجا به پایگاه داده دسترسی پیدا می‌کنیم.

هنگام شروع افزودن نخ‌های پس زمینه به برنامه‌های خود باید دو قانون مهم را در نظر بگیریم:

- تمام کارهای طولانی مدت (long-running tasks) باید در یک نخ پس زمینه انجام شوند. این کار تضمین می‌کند که نخ اصلی برای انجام کارهای مرتبط با رابط کاربری آزاد و پاسخگو کاربر باشد.
- رابط کاربری فقط می‌تواند از نخ اصلی به روز شود. اگر بخواهیم رابط کاربری را از یک نخ پس زمینه تغییر دهیم، خطایی دریافت خواهیم کرد، بنابراین باید مطمئن شویم که هر داده تولید شده از یک نخ پس زمینه برای به روز رسانی رابط کاربری به رشته اصلی ارسال می‌شود.

برای BookShelf از دو گزینه برای اجرای فراخوانی‌های پایگاه داده خود در پس زمینه استفاده خواهیم کرد. در این فصل از LiveData و در فصل ۱۲ و ۱۴ از Executor برای درج و بروزرسانی داده‌ها استفاده خواهیم کرد.

استفاده از LiveData

LiveData یک کلاس نگهدارنده داده است که در کتابخانه lifecycle-livedata مربوط به Jetpack یافت می‌شود. Room برای کار با LiveData ساخته شده است. برای استفاده از LiveData وابستگی آن را اضافه کنید.

لیست ۱۱.۱۷ افزودن وابستگی

```
dependencies {
    ...
    ksp("androidx.room:room-compiler:2.6.1")
    implementation(
        "androidx.lifecycle:lifecycle-livedata-ktx:2.10.0")
    testImplementation(libs.junit)
    ...
}
```

هدف LiveData ساده کردن انتقال داده‌ها بین بخش‌های مختلف برنامه شما است. LiveData همچنین انتقال داده‌ها را بین نخ‌ها امکان پذیر می‌کند. این کار با رعایت قوانین مربوط به نخ‌ها که در بالا بیان کردیم مناسب است.

هنگامی که کوئری‌ها را در DAO خود پیکربندی می‌کنیم تا LiveData را برگردانیم، Room به طور خودکار آن عملیات پرس و جو را در یک نخ پس زمینه اجرا می‌کند و پس از اتمام پرس و جو نتایج را در شی LiveData منتشر می‌کند. می‌توانیم اکتیویتی یا فرگمنت خود را برای مشاهده شی LiveData تنظیم کنیم، در این صورت اکتیویتی یا فرگمنت شما در نخ اصلی پس از آماده شدن نتایج از آن مطلع می‌شود.

در این فصل ما بر استفاده از قابلیت بین نخی (cross-thread) LiveData برای انجام کوئری‌های پایگاه داده تمرکز خواهیم کرد. برای شروع BookDao.kt را باز و نوع بازگشت توابع کوئری خود را به روز کنید تا یک شی LiveData را برگرداند.

لیست ۱۱.۱۸ بازگرداندن LiveData در DAO (database/BookDao.kt)

```
…
import androidx.lifecycle.LiveData
…
@Dao
interface BookDao {
    @Query("SELECT * FROM Book")
    fun getBooks(): LiveData<List<Book>>
    @Query("SELECT * FROM Book WHERE id=(:id)")
    fun getBook(id: UUID): LiveData<Book?>
}
```

با برگرداندن نمونه‌ای از LiveData از کلاس DAO، به Room سیگنال می‌دهیم تا پرس و جو را روی یک نخ پس زمینه اجرا کند. هنگامی که پرس و جو کامل شد، شی LiveData ارسال داده‌های کتاب به نخ اصلی را انجام می‌دهد.

در مرحله بعد، BookRepository را به روز کنید تا LiveData را از توابع پرس و جو خود برگرداند.

لیست ۱۱.۱۹ بازگرداندن LiveData از repository (BookRepository.kt)

```
…
import androidx.lifecycle.LiveData
…
class BookRepository private constructor(context: Context) {
    …
    private val BookDao = database.BookDao()
    fun getBooks(): LiveData<List<Book>> = BookDao.getBooks()
    fun getBook(id: UUID): LiveData<Book?> =
        BookDao.getBook(id)
    companion object {
        …
        }
    }
}
```

نمایش LiveData

برای نمایش کتاب‌ها از پایگاه داده، BookListFragment را به روز کنید تا LiveData بازگردانده شده از ()BookRepository.getBooks را به نمایش بگذارد.

ابتدا BookListViewModel.kt را باز و نام ویژگی books را تغییر دهید تا مشخص شود که چه داده‌هایی را در خود جای داده است.

لیست ۱۱.۲۰ دسترسی به repository در ViewModel (BookListViewModel.kt)

```
class BookListViewModel: ViewModel() {
    private val bookRepository = BookRepository.get()
    ~~val books = bookRepository.getBooks()~~
    val bookListLiveData = bookRepository.getBooks()
}
```

در مرحله بعد BookListFragment را تغییر دهید تا این واقعیت را که BookListViewModel اکنون LiveData بازگردانده شده از repository را نمایش دهد، منعکس کند (فهرست ۱۱.۱۸). پیاده‌سازی onCreate(…) را حذف کنید، زیرا به BookListViewModel.Books اشاره می‌کند که دیگر وجود

ندارد. در updateUI() ارجاع به BookListViewModel.Books را حذف و یک پارامتر برای پذیرش لیستی از کتاب‌ها به عنوان ورودی اضافه کنید.

در نهایت فراخوانی updateUI() را از onCreateView() حذف کنید. یک فراخوانی برای updateUI() به زودی در جای دیگری پیاده‌سازی می‌کنیم.

لیست ۱۱.۲۱ حذف ارجاعات به نسخه قدیمی ViewModel (BookListFragment.kt)

```
class BookListFragment: Fragment() {
    …
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        Log.d(TAG
        , "Total Books: ${bookListViewModel.books.size}")
    }
    …
    override fun onCreateView(
        ..
    ): View? {
        …
        bookRecyclerView.layoutManager =
            LinearLayoutManager(context)
        updateUI()
        return view
    }
    private fun updateUI(books: List<Book>) {
        val crimes = bookListViewModel.books
        adapter = BookAdapter(books)
        bookRecyclerView.adapter = adapter
    }
    …
}
```

اکنون BookListFragment را برای نمایش LiveData که لیست کتاب‌ها بازگردانده شده از پایگاه داده را شامل می‌شود، به روز کنید. از آنجایی که فرگمنت باید منتظر نتایج پایگاه داده باشد تا بتواند RecyclerView را با کتاب‌ها پر کند، آداپتور RecyclerView را با یک لیست کتاب خالی برای شروع، مقداردهی اولیه کنید.

سپس آداپتور RecyclerView را با لیست جدید کتاب‌ها زمانی که داده‌های جدید در LiveData منتشر می‌شود، تنظیم کنید.

لیست ۱۱.۲۲ اتصال RecyclerView (BookListFragment.kt)

```
…
import androidx.lifecycle.Observer
…
class BookListFragment: Fragment() {
    …
    private lateinit var bookRecyclerView: RecyclerView
    private var adapter: BookAdapter? = null
    private var adapter: BookAdapter? =
       BookAdapter(emptyList())
    …
    override fun onCreateView(
        …
    ): View? {
        …
        bookRecyclerView.layoutManager =
            LinearLayoutManager(context)
        bookRecyclerView.adapter = adapter
        return view
    }
    override fun onViewCreated(view: View
        , savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState)
        bookListViewModel.bookListLiveData.observe(
            viewLifecycleOwner,
            Observer { books ->
                books?.let {
                    Log.i(TAG, "Got crimes ${books.size}")
                    updateUI(books)
                }
            })
    }
    …
}
```

تابع LiveData.observe(LifecycleOwner, Observer) برای ثبت یک observer در نمونه LiveData و گره زدن حیات انتشار به حیات جزء دیگری مانند یک اکتیویتی یا فرگمنت استفاده می‌شود.

پارامتر دوم تابع observe(…) یک پیاده‌سازی Observer است. این شی وظیفه واکنش به داده‌های جدید از LiveData را بر عهده دارد. در این حالت، هر زمان که لیست کتاب‌ها LiveData به روز شود، بلوک کد observer اجرا می‌شود. observer فهرستی از کتاب‌ها را از LiveData دریافت و اگر مقدار null نباشد یک log ثبت می‌کند.

اگر Observer را برای گوش دادن به تغییرات LiveData کنسل نکنیم، ممکن است پیاده‌سازی Observer سعی کند ویوی فرگمنت را زمانی که در حالت نامعتبر است (مانند زمانی که view در حال تخریب است) به روز کند. و اگر این اتفاق روی دهد، برنامه ممکن است از کار بیفتد.

اینجاست که پارامتر LifecycleOwner به LiveData.observe(…) وارد می‌شود. حیات Observer به حیات مؤلفه اندروید محدود می‌شود. در کد بالا Observer را به حیات ویوی فرگمنت محدود می‌کنیم.

تا زمانی که مالک چرخه حیاتی که Observer خود را به آن محدود می‌کنیم در یک وضعیت چرخه حیات معتبر باشد شی LiveData، Observer را از هر گونه داده جدیدی که وارد می‌شود مطلع می‌کند. شی LiveData به طور خودکار زمانی که چرخه حیات مرتبط، دیگر در حالت معتبر نباشد Observer را کنسل می‌کند. از آنجایی که LiveData به تغییرات چرخه حیات واکنش نشان می‌دهد، به آن مؤلفه آگاه از چرخه حیات (lifecycle-aware) می‌گویند.

lifecycle owner مؤلفه‌ای است که رابط LifecycleOwner را پیاده‌سازی می‌کند و شامل یک شی Lifecycle است. Lifecycle شیئی است که وضعیت فعلی چرخه حیات اندروید را پیگیری می‌کند. (به یاد داشته باشید که اکتیویتی‌ها، فرگمنت‌ها، ویوها و حتی خود فرآیند درخواست همگی چرخه حیات خاص خود را دارند.) حالت‌های چرخه زندگی، مانند created و resumed، در Lifecycle.State بررسی می‌شوند. می‌توان با استفاده از Lifecycle.getCurrentState() وضعیت چرخه حیات را جویا شد.

فرگمنت مستقیما یک مالک چرخه حیات است – فرگمنت LifecycleOwner را پیاده‌سازی می‌کند و دارای شی Lifecycle است که وضعیت چرخه حیات نمونه فرگمنت را نشان می‌دهد.

چرخه حیات ویوی یک فرگمنت متعلق به FragmentViewLifecycleOwner است و به طور جداگانه توسط آن ردیابی می‌شود. هر فرگمنت دارای یک نمونه از FragmentViewLifecycleOwner است که چرخه حیات ویوی آن فرگمنت را پیگیری می‌کند.

در کد بالا با پاس کردن viewLifecycleOwner به تابع observe(…)، مشاهده تغییرات را در عوض خود فرگمنت به چرخه حیات ویو فرگمنت محدود کردیم. چرخه حیات ویوی فرگمنت اگر چه جدا از چرخه حیات خود نمونه فرگمنت است اما چرخه حیات فرگمنت را منعکس می‌کند. این امکان وجود دارد که این رفتار پیش‌فرض را با حفظ (retaining) فرگمنت تغییر دهیم (که در BookShelf انجام نخواهیم داد).

Fragment.onViewCreated(…) بعد از برگشت Fragment.onCreateView(…) فراخوانی می‌شود و نشان می‌دهد که سلسله مراتب ویوی فرگمنت در جای خود قرار دارد. LiveData را از onViewCreated(…) برای اطمینان از اینکه ویوی ما برای نمایش داده‌های کتاب آماده است، رصد می‌کنیم. همچنین به همین دلیل است که viewLifecycleOwner را به تابع observe() به جای خود

فرگمنت پاس می‌کنیم. ما فقط می‌خواهیم لیست کتاب‌ها را زمانی دریافت کنیم که ویوی ما در وضعیت مناسب باشد، بنابراین استفاده از مالک چرخه حیات ویو تضمین می‌کند که زمانی که ویو روی صفحه نیست، به روزرسانی‌ها را دریافت نخواهد کرد.

هنگامی که لیست کتاب‌ها آماده شد، observerی که تعریف کرده‌ایم یک پیام گزارش چاپ می‌کند و لیست را به updateUI() را برای آماده سازی آداپتور ارسال می‌کند.

همه چیز در جای خودش قرار گرفت، BookShelf را اجرا کنید. دیگر نباید خطای قبلی را مشاهده کنید. در عوض باید شاهد یک ویو خالی باشید، زیرا هنوز داده‌ای در پایگاه داده وارد نکرده‌ایم (شکل ۱۱.۵).

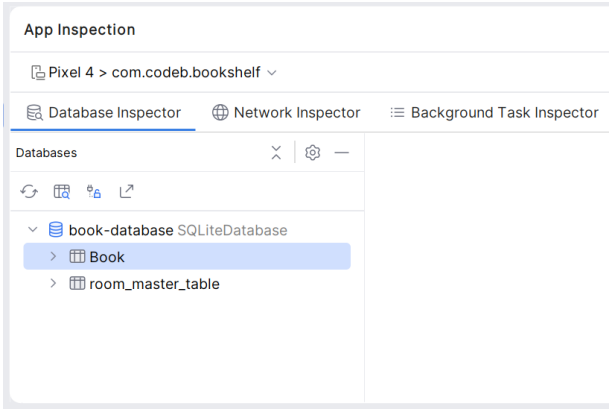
تصویر ۱۱.۵ کتاب‌ها پایگاه داده



هنوز به بخش اضافه کردن مقادیر به جدول توسط برنامه خودمان نرسیدیم. اما ما به داده نیاز داریم به همین خاطر از یک روش دیگر فقط برای یکبار و تنها برای اضافه کردن داده‌های نمونه استفاده خواهیم کرد. نگران این قسمت نباشد. فقط آن را دنبال کنید تا داده‌ای برای نمایش و کار با آن داشته باشیم.

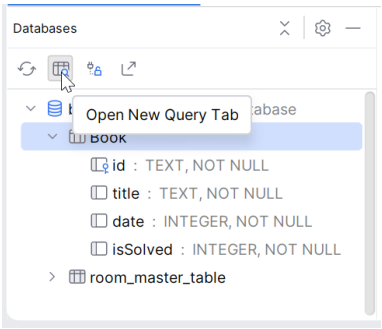
مطمئن شوید که برنامه درحال اجراست. منوی View → Tool Windows → App Inspection را انتخاب کنید. پنجره App Inspection ظاهر می‌شود.

تصویر ۱۱.۶ نمایش پایگاه داده



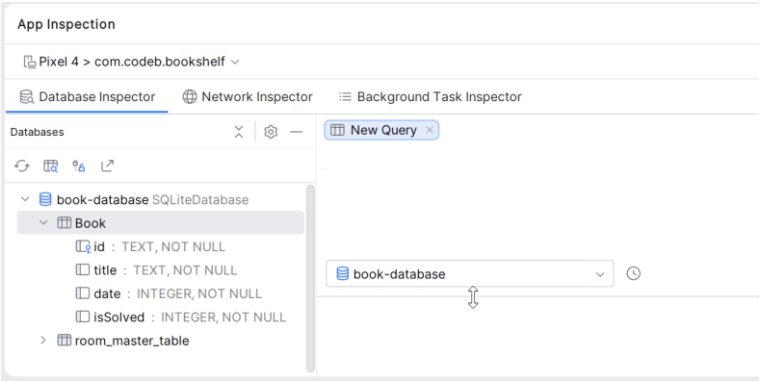
در این پنجره مطمئن شوید که دستگاهی که برنامه روی آن اجرا می‌شود، انتخاب شده است. سپس DataBase Inspector را انتخاب نمایید. همانند تصویر جدول پایگاه داده ما (Book) نشان داده می‌شود. آن را گسترش دهید. با این کار فیلدهای جدول نمایش داده می شود. در بالای این قسمت دکمه‌ای همانند شکل زیر برای اجرای یک کوئری SQL با توضیح Open new query tab وجود دارد. آن را انتخاب کنید.

تصویر ۱۱.۷ جدول پایگاه داده



برگه همانند شکل زیر برای وارد کردن کوئری باز می شود.

تصویر ۱۱.۸ کوئری

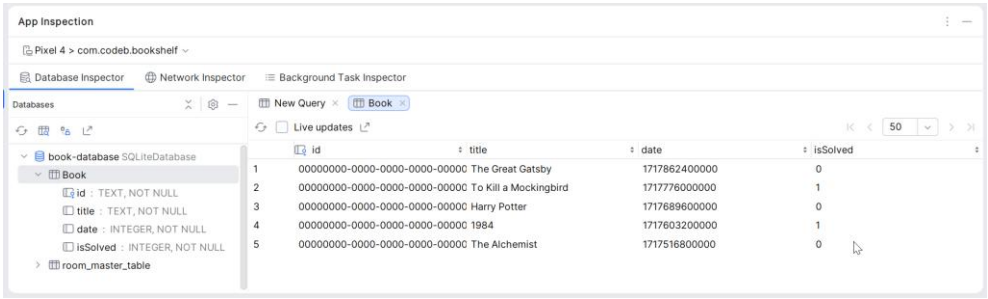


کوئری های زیر را بدون توجه به مفهوم آنها وارد کرد و کلید Run را انتخاب کنید.

```
INSERT INTO Book (id, title, date, isSolved) VALUES
('00000000-0000-0000-0000-000000000001', 'The Great Gatsby'
, 1717862400000, 0)
, ('00000000-0000-0000-0000-000000000002'
, 'To Kill a Mockingbird', 1717776000000, 1)
,('00000000-0000-0000-0000-000000000003', 'Harry Potter',
1717689600000, 0)
,('00000000-0000-0000-0000-000000000004', '1984',
1717603200000, 1)
,('00000000-0000-0000-0000-000000000005', 'The Alchemist',
1717516800000, 0);
```

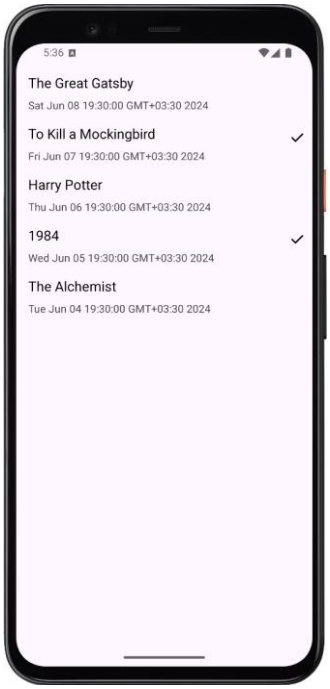
فقط این را بدانید که کوئری‌های بالا پنج کتاب را به جدول ما اضافه می کنند. برای دیدن داده‌ها تنها کافی است که روی نام جدول دابل کلیک کنید.

تصویر ۱۱.۹ مقادیر پایگاه داده



حالا برنامه را دوباره اجرا کنید. مشاهده خواهید کرد که پنج کتاب به نمایش در می‌آید.

تصویر ۱۱.۱۰ نمایش کتاب‌ها



در فصل بعدی صفحه نمایش لیست کتاب و جزئیات کتاب را به هم متصل می‌کنیم و صفحه جزئیات کتاب را با داده‌های کتابی‌که از پایگاه داده روی آن کلیک شده، پر می‌کنیم.

بیشتر بدانیم: Singletons

الگوی singleton همانطور که در BookRepository استفاده شد، در اندروید بسیار رایج است. Singletonها تصویر منفی دارند، زیرا با استفاده نادرست از آنها، نگهداری برنامه سخت می‌شود.

Singletonها اغلب در اندروید استفاده می‌شود زیرا از یک فرگمنت یا اکتیویتی بیشتر عمر می‌کنند. یک singleton همچنان در طول چرخش و حرکت بین اکتیویتی‌ها و فرگمنت‌ها در برنامه وجود خواهد داشت.

Singletonها همچنین مالک مناسبی برای آبجکت‌ها مدل شما هستند. یک برنامه پیچیده‌تر BookShelf را با اکتیویتی‌ها و فرگمنت‌ها زیادی که کتاب‌ها را اصلاح می‌کنند، تصور کنید. هنگامی که یک کنترلر یک کتاب را اصلاح می‌کند، چگونه مطمئن می‌شوید که کتاب به روز شده به کنترلرهای دیگر ارسال شده است؟

اگر BookRepository مالک Books باشد و تمام تغییرات در Books از آن عبور کند، انتشار تغییرات بسیار آسان‌تر است. همانطور که بین کنترلرها منتقل می‌شوید، می‌توانید شناسه کتاب را به عنوان شناسه ارسال و از هر کنترلر بخواهید که با استفاده از آن، شی کتاب کامل را از BookRepository بیرون بکشد.

با این حال Singletonها چند جنبه منفی دارند. به عنوان مثال در حالی که آنها امکان ذخیره داده‌ها را با طول حیات بیشتر از یک کنترلر فراهم می‌کنند باز هم عمر دارند. Singletonها به همراه تمام متغیرهای نمونه آنها از بین می‌روند، زیرا اندروید در مقطعی پس از خروج از یک برنامه حافظه را بازیابی می‌کند. Singletonها یک راه حل ذخیره‌سازی طولانی مدت نیستند. (نوشتن فایل‌ها روی دیسک یا ارسال آنها به یک وب سرور روش ذخیره‌سازی طولانی مدت است.)

Singletonها همچنین کد شما را برای تست واحد (unit test) سخت می‌کنند (در فصل ۲۰ درباره تست واحد بیشتر خواهید آموخت). و همانطور که گفتیم Singletonها پتانسیل سوء استفاده را دارند. استفاده از Singletonها برای همه چیز می‌تواند وسوسه انگیز باشد، زیرا آنها به راحت در دسترس هستند – شما می‌توانید در هر کجا که هستید به آنها دسترسی پیدا کنید و می‌توانید هر اطلاعاتی را که بعدا نیاز دارید در آنها ذخیره کنید. اما وقتی این کار را انجام می‌دهید، از پاسخ دادن به سوالات مهم اجتناب می‌کنید: این داده‌ها در کجا استفاده می‌شوند؟ این تابع در کجا مهم است؟

یک Singleton به این سوالات پاسخ نمی‌دهد. بنابراین هر کسی که بعد از شما بیاید، Singleton شما را باز می‌کند و چیزی را پیدا می‌کند که شبیه کشوی آشغال بی نظم است. باتری، زیپ کراوات، عکس‌های قدیمی؟ همه اینها اینجا برای چیست؟ اطمینان حاصل کنید که هر چیزی در Singleton شما واقعا عمومی‌است و دلیل محکمی‌برای حضور در آنجا دارد.

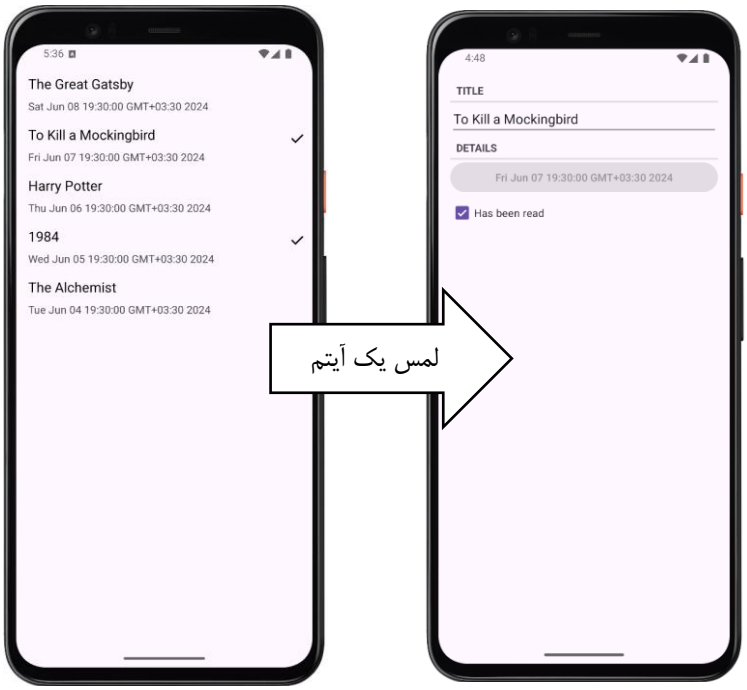
در مجموع Singletonها جزء کلیدی یک برنامه اندرویدی با معماری خوب هستند – زمانی که به درستی استفاده شود.

۱۲

تعویض فرگمنت‌ها

در این فصل بخش لیست و بخش جزئیات هر کتاب را با هم به کار خواهیم گرفت. هنگامی که کاربر آیتمی را در لیست کتاب‌ها لمس می‌کند MainActivity، BookListFragment را با یک نمونه جدید از BookFragment تعویض می‌کند. BookFragment جزئیات کتابی را که لمس شده، نمایش می‌دهد (شکل ۱۲.۱).

شکل ۱۲.۱ تعویض BookListFragment با BookFragment



برای انجام اینکار، تعویض فرگمنت‌ها را در پاسخ به اقدامات کاربر پیاده‌سازی خواهیم کرد. همچنین چگونگی انتقال داده‌ها را با استفاده از آرگومان‌های فرگمنت به یک نمونه فرگمنت دیگر، یاد خواهیم

گرفت. در نهایت، چگونگی استفاده از مبدل‌های LiveData برای بارگذاری داده‌های تغییرناپذیر یا ثابت (immutable) در پاسخ به تغییرات رابط کاربری را فرا خواهیم گرفت.

اکتیویتی واحد: رئیس فرگمنت

در GeoQuiz ما یک اکتیویتی (MainActivity) داشتیم و اکتیویتی دیگر (CheatActivity) را فراخوانی می‌کردیم. در BookShelf از یک معماری اکتیویتی واحد استفاده خواهیم کرد. برنامه‌ای که از معماری اکتیویتی واحد استفاده می‌کند، دارای یک اکتیویتی و چندین فرگمنت است. وظیفه این اکتیویتی تعویض فرگمنت‌ها در پاسخ به رویدادهای کاربر است.

برای پیاده‌سازی تعویض، از BookListFragment به BookFragment در پاسخ به لمس کاربر بر روی یک کتاب در لیست، ممکن است به راه اندازی یک تراکنش فرگمنت در FragmentManager اکتیویتی میزبانی در BookHolder.onClick(View) مربوط به BookListFragment فکر کنید. onClick(View) مدیر فرگمنت (FragmentManager) MainActivity را دریافت و یک تراکنش فرگمنت را برای جایگزین کردن BookListFragment با BookFragment درخواست می‌کند.

کد موجود در BookListFragment.BookHolder شما به این شکل است:

```
fun onClick(view: View) {
    val fragment = BookFragment.newInstance(Book.id)
    val fm = activity.supportFragmentManager
    fm.beginTransaction()
    .replace(R.id.fragment_container, fragment)
    .commit()
}
```

این کد مشکلی ندارد، اما برنامه‌نویسان حرفه‌ای اندروید این گونه عمل نمی‌کنند. فرگمنت‌ها به صورت واحدهای مستقل و قابل ترکیب در نظر گرفته شده‌اند. اگر فرگمنتی بنویسید که فرگمنت‌ها را به FragmentManager اکتیویتی اضافه می‌کند، پس آن فرگمنت در نحوه عملکرد اکتیویتی میزبان مداخله می‌کند. چنینی فرگمنتی دیگر واحد مستقل و قابل ترکیب به حساب نمی‌آید.

به عنوان مثال در کد بالا BookListFragment یک BookFragment را به MainActivity اضافه و فرض می‌کند که MainActivity یک fragment_container در لیوت خود دارد. این روالی است که باید به جای BookListFragment توسط اکتیویتی میزبان BookListFragment انجام شود.

برای حفظ استقلال فرگمنت‌ها خود، با تعریف رابط تابع پاسخ به رویداد (callback interface) در فرگمنت‌های خود، کار را به اکتیویتی میزبان واگذار می‌کنید.

رابط تابع پاسخ به رویداد فرگمنت

برای واگذاری امور به اکتیویتی میزبان، فرگمنت معمولا یک رابط سفارشی به نام Callbacks را تعریف می‌کند. این رابط، کاری را که فرگمنت نیاز به انجام آن بوسیله رئیس خود (اکتیویتی میزبانی) دارد، تعریف می‌کند. هر اکتیویتی که میزبان فرگمنت است باید این رابط را پیاده‌سازی کند.

با رابط Callbacks، فرگمنت می‌تواند توابع را در اکتیویتی میزبانی خود بدون اینکه نیازی به دانستن چیزی در مورد اینکه کدام اکتیویتی میزبان آن است، فراخوانی کند.

از یک رابط callbacks برای واگذاری رویداد کلیک از BookListFragment به اکتیویتی میزبان آن استفاده می‌کنیم. ابتدا BookListFragment را باز و یک رابط Callbacks تعریف کنید. متغییر callbacks از نوع Callbacks را برای نگه داشتن شیی که Callbacks را پیاده‌سازی می‌کند، اضافه کنید. onAttach(Context) و ()onDetach را بازنویسی کنید تا متغییر callbacks را تنظیم کند.

لیست ۱۲.۱ افزودن callback interface (BookListFragment.kt)

```
class BookListFragment: Fragment() {
    interface Callbacks{
        fun onBookSelected(bookId: UUID)
    }
    private var callbacks: Callbacks? = null
    private val bookListViewModel: BookListViewModel by
        viewModels()
    …
    companion object {
        …
    }
    override fun onAttach(context: Context) {
        super.onAttach(context)
        callbacks=context as Callbacks?
    }
    override fun onDetach() {
        super.onDetach()
        callbacks = null
    }
}
```

تابع چرخه حیات Fragment.onAttach(Context) زمان اتصال فرگمنت به اکتیویتی فراخوانی می‌شود. در اینجا آرگومان Context ارسال شده به (…)onAttach را در ویژگی callbacks ذخیره می‌کنیم؛ از آنجایی که BookListFragment در یک اکتیویتی میزبانی می‌شود، شی Context منتقل شده به (…)onAttach نمونه اکتیویتی میزبان فرگمنت است.

به یاد داشته باشید، Activity زیر کلاس Context است. بنابراین onAttach(…) یک Context را به عنوان یک پارامتر منتقل می‌کند. اطمینان حاصل کنید که از فرمت onAttach(Context) برای onAttach(…) استفاده می‌کنید و نه تابع منسوخ شده onAttach(Activity) که ممکن است در نسخه‌های بعدی API حذف شود.

به طور مشابه متغیر را در تابع چرخه حیات منسوخ شده مربوطه، Fragment.onDetach() روی null تنظیم می‌کنیم. ما متغیر را در اینجا روی null تنظیم می‌کنیم زیرا پس از آن نمی‌توانیم به اکتیویتی دسترسی داشته باشیم. توجه داشته باشید که BookListFragment یک تبدیل نوع از اکتیویتی خود را به BookListFragment.Callbacks انجام می‌دهد. این بدان معنی است که اکتیویتی میزبانی باید BookListFragment.Callbacks را پیاده‌سازی کند.

اکنون BookListFragment راهی برای فراخوانی توابع در اکتیویتی میزبانی خود دارد. مهم نیست که کدام اکتیویتی میزبانش باشد. تا زمانی که اکتیویتی BookListFragment.Callbacks را پیاده‌سازی کند، همه چیز در BookListFragment می‌تواند به همان شکل قبلی عمل کند.

در مرحله بعد شنونده کلیک را برای هر آیتم در لیست کتاب به روز کنید تا با لمس یک کتاب از طریق رابط Callbacks اکتیویتی میزبان را مطلع کند.

onBookSelected(Book) را در BookHolder.onClick(View) فراخوانی کنید.

لیست ۱۲.۲ تماس با callback! (BookListFragment.kt)

```
private inner class BookHolder(view: View)
    : RecyclerView.ViewHolder(view) {
    …
    init {
        itemView.setOnClickListener {
            callbacks?.onBookSelected(book.id)
        }
    }
    fun bind(book: Book){
        …
    }
}
```

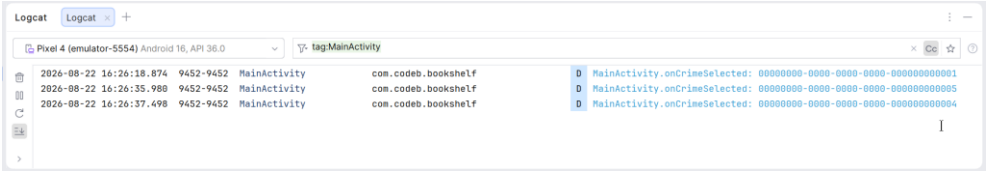
در نهایت MainActivity را برای پیاده‌سازی BookListFragment.Callbacks به روز کنید. یک دستور لاگ برای ثبت onBookSelected(UUID) را در حال حاضر در نظر بگیرید.

لیست ۱۲.۳ پیاده‌سازی callbacks (MainActivity.kt)

```
private const val TAG = "MainActivity"
class MainActivity : AppCompatActivity(),
    BookListFragment.Callbacks {
    override fun onCreate(savedInstanceState: Bundle?) {
        …
    }
    override fun onBookSelected(bookId: UUID) {
        Log.d(TAG, "MainActivity.onCrimeSelected: $bookId")
    }
}
```

BookShelf را اجرا کنید. Logcat را جستجو یا فیلتر کنید تا گزارش MainActivity را مشاهده کنید. هر بار که کتابی را در لیست لمس می‌شود، باید یک گزارش مربوط به رویداد کلیک از BookListFragment به MainActivity از طریق Callbacks.onBookSelected(UUID) منتشر شود.

شکل ۱۲.۲ رویداد کلیک



تعویض یک فرگمنت

اکنون که رابط callbacks به درستی پیاده‌سازی شد، onBookSelected(UUID) مربوط به MainActivity را به روز کنید تا زمانی که کاربر کتابی را در لیست BookListFragment لمس می‌کند BookListFragment را با نمونه‌ای از BookFragment جایگزین کند. در حال حاضر شناسه کتاب ارسال شده به callbacks را نادیده بگیرید.

لیست ۱۲.۴ جایگزینی BookListFragment با BookFragment (MainActivity.kt)

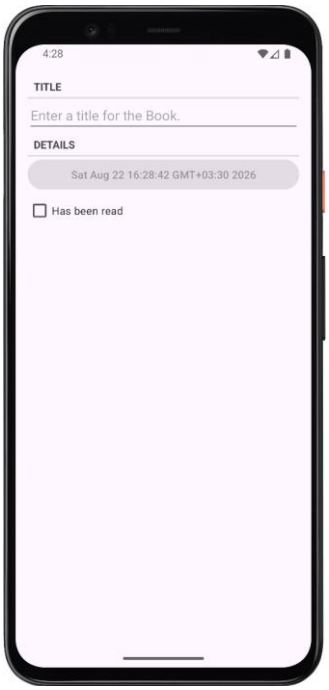
```
override fun onBookSelected(bookId: UUID) {
    Log.d(TAG, "MainActivity.onCrimeSelected: $bookId")
    val fragment = BookFragment()
    supportFragmentManager
        .beginTransaction()
        .replace(R.id.fragment_container, fragment)
        .commit()
}
```

FragmentTransaction.replace(Int, Fragment) فرگمنت میزبانی شده در اکتیویتی (در شناسه منبع مشخص شده) را با فرگمنت جدید ارائه شده جایگزین می‌کند.

اگر یک فرگمنت قبلا در این منبع مشخص شده میزبانی نشده باشد، فرگمنت جدید اضافه می‌شود، درست مانند اینکه FragmentTransaction.add(Int, fragment) را فراخوانی کرده‌اید.

BookShelf را اجرا کنید. یک کتاب را در لیست لمس کنید. صفحه جزئیات کتاب باید ظاهر شود (شکل ۱۲.۳).

شکل ۱۲.۳ یک BookFragment خالی



در حال حاضر صفحه جزئیات کتاب خالی است، زیرا به BookFragment نگفته‌ایم که کدام کتاب را نمایش دهد. به زودی صفحه جزئیات را پر خواهیم کرد. اما ابتدا باید یک نقص باقی مانده را در پیاده‌سازی جایگزینی فرگمنت خود ثبت کنیم.

دکمه بازگشت را فشار دهید. این امر MainActivity را رد می‌کند. زیرا تنها مورد در پشته برنامه‌ها، نمونه MainActivity است که هنگام راه اندازی برنامه بالا آمده است.

کاربران انتظار دارند که با فشار دادن دکمه Back از صفحه جزئیات کتاب، آنها را به لیست کتاب بازگرداند. برای اجرای این رفتار تراکنش جایگزینی را به پشته اضافه می‌کنیم.

لیست ۱۲.۵ افزودن تراکنش فرگمنت به پشته (MainActivity.kt)

```
override fun onBookSelected(bookId: UUID) {
    val fragment = BookFragment()
    supportFragmentManager
        .beginTransaction()
        .replace(R.id.fragment_container, fragment)
        .addToBackStack(null)
        .commit()
}
```

هنگامی که یک تراکنش را به پشته اضافه می‌کنیم، به این معنی است که وقتی کاربر دکمه Back را فشار می‌دهد، تراکنش معکوس می‌شود. بنابراین در این مورد BookFragment با BookListFragment جایگزین خواهد شد.

می‌توانیم با ارسال یک رشته به FragmentTransaction.addToBackStack(String) نامی برای حالتی که اضافه می‌کنیم، برگزینیم. انجام این کار اختیاری است و از آنجایی که به نام موجود در این پیاده‌سازی اهمیتی نمی‌دهیم null را پاس می‌کنیم.

برنامه خود را اجرا و یک کتاب را از لیست انتخاب کنید تا BookFragment راه اندازی شود. دکمه Back را فشار دهید تا به BookListFragment برگردید. لذت ببرید.

آرگومان‌های فرگمنت

BookListFragment اکنون اکتیویتی میزبانی خود (MainActivity) را هنگام انتخاب کتاب مطلع و شناسه کتاب انتخاب شده را ارسال می‌کند.

بسیار عالی و شیک است. اما چیزی که واقعا نیاز دارید راهی برای انتقال شناسه کتاب انتخاب شده از MainActivity به BookFragment است. به این ترتیب BookFragment می‌تواند داده‌های آن کتاب را از پایگاه داده بیرون بکشد و رابط کاربری را با آن داده‌ها پر کند.

آرگومان‌های فرگمنت راه حلی برای این مشکل ارائه می‌دهند – آنها به شما امکان می‌دهند تکه‌هایی از داده‌ها را در جایی که متعلق به فرگمنت است ذخیره کنید. "جایی که به یک فرگمنت تعلق دارد" به عنوان arguments bundle شناخته می‌شود. فرگمنت می‌تواند داده‌ها را از arguments bundle بدون تکیه بر اکتیویتی والد خود یا منبع خارجی دیگری بازیابی کند.

آرگومان‌های فرگمنت به شما کمک می‌کنند تا فرگمنت خود را کپسوله نگه دارید. یک فرگمنت خوب کپسوله شده همانند یک بلوک ساختمانی قابل استفاده مجدد است که به راحتی می‌تواند در هر اکتیویتی میزبانی شود.

برای ایجاد آرگومان‌های فرگمنت ابتدا یک شی Bundle ایجاد می‌کنیم. این Bundle شامل جفت‌های کلید–مقدار است. درست همانند Extra اینتنت یک Activity کار می‌کند. هر جفت به عنوان آرگومان شناخته می‌شود. در مرحله بعد از توابع put مخصوص نوع Bundle (مشابه توابع Intent) برای افزودن آرگومان‌ها به Bundle استفاده می‌کنیم:

```
val args = Bundle().apply {
  putSerializable(ARG_MY_OBJECT, myObject)
  putInt(ARG_MY_INT, myInt)
  putCharSequence(ARG_MY_STRING, myString)
}
```

هر نمونه فرگمنت می‌تواند شامل یک آرگومان فرگمنت Bundle متصل به آن باشد.

پیوست کردن آرگومان‌ها به یک فرگمنت

برای پیوست کردن bundle آرگومان به یک فرگمنت Fragment.setArguments(Bundle) را فراخوانی می‌کنیم. اتصال آرگومان‌ها به یک فرگمنت باید پس از ایجاد فرگمنت اما قبل از اضافه شدن به یک اکتیویتی انجام شود.

برای انجام این کار برنامه‌نویسان اندروید از قراردادی پیروی می‌کنند که بر طبق آن companion object را که حاوی newInstance(…) است به کلاس Fragment اضافه می‌کنند. این تابع نمونه فرگمنت را ایجاد و آرگومان‌های آن را تنظیم می‌کند.

هنگامی که اکتیویتی میزبان به نمونه‌ای از آن فرگمنت نیاز دارد، به جای فراخوانی مستقیم سازنده newInstance(…) را فراخوانی می‌کند. اکتیویتی می‌تواند هر پارامتر مورد نیاز را به newInstance(…) منتقل کند.

در BookFragment، یک تابع newInstance(UUID) بنویسید که یک UUID را می‌پذیرد، یک bundle آرگومان و یک نمونه فرگمنت ایجاد و سپس آرگومان‌ها را به فرگمنت متصل می‌کند.

لیست ۱۲.۶ نوشتن یک تابع newInstance(UUID) (BookFragment.kt)

```
private const val ARG_BOOK_ID = "Book_id"
class BookFragment : Fragment() {
    ...
```

```
    private lateinit var solvedCheckBox: CheckBox
    companion object {
        fun newInstance(BookId: UUID): BookFragment {
            val args = Bundle().apply {
                putSerializable(ARG_BOOK_ID, BookId)
            }
            return BookFragment().apply {
                arguments = args
            }
        }
    }
    ...
}
```

MainActivity را به روز کنید تا BookFragment.newInstance(UUID) را در صورت نیاز به ایجاد BookFragment صدا بزند. UUID را که در MainActivity.onBookSelect(UUID) دریافت کرده‌اید، پاس کنید.

لیست ۱۲.۷ با استفاده از BookFragment.newInstance(UUID) (MainActivity.kt)

```
override fun onBookSelected(bookId: UUID) {
    val fragment = BookFragment()
    val fragment = BookFragment.newInstance(bookId)
    supportFragmentManager
        .beginTransaction()
        .replace(R.id.fragment_container, fragment)
        .addToBackStack(null)
        .commit()
}
```

نیاز به استقلال هر دو طرف نیست. MainActivity باید چیزهای زیادی در مورد BookFragment بداند، از جمله اینکه دارای یک تابع newInstance(UUID) است. اکتیویتی‌های میزبان باید مشخصات نحوه میزبانی فرگمنت‌ها خود را بدانند، اما فرگمنت‌ها نباید جزئیات مربوط به اکتیویتی‌های خود بدانند. حداقل نه اگر می‌خواهید انعطاف پذیری فرگمنت‌ها مستقل را حفظ کنید.

بازیابی آرگومان‌ها

برای دسترسی به آرگومان‌های یک فرگمنت به ویژگی Fragment با نام arguments مراجعه می‌کنیم. سپس یکی از توابع "get" مخصوص نوع Bundle استفاده می‌کنیم تا مقادیر را از bundle آرگومان‌ها استخراج کنیم.

به BookFragment.onCreate(...) برگردید، UUID را از آرگومان‌های فرگمنت بازیابی کنید. شناسه را لاگ می‌کنیم تا درستی آرگومان را تایید کنیم.

لیست ۱۲.۸ دریافت شناسه کتاب از آرگومان‌ها (BookFragment.kt)

```
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    book= Book()
    val bookId: UUID = arguments?.getSerializable(
        ARG_BOOK_ID) as UUID
}
```

BookShelf را اجرا کنید. برنامه همان رفتار قبلی را دارد.

استفاده از تبدیلات LiveData

اکنون که BookFragment شناسه کتاب را دارد، باید کتاب مورد نظر را از پایگاه داده بیرون کشیده و نمایش دهد. از آنجایی که این کار نیاز به جستجوی پایگاه داده دارد و از طرفی چون نمی‌خواهیم برای هر چرخش این کار را تکرار کنیم، یک BookDetailViewModel برای مدیریت پرس و جو پایگاه داده اضافه خواهیم کرد.

هنگامی که BookFragment درخواست کتابی را با یک شناسه معین دارد، BookDetailViewModel آن باید درخواست پایگاه داده getBook(UUID) را اجرا کند. پس از تکمیل درخواست، BookDetailViewModel باید به BookFragment اطلاع دهد و شی Book بدست آمده از پرس و جو را برای آن ارسال کند.

یک کلاس جدید با نام BookDetailViewModel ایجاد و یک ویژگی LiveData را برای ذخیره و نمایش Book که از پایگاه داده استخراج شده است، تعریف کنید. از LiveData برای پیاده‌سازی رابطه‌ای استفاده کنید که در آن تغییر شناسه کتاب باعث ایجاد یک پرس و جو پایگاه داده جدید می‌شود.

لیست ۱۲.۹ افزودن ViewModel برای BookFragment (BookDetailViewModel.kt)

```
class BookDetailViewModel() : ViewModel() {
    private val bookRepository = BookRepository.get()
    private val bookIdLiveData = MutableLiveData<UUID>()
    val bookLiveData: LiveData<Book?> =
    bookIdLiveData.switchMap { bookId ->
        bookRepository.getBook(bookId)
    }
    fun loadBook(bookId: UUID) {
        bookIdLiveData.value = bookId
    }
}
```

ویژگی bookRepository یک هندل به BookRepository ذخیره می‌کند. ضروری نیست اما بعدا BookDetailViewModel در بیش از یک جا با repository ارتباط برقرار می‌کند، بنابراین این ویژگی مفید خواهد بود.

bookIdLiveData شناسه کتابی را که در حال حاضر توسط BookFragment نمایش داده می‌شود (یا در شرف نمایش است) ذخیره می‌کند. هنگام ایجاد BookDetailViewModel برای اولین بار شناسه کتاب تنظیم نمی‌شود. در نهایت BookFragment، BookDetailViewModel.loadBook(UUID) را فراخوانی می‌کند تا به ViewModel اطلاع دهد که کدام کتاب را باید بارگذاری کند.

توجه داشته باشید که ما به صراحت نوع bookLiveData را به عنوان LiveData<Book?> تعریف کرده‌ایم. از آنجایی که bookLiveData به صورت public است، باید اطمینان حاصل کنیم که به عنوان یک MutableLiveData تعریف می‌شود.

به طور کلی ViewModelها هرگز نباید MutableLiveData را تعریف کند.

پیچیدن شناسه کتاب در LiveData عجیب به نظر می‌رسد، زیرا برای BookDetailViewModel شناسه یه ویژگی غیرقابل دسترس و خصوصی است. چه چیزی در BookDetailViewModel باید به تغییرات مقدار خصوصی شناسه گوش دهد؟

پاسخ در تبدیلات LiveData نهفته است. تبدیلات LiveData راهی برای ایجاد رابطه trigger-response بین دو شی LiveData است. یک تابع تبدیل دو ورودی می‌گیرد: یک شی LiveData که به عنوان یک trigger استفاده می‌شود و یک تابع مپینگ که باید یک شی LiveData را برگرداند. تابع تبدیل یک شی LiveData جدید را برمی‌گرداند که آن را نتیجه تبدیل می‌نامیم. هر بار که مقدار جدیدی در trigger تنظیم می‌شود، مقدار آن به روز می‌شود.

مقدار نتیجه تبدیل با اجرای تابع مپینگ محاسبه می‌شود. ویژگی value در LiveData بازگردانده شده از تابع مپینگ برای تنظیم ویژگی value در نتیجه تبدیل LiveData استفاده می‌شود.

استفاده از تبدیل به این روش به این معنی است که BookFragment فقط باید یک بار BookDetailViewModel.bookLiveData نتیجه را مشاهده کند. هنگامی که فرگمنت شناسه‌ای را که می‌خواهد نمایش دهد تغییر می‌دهد، ViewModel فقط داده‌های کتاب جدید را در LiveData موجود منتشر می‌کند.

BookFragment.kt را باز و BookFragment را با BookDetailViewModel مرتبط کنید. از ViewModel درخواست کنید که Book را در onCreate(…) بارگذاری کند.

لیست کردن ۱۲.۱۰ اتصال BookFragment به BookDetailViewModel (BookFragment.kt)

```
class BookFragment : Fragment() {
    …
    private lateinit var solvedCheckBox: CheckBox
    private val bookDetailViewModel: BookDetailViewModel
       by viewModels()
    companion object {
        …
    }
    override fun onCreate(savedInstanceState: Bundle?) {
```

```
        …
        val bookId: UUID = arguments?
            .getSerializable(ARG_BOOK_ID) as UUID
        bookDetailViewModel.loadBook(bookId)
    }
```

در مرحله بعد، هر زمان که داده‌های جدید منتشر شد، رابط کاربری را به روز کنید.

فهرست ۱۲.۱۱ مشاهده تغییرات (BookFragment.kt)

```
class BookFragment : Fragment() {
    …
    companion object {
        …
    }
    override fun onViewCreated(view: View
    , savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState)
        bookDetailViewModel.bookLiveData.observe(
            viewLifecycleOwner,
            Observer { book ->
                book?.let {
                    this.book = book
                    updateUI()
                }
            })
    }
    private fun updateUI() {
        titleField.setText(book.title)
        dateButton.text = book.date.toString()
        solvedCheckBox.isChecked = book.isSolved
    }
    …
}
```

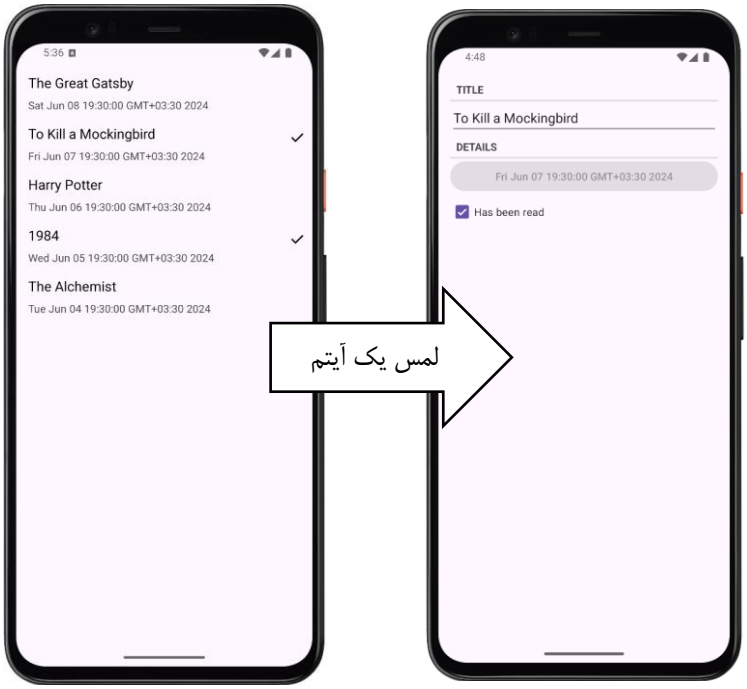
(حتما androidx.lifecycle.Observer را وارد کنید.)

ممکن است متوجه شده باشید که BookFragment، Book خود را دارد که در متغییر book ذخیره شده است. مقادیر موجود در این ویژگی نشان دهنده ویرایش‌هایی است که کاربر در حال حاضر انجام می‌دهد. Book موجود در BookDetailViewModel.BookLiveData داده‌ها را همانطور که در حال حاضر در پایگاه داده ذخیره شده است، نشان می‌دهد.

BookFragment با نوشتن داده‌های به روز شده در پایگاه داده ویرایش‌های کاربر را زمانی که فرگمنت به حالت متوقف شده منتقل می‌شود، منتشر می‌کند.

برنامه خود را اجرا و کتابی را در لیست لمس کنید. اگر همه چیز درست پیش برود، باید صفحه جزئیات را با مقادیر پر شده مشاهده نمایید (شکل ۱۲.۴).

شکل ۱۲.۴ پشتی BookShelf



هنگامی که BookFragment نمایش داده می‌شود، ممکن است کتابی را مشاهده کنید که به عنوان تمام شده علامت گذاری شده است که با انیمیشن خاصی نمایش داده می‌شود. انتظار این وضعیت را داشتیم، زیرا وضعیت چک باکس در نتیجه یک عملیات ناهمزمان تنظیم می‌شود. پرس و جو پایگاه داده برای کتاب زمانی شروع می‌شود که کاربر برای اولین بار BookFragment را راه اندازی می‌کند. وقتی کوئری پایگاه داده کامل می‌شود، observer مربوط به BookDetailViewModel.BookLiveData مطلع می‌شود و به نوبه خود داده‌های نمایش داده شده در ویجت‌ها را به روز می‌کند.

برای حذف انیمیشن فوق View.jumpDrawablesToCurrentState() را فراخوانی کنید. توجه داشته باشید که اگر تاخیر هنگام بارگیری صفحه جزئیات کتاب بر اساس نیازهای برنامه شما غیرقابل قبول باشد، می‌توانید داده‌های کتاب را از قبل در حافظه بارگذاری کنید (به عنوان مثال هنگام راه اندازی برنامه) و آن را در یک مکان مشترک ذخیره کنید. برای BookShelf تاخیر بسیاراندک است. بنابراین راه حل ساده رد شدن از انیمیشن است.

لیست ۱۲.۱۲ پرش از انیمیشن چک باکس (BookFragment.kt)

```
private fun updateUI() {
    titleField.setText(book.title)
    dateButton.text = book.date.toString()
    solvedCheckBox.isChecked = book.isSolved
    solvedCheckBox.apply {
        isChecked = book.isSolved
        jumpDrawablesToCurrentState()
    }
}
```

برنامه خود را دوباره اجرا کنید. یک کتاب مطالعه شده را در لیست لمس کنید. توجه داشته باشید که با چرخش صفحه نمایش، کادر دیگر متحرک نمی‌شود. اگر چک باکس را فشار دهید، انیمیشن به دلخواه اتفاق می‌افتد.

اکنون عنوان کتاب را ویرایش کنید. دکمه بازگشت را فشار دهید تا به صفحه لیست کتاب برگردید. متأسفانه تغییراتی که ایجاد کردید ذخیره نشدند. خوشبختانه رفع این مشکل آسان است.

بروزرسانی پایگاه داده

پایگاه داده به عنوان تنها منبع حقیقی برای داده‌های Book عمل می‌کند. در BookShelf هنگامی که کاربر صفحه جزئیات را ترک می‌کند، هر ویرایشی که انجام داده است باید در پایگاه داده ذخیره شود. (سایر برنامه‌ها ممکن است الزامات دیگری داشته باشند، مانند داشتن دکمه «ذخیره» برای ذخیره به روزرسانی‌ها پس از تایپ کاربر.)

ابتدا یک تابع به BookDao خود اضافه کنید تا یک کتاب موجود را به روز کنید. یک تابع برای درج یک کتاب جدید نیز اضافه کنید. در حال حاضر تابع insert را نادیده بگیرید، اما در فصل ۱۴ زمانی که یک گزینه منو به رابط کاربری برای ایجاد کتاب‌ها جدید اضافه می‌کنیم، از آن استفاده خواهیم کرد.

لیست ۱۲.۱۳ افزودن توابع بروزرسانی و درج پایگاه داده (database/BookDao.kt)

```
@Dao
interface BookDao {
    @Query("SELECT * FROM Book")
    fun getBooks(): LiveData<List<Book>>
    @Query("SELECT * FROM Book WHERE id=(:id)")
    fun getBook(id: UUID): LiveData<Book?>
    @Update
    fun updateBook(Book: Book)
    @Insert
    fun addBook(Book: Book)
}
```

حاشیه‌نویسی برای این توابع به هیچ پارامتری نیاز ندارد. Room می‌تواند دستور SQL مناسب را برای این عملیات تولید کند.

تابع ()updateBook از حاشیه‌نویسی @Update استفاده می‌کند. این تابع یک شی Book را می‌گیرد، از شناسه ذخیره شده در آن کتاب برای یافتن ردیف مرتبط استفاده و سپس داده‌های موجود در آن ردیف را بر اساس داده‌های جدید در شی Book به روز می‌کند.

تابع ()addBook از حاشیه‌نویسی @Insert استفاده می‌کند. پارامتر Bookی است که می‌خواهیم به جدول پایگاه داده اضافه کنیم.

در مرحله بعد repository را به روز کنید تا با توابع درج و به روز رسانی DAO تماس داشته باشد. به یاد بیاورید که Room به طور خودکار پرس و جوهای پایگاه داده را برای ()BookDao.getBooks و BookDao.getBook(UUID) در یک نخ پس زمینه انجام می‌دهد. زیرا توابع DAO، LiveData را

برمی‌گردانند. در این موارد LiveData انتقال داده‌ها را به نخ اصلی شما انجام می‌دهد تا بتوانید رابط کاربری خود را به روز کنید.

با این حال نمی‌توانیم به Room تکیه کنیم تا به طور خودکار تعاملات پایگاه داده خود را در یک نخ پس زمینه برای شما اجرا کند. در عوض باید فراخوانی‌های DAO را در یک نخ پس زمینه به صراحت اجرا کنیم. یک روش رایج برای انجام این کار استفاده از executor است.

استفاده از executor

Executor شیی است که به یک نخ ارجاع می‌دهد. یک نمونه executor دارای تابعی به نام execute است که یک بلوک کد را برای اجرا می‌پذیرد. کدی که در بلوک ارائه می‌دهیم روی هر نخی که executor به آن اشاره می‌کند، اجرا می‌شود.

ما می‌خواهیم یک executor ایجاد کنیم که از یک نخ جدید استفاده می‌کند (که همیشه یک نخ پس زمینه خواهد بود). هر کدی در بلوک روی آن نخ اجرا می‌شود، بنابراین می‌توانیم عملیات پایگاه داده خود را با خیال راحت در آنجا انجام دهیم.

شما نمی‌توانید یک executor را مستقیما در BookDao پیاده‌سازی کنید، زیرا Room پیاده‌سازی‌های تابع را بر اساس رابطی که تعریف می‌کنیم برای ما تولید می‌کند. در عوض executor را در BookRepository پیاده‌سازی کنید. یک ویژگی به executor اضافه کنید تا یک ارجاع را نگه دارید، سپس توابع درج و به روز رسانی خود را با استفاده از executor اجرا کنید.

لیست ۱۲.۱۴ درج و به روز رسانی با executor (BookRepository.kt)

```
class BookRepository private constructor(context: Context) {
    ...
    private val BookDao = database.BookDao()
    private val executor =
        Executors.newSingleThreadExecutor()
    fun getBooks(): LiveData<List<Book>> = BookDao.getBooks()
    fun getBook(id: UUID): LiveData<Book?> =
        BookDao.getBook(id)
    fun updateBook(Book: Book) {
        executor.execute {
            BookDao.updateBook(Book)
        }
    }
    fun addBook(Book: Book) {
        executor.execute {
            BookDao.addBook(Book)
        }
    }
    ...
}
```

تابع newSingleThreadExecutor() یک نمونه executor را برمی‌گرداند که به یک نخ جدید اشاره می‌کند. بنابراین هر کاری که با executor اجرا می‌کنیم خارج از نخ اصلی اتفاق می‌افتد.

هر دو ()updateBook و ()addBook فراخوانی‌های خود را به DAO در بلوک {}execute قرار می‌دهد. این کار این عملیات را از نخ اصلی خارج می‌کند تا رابط کاربری خود را مسدود نکند.

مرتبط کردن عملیات نوشتن در پایگاه داده با چرخه حیات فرگمنت

برنامه خود را به روز کنید تا مقادیری را که کاربر در صفحه جزئیات کتاب وارد می‌کند، درست قبل از اینکه کاربر صفحه را ترک کند، در پایگاه داده بنویسد.

BookDetailViewModel.kt را باز و یک تابع برای ذخیره یک شی Book به پایگاه داده اضافه کنید.

لیست ۱۲.۱۵ افزودن قابلیت ذخیره (BookDetailViewModel.kt)

```
class BookDetailViewModel() : ViewModel() {
    …
    fun loadBook(bookId: UUID) {
        bookIdLiveData.value = bookId
    }
    fun saveBook(Book: Book) {
        bookRepository.updateBook(Book)
    }
}
```

saveBook(Book) یک کتاب را می‌پذیرد و آن را در پایگاه داده می‌نویسد. از آنجایی که BookRepository اجرای درخواست به روز رسانی را در یک نخ پس زمینه انجام می‌دهد، یکپارچگی پایگاه داده در اینجا بسیار ساده است.

اکنون BookFragment را به روز کنید تا داده‌های کتاب ویرایش شده کاربر را در پایگاه داده ذخیره کنید.

لیست ۱۲.۱۶ ذخیره در ()onStop (BookFragment.kt)

```
class BookFragment : Fragment() {
    …
    private val bookDetailViewModel: BookDetailViewModel
        by viewModels()
    override fun onStop() {
        super.onStop()
        bookDetailViewModel.saveBook(book)
    }
    …
}
```

()Fragment.onStop هر زمان که فرگمنت شما به حالت stopped می‌رود (یعنی هر زمان که فرگمنت به طور کامل از دید خارج می‌شود) فراخوانی می‌شود. این بدان معناست که هنگامی که کاربر صفحه جزئیات را تمام می‌کند (مانند فشار دادن دکمه بازگشت) داده‌ها ذخیره می‌شوند. همچنین زمانی که کاربر بین برنامه‌ها جا به جا می‌شود (مانند فشار دادن دکمه Home یا استفاده از صفحه ویوی کلی) داده‌ها ذخیره می‌شوند. بنابراین ذخیره در ()onStop نیاز به ذخیره داده‌ها را در هر زمان که کاربر از صفحه

جزئیات خارج می‌شود، برآورده می‌کند و همچنین تضمین می‌کند که اگر فرآیند به دلیل فشار حافظه از بین برود، داده‌های ویرایش شده از بین نمی‌روند.

BookShelf را اجرا کنید و یک کتاب را از لیست انتخاب کنید. داده‌ها را تغییر دهید. دکمه Back را فشار دهید و برای آخرین موفقیت خود جشن بگیرید: تغییراتی که در صفحه جزئیات ایجاد کرده‌اید اکنون در صفحه لیست منعکس شده است. در فصل بعدی دکمه تاریخ را در صفحه جزئیات وصل می‌کنیم تا به کاربر اجازه دهیم تاریخ وقوع کتاب را انتخاب کند.

بیشتر بدانیم: چرا از آرگومان‌های فرگمنت استفاده کنیم؟

در این فصل یک newInstance(…) به فرگمنت خود اضافه کردیم تا هنگام ایجاد یک نمونه جدید از فرگمنت آرگومان‌ها را منتقل کنیم. این الگو نه تنها از نقطه نظر سازماندهی کد مفید است بلکه در مورد آرگومان‌های فرگمنت نیز ضروری می‌باشد. شما نمی‌توانید از یک constructor برای انتقال آرگومان‌ها به یک نمونه فرگمنت استفاده کنید.

به عنوان مثال ممکن است به جای افزودن یک تابع newInstance(UUID) یک سازنده تک آرگومانی را به BookFragment اضافه کنید که انتظار دارد یک شناسه کتاب UUID به عنوان ورودی داشته باشد. با این حال این رویکرد ناقص است. هنگامی که یک تغییر پیکربندی رخ می‌دهد، مدیر فرگمنت اکتیویتی فعلی به طور خودکار فرگمنت‌ای را که قبل از تغییر پیکربندی میزبانی شده بود دوباره ایجاد می‌کند. سپس مدیر فرگمنت آن نمونه فرگمنت جدید را به اکتیویتی جدید اضافه می‌کند.

هنگامی که مدیر فرگمنت پس از تغییر پیکربندی، فرگمنت را دوباره ایجاد می‌کند، سازنده پیش‌فرض و بدون پارامتر فرگمنت را فراخوانی می‌کند. این بدان معناست که پس از چرخش، نمونه فرگمنت جدید داده‌های شناسه کتاب را دریافت نمی‌کند.

پس آرگومان‌های فرگمنت چه تفاوتی با هم دارند؟ آرگومان‌های فرگمنت در مرگ فرگمنت ذخیره می‌شوند. مدیر فرگمنت آرگومان‌ها را دوباره به فرگمنت جدیدی که پس از چرخش دوباره ایجاد می‌کند، متصل می‌نماید. سپس آن فرگمنت جدید می‌تواند از bundle آرگومان‌ها برای ایجاد مجدد وضعیت خود استفاده کند.

همه اینها بسیار پیچیده به نظر می‌رسد. چرا هنگام ایجاد، یک متغیر نمونه روی BookFragment تنظیم نمی‌شود؟

زیرا همیشه کار نمی‌کند. هنگامی که سیستم عامل فرگمنت شما را دوباره ایجاد می‌کند – یا در یک تغییر پیکربندی یا زمانی که کاربر از برنامه شما خارج شده است و سیستم عامل حافظه را بازیابی می‌کند – همه متغیرهای نمونه شما از بین می‌روند. همچنین به یاد داشته باشید که هیچ راهی برای فریب دادن مرگ فرایند هنگام کمبود حافظه وجود ندارد، مهم نیست که چقدر تلاش کنید.

اگر چیزی می‌خواهید که در همه موارد کار کند، باید استدلال‌های خود را ادامه دهید.

یکی از گزینه‌ها استفاده از مکانیزم saved instance state است. می‌توانید شناسه کتاب را به عنوان یک متغیر نمونه معمولی ذخیره کنید، شناسه کتاب را در onSaveInstanceState(Bundle) ذخیره و آن را از Bundle در onCreate(?Bundle) بازیابی کنید. این مکانیزم در همه شرایط کار خواهد کرد.

با این حال حفظ این راه حل سخت است. اگر چند سال دیگر از این فرگمنت بازدید کنید و آرگومان دیگری اضافه کنید، ممکن است به یاد نداشته باشید که آرگومان را در onSaveInstanceState(Bundle) ذخیره کنید. رفتن به این مسیر کمتر صریح است.

توسعه دهندگان اندروید راه حل آرگومان‌های فرگمنت را ترجیح می‌دهند زیرا در اهداف خود بسیار صریح و واضح است. چند سال دیگر شما برمی‌گردید و می‌دانید که شناسه کتاب یک آرگومان است و با خیال راحت به نمونه‌های جدیدی از این فرگمنت منتقل می‌شود. اگر آرگومان دیگری اضافه کنید، می‌دانید که آن را در Bundle آرگومان‌ها ذخیره کنید.

چالش: بارگذاری کارآمد RecyclerView

در حال حاضر هنگامی که کاربر پس از ویرایش یک کتاب به صفحه لیست باز می‌گردد، BookListFragment تمام کتاب‌ها قابل مشاهده را در ویوی بازیافت کننده دوباره لیست می‌کند که به شدت ناکارآمد است، زیرا حداکثر یک کتاب تغییر خواهد کرد.

پیاده‌سازی RecyclerView مربوط به BookListFragment را به روز کنید تا فقط ردیف مرتبط با کتاب تغییر یافته را بروز کند. برای انجام این کار، BookAdapter را به روز کنید.

تا از androidx.recyclerview.widget.ListAdapter<Book, BookHolder> به جای RecyclerView.Adapter<BookHolder> گسترش یابد.

ListAdapter یک RecyclerView.Adapter است که تفاوت بین مجموعه داده‌های فعلی را که از ویوی بازیافت کننده پشتیبانی می‌کند و مجموعه داده جدیدی که برای پشتیبانی از ویوی بازیافت کننده اعمال می‌کنید، کشف می‌کند. این مقایسه در یک رشته پس زمینه اتفاق می‌افتد، بنابراین سرعت رابط کاربری را کاهش نمی‌دهد. ListAdapter به نوبه خود به RecyclerView می‌گوید که فقط ردیف‌هایی را برای داده‌هایی که تغییر کرده‌اند دوباره بروز کند.

ListAdapter از androidx.recyclerview.widget.DiffUtil برای تعیین اینکه کدام بخش از مجموعه داده تغییر کرده است استفاده می‌کند. برای پایان دادن به این چالش باید یک پیاده‌سازی DiffUtil.ItemCallback<Book> را به ListAdapter خود ارائه دهید.

همچنین باید BookListFragment را به روز کنید تا یک لیست کتاب به روز شده را به آداپتور ویوی بازیافت کننده ارسال کنید، به جای اینکه هر بار که می‌خواهید رابط کاربری را به روز کنید، آداپتور RecyclerView را به آداپتور جدید اختصاص دهید.

می‌توانید با فراخوانی ListAdapter.submitList(MutableList<T>?) یک لیست جدید ارسال کنید. یا می‌توانید LiveData را راه اندازی و تغییرات را مشاهده کنید.

برای جزئیات بیشتر در مورد نحوه استفاده از این ابزارها، به صفحات مرجع API برای androidx.recyclerview.widget.DiffUtil و androidx.recyclerview.widget.ListAdapter در developer.android.com/reference/kotlin مراجعه کنید.

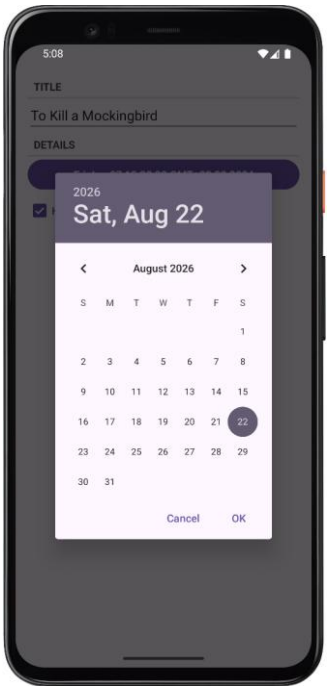
۱۳

دیالوگ‌ها

دیالوگ‌ها توجه و ورودی کاربر را می‌طلبند. آنها برای ارائه یک انتخاب یا اطلاعات مهم مفید هستند. در این فصل یک دیالوگ برای انتخاب تاریخ شروع کتاب به کاربر ارایه خواهیم کرد.

با فشار دادن دکمه date در BookFragment این کادر گفتگو نمایش داده می‌شود (شکل ۱۳.۱).

شکل ۱۳.۱ دیالوگ انتخاب تاریخ کتاب



دیالوگ شکل ۱۳.۱ نمونه‌ای از DatePickerDialog، زیر کلاس AlertDialog است. DatePickerDialog یک اعلان انتخاب تاریخ را برای کاربر به نمایش در می‌آورد و یک شنونده را برای انجام امور فراهم می‌کند. برای ایجاد دیالوگ‌های سفارشی بیشتر، AlertDialog زیر کلاس همه منظوره Dialog است که اغلب از آن استفاده خواهیم کرد.

ایجاد DialogFragment

نمایش یک DatePickerDialog در نمونه‌ای از DialogFragment زیر کلاس Fragment ایده بسیار خوبی است. گرچه نمایش DatePickerDialog بدون DialogFragment امکان پذیر است، اما توصیه نمی‌شود. مدیریت DatePickerDialog توسط FragmentManager گزینه‌های بیشتری برای ارائه دیالوگ در اختیار شما قرار می‌دهد.

علاوه بر این اگر دستگاه را بچرخانیم DatePickerDialog ناپدید می‌شود. اما اگر DatePickerDialog در یک فرگمنت قرار گیرد کادر گفتگو دوباره ایجاد و پس از چرخش دوباره روی صفحه نمایش قرار می‌گیرد.

برای BookShelf می‌خواهیم یک زیر کلاس DialogFragment با نام DatePickerFragment ایجاد کنیم. در DatePickerFragment یک نمونه از DatePickerDialog ایجاد و پیکربندی خواهیم کرد.

DatePickerFragment توسط MainActivity میزبانی خواهد شد.

اولین وظایف شما عبارتند از:

- ایجاد کلاس DatePickerFragment
- ساخت یک DatePickerFragment
- نمایش کادر گفتگو روی صفحه از طریق FragmentManager

بعدا در این فصل، داده‌های لازم را بین BookFragment و DatePickerFragment انتقال خواهیم داد.

یک کلاس جدید با نام DatePickerFragment ایجاد و سوپرکلاس آن را DialogFragment قرار دهید. حتما نسخه Jetpack، DialogFragment: androidx.fragment.app.DialogFragment را انتخاب کنید.

DialogFragment شامل توابع زیر است:

```
onCreateDialog(savedInstanceState: Bundle?): Dialog
```

FragmentManager اکتیویتی میزبان این تابع را به عنوان بخشی از قرار دادن DialogFragment روی صفحه فراخوانی می‌کند.

در DatePickerFragment.kt یک پیاده‌سازی onCreateDialog(Bundle?) اضافه کنید که یک DatePickerDialog با تاریخ فعلی را بسازد (فهرست ۱۳.۱).

لیست ۱۳.۱ ایجاد یک DialogFragment (DatePickerFragment.kt)

```
package com.codeb.bookshelf
import android.app.DatePickerDialog
import android.app.Dialog
import android.os.Bundle
import androidx.fragment.app.DialogFragment
import java.util.Calendar
class DatePickerFragment: DialogFragment() {
    override fun onCreateDialog(savedInstanceState: Bundle?):
            Dialog {
        val calendar = Calendar.getInstance()
        val initialYear = calendar.get(Calendar.YEAR)
        val initialMonth = calendar.get(Calendar.MONTH)
        val initialDay = calendar.get(Calendar.DAY_OF_MONTH)
        return DatePickerDialog(
            requireContext(),
            null,
            initialYear,
            initialMonth,
            initialDay
        )
    }
}
```

سازنده DatePickerDialog چندین پارامتر را در نظر می‌گیرد. اولین مورد یک شی context است که برای دسترسی به منابع لازم برای ویو مورد نیاز است. پارامتر دوم برای شنونده تاریخ است که بعدا در این فصل اضافه خواهیم کرد. سه پارامتر آخر عبارتند از سال، ماه و روزی که انتخابگر تاریخ باید به آن مقداردهی اولیه شود.

نمایش DialogFragment

مانند همه فرگمنت‌ها نمونه‌های DialogFragment توسط FragmentManager اکتیویتی میزبان مدیریت می‌شود.

برای اضافه شدن یک DialogFragment به FragmentManager و قرار دادن روی صفحه می‌توانیم از تابع زیر را در نمونه fragment استفاده کنیم:

```
show(parentFragmentManager, tag: String)
```

پارامتر String به طور منحصر به فرد DialogFragment را در لیست FragmentManager شناسایی می‌کند.

در BookFragment یک ثابت برای تگ DatePickerFragment اضافه کنید.

سپس در onCreateView(...) کدی را که دکمه تاریخ را غیرفعال می‌کند، حذف کنید. یک View.OnClickListener تنظیم کنید تا هنگام فشار دادن دکمه تاریخ در onStart() یک DatePickerFragment را نشان می‌دهد.

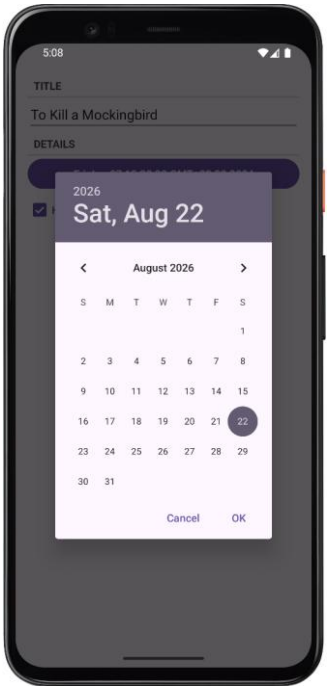
لیست ۱۳.۲ نمایش DialogFragment شما (BookFragment.kt)

```
private const val ARG_BOOK_ID = "Book_id"
private const val DIALOG_DATE = "DialogDate"
class BookFragment : Fragment() {
    ...
    override fun onCreateView(...): View? {
        ...
        dateButton.apply {
            text = book.date.toString()
            isEnabled = false
        }
        ...
    override fun onStart() {
        ...
        titleField.addTextChangedListener(titleWatcher)
        ...
        }
        dateButton.setOnClickListener {
            DatePickerFragment()
           .show(parentFragmentManager,DIALOG_DATE)
        }
    }
}
```

وقتی می‌خواهیم DialogFragment را از یک Fragment نمایش بدهیم از parentFragmentManager استفاده می‌کنیم. هر فرگمنت در اندروید برای کار با دیگر فرگمنت‌ها نیاز به یک FragmentManager دارد. این کلاس وظیفه دارد عملیات‌هایی مثل ,()add ,()replace ,()remove و ()show رو انجام دهد.

اگر فرگمنتت داخل Activity قرار گرفته باشه، parentFragmentManager همان FragmentManager مربوط به Activity می‌باشد. اگر فرگمنتت داخل یک فرگمنت دیگر باشد parentFragmentManager به فرگمنت والد اشاره می‌کند. BookShelf را اجرا و دکمه تاریخ را فشار دهید تا کادر گفتگو را ببینید (شکل ۱۳.۲).

تصویر ۱۳.۲ یک دیالوگ پیکربندی شده



دیالوگ شما روی صفحه عالی به نظر می‌رسد. در بخش بعدی، کد نویسی مربوط را برای تغییر تاریخ توسط کاربر انجام خواهیم داد.

انتقال داده بین دو فرگمنت

ما داده‌ها را بین دو اکتیویتی با استفاده از اکستراهای اینتنت، بین فرگمنت و اکتیویتی با استفاده از رابط callbacks و داده‌ها را از یک اکتیویتی به یک فرگمنت با استفاده از آرگومان‌های فرگمنت منتقل کردیم. اکنون باید داده‌ها را بین دو فرگمنت که توسط یک اکتیویتی میزبانی می‌شوند – BookFragment و DatePickerFragment – منتقل کنیم.

برای ارسال تاریخ کتاب به DatePickerFragment یک تابع newInstance(Date) خواهیم نوشت و Date را به یک آرگومان روی فرگمنت تبدیل می‌کنیم.

برای دریافت تاریخ جدید در BookFragment یک شنونده نتیجه در BookFragment تعریف و به هنگام انتخاب تاریخ از آن استفاده خواهیم کرد.

انتقال داده به DatePickerFragment

برای انتقال داده‌ها به DatePickerFragment آن را در bundle آرگومان‌های DatePickerFragment ذخیره کنید، جایی که DatePickerFragment می‌تواند به آن دسترسی داشته باشد.

ایجاد و تنظیم آرگومان‌های فرگمنت همانطور که در فصل ۱۲ مشاهده کردید، معمولا در یک تابع newInstance(…) انجام می‌شود. در DatePickerFragment.kt یک تابع newInstance(Date) را در companion object اضافه کنید.

لیست ۱۳.۳ افزودن تابع newInstance(Date) (DatePickerFragment.kt)

```
private const val ARG_DATE = "date"
class DatePickerFragment: DialogFragment() {
    override fun onCreateDialog(savedInstanceState: Bundle?):
    Dialog {
        ...
    }
    companion object {
        fun newInstance(date: Date): DatePickerFragment {
            val args = Bundle().apply {
                putLong(ARG_DATE, date.time)
            }
            return DatePickerFragment().apply {
                arguments = args
            }
        }
    }
}
```

در BookFragment فراخوانی سازنده DatePickerFragment را حذف و آن را با فراخوانی DatePickerFragment.newInstance(Date) جایگزین کنید.

لیست ۱۳.۴ افزودن فراخوانی به newInstance(...) (BookFragment.kt)

```
dateButton.setOnClickListener {
    DatePickerFragment()
        .show(parentFragmentManager,DIALOG_DATE)
    DatePickerFragment.newInstance(book.date)
        .show(parentFragmentManager, DIALOG_DATE)
}
```

DatePickerFragment باید DatePickerDialog را با استفاده از اطلاعات موجود در Date مقداردهی اولیه کند. با این حال مقداردهی اولیه DatePickerDialog به نوع Int برای ماه، روز و سال نیاز دارد. اما Date نمی‌تواند نوع Int را مستقیما ارائه دهد.

برای دریافت Int مورد نیاز Date را به شی Calendar ارائه می‌دهیم. سپس اطلاعات مورد نیاز را از Calendar بازیابی کنیم.

در onCreateDialog(Bundle?)، Date را از آرگومان‌ها دریافت و از آن به علاوه Calendar برای مقداردهی اولیه DatePickerDialog استفاده می‌کنیم.

لیست ۱۳.۵ استخراج تاریخ و مقداردهی اولیه DatePickerDialog (DatePickerFragment.kt)

```
override fun onCreateDialog(savedInstanceState: Bundle?):
Dialog {
    val date = Date(arguments?.getLong(ARG_DATE) ?: 0L)
    val calendar = Calendar.getInstance()
    calendar.time = date
    val initialYear = calendar.get(Calendar.YEAR)
    val initialMonth = calendar.get(Calendar.MONTH)
    val initialDay = calendar.get(Calendar.DAY_OF_MONTH)
    return DatePickerDialog(
        requireContext(),
        null,
        initialYear,
        initialMonth,
        initialDay
    )
}
```

اکنون BookFragment با موفقیت به DatePickerFragment می‌گوید که چه تاریخی را نشان دهد. می‌توانید BookShelf را اجرا و مطمئن شوید که همه چیز مانند قبل کار می‌کند.

برای اینکه BookFragment تاریخ را از DatePickerFragment دریافت کند، به راهی برای رابطه بین دو فرگمنت نیاز داریم.

در کار با اکتیویتی‌ها ActivityManager رابطه اکتیویتی والد–فرزند را پیگیری کرد. هنگامی که اکتیویتی فرزند از بین می‌رود ActivityManager می‌داند که کدام اکتیویتی باید نتیجه را دریافت کند.

می‌توانیم با تبدیل BookFragment به فرگمنت هدفِ DatePickerFragment اتصال مشابهی ایجاد کنیم. این اتصال به طور خودکار پس از از بین رفتن BookFragment و DatePickerFragment و ایجاد مجدد توسط سیستم عامل دوباره برقرار می‌شود. برای ایجاد این رابطه، در BookFragment.kt دو ثابت برای کلید درخواست و کلید نتیجه ایجاد و سپس BookFragment را به فرگمنت هدف نمونه DatePickerFragment تبدیل کنید.

لیست ۱۳.۶ تنظیم فرگمنت هدف (BookFragment.kt)

```
const val REQUEST_DATE = "date"
const val SELECTED_DATE = "selected_date"
class BookFragment : Fragment(){
    ...
    override fun onViewCreated(view: View
    , savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState)
        bookDetailViewModel.bookLiveData.observe(
            viewLifecycleOwner,
            Observer { book ->
                book?.let {
                    this.book = book
                    updateUI()
                }
            })

        parentFragmentManager
        .setFragmentResultListener(REQUEST_DATE, this) { _
        , bundle ->
            val date = Date(bundle.getLong(SELECTED_DATE))
            book.date = date
            updateUI()
        }
    }
    …
}
```

اکنون که BookFragment می‌تواند به تاریخ‌های جدید پاسخ دهد، DatePickerFragment باید تاریخ جدید را زمانی که کاربر انتخاب می‌کند، ارسال کند. در DatePickerFragment یک شنونده به DatePickerDialog اضافه کنید که تاریخ را به BookFragment (فهرست ۱۳.۷) ارسال می‌کند.

لیست ۱۳.۷ ارسال تاریخ (DatePickerFragment.kt)

```
private const val REQUEST_DATE = "date"
private const val SELECTED_DATE = "selected_date"
class DatePickerFragment: DialogFragment() {
    override fun onCreateDialog(savedInstanceState: Bundle?):
    Dialog {
        val listener = DatePickerDialog.OnDateSetListener
        { _, year, month, day ->
            val result = Bundle().apply {
               putLong(SELECTED_DATE,
               GregorianCalendar(year, month, day).time.time)
            }
             parentFragmentManager.
             setFragmentResult(REQUEST_DATE, result)
        }
        val date = arguments?.getSerializable(ARG_DATE)
            as Date
        val calendar = Calendar.getInstance()
        calendar.time = date
        val initialYear = calendar.get(Calendar.YEAR)
        val initialMonth = calendar.get(Calendar.MONTH)
        val initialDay = calendar.get(Calendar.DAY_OF_MONTH)
        return DatePickerDialog(
            requireContext(),
            nulllistener,
            initialYear,
            initialMonth,
            initialDay
        )
    }
    …
```

OnDateSetListener برای دریافت تاریخی که کاربر انتخاب می‌کند، استفاده می‌شود. اولین پارامتر DatePicker پارامتری است که نتیجه را بر می‌گرداند. از آنجایی که در این مورد از آن پارامتر استفاده نمی‌کنیم، آن را _ نام گذاری می‌کنید. این یک قرارداد کاتلین برای نشان دادن پارامترهایی است که استفاده نشده‌اند.

تاریخ انتخاب شده در قالب سال، ماه و روز ارائه می‌شود، اما برای ارسال به BookFragment به یک Date نیاز داریم. ما این مقادیر را به GregorianCalendar ارسال و به ویژگی time برای دریافت شی Date دسترسی پیدا می‌کنیم.

خط parentFragmentManager.setFragmentResult(REQUEST_DATE, result)، نتیجه انتخاب شده را از DatePickerFragment به BookFragment ارسال می‌کند.

یعنی DatePickerFragment می‌گوید: «من یک نتیجه دارم، اسمش request_date هست، اینم داده‌هاش.» و BookFragment که Listener ثبت کرده، نتیجه را دریافت می‌کند. BookShelf را اجرا و اطمینان حاصل کنید که روی تاریخ‌ها کنترل دارید.

چالش: دیالوگ‌های بیشتر

فرگمنت دیالوگ دیگری به نام TimePickerFragment بنویسید که به کاربر اجازه می‌دهد با استفاده از ویجت TimePicker انتخاب کند که چه زمانی از روز، کتاب شروع شده است. دکمه دیگری را به BookFragment اضافه کنید، تا TimePickerFragment نمایش داده شود.

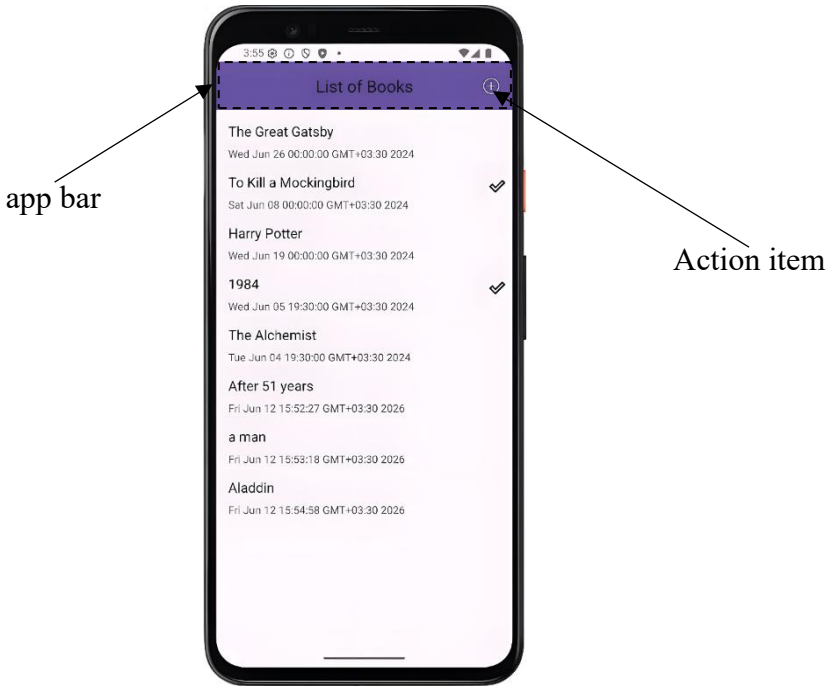
۱۴

نوار برنامه (App Bar)

یکی از اجزای کلیدی هر برنامه اندرویدی که به خوبی طراحی شده است، نوار برنامه یا app bar است. app bar شامل عملیاتی است که کاربر می‌تواند به انتخاب آنها اقدام نماید. همچنین می‌تواند شامل دکمه‌هایی برای پیمایش بهتر اپلیکیشن باشد. علاوه بر آن app bar طراحی بهتری برای برنامه بوجود می‌آورد که می‌تواند محلی برای قرار دادن برند تجاری شما فراهم آورد.

در این فصل یک منو به app bar اضافه خواهیم کرد تا به کاربران اجازه درج کتاب جدید را بدهد (شکل ۱۴.۱)

تصویر ۱۴.۱ BookShelf app bar



ساخت app bar

BookShelf در حال حاضر app bar ندارد. اندروید استودیو همه پروژه‌های جدید را این گونه تنظیم می‌کند. ما برای کنترل بیشتر برای هر فرگمنت app bar جداگانه‌ای می‌سازیم. برای این کار ابتدا باید fragment_book_list را تغییر دهیم تا میزبان یک app bar باشد.

لیست ۱۴.۱ ساخت app bar (fragment_book_list)

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:orientation="vertical"
    android:layout_width="match_parent"
    android:layout_height="match_parent">
    <com.google.android.material.appbar.MaterialToolbar
        android:id="@+id/toolbar"
        android:layout_width="match_parent"
        android:layout_height="?attr/actionBarSize"
        android:background="?attr/colorPrimary"
        android:theme="?attr/actionBarTheme"
        app:titleCentered="true"
        app:title="@string/book_list_title"/>
    <androidx.recyclerview.widget.RecyclerView
xmlns:android="http://schemas.android.com/apk/res/android"
        android:id="@+id/book_recycler_view"
        android:layout_width="match_parent"
        android:layout_height="match_parent"/>
</LinearLayout>
```

یک LinearLayout ساختیم تا میزبان یک app bar و RecyclerView ما باشد. app bar ما از نوع متریال دیزاین گوگل است. (می توانید در مورد آن جستجو کنید.) در یک جمله این عنصر که با com.google.android.material.appbar.MaterialToolbar شروع می شود یک app bar ساخته، یک عنوان از رشته book_list_title در فایل strings.xml انتخاب و عنوان را در وسط نمایش می دهد و از تم متریال دیزاین گوگل استفاده می‌کند.

در Strings.xml عضو جدید یعنی book_list_title را بسازید.

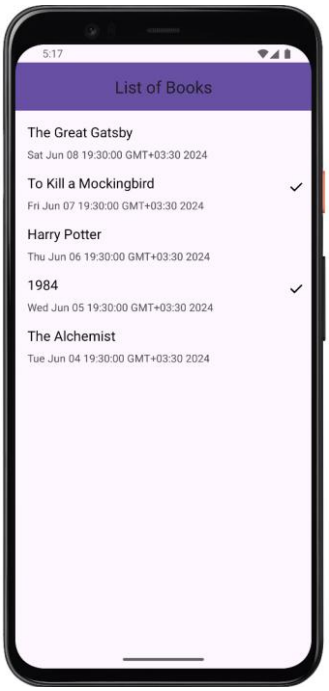
لیست ۱۴.۲ ساخت منابع رشته‌ای (Strings.xml)

```
<resources>
    ...
```

```
    <string name="book_solved_label">Has been read</string>
    <string name="book_list_title">List of Books</string>
</resources>
```

حالا app bar شما باید مانند شکل زیر باشد.

تصویر ۱۴.۲ BookShelf app bar



منوها

قسمت سمت راست بالای app bar برای منو رزرو شده است. این منو از action itemها (که گاهی اوقات به عنوان آیتم‌های منو نیز شناخته می‌شود) تشکیل شده است. این آیتم‌ها می‌توانند اقدامی را برای انجام در صفحه فعلی یا در کل برنامه، تعریف نمایند. فعلا ما یک آیتم اضافه خواهیم کرد تا به کاربر اجازه اضافه کردن کتاب جدید را بدهیم.

action item جدید شما به یک منبع رشته‌ای برای عنوان خود نیاز دارد. res/values/strings.xml را باز و یک برچسب رشته‌ای اضافه کنید که عمل آیتم جدید را توصیف می‌کند.

لیست ۱۴.۳ افزودن رشته برای منو (res/values/strings.xml)

```
<resources>
    ...
    <string name="book_solved_label">Has been read</string>
    <string name="new_book">New Book</string>
</resources>
```

تعریف منو در XML

منوها همانند لیوت نوعی منبع هستند. فایل XML یک منو را ایجاد و آن را در دایرکتوری res/menu پروژه خود قرار می‌دهیم. اندروید یک شناسه منبع برای فایل منو در نظر می‌گیرد و از آن برای بارگذاری منو در کد استفاده می‌کند.

در پنجره ابزار پروژه روی دایرکتوری res کلیک راست کرده و New → Android resource file را انتخاب کنید. نام فایل را fragment_book_list قرار دهید. Resource type را به Menu تغییر و روی OK کلیک کنید.

در اینجا از همان قرارداد نامگذاری برای فایل‌های منو استفاده می‌کنیم که برای فایل‌های لیوت استفاده کردیم. اندروید استودیو res/menu/fragment_book_list.xml را تولید می‌کند، که همان نام فایل لیوت BookListFragment را دارد. با این تفاوت که در پوشه منو قرار گرفته است. در فایل جدید به ویوی Code بروید و یک عنصر item همانطور که در فهرست ۱۴.۴ نشان داده شده است، اضافه کنید.

لیست ۱۴.۴ ایجاد یک منبع منو برای BookListFragment

```
<menu
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto">
    <item
        android:id="@+id/new_book"
        android:icon="@android:drawable/ic_menu_add"
        android:title="@string/new_book"
        app:showAsAction="ifRoom|withText"/>
</menu>
```

ویژگی showAsAction به نحوه نمایش آیتم که به دو صورت نمایش در خود app bar یا نمایش به صورت منو می‌باشد، اشاره دارد. دو مقدار ifRoom و withText را با هم ترکیب کرده‌ایم، بنابراین نماد و متن item در صورت وجود فضا در app bar ظاهر می‌شود. اگر جایی برای نماد وجود داشته باشد اما برای متن نه، فقط نماد قابل مشاهده خواهد بود. اگر جایی برای هیچ کدام وجود نداشته باشد، آیتم به صورت منو نمایش داده می‌شود.

گزینه‌های دیگر برای showAsAction عبارتند از : always برای نمایش همیشگی و never برای عدم نمایش همیشگی در app bar. استفاده از always توصیه نمی‌شود؛ بهتر است از ifRoom استفاده کنید و اجازه دهید سیستم عامل تصمیم بگیرد. استفاده از never انتخاب خوبی برای‌ایتم‌هایی است که کمتر مورد استفاده قرار می‌گیرند. به طور کلی فقط بایدایتم‌هایی را که کاربران به طور مکرر از آنها استفاده می‌کنند، در app bar قرار دهیم تا از به هم ریختگی صفحه جلوگیری کنیم.

ایجاد منو

در کد، منوها با توابع پاسخ از کلاس Activity مدیریت می‌شوند. هنگامی که منو مورد نیاز است، اندروید تابع onCreateOptionsMenu(Menu)، Activity را فراخوانی می‌کند.

با این حال، با توجه به طراحی ما پیاده‌سازی کدها باید در فرگمنت صورت بگیرد و نه در اکتیویتی. Fragment دارای مجموعه‌ای از توابع پاسخ منو خاص خود است که در BookListFragment پیاده‌سازی خواهید کرد.

در BookListFragment.kt و در ()onViewCreated منوی تعریف شده را بارگذاری کنید.

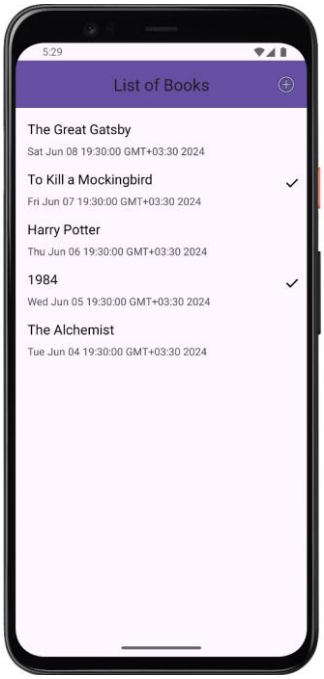
لیست ۱۴.۵ بارگذاری منو (BookListFragment.kt)

```
override fun onViewCreated(view: View, savedInstanceState:
Bundle?) {
    super.onViewCreated(view, savedInstanceState)
    bookListViewModel.bookListLiveData.observe(
        ...)
    val toolbar = view.findViewById<MaterialToolbar>(
        R.id.toolbar)
    toolbar.inflateMenu(R.menu.fragment_book_list)
}
```

ما MenuInflater.inflate(Int, Menu) را فراخوانی و شناسه منبع فایل منوی خود را به آن ارسال می‌کنیم. با این کار نمونه Menu با موارد تعریف شده، بارگذاری می‌شود.

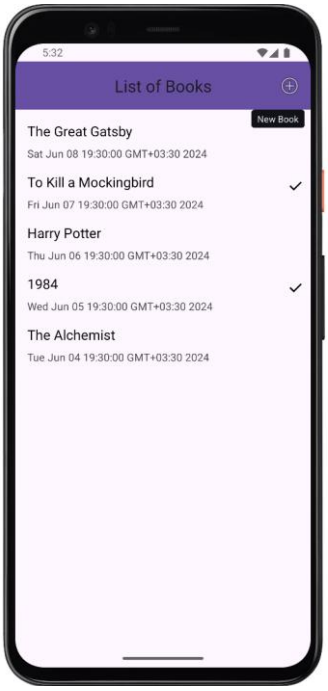
اکنون می‌توانید BookShelf را برای مشاهده منو اجرا کنید (تصویر ۱۴.۴).

تصویر ۱۴.۴ آیکون New Book در app bar



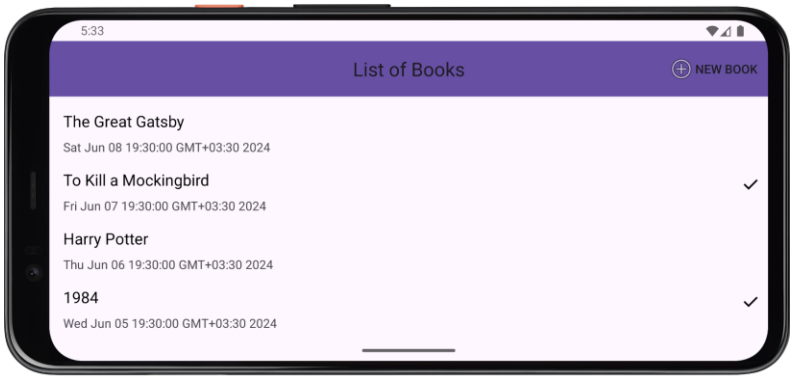
متن آیتم کجاست؟ اکثر تلفن‌ها فقط فضای کافی برای نماد در جهت عمودی دارند. می‌توانید یک نماد را در app bar به مدت طولانی فشار دهید تا عنوان آن نمایان شود (تصویر ۱۴.۵).

تصویر ۱۴.۵ با فشار دادن طولانی یک نماد در app bar عنوان قابل مشاهده است.



در جهت افقی در app bar فضایی برای نماد و متن وجود دارد (شکل ۱۴.۶).

تصویر ۱۴.۶ نماد و متن در app bar افقی



پاسخ به انتخاب‌های منو

برای پاسخ به لمس آیتم New Book توسط کاربر، ما شنونده‌ای را روی app bar با عنوان setOnMenuItemClickListener تنظیم می‌کنیم که درصورت فشرده شدن هر یک از آیتم ها واکنش نشان دهد. البته منو ما یک آیتم دارد پس باید در صورت لمس آن فراخوانی به تابع addBook(Book)، repository را داشته باشیم. برای اینکار باید این تابع را در BookListViewModel پیاده‌سازی کنیم.

لیست ۱۴.۶ افزودن کتاب جدید (BookListViewModel.kt)

```
class BookListViewModel: ViewModel() {
    private val bookRepository = BookRepository.get()
    val bookListLiveData = bookRepository.getBooks()
    fun addBook(book: Book) {
        bookRepository.addBook(book)
    }
}
```

می‌توانیم با بررسی شناسه MenuItem تعیین کنیم که کدام مورد انتخاب شده است و سپس به طور مناسب پاسخگو باشیم. این شناسه مربوط به شناسه‌ای است که به MenuItem در فایل منوی خود اختصاص دادیم.

در BookListFragment.kt با پیاده‌سازی setOnMenuItemClickListener برای ایجاد کتاب جدید، ذخیره آن در پایگاه داده و سپس اطلاع دادن به اکتیویتی والد، به انتخاب MenuItem پاسخ می‌دهیم.

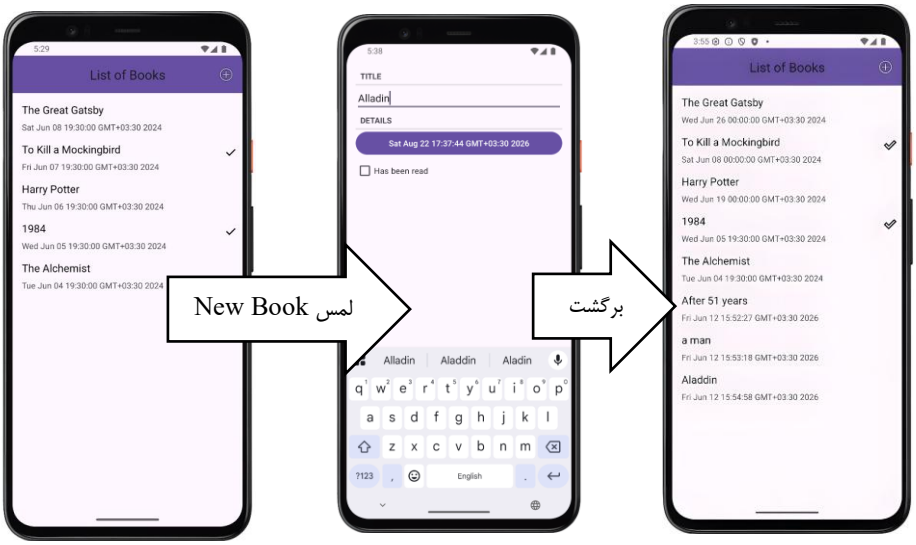
لیست ۱۴.۷ پاسخ به انتخاب منو (BookListFragment.kt)

```
override fun onViewCreated(view: View, savedInstanceState:
Bundle?) {
    …
    toolbar.inflateMenu(R.menu.fragment_book_list)
    toolbar.setOnMenuItemClickListener { item ->
        when (item.itemId) {
            R.id.new_book -> {
                val book= Book()
                bookListViewModel.addBook(book)
                callbacks?.onBookSelected(book.id)
                true
            }
            else -> false
        }
    }
}
```

توجه داشته باشید که این تابع یک مقدار بولی را برمی‌گرداند . هنگامی که MenuItem را مدیریت کردیم، باید true را برگردانیم تا نشان دهیم که پردازش بیشتری لازم نیست.

BookShelf را اجرا کنید. آیتم منوی جدیدتان را امتحان کنید تا یک کتاب جدید اضافه کنید. شما باید ببینید که کتاب جدید در لیست کتاب ظاهر می‌شود (شکل ۱۴.۷).

شکل ۱۴.۷ کتاب جدید



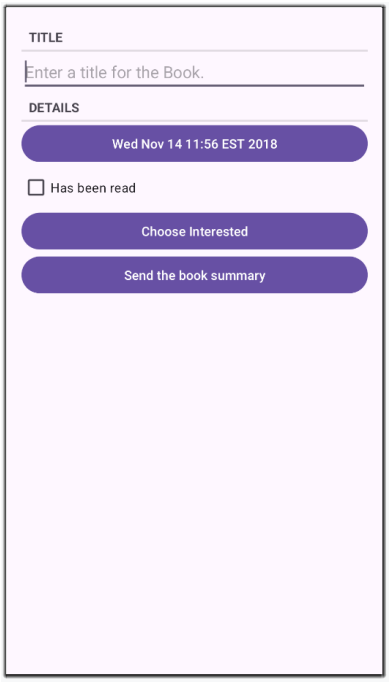
۱۵

اینتنت ضمنی (Implicit Intent)

در اندروید می‌توانیم با استفاده از یک اینتنت، اکتیویتی را در برنامه دیگری در دستگاه راه اندازی کنیم. در یک اینتنت صریح کلاس اکتیویتی را برای شروع مشخص می‌کنیم تا سیستم عامل آن را راه اندازی کند. اما در یک اینتنت ضمنی ما کاری را که باید انجام شود توصیف می‌کنیم سپس سیستم عامل اکتیویتی مناسب را از برنامه دیگری برای انجام آن کار راه اندازی می‌کند.

در BookShelf از اینتنت‌های ضمنی استفاده خواهیم کرد تا امکان انتخاب فردی از لیست مخاطبین کاربر به عنوان علاقمند به یک کتاب را فراهم کنیم. کاربر از بین هر برنامه مخاطبین که روی دستگاه نصب شده است می‌تواند یکی را انتخاب کند (شکل ۱۵.۱).

تصویر ۱۵.۱ باز کردن مخاطبین و برنامه‌های ارسال متن



استفاده از اینتنت‌های ضمنی برای استفاده از قابلیت سایر برنامه‌ها بسیار آسان‌تر از پیاده‌سازی این قابلیت‌ها در برنامه خودمان است. کاربران همچنین به خاطر امکان استفاده از برنامه‌هایی که از قبل با آن آشنا هستند، قدردان خواهند بود.

قبل از اینکه بتوانیم این اینتنت‌های ضمنی را ایجاد کنیم، تنظیماتی برای انجام در BookShelf داریم:

اول باید دکمه‌های Choose Interested و Send the book summary را به لیوت BookFragment اضافه کنید. در ادامه یک ویژگی Interested را به کلاس Book اضافه کنید که نام علاقه‌مند را در خود جای دهد. و در نهایت ایجاد گزارش کتاب با استفاده از format resource strings را پیاده می‌کنیم.

افزودن دکمه‌ها

لیوت BookFragment را به روز کنید تا شامل دکمه‌های جدید باشد: یعنی یک دکمه Choose Interested و یک دکمه Send the book summary. ابتدا رشته‌هایی را که این دکمه‌ها نمایش می‌دهند، اضافه کنید.

لیست ۱۵.۱ افزودن رشته‌های دکمه (res/values/strings.xml)

```
<resources>
    …
    <string name="new_book">New Book</string>
    <string name="book_interested_text">
        Choose Interested</string>
    <string name="book_summary_text">
        Send the book summary</string>
</resources>
```

در res/layout/fragment_book.xml همانطور که در فهرست ۱۵.۲ نشان داده شده است، دو ویجت دکمه اضافه کنید.

لیست ۱۵.۲ افزودن دکمه‌ها (fragment_Book.xml)

```
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:orientation="vertical"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:layout_margin="16dp"
    android:id="@+id/fragment_book">
    …
```

```
    <CheckBox
        android:id="@+id/book_solved"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="@string/book_solved_label"/>
    <Button
        android:id="@+id/book_interested"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="@string/book_interested_text"/>
    <Button
        android:id="@+id/book_summary"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="@string/book_summary_text"/>
</LinearLayout>
```

پیش‌نمایش لیوت به روز شده را مشاهده کنید.

افزودن Interested به لایه مدل

در مرحله بعد Book.kt را باز و متغییری اضافه کنید تا نام علاقمند را در خود جای دهد.

لیست ۱۵.۳ افزودن ویژگی interested (Book.kt)

```
@Entity
data class Book(@PrimaryKey val id: UUID = UUID.randomUUID(),
                var title: String = "",
                var date: Date = Date(),
                var isSolved: Boolean = false,
                var interested: String = "")
```

اکنون باید یک ویژگی اضافی به پایگاه داده کتاب خود اضافه کنید. این امر مستلزم افزایش نسخه کلاس BookDatabase و همچنین اعلام نحوه انتقال پایگاه داده خود بین نسخه‌ها به Room است.

برای اینکه به Room بگوییم چگونه پایگاه داده خود را از یک نسخه به نسخه دیگر تغییر دهد، یک Migration ارائه می‌دهیم. BookDatabase.kt را باز کنید، یک Migration اضافه و نسخه را افزایش دهید.

لیست ۱۵.۴ افزودن Migration پایگاه داده (database/BookDatabase.kt)

```
@Database(entities = [ Book::class ], ~~version=1~~ version=2)
@TypeConverters(BookTypeConverters::class)
abstract class BookDatabase : RoomDatabase() {
    abstract fun BookDao(): BookDao
```

```
}
val migration_1_2 = object : Migration(1, 2) {
    override fun migrate(database: SupportSQLiteDatabase) {
        database.execSQL(
            "ALTER TABLE Book ADD COLUMN interested TEXT NOT
NULL DEFAULT ''"
        )
    }
}
```

از آنجایی که نسخه اولیه پایگاه داده شما روی ۱ تنظیم شده است، آن را به ۲ افزایش می دهیم. سپس یک شی Migration ایجاد می‌کنیم که حاوی دستورالعمل‌هایی برای به روز رسانی پایگاه داده شما است.

سازنده کلاس Migration دو پارامتر را در بر می‌گیرد. اولی نسخه پایگاه داده‌ای است که از آن مهاجرت و دومی نسخه‌ای است که به آن مهاجرت می‌کنید. در این حالت شماره نسخه ۱ و ۲ را ارائه می‌دهید.

تنها تابعی که باید در شی مهاجرت خود پیاده‌سازی کنید، migrate(SupportSQLiteDatabase) است. شما از پارامتر database برای اجرای هر دستور SQL لازم برای ارتقاء جداول خود استفاده می‌کنید. (Room از SQLite در پشت صحنه، همانطور که در فصل ۱۱ می‌خوانید استفاده می‌کند.) دستور ALTER TABLE که نوشته‌اید ستون interested جدید را به جدول Book اضافه می‌کند.

پس از ایجاد Migration، باید آن را در اختیار پایگاه داده خود قرار دهید.

BookRepository.kt را باز و هنگام ایجاد نمونه BookDatabase خود، Migration را به Room ارائه دهید.

لیست ۱۵.۵ ارائه Migration به Room (BookRepository.kt)

```
class BookRepository private constructor(context: Context) {
    private val database : BookDatabase =
    Room.databaseBuilder(
        context.applicationContext,
        BookDatabase::class.java,
        DATABASE_NAME
    ).buid().addMigrations(migration_1_2).build()
    …
}
```

برای راه اندازی migration، addMigrations(…) را فراخوانی می‌کنیم. قبل از فراخوانی تابع build() addMigrations() تعداد متغیری از آبجکت‌ها Migration را می‌پذیرد، بنابراین می‌توانیم تمام Migrationهای خود را هنگام اعلام ارسال کنیم.

هنگامی که برنامه شما راه اندازی می‌شود و Room پایگاه داده را می‌سازد، ابتدا نسخه پایگاه داده موجود در دستگاه را بررسی می‌کند. اگر این نسخه با نسخه‌ای که در حاشیه‌نویسی Database@ تعریف کرده‌اید مطابقت نداشته باشد، Room برای Migrationهای مناسب برای به روز رسانی پایگاه داده به آخرین نسخه، رجوع می‌کند.

مهم است که Migrationهایی را برای تبدیل پایگاه داده خود ارائه دهیم. اگر این کار را انجام ندهیم، Room نسخه قدیمی پایگاه داده را حذف و نسخه جدیدی را دوباره ایجاد می‌کند. این بدان معنی است که تمام داده‌ها از بین می‌روند و منجر به نارضایتی کاربران می‌شوند.

هنگامی که مهاجرت شما انجام شد، BookShelf را اجرا کنید تا مطمئن شوید همه چیز به درستی عمل می‌نماید. رفتار برنامه باید مانند قبل از مهاجرت باشد و باید کتابی را که در فصل ۱۴ اضافه کرده‌اید، مشاهده نمایید. به زودی از ستون جدید اضافه شده، استفاده خواهید کرد.

استفاده از Format String

آخرین مرحله مقدماتی، ایجاد یک الگو خلاصه کتاب است. از آنجایی که تا زمان اجرا از جزئیات یک کتاب مطلع نخواهیم شد، باید از یک رشته فرمت با متغیرها استفاده کنیم که می‌تواند در زمان اجرا جایگزین شود. در اینجا Format String که باید استفاده کنیم، آمده است:

```
%1$s! The Book %2$s. %3$s, and %4$s
```

%1$s، %2$s و غیره متغیرهایی هستند که انتظار آرگومان‌های رشته‌ای را دارند. در کد، getString(…) را فراخوانی و format string و چهار رشته دیگر را به ترتیبی که باید جایگزین متغیرها شوند، ارسال کنید.

ابتدا، در strings.xml، رشته‌های نشان داده شده در فهرست ۱۵.۶ را اضافه کنید.

لیست ۱۵.۶ افزودن منابع رشته (res/values/strings.xml)

```
<resources>
    …
    <string name="book_summary_text">
        Send the book summary</string>
    <string name="book_summary">
        %1$s! The book has been started on %2$s. %3$s
        , and %4$s
    </string>
    <string name="book_summary_solved">has been read</string>
    <string name="book_summary_unsolved">
        has not been read</string>
    <string name="book_summary_no_interested">
```

```
        there is no interested.</string>
    <string name="book_summary_interested">
        the interested is %s.</string>
    <string name="book_summary_subject">
        BookShelf Book Summary</string>
    <string name="send_summary">
        Send book summary via</string>
</resources>
```

در BookFragment.kt تابعی اضافه کنید که چهار رشته ایجاد و سپس آنها را با هم ترکیب و یک گزارش کامل را برمی‌گرداند.

لیست ۱۵.۷ افزودن تابع getBookSummary() (BookFragment.kt)

```
class BookFragment : Fragment(){
    …
    private val bookDetailViewModel: BookDetailViewModel
        by viewModels()
    private fun getBookSummary(): String {
        val solvedString = if (book.isSolved) {
            getString(R.string.book_summary_solved)
        } else {
            getString(R.string.book_summary_unsolved)
        }
        val sdf = SimpleDateFormat(
            "yyyy-MM-dd", Locale.getDefault())
        val dateString = sdf.format(book.date)
        var suspect = if (book.interested.isBlank()) {
            getString(R.string.book_summary_no_interested)
        } else {
            getString(R.string.book_summary_interested
            , book.interested)
        }
        return getString(R.string.book_summary,
            book.title, dateString, solvedString, suspect)
    }
    …
}
```

اکنون مقدمات کامل شده است. حال می‌توانیم به اینتنت‌های ضمنی بپردازیم.

استفاده از اینتنت‌های ضمنی

Intent شیی است که عمل مورد نظر را برای سیستم عامل توصیف می‌کند. با اینتنت‌های صریحی که تاکنون ایجاد کرده‌ایم به صراحت اکتیویتی را که می‌خواستیم سیستم عامل شروع کند، اعلام کردیم. مانند:

```
val intent = Intent(this, CheatActivity::class.java)
startActivity(intent)
```

با یک اینتنت ضمنی کاری را که می‌خواهیم انجام دهیم برای سیستم عامل توصیف می‌کنیم. سپس سیستم عامل اکتیویتی را که خود را توانا به انجام آن کار معرفی کرده است، شروع می‌کند. اگر سیستم عامل بیش از یک اکتیویتی پیدا کند، به کاربر حق انتخاب می‌دهد.

بخش‌های یک اینتنت ضمنی

بخش‌های حیاتی یک اینتنت برای تعریف کاری که باید انجام شود، عبارتند از:

- عملی (action) که می‌خواهیم انجام دهیم: معمولا ثابت‌های کلاس Intent هستند. به عنوان مثال اگر می‌خواهیم یک URL را مشاهده کنیم از Intent.ACTION_VIEW و برای ارسال از Intent.ACTION_SEND استفاده می‌کنیم.
- داده: می‌تواند چیزی خارج از دستگاه مانند URL یک صفحه وب باشد.
- نوع داده: یک نوع MIME مانند text/html یا audio/mpeg3 است. اگر یک اینتنت شامل مکانی برای داده‌ها باشد، معمولا می‌توان نوع آن را از آن داده استنباط کرد.
- دسته‌بندی‌های اختیاری: اگر از اکشن برای توصیف کارهایی که باید انجام شود استفاده کنیم، این دسته‌بندی معمولا توصیف می‌کند کجا، چه زمانی یا چگونه می‌خواهیم از یک اکتیویتی استفاده کنیم. اندروید از دسته android.intent.category.LAUNCHER استفاده می‌کند تا نشان دهد که یک اکتیویتی باید در بالای سایر برنامه‌ها نمایش داده شود. از سوی دیگر دسته android.intent.category.INFO اکتیویتی را نشان می‌دهد که اطلاعات مربوط به یک پکیج را به کاربر نشان می‌دهد اما نباید در لانچر نشان داده شود.

بنابراین به عنوان مثال، یک اینتنت ضمنی ساده برای مشاهده یک وب سایت، شامل یک عمل Intent.ACTION_VIEW و یک داده Uri که به URL یک وب سایت اشاره می‌کند، است.

بر اساس این اطلاعات سیستم عامل اکتیویتی مناسب یک برنامه مناسب را راه اندازی می‌کند. (اگر بیش از یک گزینه پیدا کند، کاربر حق انتخاب دارد.)

یک اکتیویتی خود را به عنوان یک اکتیویتی مناسب برای عمل ACTION_VIEW از طریق یک فیلتر اینتنت در مانیفست اعلام می‌کند.

برای پاسخ به اینتنت‌های ضمنی، یک اکتیویتی باید دسته DEFAULT را به صراحت در یک intent-filter تنظیم کند. عنصر action در intent-filter به سیستم عامل می‌گوید که اکتیویتی قادر به انجام کار است و دسته DEFAULT به سیستم عامل می‌گوید که زمانی که سیستم عامل داوطلب می‌خواهد، این اکتیویتی باید برای کار در نظر گرفته شود. این دسته پیش‌فرض به طور ضمنی به هر اینتنت ضمنی اضافه می‌شود.

اینتنت‌های ضمنی درست مانند اینتنت‌های صریح می‌توانند شامل extraها باشند. اما هر extra در یک اینتنت ضمنی توسط سیستم عامل برای یافتن یک اکتیویتی مناسب استفاده نمی‌شود.

توجه داشته باشید که بخش‌های action و داده یک اینتنت را می‌توان در ارتباط با یک اینتنت صریح نیز استفاده کرد. معادل گفتن به یک اکتیویتی خاص برای انجام کاری خاص است.

ارسال خلاصه کتاب

بیایید ببینیم یک اینتنت ضمنی برای ارسال خلاصه کتاب در BookShelf چگونه عمل می‌کند. کاری که می‌خواهیم انجام دهیم ارسال متن ساده است. خلاصه کتاب یک رشته است. بنابراین اکشن اینتنت ضمنی ACTION_SEND خواهد بود. به هیچ داده‌ای اشاره نمی‌کند یا هیچ دسته‌ای ندارد، اما نوع text/plain را مشخص می‌کند.

در onCreateView(…) موجود در BookFragment به دکمه SEND BOOK REPORT ارجاع دهید، سپس یک شنونده را روی آن در onStart() تنظیم کنید. در پیاده‌سازی شنونده یک اینتنت ضمنی ایجاد و آن را به startActivity (Intent) پاس کنید.

لیست ۱۵.۸ ارسال خلاصه کتاب (BookFragment.kt)

```
class BookFragment : Fragment(){
    …
    private lateinit var sendBookSummaryButton: Button
    …
    override fun onCreateView(
        inflater: LayoutInflater,
        container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View? {
        …
        solvedCheckBox =
        view.findViewById(R.id.book_solved) as CheckBox
        sendBookSummaryButton =
        view.findViewById(R.id.book_summary) as Button
        return view
    }
    override fun onStart() {
        …
        dateButton.setOnClickListener {
            DatePickerFragment.newInstance(book.date)
            .show(parentFragmentManager, DIALOG_DATE)
        }
        sendBookSummaryButton.setOnClickListener {
            Intent(Intent.ACTION_SEND).apply {
                type = "text/plain"
                putExtra(Intent.EXTRA_TEXT, getBookSummary())
                putExtra(
                    Intent.EXTRA_SUBJECT,
                    getString(R.string.book_summary_subject))
            }.also { intent ->
                startActivity(intent)
            }
```

```
            }
        }
}
```

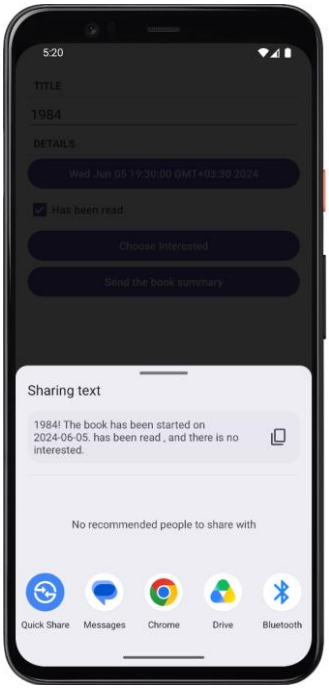
در اینجا از سازنده Intent استفاده کردیم که رشته ثابتی را به عنوان تعریف اکشن می‌پذیرد.

متن خلاصه و رشته موضوع گزارش را به عنوان extra اضافه می‌کنیم. توجه داشته باشید که این موارد اضافی از ثابت‌های تعریف شده در کلاس Intent استفاده می‌کنند. هر اکتیویتی که به این اینتنت پاسخ دهد، این ثابت‌ها و چگونگی کار با آنهار را می‌داند.

شروع یک اکتیویتی از یک فرگمنت تقریبا مانند شروع یک اکتیویتی از یک اکتیویتی دیگر است. شما تابع startActivity(Intent) موجود در Fragment را فراخوانی می‌کنید که تابع Activity مربوطه را در پشت صحنه فراخوانی می‌کند.

BookShelf را اجرا و دکمه Send the book summary را فشار دهید. از آنجایی که این اینتنت احتمالا با بسیاری از اکتیویتی‌های دستگاه مطابقت دارد، احتمالا لیستی از اکتیویتی‌های ارائه شده در یک انتخابگر را مشاهده خواهید کرد (شکل ۱۵.۲). در صورت لزوم برای مشاهده همه اکتیویتی‌ها در لیست به پایین اسکرول کنید.

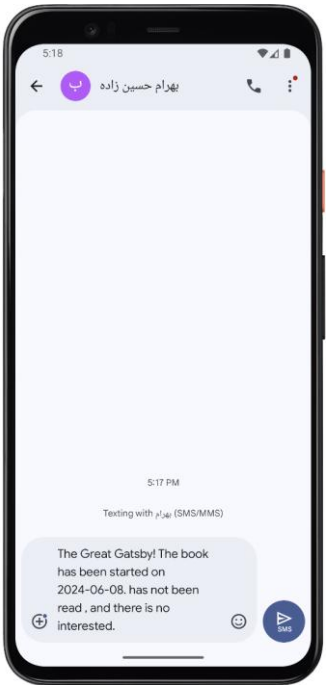
تصویر ۱۵.۲ اکتیویتی‌هایی که داوطلب ارسال گزارش کتاب شما هستند



برنامه مورد نظرتان را انتخاب کنید. گزارش کتاب در برنامه‌ای که انتخاب کرده‌اید بارگذاری خواهد شد. تنها کاری که باید انجام دهید این است که آدرس دهی و ارسال کنید.

توجه داشته باشید که برنامه‌هایی مانند Gmail و Google Drive از شما می‌خواهند با حساب Google وارد شوید. انتخاب برنامه Messages ساده‌تر است که نیازی به ورود به سیستم ندارد.

تصویر ۱۵.۳ ارسال گزارش کتاب با برنامه Messages



اگر انتخابی برای برنامه مناسب ظاهر نشود به معنای یکی از این دو موارد است. یا قبلا یک برنامه پیش‌فرض را برای یک اینتنت ضمنی یکسان تنظیم کرده‌اید و یا دستگاه شما فقط یک اکتیویتی دارد که می‌تواند به این اینتنت پاسخ دهد.

اغلب بهتر است برای یک اکشن از برنامه پیش‌فرض استفاده کنید. با این حال در BookShelf همیشه می‌خواهیم کاربر برای ACTION_SEND مجاز به انتخاب از بین برنامه‌ها باشد. چون ممکن است کاربر زمانی بخواهد محتاط باشد و گزارش کتاب را ایمیل کند، اما زمانی دیگر ممکن است پیام رسان را ترجیح دهد.

می‌توانیم یک صفحه انتخاب ایجاد کنیم تا هر بار که از یک اینتنت ضمنی برای شروع یک اکتیویتی استفاده می‌شود، نشان داده شود. پس از ایجاد اینتنت ضمنی خود مانند قبل تابع Intent.createChooser(Intent, String) را فراخوانی کنید و اینتنت ضمنی و یک رشته را برای عنوان صفحه ارسال کنید. سپس اینتنت بازگردانده شده از createChooser(…) را به startActivity(…) ارسال کنید.

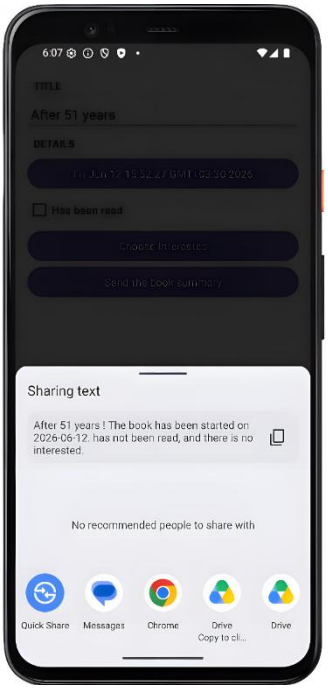
در BookFragment.kt یک انتخابگر (chooser) برای نمایش اکتیویتی‌هایی که به اینتنت ضمنی شما پاسخ می‌دهند ایجاد کنید.

لیست ۱۵.۹ با استفاده از انتخابگر (BookFragment.kt)

```
sendBookSummaryButton.setOnClickListener {
    Intent(Intent.ACTION_SEND).apply {
        type = "text/plain"
        putExtra(Intent.EXTRA_TEXT, getBookSummary())
        putExtra(
            Intent.EXTRA_SUBJECT,
            getString(R.string.book_summary_subject))
    }.also { intent ->
        startActivity(intent)
        val chooserIntent =
            Intent.createChooser(intent,
            getString(R.string.book_summary))
        startActivity(chooserIntent)
    }
}
```

BookShelf را دوباره اجرا کنید. تا زمانی که بیش از یک اکتیویتی داشته باشید که بتواند اینتنت شما را مدیریت کند، لیستی برای انتخاب به شما پیشنهاد می‌شود (شکل ۱۵.۴).

تصویر ۱۵.۴ ارسال متن با انتخابگر



درخواست مخاطب از Android

اکنون قصد داریم یک اینتنت ضمنی دیگر ایجاد کنیم که کاربران را قادر می‌سازد یک نفر را جهت انتخاب به عنوان علاقمند به کتاب، از مخاطبین خود انتخاب کنند. این اینتنت ضمنی یک اکشن و داده خواهد داشت. اکشن Intent.ACTION_PICK و داده یک مخاطب از پایگاه داده مخاطبین است که در ContactsContract.Contacts.CONTENT_URI قرار دارد. به طور خلاصه، از اندروید می‌خواهیم ما را در انتخاب مخاطب کمک کند.

ابتدا به تعریف دکمه می پردازیم.

لیست ۱۵.۱۰ افزودن یک فیلد برای دکمه Choose Interested (BookFragment.kt)

```
class BookFragment : Fragment(){
    …
    private lateinit var sendBookSummaryButton: Button
    private lateinit var chooseInterestedButton: Button
    …
    override fun onCreateView(
        inflater: LayoutInflater,
        container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View? {
        …
        sendBookSummaryButton =
        view.findViewById(R.id.book_summary) as Button
        chooseInterestedButton =
        view.findViewById(R.id.book_interested) as Button
        return view
    }
    …
}
```

یک لانچر برای گرفتن مخاطب تعریف و سپس در ()onStart شنونده کلیک را روی دکمه تنظیم کنید. اینتنت ضمنی را در شنونده برای لانچ پیاده نمایید. همچنین اگر مخاطبی اختصاص داده شد نامش را روی دکمه Choose Interested نشان دهید. در مرحله بعد ()updateUI را تغییر دهید تا متن را روی دکمه Choose Interested تنظیم کند.

فهرست ۱۵.۱۱ ارسال یک اینتنت ضمنی (BookFragment.kt)

```
class BookFragment : Fragment(){
    ...
    private val bookDetailViewModel: BookDetailViewModel
    by viewModels()
```

```
private val pickContact =
    registerForActivityResult(
    ActivityResultContracts.StartActivityForResult())
    { result ->
        if (result.resultCode == Activity.RESULT_OK) {
            val uri = result.data?.data ?:
            return@registerForActivityResult
            val cursor = context?.contentResolver?.query(
                uri,
                arrayOf(
                ContactsContract
                .CommonDataKinds.Phone.DISPLAY_NAME
                ),
                null, null, null
            )
            cursor?.use {
                if (it.moveToFirst()) {
                    val name= it.getString(0)
                    book.interested = name
                    chooseInterestedButton.text=name
                    bookDetailViewModel.saveBook(book)
                }
            }
        }
    }
...
private fun updateUI() {
    titleField.setText(book.title)
    dateButton.text = book.date.toString()
    solvedCheckBox.apply {
        isChecked = book.isSolved
        jumpDrawablesToCurrentState()
    }
    if(book.interested.isNotEmpty())
        chooseInterestedButton.text=book.interested
}

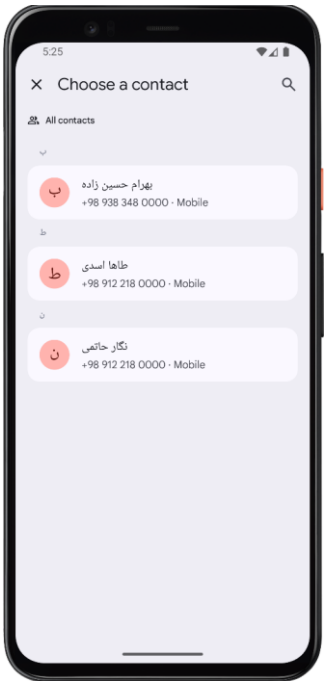
...

override fun onStart() {
    ...
    sendBookSummaryButton.setOnClickListener {
        ...
    }
    chooseInterestedButton.apply {
        val intent = Intent(Intent.ACTION_PICK)
        .apply {
        type =
```

```
            ContactsContract.CommonDataKinds.Phone.CONTENT_TYPE
            }
                setOnClickListener {
                    pickContact.launch(intent)
                }
            }
        }
}
```

BookShelf را اجرا و دکمه Choose Interested را فشار دهید. باید لیستی از مخاطبین را ببینید (تصویر ۱۵.۵).

تصویر ۱۵.۵ فهرستی از علاقه‌مندان احتمالی



اگر برنامه مخاطبین دیگری نصب کرده باشید، صفحه نمایش شما متفاوت به نظر می‌رسد. البته این یکی از مزایای اینتنت‌های ضمنی است. زیرا برای استفاده از برنامه خود لازم نیست نام برنامه مخاطبین را بدانید. کاربران می‌توانند هر برنامه‌ای را که دوست دارند نصب کنند و سیستم عامل آن را پیدا و راه اندازی می‌کند.

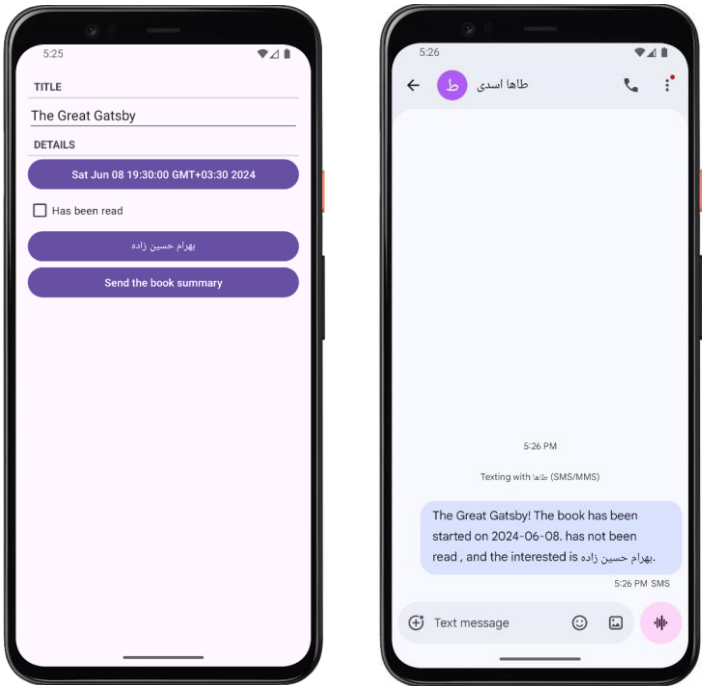
کد مربوط به لانچر داخل یک Fragment اجرا می‌شود و چون Fragment خودش مستقیماً به contentResolver دسترسی ندارد، از context?.contentResolver استفاده شده تا از Context اکتیویتی والد برای دسترسی به مخاطبین استفاده کند؛ در این کد پس از اجرای یک Implicit Intent برای انتخاب مخاطب، URI مخاطب انتخاب‌شده دریافت می‌شود و با استفاده از contentResolver.query فقط فیلد نام (DISPLAY_NAME) از دیتابیس مخاطبین خوانده می‌شود، سپس اگر Cursor روی اولین ردیف قرار گرفت، نام استخراج شده و در متغیر book.interested ذخیره می‌شود، متن دکمه chooseInterestedButton به همان نام تغییر می‌کند و در نهایت با bookDetailViewModel.saveBook(book) تغییرات ذخیره می‌شود؛ این فرآیند به‌صورت کامل انتخاب مخاطب، خواندن نام او، نمایش در UI و ذخیره‌سازی در ViewModel را انجام می‌دهد.

به محض دریافت نام مظنون، کتاب را در پایگاه داده ذخیره می‌کنیم. دلیل انجام این کار مربوط به چرخه حیات می‌شود. هنگامی که BookFragment از سر گرفته (resumed) می‌شود، onViewCreated(…) فراخوانی می‌شود و کتاب را از پایگاه داده پرس و جو می‌کند. اما onActivityResult(…) قبل از آن فراخوانی می‌شود. بنابراین هنگامی که کتاب پایگاه داده دریافت می‌شود، کتاب را با اطلاعات مظنون بازنویسی می‌کند. برای جلوگیری از از دست دادن داده‌های مظنون به محض آماده شدن آن را ذخیره می‌کنیم.

برنامه خود را اجرا کنید. مطمئن شوید که آن را روی دستگاهی اجرا می‌کنید که دارای برنامه مخاطبین است — اگر دستگاه اندرویدی شما شبیه‌ساز ندارد، از شبیه‌ساز استفاده کنید. اگر از شبیه‌ساز استفاده می‌کنید، قبل از اجرای BookShelf چند مخاطب را با استفاده از برنامه مخاطبین آن اضافه کنید. سپس برنامه خود را اجرا کنید.

یک مخاطب را انتخاب کنید. نام فردی که انتخاب کرده‌اید باید در دکمه chooseInterestedButton ظاهر شود. سپس یک گزارش کتاب ارسال کنید. نام مظنون باید در گزارش کتاب ظاهر شود (شکل ۱۵.۶).

شکل ۱۵.۶ نام مظنون روی دکمه و در گزارش کتاب



مجوزهای مخاطبین

چگونه اجازه خواندن از پایگاه داده مخاطبین را دریافت می‌کنیم؟ این برنامه مجوزهای خودش را در اختیار شما می‌گذارد. برنامه مخاطبین دارای مجوزهای کامل برای استفاده از پایگاه داده مخاطبین است. وقتی برنامه مخاطبین یک URI داده را در یک Intent به اکتیویتی والد برمی‌گرداند، همچنین پرچم Intent.FLAG_GRANT_READ_URI_PERMISSION را اضافه می‌کند. این پرچم به اندروید نشان می‌دهد که اکتیویتی والد در BookShelf باید اجازه داشته باشد یک بار از این داده‌ها استفاده کند. این ساز و کار به خوبی کار می‌کند زیرا شما واقعا نیازی به دسترسی به کل پایگاه داده مخاطبین ندارید. شما فقط نیاز به دسترسی به یک مخاطب در آن پایگاه داده دارید.

۱۶

گرفتن عکس با اینتنت

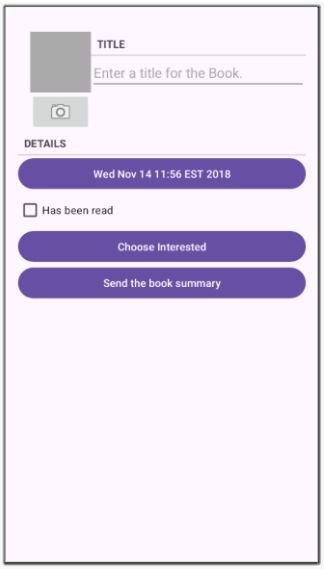
اکنون که چگونگی کار با اینتنت‌های ضمنی را یاد گرفتیم، می‌توان کتاب‌ها را با جزئیات بیشتری مستند کرد. الصاق عکس به کتاب‌ها، جزئیات آن را بیشتر نمایان می‌کند. البته گرفتن عکس چند ابزار جدید را در ترکیب با ابزاری که اخیرا با آن آشنا شده‌ایم یعنی اینتنت ضمنی، شامل می‌شود.

از یک اینتنت ضمنی می‌توان برای راه اندازی برنامه دوربین مورد علاقه کاربر و دریافت یک تصویر جدید از آن استفاده کرد. اما عکسی که دوربین ثبت می‌کند، کجا باید قرار دهیم؟ و چگونه آن را به نمایش بگذاریم؟ در این فصل به هر دوی این سوالات پاسخ خواهیم داد.

مکانی برای عکس شما

اولین قدم این است که مکانی برای عکس خود در صفحه نمایش جزئیات کتاب ایجاد کنیم. به دو View جدید نیاز داریم: یک ImageView برای نمایش عکس و یک دکمه برای گرفتن یک عکس جدید (تصویر ۱۶.۱).

تصویر ۱۶.۱ رابط کاربری جدید



اختصاص یک ردیف کامل به یک تصویر کوچک و یک دکمه، برنامه شما را نامرتب و غیرحرفه‌ای جلوه می‌دهد. باید صفحه را به شکلی معقول مرتب نماییم.

ویوهای جدید را به res/layout/fragment_book.xml اضافه کنید. از سمت چپ شروع کنید و یک ImageView برای عکس و یک ImageButton برای دکمه گرفتن عکس اضافه کنید. در پایان فایل xml شما باید همانند لیست ۱۶.۱ باشد.

لیست ۱۶.۱ افزودن یک تصویر و دکمه دوربین به لیوت (res/layout/fragment_Book.xml)

```
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/fragment_book"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:layout_margin="16dp"
    android:orientation="vertical">
    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginStart="16dp"
        android:layout_marginTop="16dp"
        android:orientation="horizontal">
```

```
        <LinearLayout
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:orientation="vertical">
            <ImageView
                android:id="@+id/book_photo"
                android:layout_width="80dp"
                android:layout_height="80dp"
                android:background=
                "@android:color/darker_gray"
                android:cropToPadding="true"
                android:scaleType="centerInside" />
            <ImageButton
                android:id="@+id/book_camera"
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:src=
                "@android:drawable/ic_menu_camera" />
        </LinearLayout>
        <LinearLayout
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_weight="1"
            android:orientation="vertical">
            <TextView
                style="?android:listSeparatorTextViewStyle"
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:text="@string/book_title_label" />
            <EditText
                android:id="@+id/book_title"
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:hint="@string/book_title_hint" />
        </LinearLayout>
    </LinearLayout>
    <TextView
        style="?android:listSeparatorTextViewStyle"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="@string/book_title_label"/>
    <EditText
        android:id="@+id/book_title"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="@string/book_title_hint"/>
<TextView
        style="?android:listSeparatorTextViewStyle"
```

```
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="@string/book_details_label" />
    <Button
        android:id="@+id/book_date"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        tools:text="Wed Nov 14 11:56 EST 2018" />
    <CheckBox
        android:id="@+id/book_solved"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="@string/book_solved_label" />
    <Button
        android:id="@+id/book_interested"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="@string/book_interested_text" />
    <Button
        android:id="@+id/book_summary"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="@string/book_summary_text" />
</LinearLayout>
```

BookShelf را اجرا و کتابی را برای مشاهده جزئیات لمس کنید. باید رابط کاربری جدید خود را دقیقا مانند شکل ۱۶.۱ ببینید. عالی به نظر می‌رسد.

اکنون برای پاسخ به لمس ImageButton و کنترل محتوای ImageView خود، به متغییرهایی نیاز داریم که به هر یک از آنها ارجاع دارد. در BookFragment طبق معمول از findViewById(Int) را برای جستجو و اتصال ویوها استفاده کنید.

لیست ۱۶.۲ افزودن ویژگی‌ها (BookFragment.kt)

```
class BookFragment : Fragment(){
    …
    private lateinit var chooseInterestedButton: Button
    private lateinit var photoButton: ImageButton
    private lateinit var photoView: ImageView
    …
    override fun onCreateView(
        …
    ): View? {
        …
        chooseInterestedButton = view.findViewById
        (R.id.book_interested) as Button
        photoButton = view.findViewById      (
```

```
            R.id.book_camera) as ImageButton
        photoView = view.findViewById(
            R.id.book_photo) as ImageView
        return view
    }
    …
}
```

فعلا کار با رابط کاربری تمام شده است. (در عرض یک یا دو دقیقه دکمه را متصل خواهیم کرد.)

ذخیره‌سازی فایل

عکس گرفته شده به چیزی بیش از یک مکان روی صفحه نیاز دارد. تصاویر با اندازه واقعی برای قرار گرفتن در پایگاه داده SQLite بسیار بزرگ هستند، چه برسد به قرار دادن آنها در یک اینتنت. آنها به مکانی برای قرار گیری در سیستم فایل دستگاه شما نیاز دارند.

خوشبختانه مکانی برای ذخیره این فایل‌ها داریم: حافظه خصوصی برنامه. به یاد بیاورید که پایگاه داده ما در حافظه خصوصی برنامه ذخیره شده است. با توابعی مانند Context.getFileStreamPath(String) و Context.getFilesDir() می‌توانیم همین کار را با فایل‌های معمولی نیز انجام دهید (یک زیر پوشه مجاور زیرپوشه databases).

اینها توابع اصلی فایل و دایرکتوری در کلاس Context هستند:

- getFilesDir(): File: یک هندل به پوشه فایل‌های خصوصی برنامه برمی‌گرداند.
- openFileInput(name: String): FileInputStream: یک فایل موجود را در دایرکتوری فایل‌ها برای خواندن باز می‌کند.
- openFileOutput(name: String, mode: Int): FileOutputStream: فایلی را در دایرکتوری فایل‌ها برای خروجی باز می‌کند. در صورت نبود فایل آن را ایجاد می‌کند.
- getDir(name: String, mode: Int): File: یک زیرشاخه را در دایرکتوری فایل‌ها دریافت می‌کند (و احتمالا ایجاد می‌کند).
- fileList(…): Array<String>: لیستی از نام فایل‌ها را در دایرکتوری فایل‌های اصلی دریافت می‌کند.
- getCacheDir(): File: یک هندل را به دایرکتوری که می‌توانید به طور خاص برای ذخیره فایل‌های کش استفاده کنید، بر می‌گرداند. باید مراقب باشید که این دایرکتوری را مرتب نگه دارید و تا حد امکان از فضای کمتری استفاده کنید.

یک نکته وجود دارد. از آنجایی که این فایل‌ها خصوصی هستند، فقط برنامه خود شما می‌تواند آنها را بخواند یا بنویسد. تا زمانی که هیچ برنامه دیگری نیازی به دسترسی به آن فایل‌ها نداشته باشد، این توابع کافی هستند.

با این حال اگر برنامه دیگری نیاز به نوشتن در فایل‌های شما داشته باشد این توابع کافی نخواهند بود. این مورد برای BookShelf صدق می‌کند، زیرا برنامه دوربین باید تصویری را که به عنوان یک فایل در برنامه شما می‌گیرد، ذخیره کند.

قبلا و در نسخه‌های قدیمی اندروید می‌توانستیم فایل‌ها را با استفاده از فضای ذخیره‌سازی خارجی در دسترس عموم قرار دهیم. اما به دلایل امنیتی در نسخه‌های اخیر اندروید این کار منسوخ شده است.

اگر نیاز به اشتراک گذاری فایل‌ها با برنامه‌های دیگر داشته باشیم و یا بخواهیم فایل‌هایی از آنها دریافت کنیم باید آن فایل‌ها را ازطریق ContentProvider در معرض دید قرار دهیم. ContentProvider به ما امکان قرار دادن URI (Uniform Resource Identifier)[۲] محتوا در معرض دید سایر برنامه‌ها را می‌دهد. سپس آنها می‌توانند از آن URI برای دانلود محتوا یا عمل نوشتن در آن استفاده نمایند. در هر صورت کنترل در دست ماست و همیشه می‌توانیم در صورت تمایل خواندن یا نوشتن را رد کنیم.

استفاده از FileProvider

هنگامی که نیاز ما دریافت یک فایل از برنامه دیگری است، پیاده‌سازی کل ContentProvider کار اضافه و بیهوده‌ای است. خوشبختانه گوگل کلاس راحتی به نام FileProvider ارائه کرده است که همه چیز به جز پیکربندی را انجام می‌دهد.

اولین قدم اعلام FileProvider به عنوان ContentProvider است که به یک authority متصل شده است. این کار را با افزودن اعلان content provider به مانیفست اندروید خود انجام دهید.

لیست ۱۶.۳ افزودن اعلان FileProvider (manifests/AndroidManifest.xml)

```
<?xml version="1.0" encoding="utf-8"?>
<manifest
xmlns:android="http://schemas.android.com/apk/res/android">
    <application
        …
        <activity
            android:name=".MainActivity"
            android:exported="true"
            android:windowSoftInputMode="adjustResize">
```

[۲] URI یک رشته استاندارد برای شناسایی یک منبع است—مثل عکس، فایل، مخاطب، آهنگ، یا هر داده‌ای که سیستم باید به آن دسترسی پیدا کند.

```
            <intent-filter>
                <action android:name=
                "android.intent.action.MAIN" />
                <category android:name=
                "android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
        <provider
            android:name="androidx.core.content.FileProvider"
            android:authorities=
            "com.codeb.bookshelf.fileprovider"
            android:exported="false"
            android:grantUriPermissions="true">
        </provider>
    </application>
</manifest>
```

Authority مکانی است که فایل‌ها در آن ذخیره می‌شوند. رشته‌ای که برای android:authorities انتخاب می‌کنیم باید در کل سیستم منحصر به فرد باشد. برای اطمینان از این امر، قرارداد این است که رشته authority را به نام پکیج خود اضافه می‌کنیم.

با اتصال FileProvider به authority، به سایر برنامه‌ها هدفی برای درخواست‌هایشان می‌دهیم. با افزودن ویژگی exported="false" اجازه استفاده از ارائه دهنده خود را به کسانی که مجوز ندادیم، ممنوع می‌کنیم. و با افزودن ویژگی grantUriPermissions این امکان را به سایر برنامه‌ها برای نوشتن در URIها به هنگام ارسال در یک اینتنت در این authority می‌دهیم.

پس از اعلام FileProvider، باید به FileProvider خود بگوییم که کدام فایل‌ها را در معرض دید قرار دهد. این قسمت پیکربندی با یک فایل XML اضافی انجام می‌شود. روی پوشه app/res خود در پنجره ابزار پروژه کلیک راست کرده و New → Android resource file را انتخاب کنید. files را برای نام وارد کنید و برای نوع منبع XML را انتخاب کنید. روی OK کلیک کنید تا اندروید استودیو فایل منبع جدید را اضافه و باز کند.

در ویوی کد res/xml/files.xml جدید، جزئیات مربوط به مسیر فایل را اضافه کنید (فهرست ۱۶.۵).

لیست ۱۶.۴ پر کردن توضیحات مسیرها (res/xml/files.xml)

```
<?xml version="1.0" encoding="utf-8"?>
<PreferenceScreen xmlns:android=
    "http://schemas.android.com/apk/res/android">
</PreferenceScreen>
<paths>
    <files-path name="book_photos" path="."/>
</paths>
```

این فایل XML می‌گوید: "مسیر ذخیره‌سازی خصوصی من را به عنوان book_photos مپ کن". ما از نام book_photos استفاده نخواهیم کرد بلکه FileProvider از آن به صورت داخلی استفاده می‌کند.

اکنون با افزودن یک تگ meta-data در خود AndroidManifest.xml به FileProvider خود متصل می‌شویم.

لیست ۱۶.۵ اتصال مسیرها (manifests/AndroidManifest.xml)

```
<provider
    android:name="androidx.core.content.FileProvider"
    android:authorities="com.codeb.bookshelf.fileprovider"
    android:exported="false"
    android:grantUriPermissions="true">
    <meta-data
        android:name="android.support.FILE_PROVIDER_PATHS"
        android:resource="@xml/files"/>
</provider>
```

تعیین مکان تصویر

وقت آن است که به تصاویر خود مکانی برای قرارگیری روی دیسک به صورت محلی بدهیم. ابتدا یک ویژگی را به Book اضافه کنید تا یک نام فایل دریافت کند.

لیست ۱۶.۶ افزودن ویژگی نام فایل (Book.kt)

```
@Entity
data class Book(@PrimaryKey val id: UUID = UUID.randomUUID(),
                var title: String = "",
                var date: Date = Date(),
                var isSolved: Boolean = false,
                var interested: String = ""){
    val photoFileName
        get() = "IMG_$id.jpg"
}
```

photoFileName مسیر پوشه عکس را شامل نمی‌شود. با این حال نام فایل منحصر به فرد خواهد بود، زیرا بر اساس شناسه کتاب است.

در مرحله بعد محل قرار گرفتن عکس‌ها را معین کنید. BookRepository مسئول داده‌های در BookShelf است، بنابراین مالک طبیعی این ایده است. یک تابع getPhotoFile(Book) را به BookRepository اضافه کنید که یک مسیر فایل محلی کامل برای تصویر Book فراهم می‌کند.

لیست ۱۶.۷ پیدا کردن محل فایل عکس (BookRepository.kt)

```
class BookRepository private constructor(context: Context) {
    private val filesDir =
    context.applicationContext.filesDir
    …
    fun getBook(id: UUID): LiveData<Book?> =
    BookDao.getBook(id)
    fun getPhotoFile(book: Book): File =
    File(filesDir, book.photoFileName)
    …
}
```

این کد هیچ فایلی در سیستم فایل ایجاد نمی‌کند. فقط آبجکت‌ها فایل را برمی‌گرداند که به مکان‌های مناسب اشاره می‌کند. بعدا از FileProvider برای اعلام این مسیرها به عنوان URI استفاده خواهیم کرد.

در نهایت یک تابع به BookDetailViewModel اضافه کنید تا اطلاعات فایل در معرض BookFragment قرار گیرد.

لیست ۱۶.۸ افشای یک فایل از طریق BookDetailViewModel (BookDetailViewModel.kt)

```
class BookDetailViewModel() : ViewModel() {
    …
    fun saveBook(Book: Book) {
        bookRepository.updateBook(Book)
    }
    fun getPhotoFile(book: Book): File {
        return bookRepository.getPhotoFile(book)
    }
}
```

استفاده از اینتنت دوربین

قدم بعدی گرفتن عکس واقعی است. این بخش آسان است: ما می‌توانیم دوباره از یک اینتنت ضمنی استفاده کنیم.

با ذخیره کردن مکان فایل عکس شروع کنید.

لیست ۱۶.۹ بدست آوردن محل فایل عکس (BookFragment.kt)

```
class BookFragment : Fragment(){
    private lateinit var book: Book
    private lateinit var photoFile: File
    …
    override fun onViewCreated(view: View,
```

```
savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState)
        bookDetailViewModel.bookLiveData.observe(
            viewLifecycleOwner,
            Observer { book ->
                book?.let {
                    this.book = book
                    photoFile =
                    bookDetailViewModel.getPhotoFile(book)
                    updateUI()
                }
            })
        ...
    }
}
```

در مرحله بعد دکمه دوربین را وصل می‌کنیم. اینتنت دوربین در MediaStore، استاد تمام عیار مربوط به رسانه‌ها تعریف شده است.

با ارسال یک اینتنت با اکشن MediaStore.ACTION_IMAGE_CAPTURE اندروید یک اکتیویتی دوربین را فعال و برای ما عکس می‌گیرد. اما یک لحظه دست نگه دارید.

ارسال اینتنت

اکنون آماده ارسال اینتنت دوربین هستیم. اکشن مورد نظر ACTION_IMAGE_CAPTURE نام دارد و در کلاس MediaStore تعریف شده است. MediaStore رابط‌های عمومی مورد استفاده در اندروید برای تعامل با رسانه‌های رایج همانند تصاویر، فیلم‌ها و موسیقی را تعریف می‌کند. همچنین شامل اینتنت ثبت تصویر است که دوربین را راه اندازی می‌کند.

به طور پیش‌فرض ACTION_IMAGE_CAPTURE دوربین را روشن و عکس می‌گیرد، اما این یک تصویر با وضوح کافی نخواهد بود. بلکه یک تصویر کوچک با وضوح پایین است که به شی Intentی متصل می‌نماید.

برای یک خروجی با وضوح خوب باید تصویر را در سیستم فایل ذخیره کند. این کار را می‌توان با ارسال یک Uri که به جایی که می‌خواهیم فایل را در MediaStore.EXTRA_OUTPUT ذخیره کنید، انجام داد. این Uri به مکانی اشاره می‌کند که توسط FileProvider سرویس دهی می‌شود.

ابتدا یک ویژگی جدید برای URI عکس ایجاد و پس از ارجاع به photoFile آن را مقداردهی اولیه کنید.

لیست ۱۶.۱۰ افزودن ویژگی URI عکس (BookFragment.kt)

```
class BookFragment : Fragment(){
    …
    private lateinit var photoFile: File
    private lateinit var photoUri: Uri
    …
    override fun onViewCreated(view: View
    , savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState)
        bookDetailViewModel.bookLiveData.observe(
            viewLifecycleOwner,
            Observer { book ->
                book?.let {
                    this.book = book
                    photoFile =
                    bookDetailViewModel.getPhotoFile(book)
                    photoUri =
                    FileProvider
                    .getUriFor(
                        requireActivity(),
                        "com.codeb.bookshelf.fileprovider",
                        photoFile)
                    updateUI()
                }
            })
        …
    }
}
```

فراخوانی FileProvider.getUriForFile(…) مسیر فایل محلی شما را به Uri که برنامه دوربین می‌تواند ببیند، ترجمه می‌کند. این تابع اکتیویتی authority ارائه دهنده و فایل عکس شما را برای ایجاد URI که به فایل اشاره می‌کند، می‌گیرد. رشته authority که به FileProvider.getUriForFile(…) ارسال می‌کنید، باید با رشته authority که در مانیفست تعریف کرده‌اید مطابقت داشته باشد.

در مرحله بعد یک اینتنت ضمنی جهت گرفتن عکس جدید در مکان ذخیره شده در photoUri بنویسید (فهرست ۱۶.۱۱). با نوشتن مقداری کد مطمئن شوید که در صورت عدم وجود برنامه دوربین یا نبود مکانی برای ذخیره عکس، دکمه غیرفعال شود.

لیست ۱۶.۱۱ ارسال اینتنت دوربین (BookFragment.kt)

```
class BookFragment : Fragment(){
    ...
    private val bookDetailViewModel: BookDetailViewModel
    by viewModels()
    private val takePhotoLauncher =
           registerForActivityResult
```

```
            (ActivityResultContracts.StartActivityForResult())
{ result ->
            if (result.resultCode == Activity.RESULT_OK) {
            }
            requireActivity().revokeUriPermission(
                photoUri,
                Intent.FLAG_GRANT_WRITE_URI_PERMISSION
            )
        }
    ...
    override fun onStart() {
        ...
        photoButton.apply {
            val pm = requireActivity().packageManager
            val captureImage =
            Intent(MediaStore.ACTION_IMAGE_CAPTURE)
            val resolvedActivity =
            pm.resolveActivity(captureImage,
            PackageManager.MATCH_DEFAULT_ONLY)
            isEnabled = resolvedActivity != null
            setOnClickListener {
            captureImage.putExtra
            (MediaStore.EXTRA_OUTPUT, photoUri)
            val cameraActivities =
            pm.queryIntentActivities(
                    captureImage,
                    PackageManager.MATCH_DEFAULT_ONLY
                )
                for (cameraActivity in cameraActivities) {
                     requireActivity().grantUriPermission
                    (cameraActivity
                    .activityInfo.packageName,
                    photoUri,
                    Intent.FLAG_GRANT_WRITE_URI_PERMISSION
                    )
                }

                takePhotoLauncher.launch(captureImage)
            }
        }
        ...
    }
    ...
}
```

برنامه دوربین برای نوشتن در photoUri به مجوز نیاز دارد. برای انجام این کار فلگ Intent.FLAG_GRANT_WRITE_URI_PERMISSION را اعطا می‌کنیم. افزودن متد android:grantUriPermissions در اعلامیه ارائه دهنده، برای استفاده از این قسمت تابع ضروری است.

BookShelf را اجرا و دکمه دوربین را برای راه اندازی برنامه دوربین لمس کنید. دقت کنید به یکی از دلایل زیر ممکن است برنامه ما روی دستگاه مجازی کار نکند:

۱. امولاتور دوربین واقعی ندارد به همین دلیل ممکن است resolveActivity = null باشد.

۲. امولاتور اجازه نوشتن روی URI را قبول نمی‌کند.

پس بهتر است برای تست از دستگاه فیزیکی استفاده کنید.

مقیاس و نمایش بیت مپ‌ها

با انجام کارهای فوق موفق به ثبت عکس شدیم. تصویر ما در فایلی در سیستم فایل ذخیره می‌شود تا بتوانیم از آن استفاده کنیم.

قدم بعدی دستیابی به فایل و بارگذاری و نمایش آن به کاربر است. برای انجام این کار باید تصویر را در یک شی Bitmap با اندازه معقول بارگذاری کنیم. جهت دریافت بیت مپ از یک فایل، تنها کاری که باید انجام دهیم استفاده از کلاس BitmapFactory به شرح زیر است:

```
val bitmap = BitmapFactory.decodeFile(photoFile.getPath())
```

نکته‌ای قابل توجه در این رابطه وجود دارد: "اندازه معقول". Bitmap یک شی ساده است که داده‌های پیکسلی را ذخیره می‌کند. این بدان معناست که حتی اگر فایل اصلی فشرده شود، هیچ فشرده سازی در خود بیت مپ اعمال نخواهد شد. بنابراین یک تصویر دوربین ۱۶ مگاپیکسلیِ ۲۴ بیتی که ممکن است فقط یک JPG ۵ مگابایتی باشد، تا ۴۸ مگابایت هنگام بارگذاری در یک شی Bitmap فضا می‌گیرد.

برای حل این مورد باید بیت مپ را کوچک کنیم. ابتدا فایل را برای بررسی بزرگی اسکن می‌کنیم تا میزان مقیاس مشخص شود و در نهایت فایل را برای ایجاد یک شی Bitmap کوچک دوباره می‌خوانیم.

فایل جدید به نام PictureUtils.kt ایجاد و یک تابع سطح فایل به نام getScaledBitmap(String, Int, Int) به آن اضافه کنید (فهرست ۱۶.۱۲).

لیست ۱۶.۱۲ ایجاد getScaledBitmap(...) (PictureUtils.kt)

```
package com.codeb.bookshelf
import android.graphics.Bitmap
```

```
import android.graphics.BitmapFactory
class PictureUtils {
    fun getScaledBitmap(path: String, destWidth: Int
    , destHeight: Int): Bitmap {
        // Read in the dimensions of the image on disk
        var options = BitmapFactory.Options()
        options.inJustDecodeBounds = true
        BitmapFactory.decodeFile(path, options)
        val srcWidth = options.outWidth.toFloat()
        val srcHeight = options.outHeight.toFloat()
        // Figure out how much to scale down by
        var inSampleSize = 1
        if (srcHeight > destHeight || srcWidth > destWidth) {
            val heightScale = srcHeight / destHeight
            val widthScale = srcWidth / destWidth
            val sampleScale = if (heightScale > widthScale) {
                heightScale
            } else {
                widthScale
            }
            inSampleSize = Math.round(sampleScale)
        }
        options = BitmapFactory.Options()
        options.inSampleSize = inSampleSize
        // Read in and create final bitmap
        return BitmapFactory.decodeFile(path, options)
    }
}
```

پارامتر کلیدی بالا inSampleSize است. این گزینه تعیین می‌کند که هر "sample" برای هر پیکسل چقدر باید بزرگ باشد – اندازه نمونه ۱ دارای یک پیکسل افقی نهایی برای هر پیکسل افقی در فایل اصلی است و اندازه نمونه ۲ دارای یک پیکسل افقی برای هر دو پیکسل افقی در فایل اصلی است. بنابراین وقتی inSampleSize، ۲ باشد، تعداد پیکسل‌ها در تصویر یک چهارم تعداد پیکسل‌ها در نسخه اصلی است.

یک خبر بد دیگر: هنگامی که فرگمنت شما در ابتدا راه اندازی می‌شود، نمی‌دانید که PhotoView چقدر بزرگ است. تا زمانی که بارگذاری لیوت اتفاق نیفتد، ویوها ابعادی روی صفحه ندارند. اولین بارگذاری لیوت بعد از onCreate(…) اتفاق می‌افتد، onStart() و onResume() در ابتدا اجرا می‌شوند، به همین دلیل PhotoView نمی‌داند چقدر بزرگ است.

دو راه حل برای این مشکل وجود دارد: یا منتظر می‌مانیم بارگذاری اتفاق بیافتد یا از یک تخمین محافظه کارانه استفاده می‌کنیم. رویکرد برآورد محافظه کارانه کارآمدتر و ساده تر است. تابع سطح فایل دیگری به

نام getScaledBitmap(String, Activity) بنویسید تا یک بیت مپ را برای اندازه یک اکتیویتی خاص مقیاس کنید. بدین ترتیب فایل PictureUtils.kt باید دوتابع getScaledBitmap داشته باشد.

فهرست ۱۶.۱۳ نوشتن یک تابع مقیاس محافظه کارانه (PictureUtils.kt)

```
fun getScaledBitmap(path: String, activity: Activity): Bitmap
{
    val size = Point()
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.R) {
        val windowMetrics = activity.windowManager.curren-
tWindowMetrics
        val bounds = windowMetrics.bounds
        size.x = bounds.width()
        size.y = bounds.height()
    } else {
        @Suppress("DEPRECATION")
        activity.windowManager.defaultDisplay.getSize(size)
    }
    return getScaledBitmap(path, size.x, size.y)
}
```

این تابع بررسی می‌کند تا ببیند صفحه نمایش چقدر بزرگ است و سپس تصویر را به آن اندازه کاهش می‌دهد. ImageViewای که در آن بارگذاری می‌کنیم همیشه کوچکتر از این اندازه خواهد بود، بنابراین این یک تخمین بسیار محافظه کارانه است.

در مرحله بعد برای بارگذاری این بیت مپ در ImageView یک تابع به BookFragment اضافه کنید تا photoView را به روز کنید.

لیست ۱۶.۱۴ به روز رسانی photoView (BookFragment.kt)

```
class BookFragment : Fragment(){
    ...
    private val bookDetailViewModel: BookDetailViewModel
    by viewModels()
    private fun updatePhotoView() {
        if (photoFile.exists()) {
            val bitmap = PictureUtils().getScaledBitmap
           (photoFile.path, requireActivity())
            photoView.setImageBitmap(bitmap)
        } else {
            photoView.setImageDrawable(null)
        }
    }
    ...
}
```

سپس آن تابع را از داخل updateUI() و ActivityResult(…) فراخوانی کنید.

لیست ۱۶.۱۵ فراخوانی updatePhotoView() (BookFragment.kt)

```
class BookFragment : Fragment(){
    …
    private val bookDetailViewModel: BookDetailViewModel
        by viewModels()

    private fun updatePhotoView() {
        if (photoFile.exists()) {
            val bitmap = PictureUtils().getScaledBitmap
            (photoFile.path, requireActivity())
            photoView.setImageBitmap(bitmap)
        } else {
            photoView.setImageDrawable(null)
        }
    }
    private val takePhotoLauncher =
         registerForActivityResult
         (ActivityResultContracts.StartActivityForResult())
         { result ->
            if (result.resultCode == Activity.RESULT_OK) {
                updatePhotoView()
            }
            requireActivity().revokeUriPermission(
                photoUri,
                Intent.FLAG_GRANT_WRITE_URI_PERMISSION
            )
        }
    …
    private fun updateUI() {
        titleField.setText(book.title)
        dateButton.text = book.date.toString()
        solvedCheckBox.apply {
            isChecked = book.isSolved
            jumpDrawablesToCurrentState()
        }
        if(book.interested.isNotEmpty())
            chooseInterestedButton.text=book.interested
        updatePhotoView()
    }
    …
    }
}
```

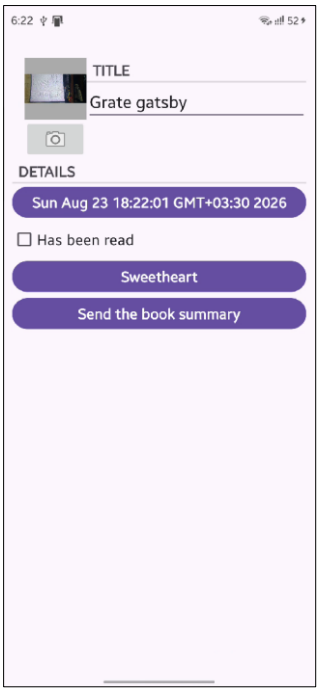
اکنون که نوشتن دوربین در فایل ما تمام شده است، می‌توانیم مجوز را لغو و دسترسی به فایل خود را دوباره ببندیم. این کار در ()ActivityResult در صورت دریافت یک نتیجه معتبر و همچنین در ()onDetach برای پوشش احتمال پاسخ نامعتبر، انجام می‌دهیم.

فهرست ۱۶.۱۶ لغو مجوزهای URI (BookFragment.kt)

```
class BookFragment : Fragment(){
    …
    private val bookDetailViewModel: BookDetailViewModel
    by viewModels()
    override fun onDetach() {
        super.onDetach()
        requireActivity().revokeUriPermission(photoUri,
            Intent.FLAG_GRANT_WRITE_URI_PERMISSION)
    }
    …
}
```

BookShelf را دوباره اجرا کنید. صفحه جزئیات کتاب را باز و از دکمه دوربین برای گرفتن عکس استفاده کنید. باید تصویر خود را در ویوی تصویر کوچک مشاهده نمایید (شکل ۱۶.۲).

شکل ۱۶.۲ تصویر کوچک با افتخار در صفحه جزئیات کتاب ظاهر می‌شود



تعریف ویژگی‌ها

پیاده‌سازی دوربین اکنون بسیار عالیست. تنها یک کار دیگر باقی می‌ماند: به کاربران بالقوه اطلاع رسانی کنید. هنگامی که برنامه شما از قابلیتی مانند دوربین یا هر ویژگی دیگری که ممکن است از دستگاهی به دستگاه دیگر متفاوت باشد استفاده می‌کند، اکیدا توصیه می‌شود که در مورد آن به اندروید اطلاع دهید. این کار به سایر برنامه‌ها (مانند play store) اجازه می‌دهد تا در صورت استفاده از ویژگی‌ای که دستگاه پشتیبانی نمی‌کند، از نصب برنامه خودداری کند.

برای اعلام استفاده از دوربین، یک برچسب <uses-feature> به AndroidManifest.xml خود اضافه کنید (فهرست ۱۶.۱۷).

لیست ۱۶.۱۷ افزودن برچسب <uses-feature> (manifests/AndroidManifest.xml)

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android=
"http://schemas.android.com/apk/res/android">
    <uses-feature android:name="android.hardware.camera"
        android:required="false"/>
    <application
        …
    </application>
</manifest>
```

ما ویژگی اختیاری android:required را در اینجا قرار می‌دهیم. چرا؟ به طور پیش‌فرض، اعلام اینکه از یک ویژگی استفاده می‌کنید به این معنی است که برنامه شما بدون آن ویژگی به درستی کار نمی‌کند. این در مورد BookShelf صدق نمی‌کند. زیرا ما resolveActivity(…) را برای بررسی یک برنامه دوربین فراخوانی و اگر آن را پیدا نکردیم به آرامی‌دکمه دوربین را غیرفعال می‌کنیم.

android:required=”false” این وضعیت را به درستی مدیریت می‌کند. ما به اندروید می‌گوییم که برنامه ما می‌تواند بدون دوربین به خوبی کار کند، اما برخی از قسمت‌ها غیرفعال می‌شوند.

چالش: نمایش جزئیات

در حالی که تصویری را که به نمایش گذاشتیم، قابل مشاهده است اما نمی‌توان به علت ابعاد آن، آن را به خوبی دید. برای اولین چالش یک DialogFragment جدید ایجاد کنید که یک نسخه بزرگنمایی شده از عکس صحنه کتاب شما را نمایش می‌دهد. وقتی تصویر کوچک را فشار می‌دهید باید DialogFragment تصویر بزرگنمایی شده را بالا بکشد.

۱۷

بومی‌سازی

با فرض اینکه BookShelf یک برنامه بسیار محبوب خواهد بود، تصمیم گرفتیم آن را برای مخاطبان بیشتری در دسترس قرار دهیم. اولین قدم این است که تمام متن‌های اکتیویتی را بومی‌سازی کنیم تا برنامه ما به زبان فارسی یا انگلیسی خوانده شود.

بومی‌سازی فرآیند ارائه منابع مناسب برای برنامه ما بر اساس تنظیمات زبان کاربر است. در این فصل ما یک نسخه فارسی res/values/strings.xml ارائه خواهیم کرد. هنگامی که زبان یک دستگاه روی فارسی تنظیم می‌شود، اندروید به طور خودکار رشته‌های فارسی را در زمان اجرا پیدا و استفاده می‌کند (شکل ۱۷.۱).

شکل ۱۷.۱ قفسه کتاب



بومی‌سازی منابع

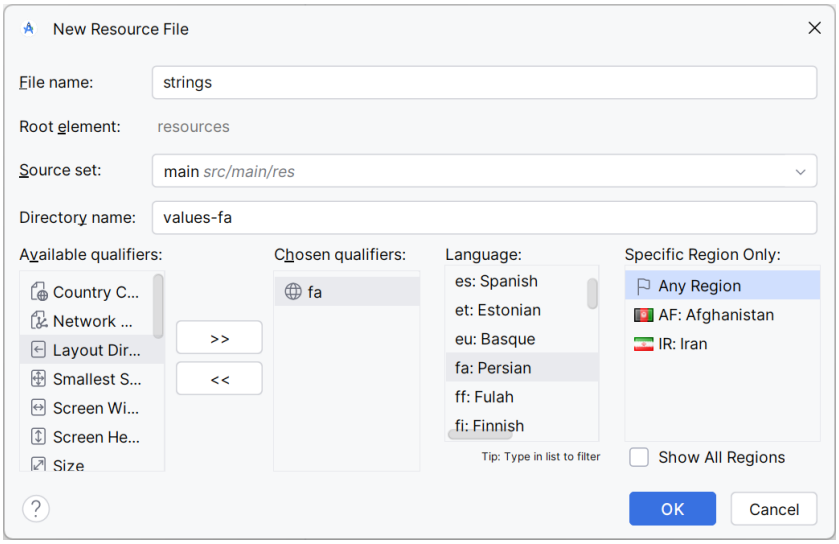
تنظیمات زبان بخشی از پیکربندی دستگاه است. اندروید برای زبان‌های مختلف توصیفگر به کار می‌برد، همانطور که برای جهت صفحه، اندازه صفحه نمایش و سایر عوامل پیکربندی به کار می‌برد. این کار بومی‌سازی را ساده می‌کند: ما زیرشاخه‌های منابع را با توصیفگرهای بسته به زبان ایجاد و منابع جایگزین را در آنها قرار می‌دهیم. سپس سیستم منابع اندروید بقیه کارها را انجام می‌دهد.

در پروژه BookShelf و در values یک فایل جدید ایجاد کنید: در پنجره ابزار پروژه روی res/values/ کلیک راست کرده و New → Values resource file را انتخاب کنید. strings را برای نام فایل وارد کنید. گزینه Source set را روی main تنظیم و مطمئن شوید که Directory name روی values تنظیم شده است.

Locale را در لیست Available qualifiers انتخاب و روی دکمه >> کلیک کنید تا Locale به بخش Chosen qualifires منتقل شود. Fa:Persian را در لیست زبان انتخاب کنید. Any Region به طور خودکار در لیست Specific Region Only مرتب می‌شود – که همان دلخواه ماست.

پنجره New Resource File باید شبیه شکل ۱۷.۲ باشد.

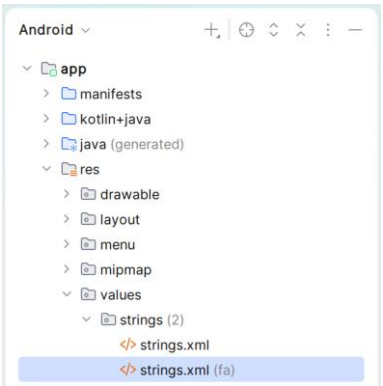
تصویر ۱۷.۲ افزودن یک فایل توصیفگر منبع رشته‌ای



توجه داشته باشید که اندروید استودیو به طور خودکار فیلد نام دایرکتوری را به values-fa تغییر می‌دهد. توصیف گر پیکربندی زبان از کدهای ISO 639-1 گرفته شده و هر کدام از دو کاراکتر تشکیل شده است. برای فارسی، توصیفگر fa- است.

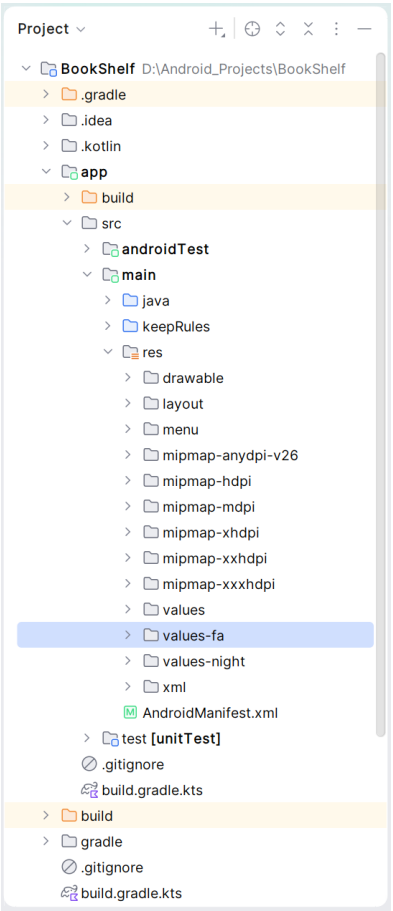
روی OK کلیک کنید. فایل strings.xml جدید در زیر res/values، با (fa) بعد از نام آن فهرست می‌شود. فایل‌های رشته‌ای در ویوی اندروید پنجره ابزار پروژه با هم گروه بندی می‌شوند (شکل ۱۷.۳).

تصویر ۱۷.۳ مشاهده strings.xml جدید در ویوی Android



با این حال اگر ساختار دایرکتوری را بررسی کنید، خواهید دید که پروژه شما اکنون شامل یک دایرکتوری values اضافی است: res/values-fa. strings.xml تازه تولید شده در داخل این دایرکتوری جدید قرار دارد (شکل ۱۷.۴).

تصویر ۱۷.۴ مشاهده strings.xml جدید در ویوی پروژه



اکنون زمان آن است که جادو اتفاق بیفتد. نسخه‌های فارسی تمام رشته‌های خود را به res/values-fa/ strings.xml اضافه کنید.

لیست ۱۷.۱ افزودن جایگزین‌های فارسی برای منابع رشته‌ای (res/values-es/strings.xml)

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <string name="app_name">کتاب قفسه</string>
    <string name="book_title_hint">
    کنید وارد خود کتاب برای عنوان یک</string>
    <string name="book_title_label">عنوان</string>
    <string name="book_details_label">جزئیات</string>
    <string name="book_solved_label">شده خوانده</string>
    <string name="book_list_title">ها کتاب لیست</string>
```

```
    <string name="new_book">جدید کتاب</string>
    <string name="book_interested_text">
    مند علاقه انتخاب</string>
    <string name="book_summary_text">کتاب خلاصه ارسال</string>
    <string name="book_summary">
    %1$s! این کتاب شروع شده در  %2$s. %3$s, and %4$s</string>
    <string name="book_summary_solved">شده خوانده</string>
    <string name="book_summary_unsolved">نشده خوانده</string>
    <string name="book_summary_no_interested">
    ندارد متدی علاقه</string>
    <string name="book_summary_interested">
    متد علاقه: %s.</string>
    <string name="book_summary_subject">
    کتاب قفسه از کتاب خلاصه</string>
    <string name="send_summary">با خلاصه ارسال</string>
</resources>
```

این تنها کاری است که برای ارائه منابع رشته‌ای محلی برای برنامه خود باید انجام دهیم. برای تأیید، با باز کردن تنظیمات و پیدا کردن تنظیمات زبان، تنظیمات دستگاه خود را به فارسی تغییر دهید.

وقتی به لیستی از گزینه‌های زبان رسیدید، گزینه فارسی را انتخاب کنید. منطقه اهمیتی نخواهد داشت زیرا توصیفگر fa- با هر دو مطابقت دارد. (توجه داشته باشید که در نسخه‌های جدیدتر اندروید، کاربران می‌توانند چندین زبان را انتخاب کرده و ترتیب اولویت را تنظیم کنند. اگر از دستگاه جدیدتری استفاده می‌کنید، مطمئن شوید که فارسی ابتدا در لیست تنظیمات زبان شما ظاهر می‌شود.)

اکنون BookShelf را اجرا کنید و از شکوه برنامه جدید بومی‌سازی شده خود لذت ببرید. وقتی به خوبی استراحت کردید، تنظیمات زبان دستگاه خود را به انگلیسی برگردانید.

منابع پیش‌فرض

توصیفگر زبان انگلیسی en- است. در بومی‌سازی ممکن است به این فکر کنید که نام دایرکتوری values موجود خود را به values-en تغییر دهید. این‌ایده خوبی نیست، اما برای لحظه‌ای وانمود کنید که این کار را انجام داده‌ایم: به روز رسانی فرضی شما به این معنی است که برنامه شما اکنون دارای یک strings.xml انگلیسی در value-en و یک strings.xml فارسی در value-fa است.

برنامه تازه به روز شده ما به خوبی ساخته می‌شود. همچنین روی دستگاه‌هایی که زبان آنها روی فارسی یا انگلیسی تنظیم شده است به خوبی اجرا می‌شود. اما اگر زبان دستگاه کاربر روی ایتالیایی تنظیم شود چه اتفاقی می‌افتد؟ اتفاق‌های بد. اتفاق‌های بسیار بد.

اگر برنامه مجاز به اجرا باشد، Android منابع رشته‌ای را که با پیکربندی فعلی مطابقت دارند پیدا نمی‌کند. برنامه ما با Resources.NotFoundException از کار بیفتد.

اندروید استودیو اقداماتی را برای نجات شما از این سرنوشت انجام می‌دهد. Android Asset Packaging Tool (AAPT) بررسی‌های زیادی را در حین پکیج بندی منابع انجام می‌دهد. اگر AAPT متوجه شود که ما از منابعی استفاده می‌کنیم که در فایل‌های منبع پیش‌فرض گنجانده نشده‌اند در زمان کامپایل خطایی ایجاد می‌کند:

```
Android resource linking failed
warn: removing resource
com.codeb.bookshelf:string/book_title_label
without required default value.
AAPT: error: resource string/book_title_label
(aka com.codeb.bookshelf:string/book_title_label)
not found.
error: failed linking file resources.
```

نتیجه اخلاقی این داستان: یک منبع پیش‌فرض برای هر یک از منابع خود ارائه می دهیم. منابع موجود در دایرکتوری‌های فاقد توصیفگر منابع پیش‌فرض ما هستند. در صورت عدم تطابق با پیکربندی فعلی دستگاه، از منابع پیش‌فرض استفاده می‌شود. اگر اندروید به دنبال منبعی باشد و نتواند منبعی را پیدا کند که با پیکربندی دستگاه یا پیش‌فرض مطابقت داشته باشد، برنامه شما رفتار نامناسبی خواهد داشت.

تراکم صفحه نمایش متفاوت عمل می‌کند

استثنا در ارائه منابع پیش‌فرض برای تراکم صفحه است. همانطور که مشاهده کردید، دایرکتوری‌های drawable یک پروژه معمولا برای تراکم صفحه نمایش با mdpi-، xxhdpi- و غیره توصیف می‌شوند.

با این حال تصمیم اندروید در مورد اینکه از کدام منبع استفاده کند، موضوع ساده‌ای برای تطبیق تراکم صفحه نمایش دستگاه یا پیش‌فرض در یک دایرکتوری فاقد توصیفگر در صورت عدم تطابق نیست.

انتخاب بر اساس ترکیبی از اندازه و تراکم صفحه نمایش است و اندروید ممکن است از دایرکتوری که با چگالی کمتر یا بالاتر از دستگاه واجد شرایط است یک drawable را انتخاب کند، اما نکته مهم این است که قرار دادن منابع قابل ترسیم پیش‌فرض در /res/drawable ضروری نیست.

بررسی پوشش رشته‌ها با استفاده از Translations Editor

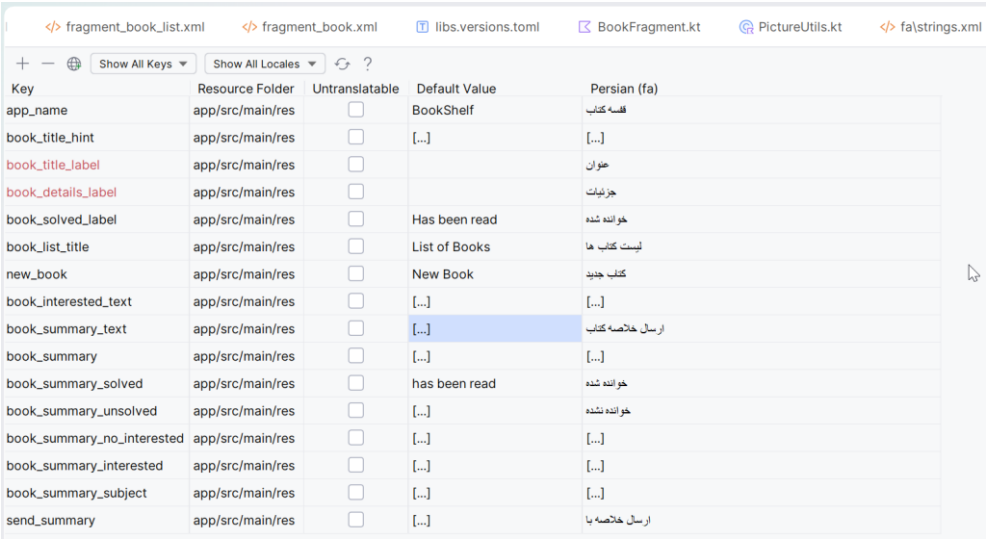
با افزایش تعداد زبان‌هایی که پشتیبانی می‌کنیم، اطمینان از ارائه نسخه‌ای از هر رشته برای هر زبان کار دشواری به نظر می‌رسد. خوشبختانه اندروید استودیو یک ویرایشگر ترجمه مفید برای دیدن همه ترجمه‌های شما در یک مکان ارائه می‌دهد. قبل از شروع strings.xml پیش‌فرض خود را باز و book_title_label و book_details_label کامنت کنید (لیست ۱۷.۲).

لیست ۱۷.۲ کامنت کردن رشته‌ها (res/values/strings.xml)

```
<resources>
    <string name="app_name">BookShelf</string>
    <string name="book_title_hint">
    Enter a title for the Book.</string>
    <!--<string name="book_title_label">Title</string>
    <string name="book_details_label">Details</string>-->
    ...
</resources>
```

برای راه اندازی Translations Editor روی یکی از strings.xml فایل‌های موجود در پنجره ابزار پروژه کلیک راست کرده و Open Translations Editor را انتخاب کنید. ویرایشگر تمام رشته‌های برنامه و وضعیت ترجمه را برای هر زبانی که برنامه شما برای آن توصیفگری را ارائه داده، نمایش می‌دهد. از آنجایی که book_title_label و book_details_label کامنت شده‌اند، نام آن فیلدها را با رنگ قرمز خواهید دید (شکل ۱۷.۵).

تصویر ۱۷.۵ استفاده از Translations Editor برای بررسی پوشش رشته



Key	Resource Folder	Untranslatable	Default Value	Persian (fa)
app_name	app/src/main/res	☐	BookShelf	قفسه کتاب
book_title_hint	app/src/main/res	☐	[...]	[...]
book_title_label	app/src/main/res	☐		عنوان
book_details_label	app/src/main/res	☐		جزئیات
book_solved_label	app/src/main/res	☐	Has been read	خوانده شده
book_list_title	app/src/main/res	☐	List of Books	لیست کتاب ها
new_book	app/src/main/res	☐	New Book	کتاب جدید
book_interested_text	app/src/main/res	☐	[...]	[...]
book_summary_text	app/src/main/res	☐	[...]	ارسال خلاصه کتاب
book_summary	app/src/main/res	☐	[...]	[...]
book_summary_solved	app/src/main/res	☐	has been read	خوانده شده
book_summary_unsolved	app/src/main/res	☐	[...]	خوانده نشده
book_summary_no_interested	app/src/main/res	☐	[...]	[...]
book_summary_interested	app/src/main/res	☐	[...]	[...]
book_summary_subject	app/src/main/res	☐	[...]	[...]
send_summary	app/src/main/res	☐	[...]	ارسال خلاصه با

لیستی از منابع برای افزودن به پروژه شما ارائه می‌شود. منابع جا افتاده را در هر پیکربندی محلی پیدا کنید و آنها را به فایل رشته‌های مرتبط اضافه کنید.

اگرچه می‌توانیم رشته‌ها را مستقیما در Translations Editor اضافه کنیم، اما در مورد شما فقط باید book_title_label و book_details_label را به حالت اولیه برگردانید. قبل از حرکت این کار را انجام دهید.

هدف قرار دادن یک منطقه

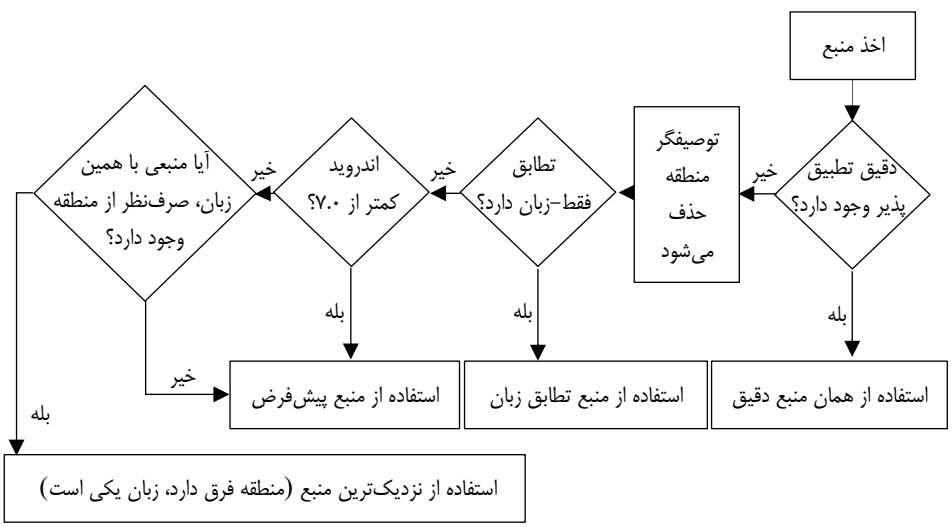
ما می‌توانیم یک دایرکتوری منبع را با یک توصیفگر زبان به علاوه منطقه توصیف کنیم تا منابع را حتی به طور خاص‌تر هدف قرار دهیم. به عنوان مثال توصیفگر فارسی صحبت شده در ایران fa-rIR- است، که در آن r نشان دهنده یک توصیفگر منطقه و IR کد برای ایران است. توصیفگرهای پیکربندی به حروف کوچک و بزرگ حساس نیستند، اما خوب است که از کنوانسیون اندروید پیروی کنیم: از یک کد زبان با حروف کوچک و یک کد منطقه با حروف بزرگ با پیشوند r استفاده کنید.

توجه داشته باشید که ممکن است یک توصیفگر زبان – منطقه مانند fa-rIR- دو توصیفگر پیکربندی مجزا به نظر برسند که با هم ترکیب شده‌اند. اما اینطور نیست و این توصیفگر یه توصیفگر واحد می‌باشد. توصیفگر منطقه به خودی خود یک توصیفگر معتبر نیست.

یک توصیفگر منبع زبان – منطقه در دو حالت فرصت تطبیق با منطقه کاربر را دارد. تطابق دقیق زمانی اتفاق می‌افتد که هر دو توصیفگر زبان و منطقه با زبان کاربر مطابقت داشته باشند. اگر تطابق دقیقی یافت نشود سیستم توصیفگر منطقه را حذف و فقط برای زبان به دنبال تطابق می‌گردد.

انتخاب منابع زمانی که تطابق دقیقی وجود ندارد بسته به نسخه اندروید دستگاه می‌تواند متفاوت باشد. شکل ۱۷.۶ استراتژی حل منابع محلی قبل و بعد از Android Nougat را نشان می‌دهد.

شکل ۱۷.۶ وضوح محلی (قبل و بعد از نوقا)



در دستگاههایی که نسخه‌های پیش از نوقا اندروید را اجرا می‌کنند، اگر زبانی تطابق‌پذیر پیدا نشود از پیش‌فرض (منبع فاقد توصیفگر) استفاده می‌شود.

نوقا و نسخه‌های جدیدتر پشتیبانی از زبان را با زبان‌های بیشتر و توانایی انتخاب بیش از یک زبان در تنظیمات دستگاه افزایش داده‌اند. این سیستم همچنین از یک استراتژی هوشمندتر برای منطقه با هدف نشان دادن زبان صحیح تا حد امکان استفاده می‌کند، حتی اگر برنامه تطابق دقیق منطقه یا زبان را ارائه ندهد. اگر تطابق دقیقی در دستگاهی که نوقا را اجرا می‌کند یافت نشود و هیچ تطابق زبانی یافت نشود، سیستم برای منبعی که توصیفگر زبان یکسان اما منطقه‌ای متفاوت است و از بهترین منبعی که این معیارها را برآورده می‌کند، استفاده می‌کند.

مثالی را در نظر بگیرید. فرض کنید زبان دستگاه خود را روی اسپانیایی و منطقه خود را روی شیلی تنظیم کرده‌اید. برنامه‌ای در دستگاهتان حاوی فایل‌های strings.xml اسپانیایی است که برای اسپانیا و مکزیک طراحی شده است (در value-es-rES و values-es-rMX). دایرکتوری values پیش‌فرض حاوی یک strings.xml انگلیسی است.

اگر دستگاه شما از اندروید پیش نوقا استفاده می‌کند، محتوای انگلیسی دایرکتوری مقادیر پیش‌فرض را مشاهده خواهید کرد. اما اگر دستگاه شما نوقا یا جدیدتر را اجرا می‌کند، تجربه بهتری خواهید داشت: محتویات values-es-rMX/strings.xml را خواهید دید – به این معنی که اسپانیایی را خواهید دید، اگرچه متناسب با شیلی نیست.

این یک مثال ساختگی است. اما یک نکته مهم را بیان می‌کند: رشته‌ها را تا حد امکان کلی ارائه دهید، تا حد امکان از دایرکتوری‌های توصیفگر زبان استفاده کنید و فقط در صورت لزوم از دایرکتوری‌های توصیفگر منطقه استفاده نمایید. به جای حفظ تمام رشته‌های فارسی در دایرکتوری‌های توصیفگر منطقه، این برنامه نمونه بهتر است بیشتر رشته‌های فارسی را در یک دایرکتوری value-fa توصیفگر زبان ذخیره کند و رشته‌های توصیفگر منطقه را فقط برای کلمات و عباراتی که در گویش‌های مختلف منطقه‌ای متفاوت هستند، ارائه دهد.

این کار نه تنها نگهداری فایل‌های رشته‌ای را برای برنامه‌نویس آسان‌تر می‌کند، بلکه به سیستم کمک می‌کند تا منابع موجود در دستگاه‌های قبل از نوقا و نوقا یا دستگاه‌های بعدی را مدیریت کند (با ارائه یک تطابق فقط زبان). توجه داشته باشید که این توصیه برای همه انواع منابع جایگزین در دایرکتوری‌های values صدق می‌کند: منابع مشترک را در دایرکتوری‌های عمومی‌تر ارائه دهید و فقط منابعی را که باید در دایرکتوری‌های توصیفگر تنظیم شوند، بگنجانید.

توصیفگرهای پیکربندی

شما تاکنون چندین توصیفگر پیکربندی را برای ارائه منابع جایگزین مشاهده و استفاده کرده‌اید: زبان (مانند values-fa)، جهت صفحه (layout-land) و تراکم صفحه نمایش (drawable-mdpi).

اندروید توصیفگرهای پیکربندی زیر را برای هدف قرار دادن منابع ارائه می‌دهد:

۱. کد کشور تلفن همراه (MCC) و کد شبکه تلفن همراه (MNC)
۲. کد زبان و به دنبال آن کد منطقه
۳. جهت چیدمان
۴. کوچکترین عرض
۵. عرض موجود
۶. ارتفاع موجود
۷. اندازه صفحه نمایش
۸. aspect صفحه نمایش
۹. صفحه گرد (سطح API 23 و بالاتر)
۱۰. طیف رنگی گسترده‌ای
۱۱. محدوده دینامیکی بالا
۱۲. جهت صفحه نمایش
۱۳. حالت رابط کاربری
۱۴. حالت شب
۱۵. تراکم صفحه نمایش (DPI)
۱۶. نوع صفحه لمسی
۱۷. در دسترس بودن صفحه کلید
۱۸. روش ورودی متن اولیه
۱۹. در دسترس بودن کلید پیمایش

۲۰. روش اصلی پیمایش غیر لمسی

۲۱. سطح API

توضیحات این ویژگی‌ها و نمونه‌هایی از توصیفگرهای پیکربندی خاص را در لینک زیر می‌توان یافت:

developer.android.com/guide/topics/resources/providing-resources.html#AlternativeResources

با توجه به انواع مختلفی از توصیفگرهای پیکربندی برای هدف قرار دادن منابع، ممکن است مواقعی وجود داشته باشد که پیکربندی دستگاه با بیش از یک منبع جایگزین مطابقت داشته باشد. هنگامی که این اتفاق می‌افتد، به ترتیب نشان داده شده در لیست بالا به توصیفگرها اولویت داده می‌شود.

کادر New Resource File دارای توصیفگرهای زیادی است. ما می‌توانیم بیش از یک توصیفگر را در یک دایرکتوری منابع قرار دهیم. هنگام استفاده از چندین توصیفگر در دایرکتوری، باید آنها را به ترتیب اولویت ردیف کنیم. بنابراین values-fa-w600dp یک نام دایرکتوری معتبر و values-w600dp-fa معتبر نیست. (استفاده از New Resource File نام دایرکتوری را به درستی برای ما پیکربندی می‌کند.)

چالش: بومی‌سازی تاریخ‌ها

ممکن است متوجه شده باشید که صرف نظر از زبان دستگاه، تاریخ‌های نمایش داده شده در BookShelf همیشه به استایل پیش‌فرض‌ایالات متحده، با ماه قبل از روز قالب بندی می‌شوند. با قالب بندی تاریخ‌ها بر اساس پیکربندی محل، محلی سازی خود را یک قدم جلوتر ببرید. ساده تر از آن چیزی است که فکر می‌کنید.

مستندات توسعه دهنده را در کلاس DateFormat که به عنوان بخشی از فریمورک اندروید ارائه شده است، بررسی کنید. DateFormat یک قالب بندی تاریخ – زمان را ارائه می‌دهد که زبان فعلی را در نظر می‌گیرد. می‌توانید با استفاده از ثابت‌های پیکربندی تعبیه شده در DateFormat، خروجی را بیشتر کنترل کنید.

۱۸

اتصال داده و MVVM

این فصل پروژه جدیدی به نام BeatBox را آغاز خواهیم کرد (شکل ۱۸.۱). BeatBox جعبه موسیقی نیست. بلکه تمرینی برای تکنیک‌های مهم برنامه‌نویسی اندروید است.

شکل ۱۸.۱ BeatBox در پایان این فصل



در این پروژه یاد خواهید گرفت که چگونه از یک کتابخانه کامپوننت معماری Jetpack به نام data binding (اتصال داده) استفاده کنید. ما از اتصال داده برای پیاده‌سازی معماری Model-View-View Model یا MVVM استفاده خواهیم کرد. علاوه بر این، نحوه استفاده از سیستم دارایی‌ها (assets system) برای ذخیره یک فایل صوتی را خواهید دید.

معماری‌های مختلف: چرا زحمت بکشیم؟

هر برنامه‌ای که تاکنون نوشته‌اید از نسخه ساده MVC استفاده کرده است. و تا کنون – اگر کار خود را به خوبی انجام داده باشیم – هر برنامه‌ای برای شما منطقی بوده است. پس چرا تغییر کنیم؟ مشکل چیست؟

معماری MVC که به شما نشان داده‌ایم برای برنامه‌های کوچک و ساده به خوبی کار می‌کند. افزودن ویژگی‌های جدید و فکر کردن در مورد فرگمنت‌ها، برنامه‌نویسی را آسان می‌کند. این روش پایه محکمی برای توسعه ایجاد می‌کند. شما به سرعت برنامه تان را راه اندازی و اجرا می‌کنید و در مراحل اولیه یک پروژه به خوبی به کار می‌رود.

مشکل زمانی بوجود می‌آید که پروژه شما بزرگتر از آنچه در این کتاب نشان می‌دهیم، باشد – همانطور که پروژه‌های دنیای واقعی اینطور هستند. ساخت فرگمنت‌ها و اکتیویتی‌های بزرگ دشوار است و درک آنها دشوارتر. ویژگی‌های جدید و رفع اشکال زمان بیشتری از شما می‌گیرد. در برخی موارد کنترل کننده‌ها باید به قطعات کوچکتر تقسیم شوند.

چگونه این کار را انجام می‌دهیم؟ ما کارهای مختلفی را که کلاس‌های کنترلر بزرگ انجام می‌دهند کشف و به هر کلاس کار خاص خود را می‌دهیم. به جای یک کلاس بزرگ، نمونه‌هایی از چند کلاس دارید که کار را به صورت تیمی انجام می‌دهند.

چگونه کارهای مختلف را تعیین و شناسایی کنیم؟ پاسخ به این سوال تعریف معماری شماست. مردم از توضیحاتی مانند "Model-View-Controller" و "Model-View-View Model" استفاده می‌کنند تا راه‌هایی را که به این سوال پاسخ می‌دهند در سطح بالایی توصیف کنند. اما پاسخ به این سوال همیشه مسئولیت شماست، بنابراین معماری شما همیشه منحصر به فرد شماست.

BeatBox، همان‌طور که گفتیم، با معماری MVVM طراحی شده است. ما طرفدار MVVM هستیم چون مقدار زیادی از کدهای خسته‌کننده کنترلر را به فایل layout منتقل می‌کند؛ جایی که به‌راحتی می‌توانید ببینید کدام بخش‌های رابط کاربری‌تان پویا هستند. هم‌زمان، کدهای پویا و مهم کنترلر را به یک کلاس view model منتقل می‌کند؛ جایی که تست‌کردن و بررسی‌کردن آن‌ها بسیار ساده‌تر است.

این‌که هر view model چقدر باید بزرگ باشد همیشه یک تصمیم سلیقه‌ای است. اگر view model شما بیش از حد بزرگ شد، می‌توانید آن را بیشتر خرد و تقسیم کنید. معماری پروژه مال شماست، نه ما.

MVVM View Models در مقابل Jetpack ViewModels

قبل از شروع پروژه جدید، نکته‌ای در مورد اصطلاحات این فصل یادتان باشد: view model که بخشی از MVVM است با کلاس Jetpack ViewModel که در فصل ۴ و فصل ۹ با آن آشنا شدید، یکسان نیست. برای جلوگیری از سردرگمی، همیشه نام کلاس Jetpack را با عنوان "ViewModel" و مفهوم MVVM را با عنوان "view model" بکار خواهیم برد.

به یاد بیاورید که Jetpack ViewModel یک کلاس خاص برای نگهداری داده‌ها در فرگمنت‌ها و اکتیویتی‌ها در طول چرخه حیات نابودی آنها است. view model در MVVM بخش مفهومی‌تری از معماری است. view modelها ممکن است با استفاده از Jetpack ViewModels پیاده‌سازی شوند، اما – همانطور که در بیشتر این فصل خواهید دید – می‌توان آنها را بدون استفاده از کلاس ViewModel نیز پیاده‌سازی کرد.

ایجاد BeatBox

زمان شروع است. اولین قدم این است که برنامه BeatBox خود را ایجاد کنید.

در اندروید استودیو File → New → New Project را برای ایجاد یک پروژه جدید انتخاب کنید. این برنامه را BeatBox نام گذاری و نام پکیج را com.codeb.beatbox و در قسمت نوع اکتیویتی Empty View Activity را انتخاب نمایید.

این اکتیویتی شبکه‌ای از دکمه‌ها را در RecyclerView نمایش می‌دهد. وابستگی RecyclerView را به فایل app/build.gradle خود اضافه کنید (همگام سازی فایل را فراموش نکنید). محتویات تولید شده خودکار res/layout/activity_main.xml را با یک RecyclerView جایگزین کنید.

لیست ۱۸.۱ – بروزرسانی فایل لیوت برای MainActivity

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
... />
...
</androidx.constraintlayout.widget.ConstraintLayout>
<androidx.recyclerview.widget.RecyclerView
xmlns:android="http://schemas.android.com/apk/res/android"
android:id="@+id/recycler_view"
android:layout_width="match_parent"
android:layout_height="match_parent" />
```

همینطور باید در MainActivity.kt نیز تغییر زیر را ایجاد کنید.

لیست ۱۸.۲ تغییر MainActivity

```
class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContentView(R.layout.activity_main)
        ViewCompat.setOnApplyWindowInsetsListener(
            findViewById(R.id.main recycler_view))
            { v, insets ->
            val systemBars = insets.getInsets(
            WindowInsetsCompat.Type.systemBars())
            v.setPadding(systemBars.left, systemBars.top
            , systemBars.right, systemBars.bottom)
            insets
        }
```

```
        }
}
```

برنامه خود را اجرا کنید. اگر یک صفحه خالی می‌بینید پس کارتان را تا این لحظه به درستی انجام داده‌اید.

پیاده‌سازی یک Data Binding ساده

کار بعدی اتصال RecyclerView است. این کاری است که قبلا انجام داده‌اید. اما این بار از Data Bindingها برای سرعت بخشیدن به کار خود استفاده خواهیم کرد.

Data Bindingها مزایای متعددی را جهت سهولت کار با لیوت‌ها ارایه می‌دهند. در موارد ساده مانند مواردی که در این بخش خواهید دید به شما امکان می‌دهد بدون نیاز به فراخوانی findViewById(…) به ویوها دسترسی داشته باشید. بعدا کاربردهای پیشرفته تری از اتصال داده را مشاهده خواهید کرد، از جمله اینکه چگونه به پیاده‌سازی MVVM کمک می‌کند.

با فعال کردن اتصال داده‌ها در فایل build.gradle برنامه خود شروع کنید.

لیست ۱۸.۲ فعال سازی اتصال داده (app/build.gradle)

```
plugins {
    alias(libs.plugins.android.application)
}
android {
    namespace = "com.codeb.beatbox"
    compileSdk {
        …
    }
    buildFeatures {
        dataBinding = true
    }
...
}
```

توجه داشته باشید که استفاده از افزونه kotlin-kapt به Data Binding اجازه می‌دهد تا پردازش حاشیه‌نویسی را در Kotlin انجام دهد. بعدا در این فصل خواهید دید که چرا این موضوع مهم است.

برای استفاده از اتصال داده در یک فایل لیوت، باید فایل را به یک فایل لیوت اتصال داده تبدیل کنیم. این کار را با قرار دادن کل فایل XML در یک تگ <layout> انجام می‌دهیم. همانطور که در فهرست ۱۹.۳ نشان داده شده است، این تغییر را در activity_main.xml ایجاد کنید.

لیست ۱۸.۳ جمع بندی آن (res/layout/activity_main.xml)

```
<layout
xmlns:android="http://schemas.android.com/apk/res/android">
    <androidx.recyclerview.widget.RecyclerView
xmlns:android="http://schemas.android.com/apk/res/android"
        android:id="@+id/recycler_view"
        android:layout_width="match_parent"
```

```
        android:layout_height="match_parent"/>
</layout>
```

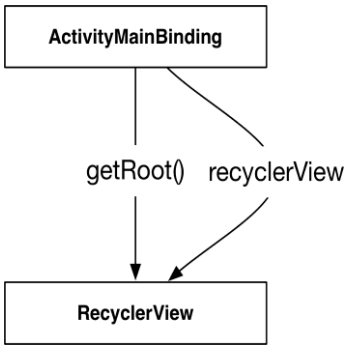
برچسب <layout> سیگنال شما به کتابخانه اتصال داده برای کار روی فایل لیوت شما است. کتابخانه برای لیوت‌هایی با این تگ، یک کلاس اتصال (binding class) ایجاد می‌کند. به طور پیش‌فرض این کلاس با نام فایل لیوت شما نامگذاری می‌شود. اما به جای استفاده از استایل تمام کلمات با حروف کوچک که با زیر خط (underline) از هم جدا می‌شوند از استایل کلمات با حروف اول بزرگ و چسبیده به هم و با پسوند Binding استفاده می‌شود.

بنابراین کتابخانه اتصال داده باید قبلا یک کلاس اتصال برای فایل activity_main.xml شما به نام ActivityMainBinding ایجاد کرده باشد. این کلاس همان چیزی است که شما برای اتصال داده‌ها از آن استفاده خواهید کرد: به جای بارگذاری سلسله مراتب view با setContentView(Int) نمونه‌ای از ActivityMainBinding را بارگذاری خواهیم کرد.

ActivityMainBinding سلسله مراتب view را برای شما در ویژگی‌ای به نام root نگه می‌دارد. علاوه بر آن ارجاعی برای هر ویویی که با android:id در فایل لیوت خود برچسب گذاری کرده‌اید، نگهداری می‌کند.

بنابراین کلاس ActivityMainBinding شما دارای دو ارجاع است: root که به کل لیوت اشاره دارد و recyclerView که فقط به RecyclerView اشاره می‌کند (شکل ۱۸.۲).

تصویر ۱۸.۲ کلاس اتصال شما



به دلیل وجود تنها یک ویو در لیوت هر دو ارجاع به یک ویو اشاره دارند: RecyclerView.

اکنون از کلاس اتصال خود استفاده کنید. onCreate(Bundle?) در MainActivity را جهت استفاده از DataBindingUtil برای بارگذاری کردن نمونه‌ای از ActivityMainBinding به روز کنید (فهرست ۱۸.۴).

باید ActivityMainBinding را مانند هر کلاس دیگری ایمپورت کنیم. اگر اندروید استودیو نتواند ActivityMainBinding را پیدا کند، به این معنی است که به دلایلی به صورت خودکار تولید نشده است. اندروید استودیو را مجددا راه اندازی کنید.

لیست ۱۸.۴ – بارگذاری یک کلاس اتصال (MainActivity.kt)

```
class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContentView(R.layout.activity_main)
        val binding: ActivityMainBinding =
            DataBindingUtil.setContentView(this
                , R.layout.activity_main)
        ...
        }
    }
}
```

با ایجاد کلاس اتصال خود اکنون می‌توانیم به RecyclerView دسترسی داشته و آن را پیکربندی کنیم.

لیست ۱۸.۵ پیکربندی RecyclerView (MainActivity.kt)

```
class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        ...
        ViewCompat.setOnApplyWindowInsetsListener
        (findViewById(R.id.recycler_view))
        ...
        binding.recyclerView.apply {
            layoutManager = GridLayoutManager(context, 3)
        }
    }
}
```

در مرحله بعد فایل لیوت res/layout/list_item_sound.xml را برای دکمه‌ها ایجاد کنید. ما در اینجا نیز از اتصال داده استفاده خواهیم کرد، بنابراین لیوت خود را با تگ <layout> احاطه می‌کنیم.

لیست ۱۸.۶ ایجاد لیوت sound (res/layout/list_item_sound.xml)

```
<?xml version="1.0" encoding="utf-8"?>
```

```
<layout
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">
    <Button
        android:layout_width="match_parent"
        android:layout_height="120dp"
        tools:text="Sound name"/>
</layout>
```

در مرحله بعد، یک SoundHolder متصل به list_item_sound.xml ایجاد کنید.

لیست ۱۸.۷ ایجاد SoundHolder (MainActivity.kt)

```
class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        ...
    }
    private inner class SoundHolder
    (private val binding: ListItemSoundBinding) :
        RecyclerView.ViewHolder(binding.root) {
    }
}
```

SoundHolder شما انتظار کلاس بایندینگی را دارد که به طور ضمنی ایجاد کرده‌اید: ListItemSoundBinding. سپس یک آداپتور متصل به SoundHolder ایجاد کنید.

لیست ۱۸.۸ ایجاد SoundAdapter (MainActivity.kt)

```
class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        ...
    }
    private inner class SoundHolder
    (private val binding: ListItemSoundBinding) :
        RecyclerView.ViewHolder(binding.root) {
    }
    private inner class SoundAdapter() :
        RecyclerView.Adapter<SoundHolder>() {
        override fun onCreateViewHolder
       (parent: ViewGroup, viewType: Int):
                SoundHolder {
            val binding = DataBindingUtil
            .inflate<ListItemSoundBinding>(
                layoutInflater,
                R.layout.list_item_sound,
                parent,
                false
            )
```

```
            return SoundHolder(binding)
        }
        override fun onBindViewHolder
        (holder: SoundHolder, position: Int) {
        }
        override fun getItemCount() = 0
    }
}
```

در نهایت SoundAdapter را در onCreate(Bundle?) متصل کنید.

لیست ۱۸.۹ متصل کردن SoundAdapter (MainActivity.kt)

```
override fun onCreate(savedInstanceState: Bundle?) {
    ...
    binding.recyclerView.apply {
        layoutManager = GridLayoutManager(context, 3)
        adapter = SoundAdapter()
    }
}
```

اکنون از اتصال داده‌ها برای تنظیم recycler view استفاده کردیم. متأسفانه هنوز چیزی برای نمایش دادن وجود ندارد. وقت آن است که با دادن چند فایل صوتی برای نمایش، آن را برطرف کنیم.

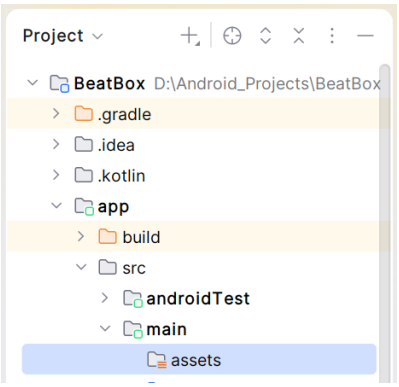
ایمپورت دارایی‌ها

ابتدا باید فایل‌های صوتی را جهت استفاده در زمان اجرا به پروژه اضافه کنیم. به جای استفاده از سیستم منابع برای این کار از raw assets استفاده خواهیم کرد. این دارایی‌ها در APK نهایی ما بدون اعمال هیچ پیکربندی از طرف ابزار سیستم پکیج بندی می‌شوند.

از برخی جهات این روش خوبی است. از آنجایی که هیچ سیستم پیکربندی وجود ندارد، می‌توانید دارایی‌ها را هر طور که می‌خواهید نامگذاری کرده و با ساختار پوشه خود سازماندهی کنید. با این حال از برخی جهات نیز بد به نظر می‌رسد. بدون سیستم پیکربندی نمی‌توان به طور خودکار به تغییرات تراکم پیکسل، زبان یا جهت پاسخ داد. همچنین نمی‌توان به طور خودکار از دارایی‌ها در فایل‌های لیوت یا سایر منابع استفاده کنید.

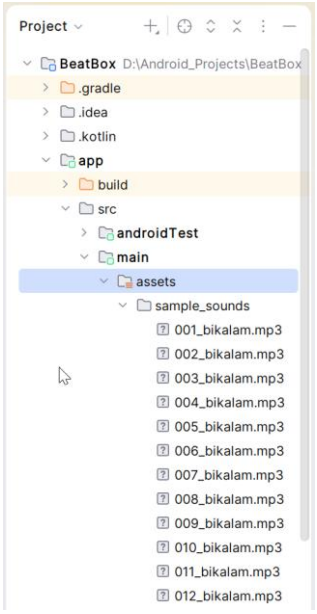
معمولا سیستم منابع راه حل بهتری است. با این حال در مواردی که فقط به صورت برنامه‌نویسی به فایل‌ها دسترسی داریم، می‌توانیم از روش raw assets استفاده کنیم.

اولین قدم ایمپورت کردن دارایی‌ها خواهد بود. ابتدا ویو را به Project تغییر دهید، سپس با کلیک راست بر روی main و انتخاب New → Directory → Assets، یک پوشه assets در پروژه خود ایجاد کنید. سپس داخل این پوشه، پوشه دیگری با نام sample_sounds بسازید (تصویر ۱۸.۳).



از کجا می‌توانیم صداها را پیدا کنید؟ می توانید تعدادی دانلود کنید. سپس فایل‌ها را به sample_sounds کپی یا منتقل کنید.

شکل ۱۸.۴ ایمپورت دارایی‌های



مرحله بعدی فهرست کردن دارایی‌ها و نشان دادن آنها به کاربر خواهد بود.

دسترسی به دارایی‌ها

BeatBox در نهایت کارهای زیادی را در رابطه با مدیریت دارایی انجام می‌دهد: یافتن دارایی‌ها، پیگیری آنها و در نهایت پخش آنها به عنوان صدا. برای مدیریت همه اینها یک کلاس جدید به نام BeatBox در com.codeb.beatbox ایجاد کنید. در ادامه چند ثابت اضافه کنید: یکی به عنوان شناسه لاگ و دیگری برای تعیین محل ذخیره‌سازی صداها.

لیست ۱۸.۱۰ کلاس جدید BeatBox (BeatBox.kt)

```
private const val TAG = "BeatBox"
private const val SOUNDS_FOLDER = "sample_sounds"
class BeatBox {
}
```

دارایی‌ها با استفاده از کلاس AssetManager قابل دسترسی هستند. می‌توان یک AssetManager را از هر Context دریافت کرد. از آنجایی که BeatBox به یکی نیاز دارد، سازنده‌ای برای آن پیاده‌سازی کنید تا یک AssetManager را بگیرد و آن را برای کارهای بعدی ذخیره کند.

لیست ۱۸.۱۱ ذخیره کردن یک AssetManager (BeatBox.kt)

```
package com.codeb.beatbox
import android.content.res.AssetManager
private const val TAG = "BeatBox"
private const val SOUNDS_FOLDER = "sample_sounds"
class BeatBox(private val assets: AssetManager) {
}
```

هنگام دسترسی به دارایی‌ها لازم نیست نگران این باشید که از کدام Context استفاده می‌کنید. در هر موقعیتی که احتمالا در عمل با آن مواجه شوید، هر AssetManager به مجموعه‌ای از دارایی‌های یکسان متصل می‌شود.

برای دریافت لیستی از آنچه در دارایی‌های خود داریم از تابع list(String) استفاده می‌کنیم. تابعی به نام loadSounds() بنویسید که با list(String) لیستی از دارایی‌های شما را ایجاد کند.

لیست ۱۸.۱۲ دارایی‌ها (BeatBox.kt)

```
class BeatBox (private val assets: AssetManager){
    fun loadSounds(): List<String> {
        try {
            val soundNames = assets.list(SOUNDS_FOLDER)!!
            Log.d(TAG, "Found ${soundNames.size} sounds")
            return soundNames.asList()
        } catch (e: Exception) {
```

```
            Log.e(TAG, "Could not list assets", e)
            return emptyList()
        }
    }
}
```

AssetManager.list(String) نام فایل‌های موجود در مسیر پوشه‌ای را که پاس می‌کنید، فهرست می‌کند. باید تمام فایل‌ها با پسوند mp3. را که در پوشه sample_sounds قرار دارد، مشاهده کنید.

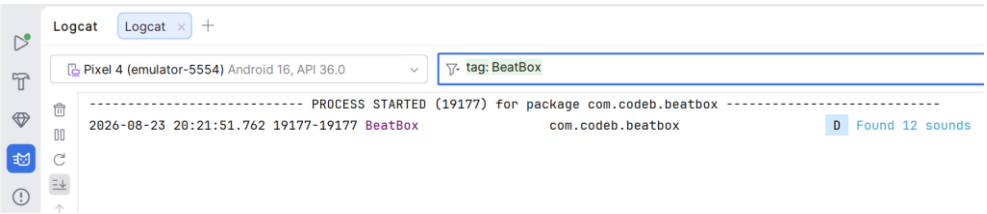
برای تأیید درستی کار یک نمونه از BeatBox در MainActivity ایجاد و متد ()loadSounds را فراخوانی کنید.

لیست ۱۸.۱۳ ایجاد یک نمونه BeatBox (MainActivity.kt)

```
class MainActivity : AppCompatActivity() {
    private lateinit var beatBox: BeatBox
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        beatBox = BeatBox(assets)
        beatBox.loadSounds()
        ...
    }
    ...
}
```

برنامه خود را اجرا و خروجی گزارش را ببینید. چند فایل صوتی پیدا شده است. ما ۱۲ فایل mp3. ارائه کردیم، بنابراین باید چنین گزارشی نمایش داده شود:

شکل ۱۸.۵ تعداد فایل‌ها



استفاده از دارایی‌ها

اکنون که لیست فایل‌های دارایی خود را داریم. پس وقت آن است که آنها را به کاربر ارائه دهیم. در نهایت ما می‌خواهیم فایل‌ها پخش شوند. بنابراین منطقی است که یک شی مسئول پیگیری نام فایل، نامی‌که کاربر باید ببیند و هر گونه اطلاعات دیگر مربوط به آن صدا را داشته باشید.

یک کلاس Sound ایجاد کنید تا همه اینها را در خود جای دهد.

لیست ۱۸.۱۴ – ایجاد آبجکت‌ها Sound (Sound.kt)

```
package com.codeb.beatbox
private const val MP3 = ".MP3"
class Sound(val assetPath: String) {
    val name = assetPath.split("/").last().removeSuffix(MP3)
}
```

در سازنده برای بدست آوردن نامی قابل ارایه کمی کد می‌نویسیم. ابتدا نام فایل را با استفاده از String.split(String).last() پیدا کرده، سپس از String.removeSuffix(String) برای حذف پسوند فایل استفاده می‌کنیم.

در مرحله بعد فهرستی از صداها را در BeatBox.loadSounds() ایجاد کنید.

لیست ۱۸.۱۵ ایجاد Soundها (BeatBox.kt)

```
class BeatBox (private val assets: AssetManager){
    val sounds: List<Sound>
    init {
        sounds = loadSounds()
    }
    fun loadSounds(): List<String> List<Sound> {
        val soundNames: Array<String>
        try {
            val soundNames = assets.list(SOUNDS_FOLDER)!!
            Log.d(TAG, "Found ${soundNames.size} sounds")
            return soundNames.asList()
        } catch (e: Exception) {
            Log.e(TAG, "Could not list assets", e)
            return emptyList()
        }
        val sounds = mutableListOf<Sound>()
        soundNames.forEach { filename ->
            val assetPath = "$SOUNDS_FOLDER/$filename"
            val sound = Sound(assetPath)
            sounds.add(sound)
        }
```

```
        return sounds
    }
}
```

اکنون SoundAdapter را به لیست Soundها متصل کنید.

لیست ۱۸.۱۶ اتصال به لیست Sound (MainActivity.kt)

```
private inner class SoundAdapter(
private val sounds: List<Sound>) :
    …
    }
    override fun onBindViewHolder(holder: SoundHolder
    , position: Int) {
    }
    override fun getItemCount() = 0 sounds.size
}
```

حالا صداهای BeatBox را در onCreate(Bundle?) پاس دهید.

لیست ۱۸.۱۷ انتقال صداها به آداپتور (MainActivity.kt)

```
override fun onCreate(savedInstanceState: Bundle?) {
    …
    binding.recyclerView.apply {
        layoutManager = GridLayoutManager(context, 3)
        adapter = SoundAdapter(beatBox.sounds)
    }
    …
}
```

در نهایت، فراخوانی تابع BeatBox.loadSounds() را در onCreate حذف کنید.

لیست ۱۸.۱۸ حذف BeatBox.loadSounds() از onCreate(…) (MainActivity.kt)

```
override fun onCreate(savedInstanceState: Bundle?) {
    …
    beatBox = BeatBox(assets)
    beatBox.loadSounds()
    …
}
```

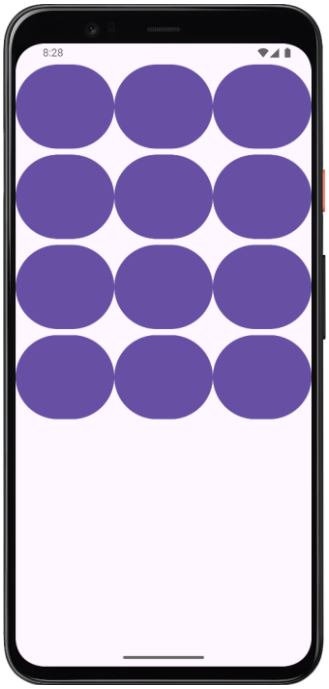
از آنجایی که BeatBox.loadSounds() دیگر از خارج از بلوک مقداردهی اولیه BeatBox فراخوانی نمی‌شود، نیازی به عمومی نگه داشتن این تابع نیست. برای اینکه شهروند خوبی باشید و از فراخوانی ناخواسته loadSounds() از سایر قسمت‌های کد جلوگیری کنید، آن را به private تغییر دهید.

لیست ۱۸.۱۹ به روز رسانی BeatBox.loadSounds() به خصوصی (BeatBox.kt)

```
class BeatBox (private val assets: AssetManager){
    …
    }
    private fun loadSounds(): List<Sound> {
        …
    }
}
```

با این کار هنگام اجرای BeatBox باید شبکه‌ای از دکمه‌ها را ببینید (شکل ۱۸.۶).

تصویر ۱۸.۶ دکمه‌های خالی



برای پر کردن دکمه‌ها با عناوین، از ابزارهای اضافی اتصال داده جدید خود استفاده خواهیم کرد.

اتصال به داده‌ها

با اتصال داده‌ها می‌توان آبجکت‌ها داده را در فایل لیوت اعلام کرد:

```
<layout
xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:tools="http://schemas.android.com/tools">
```

```
<data>
    <variable
        name="book"
        type="com.codeb.bookshelf.Book"/>
</data>
...
</layout>
```

و سپس از مقادیر آن آبجکت‌ها مستقیما در فایل لیوت خود با استفاده از اپراتورهای binding mustache استفاده کنیم، @{}:

```
<CheckBox
android:id="@+id/list_item_crime_solved_check_box"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:layout_alignParentRight="true"
android:checked="@{book.isSolved()}"
android:padding="4dp"/>
```

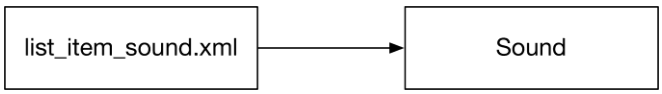
در یک آبجکت دیاگرام، مانند شکل ۱۸.۷ به نظر می‌رسد:

شکل ۱۸.۷ اتصال



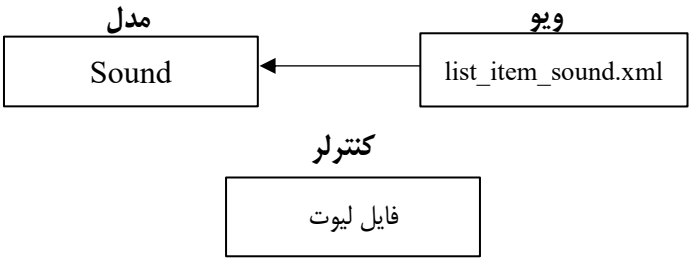
هدف ما در این مرحله آن است که نام صداها را بر روی دکمه‌ها قرار دهیم. مستقیم‌ترین روش برای انجام این کار با استفاده از data binding آن است که در فایل list_item_sound.xml مستقیماً یک شی Sound را مورد اتصال (bind) قرار دهیم (شکل ۱۸.۸):

شکل ۱۸.۸ اتصال مستقیم



با این حال این امر باعث برخی مشکلات معماری می‌شود. برای اینکه ببینید چرا، از منظر MVC به آن نگاه کنید (شکل ۱۸.۹).

شکل ۱۸.۹ MVC شکسته

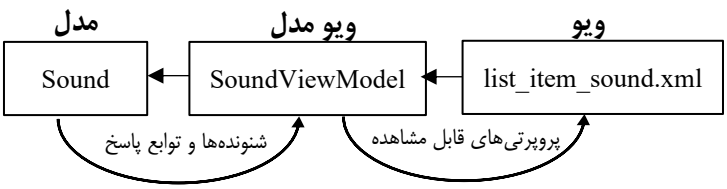


اصل راهنمای هر معماری، اصل مسئولیت واحد (Single Responsibility Principle) است. این اصل اشاره به آن دارد که هر کلاسی که می‌سازید باید دقیقا یک مسئولیت داشته باشد. MVC به شما این ایده را می‌دهد که این مسئولیت‌ها باید چه باشند: مدل نشان دهنده نحوه عملکرد برنامه شما است، کنترلر تصمیم می‌گیرد که چگونه برنامه شما را نمایش دهد، و ویو آن را روی صفحه نمایش می‌دهد.

استفاده از اتصال داده‌ها همانطور که در شکل ۱۹.۸ نشان داده شده است این تقسیم مسئولیت‌ها را از بین می‌برد. زیرا شی مدل Sound احتمالا با کدی باید به پایان می‌رسید که داده‌ها را برای نمایش ویو آماده می‌کند. این به سرعت برنامه شما را به هم می‌ریزد، زیرا Sound.kt با دو نوع کد پر می‌شود: کدی که نحوه عملکرد برنامه شما را نشان می‌دهد و کدی که داده‌ها را برای نمایش نمایش آماده می‌کند.

به جای گل آلود کردن مسئولیت Sound، یک شی جدید به نام view model را برای استفاده با اتصال داده‌ها معرفی خواهیم کرد. این مدل ویو مسئولیت آماده سازی داده‌ها برای نمایش ویو را بر عهده خواهد گرفت (شکل ۱۸.۱۰).

شکل ۱۸.۱۰ Model-View-View Model



همانطور که گفتیم این معماری MVVM نامیده می‌شود که مخفف Model-View-View Model است. بیشتر کارهایی که کلاس‌های کنترلر شما زمانی در زمان اجرا برای قالب بندی داده‌ها برای نمایش انجام می‌دادند، در view model انجام می‌شود. ویجت‌های متصل با آن داده‌ها مستقیما در فایل لیوت با استفاده از اتصال داده به آن مدل ویو مدیریت می‌شوند. اکتیویتی یا فرگمنت شما مسئول مواردی مانند مقداردهی اولیه اتصال و view model و ایجاد پیوند بین این دو خواهد بود.

هیچ کنترلری با MVVM وجود ندارد. بلکه اکتیویتی‌ها و فرگمنت‌ها بخشی از ویو محسوب می‌شوند.

ایجاد یک view model

بیایید ویو مدل خودمان را ایجاد کنیم. یک کلاس جدید به نام SoundViewModel ایجاد و به آن دو ویژگی بدهید: یک Sound برای استفاده و یک BeatBox برای پخش.

لیست ۱۸.۲۰ ایجاد SoundViewModel (SoundViewModel.kt)

```
class SoundViewModel {
    var sound: Sound? = null
        set(sound) {
            field = sound
        }
}
```

این ویژگی‌ها رابطی هستند که آداپتور شما از آن استفاده می‌کند. برای فایل لیوت یک تابع اضافی برای دریافت عنوانی که دکمه باید نمایش دهد، می‌خواهیم. اکنون آن را به SoundViewModel اضافه کنید.

لیست ۱۸.۲۱ افزودن توابع اتصال (SoundViewModel.kt)

```
class SoundViewModel {
    var sound: Sound? = null
        set(sound) {
            field = sound
        }
    val title: String?
        get() = sound?.name
}
```

اتصال به یک view model

اکنون view model را در فایل لیوت خود اضافه کنید. اولین قدم این است که یک ویژگی را در فایل لیوت خود اعلام نمایید.

لیست ۱۸.۲۲ اعلام ویژگی view model (res/layout/list_item_sound.xml)

```
<layout
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">
    <data>
        <variable
            name="viewModel"
            type="com.codeb.beatbox.SoundViewModel"/>
    </data>
    ...
</layout>
```

این یک ویژگی به نام viewModel را در کلاس اتصال شما تعریف می‌کند که شامل یک getter و setter می‌شود. در کلاس اتصال خود، می‌توانید از viewModel در عبارات اتصال استفاده کنید.

لیست ۱۸.۲۳ اتصال عنوان دکمه‌ها (res/layout/list_item_sound.xml)

```
<layout
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">
    <data>
        <variable
            name="viewModel"
            type="com.codeb.beatbox.SoundViewModel"/>
    </data>
    <Button
        android:layout_width="match_parent"
        android:layout_height="120dp"
        tools:text="Sound name"
        android:text="@{viewModel.title}"/>
</layout>
```

در binding mustache می‌توان عبارات ساده جاوا از جمله فراخوانی‌های تابع زنجیره‌ای، ریاضیات و چیزهای دیگر استفاده کرد.

آخرین مرحله اتصال view model است. یک SoundViewModel ایجاد و آن را به کلاس اتصال خود وصل کنید. سپس یک تابع اتصال به SoundHolder خود اضافه کنید.

لیست ۱۸.۲۴ اتصال view model (MainActivity.kt)

```
private inner class SoundHolder(private val binding:
ListItemSoundBinding) :
    RecyclerView.ViewHolder(binding.root) {
    init {
        binding.viewModel = SoundViewModel()
    }
    fun bind(sound: Sound) {
        binding.apply {
            viewModel?.sound = sound
            executePendingBindings()
        }
    }
}
```

در داخل سازنده، view model خود را ساخته و ضمیمه می‌کنیم. سپس در تابع bind، داده‌هایی را که view model با آنها کار می‌کند به روز می‌کنیم.

فراخوانی executePendingBindings() معمولا ضروری نیست. با این حال در اینجا ما داده‌های اتصال را در داخل یک RecyclerView به روز می‌کنیم که ویوها را با سرعت بسیار بالا به روز می‌کند. با فراخوانی این تابع به جای اینکه یک یا دو میلی ثانیه منتظر بمانیم، لیوت را بلافاصله مجبور به به روز رسانی می‌کنیم. این کار RecyclerView را با RecyclerView.Adapter آن همگام نگه می‌دارد.

در نهایت اتصال view model خود را با پیاده‌سازی onBindViewHolder(…) به پایان می‌رسانیم.

لیست ۱۸.۲۵ فراخوانی bind(Sound) (MainActivity.kt)

```
private inner class SoundAdapter(
private val sounds: List<Sound>) :
    RecyclerView.Adapter<SoundHolder>() {
    override fun onCreateViewHolder(parent: ViewGroup
    , viewType: Int):
            SoundHolder {
        ...
    }
    override fun onBindViewHolder(holder: SoundHolder
    , position: Int) {
        val sound = sounds[position]
        holder.bind(sound)
    }
    override fun getItemCount() = sounds.size
}
```

برنامه خود را اجرا کنید، عناوین را روی تمام دکمه‌های خواهید دید (تصویر ۱۸.۱۱).

تصویر ۱۸.۱۱ عناوین دکمه‌ها



بیشتر بدانیم: اطلاعات بیشتر در مورد Data Binding

پوشش کامل Data Binding خارج از محدوده این کتاب است. ولی با این حال باید تلاش کنیم.

عبارات لامبدا

می‌توان با استفاده از عبارات لامبدا (lambda expressions) توابع پاسخ کوتاه را مستقیما در فایل لیوت خود بنویسید. اینها نسخه‌های ساده شده عبارات لامبدا جاوا هستند:

```
<Button
    android:layout_width="match_parent"
    android:layout_height="120dp"
    android:text="@{viewModel.title}"
    android:onClick="@{(view) -> viewModel.onButtonClick()}"
    tools:text="Sound name"/>
```

همچنین برخلاف جاوا لامبدا اگر از پارامترهای لامبدا استفاده نمی‌کنید، می‌توانید آنها را حذف کنید:

```
android:onClick="@{() -> viewModel.onButtonClick()}"
```

۱۹

پخش صدا

در این فصل، در نهایت تمام فایل‌های mp3. را که در فصل قبل بارگذاری کردیم، پخش خواهیم کرد.

APIهای صوتی اندروید در بیشتر موارد سطح پایینی دارند، اما ابزاری وجود دارد که عملا برای برنامه‌ای که می‌نویسیم طراحی شده است: SoundPool. SoundPool می‌تواند مجموعه بزرگی از صداها را در حافظه بارگذاری و حداکثر تعداد صداهایی را که در هر زمان پخش می‌شوند، کنترل کند. بنابراین اگر کاربر برنامه ما کمی‌بیش از حد هیجان زده شود و همه دکمه‌ها را به طور همزمان لمس کند، برنامه ما از کار نمی‌افتد و یا گوشی کاربر را بیش از حد تحت فشار قرار نمی‌دهد.

آماده‌اید؟ زمان شروع است.

ایجاد یک SoundPool

اولین کار پخش صدا در داخل BeatBox است. برای انجام این کار، ابتدا یک شی SoundPool (لیست ۱۹.۱) ایجاد کنید.

لیست ۱۹.۱ ایجاد یک SoundPool (BeatBox.kt)

```
private const val TAG = "BeatBox"
private const val SOUNDS_FOLDER = "sample_sounds"
private const val MAX_SOUNDS = 5
class BeatBox (private val assets: AssetManager){
    val sounds: List<Sound>
    private val soundPool = SoundPool.Builder()
        .setMaxStreams(MAX_SOUNDS)
        .build()
    ...
}
```

یک Builder برای ایجاد یک نمونه SoundPool استفاده می‌شود. گزینه setMaxStreams(Int) در سازنده مشخص می‌کند که چند صدا می‌توانند در هر زمان پخش شوند. اگر پنج صدا در حال پخش باشد و کاربر سعی کند ششمین صدا را پخش کند، SoundPool پخش قدیمی‌ترین صدا را متوقف می‌کند.

علاوه بر گزینه max streams، سازنده SoundPool به شما امکان می‌دهد ویژگی‌های مختلف صوتی را با گزینه setAudioAttributes(AudioAttributes) مشخص کنید. می‌توانید از مستندات برای انجام این کار بهره بگیرید.

دسترسی به دارایی‌ها

به یاد بیاورید که فایل‌های صوتی شما در دارایی‌های برنامه ذخیره می‌شوند. قبل از دسترسی به آن فایل‌ها برای پخش صدا، اجازه دهید کمی‌بیشتر در مورد نحوه عملکرد دارایی‌ها بحث کنیم.

شی Sound شما دارای یک مسیر فایل دارایی تعریف شده روی آن است. اگر بخواهید آنها را با یک File باز کنید، مسیر فایل دارایی به درد نمی‌خورد بلکه باید از یک AssetManager استفاده کنید:

```
val assetPath = sound.assetPath
val assetManager = context.assets
val soundData = assetManager.open(assetPath)
```

این کار یک InputStream استاندارد برای داده‌ها در اختیار شما می‌گذارد که می‌توانیم مانند هر InputStream دیگری در Kotlin از آن استفاده کنیم.

برخی از APIها همانند SoundPool به جای آن به FileDescriptors نیاز دارند. برای انجام این کار AssetManager.openFd(String) را فراخوانی می‌کنیم:

```
val assetPath = sound.assetPath
val assetManager = context.assets
// AssetFileDescriptors are different from FileDescriptors...
val assetFileDescriptor = assetManager.openFd(assetPath)
// ... but you can get a regular FileDescriptor easily if you
need to
val fileDescriptor = assetFileDescriptor.fileDescriptor
```

بارگذاری صداها

کار بعدی که باید با SoundPool انجام دهیم بارگذاری صداها در آن است. مزیت اصلی استفاده از SoundPool نسبت به برخی از روش‌های دیگر پخش صدا، سرعت پاسخگویی بالای آن است.

هر صدایی که بارگذاری می‌کنیم، شناسه خود را دریافت می‌کند. برای ردیابی این شناسه، یک ویژگی soundId به Sound اضافه کنید.

لیست ۱۹.۲ افزودن ویژگی شناسه صدا (Sound.kt)

```
class Sound(val assetPath: String, var soundId: Int? = null)
{
    val name = assetPath.split("/").last().removeSuffix(MP3)
}
```

حالا برای بارگذاری صداها تابع load(Sound) را به BeatBox اضافه کنید تا یک Sound را در SoundPool بارگذاری کند.

لیست ۱۹.۳ بارگذاری soundها در SoundPool (BeatBox.kt)

```
class BeatBox (private val assets: AssetManager){
    …
    private fun loadSounds(): List<Sound> {
        …
    }
    private fun load(sound: Sound) {
        val afd: AssetFileDescriptor =
            assets.openFd(sound.assetPath)
        val soundId = soundPool.load(afd, 1)
        sound.soundId = soundId
    }
}
```

فراخوانی soundPool.load (AssetFileDescriptor, Int) یک فایل را برای پخش بعدی در SoundPool بارگذاری می‌کند.

برای پیگیری صدا و پخش مجدد آن (یا تخلیه آن)، soundPool.load(…) یک شناسه برمی‌گرداند.

توجه داشته باشید که فراخوانی openFd(String) می‌تواند IOException را در پی داشته باشد، همچنین load(Sound) نیز می‌تواند همین مشکل را بوجود آورد. این بدان معناست که شما باید هر زمان که load(Sound) را فراخوانی می‌کنید باید مدیریت استثنا داشته باشید. اکنون تمام صداهای خود را با فراخوانی load(Sound) در داخل BeatBox.loadSounds() بارگذاری کنید.

لیست ۱۹.۴ بارگیری تمام صداهای (BeatBox.kt)

```
private fun loadSounds(): List<Sound> {
    val soundNames: Array<String>
    try {
        soundNames = assets.list(SOUNDS_FOLDER)!!
    } catch (e: Exception) {
        Log.e(TAG, "Could not list assets", e)
        return emptyList()
    }
    val sounds = mutableListOf<Sound>()
    soundNames.forEach { filename ->
```

```
            val assetPath = "$SOUNDS_FOLDER/$filename"
            val sound = Sound(assetPath)
            sounds.add(sound)
            try {
                load(sound)
                sounds.add(sound)
            } catch (ioe: IOException) {
                Log.e(TAG, "Cound not load sound $filename", ioe)
            }
        }
        return sounds
}
```

BeatBox را اجرا و مطمئن شوید که تمام صداها به درستی بارگذاری شده‌اند. اگر این کار را به درستی انجام نگیرد، گزارش‌های استثنای قرمز را در Logcat مشاهده می‌شود.

پخش صداها

BeatBox باید قادر به پخش صداها باشد. تابع play(Sound) را به BeatBox اضافه کنید.

لیست ۱۹.۵ پخش صداها (BeatBox.kt)

```
class BeatBox (private val assets: AssetManager){
    …
    private fun load(sound: Sound) {
        …
    }
    fun play(sound: Sound) {
        sound.soundId?.let {
            soundPool.play(it, 1.0f, 1.0f, 1, 0, 1.0f)
        }
    }
}
```

قبل از پخش، مطمئن شوید که soundId تهی یا null نیست. در صورت عدم بارگذاری صدا، ممکن است این اتفاق بیافتد. بعد از اطمینان، صدا را با فراخوانی soundPool.play(Int, Float, Float, Int, Int, Float) پخش می‌کنیم. این پارامترها به ترتیب عبارتند از: شناسه صدا، میزان صدا در سمت چپ، میزان صدا در سمت راست، اولویت، اینکه آیا صدا باید تکرار شود یا خیر، و نرخ پخش. برای حجم کامل و نرخ پخش معمولی، عدد ۱.۰ را پاس می‌کنید. با پاس ۰ برای مقدار تکرار می‌گوییم "تکرار صورت نگیرید". (اگر می‌خواهید صدا در یک حلقه بی پایان تکرار شود عدد ۱- را پاس نمایید.)

با نوشتن این تابع، اکنون آماده ادغام پخش صدا در SoundViewModel هستید. ما این ادغام را به صورت تست واحد انجام خواهیم داد – یعنی ابتدا یک تست واحد ناموفق می‌نویسیم، سپس ادغام را پیاده‌سازی می‌کنیم تا تست را بگذرانیم.

در مرحله بعد ()onButtonClicked را پیاده‌سازی کنید (فهرست ۱۹.۶).

لیست ۱۹.۶ پیاده‌سازی ()onButtonClicked (SoundViewModel.kt)

```
class SoundViewModel(private val beatBox: BeatBox) :
BaseObservable(){
    var sound: Sound? = null
        set(sound) {
            field = sound
            notifyChange()
        }
    @get:Bindable
    val title: String?
        get() = sound?.name
```

```
    fun onButtonClicked() {
        sound?.let {
            beatBox.play(it)
        }
    }
}
```

توابع پاسخ Data Binding

تنها یک مرحله باقی می‌ماند تا دکمه‌های کار کنند: باید ()onButtonClicked را به دکمه‌های خود اتصال دهیم.

به یاد دارید که چگونه از اتصال داده برای قرار دادن داده در رابط کاربری خود استفاده کردید؟ همچنین می‌توانیم از آن برای اتصال شنوندگان کلیک و موارد مشابه با استفاده از عبارات لامبدا استفاده کرد.

یک تابع پاسخ اتصال داده برای اتصال دکمه‌های خود به ()SoundViewModel.onButtonClicked اضافه کنید.

لیست ۱۹.۷ درگیر کردن دکمه‌ها (list_item_sound.xml)

```
<Button
    android:layout_width="match_parent"
    android:layout_height="120dp"
    tools:text="Sound name"
    android:onClick="@{() -> viewModel.onButtonClicked()}"
    android:text="@{viewModel.title}"/>
```

اکنون BeatBox را اجرا و روی دکمه‌های خود کلیک کنید. باید بشنوید که برنامه شما صداهایی ایجاد می‌کند. نترسید – این همان کاری است که BeatBox قرار بود انجام دهد.

تخلیه SoundPool

برنامه کار می‌کند، اما برای اینکه یک شهروند خوب باشید، باید SoundPool خود را با فراخوانی ()SoundPool.release وقتی کارتان تمام شد، تخلیه کنید (فهرست ۱۹.۸).

لیست ۱۹.۸ تخلیه SoundPool (BeatBox.kt)

```
class BeatBox (private val assets: AssetManager){
    ...
    fun play(sound: Sound) {
        ...
```

```
    }
    fun release() {
        soundPool.release()
    }
}
```

سپس تابع BeatBox.release() را در MainActivity (فهرست ۱۹.۹) اضافه کنید.

لیست ۱۹.۹ تخلیه BeatBox (MainActivity.kt)

```
class MainActivity : AppCompatActivity() {
    ...
    private inner class SoundHolder
        (private val binding: ListItemSoundBinding) :
        RecyclerView.ViewHolder(binding.root) {
        init {
            binding.viewModel = SoundViewModel(beatBox)
        }
        fun bind(sound: Sound) {
            …
            }
        }
    }
    private inner class SoundAdapter
    (private val sounds: List<Sound>) :
        ...
    }
    override fun onDestroy() {
        super.onDestroy()
        beatBox.release()
    }
}
```

برنامه خود را دوباره اجرا و مطمئن شوید که با تابع release() به درستی کار می‌کند.

چالش: کنترل سرعت پخش

در این چالش، کنترل سرعت پخش را به BeatBox اضافه خواهیم کرد (تصویر ۱۹.۸). در MainActivity یک SeekBar را متصل کنید تا مقدار نرخی را که تابع پخش play(Int, Float, Float, Int, Int, Float) می‌پذیرد، کنترل کنید.

چالش: پخش صدا در طول چرخش

در حال حاضر اگر دستگاه را بچرخانید، BeatBox پخش صدا را متوقف می‌کند. این چالش این مشکل را برطرف خواهد کرد.

مشکل اصلی محل ذخیره‌سازی شی BeatBox در برنامه است. MainActivity ارجاعی به BeatBox دارد، اما در طول چرخش از بین می‌رود و دوباره ایجاد می‌شود. این بدان معنی است که BeatBox شما SoundPool را آزاد و هر بار که می‌چرخانید آن را دوباره ایجاد می‌کند.

قبلا دیده‌اید که چگونه اطلاعات را در سراسر چرخش در GeoQuiz و BookShelfion حفظ کنید. یک Jetpack ViewModel به BeatBox اضافه کنید تا آبجکت‌ها BeatBox خود را در طول چرخش زنده نگه دارید.

می‌توانید شی BeatBox را به عنوان یک ویژگی عمومی‌از ViewModel خود تعریف کنید. به این ترتیب، MainActivity می‌تواند به نمونه BeatBox از Jetpack ViewModel دسترسی داشته باشد تا آن را به مدل ویوی اتصال داده‌ها تحویل دهد.

۲۰

استایل‌ها (Styles) و تم‌ها (Themes)

BeatBox ترسناک به نظر می‌رسد. اما بد نیست کمی به این شکوه اضافه کنیم.

تا کنون BeatBox از استایل‌های پیش‌فرض رابط کاربری استفاده کرده است و برند خاص خود را ندارد. ما می‌توانیم آن را تغییر دهیم. شکل ۲۰.۱ BeatBoxی بهتر، قوی تر و سریع تر – یا حداقل شیک تر – را نشان می‌دهد.

شکل ۲۰.۱ یک BeatBox با تم



منابع رنگ

با تعریف چند رنگ که در استفاده خواهیم کرد، شروع می‌کنیم. فایل res/values/colors.xml را ویرایش کنید تا با لیست ۲۰.۱ مطابقت داشته باشد.

لیست ۲۰.۱ تعریف چند رنگ (res/values/colors.xml)

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <color name="black">#FF000000</color>
    <color name="white">#FFFFFFFF</color>
    <color name="red">#F44336</color>
    <color name="dark_red">#C3352B</color>
    <color name="gray">#607D8B</color>
    <color name="soothing_blue">#0083BF</color>
    <color name="dark_blue">#005A8A</color>
</resources>
```

منابع رنگ یک راه مناسب برای تعیین مقادیر رنگی است که در طول برنامه خود به آن ارجاع می‌دهیم.

استایل‌ها

اکنون دکمه‌های BeatBox را با یک استایل به روز می‌کنیم. استایل مجموعه‌ای از ویژگی‌هایی است که می‌توان روی ویجت اعمال کرد.

به res/values/themes.xml بروید و استایلی به نام BeatBoxButton (فهرست ۲۰.۲) اضافه کنید. (هنگامی که BeatBox را ایجاد کردید، پروژه جدید شما باید شامل دو نسخه – themes.xml و themes.xml(night) – باشد. اگر پروژه شما این فایل را ندارد آن را ایجاد کنید.) لیست ۲۰.۲ افزودن یک استایل (res/values/themes.xml)

```
<resources xmlns:tools="http://schemas.android.com/tools">
    ...
    <style name="Theme.BeatBox" parent="Base.Theme.BeatBox"/>
    <style name="BeatBoxButton">
    <item
    name="android:backgroundTint">@color/dark_blue</item>
    </style>
</resources>
```

در اینجا استایلی به نام BeatBoxButton ایجاد کردیم. این استایل ویژگی واحد android: backgroundTint را تعریف و مقدار آن را به رنگ آبی تیره تنظیم می‌کند. می‌توان این استایل را روی هر تعداد ویجت اعمال کرد.

اکنون که استایل تعریف شده است، BeatBoxButton را روی دکمه‌های خود اعمال کنید.

لیست ۲۰.۳ استفاده از یک استایل (res/layout/list_item_sound.xml)

```
<Button
    style="@style/BeatBoxButton"
    android:layout_width="match_parent"
    android:layout_height="120dp"
    tools:text="Sound name"
    android:onClick="@{() -> viewModel.onButtonClicked()}"
    android:text="@{viewModel.title}"/>
```

BeatBox را اجرا و مشاهده کنید که همه دکمه‌ها اکنون دارای رنگ پس زمینه آبی تیره هستند (تصویر ۲۰.۲).

شکل ۲۰.۲ BeatBox با استایل‌های دکمه.



می‌توان برای هر مجموعه‌ای از ویژگی‌هایی که می‌خواهیم در برنامه خود استفاده مجدد می‌کنیم، یک استایل ایجاد کرد. خیلی مفید است.

وراثت استایل

استایل‌ها همچنین از وراثت پشتیبانی می‌کنند: یک استایل می‌تواند ویژگی‌های استایل دیگری را به ارث ببرد یا آن را بازنویسی کند.

یک استایل جدید به نام BeatBoxButton.Strong ایجاد کنید که از BeatBoxButton ارث می‌برد و همچنین متن را پررنگ به نمایش می‌گذارد.

لیست ۲۰.۴ به ارث بردن از BeatBoxButton (res/values/themes.xml)

```
<resources xmlns:tools="http://schemas.android.com/tools">
    …
    <style name="BeatBoxButton">
        <item
        name="android:backgroundTint">@color/dark_blue</item>
    </style>
    <style name="BeatBoxButton.Strong">
        <item name="android:textStyle">bold</item>
    </style>
</resources>
```

(در حالی که می‌توانستیم ویژگی android:textStyle را مستقیما به استایل BeatBoxButton اضافه کنیم، BeatBoxButton.Strong را برای نشان دادن وراثت استایل ایجاد کردیم.)

قرارداد نامگذاری در اینجا کمی عجیب است. انتخاب نام استایل BeatBoxButton.Strong به معنی ارث بردن تم شما از BeatBoxButton است.

همچنین روش جایگزین برای انجام این عمل وجود دارد. بدین طریق که هنگام اعلام استایل می‌توان یک والد مشخص کرد:

```
<style name="BeatBoxButton">
    <item name="android:background">@color/dark_blue</item>
</style>
<style name="StrongBeatBoxButton"
    parent="@style/BeatBoxButton">
    <item name="android:textStyle">bold</item>
</style>
```

res/layout/list_item_sound.xml را به روز کنید تا از استایل جدید خود استفاده کنید.

لیست ۲۰.۵ استفاده از استایل پررنگ تر (res/layout/list_item_sound.xml)

```
<Button
    style="@style/BeatBoxButton"
```

```
style="@style/BeatBoxButton.Strong"
android:layout_width="match_parent"
android:layout_height="120dp"
tools:text="Sound name"
android:onClick="@{() -> viewModel.onButtonClicked()}"
android:text="@{viewModel.title}"/>
```

BeatBox را اجرا و شاهد دکمه‌ها با متن پررنگ باشید.

تم

استایل‌ها جالب هستند. آنها به ما این امکان را می‌دهند که مجموعه‌ای از ویژگی‌ها را تعریف و سپس آنها را روی هر تعداد ویجت اعمال کنیم. نقطه ضعف استایل‌ها این است که شما باید آنها را برای هر ویجت، یکی یکی اعمال کنید. اگر برنامه پیچیده‌تری با دکمه‌های زیاد در لیوت‌های زیاد داشتید چه؟ افزودن استایل BeatBoxButton به همه آنها می‌تواند کار بزرگی باشد.

اینجاست که تم‌ها وارد می‌شوند. تم‌ها استایل‌ها را یک قدم جلوتر می‌برند: آنها امکان تعریف مجموعه‌ای از ویژگی‌ها را مانند یک استایل فراهم می‌آورند، اما این ویژگی‌ها به طور خودکار در سراسر برنامه اعمال می‌شوند. ویژگی‌های تم می‌توانند ارجاعی به منابع عینی مانند رنگ‌ها یا ارجاعی به استایل‌ها را ذخیره کنند. در یک تم می‌توانیم به عنوان مثال بگوییم "من می‌خواهم همه دکمه‌ها از این استایل استفاده کنند." با وجود تم‌ها نیازی نیست که هر ویجت دکمه را پیدا کرده و به آن بگوییم که از کدام استایل استفاده کند.

هنگامی که BeatBox را ایجاد کردیم، یک تم پیش‌فرض به آن داده شد.

به manifests/AndroidManifest.xml رفته و به ویژگی theme تگ application نگاه کنید:

```
<manifest xmlns:android=
"http://schemas.android.com/apk/res/android">
    <application
        ...
        android:supportsRtl="true"
        android:theme="@style/Theme.BeatBox">
        <activity
            ...
        </activity>
    </application>
</manifest>
```

ویژگی theme به تمی به نام Theme.BeatBox اشاره می‌کند. Theme.BeatBox در themes.xml پیاده‌سازی شده است.

همانطور که می‌بینید، تم نیز یک استایل است. اما تم‌ها ویژگی‌های متفاوتی را نسبت به یک استایل مشخص می‌کنند (همانطور که خواهید دید). تم‌ها نیز با اعلام در مانیفست کاربردی می‌شوند. این کار باعث اعمال خودکار تم در کل برنامه اعمال شود.

با Ctrl+click بر روی Theme.BeatBox اندروید استودیو شما را به res/values/themes.xml و به تعریف تم Theme.BeatBox می‌برد.

هنگامی که پروژه جدید ایجاد می شود یک تم پیش‌فرض به پروژه داده می‌شود. Theme.BeatBox ویژگی‌هایی را از Theme.Material3.DayNight.NoActionBar به ارث می‌برد.
Theme.Material3.DayNight.NoActionBar یک تم آماده Material Design 3 (Material You) هست که گوگل برای پروژه‌های مدرن اندروید فراهم آورده است که اجزای آن عبارتند از:

- Theme.Material3: یعنی از نسل سوم متریال (Material You) استفاده می‌کنیم. این نسخه رنگ‌ها را داینامیک از والپیپر کاربر می‌گیرد و کامپوننت‌های جدید مثل FilledButton، NavigationBar مدرن و ... رو پشتیبانی می‌کند.
- DayNight: یعنی اپلیکیشن به‌طور خودکار بسته به تنظیمات سیستم بین حالت روشن و تاریک سوییچ می‌کند.
- NoActionBar: یعنی این تم هیچ ActionBar پیش‌فرضی ندارد. ما همانطور که قبلا انجام دادیم باید خودمان یک Toolbar یا App Bar اضافه کنیم و کنترل کامل روی ظاهرش در دست ماست.

Material You که با نام Material Design 3 (M3) نیز شناخته می‌شود، نسل سوم چارچوب طراحی رابط کاربری گوگل است که نخستین بار در اندروید ۱۲ معرفی شد. این سیستم طراحی با هدف ایجاد تجربه کاربری شخصی‌سازی شده، تطبیقی و هماهنگ با محیط کاربر توسعه یافته است.

Material You یک تحول بنیادین در طراحی رابط کاربری اندروید محسوب می‌شود که با بهره‌گیری از رنگ‌های داینامیک، تایپوگرافی ساختاریافته و سیستم شکل استاندارد، تجربه‌ای شخصی‌سازی شده و سازگار با دستگاه‌های مدرن ارائه می‌دهد. این رویکرد، گامی مهم در جهت افزایش انعطاف‌پذیری و کاربرمحوری در طراحی نرم‌افزارهای موبایل است.

برای اینکه کاملاً با Material3 هماهنگ باشیم، لازم است دکمه های خود را به نوع MaterialButton تغییر دهیم تا اینطوری از رنگ‌ها و حالت‌های جدید (Filled, Tonal, Outlined) استفاده نماییم. برای اینکار res/layout/list_item_sound.xml را بروز کنید تا از MaterialButton استفاده کند.

لیست ۲۰۶ استفاده از MaterialButton (res/layout/list_item_sound.xml)

```
<Button com.google.android.material.button.MaterialButton
    style="@style/BeatBoxButton.Strong"
    android:layout_width="match_parent"
    android:layout_height="120dp"
    tools:text="Sound name"
    android:onClick="@{() -> viewModel.onButtonClicked()}"
    android:text="@{viewModel.title}"/>
```

البته تغییر دکمه ها موردی صلیقه‌ای است. دکمه های ساده هم کار خواهند کرد اما ظاهرشان متریال را نخواهد گرفت. یعنی رنگ‌ها و استایل‌های Material3 اعمال نخواهند شد.

بازنویسی ویژگی‌های تم

می‌توان مقادیری را به تم والد اضافه و یا مقادیر موجود آن را بازنویسی کرد. اکنون وقت آن است که پیش رفته و ببینیم چه ویژگی‌های وجود دارد که می‌توان آن را بازنویسی کرد. البته این کار، کار سختی است. مستندات کمی در مورد اینکه کدام ویژگی‌ها را می‌توان بازنویسی کرد، وجود دارد. نگران نباشید ما در کنار شما هستیم. خوش به حالتان که این کتاب را دارید تا راهنمای شما باشد.

اولین هدف تغییر رنگ پس زمینه است. می‌توانیم با رفتن به res/layout/activity_main.xml و تنظیم دستی ویژگی android:background ویو RecyclerView – و سپس انجام این فرآیند در هر فرگمنت و فایل لیوت اکتیویتی دیگری که ممکن است وجود داشته باشد — این کار را انجام دهیم. البته این کار بیهوده خواهد بود. بدیهی است که وقت خودمان را تلف می‌کنیم. تم مسئولیت تنظیم رنگ پس زمینه را بر عهده دارد. این کار را باید به آن سپرد.

حرکت به سمت ریشه تم

در عوض، ویژگی رنگ پس زمینه را روی تم بازنویسی می‌کنیم. برای کشف نام این ویژگی باید نگاهی به چگونگی تنظیم این ویژگی توسط تم والد – Theme.Material3.DayNight.NoActionBar – بیاندازیم.

ممکن است فکر کنید، "چگونه می‌توانم بفهمم که کدام ویژگی را بازنویسی کنم در حالی که نام آن را نمی‌دانم؟" ما نام ویژگی‌ها را تا زمانی که باعث شود فکر کنیم "این یکی درست به نظر می‌رسد." می‌خوانیم. سپس آن ویژگی را بازنویسی می‌کنیم، برنامه را اجرا و امیدوار هستیم که عاقلانه انتخاب کرده باشیم.

کاری که در پیش داریم جستجوی اجداد تم خود است. برای انجام این کار تا زمانی که یک ویژگی مناسب پیدا کنیم به سمت تم والد یکی پس از دیگری حرکت خواهیم کرد.

فایل themes.xml خود را باز کنید.

Theme.BeatBox ویژگی‌ها را از Base.Theme.BeatBox به ارث می‌برد. جالب اینجاست که Theme.BeatBox خود هیچ ویژگی را باز نویسی نمی‌کند. فقط و فقط به والد خود اشاره می‌کند.

روی Base.Theme.BeatBox کلید Ctrl+click بزنید. اندروید استودیو به شما می‌گوید که این تم یک توصیفگر منبع است. چند دو نسخه مختلف از این تم وجود دارد.

values/themes.xml را انتخاب کنید (شکل ۲۰.۳) تا به تعریف Base.Theme.BeatBox برسید.

تصویر ۲۰.۳ انتخاب والد




```
<resources xmlns:tools="http://schemas.android.com/tools">
    <!-- Base application theme. -->
    <style name="Base.Theme.BeatBox" parent="Theme.Material3.DayNight.NoActionBar">
        <!-- Customize yo
        <!-- <item name="
    </style>

    <style name="Theme.BeatBox" parent="Base.Theme.BeatBox" />
    <style name="BeatBoxButton">
    <item
        name="android:backgroundTint">@color/dark_blue</item>
    </style>
    <style name="BeatBoxButton.Strong">
        <item name="android:textStyle">bold</item>
    </style>
</resources>
```

Base.Theme.BeatBox تم دیگری است که هیچ ویژگی‌ای را بازنویسی نمی‌کند. با تم اصلی آن ادامه دهید: Theme.Material3.DayNight.NoActionBar.

نزدیک شدیم. Theme.Material3.Light.NoActionBar را ادامه دهید.

سپس Base.Theme.Material3.Light و بعد از آن Base.V14.Theme.Material3.Light به نظر می‌رسد رسیدیم. لیست ویژگی‌ها را در Base.V14.Theme.Material3.Light اسکن کنید. مشخصه‌ای برای تغییر رنگ پس زمینه مشاهده می‌شود: windowBackground. به نظر می‌رسد که این ویژگی پس زمینه تم باشد.

```
<item name="android:windowBackground">
    ?android:attr/colorBackground
</item>
```

این ویژگی است که در BeatBox بازنویسی می‌کنیم. به فایل themes.xml برگردید و ویژگی windowBackground را بازنویسی کنید.

لیست ۲۰.۷ تنظیم پس زمینه (res/values/styles.xml)

```
<resources xmlns:tools="http://schemas.android.com/tools">
    …
    <style name="Base.Theme.BeatBox"
    parent="Theme.Material3.DayNight.NoActionBar">
        …
        <item name="android:windowBackground">
        @color/soothing_blue</item>
    </style>
    …
</resources>
```

توجه داشته باشید که هنگام بازنویسی این ویژگی باید از فضای نام اندروید استفاده کنید، زیرا windowBackground در سیستم عامل اندروید اعلام شده است.

BeatBox را اجرا کنید تا آبی آرامش بخش را مانند شکل ۲۰.۴ مشاهده نمایید.

تصویر ۲۰.۴ BeatBox با پس زمینه



مراحلی که به تازگی برای یافتن ویژگی windowBackground طی کرده‌اید، همان مراحلی است که هر توسعه دهنده اندروید هنگام تغییر تم یک برنامه انجام می‌دهد. در سلسله مراتب تم تا زمانی که به تم ریشه Theme.BeatBox برسیم، پیمایش کردیم.

توجه داشته باشید که این سلسله مراتب تم ممکن است در طول زمان تغییر کند. اما پیمایش سلسله مراتب همیشه به این شکل خواهند ماند.

اصلاح ویژگی‌های دکمه

پیش از این، دکمه‌ها را با تنظیم دستی یک ویژگی style در فایل res/layout/list_item_sound.xml سفارشی کردیم. اگر برنامه پیچیده‌تری با دکمه‌هایی بسیار در اکتیویتی‌ها یا فرگمنت‌ها مختلف داشته باشیم، تنظیم ویژگی style در هر دکمه کار بسیار سخت و حتی غیرممکن است. این موضوع را می‌توان با تعریف استایلی در تم برای دکمه‌ها حل نمود.

قبل از افزودن یک استایل دکمه به تم خود، ویژگی style را از list_item_sound.xml حذف کنید.

لیست ۲۰.۸ ما راه بهتری داریم (res/layout/list_item_sound.xml)

```
<com.google.android.material.button.MaterialButton
    style="@style/BeatBoxButton.Strong"
    android:layout_width="match_parent"
    android:layout_height="120dp"
    tools:text="Sound name"
    android:onClick="@{() -> viewModel.onButtonClicked()}"
    android:text="@{viewModel.title}"/>
```

BeatBox را دوباره اجرا و مطمئن شوید که دکمه‌ها به ظاهر قدیمی و ملایم بازگشته‌اند (شکل ۲۰.۵).

تصویر ۲۰.۵ BeatBox با دکمه‌های قدیمی‌و ملایم.



دوباره به پیمایش تم‌ها بروید. این بار به دنبال materialButtonStyle بگردید. به احتمال زیاد آن را در Base.V14.Theme.Material3.Light پیدا خواهید کرد.

```
<item name="materialButtonStyle">
    @style/Widget.Material3.Button
</item>
```

materialButtonStyle استایل هر دکمه متریال در برنامه شما را مشخص می‌کند. ویژگی materialButtonStyle به جای یک مقدار به یک منبع استایل اشاره می‌کند. هنگام تنظیم ویژگی windowBackground، مقدار color را ارسال کردیم. در مورد materialButtonStyle باید استایل ارسال شود. برای دیدن استایل دکمه به Widget.Material3.Button بروید.

Widget.Material3.Button دوباره استایل‌هایی را تعریف می‌کند. برای دیدن ویژگی‌ها به والد آن یعنی Widget.MaterialComponents.Button، سپس والد بعدی یعنی Widget.AppCompat.Button بروید. به هر دکمه‌ای که در BeatBox استفاده می‌کنیم این ویژگی‌ها داده می‌شود.

آنچه را که در تم خود اندروید در BeatBox اتفاق می‌افتد، تکرار کنید. والد BeatBoxButton را تغییر دهید تا از استایل موجود دکمه ارث‌بری داشته باشد (فهرست ۲۱.۱۰). همچنین استایل BeatBoxButton.Strong خود را حذف کنید.

لیست ۲۰.۹ ایجاد یک استایل دکمه (res/values/styles.xml)

```
<resources
xmlns:tools="http://schemas.android.com/tools">
    …
    <style name="Theme.BeatBox"
        parent="Base.Theme.BeatBox" />
    <style name="BeatBoxButton"
        parent="Widget.AppCompat.Button">
        <item name=
        "android:background">@color/~~dark_blue~~ red</item>
    </style>
    ~~<style name="BeatBoxButton.Strong">~~
        ~~<item name="android:textStyle">bold</item>~~
    ~~</style>~~
</resources>
```

ما Widget.AppCompat.Button را به عنوان والد مشخص کردیم. قصد داریم دکمه‌های ما تمام ویژگی‌هایی را که یک دکمه معمولی دریافت می‌کند به ارث ببرد و سپس به طور انتخابی ویژگی‌ها را تغییر دهیم.

اگر تم والد را برای BeatBoxButton مشخص نکنیم، دکمه‌های به چیزی تبدیل می‌شوند که اصلا شبیه دکمه نیست. ویژگی‌هایی که انتظار داریم، مانند متنی که در مرکز دکمه قرار دارد، از بین خواهند رفت.

اکنون که BeatBoxButton را به طور کامل تعریف کردیم، زمان استفاده از آن فرا رسیده است. به ویژگی materialButtonStyle که قبلا هنگام کاوش در تم‌های اندروید پیدا کرده‌اید، نگاه کنید. این ویژگی را در لیوت زمینه خود تکثیر کنید.

لیست ۲۰.۱۰ – استفاده از استایل BeatBoxButton (res/values/styles.xml)

```
<resources>
    <style name="Base.Theme.BeatBox"
    parent="Theme.Material3.DayNight.NoActionBar">
        <item name="android:windowBackground">
        @color/soothing_blue</item>
        <item name="materialButtonStyle">
        @style/BeatBoxButton</item>
    </style>
    <style name="Theme.BeatBox" parent="Base.Theme.BeatBox"/>
    <style name="BeatBoxButton"
```

```
    parent="Widget.AppCompat.Button">
        <item name="android:background">@color/red</item>
    </style>
</resources>
```

توجه داشته باشید که هنگام تعریف materialButtonStyle از پیشوند :android استفاده نمی‌کنیم. به این دلیل است که materialButtonStyle که در حال بازنویسی هستیم در کتابخانه AppCompat پیاده‌سازی می‌شود. اکنون ویژگی materialButtonStyle را بازنویسی کرده و استایل خود را جایگزین می‌کنیم: BeatBoxButton.

BeatBox را اجرا کنید. متوجه خواهید شد که همه دکمه‌های شما قرمز هستند (تصویر ۲۱.۹). شما ظاهر هر دکمه معمولی در BeatBox را بدون تغییر مستقیم فایل‌های لیوت تغییر دادید. قدرت تم‌ها را در اندروید می‌بینید!

تصویر ۲۰.۶ BeatBox کاملا تم داده شده



مطمئنا اگر دکمه‌ها دکمه‌های واضح تری باشند خوب خواهد بود. در فصل بعدی، این مشکل را برطرف خواهیم کرد.

بیشتر بدانیم: اطلاعات بیشتر در مورد وراثت استایل

توصیف وراثت استایل در اوایل این فصل داستان کاملی نیست. ممکن است هنگام کاوش در سلسله مراتب تم، متوجه تغییر در استایل وراثت شده باشید. تم‌های AppCompat از نام تم برای نشان دادن وراثت استفاده می‌کردند تا زمانی که به تم Platform.AppCompat رسیدیم.

```
<style name="Platform.AppCompat" parent="android:Theme.Holo">
    ...
</style>
```

در اینجا، استایل نامگذاری وراثت به استایل ویژگی parent صریح تغییر می‌کند. چرا؟

مشخص کردن تم والد در نام تم فقط برای تم‌هایی که در یک پکیج وجود دارند، ممکن است. بنابراین خواهید دید که تم‌های سیستم عامل اندروید بیشتر اوقات از استایل وراثت نام تم استفاده می‌کنند. همچنین خواهید دید که کتابخانه AppCompat نیز همین کار را انجام می‌دهد. اما هنگامی که کتابخانه AppCompat به یک والد خارج از خود می‌رسد از ویژگی parent صریح استفاده می‌شود.

ایده خوبی است در برنامه‌های خود از همان قرارداد پیروی کنیم.

بیشتر بدانیم: دسترسی به ویژگی‌های تم

هنگامی که ویژگی‌ها در تم اعلام شدند، می‌توان به صورت XML یا به صورت کد به آنها دسترسی داشته باشید. هنگام ارجاع به یک مقدار در XML مانند یک رنگ از نماد @ استفاده می‌کنیم. به عنوان مثال @color/gray به یک منبع خاص اشاره می‌کند.

هنگام ارجاع به یک منبع در تم، از نماد ؟ استفاده می‌کنید:

```
<Button
    xmlns:android=
    "http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/list_item_sound_button"
    android:layout_width="match_parent"
    android:layout_height="120dp"
    android:background="?attr/colorAccent"
    tools:text="Sound name"/>
```

نماد ؟ می‌گوید از منبعی استفاده کن که ویژگی colorAccent در تم به آن اشاره دارد.

همچنین می‌توانید از ویژگی‌های تم در کد استفاده کنید، اگرچه کار بسیار بیشتری می‌برد.

```
val theme: Resources.Theme = activity.theme
val attrsToFetch = intArrayOf(R.attr.colorAccent)
val a: TypedArray = theme.obtainStyledAttributes
    (R.style.AppTheme, attrsToFetch)
val accentColor = a.getInt(0, 0)
a.recycle()
```

در خط اول تم برنامه گرفته و در شی theme ذخیره می‌شود. در خط بعدی از میان آرایه‌ای از ویژگی‌ها بدست آمده، ویژگی colorAccent در attrsToFetch ذخیره می‌شود. سپس با استفاده از تابع obtainStyledAttributes() ویژگی colorAccent مربوط به تم AppTheme گرفته و سپس مقدار رنگ به عنوان یک عدد صحیح در متغییر accentColor ذخیره می‌شود و با دستور a.recycle() آرایه آزاد می‌شود.

حال می‌توانیم از آن رنگ برای تغییر پس زمینه یک دکمه استفاده کنید.

tool bar و دکمه‌ها در BeatBox دقیقا این کار را انجام می‌دهند تا خود را بر اساس ویژگی‌های تم شما استایل دهند.

۲۱

Drawableهای XML

اکنون که کار تم BeatBox به پایان رسیده، وقت آن است که کاری برای دکمه‌ها انجام دهیم.

در حال حاضر دکمه‌ها وقتی فشرده می‌شوند هیچ بازخوردی ندارند و تنها شبیه به جعبه‌های قرمز به نظر می‌رسند. در این فصل از drawableهای XML برای ارتقای BeatBox به سطح بعدی استفاده خواهیم کرد (تصویر ۲۱.۱).

شکل ۲۱.۱ تغییر BeatBox



اندروید هر چیزی را که قرار است روی صفحه ترسیم شود را "drawable" می‌نامد. خواه یک شکل انتزاعی، یک بیت کد هوشمندانه که کلاس Drawable را زیر کلاس می‌کند، یا تصویر بیت مپ باشد. در این فصل drawableهای shape، state list و layer list را مشاهده خواهید کرد. هر سه این موارد

در فایل‌های XML تعریف می‌شوند، بنابراین ما آنها را در دسته drawableهای XML گروه بندی می‌کنیم.

ساخت دکمه‌های یکنواخت

قبل از ایجاد هر لیوت XML، res/layout/list_item_sound.xml را برای تغییرات مربوط به دکمه‌های آماده می‌کنیم.

لیست ۲۱.۱ فاصله دکمه‌ها (res/layout/list_item_sound.xml)

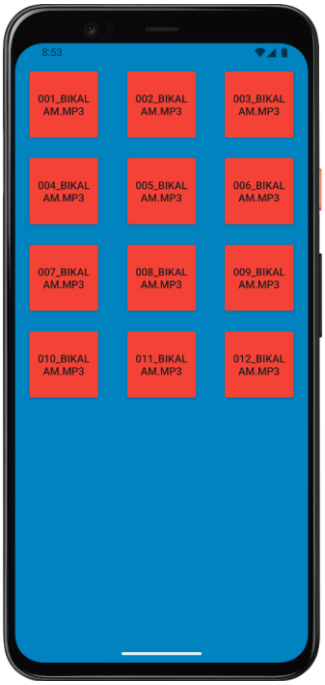
```
<?xml version="1.0" encoding="utf-8"?>
<layout
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">
    <data>
        <variable
            name="viewModel"
            type="com.codeb.beatbox.SoundViewModel"/>
    </data>
    <FrameLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_margin="8dp">
        <com.google.android.material.button.MaterialButton
            android:layout_width="match_parent"
            android:layout_height="120dp"
            android:layout_width="100dp"
            android:layout_height="100dp"
            android:layout_gravity="center"
            tools:text="Sound name"
            android:onClick=
                "@{() -> viewModel.onButtonClicked()}"
            android:text="@{viewModel.title}"/>
    </FrameLayout>
</layout>
```

به هر دکمه عرض و ارتفاع ۱۰۰dp دادیم تا بعدا وقتی که دکمه‌ها دایره‌ای شکل می‌شوند، جا داشته باشند.

recycler view صرف نظر از اندازه صفحه نمایش همیشه سه ستون را نشان می‌دهد. اگر فضای اضافی وجود داشته باشد، recycler view ستون‌ها را متناسب با دستگاه کش می‌دهد. نمی‌خواهیم recycler view دکمه‌های را بکشد، به همین خاطر باید آنها را در یک frame layout قرار دهیم. frame layout خودش کشیده می‌شود اما دکمه‌ها به همان صورت باقی می‌مانند.

BeatBox را اجرا کنید. خواهید دید که دکمه‌های شما همگی به یک اندازه هستند و مقداری فاصله بین آنها وجود دارد (تصویر ۲۱.۲).

تصویر ۲۱.۲ دکمه‌های فاصله دار



Drawableهای Shape

اکنون دکمه‌های خود را با Drawableهای Shape گرد می‌کنیم.

از آنجایی که drawableهای XML محدود به چگالی نیستند، در پوشه پیش‌فرض drawable قرار می‌گیرند.

در پنجره ابزار پروژه یک فایل جدید در res/drawable به نام button_beat_box_normal.xml (لیست ۲۱.۲) ایجاد کنید.

لیست ۲۱.۲ ساخت یک شکل گرد (res/drawable/button_beat_box_normal.xml)

```
<?xml version="1.0" encoding="utf-8"?>
<shape
xmlns:android="http://schemas.android.com/apk/res/android"
    android:shape="oval">
    <solid android:color="@color/dark_blue"/>
</shape>
```

این فایل یک شکل بیضی پر شده با رنگ آبی تیره ایجاد می‌کند. گزینه‌های از جمله rectangles برای ترسیم مستطیل، lines برای ترسیم خطوط و همچنین gradients برای ترسیم گرادیان‌ها، در دسترس هستند. button_beat_box_normal را به عنوان پس زمینه دکمه‌های خود اعمال کنید.

لیست ۲۱.۳ تغییر پس زمینه (res/values/styles.xml)

```
<resources>
    <style name="Base.Theme.BeatBox"
    parent="Theme.Material3.DayNight.NoActionBar">
        ...
    </item>
    </style>
    <style name="Theme.BeatBox" parent="Base.Theme.BeatBox"/>
    <style name="BeatBoxButton"
        parent="Widget.AppCompat.Button">
        <item name="android:background">@color/red</item>
        <item name="android:background">
        @drawable/button_beat_box_normal</item>
    </style>
</resources>
```

BeatBox را اجرا کنید. دکمه‌های شما اکنون دایره‌ای شکل هستند (شکل ۲۱.۳).

شکل ۲۱.۳ دکمه‌های دایره‌ای



در صورت لمس دکمه‌ای صدا مربوطه پخش می‌شود، اما دکمه ظاهر خود را تغییر نمی‌دهد. بهتر است دکمه پس از لمس متفاوت به نظر برسد.

Drawableهای State List

برای رفع این مشکل، یک شکل جدید که برای حالت فشرده دکمه استفاده می‌شود، تعریف کنید. button_beat_box_pressed.xml را در res/drawable ایجاد کنید. مانند نسخه معمولی اما با رنگ پس زمینه قرمز باشد.

لیست ۲۱.۴ تعریف شکل(res/drawable/button_beat_box_pressed.xml)

```
<?xml version="1.0" encoding="utf-8"?>
<shape
   xmlns:android="http://schemas.android.com/apk/res/android"
    android:shape="oval">
    <solid android:color="@color/red"/>
</shape>
```

در مرحله بعد، زمانی که کاربر دکمه را لمس می‌کند، از این نسخه استفاده خواهیم کرد. برای انجام این کار، از یک state list استفاده می‌کنیم.

state list یک drawable است که بر اساس وضعیت چیزی به سایر drawableها اشاره می‌کند. یک دکمه دارای حالت فشرده و فشرده نشده است. از یک state list برای مشخص کردن drawable پس زمینه دکمه به هنگام فشرده شدن و مشخص کردن drawable دیگری به هنگام فشرده نشدن، استفاده خواهیم کرد.

یک state list در یک فایل جدید در پوشه res/drawable تعریف کنید.

لیست ۲۱.۵ ایجاد یک state list (res/drawable/button_beat_box.xml)

```
<selector
xmlns:android="http://schemas.android.com/apk/res/android">
    <item android:drawable=
    "@drawable/button_beat_box_pressed"
        android:state_pressed="true"/>
    <item android:drawable=
    "@drawable/button_beat_box_normal" />
</selector>
```

اکنون استایل دکمه خود را تغییر دهید تا از state list به عنوان پس زمینه دکمه استفاده کند.

لیست ۲۱.۶ اعمال یک state list (res/values/styles.xml)

```
<style name="BeatBoxButton" parent="Widget.AppCompat.Button">
    <item name="android:background">
    @drawable/button_beat_box_normal</item>
    <item name="android:background">
```

```
    @drawable/button_beat_box</item>
</style>
```

هنگامی که دکمه در حالت فشرده قرار دارد، از button_beat_box_pressed به عنوان پس زمینه استفاده می‌شود. در غیر این صورت، پس زمینه دکمه button_beat_box_normal خواهد بود. BeatBox را اجرا و یک دکمه را لمس کنید. پس زمینه دکمه تغییر می‌کند (شکل ۲۲.۴).

تصویر ۲۱.۴ BeatBox اکنون با حالت دکمه‌ای که فشار داده شده است



State listها یک ابزار سفارشی سازی مفید هستند که از بسیاری از stateهای دیگر از جمله disabled، focused و activated پشتیبانی می‌کنند.

Drawableهای Layer List

BeatBox ظاهر خوبی دارد. اکنون دکمه‌های گرد و به صورت بصری به فشار دادن پاسخ می‌دهند. زمان چیزی کمی پیشرفته‌تر است.

Layer list امکان ترکیب دو Drawable را در یکی فراهم می‌آورد. با استفاده از این ابزار، زمانی که دکمه در حالت فشرده قرار دارد، یک حلقه تیره در اطراف دکمه اضافه خواهیم کرد.

لیست ۲۱.۷ استفاده از یک Layer list (res/drawable/button_beat_box_pressed.xml)

```
<layer-list
xmlns:android="http://schemas.android.com/apk/res/android">
    <item>
        <shape
         xmlns:android=
         "http://schemas.android.com/apk/res/android"
            android:shape="oval">
            <solid android:color="@color/red"/>
        </shape>
    </item>
    <item>
        <shape android:shape="oval">
            <stroke android:width="4dp"
                android:color="@color/dark_red"/>
        </shape>
    </item>
</layer-list>
```

دو drawable را در این layer list مشخص کردیم. اولی یک دایره قرمز است، همانطور که قبل از این تغییر بود. دومین drawable در بالای اولی کشیده می‌شود که یک بیضی با stroke عرض dp۴ است. این کار یک حلقه قرمز تیره ایجاد می‌کند.

این دو drawable با هم ترکیب و layer list را تشکیل می‌دهند. می‌توان بیش از دو drawable را در یک layer list ترکیب و چیز پیچیده‌تری را به وجود آورد.

BeatBox را اجرا کنید و یک دکمه را فشار دهید. یک حلقه زیبا در اطراف آن خواهید دید (شکل ۲۲.۵).

شکل ۲۱.۵ BeatBox کامل شد



با افزودن layer list، BeatBox اکنون کامل شده است. به یاد دارید که BeatBox قبلا چقدر ساده به نظر می‌رسید؟ اکنون چیزی خاص و منحصر به فرد دارید.

بیشتر بدانیم: چرا با Drawableهای XML به خود زحمت بدیم؟

ما همیشه یک حالت فشرده برای دکمه‌های خود می‌خواهیم، بنابراین state list جزء حیاتی هر برنامه اندرویدی است. اما در مورد shape و layer list چطور؟ آیا باید از آنها استفاده کنیم.

drawableهای XML انعطاف پذیر هستند. می‌توان از آنها برای اهداف زیادی استفاده کرد، همچنین به راحتی آنها را در آینده می‌توانیم به روز کنیم. با ترکیبی از layer list و shape می‌توان پس زمینه‌های پیچیده‌ای را بدون استفاده از ویرایشگر تصویر ایجاد کرد. اگر تصمیم به تغییر شِمای رنگ در BeatBox را دارید، به روز رسانی رنگ‌ها در یک drawable آسان است.

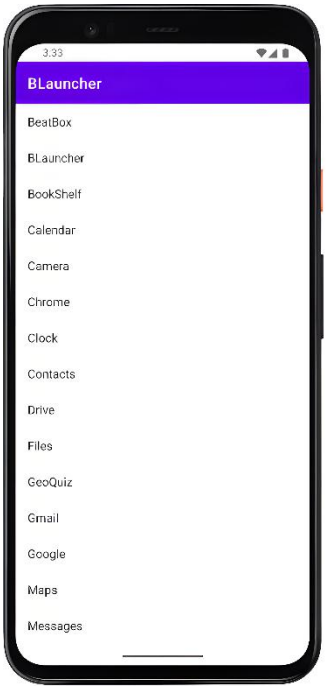
در این فصل ما drawableهای XML خود را در دایرکتوری drawable بدون هیچ توصیفگر منبعی برای تراکم صفحه، تعریف کردیم. دلیل انجام این کار استقلال drawableهای XML از تراکم صفحه است. یک عکس به عنوان پس زمینه معمولا چندین نسخه در تراکم‌های مختلف می‌خواهد تا در اکثر دستگاه‌ها وضوح خود را حفظ کند. drawableهای XML فقط باید یک بار تعریف شوند و در هر تراکم صفحه نمایش واضح به نظر می‌رسند.

۲۲

اطلاعات بیشتر در مورد اینتنت‌ها و Taskها

در این فصل، از اینتنت‌های ضمنی برای ایجاد یک برنامه لانچر برای جایگزینی برنامه لانچر پیش‌فرض اندروید استفاده خواهیم کرد. شکل ۲۲.۱ نشان می‌دهد که این برنامه چگونه خواهد بود.

شکل ۲۲.۱ محصول نهایی BLauncher



BLauncher لیستی از برنامه‌های دستگاه را نمایش می‌دهد. کاربر برای اجرای برنامه‌ای می‌تواند آن را از لیست انتخاب نماید.

این برنامه درک ما را از intentها، فیلترهای intent و تعامل اپلیکیشن‌ها در محیط اندروید عمیق‌تر خواهد کرد.

شروع BLauncher

یک پروژه با نام BLauncher ایجاد نمایید، اما این بار بدون اکتیویتی. هنگام ایجاد از گزینه No Activity استفاده نمایید.

پس از اینکه اندروید استودیو پروژه را مقداردهی اولیه کرد، یک اکتیویتی از نوع Empty Views Activity با نام اکتیویتی BLauncherActivity بسازید و کادر Launcher Activity و Generate a Layout File را علامت بزنید.

BLauncherActivity لیستی از نام برنامه‌ها را در RecyclerView نمایش خواهد داد. وابستگی recyclerview را به app/build.gradle اضافه کنید.

محتویات layout/activity_blauncher.xml را با RecyclerView نشان داده شده در لیست ۲۲.۱ جایگزین کنید.

لیست ۲۲.۱ – بروزرسانی لیوت BLauncherActivity (layout/activity_blauncher.xml)

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android=
        "http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".BLuncherActivity">
</androidx.constraintlayout.widget.ConstraintLayout>
```

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.recyclerview.widget.RecyclerView
xmlns:android="http://schemas.android.com/apk/res/android"
    android:id="@+id/app_recycler_view"
    android:layout_width="match_parent"
    android:layout_height="match_parent"/>
```

BLauncherActivity.kt را باز و یک ارجاع به شی RecyclerView را در یک متغییر ذخیره و همچنین شناسه main را اصلاح نمایید.

فهرست ۲۲.۲ پیاده‌سازی اولیه BLauncherActivity (BLauncherActivity.kt)

```
package com.codeb.bluncher
import android.os.Bundle
import androidx.activity.enableEdgeToEdge
```

```
import androidx.appcompat.app.AppCompatActivity
import androidx.core.view.ViewCompat
import androidx.core.view.WindowInsetsCompat
import androidx.recyclerview.widget.LinearLayoutManager
import androidx.recyclerview.widget.RecyclerView
class BLuncherActivity : AppCompatActivity() {
    private lateinit var recyclerView: RecyclerView
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContentView(R.layout.activity_bluncher)
        ViewCompat.setOnApplyWindowInsetsListener(
        findViewById(
        R.id.main app_recycler_view)) { v, insets ->
            val systemBars = insets.getInsets(
            WindowInsetsCompat.Type.systemBars())
            v.setPadding(systemBars.left, systemBars.top
            , systemBars.right, systemBars.bottom)
            insets
        }
        recyclerView = findViewById(R.id.app_recycler_view)
        recyclerView.layoutManager =
            LinearLayoutManager(this)
    }
}
```

برنامه خود را اجرا و مطمئن شوید که همه چیز به درستی تا این مرحله پیشرفته است. اگر چنین است، شما صاحب برنامه‌ای با عنوان BLauncher خواهید بود که یک RecyclerView خالی را نمایش می‌دهد (شکل ۲۲.۲).

شکل ۲۲.۲ شروع BLauncher



تعیین یک اینتنت ضمنی

BLauncher لیستی از برنامه‌های قابل اجرا در دستگاه را به کاربر نشان می‌دهد. (برنامه قابل اجرا برنامه‌ای است که کاربر می‌تواند با کلیک بر روی یک آیکون در صفحه اصلی یا صفحه برنامه‌ها باز کند.) برای انجام این کار روی سیستم برای بازیابی لیست اکتیویتی‌های اصلی قابل راه اندازی کوئری می‌زنیم.

PackageManager که در قبلا با آن آشنا شدیم، برای تعیین اکتیویتی‌ها استفاده می‌شود. اکتیویتی‌های اصلی قابل راه اندازی، با intent filterی که شامل یک اکشن MAIN و یک دسته LAUNCHER است، مشخص می‌شوند. این فیلتر اینتنت را در فایل manifests/AndroidManifest.xml در پروژه‌های قبلی خود دیدید:

```
<intent-filter>
    <action android:name="android.intent.action.MAIN"/>
    <category android:name=
        "android.intent.category.LAUNCHER"/>
</intent-filter>
```

هنگامی که BLauncherActivity را به یک اکتیویتی لانچر تبدیل کردیم، این فیلتر به طور خودکار برای برنامه ما اضافه شد. (اگر دوست دارید مانیفست را بررسی کنید.)

در BLauncherActivity.kt تابعی به نام setupAdapter() اضافه و آن را از onCreate(…) فراخوانی کنید. (این تابع یک نمونه RecyclerView.Adapter را تنظیم و آن را به RecyclerView متصل می‌کند. در حال حاضر فقط لیستی از داده‌های برنامه را ایجاد می‌کند.)

همچنین یک اینتنت ضمنی ایجاد و فهرستی از اکتیویتی‌هایی که با اینتنت مطابقت دارند را از فایل PackageManager بدست آورید؛ تعداد اکتیویتی‌هایی را که PackageManager برمی‌گرداند، Log کنید.

لیست ۲۲.۳ کوئری PackageManager (BLauncherActivity.kt)

```
private const val TAG = "BLauncherActivity"
class BLauncherActivity : AppCompatActivity() {
    private lateinit var recyclerView: RecyclerView
    override fun onCreate(savedInstanceState: Bundle?) {
        …
        recyclerView.layoutManager =
            LinearLayoutManager(this)
        setupAdapter()
    }
    private fun setupAdapter() {
        val startupIntent = Intent(Intent.ACTION_MAIN)
        .apply   {
            addCategory(Intent.CATEGORY_LAUNCHER)
        }
        val activities =
            packageManager.queryIntentActivities (
            startupIntent, 0)
        Log.i(TAG, "Found ${activities.size} activities")
    }
}
```

در اینجا یک اینتنت ضمنی با اکشن ACTION_MAIN ایجاد می‌کنیم.

همچنین CATEGORY_LAUNCHER را به دسته‌بندی اینتنت اضافه کردیم.

فراخوانی PackageManager.queryIntentActivities(Intent, Int) فهرستی حاوی ResolveInfo را برای همه اکتیویتی‌هایی که دارای فیلتر مطابق با اینتنت داده شده هستند، برمی‌گرداند. می‌توانیم فلگ‌هایی را برای تغییر نتایج مشخص کنیم.

به عنوان مثال فلگ PackageManager.GET_SHARED_LIBRARY_FILES باعث می‌شود که کوئری اطلاعات اضافی را شامل شود (مسیرهایی به کتابخانه‌هایی که با هر برنامه منطبق مرتبط هستند). در اینجا ۰ جهت عدم تغییر نتایج پاس می‌کنیم. BLauncher را اجرا و Logcat را بررسی کنید تا ببینید PackageManager چه تعداد برنامه را برگردانده است.

در BookShelf ما از یک اینتنت ضمنی برای ارسال گزارش کتاب استفاده کردیم. ما یک انتخابگر اکتیویتی را با ایجاد یک اینتنت ضمنی و قرار دادن یک اینتنت انتخابگر و ارسال آن به سیستم عامل با startActivity(Intent) ارائه کردیم:

```
val intent = Intent(Intent.ACTION_SEND)
... // Create and put intent extras
chooserIntent = Intent.createChooser(intent,
    getString(R.string.send_report)
startActivity(chooserIntent)
```

ممکن است تعجب کنید که چرا در اینجا از این روش استفاده نمی‌کنیم. توضیح کوتاه این است که فیلتر اینتنت MAIN/LAUNCHER ممکن است با اینتنت ضمنی MAIN/LAUNCHER که از طریق startActivity(Intent) ارسال می‌شود، مطابقت داشته باشد یا نداشته باشد.

startActivity(Intent) به این معنی نیست که "اکتیویتی منطبق با این اینتنت ضمنی را شروع کن". این بدان معناست که " اکتیویتی پیش‌فرض مطابق با این اینتنت ضمنی را شروع کن". هنگامی که یک اینتنت ضمنی را از طریق startActivity(Intent) فراخوانی می‌کنیم، سیستم عامل مخفیانه دسته Intent.CATEGORY_DEFAULT را به اینتنت اضافه می‌کند.

بنابراین اگر بخواهیم یک فیلتر اینتنت با اینتنت‌های ضمنی ارسال شده از طریق startActivity(Intent) مطابقت داشته باشد، باید دسته DEFAULT را در آن فیلتر اینتنت قرار دهید.

اکتیویتی که دارای فیلتر اینتنت MAIN/LAUNCHER است، نقطه ورود اصلی یک برنامه می‌باشد که به آن تعلق دارد. ما فقط نقطه ورود اصلی را برای برنامه‌ها می‌خواهیم و اهمیتی ندارد که نقطه ورود اصلی "پیش‌فرض" باشد. بنابراین لازم نیست دسته CATEGORY_DEFAULT را نیز شامل نماییم.

چون فیلترهای اینتنت MAIN/LAUNCHER ممکن است شامل CATEGORY_DEFAULT نباشند، نمی‌توانیم آنها را با یک اینتنت ضمنی ارسال شده از طریق startActivity(Intent) مطابقت دهیم. در عوض، از اینتنت برای کوئری مستقیم روی PackageManager برای اکتیویتی‌هایی با فیلتر اینتنت MAIN/LAUNCHER استفاده می‌کنیم.

مرحله بعدی نمایش برچسب‌های این اکتیویتی‌ها است. برچسب یک اکتیویتی، نام نمایشی آن است. با توجه به اینکه این اکتیویتی‌ها اکتیویتی‌های لانچر هستند، برچسب به احتمال زیاد نام برنامه است.

برچسب‌های اکتیویتی‌ها را به همراه سایر متاداده‌ها در آبجکت‌ها ResolveInfo برگردانده شده از PackageManager پیدا می‌کنیم.

ابتدا آبجکت‌ها ResolveInfo بازگردانده شده از PackageManager را بر اساس حروف الفبا با استفاده از متد ResolveInfo.loadLabel(PackageManager) ردیف کنید.

فهرست ۲۲.۴ مرتب سازی بر اساس حروف الفبا (BLauncherActivity.kt)

```
private fun setupAdapter() {
    val startupIntent = Intent(Intent.ACTION_MAIN).apply {
        addCategory(Intent.CATEGORY_LAUNCHER)
    }
    val activities =
        packageManager.queryIntentActivities(startupIntent,0)
    activities.sortWith(Comparator { a, b ->
        String.CASE_INSENSITIVE_ORDER.compare(
            a.loadLabel(packageManager).toString(),
            b.loadLabel(packageManager).toString()
        )
    })
    Log.i(TAG, "Found ${activities.size} activities")
}
```

اکنون یک ViewHolder را برای نمایش برچسب یک اکتیویتی تعریف کنید. ResolveInfo را در یک متغییر ذخیره کنید (بعدا بیش از یک بار از آن استفاده خواهید کرد).

لیست ۲۲.۵ پیاده‌سازی ViewHolder (BLauncherActivity.kt)

```
class BLauncherActivity : AppCompatActivity() {
    private lateinit var recyclerView: RecyclerView
    override fun onCreate(savedInstanceState: Bundle?) {
        …
    }
    private fun setupAdapter() {
        …
    }
    private class ActivityHolder(itemView: View) :
        RecyclerView.ViewHolder(itemView) {
        private val nameTextView = itemView as TextView
        private lateinit var resolveInfo: ResolveInfo
        fun bindActivity(resolveInfo: ResolveInfo) {
            this.resolveInfo = resolveInfo
            val packageManager = itemView.context
```

```
                .packageManager
            val appName =
                resolveInfo.loadLabel(
                    packageManager).toString()
            nameTextView.text = appName
        }
    }
}
```

بعد یک پیاده‌سازی RecyclerView.Adapter اضافه کنید.

لیست ۲۲.۶ پیاده‌سازی RecyclerView.Adapter

```
class BLauncherActivity : AppCompatActivity() {
    …
    private class ActivityHolder(itemView: View) :
        RecyclerView.ViewHolder(itemView) {
        …
    }
    private class ActivityAdapter(val activities:
        List<ResolveInfo>) :
        RecyclerView.Adapter<ActivityHolder>() {
        override fun onCreateViewHolder
        (container: ViewGroup, viewType: Int):
                ActivityHolder {
            val layoutInflater =
                LayoutInflater.from(container.context)
            val view = layoutInflater
                .inflate(android.R.layout.simple_list_item_1
                , container, false)
            return ActivityHolder(view)
        }
        override fun onBindViewHolder(holder: ActivityHolder
        , position: Int) {
            val resolveInfo = activities[position]
            holder.bindActivity(resolveInfo)
        }
        override fun getItemCount(): Int {
            return activities.size
        }
    }
}
```

در اینجا android.R.layout.simple_list_item_1 را در onCreateViewHolder(…) بارگذاری می‌کنیم. فایل لیوت simple_list_item_1 بخشی از فریمورک اندروید است، به همین دلیل است که از android.R.layout به جای R.layout استفاده می‌کنیم. این فایل شامل یک TextView است.

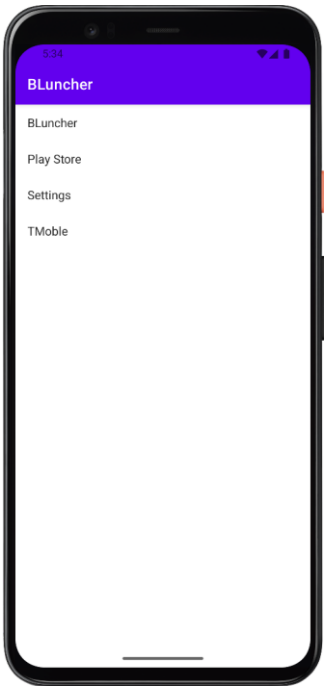
در آخر setupAdapter() را به روز کنید تا نمونه‌ای از ActivityAdapter ایجاد و آن را به عنوان آداپتور RecyclerView متصل کند.

لیست ۲۲.۷ تنظیم آداپتور RecyclerView (BLauncherActivity.kt)

```
private fun setupAdapter() {
    …
    Log.i(TAG, "Found ${activities.size} activities")
    recyclerView.adapter = ActivityAdapter(activities)
}
```

BLauncher را اجرا کنید. RecyclerView با برچسب‌های اکتیویتی پر شده است (شکل ۲۲.۳).

تصویر ۲۲.۳ تمام اکتیویتی‌های



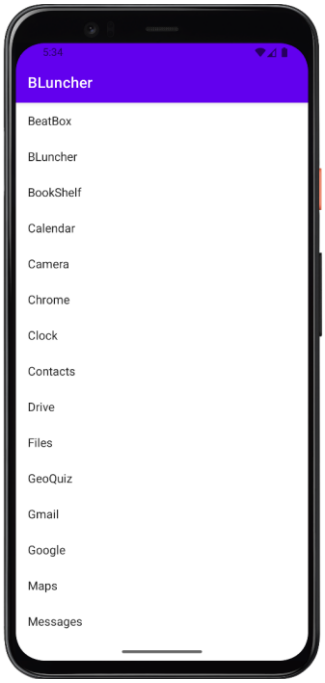
حتما به این پی برده‌اید که بسیاری از برنامه‌ها در لیست ما قرار ندارند. از Android 11 (API 30) به بعد، اندروید اجازه نمی‌دهد همه اَپ‌ها را ببینیم مگر اینکه صریحاً اجازه اخذ نماییم. برای این کار باید مانیفست را همانند لیست ۲۲.۸ به روز کنیم.

لیست ۲۲.۸ به روز رسانی مانیفست (Manifest.xml)

```
<manifest xmlns:android=
"http://schemas.android.com/apk/res/android">
    <queries>
        <intent>
            <action android:name=
                "android.intent.action.MAIN" />
            <category android:name=
                "android.intent.category.LAUNCHER" />
        </intent>
    </queries>
    <application
        ...
    </application>
</manifest>
```

دوباره برنامه را اجرا کنید این بار اکتیویتی های بیشتری نمایش داده می شود.

تصویر ۲۲.۴ تمام اکتیویتی‌های



ایجاد اینتنت‌های صریح در زمان اجرا

از یک اینتنت ضمنی برای جمع آوری اکتیویتی‌های مورد نظر و ارائه آنها در یک لیست استفاده کردیم. مرحله بعدی پاسخ به لمس یک مورد از لیست است. باید اکتیویتی انتخاب شده را اجرا کنیم. اجرای یک اکتیویتی با استفاده از یک اینتنت صریح انجام می‌شود.

برای ایجاد اینتنت صریح باید نام پکیج و نام کلاس اکتیویتی را از ResolveInfo دریافت کنیم. می‌توان این داده‌ها را از بخشی از ResolveInfo به نام ActivityInfo دریافت کرد (به صفحه مستندات مراجعه کنید).

ActivityHolder را برای پیاده‌سازی شنونده کلیک به روز کنید. هنگامی که اکتیویتی در لیست انتخاب می‌شود، از ActivityInfo برای ایجاد یک اینتنت صریح استفاده کنید. سپس از آن اینتنت صریح برای راه اندازی اکتیویتی استفاده کنید.

لیست ۲۲.۹ راه اندازی اکتیویتی لمس شده (BLauncherActivity.kt)

```
private class ActivityHolder(itemView: View) :
    RecyclerView.ViewHolder(itemView),
    View.OnClickListener {
    private val nameTextView = itemView as TextView
    private lateinit var resolveInfo: ResolveInfo
    init {
        nameTextView.setOnClickListener(this)
    }
    fun bindActivity(resolveInfo: ResolveInfo) {
        …
    }
    override fun onClick(view: View) {
        val activityInfo = resolveInfo.activityInfo
        val intent = Intent(Intent.ACTION_MAIN).apply {
            setClassName(activityInfo
            .applicationInfo.packageName,activityInfo.name)
        }
        val context = view.context
        context.startActivity(intent)
    }
}
```

توجه داشته باشید که در این اینتنت اکشنی را ارسال می‌کنیم. اکثر برنامه‌ها چه اکشن را اضافه کنیم و چه نکنیم رفتار یکسانی خواهند داشت. با این حال برخی ممکن است رفتار خود را تغییر دهند. یک اکتیویتی بسته به نحوه راه اندازی می‌تواند رابطهای مختلفی را نمایش دهد. به عنوان یک برنامه‌نویس بهتر است اینتنت های خود را به وضوح اعلام و اجازه دهیم اکتیویتی‌ها آنچه را که می‌خواهند انجام دهند.

در لیست ۲۲.۹ نام پکیج و نام کلاس را از متاداده دریافت می‌کنیم و از آن برای ایجاد یک اینتنت صریح استفاده می‌کنیم:

```
fun setClassName(packageName: String, className: String):
    Intent
```

این با نحوه ایجاد اینتنت‌ها صریح در گذشته متفاوت است. قبلا از سازنده Intent که Context و یک شی Class را می‌پذیرفت، استفاده می‌کردیم:

```
Intent(packageContext: Context, cls: Class<?>)
```

این سازنده از پارامترهای خود برای به دست آوردن آنچه که Intent واقعا نیاز دارد، یعنی یک ComponentName استفاده می‌کند. ComponentName یک نام پکیج و یک نام کلاس به هم چسبیده است. هنگامی که یک Activity و یک کلاس را برای ایجاد یک Intent ارسال می‌کنیم، سازنده نام پکیج کاملا واجد شرایط را از Activity تعیین می‌کند.

```
fun setComponent(component: ComponentName): Intent
```

با این حال استفاده از setClassName(…) کد کمتری دارد، که نام component را در پشت صحنه ایجاد می‌کند.

BLauncher را اجرا و برخی از برنامه‌ها را راه اندازی کنید.

تسک‌ها (Tasks) و پشته بازگشت (Back Stack)

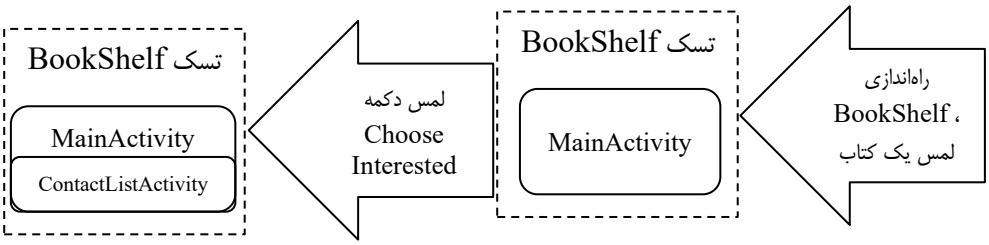
اندروید از تسک‌ها برای پیگیری وضعیت برنامه‌های در حال اجرا استفاده می‌کند. هر برنامه‌ای که از لانچر پیش‌فرض اندروید اجرا می‌شود، تسک خاص خود را دارد. این رفتار مطلوب مورد نظر است – اما متأسفانه برای BLauncher این گونه رفتار نمی‌کند. قبل از اینکه اپلیکیشن‌ها را مجبور به راه اندازی تسک‌هایشان کنیم، بیایید مطالبی در مورد تسک‌ها و نحوه عملکرد آنها بیان کنیم.

یک تسک پشته‌ای از اکتیویتی‌هایی است که کاربر متوجه آنهاست. اکتیویتی پایین پشته base activity نامیده می‌شود، و اکتیویتی بالای پشته، اکتیویتی است که کاربر می‌بیند. وقتی دکمه Back را فشار می‌دهیم، اکتیویتی بالایی را از پشته خارج می‌کنیم. اگر هنگام مشاهده base activity دکمه Back را فشار دهیم، به صفحه اصلی بر خواهیم گشت.

به طور پیش‌فرض، اکتیویتی‌های جدید در تسک فعلی راه اندازی می‌شوند. در BookShelf هر زمان که اکتیویتی جدیدی را شروع می‌کردیم، آن اکتیویتی به تسک فعلی اضافه می‌شد (همانطور که در شکل ۲۳.۴

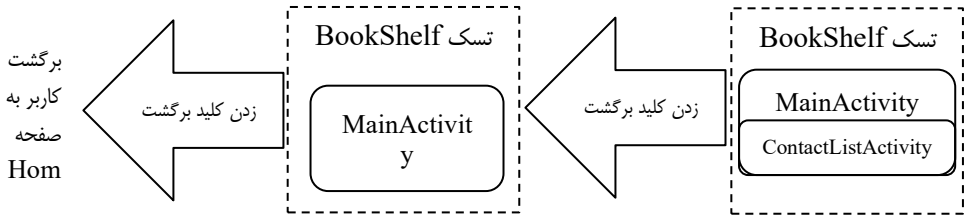
نشان داده شده است) – حتی اگر این اکتیویتی بخشی از برنامه BookShelf نبود، مانند زمانی که اکتیویتی را برای انتخاب مظنون راه اندازی می‌کردیم.

شکل ۲۲.۵ تسک BookShelf



مزیت افزودن یک اکتیویتی به تسک فعلی این است که کاربر می‌تواند به جای سلسله مراتب برنامه (همانطور که در شکل ۲۲.۶ نشان داده شده است) از طریق تسک حرکت کند.

تصویر ۲۲.۶ فشار دادن دکمه Back در BookShelf

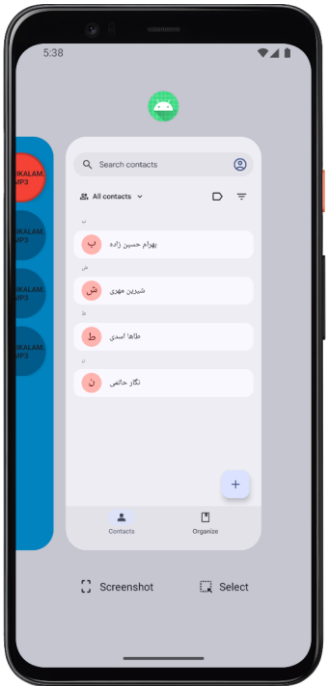


جابه جایی بین تسک‌ها

با استفاده از صفحه ویوی کلی می‌توانید بدون تأثیر بر وضعیت تسکها، بین آن‌ها جابجا شد. به عنوان مثال اگر شروع به وارد کردن یک مخاطب جدید کنید و سپس به بررسی توییتر خود بپردازید، دو تسک شروع خواهید کرد. حال اگر دوباره برای اتمام ویرایش مخاطب برگردید، موقعیت شما در هر دو تسک ذخیره می‌شود.

با استفاده از صفحه ویوی کلی در دستگاه یا شبیه‌ساز تسک‌ها را تغییر دهید. ابتدا BookShelf را از صفحه اصلی یا از لانچر خود راه اندازی کنید. کتابی را از فهرست کتاب انتخاب نمایید. سپس دکمه Home را فشار دهید تا به صفحه اصلی برگردید. در مرحله بعد، BeatBox را راه اندازی کنید. در نهایت، صفحه ویوی کلی را باز کنید (تصویر ۲۲.۷).

تصویر ۲۲.۷ صفحه ویوی کلی



ورودی نمایش داده شده برای هر برنامه (معروف به کارت) نشان دهنده تسک برنامه است. یک اسکرین‌شات از اکتیویتی در بالای پشته هر تسک نمایش داده می‌شود. می‌توانید کارت BeatBox یا BookShelf را برای بازگشت به آن لمس نمایید (و به هر اکتیویتی که در آن برنامه با آن تعامل داشتید).

می‌توانید با کشیدن کارت برنامه، تسک آن را پاک نمایید. پاک کردن تسک، تمام اکتیویتی‌ها را از پشته برنامه حذف می‌کند.

تسک BookShelf را حذف کنید، سپس برنامه را دوباره راه اندازی نمایید. به جای کتابی‌که قبل از پاک کردن کار ویرایش می‌کردید، لیست کتاب‌ها را خواهید دید.

شروع یک تسک جدید

در حال حاضر بعضی از اکتیویتی‌ها که از BLauncher شروع می‌شود، به تسک BLauncher اضافه می‌شود. علت اینکار در تعاریف مانیفست برنامه هاست. این قابلیت نیز از اندروید ۱۱ اضافه شده است.

می‌توان با پاک کردن تمام تسک‌های نمایش داده شده در صفحه ویوی کلی این موضوع را تأیید کنید.

ما می‌خواهیم BLauncher اکتیویتی‌ها را در تسک‌های جدید شروع کند. به این ترتیب هر برنامه که اجرا می‌شود تسک خود را دریافت می‌کند و کاربران می‌توانند از طریق صفحه ویوی کلی به دلخواه بین برنامه‌های در حال اجرا جابجا شوند.

برای شروع یک تسک جدید هنگام شروع اکتیویتی جدید، یک فلگ به اینتنت اضافه خواهیم کرد.

لیست ۲۲.۱۰ افزودن یک فلگ تسک جدید به اینتنت (BLauncherActivity.kt)

```
override fun onClick(view: View) {
    val activityInfo = resolveInfo.activityInfo
    val intent = Intent(Intent.ACTION_MAIN).apply {
        setClassName(activityInfo.applicationInfo.packageName
        ,activityInfo.name)
        addFlags(Intent.FLAG_ACTIVITY_NEW_TASK)
    }
    val context = view.context
    context.startActivity(intent)
}
```

تسک‌های فهرست شده در صفحه ویوی کلی خود را پاک کنید. BLauncher را اجرا و مشاهده کنید این بار هنگامی که صفحه ویوی کلی را بالا می‌آورید، یک تسک جداگانه برای هر اکتیویتی ایجاد می شود.

اگر اکتیویتی را دوباره از BLauncher راه اندازی کنید، دومین تسک ایجاد نخواهد شد. پرچم FLAG_ACTIVITY_NEW_TASK به خودی خود یک تسک برای هر اکتیویتی ایجاد می‌کند. بنابراین اندروید به جای شروع یک تسک جدید، به آن تسک سوییچ می‌کند.

استفاده از BLauncher به عنوان صفحه اصلی

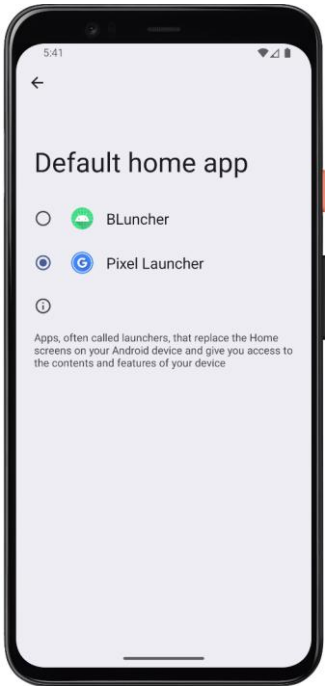
چه کسی می‌خواهد یک برنامه را برای راه اندازی برنامه‌های دیگر استفاده کند؟ منطقی‌تر است که BLauncher را به عنوان جایگزینی برای صفحه اصلی ارائه دهیم. فایل AndroidManifest.xml موجود در BLauncher را باز و به فیلتر اینتنت اصلی آن طبق لیست زیر اضافه کنید.

لیست ۲۲.۱۱ تغییر دسته‌بندی‌های BLauncherActivity (manifests/AndroidManifest.xml)

```
<intent-filter>
    <action android:name="android.intent.action.MAIN" />
    <category android:name=
    "android.intent.category.LAUNCHER"/>
    <category android:name="android.intent.category.HOME"/>
    <category android:name=
    "android.intent.category.DEFAULT"/>
</intent-filter>
```

با افزودن دسته‌بندی‌های HOME و DEFAULT به فیلتر، BLauncherActivity درخواست می‌کند که به عنوان گزینه‌ای برای صفحه اصلی ارائه شود. حال اگر به قسمت Apps→Default apps سپس Home app در تنظیمات گوشی بروید BLauncher یکی از گزینه ها خواهد بود (شکل ۲۲.۸).

تصویر ۲۲.۸ انتخاب یک برنامه Home



اگر BLauncher را به صفحه اصلی تبدیل کنیم، می‌توان بعدا به راحتی آن را تغییر داد.

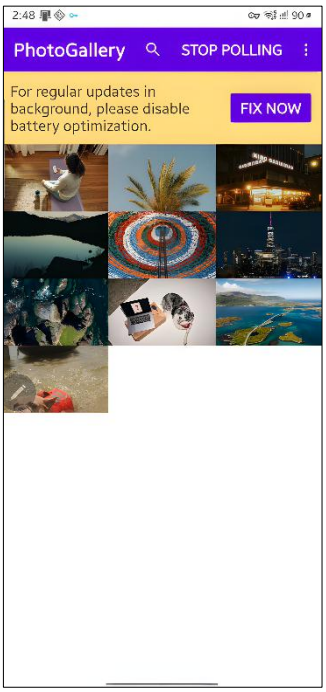
۲۳

ارتباطات شبکه، وب‌سرویس‌ها و جریان‌های داده

برنامه‌هایی که بر روان کاربران تسلط دارند، برنامه‌های متصل به شبکه هستند. کاربرانی که دیوانه‌وار فیدهای خبری خود را بررسی می‌کنند، به پیام‌ها پاسخ می‌دهند یا بازی‌های آنلاین انجام می‌دهند، همگی در حال تعامل با وب‌سرویس‌ها هستند.

برای شروع کار با شبکه در اندروید، قصد داریم یک برنامه جدید به نام PhotoGallery ایجاد کنیم. این برنامه، یک کلاینت برای سایت اشتراک‌گذاری عکس Unsplash است که جدیدترین و جذاب‌ترین عکس‌های عمومی روز را واکشی کرده و نمایش می‌دهد.

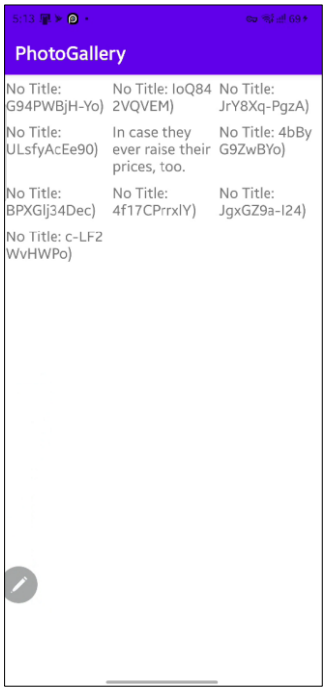
تصویر ۲۳.۱ گالری عکس کامل



در این فصل یاد می‌گیریم که چگونه از کتابخانه قدرتمند Retrofit برای مدیریت درخواست‌های HTTP استفاده کنیم و به کمک ابزارهای مدرن کاتلین یعنی Flow و Coroutines، داده‌ها را به صورت ناهمگام (Async) و بدون درگیر شدن با کالبک‌های پیچیده‌ای که در فصل های قبل دیدید به لایه رابط کاربری (UI) هدایت کنیم. همچنین از کتابخانه Gson برای تبدیل پاسخ‌های JSON به اشیاء خوانای کاتلین بهره خواهیم برد.

در پایان این فصل، عناوین عکس‌ها را از Unsplash واکشی و نمایش می‌دهیم. عملیات دانلود و نمایش خود تصاویر را در فصل بعد پی خواهیم گرفت.

شکل ۲۳.۲ گالری عکس در پایان فصل



ایجاد پروژه PhotoGallery

یک پروژه جدید در اندروید استودیو ایجاد کنید. در بخش Phone and Tablet، گزینه No Activity را انتخاب کرده و نام پروژه را PhotoGallery بگذارید. نام پکیج را com.codeb.photogallery، زبان را Kotlin و حداقل سطح API را API 24 تنظیم کنید.

پس از ساخته شدن پروژه، روی پوشه app راست‌کلیک کرده و از مسیر File → New → Activity → Empty Views Activity یک اکتیویتی به نام PhotoGalleryActivity بسازید. تیک گزینه Launcher Activity را بزنید تا این اکتیویتی نقطه شروع برنامه باشد.

فایل res/layout/activity_photo_gallery.xml را باز کرده و تکه کد زیر را جایگزین محتوای پیش‌فرض کنید تا لیوت پایه برای میزبانی فرگمنت داشته باشیم.

لیست ۲۳.۱ افزودن fragmentContainer (res/layout/activity_photo_gallery.xml)

```
<?xml version="1.0" encoding="utf-8"?>
<FrameLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/fragmentContainer"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".PhotoGalleryActivity"/>
```

حالا فایل PhotoGalleryActivity.kt را باز کنید. در متد onCreate بررسی می‌کنیم که آیا اکتیویتی برای بار اول اجرا شده است یا خیر (savedInstanceState == null). اگر اولین بار باشد، یک نمونه از فرگمنت گالری را به کانتینر اضافه می‌کنیم.

لیست ۲۳.۲ راه اندازی اکتیویتی (PhotoGalleryActivity.kt)

```
class PhotoGalleryActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContentView(R.layout.activity_photo_gallery)
        ViewCompat.setOnApplyWindowInsetsListener
        (findViewById(R.id.~~main~~fragmentContainer))
        { v, insets ->
            val systemBars = insets.getInsets
            (WindowInsetsCompat.Type.systemBars())
            v.setPadding(systemBars.left, systemBars.top
            ,systemBars.right, systemBars.bottom)
            insets
        }
```

```
        if (savedInstanceState == null) {
            supportFragmentManager
                .beginTransaction()
                .add(R.id.fragmentContainer
                , PhotoGalleryFragment.newInstance())
                .commit()
        }
    }
}
```

در BookShelf با فراخوانی findFragmentById() بررسی کردیم که آیا یک فرگمنت قبلا در نگهدارنده میزبانی شده است یا خیر. این بررسی ضروری بود زیرا مدیر فرگمنت به طور خودکار فرگمنت‌ها میزبانی شده را پس از تغییر پیکربندی یا پس از مرگ فرآیند آغاز شده توسط سیستم ایجاد و به اکتیویتی اضافه می‌کند. فقط در صورتی می‌خواهیم فرگمنت را اضافه کنیم که قبلا فرگمنتی در آنجا وجود نداشته باشد.

PhotoGalleryActivity رویکرد متفاوتی برای تعیین اینکه آیا یک فرگمنت از قبل میزبانی شده است یا خیر، اتخاذ می‌کند: بررسی می‌کند که آیا bundle موجود در savedInstanceState ارسال شده به onCreate(…) تهی است یا خیر. به یاد بیاورید که اگر باندل null باشد، این یک راه اندازی جدید از اکتیویتی است و می‌توان با خیال راحت فرض را بر آن گذاشت که هیچ فرگمنت‌ای به طور خودکار بازیابی و میزبانی مجدد نشده است. اگر باندل null نباشد، به این معنی است که اکتیویتی پس از تخریب بازسازی شده و هر فرگمنت‌ای که قبل از تخریب میزبانی شده بود دوباره ایجاد شده و دوباره به نگهدارنده مربوطه خود اضافه شده است.

هر یک از این رویکردها کارآمد هستند. استفاده از هر کدام بیشتر به استایل شما بستگی دارد.

حالا ویوی فرگمنت را تنظیم کنید. PhotoGallery نتایج خود را در یک RecyclerView نمایش و از GridLayoutManager داخلی برای مرتب کردن موارد در یک شبکه استفاده می‌کند. ابتدا کتابخانه RecyclerView را به عنوان یک وابستگی اضافه کنید. پس از انجام این تغییرات فایل Gradle را همگام سازی کنید.

نوبت به طراحی لیوت فرگمنت می‌رسد. این برنامه قرار است تصاویر را در یک شبکه (Grid) نمایش دهد، پس بهترین گزینه استفاده از RecyclerView است.

ابتدا فایل res/layout/fragment_photo_gallery.xml را بسازید و کامپوننت ریشه آن را روی RecyclerView تنظیم کنید.

پس از اتمام کار محتویات res/layout/ fragment_photo_gallery.xml باید با فهرست ۲۳.۳ مطابقت داشته باشد.

لیست ۲۳.۳ افزودن ویوی RecyclerView (res/layout/fragment_photo_gallery.xml)

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.recyclerview.widget.RecyclerView
xmlns:android="http://schemas.android.com/apk/res/android"
    android:id="@+id/photo_recycler_view"
    android:layout_width="match_parent"
    android:layout_height="match_parent"/>
```

در نهایت کلاس PhotoGalleryFragment را ایجاد کنید. لیوت مربوطه را بارگذاری و یک متغیر عضو جهت ارجاع به RecyclerView تنظیم نمایید. layoutManager ویوی RecyclerView را روی نمونه جدیدی از GridLayoutManager قرار دهید. در حال حاضر تعداد ستون‌ها را به ۳ ستون تنظیم کنید. وقتی کارتان تمام شد، کد فرگمنت شما باید با فهرست ۲۳.۴ مطابقت داشته باشد.

لیست ۲۳.۴ راه اندازی فرگمنت (PhotoGalleryFragment.kt)

```
package com.codeb.photogallery
import android.os.Bundle
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import androidx.fragment.app.Fragment
import androidx.recyclerview.widget.GridLayoutManager
import androidx.recyclerview.widget.RecyclerView
private const val TAG:String = "PhotoGalleryFragment"
class PhotoGalleryFragment : Fragment() {
    private lateinit var photoRecyclerView: RecyclerView
    override fun onCreateView(
        inflater: LayoutInflater,
        container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View {
        val view = inflater.inflate(R.layout
            .fragment_photo_gallery, container, false)
        photoRecyclerView = view.findViewById(R.id
            .photo_recycler_view)
        photoRecyclerView.layoutManager =
            GridLayoutManager(context, 3)
        return view
    }
    companion object {
        fun newInstance() = PhotoGalleryFragment()
```

```
    }
}
```

PhotoGallery را اجرا کنید تا قبل از حرکت مطمئن شویم که همه چیز به درستی انجام شده است. اگر همه چیز خوب باشد، یک صفحه خالی بسیار زیبا خواهید داشت.

مبانی شبکه‌سازی مدرن با Retrofit و Flow

کتابخانه Retrofit ساخت شرکت Square، ابزاری استاندارد برای تبدیل APIهای وب به اینترفیس‌های ساده کاتلین است. در دنیای برنامه‌نویسی موبایل و اندروید، APIهای وب (Web APIs) پل‌های ارتباطی هستند که به اپلیکیشن شما اجازه می‌دهند با سرورهای دوردست در سراسر اینترنت صحبت کنند، اطلاعات بفرستند یا داده‌های جدید را دانلود کنند.

در گذشته برای استفاده از رتروفیت مجبور بودیم از کالبک‌های تودرتو استفاده کنیم که مدیریت خطا و نخ‌ها را سخت می‌کرد. اما امروز، رتروفیت به طور کامل از توابع معلق (suspend functions) و کوروتین‌های (Coroutine) کاتلین پشتیبانی می‌کند. توابع معلق و کوروتین‌ها مکانیزم اصلی کاتلین برای نوشتن کد ناهمزمان (asynchronous) و غیر مسدود کننده (nom-blocking) به صورت ساده و خوانا هستند.

برای استفاده از آنها، ابتدا وابستگی‌های مورد نیاز را به فایل build.gradle.kts (ماژول app) اضافه و پروژه را سینک کنید.

لیست ۲۳.۵ افزودن وابستگی Retrofit (app/build.gradle)

```
dependencies {
...
implementation("com.squareup.retrofit2:retrofit:3.0.0")
implementation("com.squareup.retrofit2:converter-scalars:3.0.0")
implementation("org.jetbrains.kotlinx:kotlinx-coroutines-android:1.11.0")
...
}
```

قصد داریم صفحه اصلی Unsplash API را واکشی و محتویات آن را log کنیم. شروع ساده به ما امکان یادگیری اصولی را می‌دهد. بعدا بر اساس این پیاده‌سازی درخواست‌ها و تجزیه پاسخ‌ها را پیاده‌سازی خواهیم کرد.

تعریف اینترفیس API به صورت معلق (Suspend)

یک پکیج جدید به نام api در پروژه خود ایجاد و درون آن یک Interface به نام UnsplashAPI بسازید. در اینجا تابع را با کلمه کلیدی suspend تعریف می‌کنیم. وقتی پشت متد رتروفیت کلمه suspend را می‌گذاریم، به رتروفیت می‌گوییم: «این کار شبکه ممکن است چند ثانیه طول بکشد، پس نخ اصلی گوشی را قفل نکن. برو در پس‌زمینه کار را انجام بده و هروقت String آماده شد، آن را برگردان.»

حالا رابط شبکه را پیاده‌سازی می‌کنیم.

لیست ۲۳.۶ افزودن رابط Retrofit API (api/UnsplashAPI.kt)

```
package com.codeb.photogallery.api
import retrofit2.http.GET
interface UnsplashAPI {
    @GET("/")
    suspend fun fetchContents(): String
}
```

بیایید اجزای این تابع را کالبد شکافی کنیم:

حاشیه‌نویسی@GET("/") نوع درخواست ما را از نوع HTTP GET (برای واکشی داده‌ها) تعیین می‌کند و آدرس نسبی را روی ریشه یا همان آدرس پایه (/) قرار می‌دهد. آدرس پایه (Base URL) ریشه، ستون فقرات و آدرس اصلی سرور است که در تمام درخواست‌های برنامه ما ثابت است. این آدرس مشخص می‌کند که اپلیکیشن ما اصلاً با کدام شرکت یا سرور در اینترنت کار دارد. در برنامه ما این آدرس https://api.unsplash.com/ می باشد.

آدرس نسبی (Relative Path) یک آدرس کوتاه، متغیر و اختصاصی است که مشخص می‌کند ما دقیقاً به کدام بخش از آن سرور کار داریم. آدرس نسبی به انتهای آدرس پایه می‌چسبد. در ادامه مثال‌هایی مانند "photos" یا "search" را خواهید دید.

کلمه کلیدی suspend جادویی‌ترین بخش کد است! این کلمه به کامپایلر کاتلین می‌گوید که متد fetchContents یک تابع معلق است. توابع معلق می‌توانند بدون قفل کردن یا مسدود کردن ترد اصلی گوشی (Main Thread)، در پس‌زمینه اجرا شوند، منتظر پاسخ سرور بمانند و پس از اتمام کار، بدون نیاز به هیچ کالبک یا متد تودرتویی، نتیجه را در همان خط بازگردانند.

نوع خروجی را String مشخص کردیم. رتروفیت خودش در پشت صحنه درخواست را اجرا کرده، با کمک مبدل اسکالر پاسخ خروجی سرور را به رشته تبدیل می‌کند و رشته‌ی نهایی را تحویل ما می‌دهد.

حرکت به سمت الگوی repository

برای رعایت اصول معماری پیشنهادی گوگل، نباید کدهای شبکه را مستقیماً داخل اکتیویتی یا فرگمنت بنویسیم. رابط کاربری (UI) فقط باید درخواست داده بدهد و کاری نداشته باشد که این داده از کجا و چگونه تامین می‌شود.

یک کلاس جدید به نام UnsplashFetcher در پروژه ایجاد کنید. این کلاس نقش مخزن (Repository) ما را بازی می‌کند. خروجی تابع واکشی را از نوع Flow<String> تعریف می‌کنیم.

وقتی کارتان تمام شد، UnsplashFetcher باید با لیست ۲۳.۷ مطابقت داشته باشد.

لیست ۲۳.۷ ایجاد UnsplashFetcher (UnsplashFetcher.kt)

```
package com.codeb.photogallery
import android.util.Log
import com.codeb.photogallery.api.UnsplashAPI
import kotlinx.coroutines.flow.Flow
import kotlinx.coroutines.flow.flow
import retrofit2.Retrofit
import retrofit2.converter.scalars.ScalarsConverterFactory
private const val TAG = "UnsplashFetcher"
class UnsplashFetcher {
    private val unsplashAPI: UnsplashAPI
    init {
        val retrofit: Retrofit = Retrofit.Builder()
            .baseUrl("https://api.unsplash.com/")
            .addConverterFactory(
                ScalarsConverterFactory
                .create())
            .build()
        unsplashAPI =
            retrofit.create(UnsplashAPI::class.java)
    }
    fun fetchContents(): Flow<String> {
        return flow {
            try {
                Log.d(TAG, "webservice req started...")
                val response = unsplashAPI.fetchContents()
                Log.d(TAG, "response: $response")
                emit(response)
            } catch (t: Throwable) {
                Log.e(TAG, "error in connection", t)
                emit("error to load data")
            }
        }
```

```
        }
}
```

چرا Flow به جای LiveData؟

وقت آن است که جایگزین امروزی و بسیار مفید LiveData که در فصل‌های قبل از آن استفاده کردیم معرفی کنیم: Flow. کاتلین Flow را به LiveData ترجیح می‌دهد، چرا؟

بزرگ‌ترین مشکل LiveData این است که جزئی از معماری اندروید (androidx.lifecycle) است. این یعنی شما نمی‌توانید از لایودیتا در پروژه‌های جاوا/کاتلین خالص یا در لایه‌های عمیق‌تر پروژه (مثل لایه داده یا دیتابیس) بدون وابستگی به فریم‌ورک اندروید استفاده کنید. اما Flow بخشی از زبان کاتلین خالص است. این ویژگی باعث می‌شود که کد شما برای نوشتن تست‌های واحد (Unit Tests) بسیار راحت‌تر باشد و حتی بتوان از آن در پروژه‌های مالتی‌پلتفرم (Kotlin Multiplatform) نیز استفاده کرد. Flow معماری پاک دارد و به اندروید وابسته نیست.

Flow قدرت بی‌نظیر در تبدیل و فیلتر کردن داده‌ها (Operators) دارد. فرض کنید داده‌ای که از شبکه (ترروفیت) می‌آید، نیاز به پردازش، فیلتر یا تغییر داشته باشد. ابزار LiveData ابزارهای بسیار محدودی برای تغییر داده‌ها در اختیار ما می‌گذاشت. اما Flow مجهز به صدها اپراتور قدرتمند و آماده (مثل map, filter, debounce, combine, zip) است. به عنوان مثال، اگر کاربر در حال تایپ در کادر جستجو باشد، با Flow و اپراتور debounce می‌توان به راحتی کاری کرد که تا تایپ کاربر تمام نشده، درخواست شبکه ارسال نشود؛ کاری که انجامش با لایودیتا شبیه به یک کابوس بود!

Flow مدیریت نخ‌ها (Threading) را بی‌نقص انجام می‌دهد. در LiveData کنترل زیادی روی اینکه عملیات روی چه نخی (نخ پس‌زمینه یا اصلی) انجام شود نداشتیم؛ تغییرات حتماً باید با متدهایی مثل postValue هندل می‌شدند. در Flow مدیریت نخ‌ها با اپراتوری به نام flowOn مثل آب خوردن است. ما می‌توانیم مشخص کنیم که بخش شبکه روی نخ پس‌زمینه (Dispatchers.IO) اجرا شود و بخش نمایش روی نخ اصلی (Dispatchers.Main) اتفاق بیفتد.

مفهوم جریان سرد (Cold Stream) و بهینه‌سازی مصرف باتری یکی از چالش‌های پیش‌رو است. ابزار LiveData یک جریان گرم (Hot Stream) است؛ یعنی به محض اینکه داده‌ای تغییر کند، آن را نگه می‌دارد و آماده منتشر کردن است، چه کسی گوش بدهد و چه ندهد! اما Flow به صورت پیش‌فرض یک جریان سرد (Cold Stream) است. جریان سرد یعنی تا زمانی که فرگمنت یا اکتیویتی متد ()collect را صدا نزند و رسماً اعلام نیاز نکند، Flow هیچ انرژی و پهنای باندی مصرف نمی‌کند و درخواستی به اینترنت نمی‌فرستد. این یعنی صرفه‌جویی فوق‌العاده در مصرف باتری و اینترنت گوشی کاربر.

پس نتیجه می‌گیریم: لایودیتا (LiveData) ابزار خوبی برای سناریوهای بسیار ساده در لایه UI است، اما وقتی پای عملیات‌های پیچیده، پردازش داده‌های شبکه، خطایابی و معماری‌های مدرن وسط می‌آید، Flow با انعطاف‌پذیری و اپراتورهای قدرتمندش، برنده مطلق میدان است.

حالا به PhotoGalleryFragment برگردید و تغییرات زیر را در متد onViewCreated فرگمنت اعمال کنید (لیست ۲۳.۸).

لیست ۲۳.۸ استفاده از UnsplashFetcher (PhotoGalleryFragment.kt)

```
class PhotoGalleryFragment : Fragment() {
    private lateinit var photoRecyclerView: RecyclerView
    private val unsplashFetcher = UnsplashFetcher()
    override fun onCreateView(
        inflater: LayoutInflater,
        container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View {
        val view = inflater.inflate
        (R.layout.fragment_photo_gallery, container, false)
        photoRecyclerView = view.findViewById
        (R.id.photo_recycler_view)
        photoRecyclerView.layoutManager =
        GridLayoutManager(context, 3)
        return view
    }
    override fun onViewCreated(view: View
    , savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState)
        viewLifecycleOwner.lifecycleScope.launch {
            unsplashFetcher.fetchContents()
            .collect { responseString ->
                Log.d(TAG
                , "response received: $responseString")
            }
        }
    }
    companion object {
        fun newInstance() = PhotoGalleryFragment()
    }
}
```

این جداسازی برنامه شما را به پیروی از الگوی repository توصیه شده توسط Google در راهنمایی معماری برنامه نزدیکتر می‌کند. UnsplashFetcher به عنوان یک repository خیلی پایه عمل می‌کند. یک کلاس repository منطق دسترسی به داده‌ها از یک منبع واحد یا مجموعه‌ای از منابع را در بر

می‌گیرد و تعیین می‌کند که چگونه مجموعه خاصی از داده‌ها – چه به صورت محلی در یک پایگاه داده و چه از یک سرور راه دور – را واکشی و ذخیره کند. کد رابط کاربری شما تمام داده‌ها را از repository درخواست می‌کند، زیرا رابط کاربری اهمیتی نمی‌دهد که داده‌ها واقعا چگونه ذخیره یا واکشی می‌شوند. اینها جزئیات پیاده‌سازی خود repository است.

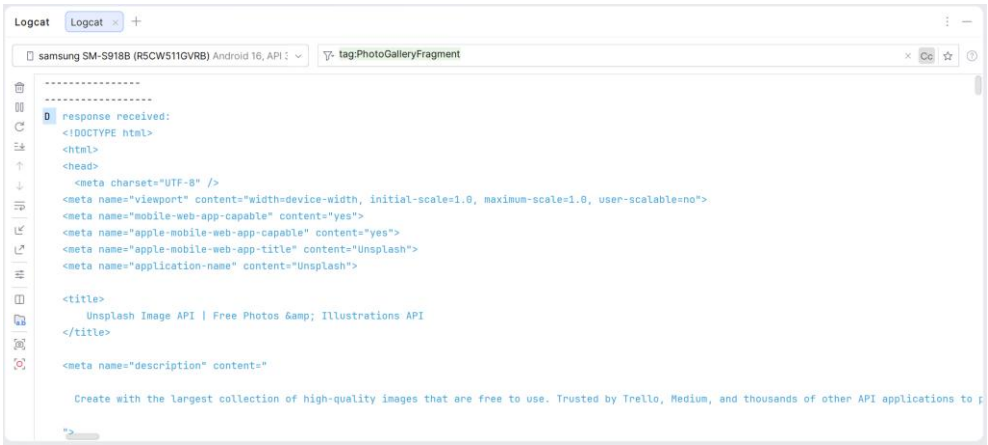
در حال حاضر تمام داده‌های برنامه شما مستقیما از وب سرور Unsplash می‌آید. با این حال در آینده ممکن است تصمیم بگیرید که آن داده‌ها را در یک پایگاه داده محلی ذخیره کنید. در این صورت repository مدیریت دریافت داده‌ها را از مکان مناسب بر عهده می‌گیرد. سایر مؤلفه‌های برنامه شما می‌توانند از repository برای دریافت داده‌ها بدون نیاز به دانستن اینکه داده‌ها از کجا می‌آیند، استفاده کنند.

برای اجرا برنامه نیاز به کسب مجوز استفاده از اینترنت را داریم. AndroidManifest.xml را باز کرده و تغییر زیر را اعمال نمایید.

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android=
    "http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">
    <uses-permission android:name=
       "android.permission.INTERNET" />
    <application>
       …
    </application>
</manifest>
```

حالا برنامه را اجرا کنید. همانطور که در شکل ۲۳.۳ نشان داده شده است، باید محتویات صفحه اصلی Unsplash را در Logcat چاپ کنید.

تصویر ۲۳.۳ داده‌ها



واکشی JSON از Unsplash

JSON مخفف JavaScript Object Notation است. یک فرمت داده محبوب به ویژه برای سرویس‌های وب است. می‌توانید اطلاعات بیشتری درباره JSON به عنوان یک فرمت در json.org دریافت کنید.

Unsplash یک JSON API خوب ارائه می‌دهد. آن را در مرورگر وب مورد علاقه خود بکشید و لیست فرمت‌های درخواست را پیدا کنید. نقطه ارتباط API این سایت، https://api.unsplash.com/ است. می‌توان متدهایی را که Unsplash در این نقطه ارائه می‌دهد همانظور که در لیست زیر آمده است، فراخوانی کرد.

```
GET https://api.unsplash.com/photos
GET https://api.unsplash.com/search/photos
GET https://api.unsplash.com/collections
GET https://api.unsplash.com/topics
GET https://api.unsplash.com/users/{username}/photos
```

قصد داریم عکس های محبوب این سایت را واکشی کنیم، به همین خاطر URL درخواست GET ما چیزی شبیه به این خواهد بود:

```
https://api.unsplash.com/photos?order_by=popular
&client_id=YOUR_ACCESS_KEY
```

این دقیقاً نقطه ارتباط رسمی و درست برای گرفتن عکس‌های محبوب (Popular Photos) از API سایت Unsplash است. این Endpoint دقیقاً شامل چه چیزی است؟

۱. متد: GET

۲. آدرس نسبی: /photos

۳. پارامترهای کوئری:

- order_by=popular: مرتب‌سازی عکس‌ها بر اساس میزان محبوبیت.
- client_id=YOUR_ACCESS_KEY: کلید اختصاصی توسعه‌دهنده (Access Key) که پس از ثبت‌نام در پنل توسعه‌دهندگان Unsplash دریافت می‌کنید. بدون این کلید محرمانه، سرور به درخواست‌های شما پاسخ معتبر نخواهد داد.

در ارتباط با استفاده از سرویس Unsplash API، لازم است مقدار YOUR_ACCESS_KEY که پس از ثبت‌نام در بخش Unsplash Developers صادر می‌شود، در تمامی درخواست‌های HTTP به‌عنوان پارامتر client_id ارسال گردد. این کلید، شناسه منحصربه‌فردی است که دسترسی برنامه کاربر را به منابع Unsplash تأیید می‌کند و بدون ارائه آن، سرور هیچ‌گونه پاسخ معتبر ارائه نخواهد داد.

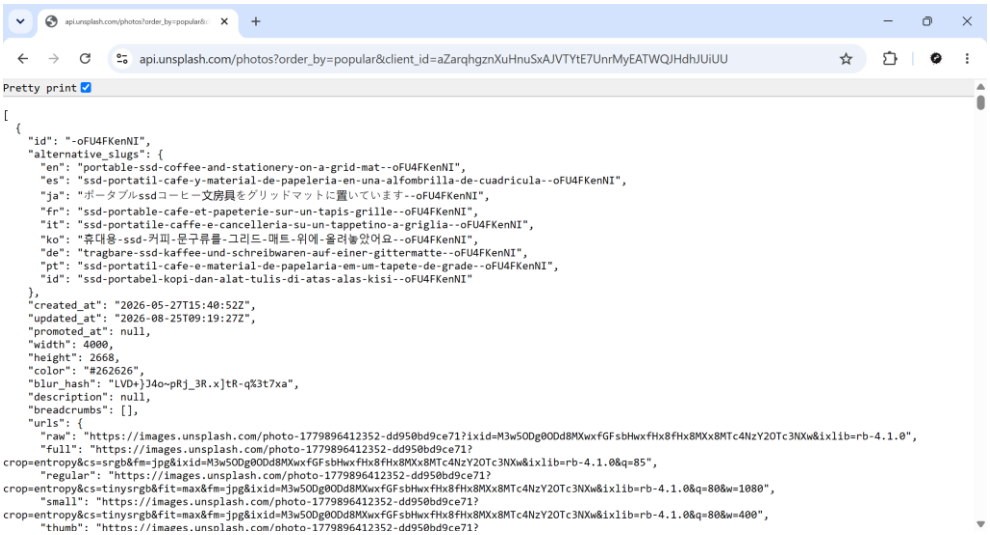
در این ساختار، عبارت YOUR_ACCESS_KEY باید با کلید واقعی صادرشده برای حساب توسعه‌دهنده جایگزین شود. این کلید محرمانه بوده و نگهداری صحیح آن برای جلوگیری از سوءاستفاده ضروری است.

این کلید فرمتی شبیه به رشته زیر را دارد.

aTarqhgznNuHnuSxAJVTYtE7UnrMyEATWQJHdhJUiUU

URL نمونه را در مرورگر خود کپی وکلید API خود را جایگزین کنید. با این کار نمونه‌ای از پاسخ را همانطور که در شکل ۲۳.۴ نشان داده شده است، می‌توانید مشاهده کنید.

شکل ۲۳.۴ نمونه خروجی JSON



زمان آن رسیده است که کد شبکه خود را به جای درخواست محتویات صفحه اصلی Unsplash API برای دریافت داده‌های عکس‌های جالب اخیر از API به روز کنیم. ابتدا یک تابع به رابط API خود اضافه کنید. باز هم YOUR_ACCESS_KEY را با کلید API خود جایگزین کنید. در حال حاضر پارامترهای کوئری URL را مستقیما وارد کنید. (بعدا، این پارامترهای کوئری را انتزاع کرده و به صورت برنامه‌نویسی اضافه می‌کنید.)

فهرست ۲۳.۹ تعریف درخواست "واکشی عکس‌های جالب اخیر" (api/UnsplashAPI.kt)

```
interface UnsplashAPI {
    @GET("/")
    suspend fun fetchContents(): String
    @GET("photos" +
            "?order_by=popular" +
```

```
            "&client_id=aZarqhgznXuHnuSxAJVTYtE7UnrMyEATWQJHdhJUiUU"
    )
    suspend fun fetchPhotos(): String
}
```

توجه داشته باشید که مقداری را برای YOUR_ACCESS_KEY اضافه کردیم.

کد پیکربندی نمونه رتروفیت را در UnsplashFetcher به روز کنید. نام تابع ()fetchContents را به ()fetchPhotos تغییر و تابع جدید ()fetchPhotos در رابط API فراخوانی کنید.

لیست ۲۳.۱۰ به روز رسانی URL پایه (UnsplashFetcher.kt)

```
class UnsplashFetcher {
    private val unsplashAPI: UnsplashAPI
    init {
        val retrofit: Retrofit = Retrofit.Builder()
            .baseUrl("https://api.unsplash.com/")
            .addConverterFactory(ScalarsConverterFactory
            .create())
            .build()
        unsplashAPI =
        retrofit.create(UnsplashAPI::class.java)
    }
    fun ~~fetchContents()~~fetchPhotos(): Flow<String> {
        return flow {
            try {
                Log.d(TAG, "webservice req started...")
                val response =
                unsplashAPI.~~fetchContents()~~fetchPhotos()
                Log.d(TAG, "response: $response")
                emit(response)
            } catch (t: Throwable) {
                Log.e(TAG, "error in connection", t)
                emit("error to load data")
            }
        }
    }
}
```

توجه داشته باشید که URL پایه‌ای که تنظیم می‌کنیم api.Unsplash.com است، اما نقطه ارتباط که می‌خواهیم برویم در api.Unsplash.com/photos قرار دارد. به این دلیل است که ما سرویس‌ها و قسمت‌های rest مسیر را در حاشیه‌نویسی @GET خود در UnsplashAPI مشخص کرده‌ایم. مسیر و سایر اطلاعاتی که در حاشیه‌نویسی @GET گنجانده‌اید قبل از صدور درخواست وب، توسط Retrofit به URL اضافه می‌شود.

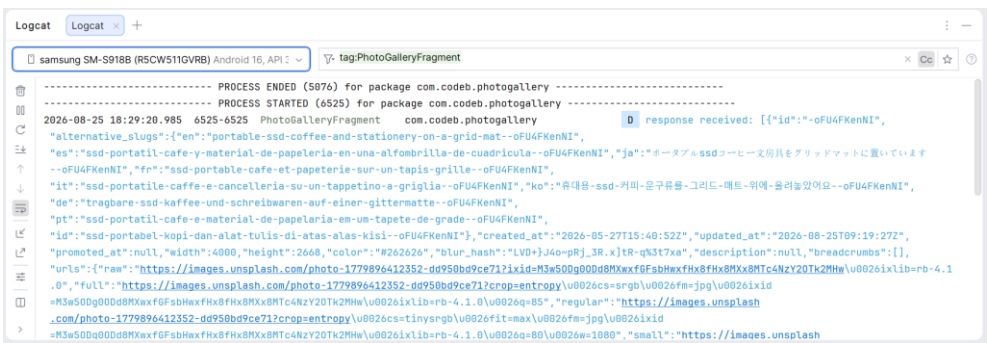
در نهایت PhotoGalleryFragment را برای اجرای درخواست وب به روز کنید تا به جای محتویات صفحه اصلی Unsplash، عکس‌های جالب اخیر را واکشی کند. فراخوانی fetchContents() را با فراخوانی تابع جدید fetchPhotos() جایگزین کنید.

لیست ۲۳.۱۱ اجرای درخواست "واکشی عکس‌های جالب اخیر" (PhotoGalleryFragment.kt)

```
class PhotoGalleryFragment : Fragment() {
    …
    override fun onViewCreated(view: View
    , savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState)
        viewLifecycleOwner.lifecycleScope.launch {
            unsplashFetcher
            .fetchContents()fetchPhotos().collect
            { responseString ->
                Log.d(TAG, "response received:
                $responseString")
            }
        }
    }
    companion object {
        fun newInstance() = PhotoGalleryFragment()
    }
}
```

ایجاد این چند ترفند در کد موجود، برنامه ما را برای واکشی و لاگ کردن داده‌های Unsplash آماده می‌کند. PhotoGallery را اجرا کنید. باید JSON را مانند شکل ۲۳.۵ در Logcat ببینید.

شکل ۲۳.۵ Unsplash JSON در Logcat



(اگر نتایجی مانند ما دریافت نکردید، وحشت نکنید. برنامه را مجددا تست نمایید.)

اکنون که چنین داده‌های JSON خوبی از Unsplash داریم، چه کاری باید با آن انجام دهیم؟ کاری را که با تمام داده‌ها انجام می‌دهیم، انجام خواهیم داد – آن را در یک یا چند شی مدل قرار می‌دهیم. کلاس مدلی که می‌خواهیم برای PhotoGallery ایجاد کنید UnsplashPhoto نام دارد. یک مورد از آن دارای اطلاعات متا برای یک عکس، از جمله عنوان، شناسه و نشانی اینترنتی برای دانلود تصویر است.

کلاس داده UnsplashPhoto را ایجاد و کد زیر را اضافه کنید.

لیست ۲۳.۱۲ ایجاد یک کلاس آبجکت‌ها مدل (UnsplashPhoto.kt)

```
data class UnsplashPhoto(
    val id: String,
    val title: String?,
    val url: Urls
)
class Urls(
    val raw: String,
    val full: String,
    val regular: String,
    val small: String,
    val thumb: String
)
```

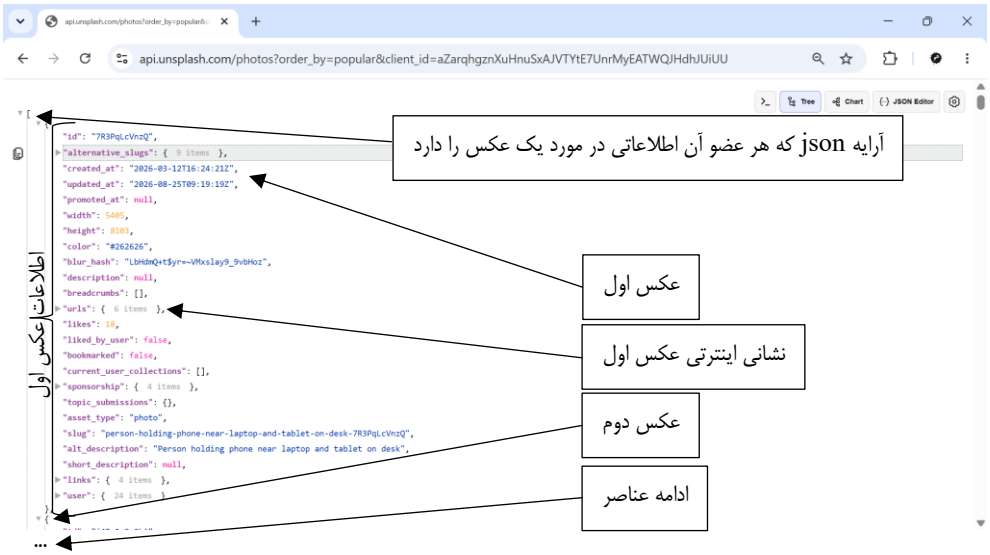
چون نشانی اینترنتی هر عکس شامل تگ فرزند است، یک کلاس جداگانه با نام Urls برای آن ایجاد کردیم. دقت کنید urls کلاس UnsplashPhoto از همین نوع است.

پس از تعریف مدل زمان ایجاد نمونه‌هایی از آن شی و پر کردن آنها با داده‌های خروجی JSON دریافتی از Unsplash است.

تجزیه JSON به آبجکت‌های مدل

پاسخ JSON نمایش داده شده در مرورگر و پنجره Logcat به سختی قابل خواندن است. اگر پاسخ را در یک jsonviewer مشاهده کنیم، چیزی شبیه به لیست ۲۴.۶ به نظر می‌رسد.

شکل ۲۳.۶ متن JSON، سلسله مراتب JSON



یک شی JSON مجموعه‌ای از جفت‌های نام – مقدار است که بین { } محصور شده‌اند. آرایه JSON یک لیست جدا شده با کاما از آبجکت‌ها JSON است که در پرانتز محصور شده‌اند. ما می‌توانیم آبجکت‌ها را در داخل یکدیگر قرار دهیم و در نتیجه یک سلسله مراتب ایجاد کنیم. با توجه به تصویر ۲۴.۶ عناصر آرایه Json در برنامه ما باید در مدل UnsplashPhoto و نشانی اینترنتی هر کدام از UnsplashPhotoها در مدل Urls قرار گیرند.

اندروید شامل پکیج استاندارد org.json است که دارای کلاس‌هایی است که دسترسی به ایجاد و تجزیه متن JSON (مانند JSONObject و JSONArray) را فراهم می‌کند. با این حال، بسیاری از افراد باهوش کتابخانه‌هایی ایجاد کرده‌اند تا فرآیند تبدیل متن JSON به آبجکت‌ها جاوا و بازگشت مجدد را ساده کنند.

یکی از این کتابخانه‌ها Gson (github.com/google/gson) است. Gson داده‌های JSON را به طور خودکار به آبجکت‌ها کاتلین برای شما نگاشت می‌کند. این بدان معناست که شما نیازی به نوشتن کد تجزیه ندارید. در عوض، شما کلاس‌های کاتلین را تعریف می‌کنید که به سلسله مراتب آبجکت‌هاهای JSON نگاشت می‌شوند و به Gson اجازه می‌دهید بقیه کارها را انجام دهد.

Square یک مبدل Gson برای Retrofit ایجاد کرد که اتصال Gson به اجرای Retrofit شما را آسان می‌کند. ابتدا وابستگی‌های کتابخانه مبدل Gson و Retrofit Gson را به فایل Gradle ماژول برنامه خود اضافه کنید.

لیست ۲۳.۱۳ افزودن وابستگی‌های Gson (app/build.gradle)

```
dependencies {
    …
    implementation("com.squareup.retrofit2:converter-scalars:3.0.0")
    implementation("com.google.code.gson:gson:2.14.0")
    implementation("com.squareup.retrofit2:converter-gson:3.0.0")
    …
}
```

در مرحله بعد، آبجکت‌ها مدلی ایجاد کنید که به داده‌های JSON در پاسخ Unsplash نگاشت شوند. ما در حال حاضر UnsplashPhoto را داریم که تقریبا مستقیما به یک شی جداگانه در آرایه JSON نگاشت می‌کند. به طور پیش‌فرض Gson نام آبجکت‌ها JSON را به نام ویژگی‌ها نگاشت می‌کند. اگر نام ویژگی‌های ما مستقیما با نام آبجکت‌های JSON مطابقت دارد، می‌توانیم آنها را همانطور که هست رها کنید. اما نام ویژگی‌های ما نیازی به مطابقت با نام آبجکت‌ها JSON ندارد. ویژگی UnsplashPhoto.title خود را در مقابل فیلد "description" در داده‌های JSON در نظر بگیرید. UnsplashPhoto.title در کد ما معنادارتر است، بنابراین بهتر است آن را حفظ کنیم. یک حاشیه‌نویسی @SerializedName به ویژگی اضافه کنید تا به Gson بگویید که ویژگی به کدام فیلد JSON نگاشت شود. همچنین اینکار را با url نیز انجام دهید.

اکنون UnsplashPhoto را جهت انجام این کار به روز کنید.

فهرست ۲۳.۱۴ override نگاشت پیش‌فرض name-property (UnsplashPhoto.kt)

```
data class UnsplashPhoto(
    val id: String,
    @SerializedName("description")val title: String?,
    @SerializedName("urls")val url: Urls
)
```

اکنون زمان آن رسیده است که جادو کنیم: پیکربندی Retrofit برای استفاده از Gson برای تجزیه داده‌های خود در آبجکت‌ها مدلی که به تازگی تعریف کرده‌ایم. ابتدا نوع بازگشتی مشخص شده در رابط Retrofit API را به روز کنید تا با شی مدلی که برای نگاشت به بالاترین شی JSON تعریف کرده‌اید مطابقت داشته باشد. این به Gson می‌گوید که باید از UnsplashPhoto برای تجزیه داده‌های پاسخ JSON استفاده کند.

لیست ۲۳.۱۵ به روز رسانی نوع بازگشت ()fetchPhotos (UnsplashAPI.kt)

```
interface UnsplashAPI {
    @GET(
        "photos" +
        "?order_by=popular" +
        "&client_id="+
        "aZarqhgznXuHnuSxAJVTYtE7UnrMyEATWQJHdhJUiUU"
    )
```

```
    suspend fun fetchPhotos():~~String~~ List<UnsplashPhoto>
}
```

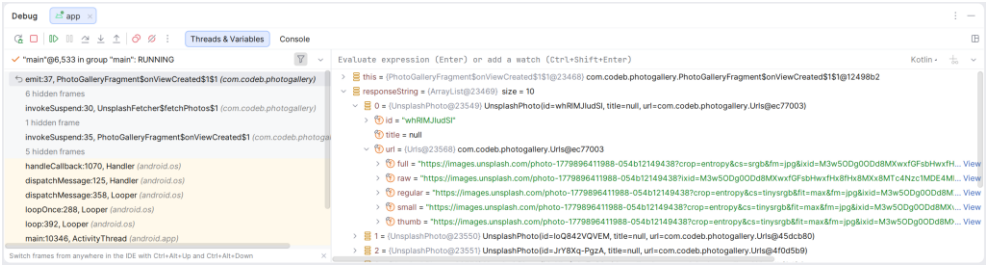
در مرحله بعد UnsplashFetcher را به روز کنید. converter factory اسکالر را با converter factory Gson تغییر دهید. ()fetchPhotos را برای برگرداندن Flow که لیستی از UnsplashPhoto را در بردارد، به روز کنید. در نهایت کد شما دقیقا باید همانند لیست ۲۳.۱۶ باید باشد.

لیست ۲۳.۱۶ به روز رسانی UnsplashFetcher برای Gson (UnsplashFetcher.kt)

```
class UnsplashFetcher {
    private val unsplashAPI: UnsplashAPI
    init {
        val retrofit: Retrofit = Retrofit.Builder()
            .baseUrl("https://api.unsplash.com/")
            .addConverterFactory(GsonConverterFactory
            .create())
            .build()
        unsplashAPI = retrofit
        .create(UnsplashAPI::class.java)
    }
    fun fetchPhotos(): Flow<List<UnsplashPhoto>> = flow {
        try {
            Log.d(TAG,
            "Fetching popular photos from Unsplash...")
            val fetchedPhotos = unsplashAPI.fetchPhotos()
            val filteredPhotos = fetchedPhotos.filter {
            it.url.small.isNotBlank() }
            emit(filteredPhotos)
        } catch (e: Exception) {
            Log.e(TAG, "Failed to fetch or parse photos", e)
            emit(emptyList())
        }
    }
}
```

توجه داشته باشید که Unsplash همیشه یک مقدار url_s معتبر برای هر تصویر برنمی‌گرداند. کد بالا آیتم‌های گالری را با مقادیر URL خالی با استفاده از filter{...} فیلتر می‌کند. برنامه را برای آزمایش کد تجزیه JSON خود اجرا کنید. باید خروجی ()toString لیست UnsplashPhoto را که در Logcat چاپ شده است مشاهده کنید. اگر می‌خواهید نتایج را بیشتر بررسی کنید، یک breakpoint روی خط ورود به سیستم در Observer تنظیم و از دیباگ برای بررسیUnsplashPhoto استفاده کنید (شکل ۲۴.۷).

تصویر ۲۳.۷ بررسی پاسخ Unsplash



اگر با UninitializedPropertyAccessException مواجه شدید، مطمئن شوید که درخواست وب شما به درستی قالب بندی شده است. در برخی موارد (مانند زمانی که کلید API نامعتبر است)، Unsplash API پاسخی را با کد پاسخ خوب (۲۰۰) اما بدنه پاسخ خالی برمی‌گرداند و Gson نمی‌تواند مدل‌های late-initialized شما را مقداردهی اولیه کند.

کار شبکه در تغییرات پیکربندی

اکنون که برنامه ما به خوبی JSON را به آبجکت‌ها مدل تجزیه می‌کند، بیایید نگاهی دقیق‌تر به نحوه رفتار پیاده‌سازی ما در هنگام تغییر پیکربندی بیندازیم. برنامه را اجرا کنید، سپس دستگاه را به سرعت پنج بار یا بیشتر پشت سر هم بچرخانید.

خروجی Logcat را بررسی کنید. اینجا چه خبر است؟ هر بار که دستگاه را می‌چرخانیم، یک درخواست شبکه جدید ایجاد می‌شود. دلیل این کار این است که ما درخواست را در onCreate(…) ارسال می‌کنیم. از آنجایی که فرگمنت در هر چرخش از بین می‌رود و دوباره ایجاد می‌شود، درخواست جدیدی برای دانلود مجدد داده‌ها (غیرضروری) صادر می‌شود. این مشکل ایجاد می‌کند، زیرا ما در حال انجام کارهای تکراری هستیم. ما باید هنگام ایجاد فرگمنت برای اولین بار درخواست دانلود صادر کنیم. درخواست (و داده‌های حاصل) باید در طول چرخش حفظ شوند تا از تجربه سریع کاربر اطمینان حاصل شود (و از استفاده غیرضروری از داده‌های کاربر در صورت عدم استفاده از WiFi جلوگیری شود).

به جای راه اندازی یک درخواست وب جدید هر بار به هنگام تغییر پیکربندی، باید یک بار داده‌های عکس را زمانی که فرگمنت در ابتدا ایجاد و روی صفحه نمایش داده می‌شود، واکشی کنیم. سپس با ذخیره‌سازی نتایج به درخواست وب اجازه دهیم تا در صورت تغییر پیکربندی به اجرا ادامه دهد.

ViewModel ابزار مناسبی برای کمک در این کار است.

ابتدا وابستگی lifecycle-extensions را به app/build.gradle اضافه کنید.

لیست ۲۳.۱۷ افزودن وابستگی lifecycle-extensions (app/ build.gradle)

```
dependencies {
    …
    implementation("com.squareup.retrofit2:converter-gson:3.0.0")
    implementation("androidx.lifecycle:lifecycle-viewmodel-ktx:2.11.0")
    implementation("androidx.fragment:fragment-ktx:1.8.9")
    …
}
```

در معماری مدرن اندروید، داده‌ها در لایه‌های پایینی (شبکه و دیتابیس) به صورت Flow تولید می‌شوند، اما وقتی به ViewModel می‌رسند، حتماً باید تبدیل به StateFlow شوند تا در برابر چرخش صفحه مقاوم باشند و رابط کاربری (UI) بتواند به صورت کاملاً امن با آن‌ها کار کند. اگر از Flow معمولی استفاده کنید، با هر بار چرخش صفحه، فرگمنت دوباره متد ()collect را صدا می‌زند و فلو سرد مجبور می‌شود درخواست اینترنت را از صفر تکرار کند! این یعنی مصرف حجم اینترنت و باتری کاربر و کندی برنامه.

اما StateFlow یک جریان گرم است. دیتایی که از اینترنت آمده را مانند یک گاوصندوق در حافظه ویومدل نگه می‌دارد. وقتی صفحه می‌چرخد و فرگمنتِ جدید متولد می‌شود، StateFlow فوراً آخرین دیتای موجود را بدون ارسال حتی یک درخواست جدید به اینترنت، به فرگمنتِ جدید تحویل می‌دهد.

در مرحله بعد یک کلاس ViewModel با نام PhotoGalleryViewModel ایجاد و کد فهرست ۲۴.۳۰ را در آن قرار دهید.

لیست ۲۳.۱۸ ViewModel جدید (PhotoGalleryViewModel.kt)

```
package com.codeb.photogallery

import androidx.lifecycle.ViewModel
import androidx.lifecycle.viewModelScope
import kotlinx.coroutines.flow.SharingStarted
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.stateIn

class PhotoGalleryViewModel : ViewModel() {
    private val unsplashFetcher = UnsplashFetcher()
    val unsplashPhotos:
            StateFlow<List<UnsplashPhoto>> = unsplashFetcher
        .fetchPhotos()
        .stateIn(
            scope = viewModelScope,
            started = SharingStarted.WhileSubscribed(5000),
            initialValue = emptyList()
        )
}
```

بیایید پارامترهای داخل متد stateIn را کالبدشکافی کنیم تا ببینیم چطور یک فلو معمولی و سرد شبکه را به یک StateFlow زنده و مقاوم تبدیل کرده‌ایم:

پارامتر scope = viewModelScope مشخص می‌کند که این جریان داده به عمر چه کسی وابسته است. viewModelScope تا زمانی زنده است که کاربر در این صفحه حضور دارد. به محض اینکه کاربر دکمه بازگشت را بزند و فرگمنت کلاً بسته شود، ویومدل نابود شده و این اسکوپ تمام درخواست‌های اینترنت نیمه‌کاره را فوراً لغو (Cancel) می‌کند تا از نشت حافظه جلوگیری کند.

پارامتر initialValue = emptyList()، یک StateFlow برخلاف فلوهای معمولی، هرگز نمی‌تواند بدون مقدار باشد. از آنجا که درخواست شبکه ممکن است چند ثانیه طول بکشد، ما یک لیست خالی به عنوان مقدار اولیه به آن می‌دهیم. رابط کاربری (UI) به محض متصل شدن، این لیست خالی را می‌بیند و متوجه می‌شود که فعلاً دیتایی نیست (مثلاً شبیه‌ساز لودینگ را نشان می‌دهد) تا زمانی که جواب اصلی از اینترنت برسد.

پارامتر جادویی started = SharingStarted.WhileSubscribed(5000) هوشمندانه‌ترین بخش مدیریت مصرف باتری و اینترنت است. این عدد ۵۰۰۰ (پنج هزار میلی‌ثانیه یا ۵ ثانیه) یک بافر زمانی برای شرایط خاص است:

هنگام چرخش صفحه (Configuration Change): وقتی کاربر گوشی را می‌چرخاند، فرگمنت قبلی نابود می‌شود و فرگمنت جدید ساخته می‌شود. در این میان، برای چند کسری از ثانیه، تعداد شنوندگان (Subscribers) این ویومدل به صفر می‌رسد.

اگر این تاخیر ۵ ثانیه‌ای نبود، ویومدل فکر می‌کرد دیگر کسی به دیتا نیاز ندارد، پس لوله‌کشی شبکه را کلاً قطع می‌کرد و با آمدن فرگمنت جدید، مجبور می‌شد درخواست اینترنت را از اول تکرار کند!

با وجود این تاخیر ۵ ثانیه‌ای: ویومدل ۵ ثانیه دندان روی جگر می‌گذارد و صبر می‌کند. فرگمنت جدید در کمتر از چند میلی‌ثانیه متولد شده و دوباره به آن متصل می‌شود. در نتیجه، هیچ درخواست اینترنت اضافه‌ای ارسال نمی‌شود و کاربر بدون هیچ وقفه یا لگی، عکس‌ها را روی صفحه می‌بیند. اما اگر کاربر کلاً برنامه را ببندد یا به یک صفحه دیگر برود و بیش از ۵ ثانیه بگذرد، لوله‌کشی شبکه واقعاً متوقف می‌شود تا منابع گوشی هدر نروند.

حالا فرگمنت را به ویومدل متصل می‌کنیم. برای گوش دادن به داده‌های یک StateFlow در لایه رابط کاربری (UI)، از متد فوق‌العاده امن repeatOnLifecycle استفاده می‌کنیم. این متد تضمین می‌کند که اگر کاربر دکمه هوم را زد و برنامه به پس‌زمینه رفت، جمع‌آوری داده‌ها فوراً متوقف شود تا باتری گوشی هدر نرفته و برنامه کرش نکند.

لیست ۲۳.۱۹ دریافت نمونه ViewModel از ارائه دهنده (PhotoGalleryFragment.kt)

```
class PhotoGalleryFragment : Fragment() {
    private lateinit var photoRecyclerView: RecyclerView
    private val photoGalleryViewModel:
        PhotoGalleryViewModel by viewModels()
    private val unsplashFetcher = UnsplashFetcher()
    …
    override fun onViewCreated(view: View
    , savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState)
        viewLifecycleOwner.lifecycleScope.launch {
            unsplashFetcher.fetchPhotos().collect {
            photosList ->
                Log.d(TAG, "response received: $photosList")
            }
        }
        viewLifecycleOwner.lifecycleScope.launch {
            viewLifecycleOwner.repeatOnLifecycle(
                Lifecycle.State.STARTED) {
                photoGalleryViewModel.unsplashPhotos
                .collect { unsplashPhotos ->
                    if (unsplashPhotos.isNotEmpty()) {
                    Log.d(TAG
                    , "Data:${unsplashPhotos.size} photos")
                    }
                }
            }
        }
    }
    companion object {
        fun newInstance() = PhotoGalleryFragment()
    }
}
```

در نهایت موارد مرتبط با رابط کاربری (مانند آداپتور recycler view) را در پاسخ به تغییرات داده‌ها به روز خواهیم کرد.

برنامه خود را اجرا کنید. Logcat را توسط UnsplashFetcher فیلتر کنید. شبیه‌ساز را چندین بار بچرخانید. شما فقط باید …Fetching popular photos from Unsplash را یک بار در پنجره Logcat ببینید، مهم نیست چند بار بچرخانید.

نمایش نتایج در RecyclerView

برای آخرین کار خود در این فصل، به لایه view رفته و RecyclerView را برای نمایش برخی عناوین آماده خواهیم کرد.

برای درک عمیق روند کار، از کوچک‌ترین و پایه‌ای‌ترین جزء لایه نمایش، یعنی ViewHolder شروع می‌کنیم. در معماری RecyclerView، لایه ViewHolder نقش «نگهدارنده و مدیریت‌کننده ویوهای یک آیتم منفرد» را بازی می‌کند. هر زمان که یک خانه (سلول) از جدول یا لیست قرار است روی صفحه نمایش داده شود، یک نمونه از این کلاس وظیفه پر کردن و آپدیت کردن المان‌های بصری آن خانه را بر عهده دارد.

کلاس ViewHolder را همانند لیست ۲۳.۲۰ در PhotoGalleryFragment تعریف کنید.

لیست ۲۳.۲۰ افزودن پیاده‌سازی ViewHolder (PhotoGalleryFragment.kt)

```
class PhotoGalleryFragment : Fragment() {
    ...
    companion object {
        ...
    }
    private class PhotoHolder(private val itemTextView
    : TextView) :
        RecyclerView.ViewHolder(itemTextView) {
        fun bind(photo: UnsplashPhoto) {
            itemTextView.text = photo.title ?:
               "No Title: ${photo.id})"
        }
    }
}
```

جزئیات و مراحل دقیق کارکرد این لایه به شرح زیر است:

کلاس PhotoHolder در ورودی خود یک شیء از جنس TextView (یا هر ویوی دیگری که لایه گرافیکی آیتم ما را می‌سازد) دریافت می‌کند. این ویو مستقیماً به سازنده‌ی کلاس پدر، یعنی RecyclerView.ViewHolder(itemTextView) پاس داده می‌شود.

با این کار، سیستم اندروید این ویو را به عنوان یک «سلول قابل بازیافت» در حافظه علامت‌گذاری می‌کند.

متغیر داخلی و پیش‌فرض itemView در این کلاس متولد می‌شود که به کل ریشه‌ی گرافیکی آن آیتم دسترسی دارد. متد bind قلب تپنده‌ی هولدر است. آداپتر در زمان مناسب، مدل داده‌ی مربوط به آن موقعیت از لیست (که اینجا شیئی از کلاس UnsplashPhoto است) را به این تابع تحویل می‌دهد. برای

مدیریت مقادیر پوچ (Null Safety) درون متد bind، از عملگر ال‌ویس کاتلین (?:) استفاده شده است. از آنجا که در پاسخ‌های وب‌سرویس ممکن است برخی از عکس‌ها در سایت Unsplash عنوانی نداشته باشند (photo.title پوچ یا همان null باشد)، برنامه کرش نمی‌کند؛ بلکه به صورت هوشمند یک متن جایگزین همراه با شناسه منحصربه‌فرد عکس (photo.id) را درون کادر متنی قرار می‌دهد. این کلاس هیچ وضعیت (State) داخلی را در خود ذخیره نمی‌کند. وظیفه آن فقط و فقط این است: «داده را بگیر و فوراً روی متنِ ویو بنویس». این سادگی به سیستم اجازه می‌دهد وقتی این ویو از صفحه خارج شد، به سرعت متن قبلی را پاک کرده و متن عکس بعدی را بدون هیچ‌گونه تداخل یا سنگینی در حافظه، جایگزین کند.

در مرحله بعد یک RecyclerView.Adapter اضافه کنید تا PhotoHolders را در صورت نیاز بر اساس لیستی از UnsplashPhotoها صدا بزند.

لیست ۲۳.۲۱ افزودن پیاده‌سازی RecyclerView.Adapter (PhotoGalleryFragment.kt)

```
class PhotoGalleryFragment : Fragment() {
    …
    private class PhotoHolder(private val itemTextView
    : TextView) :
        RecyclerView.ViewHolder(itemTextView) {
        fun bind(photo: UnsplashPhoto) {
            itemTextView.text = photo.title ?
            : "No Title: ${photo.id})"
        }
    }
    private class PhotoAdapter : ListAdapter<UnsplashPhoto
    , PhotoHolder>(PhotoDiffCallback()) {
        override fun onCreateViewHolder(parent: ViewGroup
        , viewType: Int): PhotoHolder {
            val textView = TextView(parent.context).apply {
                setPadding(16, 16, 16, 16)
            }
            return PhotoHolder(textView)
        }
        override fun onBindViewHolder(holder: PhotoHolder
        , position: Int) {
            val unsplashPhoto = getItem(position)
            holder.bind(unsplashPhoto)
        }
    }
    …
}
```

جزئیات و وظایف فنی این کلاس را گام به گام بررسی می‌کنیم:

آداپتر ما از ListAdapter ارث‌بری دارد و در خط اول، به سیستم اعلام می‌کند که قرار است با چه نوع داده‌ای و چه نوع هولدری کار کند: UnsplashPhoto، نوع داده‌ای که لیست آداپتر را تشکیل می‌دهد و PhotoHolder، نوع هولدری که برای رندر آیتم‌ها استفاده می‌شود.

PhotoDiffCallback() مغز متفکری که در ورودی سازنده (Constructor) پاس داده می‌شود تا لیست‌ها را هوشمندانه مقایسه کند (در مرحله بعد به آن می‌رسیم). با این کار، دیگر نیازی به پیاده‌سازی متد سنتی getItemCount() نیست، چون ListAdapter خودش تعداد لیست را مدیریت می‌کند.

متد onCreateViewHolder (کارگاه ساخت سلول‌ها) زمانی توسط اندروید صدا زده می‌شود که RecyclerView به یک سلول جدید روی صفحه نیاز دارد. ساخت فیزیکی ویو و تنظیمات ظاهری با بلوک apply، یک فاصله داخلی (Padding) به اندازه ۱۶dp به چهار طرف متن داده می‌شود تا نوشته‌ها در جدول به هم نچسبند. خروجی یعنی ویوی متنی ساخته شده، درون کلاس PhotoHolder بسته‌بندی شده و به سیستم تحویل داده می‌شود.

این متد فقط و فقط به تعداد محدود (مثلاً به اندازه‌ای که صفحه گوشی را پر کند + چند آیتم زاپاس) اجرا می‌شود و پس از آن، سیستم ساخت ویوی جدید را متوقف می‌کند.

متد onBindViewHolder (اتصال داده به سلول بازیافت شده) پرکارترین بخش آداپتر در زمان اسکرول کردن کاربر است. هر زمان که یک سلول قرار است روی صفحه نمایش داده شود (چه در ابتدا و چه هنگام اسکرول) این متد، با متد داخلی getItem(position) به صورت کاملاً بهینه، شیء UnsplashPhoto مربوط به آن ردیف یا خانه از جدول را از لیستِ در حال نمایش بیرون می‌کشد و داده‌ی استخراج شده مستقیماً به تابع bind از PhotoHolder (که در مرحله قبل ساختیم) تحویل داده می‌شود تا روی صفحه نقش ببندد.

کلاس PhotoDiffCallback مغز متفکر و موتور محرک بهینه‌سازی در ListAdapter است. این کلاس از ابزاری به نام DiffUtil ارث‌بری می‌کند که وظیفه آن مقایسه دو لیست (لیست قدیمی موجود روی صفحه و لیست جدید دریافتی از اینترنت) و پیدا کردن تفاوت‌های آن‌هاست.

این کلاس به صورت کاملاً خودکار در یک ترد پس‌زمینه (Background Thread) اجرا می‌شود تا محاسبات سنگین مقایسه، باعث قفل شدن یا لگ زدن رابط کاربری برنامه نشود.

کد این بخش به این صورت پیاده‌سازی شده است:

لیست ۲۳.۲۲ افزودن PhotoDiffCallback (PhotoGalleryFragment.kt)

```
class PhotoGalleryFragment : Fragment() {
    ...
    private class PhotoAdapter : ListAdapter<UnsplashPhoto
    , PhotoHolder>(PhotoDiffCallback()) {
        ...
    }
    private class PhotoDiffCallback :
    DiffUtil.ItemCallback<UnsplashPhoto>() {
        override fun areItemsTheSame(oldItem: UnsplashPhoto
        , newItem: UnsplashPhoto): Boolean {
            return oldItem.id == newItem.id
        }
        override fun areContentsTheSame(oldItem:
        UnsplashPhoto, newItem: UnsplashPhoto): Boolean {
            return oldItem == newItem
        }
    }
}
```

این کلاس دو متد حیاتی دارد که روند مقایسه را در دو مرحله مجزا انجام می‌دهند.

مرحله اول: متد areItemsTheSame (بررسی هویت آیتم)

این متد اولین خط دفاعی مقایسه است. اندروید تک‌تک آیتم‌های لیست قدیم و جدید را به این متد می‌فرستد تا بفهمد آیا این دو شیء، دقیقاً به یک آیتم واقعی اشاره دارند یا خیر.

نحوه کارکرد: ما در این متد شناسه منحصربه‌فرد عکس‌ها یعنی oldItem.id == newItem.id را با هم مقایسه می‌کنیم. اگر شناسه‌ها برابر نباشند (false): سیستم می‌فهمد که این یک آیتم کاملاً جدید است یا آیتم قبلی حذف شده است. اگر شناسه‌ها برابر باشند (true): سیستم می‌فهمد هویت آیتم یکی است (مثلاً عکس شماره ۴۵ است)؛ اما هنوز مطمئن نیست که جزییات داخلی آن تغییر کرده یا نه. پس بلافاصله مرحله دوم را صدا می‌زند.

مرحله دوم: متد areContentsTheSame (بررسی محتوای آیتم)

این متد تنها زمانی اجرا می‌شود که مرحله اول تایید کرده باشد هویت دو آیتم یکی است (areItemsTheSame خروجی true داده باشد). وظیفه این متد این است که بررسی کند آیا اطلاعات داخلی آن آیتم دستخوش تغییر شده است یا خیر.

نحوه کارکرد: ما از دستور oldItem == newItem استفاده می‌کنیم. از آنجا که UnsplashPhoto یک data class در کاتلین است، متد equals به صورت خودکار تمام متغیرهای داخلی آن (مانند عنوان، آدرس URL، تعداد لایک‌ها و...) را تک‌تک با هم مقایسه می‌کند.

اگر خروجی true باشد، یعنی عکس همان عکس قبلی است و هیچ آپدیتی در عنوان یا ظاهر آن رخ نداده؛ پس RecyclerView هیچ کار اضافه‌ای انجام نمی‌دهد و سلول را بازنویسی نمی‌کند (صرفه‌جویی در مصرف پردازنده). اگر خروجی false باشد، یعنی هویت عکس یکی است اما مثلاً عنوان آن ویرایش شده است. در این حالت، آداپتر فقط و فقط همان یک سلول را روی صفحه بازنویسی (re-bind) می‌کند.

چرا این پیاده‌سازی بسیار بهینه است؟

در روش‌های سنتی، با تغییر دیتا کل لیست از اول رندر می‌شد. اما با کمک PhotoDiffCallback:

اگر یک عکس به وسط لیست اضافه شود، سیستم متوجه می‌شود و به جای بازنویسی کل صفحه، فقط آن آیتم را با یک انیمیشن نرم درج (Insert) می‌کند.

اگر ترتیب عکس‌ها عوض شود، بدون رندر مجدد، آیتم‌ها روی صفحه جابجا (Move) می‌شوند.

اگر اطلاعاتی تغییر نکرده باشد، هیچ دستوری به لایه گرافیک صادر نمی‌شود که این یعنی بازدهی حداکثری برنامه.

حالا به آخرین و بالاترین لایه، یعنی مدیریت چرخه عمر (Lifecycle) و جمع‌آوری داده‌ها (Data Collection) در فرگمنت می‌رسیم. این بخش مانند ایستگاه کنترل مرکزی عمل می‌کند؛ جایی که فرگمنت گوش به زنگ می‌نشیند تا به محض آماده شدن داده‌ها در ویومدل، آن‌ها را تحویل گرفته و به آداپتر بسپارد.

کد این بخش در متد onViewCreated فرگمنت قرار دارد.

لیست ۲۳.۲۳ افزودن یک آداپتور (PhotoGalleryFragment.kt)

```
class PhotoGalleryFragment : Fragment() {
    private val photoAdapter = PhotoAdapter()
    override fun onCreateView(
        …
    ): View {
        …
        photoRecyclerView.layoutManager =
        GridLayoutManager(context, 3)
        photoRecyclerView.adapter = photoAdapter
        return view
    }
    ...
    override fun onViewCreated(view: View
    , savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState)
```

```
        viewLifecycleOwner.lifecycleScope.launch {
            viewLifecycleOwner.repeatOnLifecycle(
            Lifecycle.State.STARTED) {
                 photoGalleryViewModel.unsplashPhotos
                 .collec { unsplashPhotos ->
                    if (unsplashPhotos.isNotEmpty()) {
                        Log.d(TAG, "Data received in UI
                        from ViewModel:
                        ${unsplashPhotos.size} photos")
                        photoAdapter.submitList(
                        unsplashPhotos)
                    }
                }
            }
        }
    }
}
```

با انجام این مورد، کار شما برای این فصل کامل به پایان رسید. PhotoGallery را اجرا کنید و باید متن نمایش داده شده را برای هر UnsplashPhoto که دانلود کرده‌اید را ببینید (مشابه شکل ۲۴.۲).

بیشتر بدانیم: تجزیه کننده‌های جایگزین و فرمت‌های داده

Gson یک تجزیه کننده محبوب برای JSON است. اما می‌توان از موارد دیگری نیز استفاده کرد. Moshi (github.com/square/moshi) یکی دیگر از کتابخانه‌های محبوب Square است که تجزیه داده‌های JSON را انجام می‌دهد. Moshi ایده‌هایی را از Gson قرض و سعی می‌کند آنها را سریع تر و کارآمدتر کند. Square یک مبدل برای Moshi ایجاد کرده تا بتواند مستقیما با Retrofit کار کند.

Retrofit شما را فقط به استفاده از JSON به عنوان فرمت داده محدود نمی‌کند. در صورت تمایل می‌توانید از XML یا حتی Protobufs نیز استفاده کنید. کتابخانه‌های تجزیه زیادی برای Retrofit در دسترس هستند، بنابراین می‌توانید از کتابخانه‌ای استفاده کنید که به بهترین وجه با پیکربندی انتقال داده شما مطابقت دارد.

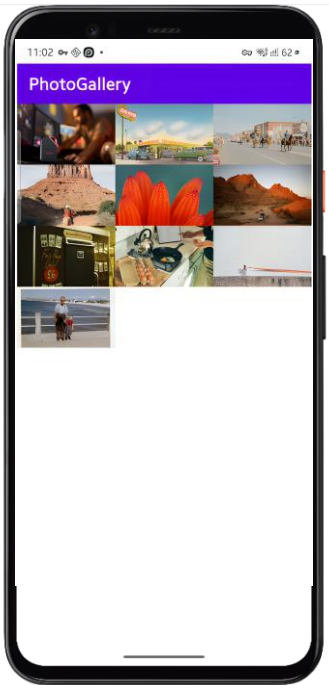
۲۴

نمایش تصاویر در RecyclerView

اکنون که داده‌های JSON را از Unsplash دانلود و تجزیه کرده‌ایم، وظیفه بعدی ما دانلود و نمایش تصاویر است. در این فصل یاد خواهیم گرفت که کتابخانه Coil چیست و چگونگی استفاده از آن را برای دانلود و نمایش پویا عکس‌ها در PhotoGallery خواهیم دید.

در پایان این فصل برنامه ما همانند تصویر ۲۴.۱ تصاویر دانلود شده را نمایش خواهد داد.

تصویر ۲۴.۱ تصاویر Unsplash



Coil چیست و دقیقاً چه کاربردی دارد؟

کتابخانه‌ی Coil مخفف Coroutine Image Loader است. این کتابخانه در سال ۲۰۱۹ توسط شرکت Instacart معرفی شد و به سرعت به استاندارد رسمی و محبوب اندروید تبدیل گردید، تا جایی که امروزه در ابزارهای مدرنی مثل Jetpack Compose حرف اول را می‌زند.

کاربرد اصلی Coil، مدیریت صفر تا صد چرخه بارگذاری تصاویر در اندروید است. شاید در ظاهر فکر کنیم بارگذاری یک عکس ساده است (یک آدرس اینترنتی می‌گیریم و روی صفحه نشان می‌دهیم)، اما در واقعیت، این فرآیند یکی از سنگین‌ترین و چالش‌برانگیزترین کارهای مهندسی در موبایل است. زمانی که شما کدی مانند imageView.load("https://unsplash.com/... ") را می‌نویسید، Coil وظایف زیر را با جزئیات کامل مدیریت می‌کند:

- درخواست شبکه ناهمگام (Async Network Request): بدون اینکه نخ اصلی (UI) را حتی برای یک میلی‌ثانیه قفل کند، درخواست دانلود عکس را به لایه شبکه می‌فرستد.
- رمزگشایی هوشمند (Smart Decoding): بایت‌های خامِ دانلود شده از اینترنت (مثل فرمت‌های JPEG، PNG یا WebP) را به بیت‌مپ (Bitmap) تبدیل می‌کند. بیت‌مپ مجموعه‌ای از پیکسل‌هاست که پردازنده گرافیکی گوشی (GPU) می‌تواند آن را روی صفحه رندر کند.
- تغییر سایز در حافظه (Downsampling): این بخش حیاتی‌ترین کار Coil است. اگر یک عکس با کیفیت 4K (مثلاً با حجم ۵ مگابایت) از Unsplash دانلود شود، Coil نگاه می‌کند که سایز ImageView شما روی صفحه چقدر است (مثلاً 300*300 پیکسل). سپس عکس را دقیقاً به اندازه نیاز کوچک و دکود می‌کند. این کار جلوه خطای معروف و ترسناک کمبود حافظه یا همان OutOfMemoryError (OOM) را می‌گیرد.
- مدیریت انیمیشن و لایه‌ها: نمایش یک تصویر موقت (Placeholder) تا زمان دانلود، نمایش تصویر خطا در صورت قطع بودن اینترنت، و اعمال انیمیشن نرمِ محو شدن و ظاهر شدن تصویر (Crossfade).

چرا باید در برنامه‌های خود از Coil استفاده کنیم؟

اگر Coil را کنار بگذاریم و بخواهیم خودمان با کدهای سنتی (مثل HttpURLConnection یا حتی Looper/Handler) عکس‌ها را لود کنیم، با چالش‌های بزرگی روبرو می‌شویم. دلایل اصلی انتخاب Coil عبارتند از:

۱. معماری بومی بر پایه کاتلین و کوروتین‌ها (Kotlin-First): کتابخانه‌های قدیمی‌تر (مانند Glide یا Picasso) با زبان جاوا نوشته شده‌اند. اما Coil از پایه با کاتلین طراحی شده است. Coil از کوروتین‌ها (Coroutines) برای مدیریت تردها استفاده می‌کند. این یعنی:

- هماهنگی بی‌نظیر با کدهای مدرن شما در ViewModel و Fragment.
- استفاده از قابلیت‌های پیشرفته کاتلین مثل Extension Functions که باعث می‌شود کل عملیات دانلود فقط در یک متد ()load خلاصه شود.

۲. سیستم کِش دوگانه (Dual Caching System): Coil برای اینکه اینترنت کاربر بیهوده مصرف نشود و سرعت برنامه بالا برود، از دو لایه کش فوق‌العاده استفاده می‌کند:

- کِش حافظه (Memory Cache): عکس‌هایی که در حال حاضر روی صفحه هستند یا کاربر تازه آن‌ها را دیده، در رم (RAM) گوشی ذخیره می‌شوند. اگر کاربر اسکرول کند و بالا برود، عکس‌ها در کسری از میلی‌ثانیه و بدون لودینگ دوباره ظاهر می‌شوند.
- کِش دیسک (Disk Cache): عکس‌ها را در حافظه داخلی گوشی ذخیره می‌کند. اگر کاربر برنامه را کاملاً ببندد و فردا دوباره باز کند، تصاویر بدون نیاز به دانلود مجدد از اینترنت، از روی هارد گوشی لود می‌شوند.

۳. مدیریت خودکار چرخه عمر (Lifecycle Awareness): یکی از کابوس‌های اندروید این است که یک عکس در حال دانلود باشد و کاربر ناگهان دکمه بازگشت (Back) را بزند و از صفحه خارج شود. در کدهای قدیمی، این اتفاق باعث نشت حافظه (Memory Leak) یا کرش برنامه می‌شد. کُویل به Lifecycle فرگمنت یا اکتیویتی متصل می‌شود. اگر ویو نابود شود، کُویل فوراً و به صورت خودکار درخواست دانلود شبکه را لغو (Cancel) می‌کند و منابع حافظه را آزاد می‌سازد.

۴. فوق‌العاده سبک و بهینه (Lightweight): کوئیل بسیار کم‌حجم است. این کتابخانه تعداد متدهای بسیار کمتری را به فایل نهایی APK شما اضافه می‌کند (حدود ۱۵۰۰ متد در برابر چندین هزار متد کتابخانه‌های دیگر). این موضوع برای توسعه‌دهندگانی که روی حجم نهایی برنامه حساس هستند، یک برگ برنده است.

جریان حرکت یک عکس در قلب Coil

برای اینکه بدانید وقتی یک آدرس وب را به Coil می‌دهید چه مراحلی طی می‌شود، این ساختار زنجیره‌ای را در نظر بگیرید:

۱. بررسی حافظه موقت (Memory Cache): Coil ابتدا چک می‌کند که آیا این عکس قبلاً دانلود شده و در رم (RAM) موجود است؟ اگر بله، فوراً رندر می‌شود.

۲. بررسی حافظه داخلی (Disk Cache): اگر در رم نبود، حافظه گوشی را چک می‌کند. در صورت وجود، عکس دکود شده و به لایه نمایش می‌رود.

۳. درخواست آنلاین و دانلود (Network): اگر عکس کاملاً جدید باشد، کوروتینِ Coil روی Dispatchers.IO فعال شده و با کمک کتابخانه OkHttp بایت‌های عکس را از اینترنت دانلود می‌کند.

۴. بهینه‌سازی سایز (Downsampling): بایت‌های خام متناسب با ابعاد کامپوننت گرافیکی شما برش و سایز داده می‌شوند تا رم گوشی پر نشود.

۵. رندر نهایی بر روی نخ اصلی (UI): با Handler مخفی شده در Dispatchers.Main، بیت‌مپ نهایی به ImageView تحویل داده می‌شود تا کاربر عکس را ببیند.

به زبان ساده، Coil تمام پیچیدگی‌های مدیریت لایه شبکه، محاسبات سنگین ریاضیِ بیت‌مپ‌ها، مدیریت حافظه رم و هماهنگی با چرخه عمر اندروید را از روی دوش شما برمی‌دارد تا شما فقط روی منطق اصلی برنامه‌تان تمرکز کنید.

آماده سازی برای نمایش تصاویر

حالا با اطلاعاتی که در اختیار داریم وقت آن است که Coil را در برنامه خود به کار بگیریم. ابتدا وابستگی کتابخانه Coil را به فایل build.gradle پروژه اضافه کنید.

لیست ۲۴.۱ وابستگی Coil (build.gradle)

```
dependencies {
    …
    implementation("androidx.fragment:fragment-ktx:1.8.9")
    implementation("io.coil-kt:coil:2.6.0")
    …
}
```

در فصل قبل، ما فقط متن را نشان می‌دادیم. حالا لایه متنی را به یک تصویر واقعی (ImageView) تغییر می‌دهیم. ابتدا کلاس PhotoHolder را بازنویسی می‌کنیم تا با استفاده از Coil، عکس را مستقیم از آدرس وب‌سایت Unsplash دانلود و رندر کند.

لیست ۲۴.۲ به روز رسانی PhotoHolder (PhotoGalleryFragment.kt)

```
class PhotoGalleryFragment : Fragment() {
    …
    companion object {
        …
    }
    private class PhotoHolder(private val ~~itemTextView~~
    imageview: ~~TextView~~ ImageView) :
        RecyclerView.ViewHolder(~~itemTextView~~ imageview) {
        fun bind(photo: UnsplashPhoto) {
            ~~itemTextView.text = photo.title ?:~~
            ~~"No Title: ${photo.id})"~~
            imageview.load(photo.url.small) {
                crossfade(true)
                placeholder(android.R.drawable
                .progress_horizontal)
                error(android.R.drawable.stat_notify_error)
                memoryCachePolicy(CachePolicy.ENABLED)
                diskCachePolicy(CachePolicy.ENABLED)
            }
        }
    }
    …
}
```

وقتی متد bind توسط آداپتر صدا زده می‌شود، این بلاک با جزئیات زیر شروع به کار می‌کند:

۱. متد الحاقی imageView.load(photo.url): این متد یک Extension Function (تابع الحاقی) کاتلینی است که روی شیء ImageView سوار شده است. Coil با این دستور، بلافاصله چرخه عمر ویو را در دست می‌گیرد. آدرس اینترنتی عکس (photo.url.small) را به عنوان کلید منحصربه‌فردِ این تسک ذخیره کرده و آماده هماهنگی با لایه‌های زیرین (کوروتین‌ها) می‌شود.

۲. انیمیشن محو شدن و پدیدار شدن crossfade(true): بدون این خط، به محض دانلود شدن عکس، تصویر ناگهان روی صفحه ظاهر می‌شود که ظاهر چندان خوشایندی ندارد (یک شوک بصری به کاربر وارد می‌کند). کُویل با فعال بودن این فلگ، از یک لایه‌ی متحرک (Transition) استفاده می‌کند. به محض آماده شدن عکس، تصویر پس‌زمینه قدیمی یا لودینگ را در مدت

زمان کوتاهی (به صورت پیش‌فرض ۱۰۰ تا ۲۰۰ میلی‌ثانیه) محو کرده و تصویر جدید را به آرامی نمایان می‌کند.

۳. مدیریت وضعیت لودینگ placeholder(…): اینترنت هر چقدر هم سریع باشد، چند میلی‌ثانیه یا چند ثانیه طول می‌کشد تا عکس از سرورهای Unsplash دانلود شود. خالی گذاشتن کادر در این مدت حس خرابی برنامه را به کاربر می‌دهد. Coil تا قبل از رسیدن بایت‌های اصلی عکس از اینترنت، این تصویر پیش‌فرض (که اینجا نوار پیشرفت افقی اندروید است) را روی صفحه رندر نگه می‌دارد.

۴. مدیریت وضعیت خطا error(…): اگر لنگر کشتی اینترنت قطع شود، سرور Unsplash پاسخ ندهد یا آدرس عکس خراب باشد، این متد وارد عمل می‌شود. کُویل به صورت خودکار متوجه خطای شبکه یا دکودینگ می‌شود، عملیات دانلود را متوقف کرده و آیکون خطا را جایگزین می‌کند تا کاربر متوجه شود بارگذاری با مشکل مواجه شده است.

۵. تنظیمات سیاست کِش دوگانه (CachePolicy.ENABLED): این دو خط، ضامن سرعت کهکشانی گالری شما در اسکرول‌های بعدی هستند. memoryCachePolicy به Coil اجازه می‌دهد به محض دانلود شدن عکس، نسخه باکیفیت و آماده‌ی رندرِ آن را در حافظه موقت (RAM) نگه دارد. با اسکرول مجدد کاربر، لودینگ حذف شده و عکس در یک میکروثانیه ظاهر می‌شود. diskCachePolicy به Coil دستور می‌دهد فایل فشرده‌شده عکس را در حافظه دائمی (Disk) ذخیره کند تا اگر کاربر کلاً از برنامه خارج شد و فردا بازگشت، بدون مصرف حتی یک کیلوبایت اینترنت، عکس‌ها از روی لایه هارد گوشی لود شوند.

حالا آداپتر را به جای TextView مجهز به یک ImageView می‌کنیم تا برای نمایش عکس‌های گالری آماده شود.

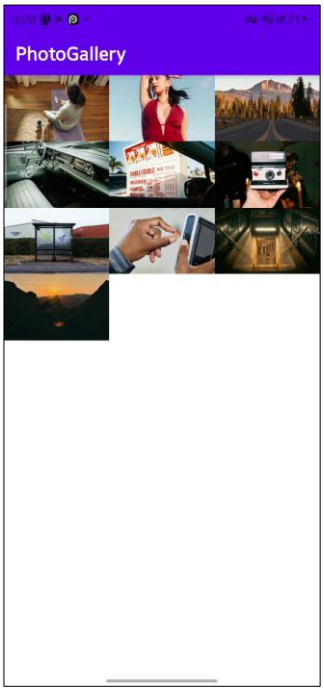
لیست ۲۴.۳ به روز رسانی OnCreateViewHolder (PhotoGalleryFragment.kt)

```
class PhotoGalleryFragment : Fragment() {
    …
    private class PhotoAdapter : ListAdapter<UnsplashPhoto
    , PhotoHolder>(PhotoDiffCallback()) {
        override fun onCreateViewHolder(parent: ViewGroup
        , viewType: Int): PhotoHolder {
            val textView = TextView(parent.context).apply {
                setPadding(16, 16, 16, 16)
            }
            val imageView = ImageView(parent.context).apply {
                layoutParams = ViewGroup.LayoutParams(
```

```
                    ViewGroup.LayoutParams.MATCH_PARENT,
                    300
                )
                scaleType = ImageView.ScaleType.CENTER_CROP
            }
            return PhotoHolder(textView)
            return PhotoHolder(imageView)
        }
        override fun onBindViewHolder(holder: PhotoHolder
        , position: Int) {
            val unsplashPhoto = getItem(position)
            holder.bind(unsplashPhoto)
        }
    }
```

PhotoGallery را اجرا و برای دیدن تصاویر پیمایش کنید (تصویر ۲۴.۲).

تصویر ۲۴.۲ تصاویر نمایش داده شده



تقریبا کارمان تمام است، اگر یک مورد دیگر را حل کنیم و آن هم کنترل خطاهای برنامه مان است.

در حال حاضر، کلاس UnsplashFetcher در صورت بروز خطا، یک لیست خالی (emptyList) به سمت ویومدل می‌فرستد (emit(emptyList())). این کار باعث می‌شود که ویومدل و فرگمنت متوجه نوع خطا (مثلاً قطعی اینترنت در برابر فیلتر یا در دسترس نبودن سرور) نشوند و صرفاً یک صفحه خالی بدون هیچ پیغامی به کاربر نشان داده شود.

برای اینکه بتوانیم خطاها را به صورت کاملاً تفکیک‌شده و ساختاریافته به ویومدل و سپس فرگمنت برگردانیم، بهترین کار این است که متد fetchPhotos را به جای ارسال لیست خام، مجهز به یک ساختار نتیجه‌محور (Result-based) کنیم.

متد fetchPhotos در کلاس UnsplashFetcher را مانند لیست زیر اصلاح کنید.

لیست ۲۴.۴ اصلاح fetchPhotos (UnsplashFetcher.kt)

```
class UnsplashFetcher {
    private val unsplashAPI: UnsplashAPI
    init {…}
    fun fetchPhotos(): Flow<List<UnsplashPhoto>> = flow {
        try {
            Log.d(TAG, "Fetching popular photos
            from Unsplash...")
            val fetchedPhotos = unsplashAPI.fetchPhotos()
            val filteredPhotos = fetchedPhotos.filter
            { it.url.small.isNotBlank() }
            emit(filteredPhotos)
        } catch (e: Exception) {
            Log.e(TAG, "Failed to fetch or parse photos", e)
            emit(emptyList()filteredPhotos)
        }
    }
}
```

چه تغییراتی اعمال شد و چرا؟

حذف بلوک try-catch داخلی جهت جلوگیری از خفه کردن خطا. چون اگر اینجا catch بنویسیم و لیست خالی بفرستیم، ویومدل هیچ راهی برای فهمیدن اینکه «خطا رخ داده» ندارد. با حذف این بلوک، خطا مستقیماً به لایه بالاتر (ViewModel) پاس داده می‌شود تا ویومدل بتواند لایه گرافیکی (UI) را روی حالت خطا ببرد.

در ویومدل جریان داده‌ها با اپراتور stateIn مستقیماً به یک StateFlow<List<UnsplashPhoto>> تبدیل می‌شود. این ساختار برای حالت‌های موفقیت عالی است، اما دو چالش جدی دارد:

۱. مقدار اولیه همیشه یک لیست خالی (emptyList) است؛ پس فرگمنت نمی‌تواند تفاوت بین "در حال لود شدن" و "لیست واقعاً خالی است" را تشخیص دهد.

۲. اگر در UnsplashFetcher خطایی رخ دهد (که در گام قبل تنظیم کردیم خطا پرتاب شود)، این خطا جریان Flow را می‌کشند و برنامه دیگر کار نمی‌کند، یا اصلاً فرگمنت متوجه رخ دادن خطا نمی‌شود.

برای حل این مشکل، ابتدا یک کلاس وضعیت (جهت پشتیبانی از لودینگ، موفقیت و خطا) می‌سازیم و سپس ویومدل را بازنویسی می‌کنیم تا خطاها را شکار کرده و به فرگمنت مخابره کند.

ساخت کلاس وضعیت (PhotoUiState)

این اینترفیس را در یک فایل جدید به نام PhotoUiState.kt در پکیج api قرار دهید تا وضعیت‌های صفحه را مشخص کند.

لیست ۲۴.۵ وضعیت‌های صفحه (PhotoUiState.kt)

```
sealed interface PhotoUiState {
    object Loading : PhotoUiState
    data class Success(val photos: List<UnsplashPhoto>)
    : PhotoUiState
    data class Error(val exception: Throwable) : PhotoUiState
}
```

حالا کلاس PhotoGalleryViewModel را به این صورت بازنویسی می‌کنیم. با استفاده از اپراتور مدرن map داده‌های موفق را بسته‌بندی می‌کنیم و با اپراتور catch جلوی کرش را گرفته و خطای شبکه را به فرگمنت هدایت می‌کنیم.

لیست ۲۴.۶ هدایت خطای شبکه (PhotoGalleryViewModel.kt)

```
class PhotoGalleryViewModel : ViewModel() {
    private val unsplashFetcher = UnsplashFetcher()
    val uiState: StateFlow<PhotoUiState> = unsplashFetcher
        .fetchPhotos()
        .map<List<UnsplashPhoto>, PhotoUiState>
        { photos ->
            PhotoUiState.Success(photos)
        }
        .catch { exception ->
            emit(PhotoUiState.Error(exception))
        }
```

```
        .stateIn(
            scope = viewModelScope,
            started = SharingStarted.WhileSubscribed(5000),
            initialValue = PhotoUiState.Loading
        )
}
```

این معماری چطور کار می‌کند؟

initialValue = PhotoUiState.Loading: به محض اینکه فرگمنت صفحه را باز کند، این وضعیت را دریافت می‌کند و می‌فهمد که باید چرخ دنده لودینگ را بچرخاند.

اپراتور map: لیست تصاویر شما را می‌گیرد و آن را درون کپسولِ قانونمندِ PhotoUiState.Success تزریق می‌کند.

اپراتور catch: اگر سرور در دسترس نباشد، اینترنت قطع باشد یا کدهای احراز هویت Unsplash منقضی شده باشند، خطا بدون متوقف کردن برنامه در قالب کپسول PhotoUiState.Error به فرگمنت ارسال می‌شود.

به آخرین ایستگاه یعنی اصلاح فایل PhotoGalleryFragment رسیدیم. در حال حاضر، فرگمنت شما دیتای خود را از unsplashPhotos می‌گیرد که در گام قبل آن را به ساختار بهینه‌ترِ uiState تغییر دادیم. همچنین برای نشان دادن وضعیت‌های خطای عدم دسترسی به سرور یا اینترنت، باید چک کنیم چه وضعیتی از سمت ویومدل فرستاده شده است.

از آنجا که در این فایل از کدهای لایه‌بندی گرافیکی استفاده نکرده‌اید، ما روش استفاده از Toast و مدیریت پیام‌های خطا با توجه به نوع استثنا (خطای اینترنت یا سرور) را در کدها پیاده‌سازی می‌کنیم.

اول onViewCreated(…) را به صورت زیر بازنویسی کنید.

لیست ۲۴.۷ لاگ خطای شبکه (PhotoGalleryFragment.kt)

```
override fun onViewCreated(view: View, savedInstanceState:
Bundle?) {
    super.onViewCreated(view, savedInstanceState)
    viewLifecycleOwner.lifecycleScope.launch {
        viewLifecycleOwner.repeatOnLifecycle
        (Lifecycle.State.STARTED) {
            photoGalleryViewModel.~~unsplashPhotos~~ uiState
            .collect
            { ~~unsplashPhotos~~ state ->
                ~~if (unsplashPhotos.isNotEmpty()) {~~
                    ~~Log.d(TAG,~~
                    ~~"Data:${unsplashPhotos.size} photos")~~
```

```
                            photoAdapter.submitList(
                            unsplashPhotos)
                        }
                when (state) {
                    is PhotoUiState.Loading -> {
                        Log.d(TAG, "Loading photos...")
                    }
                    is PhotoUiState.Success -> {
                        Log.d(TAG,
                        "Data:${state.photos.size}photos")
                        photoAdapter.submitList(state.photos)
                    }
                    is PhotoUiState.Error -> {
                        handleError(state.exception)
                    }
                }
            }
        }
    }
}
```

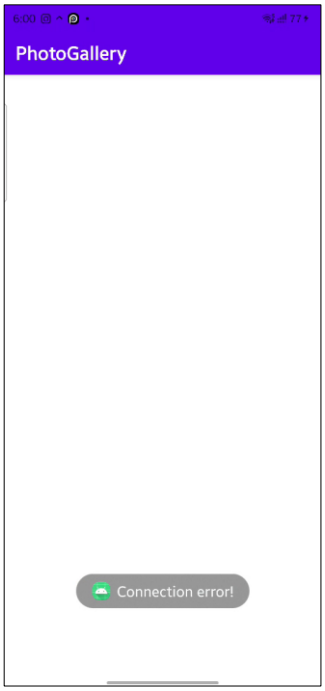
سپس متد handleError را برای آنالیز و ترجمه خطای لایه شبکه به پیام‌های کاربرپسند پیاده کنید.

لیست ۲۴.۸ ترجمه خطای لایه شبکه (PhotoGalleryFragment.kt)

```
class PhotoGalleryFragment : Fragment() {
    ...
    private fun handleError(throwable: Throwable) {
        val errorMessage = when (throwable) {
            is IOException -> {
                "Connection error!"
            }
            is HttpException -> {
                "The server is currently not responding"
            }
            else -> {
                "An unspecified error occurred"
            }
        }
        Toast.makeText(requireContext(), errorMessage
            , Toast.LENGTH_LONG).show()
    }
    ...
}
```

حالا برنامه را اجرا کنید. اینترنت را قطع کنید تا شاهد خطایی همانند تصویر ۲۴.۳ باشید.

تصویر ۲۴.۳ خطای عدم ارتباط شبکه



این فصل در اینجا به پایان می رسد. در چند فصل بعدی، عملکردهای بیشتری مانند جستجوی عکس‌ها و باز کردن صفحه Unsplash هر عکس در یک ویوی وب اضافه خواهید کرد.

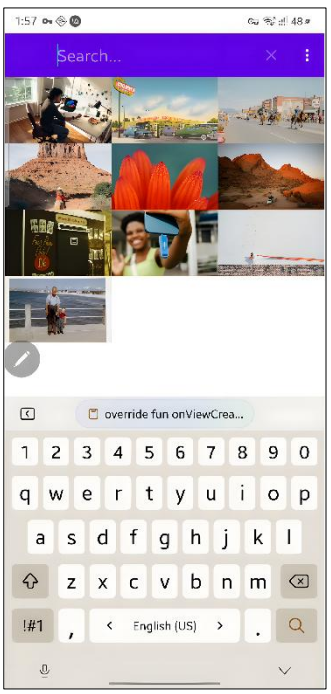
۲۵

SearchView و SharedPreferences

وظیفه بعدی ما با PhotoGallery جستجوی عکس‌ها در Unsplash است. ما یاد خواهید گرفت که چگونه جستجو را با استفاده از SearchView در اپلیکیشن خود ادغام کنیم. SearchView یک کلاس action view است که می‌تواند مستقیما در نوار ابزار جاسازی شود. همچنین یاد خواهید گرفت که چگونه به راحتی داده‌ها را با استفاده از SharedPreferences در فایل سیستم دستگاه ذخیره کنید.

در پایان این فصل، کاربر می‌تواند SearchView را لمس کرده، یک رشته کوئری را تایپ و آن را ارسال کند. ارسال کوئری، رشته کوئری را به API جستجوی Unsplash ارسال و RecyclerView را با نتایج جستجو پر می‌کند (شکل ۲۵.۱). خود رشته کوئری در سیستم فایل باقی می‌ماند. این بدان معناست که آخرین کوئری کاربر در هنگام راه اندازی مجدد برنامه و حتی دستگاه قابل دسترسی خواهد بود.

تصویر ۲۵.۱ پیش‌نمایش برنامه



جستجوی Unsplash

بیایید از سمت Unsplash شروع کنیم. در اینجا درخواست GET برای جستجوی متن "cat" پیاده شده:

```
https://api.unsplash.com/search/photos?
query=cat&client_id=YOUR_ACCESS_KEY
```

پارامتر query روی هر رشته‌ای که به دنبال آن هستیم تنظیم شده است (در این مورد "cat").

برخی از جفت‌های پارامتر–مقدار، مانند client_id=YOUR_ACCESS_KEY، در هر دو URL درخواست api.unsplash.com/search و api.unsplash.com/photos ثابت هستند. ما می‌خواهیم این جفت پارامتر–مقدار مشترک را به یک interceptor (شنودکننده) انتزاع کنیم. یک interceptor آنچه را که ممکن است انتظار داشته باشید انجام می‌دهد – یک درخواست یا پاسخ را رهگیری و به شما امکان می‌دهد محتویات را قبل از تکمیل درخواست یا پاسخ دستکاری کنید یا اقداماتی انجام دهید.

یک کلاس Interceptor جدید با نام PhotoInterceptor در پوشه API خود ایجاد کنید. برای دسترسی به یک درخواست (intercept(chain را بازنویسی کنید، جفت‌های پارامتر–مقدار مشترک را به آن اضافه کنید و URL اصلی را با URL تازه ساخته شده بازنویسی کنید.

لیست ۲۵.۱ افزودن یک رهگیر برای درج ثابت‌های URL (api/PhotoInterceptor.kt)

```
package com.codeb.photogallery.api
import okhttp3.HttpUrl
import okhttp3.Interceptor
import okhttp3.Request
import okhttp3.Response
private const val API_KEY =
    "aZarqhgznXuHnuSxAMVTYtE7UnrMyEATWQJHdhJUiUU"
class PhotoInterceptor : Interceptor {
    override fun intercept(chain: Interceptor.Chain)
    : Response {
        val originalRequest = chain.request()
        val newUrl: HttpUrl = originalRequest.url
        .newBuilder()
            .addQueryParameter("client_id", API_KEY)
            .addQueryParameter("order_by", "popular")
            .build()
        val newRequest: Request =
        originalRequest.newBuilder()
            .url(newUrl)
            .build()
        return chain.proceed(newRequest)
    }
}
```

اندروید استودیو هنگام ایمپورت کردن Request و Response، گزینه‌های متعددی را به شما ارائه می‌دهد. گزینه okhttp3 در هر دو مورد انتخاب کنید.

برای دسترسی به درخواست اصلی، ()chain.request را فراخوانی می‌کنیم. ()originalRequest.url، URL اصلی را از درخواست بیرون می‌کشد و ما از HttpUrl.Builder برای افزودن پارامترهای کوئری به آن استفاده می‌کنیم.

HttpUrl.Builder یک درخواست جدید بر اساس درخواست اصلی ایجاد می‌کند و URL اصلی را با URL جدید بازنویسی می‌کند. در نهایت chain.proceed(newRequest) را برای تولید یک پاسخ فراخوانی می‌کنیم. اگر chain.proceed(…) را فراخوان نشود، درخواست شبکه اتفاق نمی‌افتد.

اکنون UnsplashFetcher.kt را باز و interceptor را به پیکربندی Retrofit خود اضافه کنید.

لیست ۲۵.۲ افزودن یک interceptor به پیکربندی Retrofit خود (UnsplashFetcher.kt)

```
class UnsplashFetcher {
    private val unsplashAPI: UnsplashAPI
    init {
        val okHttpClient = OkHttpClient.Builder()
            .addInterceptor(PhotoInterceptor())
            .build()
        val retrofit: Retrofit = Retrofit.Builder()
            .baseUrl("https://api.unsplash.com/")
            .addConverterFactory
            (GsonConverterFactory.create())
            .client(okHttpClient)
            .build()
        unsplashAPI =
        retrofit.create(UnsplashAPI::class.java)
    }
    fun fetchPhotos(): Flow<List<UnsplashPhoto>> = flow {
        …
    }
}
```

ما یک نمونه OkHttpClient ایجاد و PhotoInterceptor را به عنوان یک interceptor اضافه می‌کنیم. سپس کلاینت تازه پیکربندی شده را روی نمونه Retrofit خود تنظیم می‌کنیم که جایگزین کلاینت پیش‌فرضی می‌شود که قبلا در جای خود بوده است. اکنون Retrofit از کلاینتی که ارائه کرده‌اید استفاده می‌کند و به نوبه خود PhotoInterceptor.intercept() را به هر درخواستی که انجام می‌شود، اعمال می‌کند.

دیگر نیازی به api.unsplash.com/photos مشخص شده در UnsplashAPI نداریم. آن را پاک کنید و در عوض، یک تابع searchPhotos() اضافه کنید تا یک درخواست جستجو را به پیکربندی Retrofit API خود تعریف کنید.

لیست ۲۵.۳ افزودن تابع جستجو به UnsplashAPI (api/UnsplashAPI.kt)

```
package com.codeb.photogallery.api

import com.codeb.photogallery.UnsplashPhoto
import com.google.gson.annotations.SerializedName
import retrofit2.http.GET
import retrofit2.http.Query

interface UnsplashAPI {
    @GET(
        "photos" +
                "?order_by=popular" +
                "&client_id="+
                "aZarqhgznXuHnuSxAJVTYtE7UnrMyEATWQJHdhJUiUU"
    )
    suspend fun fetchPhotos(): List<UnsplashPhoto>
    @GET("photos")
    suspend fun fetchPhotos(): List<UnsplashPhoto>
    @GET("search/photos")
    suspend fun searchPhotos(@Query("query")
        searchTerm: String): UnsplashSearchResponse
    data class UnsplashSearchResponse(
        @SerializedName("results") val results
        : List<UnsplashPhoto>
    )
}
```

حاشیه‌نویسی Query@ به ما امکان می‌دهد به صورت پویا یک پارامتر کوئری را به URL اضافه کنیم. دقت کنید یه کلاس مدل به UnsplashAPI اضافه کردیم زیرا نوع خروجی متد (ReturnType) باید دقیقاً با ساختار JSON که سرور Unsplash برمی‌گرداند همخوانی داشته باشد. اندپوینت "photos" مستقیم یک لیست برمی‌گرداند؛ پس خروجی List<UnsplashPhoto> برای آن کاملاً درست است.

اما اندپوینت "search/photos" پاسخ را درون یک پکیج یا کپسول حاوی فیلدهای total و results می‌فرستد؛ پس خروجی آن نمی‌تواند مستقیم List<UnsplashPhoto> باشد و رتروفیت در اینجا کرش می‌کند یا فیلدها را نمی‌شناسد. برای حل این مشکل، ابتدا باید کلاس مدل کپسول پاسخ سرور را تعریف کنیم و سپس خروجی متد دوم را اصلاح کنیم.

یک تابع جستجو به UnsplashFetcher اضافه کنید.

لیست ۲۵.۴ افزودن یک تابع جستجو به UnsplashFetcher

```
class UnsplashFetcher {
    ...
    fun searchPhotos(query: String):
        Flow<List<UnsplashPhoto>> = flow {
        Log.d(TAG, "Searching photos for query: $query")
        val searchResponse = unsplashAPI.searchPhotos(query)
        val fetchedPhotos = searchResponse.results
        val filteredPhotos = fetchedPhotos.filter {
        it.url.small.isNotBlank() }
        emit(filteredPhotos)
    }
}
```

در نهایت PhotoGalleryViewModel را به روز کنید تا جستجوی Unsplash را شروع کند. این فایل یک چالش پیدا می‌کند: این مدلِ جریان دیتا (Static Flow)، قابلیت دریافت کلمه کلیدی (query) از سمت کاربر و آپدیت شدن مجدد را ندارد.

برای اینکه هم ساختار پیشرفته و Cold-to-Hot اپراتور stateIn را حفظ کنیم و هم بتوانیم قابلیت جستجو (Search) را در همین فایل پیاده کنیم، باید از یک MutableStateFlow برای کلمه کلیدی استفاده کنیم و با اپراتور فوق‌العاده‌ی flatMapLatest هر زمان کلمه کلیدی تغییر کرد، فلو شبکه را مجدداً به کار بیندازیم.

پس نسخه نهایی PhotoGalleryViewModel باید همانند لیست ۲۵.۵ باشد.

لیست ۲۵.۵ شروع درخواست جستجو (PhotoGalleryViewModel.kt)

```
package com.codeb.photogallery

import androidx.lifecycle.ViewModel
import androidx.lifecycle.viewModelScope
import kotlinx.coroutines.ExperimentalCoroutinesApi
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.SharingStarted
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.flatMapLatest
import kotlinx.coroutines.flow.flow
import kotlinx.coroutines.flow.stateIn

class PhotoGalleryViewModel : ViewModel() {
    private val unsplashFetcher = UnsplashFetcher()
    private val _searchQuery = MutableStateFlow("")
    fun setQuery(query: String) {
        _searchQuery.value = query
```

```
    }
    @OptIn(ExperimentalCoroutinesApi::class)
    val uiState: StateFlow<PhotoUiState> = _searchQuery
        .flatMapLatest { query ->
            flow {
                emit(PhotoUiState.Loading)
                try {
                    val photos = if (query.isBlank()) {
                        unsplashFetcher.fetchPhotos()
                    } else {
                        unsplashFetcher.searchPhotos(query)
                    }
                    photos.collect {
                    emit(PhotoUiState.Success(it)) }
                } catch (e: Exception) {
                    emit(PhotoUiState.Error(e))
                }
            }
        }
        .stateIn(
            scope = viewModelScope,
            started = SharingStarted.WhileSubscribed(5000),
            initialValue = PhotoUiState.Loading
        )
}
```

چرا این روش، پادشاه معماری در کاتلین است؟

flatMapLatest به شدت هوشمند است. اگر کاربر تایپ کند "cat" و بلافاصله پشیمان شود و سرچ را پاک کند، این اپراتور درخواست شبکه‌ی قبلی را وسط راه لغو (Cancel) می‌کند و بلافاصله فلو جدید را استارت می‌زند تا پهنای باند و باتری گوشی هدر نرود.

فرگمنت شما همچنان نیازی به تغییر ندارد! همان کدهای قبلی فرگمنت که uiState را واکشی می‌کردند بدون کوچک‌ترین دستکاری کار خواهند کرد. فقط وقتی کاربر دکمه سرچ را زد، فرگمنت متد viewModel.setQuery("cat") را صدا می‌زند. اکنون که UnsplashFetcher از جستجو پشتیبانی می‌کند، زمان آن رسیده است که با افزودن SearchView راهی برای آغاز جستجو و اضافه کردن رشته کوئری توسط کاربر بوجود آوریم.

همانطور که در ابتدای فصل گفتیم، SearchView یک action view است. به این معنی که کل رابط جستجوی ما می‌تواند در tool bar برنامه قرار گیرد.

یک فایل XML منوی جدید به نام res/menu/fragment_photo_gallery.xml ایجاد کنید. این فایل مواردی را که باید در نوار ابزار ظاهر شوند مشخص می‌کند.

لیست ۲۵.۶ افزودن یک فایل XML منو (res/menu/fragment_photo_gallery.xml)

```
<?xml version="1.0" encoding="utf-8"?>
<menu
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto">
    <item android:id="@+id/menu_item_search"
        android:title="@string/search"
        app:actionViewClass=
        "androidx.appcompat.widget.SearchView"
        app:showAsAction="ifRoom" />
    <item android:id="@+id/menu_item_clear"
        android:title="@string/clear_search"
        app:showAsAction="never" />
</menu>
```

چند خطا در مورد رشته‌ها برای ویژگی‌های android:title به نمایش در می‌آید. فعلا آنها را نادیده بگیرید. بعدا آن را برطرف خواهیم کرد.

اولین item در لیست ۲۶.۶ با مشخص کردن مقدار androidx.appcompat.widget.SearchView برای ویژگی app:actionViewClass به نوار ابزار می‌گوید که یک SearchView نمایش دهد.

Item دوم در لیست ۲۵.۶ گزینه "Clear Search" را اضافه می‌کند. این گزینه همیشه در منوی سرریز نمایش داده می‌شود زیرا app:showAsAction را روی never تنظیم کرده‌ایم. بعدا آن را طوری پیکربندی می‌کنیم که با لمس کردنش، کوئری ذخیره شده کاربر از حافظه پاک می‌شود. در حال حاضر می‌توانید این مورد را نادیده بگیرید.

اکنون وقت آن است که خطاهای موجود در منوی XML خود را برطرف کنیم. res/values/strings.xml را باز و رشته‌های مورد نیاز را اضافه کنید.

لیست ۲۵.۷ افزودن رشته‌های جستجو (res/values/strings.xml)

```
<resources>
    <string name="app_name">PhotoGallery</string>
    <string name="search">Search</string>
    <string name="clear_search">Clear Search</string>
</resources>
```

در اندروید مدرن، گوگل ابزار جدید و بسیار تمیزی به نام MenuProvider را معرفی کرده است. مزیت بزرگ MenuProvider این است که دقیقاً با چرخه حیات فرگمنت (Lifecycle) هماهنگ می‌شود و وقتی فرگمنت از بین برود، منو هم بدون نشتی حافظه به صورت خودکار پاکسازی می‌شود.

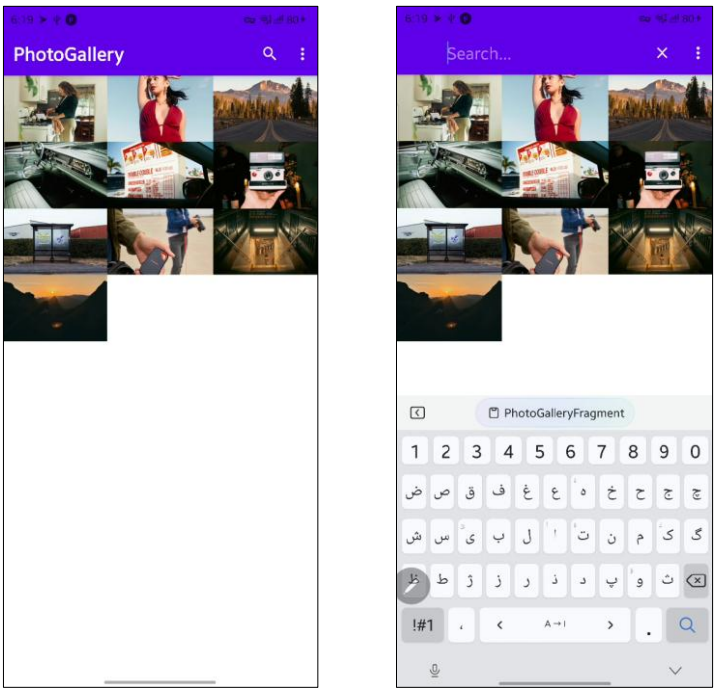
بیایید بخش مدیریت منو را در PhotoGalleryFragment با این استاندارد جدید و بدون نقص بازنویسی کنیم. کدهای داخل فرگمنت خود را به این صورت اصلاح کنید.

لیست ۲۵.۸ بازنویسی MenuProvider (PhotoGalleryFragment.kt)

```
override fun onViewCreated(view: View, savedInstanceState:
Bundle?) {
    ...
    requireActivity().addMenuProvider(object : MenuProvider {
        override fun onCreateMenu(menu: Menu, menuInflater
        : MenuInflater) {
            menuInflater.inflate(
            R.menu.fragment_photo_gallery, menu)
            val searchItem: MenuItem =
            menu.findItem(R.id.menu_item_search)
            val searchView = searchItem.actionView
            as SearchView
            searchView.setOnQueryTextListener(object
            : SearchView.OnQueryTextListener {
                override fun onQueryTextSubmit(
                    query: String?): Boolean {
                    Log.d(TAG, "QueryTextSubmit: $query")
                    photoGalleryViewModel
                    .setQuery(query.orEmpty())
                    searchView.clearFocus()
                    return true
                }
                override fun onQueryTextChange
                (newText: String?): Boolean {
                    return false
                }
            })
        }
        override fun onMenuItemSelected(menuItem: MenuItem)
        : Boolean {
            return false
        }
    }, viewLifecycleOwner, Lifecycle.State.RESUMED)
}
```

PhotoGallery را اجرا و SearchView را مشاهده نمایید. با فشار دادن آیکون جستجو، ویو برای درج متن جستجو گسترش می‌یابد (تصویر ۲۵.۲).

شکل ۲۵.۲ SearchView جمع شده و گسترش یافته است



هنگامی که SearchView گسترش می‌یابد، علامت x در سمت راست ظاهر می‌شود. با فشار دادن آن آنچه را که تایپ شده پاک می‌شود. با لمس مجدد x، SearchView به یک نماد ذربین باز می‌گردد.

دوباره برنامه خود را اجرا و یک متن جستجو ارسال کنید. باید تصاویر نمایش داده شده را بر اساس عبارت جستجویی که وارد می‌کنید، متغییر ببینید (شکل ۲۵.۳).

شکل ۲۵.۳ SearchView



توجه داشته باشید که اگر از صفحه کلید سخت افزاری برای ارسال عبارت جستجوی خود در یک شبیه‌ساز (در مقابل صفحه کلید روی صفحه شبیه‌ساز) استفاده می‌کنید، ممکن است جستجو را دو بار یکی پس از دیگری انجام دهید. این به این دلیل است که یک اشکال کوچک در SearchView وجود دارد. می‌توانید این رفتار را نادیده بگیرید زیرا این یک عارضه جانبی استفاده از شبیه‌ساز است و زمانی که برنامه شما روی یک دستگاه اندرویدی واقعی اجرا می‌شود، تاثیری نخواهد داشت.

ماندگاری با SharedPreferences

در برنامه ما، هر بار فقط یک کوئری فعال وجود خواهد داشت. PhotoGalleryViewModel کوئری را برای حیات درک شده کاربر از فرگمنت ادامه می‌دهد. اما کوئری باید بین راه اندازی مجدد برنامه نیز حفظ شود (حتی اگر کاربر دستگاه را خاموش کند).

با استفاده از shared preferences به این مهم دست خواهیم یافت. هر زمان که کاربر کوئری ارسال می‌کند، عبارت جستجو را در shared preferences می‌نویسیم و اگر کوئری قبلا وجود داشته است آن را بازنویسی می‌کنیم. هنگامی که برنامه راه اندازی می‌شود، کوئری ذخیره شده را از shared preferences بیرون کشیده و از آن برای اجرای جستجوی Unsplash استفاده می‌کنیم.

shared preferences فایل‌هایی در سیستم فایل هستند که با استفاده از کلاس SharedPreferences خوانده و ویرایش می‌کنیم. نمونه‌ای از SharedPreferences دقیقا همانند Bundle با استفاده از الگوی کلید–مقدار عمل می‌کند، تنها با این تفاوت که می‌توان آن را به صورت دایم ذخیره کرد. کلیدها رشته و مقادیر انواع داده‌ها را شامل می‌شوند. اگر به آنها نگاه کنیم، یک فایل XML ساده خواهیم دید. نادیده گرفتن جزئیات توسط SharedPreferences پیاده‌سازی را آسان می‌کند.

به هر حال، فایل‌های SharedPreferences در سندباکس برنامه ما ذخیره می‌شوند، بنابراین نباید اطلاعات حساس (مانند رمز عبور) را در آنجا ذخیره کرد.

یک فایل جدید به نام QueryPreferences.kt اضافه کنید تا به عنوان یک رابط کاربری مناسب برای خواندن و نوشتن کوئری در SharedPreferences عمل کند.

لیست ۲۵.۹ افزودن یک شی برای مدیریت یک کوئری (QueryPreferences.kt)

```
package com.codeb.photogallery

import android.content.Context
import androidx.core.content.edit

private const val PREF_SEARCH_QUERY = "searchQuery"

object QueryPreferences {
    fun getStoredQuery(context: Context): String {
        val prefs = context.getSharedPreferences(
            "photo_gallery_prefs", Context.MODE_PRIVATE)
        return prefs.getString(
            PREF_SEARCH_QUERY, "").orEmpty()
    }
    fun setStoredQuery(context: Context, query: String) {
        val prefs = context.getSharedPreferences(
```

```
            "photo_gallery_prefs", Context.MODE_PRIVATE)
        prefs.edit {
            putString(PREF_SEARCH_QUERY, query)
        }
    }
}
```

برنامه ما فقط به یک نمونه از QueryPreferences نیاز دارد که می‌تواند در تمام اجزای دیگر به اشتراک گذاشته شود. به همین دلیل، ما از کلمه کلیدی object (به جای class) برای مشخص کردن آن استفاده می‌کنیم.

QueryPreferences یک singleton است. این کار قصد ما را مبنی بر ایجاد فقط یک نمونه در برنامه اعمال می‌کند. همچنین به ما امکان می‌دهد با استفاده از ClassName.functionName(…) به توابع موجود در شی دسترسی داشته باشیم (همانطور که به زودی خواهید دید).

PREF_SEARCH_QUERY به عنوان کلید به کار برده می‌شود. هر زمان که مقدار کوئری را می‌خوانیم یا می‌نویسیم، از این کلید استفاده خواهیم کرد.

PreferenceManager.getDefaultSharedPreferences(Context) یک نمونه با نام پیش‌فرض و مجوزهای خصوصی برمی‌گرداند (به طوری که SharedPreferences فقط از داخل برنامه ما در دسترس هستند). برای دریافت یک نمونه خاص از SharedPreferences می‌توانیم از تابع Context.getSharedPreferences(String, Int) استفاده کنیم. با این حال، در عمل نمونه خاص اهمیت چندانی ندارد، فقط اشتراک در کل برنامه هدف است.

تابع getStoredQuery(Context) مقدار کوئری ذخیره شده در SharedPreferences را برمی‌گرداند. این کار را ابتدا با به دست آوردن SharedPreferences پیش‌فرض برای Contex داده شده انجام می‌دهد. (زیرا QueryPreferences Context خاص خود را ندارد، مؤلفه فراخوانی باید Context خود را به عنوان ورودی ارسال کند.)

دریافت مقداری که قبلا ذخیره کرده‌ایم به سادگی با فراخوانی SharedPreferences.getString(…) انجام می‌شود. SharedPreferences.getInt(…)، یا هر تابعی که برای نوع داده شما مناسب باشد را فراخوانی کنید.

ورودی دوم SharedPreferences.getString(String, String) مقدار بازگشتی پیش‌فرض را مشخص می‌کند که در صورت عدم وجود ورودی برای کلید PREF_SEARCH_QUERY باید از آن استفاده شود.

نوع بازگشت برای SharedPreferences.getString(…)یک String غیرتهی تعریف می‌شود زیرا کامپایلر نمی‌تواند تضمین کند که مقدار مرتبط با PREF_SEARCH_QUERY وجود دارد و تهی نیست. ما می‌دانیم که هرگز یک مقدار تهی را برای PREF_SEARCH_QUERY ذخیره نمی‌کنیم. بنابراین استفاده از عملگر (!!) بی خطر است.

تابع setStoredQuery(Context) کوئریو ورودی را در SharedPreferences پیش‌فرض برای Context داده شده می‌نویسد. در QueryPreferences() تابع SharedPreferences.edit را برای دریافت نمونه‌ای از SharedPreferences.Editor فراخوانی می‌کنیم. این کلاسی است که برای ذخیره مقادیر در SharedPreferences خود استفاده می‌کنیم. به ما امکان گروه بندی مجموعه‌ای از تغییرات را در تراکنش‌ها دقیقا مانند کاری که با FragmentTransaction انجام می‌دادیم، می‌دهد.

پس از اتمام انجام تمام تغییرات، apply() را در editor خود فراخوانی می‌کنیم. تابع apply() بلافاصله تغییر حافظه را ایجاد و سپس نوشتن فایل واقعی را روی یک نخ پس زمینه انجام می‌دهد.

QueryPreferences کل موتور ماندگاری برای PhotoGallery است.

اکنون که راهی برای ذخیره و دسترسی آسان به جدیدترین کوئری کاربر ایجاد کردیم، PhotoGalleryViewModel را برای خواندن و نوشتن کوئری در SharedPreferences بروز می‌کنیم. هنگامی که ViewModel برای اولین بار ایجاد می‌شود، کوئری را می‌خواند و از مقدار آن برای مقداردهی اولیه mutableSearchTerm استفاده می‌کند. هر زمان که mutableSearchTerm تغییر کرد، کوئری را ثبت می‌کنیم.

لیست ۲۵.۱۰ حفظ کوئری (PhotoGalleryViewModel.kt)

```
class PhotoGalleryViewModel(app: Application) :
AndroidViewModel(app) {
~~class PhotoGalleryViewModel : ViewModel() {~~
    private val unsplashFetcher = UnsplashFetcher()
    private val context = app.applicationContext
    ~~private val _searchQuery = MutableStateFlow("")~~
    private val _searchQuery = MutableStateFlow(
        QueryPreferences.getStoredQuery(context))
    fun setQuery(query: String) {
        _searchQuery.value = query
        QueryPreferences.setStoredQuery(context, query)
    }
    …
}
```

ViewModel ما برای استفاده از توابع QueryPreferences به یک context نیاز دارد.

تغییر کلاس والد PhotoGalleryViewModel از ViewModel به AndroidViewModel به PhotoGalleryViewModel امکان دسترسی به context برنامه را می‌دهد.

برای PhotoGalleryViewModel بی‌خطر است که به context برنامه ارجاع دهد، زیرا context برنامه بیشتر از PhotoGalleryViewModel عمر می‌کند.

در مرحله بعد، هنگامی که کاربر Clear Search را از منو انتخاب کرد، عبارت ذخیره شده را پاک کنید. برای انجام این کار کد MenuProvider را همانند لیست زیر اصلاح کنید.

لیست ۲۵.۱۱ پاک کردن یک کوئری ذخیره شده (PhotoGalleryFragment.kt)

```
requireActivity().addMenuProvider(object : MenuProvider {
        override fun onCreateMenu(menu: Menu, menuInflater
            : MenuInflater) {
            menuInflater.inflate
              (R.menu.fragment_photo_gallery, menu)
            val searchItem: MenuItem =
               menu.findItem(R.id.menu_item_search)
            val searchView = searchItem.actionView
               as SearchView
            searchView.setOnQueryTextListener(object :
                SearchView.OnQueryTextListener {
                override fun onQueryTextSubmit(query:
                    String?): Boolean {
                    Log.d(TAG, "QueryTextSubmit: $query")
                    photoGalleryViewModel.
                        setQuery(query.orEmpty())
                    searchView.clearFocus()
                    return true
                }
                override fun onQueryTextChange
                    (newText: String?): Boolean {
                    return false
                }
            })
        }
        override fun onMenuItemSelected(menuItem: MenuItem):
            Boolean {
            return false
            return when (menuItem.itemId) {
                R.id.menu_item_clear -> {
                    photoGalleryViewModel.setQuery("")
                    activity?.invalidateOptionsMenu()
                    true
                }
                else -> false
```

```
                }
            }
        }, viewLifecycleOwner, Lifecycle.State.RESUMED)
}
```

۲۶

WorkManager

PhotoGallery اکنون می‌تواند تصاویر جذاب را از سرور Unsplash دانلود کند، نتایج را بر اساس کلمات کلیدی کاربر فیلتر و حتی به کمک SharedPreferences آخرین کوئری را پس از بستن برنامه به خاطر بسپارد.

اما تمام این جادوها فقط زمانی اتفاق می‌افتند که اپلیکیشن جلو چشم کاربر (Foreground) باز باشد. به محض اینکه کاربر برنامه را ترک کند، خط تولید متوقف می‌شود. در این فصل، می‌خواهیم قابلیت بررسی دوره‌ای (Polling) را به برنامه اضافه کنیم. برنامه باید در پس‌زمینه گوش‌به‌زنگ باشد تا ببیند آیا عکس‌های جدیدی متناسب با آخرین جستجوی کاربر در Unsplash آپلود شده است یا خیر.

این پردازش کاملاً در پس‌زمینه (Background) رخ می‌دهد؛ یعنی حتی اگر کاربر برنامه را به طور کامل بسته باشد یا در حال بازی و کار با اپلیکیشن دیگری باشد، این بررسی بدون اختلال انجام خواهد شد. اگر سیستم متوجه حضور عکس‌های جدیدی شود که کاربر هنوز آن‌ها را ندیده است، یک اعلان (Notification) در نوار وضعیت گوشی ظاهر می‌شود تا کاربر را تشویق کند به برنامه بازگردد و گالری را تماشا کند.

برای پیاده‌سازی این قابلیت، از قدرتمندترین ابزار گوگل در کتابخانه Jetpack یعنی WorkManager استفاده خواهیم کرد. وظیفه این ابزار، مدیریت و اجرای کارهای دوره‌ای و تضمین‌شده است. در این فصل ابتدا یک کلاس Worker (از نوع CoroutineWorker برای هماهنگی با کوروتین‌ها) می‌سازیم تا کار اصلی یعنی درخواست شبکه در پس‌زمینه را انجام دهد. سپس به کمک PeriodicWorkRequest زمان‌بندی اجرای آن را در بازه‌های منظم تنظیم می‌کنیم و در نهایت، با شکار دیتای جدید، به کمک NotificationManager یک کانال امن برای ارسال اعلاناتِ مدرن به کاربر ایجاد خواهیم کرد.

ایجاد یک Worker

کلاس Worker قلب تپنده‌ی سیستم پردازش پس‌زمینه است؛ یعنی دقیقاً همان جایی که کدهای اصلی و منطقی را که می‌خواهیم در غیاب کاربر انجام شوند (مثل درخواست شبکه به Unsplash) قرار می‌دهیم.

پس از اینکه ساختار کلاس Worker آماده شد، لایه‌ی بعدی یعنی WorkRequest را می‌سازیم. وظیفه‌ی WorkRequest این است که به سیستم‌عامل بگویند این کار چه زمانی، چند وقت یک‌بار و تحت چه شرایطی (مثلاً فقط هنگام اتصال به وای‌فای) باید اجرا شود.

قبل از اینکه بتوانیم کلاس Worker خود را بنویسیم، ابتدا باید کتابخانه‌ی رسمی WorkManager را به پروژه معرفی کنیم. برای این کار، فایل app/build.gradle.kts (نسخه ماژول App) را باز کرده و وابستگی را اضافه کنید.

لیست ۲۶.۱ افزودن وابستگی WorkManager (app/build.gradle)

```
dependencies {
    ...
    implementation("io.coil-kt:coil:2.6.0")
    implementation("androidx.work:work-runtime-ktx:2.9.1")
    ...
}
```

پروژه را همگام سازی کنید.

حالا که کتابخانه به پروژه اضافه شده است، آماده‌ایم تا اولین Worker پس‌زمینه خود را خلق کنیم. در اندروید مدرن و کاتلین، به جای کلاس پایه و قدیمی Worker، از کلاس پیشرفته‌ترِ CoroutineWorker استفاده می‌کنیم. این کلاس به ما اجازه می‌دهد در آینده کدهای ناهمگام و درخواست‌های شبکه رتروفیت (که suspend هستند) را به راحتی درون آن اجرا کنیم.

یک فایل کاتلین جدید به نام PollWorker.kt بسازید و کدهای زیر را درون آن قرار دهید.

لیست ۲۶.۲ ایجاد Worker (PollWorker.kt)

```
package com.codeb.photogallery
import android.content.Context
import android.util.Log
import androidx.work.CoroutineWorker
import androidx.work.WorkerParameters
private const val TAG = "PollWorker"

class PollWorker(
    context: Context,
    workerParams: WorkerParameters
) : CoroutineWorker(context, workerParams) {
    override suspend fun doWork(): Result {
        Log.i(TAG, "WorkManager
            : Background work started successfully!")

        return Result.success()
```

```
    }
}
```

تابع doWork() به صورت پیش‌فرض بر روی یک نخ پس‌زمینه (Background Thread) اختصاصی اجرا می‌شود. به همین دلیل، شما دستتان کاملاً باز است تا هرگونه عملیات سنگین و زمان‌بری (مانند درخواست‌های شبکه، پردازش تصویر یا کار با دیتابیس) را بدون نگرانی از مسدود شدن نخ اصلی برنامه (UI Thread)یا لگ زدن رابط کاربری، درون آن انجام دهید.

موفقیت یا شکست این عملیات سنگین، توسط مقادیر بازگشتی از نوع Result مشخص می‌شود که سیستم‌عامل اندروید بر اساس آن‌ها تصمیم می‌گیرد چه بلایی سر این تسک بیاورد.

Result.success() به سیستم اعلام می‌کند که کار با موفقیت صددرصد به پایان رسیده است. در کد فعلی ما، چون کار فقط چاپ یک لاگ ساده در کنسول است، همین مقدار برگردانده می‌شود.

Result.failure() زمانی استفاده می‌شود که کار به دلیل یک خطای دائمی و مهلک (Fatal Error) نیمه‌کاره رها شده است. با بازگرداندن این مقدار، سیستم‌عامل متوجه می‌شود که نباید این درخواست را دوباره اجرا کند و پرونده‌اش بسته می‌شود.

Result.retry() برای خطاهای موقتی (مانند قطع شدن ناگهانی اینترنت یا تایم‌اوت سرور) است. وقتی این مقدار برگردانده شود، سیستم‌عامل به صورت هوشمند و بر اساس یک الگوریتم تاخیر تصاعدی (Exponential Backoff)، مدتی بعد دوباره شانسش را برای اجرای این کار امتحان می‌کند.

برنامه ریزی کار (Work Scheduling)

برای برنامه ریزی یک Worker برای اجرا، به یک WorkRequest نیاز داریم. کلاس WorkRequest خود انتزاعی است، بنابراین بسته به نوع کاری که باید اجرا شود، از یکی از زیر کلاس‌های آن استفاده می‌کنیم. اگر موردی فقط یک بار باید اجرا شود، از OneTimeWorkRequest استفاده می‌کنیم. اگر کار ما چیزی است که باید به صورت دوره‌ای اجرا شود، از PeriodicWorkRequest استفاده می‌کنیم.

در حال حاضر، ما قصد داریم از OneTimeWorkRequest استفاده کنیم. این کار برای تأیید درستی کار PollWorker پی بردن به نحوه ایجاد و کنترل درخواست‌ها می‌باشد. بعدا برنامه خود را برای استفاده از PeriodicWorkRequest به روز خواهیم کرد.

برای اجرای این تسک، فایل PhotoGalleryFragment.kt را باز کنید و کدهای زیر را درون متد onViewCreated قرار دهید.

لیست ۲۶.۳ برنامه ریزی یک درخواست کار (PhotoGalleryFragment.kt)

```
private const val TAG = "PhotoGalleryFragment"
private const val POLL_WORK_NAME = "unsplash_one_time_poll"
class PhotoGalleryFragment : Fragment() {
    …
    override fun onViewCreated(view: View
        , savedInstanceState: Bundle?) {
        …
        setupBackgroundWork()
    }
    private fun setupBackgroundWork() {
        val workRequest =
            OneTimeWorkRequestBuilder<PollWorker>().build()
        WorkManager.getInstance(requireContext()
            .applicationContext)
            .enqueueUniqueWork(
                POLL_WORK_NAME,
                ExistingWorkPolicy.KEEP,workRequest
            )
    }

    ...
}
```

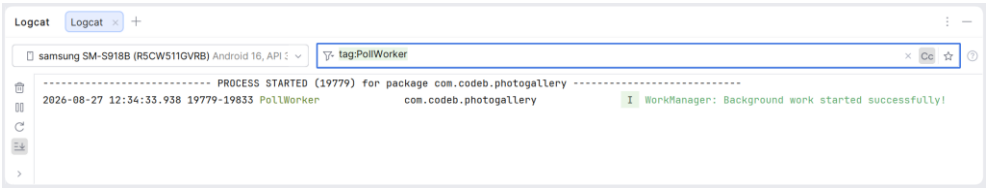
OneTimeWorkRequestBuilder<PollWorker>() این متد جادویی کاتلین با بهره‌گیری از قابلیت Generics و Reified Type کلاس Worker ما را دریافت کرده و یک سازنده (Builder) برای ساخت درخواست یک‌باره تحویل می‌دهد. متد ()build شیء نهایی را خلق می‌کند.

بسیار حیاتی است که برای گرفتن نمونه WorkManager همیشه از applicationContext استفاده کنیم (requireContext().applicationContext). این کار تضمین می‌کند که حتی با بسته شدن یا چرخیدن صفحه فرگمنت، سیستم دچار نشتی حافظه (Memory Leak) نشود.

enqueue(workRequest) درخواست را به صف پردازش سیستم‌عامل اندروید می‌سپارد. سیستم‌عامل در سریع‌ترین زمان ممکن و زمانی که پردازنده آزاد باشد، نخ پس‌زمینه را بیدار کرده و تابع ()doWork کلاس PollWorker را اجرا می‌کند.

اگر اکنون برنامه را اجرا کنید، در بخش Logcat اندروید استودیو خواهید دید که لاگ "WorkManager: Background work started successfully!" با موفقیت چاپ می‌شود.

تصویر ۲۶.۱ گزارش کار



اعمال محدودیت‌های هوشمند با کلاس Constraints

یکی از بزرگ‌ترین مزایای WorkManager این است که بر خلاف سرویس‌های سنتی، کورکورانه کار را در پس‌زمینه اجرا نمی‌کند. اگر تسک شما به شبکه گره خورده باشد و کاربر اینترنت ضعیفی داشته باشد یا در حال استفاده از اینترنت پولی گران‌قیمت (Metered Network) باشد، اجرای کار منطقی نیست.

به کمک کلاس Constraints، شما می‌توانید به سیستم‌عامل بگویید: «این کار را فقط و فقط زمانی اجرا کن که این شرایط مهیا باشند.» سیستم‌عامل اندروید تا زمان فراهم شدن تک‌تک این شرایط، کارگر (Worker) را در حالت انتظار (Enqueued) نگه می‌دارد. برخی از این محدودیت‌های کاربردی عبارتند از:

- NetworkType: تعیین نوع شبکه (مثلاً اتصال به هر شبکه‌ای، یا الزاماً یک شبکه نامحدود و رایگان مثل وای‌فای).
- RequiresCharging: دستگاه حتماً باید به شارژر متصل باشد.
- RequiresBatteryNotLow: میزان باتری گوشی نباید در وضعیت هشدار یا کم (Low Battery) باشد.

- RequiresDeviceIdle: دستگاه در حالت استراحت باشد و کاربر به طور فعال با آن کار نکند (مناسب برای کارهای بسیار سنگین).

برای اضافه کردن این ویژگی به پروژه، متد setupBackgroundWork() را در فایل به شکل زیر ویرایش کنید. در این ساختار مدرن، ما محدودیت اتصال به شبکه غیراندازه‌گیری‌شده (Unmetered مثل Wi-Fi) را به کارگر اعمال می‌کنیم.

لیست ۲۶.۴ اضافه کردن محدودیت‌های کاری (PhotoGalleryFragment.kt)

```
private fun setupBackgroundWork() {
    val constraints = androidx.work.Constraints.Builder()
        .setRequiredNetworkType(
            androidx.work.NetworkType.UNMETERED)
        .build()
    val workRequest = OneTimeWorkRequestBuilder<PollWorker>()
        .setConstraints(constraints)
        .build()
    WorkManager.getInstance(requireContext()
        .applicationContext)
        .enqueueUniqueWork(
            POLL_WORK_NAME,
            ExistingWorkPolicy.KEEP,
            workRequest
        )
}
```

بررسی عکس‌های جدید

اکنون که worker ما به درستی کار می‌کند، می‌توانیم منطق برنامه را برای بررسی عکس‌های جدید پیاده کنیم. برای این عملکرد چند قطعه مورد نیاز است. ابتدا به راهی برای ذخیره شناسه جدیدترین عکسی که کاربر دیده است نیاز داریم، سپس باید کلاس worker خود را به روز کنیم تا عکس‌های جدید را بیرون کشیده و شناسه ذخیره شده را با جدیدترین شناسه سرور مقایسه کند. worker همچنین در صورت وجود عبارت جستجوی موجود، انتخاب نوع درخواست را مدیریت می‌کند. برای اینکه کارگر (Worker) ما بتواند بفهمد عکسی که از سرور دانلود کرده «جدید» است یا خیر، به یک معیار مقایسه نیاز دارد. این معیار، شناسه (ID) آخرین تصویری است که در آخرین اجرای برنامه دریافت شده بود.

روند کار به این صورت است:

شناسه آخرین عکس دریافت شده را درون SharedPreferences ذخیره می‌کنیم. سپس کلاس PollWorker در پس‌زمینه بیدار شده و آخرین عکس‌های Unsplash را (با در نظر گرفتن کوئری سرچ کاربر) دانلود می‌کند. و در نهایت کارگر، شناسه اولین عکسِ لیست جدید را با شناسه‌ای که قبلاً در حافظه گوشی ذخیره شده بود مقایسه می‌کند. اگر این دو شناسه متفاوت باشند، یعنی عکس جدیدی آمده است!

گام اول به‌روزرسانی کلاس QueryPreferences است. ابتدا فایل QueryPreferences.kt را باز کنید و دو متد جدید برای ذخیره (setStoredLastResultId) و بازیابی (getStoredLastResultId) شناسه آخرین عکس اضافه کنید.

لیست ۲۶.۵ ذخیره آخرین شناسه عکس (QueryPreference.kt)

```
private const val PREF_SEARCH_QUERY = "searchQuery"
private const val PREF_LAST_RESULT_ID = "lastResultId"
object QueryPreferences {
    fun getStoredQuery(context: Context): String {
        ...
    }
    fun setStoredQuery(context: Context, query: String) {
        ...
    }
    fun getStoredLastResultId(context: Context): String {
        val prefs = context.getSharedPreferences(
        "photo_gallery_prefs", Context.MODE_PRIVATE)
        return prefs.getString(PREF_LAST_RESULT_ID
            , "").orEmpty()
    }
    fun setStoredLastResultId(context: Context
        , lastResultId: String) {
        val prefs = context.getSharedPreferences(
```

```
            "photo_gallery_prefs", Context.MODE_PRIVATE)
            prefs.edit {
                putString(PREF_LAST_RESULT_ID, lastResultId)
            }
        }
}
```

با استفاده از متد { ... }prefs.edit که یک تابع پسوند (Extension Function) مدرن در کاتلین اندروید کی‌تی‌ایکس (Android KTX) است، عملیات ذخیره‌سازی به صورت کاملاً بهینه و بدون نیاز به نوشتن دستور تکراری apply یا commit در دیسک انجام می‌شود.

حالا که شناسه عکس را ذخیره کردیم، می‌توانیم برای عکس‌های جدید به سراغ بررسی دوره ای برویم. فایل PollWorker.kt را باز کنید و بدنه تابع ()doWork را به این شکل بازنویسی کنید تا منطق دانلود، مقایسه شناسه‌ها و به‌روزرسانی دیسک تکمیل شود.

لیست ۲۶.۷ واکشی عکس‌های اخیر (PollWorker.kt)

```
class PollWorker(
    private val context: Context,
    workerParams: WorkerParameters
) : CoroutineWorker(context, workerParams) {
    override suspend fun doWork(): Result {
        Log.i(TAG, "WorkManager
            : Background work started successfully!")
        return Result.success()
        val query = QueryPreferences.getStoredQuery(context)
        val lastResultId = QueryPreferences
            .getStoredLastResultId(context)
        val unsplashFetcher = UnsplashFetcher()
        val photos: List<UnsplashPhoto> = try {
            if (query.isBlank()) {
                unsplashFetcher.fetchPhotos().first()
            } else {
                unsplashFetcher.searchPhotos(query).first()
            }
        } catch (e: Exception) {
            Log.e(TAG, "Background network fetch failed", e)
            return Result.retry()
        }
        if (photos.isEmpty()) {
            Log.i(TAG, "Server returned 0 photos.")
            return Result.success()
        }
        val currentResultId = photos.first().id
        if (currentResultId == lastResultId) {
            Log.i(TAG, "No new photos found.ID matches:
```

```
$currentResultId")
        } else {
            Log.i(TAG,
                "New photos detected! New
ID:$currentResultId, Old ID: $lastResultId")
            QueryPreferences.setStoredLastResultId(context
                , currentResultId)
        }
        return Result.success()
    }
}
```

این سیستم در پس‌زمینه چطور عمل می‌کند؟

مدیریت هوشمند خطا: اگر اینترنت قطع باشد، بلوک try-catch خطا را مهار کرده و دستور Result.retry() را صادر می‌کند تا باتری گوشی بیهوده برای تلاش‌های تکراری مصرف نشود.

استخراج مستقیم دیتا: متد first() روی تکانه‌های خروجی Flow گوش می‌ایستد، اولین لیستِ دریافتی از سرور Unsplash را برمی‌دارد و بلافاصله به فلو خاتمه می‌دهد که بهترین روش برای استفاده از فلوها درون محیط‌های بسته مثل Worker است.

تصمیم‌گیری نهایی: اگر آیدی جدید با آیدی ذخیره‌شده روی هارد گوشی فرق داشته باشد، آیدی جدید ذخیره شده و همه‌چیز آماده ارسال نوتیفیکیشن به کاربر می‌شود.

برنامه خود را روی دستگاه یا شبیه‌ساز اجرا کنید. اولین باری که آن را اجرا می‌کنید، آخرین شناسه نتیجه در QueryPreferences ذخیره نخواهد شد، بنابراین باید بیانیه گزارش را ببینید که نشان می‌دهد PollWorker نتیجه جدیدی پیدا کرده است. اگر به سرعت برنامه را دوباره اجرا کنید، باید ببینید که worker شما همان شناسه را پیدا می‌کند (شکل ۲۶.۲).

شکل ۲۶.۲ یافتن نتایج جدید و قدیمی



در حال حاضر کد بررسی ما به عنوان یک تسک یک‌باره (OneTimeWorkRequest) تنظیم شده است. به همین خاطر در ساختار فعلی فرگمنت ما، این کد فقط یک بار اجرا می‌شود.

برای اینکه کار ما به صورت دوره‌ای و مداوم در پس‌زمینه تکرار شود (حتی زمانی که کاربر برنامه را کاملاً می‌بندد یا گوشی را ریستارت می‌کند)، باید از PeriodicWorkRequestBuilder به جای OneTimeWorkRequestBuilder استفاده کنیم.

سیستم‌عامل اندروید برای حفظ باتری گوشی، محدودیت‌هایی روی کارهای دوره‌ای اعمال می‌کند:

- حداقل زمان دوره (Minimum Interval): طبق قوانین اندروید، یک کار دوره‌ای نمی‌تواند زودتر از هر ۱۵ دقیقه یک‌بار اجرا شود. اگر زمانی کمتر از این (مثلاً ۵ دقیقه) تنظیم کنید، سیستم‌عامل به صورت خودکار آن را به ۱۵ دقیقه تغییر می‌دهد.
- ماندگاری پس از بستن اپلیکیشن: ابزار WorkManager ذاتاً طوری طراحی شده است که تسک‌ها را در دیتابیس داخلی خود ذخیره می‌کند. بنابراین با بستن برنامه، تسک شما کلاً از بین نمی‌رود و در زمان مشخص‌شده توسط سیستم‌عامل بیدار می‌شود.

حالا کد متد setupBackgroundWork را جهت اجرای دوره ای به روز می‌کنیم. ابتدا نام تسک خود را در بالای فرگمنت تغییر دهید تا با کارهای قبلی تداخل نداشته باشد (مثلاً از unsplash_one_time_poll به unsplash_periodic_poll).

لیست ۲۶.۸ تغییر تسک (PhotoGalleryFragment.kt)

```
private const val TAG = "PhotoGalleryFragment"
~~private const val POLL_WORK_NAME = "unsplash_one_time_poll"~~
private const val POLL_WORK_NAME = "unsplash_periodic_poll"
class PhotoGalleryFragment : Fragment() {
    …
}
```

سپس متد را در کلاس PhotoGalleryFragment.kt به این شکل بازنویسی کنید.

لیست ۲۶.۹ بازنویسی متد (PhotoGalleryFragment.kt)

```
private fun setupBackgroundWork() {
        val constraints = androidx.work.Constraints.Builder()
            .setRequiredNetworkType(
             androidx.work.NetworkType.CONNECTED)
            .build()
        ~~val workRequest = OneTimeWorkRequestBuilder
        <PollWorker>().setConstraints(constraints).build()~~
        val workRequest =
            PeriodicWorkRequestBuilder<PollWorker>(
            15, TimeUnit.MINUTES
        )
            .setConstraints(constraints)
```

```
            .build()
        WorkManager.getInstance(requireContext()
            .applicationContext)
            .enqueueUniqueWork(
            .enqueueUniquePeriodicWork(
                POLL_WORK_NAME,
                ExistingWorkPolicy.KEEP,
                androidx.work
                .ExistingPeriodicWorkPolicy.KEEP,
                workRequest
            )
    }
```

دقت کنید ایمپورت زیر را حتما وارد کنید.

```
import java.util.concurrent.TimeUnit
```

حالا برنامه ما عالی کار می‌کند اما دو نکته بسیار مهم برای اجرای موفق در پس‌زمینه گوشی واقعی را در نظر بگیرید:

- لغو تسک‌های یک‌باره‌ی قبلی: از آنجایی که نام تسک و نوع آن را عوض کرده‌اید، حتماً یک‌بار برنامه را از روی گوشی کلاً پاک (Uninstall) کنید تا صف‌های قدیمی و تسک‌های یک‌باره پاک شوند، سپس نسخه جدید را نصب کنید.
- قاتل پس‌زمینه شرکت‌های سازنده (Doze Mode & Battery Savers): اندروید تضمین می‌کند که این کار در پس‌زمینه اجرا شود، اما شرکت‌هایی مثل سامسونگ، شیائومی یا هواوی قابلیتی به نام App Standby یا Battery Optimization دارند که اگر کاربر برنامه‌ای را چند روز باز نکند، کل دسترسی‌های پس‌زمینه آن را قطع می‌کنند. برای تست مطمئن، در تنظیمات گوشی خود به بخش App Info > PhotoGallery > Battery بروید و آن را روی حالت Unrestricted (بدون محدودیت) قرار دهید.

اطلاع رسانی به کاربر

worker اکنون در حال اجرا است و عکس‌های جدید را در پس زمینه بررسی می‌کند، اما کاربر چیزی در مورد آن نمی‌داند. هنگامی که PhotoGallery عکس‌های جدیدی را پیدا می‌کند که کاربر هنوز ندیده است، باید از کاربر بخواهد که برنامه را باز و محتوای جدید را ببیند.

هنگامی که برنامه شما نیاز به برقراری ارتباط با کاربر دارد، ابزار مناسب تقریبا همیشه یک اعلان (notification) است. اعلان‌ها مواردی هستند که در کشوی اعلان‌ها ظاهر می‌شوند و کاربر می‌تواند با

کشیدن از بالای صفحه به پایین به آنها دسترسی داشته باشد. در اندروید مدرن (به‌ویژه از اندروید ۱۳ به بعد)، این بخش دستخوش تغییرات امنیتی بسیار شدیدی شده است که اگر آن‌ها را رعایت نکنید، اعلان شما هرگز روی گوشی کاربر نشان داده نخواهد شد.

سیستم اعلان‌ها و مفهوم کانال (Notification Channels)

اعلان‌ها بهترین ابزار برای بازگرداندن کاربر به درون اپلیکیشن هستند. اما از اندروید ۸.۰ (API 26) به بعد، گوگل مفهوم جدیدی به نام Notification Channel (کانال اعلان) را معرفی کرد.

هدف از ساخت کانال‌ها این است که کاربر کنترل کاملی روی رفتارهای اپلیکیشن داشته باشد. به جای اینکه کاربر مجبور شود کل نوتیفیکیشن‌های برنامه شما را مسدود (Block) کند، می‌تواند به صورت تفکیک‌شده تصمیم بگیرد؛ مثلاً اعلان‌های مربوط به «تصاویر جدید» را روشن نگه دارد، اما اعلان‌های مربوط به «تبلیغات یا پیشنهادات» برنامه را خاموش کند. همچنین کاربر می‌تواند صدا، لرزش (Vibration) و میزان اهمیت (Importance) هر کانال را به دلخواه خود در تنظیمات گوشی شخصی‌سازی کند.

برنامه ما باید حداقل یک کانال ایجاد کند.

یک کلاس جدید به نام PhotoGalleryApplication اضافه کنید. آن را از Application گسترش و Application.onCreate() را برای ایجاد و افزودن یک کانال گسترش دهید.

لیست ۲۶.۱۰ ایجاد یک کانال اطلاع رسانی (PhotoGalleryApplication.kt)

```
package com.codeb.photogallery
import android.app.Application
import android.app.NotificationChannel
import android.app.NotificationManager
import android.content.Context
import android.os.Build
const val NOTIFICATION_CHANNEL_ID =
    "unsplash_polling_channel"
class PhotoGalleryApplication : Application() {
    override fun onCreate() {
        super.onCreate()
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
            val name = "Unsplash New Photos"
            val descriptionText = "Notifies you when new pho-
tos are uploaded matching your query"
            val importance = NotificationManager
            .IMPORTANCE_DEFAULT
            val channel = NotificationChannel(
            NOTIFICATION_CHANNEL_ID, name
```

```
                , importance).apply {
                description = descriptionText
            }
            val notificationManager = getSystemService(
            Context.NOTIFICATION_SERVICE)
            as NotificationManager
            notificationManager
            .createNotificationChannel(channel)
        }
    }
}
```

اندروید به صورت خودکار کلاس سفارشی Application شما را نمی‌شناسد. برای اینکه سیستم‌عامل این کلاس را به عنوان نقطه ورود اصلی برنامه شناسایی کند، باید نام آن را به تگ <application> در فایل AndroidManifest.xml معرفی کنید.

لیست ۲۶.۱۱ به روز رسانی برچسب برنامه در مانیفست (manifests/AndroidManifest.xml)

```
<manifest xmlns:android="http://schemas.an-
droid.com/apk/res/android">
    <uses-permission
        android:name="android.permission.INTERNET" />
    <application
        android:name=".PhotoGalleryApplication"
        ...>
        <activity
            ...
        </activity>
    </application>
</manifest>
```

برای ارسال اعلان، باید یک شی Notification ایجاد کنید. Notificationها با استفاده از یک شی سازنده ایجاد می‌شوند، بسیار شبیه به AlertDialog که استفاده کردیم. حداقل، Notification شما باید موارد زیر را داشته باشد:

- آیکونی برای نمایش در نوار وضعیت
- ویویی برای نمایش در کشوی اعلان برای نشان دادن خود اعلان
- یک PendingIntent در انتظار شلیک برای زمانی که کاربر اعلان را در کشو فشار می‌دهد
- یک NotificationChannel برای اعمال استایل و ارائه کنترل کاربر بر اعلان

همچنین متن ticker را به اعلان اضافه خواهیم کرد. این متن هنگام نمایش اعلان نمایش داده نمی‌شود، اما به سرویس‌های دسترسی ارسال می‌شود تا مواردی مانند صفحه خوان‌ها بتوانند به کاربران دارای اختلالات بینایی اطلاع دهند.

هنگامی که یک شی Notification ایجاد کردیم، می‌توان آن را با فراخوانی notify(Int, Notification) در سرویس سیستم NotificationManager ارسال کرد. Int شناسه اعلان برنامه شما است.

ابتدا باید مقداری کد لوله کشی اضافه کنید که در لیست ۲۷.۱۲ نشان داده شده است.

PhotoGalleryActivity.kt را باز و یک تابع newIntent(Context) اضافه کنید. این تابع یک نمونه Intent را برمی‌گرداند که می‌تواند برای راه اندازی PhotoGalleryActivity هنگام لمس اعلان توسط کاربر، استفاده شود. (در نهایت، PollWorker تابع PhotoGalleryActivity.newIntent() را فراخوانی و اینتنت حاصل را در PendingIntent قرار داده و به یک اعلان منتقل می‌کند.)

لیست ۲۶.۱۲ افزودن newIntent(…) به PhotoGalleryActivity (PhotoGalleryActivity.kt)

```
class PhotoGalleryActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        ...
    }
companion object {
    fun newIntent(context: Context): Intent {
        return Intent(context
        , PhotoGalleryActivity::class.java).apply {
            flags = Intent.FLAG_ACTIVITY_NEW_TASK or
                Intent.FLAG_ACTIVITY_CLEAR_TOP
            }
        }
    }
}
```

ترکیب دو پرچم فوق باعث می‌شود که اگر برنامه از قبل باز باشد، به جای ساخت یک صفحه تکراری، همان صفحه‌ی قبلی از حافظه بیدار شده و جلو بیاید، و اگر برنامه بسته باشد، آن را در یک تسک جدید و امن اجرا کند.

حالا نوبت ساخت اعلان است، فایل PollWorker.kt را باز کنید و متد showNotification() را به شکل زیر اضافه کنید. همچنین مطمئن شوید که این متد در بلوک else (جایی که عکس جدید کشف می‌شود) صدا زده شده است.

لیست ۲۶.۱۳ افزودن نوتیفیکیشن (PollWorker.kt)

```
private const val NOTIFICATION_ID = 1
class PollWorker(
    private val context: Context,
    workerParams: WorkerParameters
) : CoroutineWorker(context, workerParams) {
    override suspend fun doWork(): Result {
        ...
        val currentResultId = photos.first().id
        if (currentResultId == lastResultId) {
            ...
        } else {
        Log.i(TAG, "New photos detected! New ID
            : $currentResultId, Old ID: $lastResultId")
        QueryPreferences.setStoredLastResultId(context
            , currentResultId)
        val intent = PhotoGalleryActivity.newIntent(context)
        val pendingIntent = PendingIntent.getActivity(
            context,
            0,
            intent,
            PendingIntent.FLAG_IMMUTABLE
        )
        val notification = NotificationCompat.Builder(
        context, NOTIFICATION_CHANNEL_ID)
        .setSmallIcon(
        android.R.drawable.ic_menu_report_image)
        .setContentTitle("New Photos!")
        .setContentText("new photos to Unsplash!")
        .setContentIntent(pendingIntent)
        .setAutoCancel(true)
        .build()
        val notificationManager = NotificationManagerCompat
        .from(context)
        if (ContextCompat.checkSelfPermission(
            context,
             Manifest.permission.POST_NOTIFICATIONS
        ) == PackageManager.PERMISSION_GRANTED
        ) {
            notificationManager.notify(0, notification)
        }
     }
     return Result.success()
    }
}
```

این کد یک اعلان امن و مدرن می‌سازد، آن را به اکتیویتی برنامه متصل و با بررسی دقیق مجوزها در اندروید ۱۳ به بالا، بدون خطر کرش کردن، اعلان را به کاربر نشان می‌دهد. تحلیل خط به خط کدهای بالا به این صورت است که:

- PendingIntent.FLAG_IMMUTABLE، این پرچم برای امنیت در اندروید ۱۲ به بالا الزامی است و اجازه نمی‌دهد برنامه‌های دیگر محتویات اینتنت شما را تغییر دهند.
- setTicker، برای کمک به ابزارهای دسترسی‌پذیری (صفحه‌خوان‌های نابینایان) جهت خواندن صوتی اعلان استفاده می‌شود.
- setAutoCancel(true)، باعث می‌شود به محض اینکه کاربر روی اعلان کلیک کرد، اعلان خودبخود از بالای صفحه پاک شود.
- areNotificationsEnabled() همراه با try-catch، یک زره امنیتی دو لایه است که در اندروید ۱۳ به بالا، اگر کاربر دسترسی نوتیفیکیشن را قطع کرده باشد، جلوی کرش کردن برنامه را به طور کامل می‌گیرد.

تنها یک مرحله باقی مانده است: گرفتن مجوز. باید در اولین اجرا و یا درصورت نگرفتن مجوز در اجراهای قبلی، کادری برای اخذ مجوز به کاربر نشان دهیم.

جریان بررسی و درخواست مجوز نوتیفیکیشن در برنامه ما به این شکل است:

۱. ثبت نام برای دریافت نتیجه (Register Launcher): ابتدا در بالاترین سطح فرگمنت، یک شنونده قرار می دهیم تا وقتی کاربر روی کادر دسترسی کلیک کرد، برنامه نتیجه را بفهمد. فایل PhotoGalleryFragment.kt را باز کنید. ابتدا یک ActivityResultLauncher در بالاترین سطح کلاس تعریف کنید تا نتیجه‌ی کلیک کاربر روی کادر دسترسی را دریافت کند.

لیست ۲۶.۱۴ دریافت نتیجه (PhotoGalleryFragment.kt)

```
class PhotoGalleryFragment : Fragment() {
    ...
    private val requestPermissionLauncher =
        registerForActivityResult(
        ActivityResultContracts.RequestPermission()
    ) { isGranted: Boolean ->
        if (isGranted) {
            Log.d(TAG
            , "Notification permission granted by user.")
            setupBackgroundWork()
        } else {
            Log.w(TAG
```

```
            , "Notification permission denied by user.")
            }
        }
        …
}
```

۲. ما در این مرحله قبل نحوه بررسی مجوز را کردیم که آیا داریم یا خیر و در صورت نبود آن را از کاربر درخواست می‌کنیم، پیاده کردیم. این کد زمان درخواست مجوز و در هنگامی که فرگمنت لود می‌شود، توسط تابع دیگری به نام checkNotificationPermission که آن را در ادامه پیاده می‌کنیم و بررسی می‌کند که آیا گوشی کاربر اصلاً اندروید ۱۳ به بالا هست یا خیر، چک شود. زیرا فقط در صورت بالا بودن نسخه اندروید از ۱۳ به بالا به مجوز نیاز داریم. در فایل PhotoGalleryFragment.kt تابع checkNotificationPermission را در بالاترین سطح کلاس طبق لیست زیر پیاده و تابع ()setupBackgroundWork را به درون آن و در صورتی که اندروید زیر ۱۳ یا مجوز توسط کاربر تایید شود فراخوانی و از (…)onViewCreated حذف نمایید.

لیست ۲۶.۱۵ اخذ مجوز (PhotoGalleryFragment.kt)

```
class PhotoGalleryFragment : Fragment() {
    ...
    private val requestPermissionLauncher =
        registerForActivityResult(
        ActivityResultContracts.RequestPermission()
    ) { ...
    }
    private fun checkNotificationPermission() {
        if (Build.VERSION.SDK_INT >=
            Build.VERSION_CODES.TIRAMISU) {
            val permission =
            Manifest.permission.POST_NOTIFICATIONS
            when {
            ContextCompat.checkSelfPermission(
            requireContext(),
            permission) == PackageManager
            .PERMISSION_GRANTED -> {
                    setupBackgroundWork()
                }
                else -> {
                requestPermissionLauncher.launch(permission)
                }
            }
        } else {
            setupBackgroundWork()
        }
```

```
    }
    ...

    override fun onViewCreated(view: View
    , savedInstanceState: Bundle?) {
        ...
        setupBackgroundWork()
        checkNotificationPermission()
    }
    ...
}
```

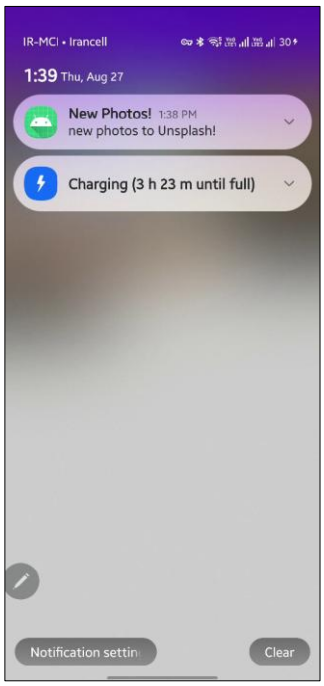
۳. فقط یک مورد دیگر برای نهایی شدن مجوز باقی است: اعلام در مانیفست. فایل AndroidManifest.xml را باز و مجوز زیر را اضافه نمایید.

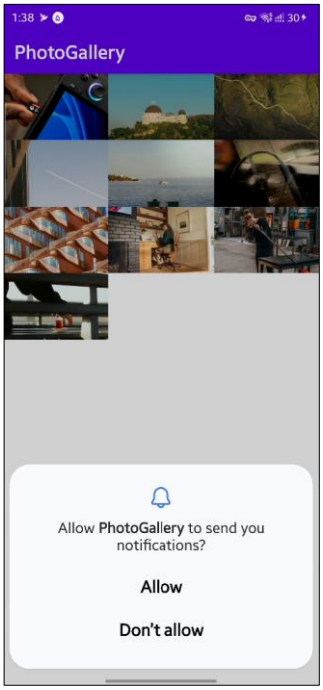
لیست ۲۶.۱۶ اعلام در مانیفست (Manifest.xml)

```
<manifest
xmlns:android="http://schemas.android.com/apk/res/android">
    <uses-permission android:name=
        "android.permission.INTERNET" />
    <uses-permission android:name=
        "android.permission.POST_NOTIFICATIONS" />
    <application
        ...
    </application>
</manifest>
```

کار تمام است. برای اینکه بتوانید کادر اخذ مجوز و آمدن نوتیفیکیشن را با چشمان خود تست کنید، برنامه را یک‌بار پاک کنید (Uninstall) و دوباره نصب کنید تا حافظه SharedPreferences خالی شود. به محض اجرای برنامه، کادر اخذ مجوز ظاهر می شود و در صورت تایید، چون شناسه قبلی وجود ندارد، سیستم‌عامل نوتیفیکیشن را در بالای صفحه گوشی نمایش خواهد داد. با کلیک روی نوتیفیکیشن، سیستم از طریق PendingIntent صفحه PhotoGalleryActivity را برای کاربر باز می‌کند. (نکته: در اندروید ۱۳ به بالا، مطمئن شوید هنگام اجرای برنامه، دکمه‌ی Allow یا اجازه دادن به نوتیفیکیشن‌ها را فشرده‌اید).

تصویر ۲۶.۳ مجوز اعلان





وقتی کاربر اعلان (Notification) را لمس می‌کند، دقیقاً سناریوی زیر به ترتیب در سیستم‌عامل اندروید رخ می‌دهد:

۱. اجرای دستور PendingIntent و باز شدن برنامه:

به دلیل اینکه ما اینتنت اکتیویتی شما را درون یک PendingIntent بسته‌بندی کرده‌ایم، سیستم‌عامل اندروید متوجه می‌شود که باید برنامه را بیدار کند. در نتیجه، بلافاصله اکتیویتی PhotoGalleryActivity اجرا شده و روی صفحه گوشی کاربر نمایش داده می‌شود.

۲. حذف خودکار اعلان از منوی گوشی (AutoCancel):

به دلیل استفاده از متد setAutoCancel(true). در کدهای ساخت اعلان، به محض اینکه کاربر انگشتش را روی نوتیفیکیشن می‌زند و برنامه باز می‌شود، آن اعلان به صورت خودکار از کشوی اعلان‌های بالای گوشی (Notification Drawer) پاک می‌شود تا فضا شلوغ نماند.

۳. ساخته شدن مجدد فرگمنت و اجرای فرآیند واکشی (Fetch) جدید:

وقتی PhotoGalleryActivity بالا می‌آید، طبق کدی که درون آن نوشته‌اید، بررسی می‌کند که آیا بار اول است که اجرا می‌شود یا خیر .(savedInstanceState == null) از آنجایی که برنامه از ابتدا باز شده است، فرگمنت PhotoGalleryFragment را به صورت تازه روی صفحه می‌نشاند. با ساخته شدن فرگمنت، کدهای بخش onViewCreated اجرا شده و اکشنِ collectروی uiState از ویومدل فعال می‌شود. در نتیجه، برنامه بلافاصله آخرین تصاویر Unsplashرا دانلود کرده و به کاربر نشان می‌دهد (محتوای جدیدی که به خاطرش نوتیفیکیشن آمده بود، روی صفحه رندر می‌شود).

قاعدتا برنامه باید به خوبی کار کند و اعلان ها را بعد از بستن برنامه در مورد عکس های جدید نشان دهد اما بعضی مواقع این اتفاق نمی افتد چرا؟

این مشکل که با بستن برنامه نوتیفیکیشن دیگر نمی‌آید، یک رفتار کاملاً شناخته‌شده در اندروید مدرن است و به کد شما مربوط نمی‌شود، بلکه به سیاست‌های سخت‌گیرانه سیستم‌عامل و شرکت‌های سازنده گوشی برمی‌گردد.

ابزار WorkManager وظیفه‌اش را درست انجام می‌دهد و تسک را در دیتابیس داخلی گوشی ذخیره می‌کند، اما سیستم‌عامل اندروید به دلایل زیر جلوی آن را می‌گیرد:

دلیل اصلی وضعیت Force Stop (توقف اجباری) است. وقتی شما برنامه را با کشیدن (Swipe) از لیست برنامه‌های اخیر (Recent Apps) می‌بندید، در برخی از گوشی‌ها (مخصوصاً سامسونگ، شیائومی، هواوی و وان‌پلاس)، سیستم‌عامل این کار را به منزله Force Stop در نظر می‌گیرد. طبق قوانین امنیتی گوگل، وقتی یک اپلیکیشن به وضعیت Force Stop می‌رود، سیستم‌عامل اندروید تمام کارهای پس‌زمینه، آلارم‌ها و کارگرهای WorkManager آن برنامه را تا زمانی که کاربر مجدداً خودش به صورت دستی اپلیکیشن را باز نکند، کاملاً فریز و متوقف می‌کند.

راه‌حل‌های حل غیرفعال کردن بهینه‌سازی باتری (اصلی‌ترین راه) است. به این ترتیب که روی آیکون اپلیکیشن PhotoGallery انگشت خود را نگه دارید و وارد App Info (اطلاعات برنامه) شوید. به بخش Battery (باتری) یا Battery Optimization بروید. وضعیت آن را از حالت Optimized به حالت Unrestricted (بدون محدودیت) تغییر دهید. این کار به سیستم‌عامل دستور می‌دهد که کارگرهای این برنامه را فریز نکند.

و مورد دیگر فعال کردن Auto-start (مخصوص گوشی‌های شیائومی و پوکو) است. اگر گوشی شما شیائومی است، در همان صفحه App Info، گزینه‌ای به نام Auto-start (شروع خودکار) وجود دارد. حتماً آن را روشن کنید تا سیستم‌عامل بعد از بستن برنامه، به کارگرها اجازه دویدن بدهد.

اما ما به عنوان برنامه‌نویس باید این وضعیت را مدیریت کنیم. در خود کد برنامه‌نویسی هم ترفندهایی وجود دارد که می‌توان تا حد زیادی به پایداری این تسک در پس‌زمینه کمک کرد و به سیستم‌عامل نشان داد که این برنامه یک اپلیکیشن حیاتی است.

در اندروید مدرن، ۲ راهکار فنی اصلی درون برنامه برای حل این مشکل وجود دارد:

۱. تبدیل کارگر به یک «تسک فوری» (Foreground Service / Expedited Job): به طور پیش‌فرض، سیستم‌عامل به کارهای WorkManager به چشم یک تسک کم اهمیت نگاه می‌کند که هر وقت گوشی بیکار شد باید اجرا شوند. شما می‌توانید با اضافه کردن قابلیت Expedited به کارگر خود، به اندروید بفهمانید که این کار بالاترین اولویت را دارد و حتی در صورت بستن برنامه یا در وضعیت بهینه‌سازی باتری (Doze Mode) نیز باید با اولویت بالا اجرا شود. برای این کار، ابتدا در فایل PollWorker.kt متد مربوط به سیستم فوری را بازنویسی کنید:

لیست ۲۶.۱۷ تبدیل به تسک فوری (PollWorker.kt)

```
class PollWorker(
    private val context: Context,
    workerParams: WorkerParameters
) : CoroutineWorker(context, workerParams) {
    ...
    override suspend fun getForegroundInfo()
    : ForegroundInfo {
        val notification = NotificationCompat.Builder(
        context, NOTIFICATION_CHANNEL_ID)
            .setSmallIcon(android.R.drawable
                .ic_menu_report_image)
            .setContentTitle(
            "PhotoGallery is checking for updates...")
            .build()
        return ForegroundInfo(NOTIFICATION_ID, notification)
    }
}
```

این متد تکه کدی است که به WorkManager اجازه می‌دهد تا کارگر معمولی شما را به یک تسک فوری با اولویت بالا (Expedited Job) تبدیل کند. در اندروید مدرن، اگر بخواهید کاری در پس‌زمینه با اولویت بالا انجام شود، سیستم‌عامل شما را مجبور می‌کند که یک اعلان سیستم (Notification) به کاربر نشان دهید تا او بداند برنامه‌ای در پس‌زمینه در حال مصرف باتری یا پردازش است. وظیفه متد getForegroundInfo دقیقاً ساختن همین اعلانِ موقت است.

تحلیل خط به خط کد:

```
override suspend fun getForegroundInfo(): ForegroundInfo
```

این یک متد داخلی از کلاس CoroutineWorker است. وقتی سیستم‌عامل تصمیم می‌گیرد کارگر شما را به عنوان یک تسک فوری اجرا کند، ابتدا این متد را صدا می‌زند تا اطلاعات اعلانِ لایه Foreground را دریافت کند.

```
val notification = NotificationCompat.Builder
    (context, NOTIFICATION_CHANNEL_ID)
```

در این بخش، شما یک نوتیفیکیشنِ ساده و موقت می‌سازید. این نوتیفیکیشن متنی مثل " PhotoGallery is checking for updates…" دارد و به کاربر نشان می‌دهد که اپلیکیشن در این لحظه در حال چک کردن تصاویر جدید در پس‌زمینه است.

```
return ForegroundInfo(NOTIFICATION_ID, notification)
```

در نهایت، این اعلان درون شیئی به نام ForegroundInfo بسته‌بندی شده و به سیستم‌عامل تحویل داده می‌شود. شناسه NOTIFICATION_ID به اندروید می‌گوید که این اعلان را با چه آیدیِ مشخصی روی پنل اعلان‌ها مدیریت یا در صورت اتمام کار، حذف کند.

چرایی اهمیت این کد (سیستم چطور کار می‌کند؟)

بدون این متد، کارگر شما در لایه Background اجرا می‌شود و با بستن برنامه، سیستم‌عامل به راحتی آن را نابود می‌کند. اما با اضافه شدن این متد، وقتی تسکِ دوره‌ای بیدار می‌شود، سیستم‌عامل موقتاً این اعلان را بالای گوشی نشان می‌دهد، کارگر را به لایه Foreground منتقل می‌کند تا از گزند سیستم‌های بهینه‌سازی باتری و بسته شدن برنامه در امان بماند، فرآیند دانلود عکس را انجام می‌دهد و به محض اینکه متد ()doWork تمام شد و ()Result.success برگشت، این اعلانِ موقت نیز خودبخود از بالای صفحه گوشی ناپدید می‌شود.

۲. نمایش مستقیم کادر «غیرفعال‌سازی بهینه‌سازی باتری» به کاربر: به جای اینکه کاربر را بفرستید تا در تنظیمات پیچیده اندروید به دنبال گزینه‌ی Unrestricted بگردد، می‌توانید یک دکمه یا دیالوگ در برنامه بگذارید که با کلیک روی آن، کاربر مستقیماً به صفحه تنظیمات باتریِ مخصوص اپلیکیشن شما هدایت شود تا دسترسی کامل پس‌زمینه را به شما بدهد. کد زیر را می‌توانید در اکتیویتی یا فرگمنت خود اجرا کنید (مثلاً در یک دیالوگ به کاربر بگویید: «برای دریافت به موقع نوتیفیکیشن‌ها، لطفاً باتری را روی حالت بدون محدودیت بگذارید»):

لیست ۲۶.۱۸ غیرفعال‌سازی بهینه‌سازی باتری (PollWorker.kt)

```
private val photoAdapter = PhotoAdapter()
fun requestIgnoreBatteryOptimizations() {
    val intent = Intent
    (Settings.ACTION_REQUEST_IGNORE_BATTERY_OPTIMIZATIONS)
    .apply {
        data =
         Uri.parse("package:${requireContext().packageName}")
    }
    startActivity(intent)
}
```

برای استفاده از کد بالا، باید مجوز زیر را در مانیفست بنویسید

لیست ۲۶.۱۹ اعلان (Manifest.xml)

```
<uses-permission android:name="android
    .permission.REQUEST_IGNORE_BATTERY_OPTIMIZATIONS"/>
```

این تابع برای به حداقل رساندن محدودیت‌های پس‌زمینه اندروید روی برنامه شماست. وقتی این تابع فراخوانی می‌شود، یک پنجره (دیالوگ سیستم‌عامل) یا یک صفحه مستقیم از تنظیمات گوشی را به کاربر نشان می‌دهد. سیستم‌عامل در این صفحه از کاربر می‌پرسد: «آیا به این برنامه اجازه می‌دهید بهینه‌سازی باتری را نادیده بگیرد؟»

ترکیب این دو کار در کد نویسی، استانداردترین روشی است که اپلیکیشن‌های بزرگی مثل واتس‌اپ و تلگرام برای زنده ماندن نوتیفیکیشن‌هایشان در پس‌زمینه اندروید استفاده می‌کنند.

این کد را کجا بگذاریم بهتر است؟ از آنجایی که این درخواست نوعی مزاحمت برای کاربر محسوب می‌شود و گوگل نیز به برنامه‌هایی که بدون دلیل این مجوز را درخواست می‌کنند حساس است، نباید آن را به محض باز شدن برنامه یا به صورت اجباری اجرا کنید.

بهترین محل برای قرار دادن آن، استفاده از روش تشویقی یا هشداری (Rationale) است. مراحل زیر بهترین بازخورد را دارد:

۱. یک دیالوگ ساده یا یک بنر کوچک در بالای فرگمنت PhotoGalleryFragment طراحی کنید.

۲. روی آن بنویسید: «برای دریافت منظم نوتیفیکیشن‌ها و تصاویر جدید در پس‌زمینه، لطفاً بهینه‌سازی باتری را برای این برنامه خاموش کنید.»

۳. یک دکمه به نام «تنظیمات باتری» بگذارید.

۴. این تابع را دقیقاً درون کدهای کلیکِ آن دکمه قرار دهید.

برای پیاده‌سازی سناریو اضافه کردن بنر هشدار باتری بالای لیست، باید تغییراتی را در دو بخش انجام دهیم. ابتدا فایل res/layout/fragment_photo_gallery.xml را به‌روزرسانی کنید.

لیست ۲۶.۲۰ بنر هشدار باتری (fragment_photo_gallery.xml)

```
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical">
    <LinearLayout
        android:id="@+id/battery_warning_container"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="horizontal"
        android:background="#FFE082"
        android:padding="12dp"
        android:gravity="center_vertical"
        android:visibility="gone"> <TextView
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:text="For regular updates in background
            , please disable battery optimization."
        android:textColor="#5D4037"
        android:textSize="14sp" />
        <Button
            android:id="@+id/btn_fix_battery"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Fix Now"
            style="?android:attr/buttonStyleSmall"/>
    </LinearLayout>
    <androidx.recyclerview.widget.RecyclerView
        android:id="@+id/photo_recycler_view"
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="1"/>
</LinearLayout>
```

حالا باید در کدهای کاتلین فرگمنت، این بنر را مدیریت کنید. متد checkBatteryOptimization و requestIgnoreBatteryOptimizations را به انتهای کلاس فرگمنت اضافه کنید و در متد onCreateView دکمه و بنر را پیدا کرده و شنونده کلیک را برقرار کنید.

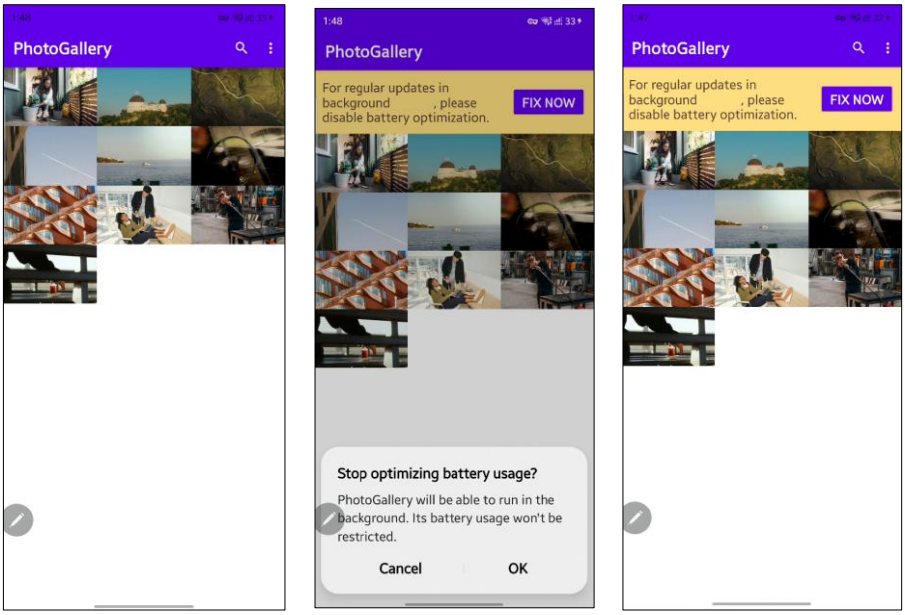
لیست ۲۶.۲۱ مدیریت بنر (PhotoGalleryFragment.kt)

```
class PhotoGalleryFragment : Fragment() {
    ...
    override fun onCreateView(
        inflater: LayoutInflater,
        container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View {
        ...
        photoRecyclerView.adapter = photoAdapter
        val batteryWarningContainer = view.findViewById<
        LinearLayout>(R.id.battery_warning_container)
        val btnFixBattery =
        view.findViewById<Button>(R.id.btn_fix_battery)
        val powerManager = requireContext().getSystemServic(
        Context.POWER_SERVICE) as PowerManager
        if (!powerManager.isIgnoringBatteryOptimizations
        (requireContext().packageName)) {
            batteryWarningContainer.visibility = View.VISIBLE
        }
        btnFixBattery.setOnClickListener {
            requestIgnoreBatteryOptimizations()
        }
        return view
    }
    private fun requestIgnoreBatteryOptimizations() {
        val intent =
        Intent(Settings
        .ACTION_REQUEST_IGNORE_BATTERY_OPTIMIZATIONS)
        .apply {
            data = Uri.parse(
            "package:${requireContext().packageName}")
        }
        startActivity(intent)
    }
    …
}
```

حالا با اجرای برنامه، اگر بهینه‌سازی باتری روی گوشی شما برای این اپلیکیشن فعال باشد، یک بنر زرد رنگ بالای لیست ظاهر می‌شود که با زدن دکمه "Fix Now"، مستقیماً کادر سیستم‌عامل برای خروج از بهینه‌سازی باتری باز خواهد شد.

برنامه را از روی گوشی پاک کنید، سپس آن را اجرا و نتیجه را مشاهده نمایید.

تصویر ۲۶.۴ بهینه سازی باتری



مشاهده می کنید که برنامه در اجرا اول فعال بودن بهینه سازی باتری را چک می‌کند و در صورت فعال بودن مجوز لغو را درخواست می‌کند و درصورت نبودن به کار خود ادامه می دهد و پیغام بالای صفحه را نشان نمی دهد.

آماده سازی کنترل کاربر بر بررسی دوره ای

برخی از کاربران ممکن است نخواهند برنامه ما در پس زمینه اجرا شود. یک کنترل مهم برای ارائه به کاربران، توانایی فعال و غیرفعال کردن بررسی دوره ای پس زمینه است.

برای اینکه به کاربر کنترل کامل بدهید تا بتواند بررسی دوره‌ای در پس‌زمینه را فعال یا غیرفعال کند، باید فرآیند را در ۳ گام ساده پیاده‌سازی کنیم:

۱. ذخیره وضعیت در QueryPreferences (برای به خاطر سپردن انتخاب کاربر).

۲. اضافه کردن آیتم منو به نوار ابزار (با متن پویای "Start Polling" یا "Stop Polling").

۳. تغییر منطق اجرا در فرگمنت (روشن یا خاموش کردن WorkManager).

گام اول: تکمیل کلاس QueryPreferences

ابتدا باید یک کلید و دو متد جدید به کلاس QueryPreferences خود اضافه کنیم تا وضعیت فعال/غیرفعال بودن کارگر (Polling) را در SharedPreferences ذخیره و بازیابی کند.

کد زیر را به کلاس QueryPreferences.kt اضافه کنید.

لیست ۲۶.۲۲ ذخیره وضعیت Worker (QueryPreferences.kt)

```
…
private const val PREF_IS_POLLING = "isPolling"
object QueryPreferences {
    ...
    fun isPolling(context: Context): Boolean {
        val prefs = context.getSharedPreferences(
        "photo_gallery_prefs", Context.MODE_PRIVATE)
        return prefs.getBoolean(PREF_IS_POLLING, false)
    }
    fun setPolling(context: Context, isPolling: Boolean) {
        val prefs = context.getSharedPreferences(
        "photo_gallery_prefs", Context.MODE_PRIVATE)
        prefs.edit {
            putBoolean(PREF_IS_POLLING, isPolling)
        }
    }
}
```

گام دوم: اضافه کردن آیتم به منو (fragment_photo_gallery.xml)

فایل منوی خود در مسیر res/menu/fragment_photo_gallery.xml را باز کرده و آیتم زیر را به آن اضافه کنید. عنوان (Title) این آیتم در کد کاتلین به صورت پویا تغییر خواهد کرد.

لیست ۲۶.۲۳ افزودن یک مورد نظرسنجی (res/menu/fragment_photo_gallery.xml)

```
<menu
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto">
    …
    <item
        android:id="@+id/menu_item_toggle_polling"
        android:title="Toggle Polling"
        app:showAsAction="ifRoom" />
</menu>
```

گام سوم: به‌روزرسانی PhotoGalleryFragment.kt

حالا باید منو را مدیریت کنیم. منطق کار به این صورت است: اگر کارگر روشن باشد، منو گزینه " Stop Polling" را نشان می‌دهد و در صورت کلیک، آن را لغو می‌کند. اگر خاموش باشد، گزینه " Start Polling" نمایش داده شده و با کلیک، آن را فعال می‌کند.

تغییرات زیر را در متد addMenuProvider در کلاس PhotoGalleryFragment.kt اعمال کنید.

لیست ۲۶.۲۴ تنظیم صحیح متن آیتم منو (PhotoGalleryFragment.kt)

```
override fun onViewCreated(view: View, savedInstanceState:
Bundle?) {
    ...
    requireActivity().addMenuProvider(object : MenuProvider {
        override fun onCreateMenu(menu: Menu, menuInflater
        : MenuInflater) {
            menuInflater.inflate
           (R.menu.fragment_photo_gallery, menu)
            ...
            val toggleItem = menu.findItem(
            R.id.menu_item_toggle_polling)
            val isPolling = QueryPreferences.isPolling(
            requireContext())
            val toggleItemTitle = if (isPolling)
            "Stop Polling" else "Start Polling"
            toggleItem.title = toggleItemTitle
        }
        override fun onMenuItemSelected(menuItem: MenuItem)
        : Boolean {
            return when (menuItem.itemId) {
                R.id.menu_item_clear -> {
```

```
                ...
            }
            R.id.menu_item_toggle_polling -> {
                val isPolling =
            QueryPreferences.isPolling(requireContext())
                if (isPolling) {
                    WorkManager.getInstance(
                    requireContext().applicationContext)
                        .cancelUniqueWork(POLL_WORK_NAME)
                    QueryPreferences.setPolling
                    (requireContext(), false)
                } else {
                    setupBackgroundWork()
                    QueryPreferences.setPolling(
                    requireContext(), true)
                }
                activity?.invalidateOptionsMenu()
                true
            }
            else -> false
        }
    }
}, viewLifecycleOwner, Lifecycle.State.RESUMED)
checkNotificationPermission()
}
```

در نهایت، یک اصلاح کوچک در onViewCreated، برای اینکه برنامه در بدو ورود کورکورانه کارگر را استارت نزند، خط انتهای onViewCreated که متد checkNotificationPermission یا setupBackgroundWork را مستقیماً صدا می‌زد، به این صورت شرطی کنید.

فهرست ۲۶.۲۵ مدیریت کلیک‌های مورد تغییر نظرسنجی (PhotoGalleryFragment.kt)

```
override fun onViewCreated(view: View, savedInstanceState
    : Bundle?) {
    ...
    checkNotificationPermission()
    if (QueryPreferences.isPolling(requireContext())) {
        checkNotificationPermission()
    }
}
```

کار تمام است، وقتی برنامه برای اولین بار روی گوشی کاربر نصب و باز می‌شود، جریان زیر اتفاق می‌افتد:

- سیستم وارد متد onViewCreated می‌شود و می‌خواهد شرط بالا را بررسی کند.
- برای ارزیابی شرط، متد QueryPreferences.isPolling() صدا زده می‌شود.

- درون این متد، سیستم به سراغ حافظه تنظیمات (SharedPreferences) می‌رود تا ببیند آیا مقداری برای کلید isPolling ذخیره شده است یا خیر.
- چون برنامه تازه نصب شده و اولین اجراست، هیچ مقداری وجود ندارد.
- به دلیل نبودن مقدار، سیستم مقدار پیش‌فرضی که خودمان تعیین کردیم (یعنی false) را برمی‌گرداند.
- در نتیجه، شرط false می‌شود، کدهای داخل آن (که کارگر را استارت می‌زدند) اجرا نمی‌شوند و Polling خاموش می‌ماند.

برنامه را پس از حذف اجرا نمایید. خواهید دید که در اولین اجرا آیتمی در منو با عنوان START POLLING به نمایش در می‌آید که اگر آن را لمس کنید، عنوان آن به STOP POLLING تغییر می‌کند و بررسی دوره ای را استارت میزند.

تصویر ۲۶.۵ بررسی دوره‌ای





PhotoGallery اکنون می‌تواند کاربر را به طور خودکار با آخرین تصاویر به روز نگه دارد، حتی زمانی که برنامه اجرا نمی‌شود. اما یک مشکل وجود دارد: هر زمان که تصاویر جدید وارد شوند، کاربر اعلان دریافت می‌کند – حتی زمانی که برنامه از قبل در حال اجرا است. این مطلوب نیست، زیرا توجه کاربر را از برنامه شما دور می‌کند.

در فصل بعدی با جلوگیری از نمایش اعلان‌ها در حین اجرای PhotoGallery این مشکل را برطرف خواهید کرد. در انجام این به روزرسانی‌ها، یاد خواهید گرفت که چگونه به broadcast intents گوش دهید و چگونه با استفاده از broadcast receiver چنین اینتنت‌هایی را مدیریت کنید.

۲۷

اینتنت‌های Broadcast

هم دریافت اعلان و هم مشاهده به روز رسانی نتایج در صفحه زمانی که به طور فعال یک برنامه را مشاهده می‌کنید، آزاردهنده و اضافی است. اگر کاربر همین الان داخل برنامه است و دارد عکس‌ها را می‌بیند، چرا باید با یک نوتیفیکیشن تکراری مزاحمش شویم؟ نوتیفیکیشن فقط برای زمانی است که کاربر بیرون از برنامه حضور دارد.

در این فصل، رفتار بررسی دوره ای PhotoGallery را با جلوگیری از نمایش اعلان‌های مربوط به عکس‌های جدید در حین اجرای برنامه، اصلاح می‌کنیم.

در انجام این به روزرسانی‌ها، یاد خواهیم گرفت که چگونه به اینتنت‌های Broadcast گوش دهیم و چگونه با استفاده از broadcast receiver چنین اینتنت‌هایی را مدیریت کنیم. همچنین به صورت پویا اینتنت‌های Broadcast را در برنامه خود در زمان اجرا، ارسال و دریافت خواهیم کرد. در نهایت، از Ordered Broadcasts استفاده خواهیم کرد تا تعیین کنیم که آیا برنامه ما در حال حاضر در پیش زمینه اجرا می‌شود یا خیر.

اینتنت‌های Regular در مقابل اینتنت‌های Broadcast

رویدادهای مختلفی در دستگاه اندرویدی اتفاق می‌افتد. وای فای در حال ورود و خروج از محدوده است، پکیج‌ها در حال نصب هستند، و تماس‌های تلفنی و پیام‌های متنی در حال آمدن و رفتن می‌باشند.

ممکن است مولفه‌های زیادی در سیستم وجود داشته باشد که باید از این نوع رویدادها آگاه باشند تا بتوانند بر اساس آن واکنش نشان دهند. برای اطلاع رسانی به این مؤلفه‌ها، اندروید از broadcast intent استفاده می‌کند تا به همه در مورد آن بگوید.

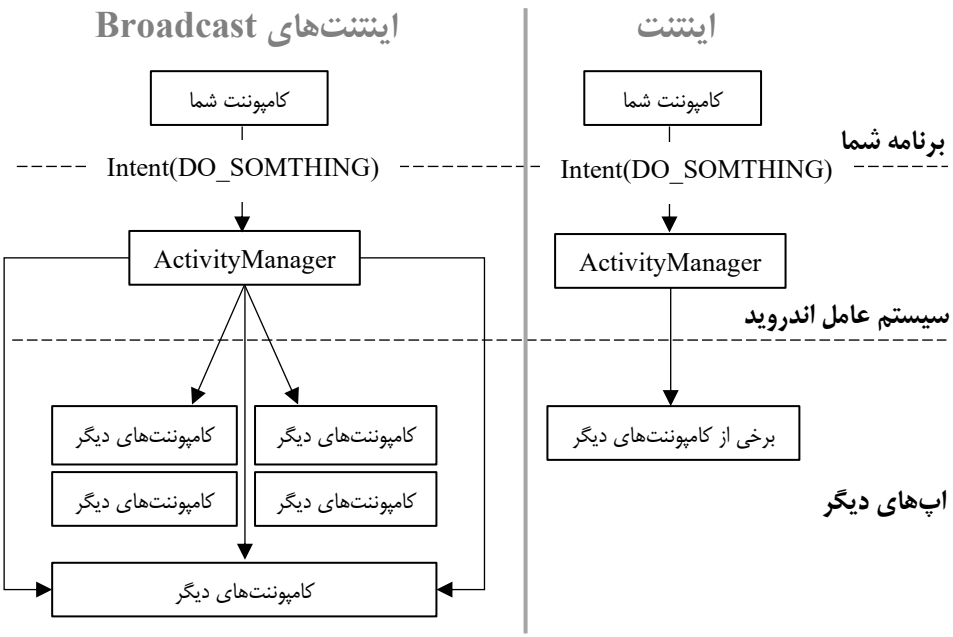
اینتنت‌های معمولی مثل یک نامه‌ی سفارشی و آدرس‌دار هستند. شما نامه‌ای می‌نویسید و روی آن دقیقاً قید می‌کنید که مقصد کدام اکتیویتی یا سرویس است (مثلاً باز کردن صفحه جزئیات یک عکس). سیستم‌عامل اندروید مطمئن می‌شود که این نامه فقط و فقط به دست همان یک مولفه‌ی مشخص برسد.

اینتنت‌های همگانی مثل یک فرستنده رادیویی یا یک بلندگوی بزرگ در وسط شهر هستند. وقتی سیستم‌عامل یا برنامه شما یک Broadcast ارسال می‌کند، هیچ مقصد مشخصی روی آن نوشته نشده است. پیام در فضای اندروید فریاد زده می‌شود: «وای‌فای قطع شد!» یا «عکس جدید دانلود شد!».

Broadcastهایی که از سیستم ارسال می‌شوند، به عنوان Broadcast سیستم شناخته می‌شوند، اما می‌توانیم Broadcastهای سفارشی (custom broadcasts) خود را نیز ارسال و دریافت کنیم. مکانیسم دریافت هر دو Broadcastهای سیستمی و سفارشی یکسان هستند، اما ما فقط با Broadcastهای سفارشی در این فصل کار خواهیم کرد.

اینتنت‌های Broadcast مشابه اینتنت‌های که قبلا می‌شناسید و دوست دارید کار می‌کنند، با این تفاوت که می‌توانند توسط چندین مولفه به نام broadcast receiver به طور همزمان دریافت شوند (شکل ۲۸.۱).

شکل ۲۷.۱ اینتنت‌های عادی در مقابل اینتنت‌های Broadcast



اکتیویتی‌ها و serviceها باید هر زمان که به عنوان بخشی از یک API عمومی استفاده می‌شوند، به اینتنت‌های ضمنی پاسخ دهند. هنگامی که آنها بخشی از یک API عمومی‌نیستند، اینتنت های صریح تقریبا همیشه کافی هستند. از سوی دیگر، Broadcast receiverها فقط برای ارسال اطلاعات به بیش از یک شنونده وجود دارد. بنابراین در حالی که Broadcast receiverها می‌توانند به اهداف صریح پاسخ

دهند، به ندرت از این روش استفاده می‌شوند، زیرا اینتنت‌های صریح فقط می‌توانند یک گیرنده داشته باشند.

فیلتر کردن اعلان‌ها در پیش‌زمینه (Foreground Notification Filtering)

اعلان‌های ما عالی کار می‌کنند، اما حتی زمانی که کاربر از قبل برنامه را باز کرده باشد، ارسال می‌شوند. می‌توانیم از اینتنت‌های Broadcast برای تغییر رفتار PollWorker بر اساس اینکه آیا برنامه شما در پیش زمینه است یا خیر، استفاده کنیم.

در توسعه اندروید مدرن، مدیریت تجربه کاربری (UX) ایجاب می‌کند که از ارسال اعلان‌های تکراری و آزاردهنده زمانی که کاربر به طور فعال در حال تعامل با محیط برنامه است، خودداری شود. از آنجایی که کامپوننت‌های پس‌زمینه مانند WorkManager (یا همان PollWorker) در لایه‌ای مجزا از لایه رابط کاربری (UI) اجرا می‌شوند، به طور مستقیم به چرخه حیات (Lifecycle) فرگمنت‌ها یا اکتیویتی‌ها دسترسی ندارند تا وضعیت پیش‌زمینه بودن برنامه را تشخیص دهند.

برای حل این چالش، از معماری ارسال سیگنال همگانی (Broadcast Intents) و سیستم اولویت‌بندی گیرنده‌ها (Ordered Broadcasts) استفاده می‌شود. ابتدا، هر زمان که عکس‌های جدیدی دریافت می‌شود، broadcast intent را از PollWorker خود ارسال خواهیم کرد. در مرحله بعد، دو broadcast receiver را ثبت خواهیم کرد. اولین receiver در مانیفست اندروید ما ثبت می‌شود. هر زمان که broadcast از PollWorker دریافت کند، اعلان را مانند گذشته برای کاربر ارسال می‌کند. گیرنده دوم به صورت پویا ثبت می‌شود تا تنها زمانی فعال باشد که برنامه ما برای کاربر قابل مشاهده باشد. وظیفه آن رهگیری broadcast از تحویل به گیرنده پخش در مانیفست شما خواهد بود که از ارسال اعلان جلوگیری می‌کند.

به دلیل عدم وجود یک متد مستقیم برای بررسی وضعیت رویت برنامه توسط کامپوننت‌های پس‌زمینه، استراتژی پیاده‌سازی بر پایه دو گیرنده مجزا با سطوح دسترسی متفاوت بنا می‌شود:

۱. گیرنده ایستا یا ثبت‌شده در مانیفست (Static Receiver): این گیرنده به صورت دائم در فایل AndroidManifest.xml تعریف می‌شود. وظیفه این کامپوننت، گوش دادن به سیگنال جدید آمده از سمت PollWorker در هر شرایطی است. اگر برنامه بسته باشد، این گیرنده پیام را دریافت کرده و اعلان سیستم (Notification) را به کاربر نشان می‌دهد.

۲. گیرنده پویا یا ثبت‌شده در زمان اجرا (Dynamic/Runtime Receiver): این گیرنده صرفاً زمانی که کاربر درون برنامه است و فرگمنت در وضعیت فعال (Resumed) قرار دارد، به صورت

پویا در کد کاتلین ثبت (Register) می‌شود. این گیرنده دارای اولویت بالاتری (Higher Priority) نسبت به گیرنده مانیفست است.

ممکن است استفاده از دو broadcast receiver برای انجام این کار غیرعادی به نظر برسد، اما اندروید مکانیزمی‌برای تعیین اینکه کدام اکتیویتی‌ها یا فرگمنت‌ها در حال حاضر در حال اجرا هستند ارائه نمی‌دهد. از آنجایی که PollWorker نمی‌تواند مستقیما بگوید که‌ایا رابط کاربری شما قابل مشاهده است یا خیر، نمی‌توانیم از ارسال اعلان با استفاده از یک عبارت if ساده در PollWorker صرف نظر کنیم. به همین ترتیب، نمی‌توان انتخاب کرد که broadcast را به صورت مشروط بر اساس قابل مشاهده بودن PhotoGallery ارسال کنیم، زیرا راهی برای تعیین اینکه برنامه ما در کدام حالت قرار دارد وجود ندارد. با این حال، می‌توانید از دو گیرنده استفاده کنید و آنها را طوری تنظیم کنید که فقط یکی از آنها به پخش واکنش نشان دهد. این کاری است که ما انجام خواهیم داد.

کلید اصلی این فرآیند، ارسال پیام به صورت Ordered Broadcast است. Ordered Broadcast یعنی برادکست ترتیبی در اندروید؛ یعنی سیستم پیام را به چند Receiver می‌فرستد اما به‌صورت نوبتی و پشت‌سرهم و نه هم‌زمان. در این حالت، پیام‌ها به صورت همزمان به همه فرستاده نمی‌شوند، بلکه به ترتیب اولویت توزیع می‌گردند:

- برنامه در پیش‌زمینه است: ابتدا گیرنده پویا (که اولویت بالاتری دارد) پیام را دریافت می‌کند. این گیرنده متوجه می‌شود که کاربر در حال تماشای برنامه است؛ در نتیجه، پیام را سقط (Abort) می‌کند. با سقط شدن پیام، انتشار آن متوقف شده و هرگز به دست گیرنده مانیفست نمی‌رسد. نتیجه: هیچ اعلانی نمایش داده نمی‌شود.
- برنامه در پس‌زمینه است یا بسته شده است: گیرنده پویا به دلیل خروج کاربر از برنامه، از چرخه سیستم حذف شده است (Unregistered). بنابراین، پیام مستقیماً به تنها گیرنده موجود، یعنی گیرنده مانیفست می‌رسد. نتیجه: اعلان سیستم به کاربر نشان داده می‌شود.

ارسال اینتنت‌های Broadcast

اولین قدم ساده است: باید اینتنت Broadcast خود را ارسال و به مولفه‌های علاقه مند اطلاع دهیم که اعلان نتایج جستجوی جدید آماده ارسال است. برای ارسال یک اینتنت Broadcast، یک اینتنت ایجاد و آن را به sendBroadcast(Intent) ارسال کنید. در این مورد، ما می‌خواهیم اقدامی‌را که تعریف کرده‌ایم Broadcast کنیم، بنابراین یک ثابت اکشن تعریف کنید.

لیست ۲۷.۱ ارسال اینتنت Broadcast (PollWorker.kt)

```
class PollWorker(
    private val context: Context,
    workerParams: WorkerParameters
) : CoroutineWorker(context, workerParams) {
    @RequiresPermission(
    Manifest.permission.POST_NOTIFICATIONS)
    override suspend fun doWork(): Result {
        …
        if (currentResultId == lastResultId) {
            Log.i(…)
        } else {
            …
            val intent = PhotoGalleryActivity.newIntent(
            context)
            val pendingIntent = PendingIntent.getActivity(
                context,
                0,
                intent,
                PendingIntent.FLAG_IMMUTABLE
            )
            val notification = NotificationCompat.Builder(
                context, NOTIFICATION_CHANNEL_ID)
                .setSmallIcon(
                android.R.drawable.ic_menu_report_image)
                .setContentTitle("New Photos!")
                .setContentText("new photos to Unsplash!")
                .setContentIntent(pendingIntent)
                .setAutoCancel(true)
                .build()
            val notificationManager =
                NotificationManagerCompat.from(context)
            if (ContextCompat.checkSelfPermission(
                    context,
                    Manifest.permission.POST_NOTIFICATIONS
                ) == PackageManager.PERMISSION_GRANTED
            ) {
                notificationManager.notify(0, notification)
            }
            context.sendBroadcast(
            Intent(ACTION_SHOW_NOTIFICATION))
        }
        return Result.success()
    }
    override suspend fun getForegroundInfo()
            : ForegroundInfo {
        …
    }
    companion object {
```

```
        const val ACTION_SHOW_NOTIFICATION =
            "com.codeb.photogallery.SHOW_NOTIFICATION"
    }

}
```

ثابت ACTION_SHOW_NOTIFICATION تعریف شد و دستور sendBroadcast یک سیگنال عمومی با این اکشن در سیستم شلیک می‌کند.

ایجاد و رجیستر گیرنده

اگرچه Broadcast ما ارسال می‌شود، اما هنوز هیچ کسی به آن گوش نمی‌دهد. برای واکنش به Broadcast، یک BroadcastReceiver را پیاده‌سازی خواهیم کرد. دو نوع گیرنده Broadcast وجود دارد. در اینجا از یک گیرنده Broadcast مستقل (standalone) استفاده خواهیم کرد.

در معماری اندروید، گیرنده‌های همگانی به دو دسته کلی مستقل (Standalone / Static) و پویا (Dynamic / Runtime) تقسیم می‌شوند.

یک گیرنده مستقل، کامپوننتی است که به طور رسمی در فایل مانیفست (AndroidManifest.xml) برنامه اعلام (Declare) می‌شود. مزیت کلیدی این نوع گیرنده آن است که به چرخه حیات رابط کاربری یا باز بودن برنامه وابسته نیست؛ یعنی حتی اگر فرآیند (Process) اپلیکیشن توسط سیستم‌عامل یا کاربر به طور کامل بسته شده باشد، سیستم‌عامل اندروید با وقوع رویداد مورد نظر، فرآیند برنامه را مجدداً احیا کرده، کلاس گیرنده را نمونه‌سازی و متد پردازشی آن را اجرا می‌کند.

قبل از اینکه بتوانیم گیرنده Broadcast خود را ثبت کنیم، باید آن را پیاده کنیم. یک کلاس کاتلین جدید به نام NotificationReceiver که زیر کلاس android.content.BroadcastReceiver است، ایجاد کنید.

لیست ۲۷.۲ اولین گیرنده Broadcast شما (NotificationReceiver.kt)

```
package com.codeb.photogallery
import android.content.BroadcastReceiver
import android.content.Context
import android.content.Intent
import android.util.Log
private const val TAG = "NotificationReceiver"
class NotificationReceiver : BroadcastReceiver() {
    override fun onReceive(context: Context, intent: Intent)
{
        Log.i(TAG,
```

```
            "Received broadcast intent: ${intent.action}")
    }
}
```

گیرنده Broadcast مؤلفه‌ای است که درست مانند یک سرویس یا یک اکتیویتی، اینتنت‌ها را دریافت می‌کند. هنگامی که اینتنتی برای NotificationReceiver صادر می‌شود، onReceive() تابع فراخوانی خواهد شد.

در مرحله بعد، manifests/AndroidManifest.xml را باز و NotificationReceiver را به عنوان یک گیرنده مستقل وصل کنید.

لیست ۲۷.۳ اضافه کردن گیرنده خود به مانیفست (manifests/AndroidManifest.xml)

```
<manifest
xmlns:android="http://schemas.android.com/apk/res/android">
    ...
    <application...>
        <activity.../>
        <receiver
            android:name=".NotificationReceiver">
        </receiver>
    </application>
</manifest>
```

برای دریافت Broadcastها توسط گیرنده ما باید یک فیلتر اینتنت نیز به آن بدهیم. این فیلتر دقیقا مانند فیلترهایی که با اینتنت‌های ضمنی استفاده کرده‌ایم رفتار می‌کند، با این تفاوت که به جای اینتنت‌های معمولی، اینتنت‌های Broadcast را فیلتر می‌کند. یک فیلتر اینتنت به گیرنده Broadcast خود اضافه کنید تا اینتنت‌های را با عمل SHOW_NOTIFICATION بپذیرید .

لیست ۲۷.۴ افزودن یک فیلتر اینتنت به گیرنده (manifests/AndroidManifest.xml)

```
<manifest
xmlns:android="http://schemas.android.com/apk/res/android">
    ...
    <application...>
        <activity.../>
        <receiver android:name=".NotificationReceiver"
    android:exported="true">
    <intent-filter>
        <action android:name=
            "com.codeb.photogallery.SHOW_NOTIFICATION" />
    </intent-filter>
</receiver>
    </application>
</manifest>
```

ریسیور جدید با پرچم android:exported="true" درون مانیفست ثبت شد تا سیستم‌عامل اندروید بداند به محض شلیک این اکشن، باید کلاس NotificationReceiver را بیدار کند.

اگر پس از انجام این تغییرات، PhotoGallery را روی دستگاهی با Android Oreo یا بالاتر اجرا کنید، گزارش خود را نخواهید دید. در واقع، گیرنده شما onReceive(…) تابع به هیچ وجه فراخوانی نخواهد شد. اما در نسخه‌های قدیمی اندروید، گزارش شما همانطور که انتظار دارید ظاهر می‌شود. این به دلیل محدودیت‌هایی است که نسخه‌های جدیدتر اندروید در Broadcast اعمال می‌کنند. اما نترسید – کار شما تا کنون بیهوده نیست.

محدود کردن Broadcast به برنامه با استفاده از مجوزهای خصوصی

یکی از مشکلات Broadcast این است که هر کسی در سیستم می‌تواند به آن گوش دهد یا گیرنده‌های شما را فعال کند. ما معمولا نمی‌خواهیم هیچ یک از این چیزها اتفاق بیفتد. به راحتی، مشخص کردن مجوز در Broadcast ما باعث می‌شود گیرنده Broadcast جدید ما روی نسخه‌های جدیدتر اندروید کار کند، بنابراین می‌توانیم دو باگ را با قیمت یکی برطرف کنید.

ما می‌توانیم با اعمال مجوز سفارشی به گیرنده و تنظیم ویژگی android:exported گیرنده روی "false" از این نفوذ غیرمجاز به کسب و کار شخصی خود جلوگیری کنیم. این مانع از قابل مشاهده شدن گیرنده برای سایر برنامه‌های سیستم می‌شود. اعمال مجوز به این معنی است که فقط مولفه‌هایی که مجوز درخواست کرده‌اند (و اعطا شده‌اند) می‌توانند Broadcast را به گیرنده تحویل دهند.

ابتدا اجازه خود را در AndroidManifest.xml اعلام و دریافت کنید.

لیست ۲۷.۵ افزودن مجوز خصوصی (manifests/AndroidManifest.xml)

```
<manifest
xmlns:android="http://schemas.android.com/apk/res/android">
…
    <permission android:name=
        "com.codeb.photogallery.PRIVATE_SHOW_NOTIFICATION"
        android:protectionLevel="signature" />
    <uses-permission android:name=
        "com.codeb.photogallery.PRIVATE_SHOW_NOTIFICATION" />
</manifest>
```

توجه داشته باشید که یک مجوز سفارشی با سطح محافظت از signature تعریف می‌کنیم. با سطوح حفاظت بیشتر آشنا خواهید شد. خود مجوز یک رشته ساده است، درست مانند اکشن‌های اینتنت، دسته‌ها و مجوزهای سیستمی‌که استفاده کرده‌اید. ما همیشه باید اجازه استفاده از آن را دریافت کنیم، حتی زمانی که خودتان آن را تعریف کرده‌اید. قانون به این شکل است.

به هر حال به مقدار ثابت سایه دار بالا توجه کنید. این رشته شناسه منحصر به فرد برای مجوز سفارشی است که از آن برای ارجاع به مجوز در جای دیگری در مانیفست خود و همچنین از کد کاتلین خود هنگام صدور اینتنت Broadcast برای گیرنده استفاده خواهیم کرد. شناسه باید در هر مکان یکسان باشد. عاقلانه است که به جای تایپ با دست، آن را کپی و جایگذاری کنید.

در مرحله بعد، مجوز را به برچسب گیرنده اعمال کنید و ویژگی exported را روی "false" تنظیم کنید.

لیست ۲۷.۶ اعمال مجوز (manifests/AndroidManifest.xml)

```
<receiver android:name=".NotificationReceiver"
    android:exported="~~true~~false">
    <intent-filter>
        <action android:name=
            "com.codeb.photogallery.SHOW_NOTIFICATION" />
    </intent-filter>
</receiver>
```

اکنون، از مجوز خود با تعریف یک ثابت در کد و سپس ارسال آن به فراخوانی sendBroadcast(…) استفاده کنید.

لیست ۲۷.۷ ارسال با مجوز (PollWorker.kt)

```
class PollWorker(
    private val context: Context,
    workerParams: WorkerParameters
) : CoroutineWorker(context, workerParams) {
    companion object {
        const val ACTION_SHOW_NOTIFICATION =
            "com.codeb.photogallery.SHOW_NOTIFICATION"
        const val PERM_PRIVATE =
        "com.codeb.photogallery.PRIVATE_SHOW_NOTIFICATION"
    }
    override suspend fun doWork(): Result {
        …
            QueryPreferences.setStoredLastResultId
            (context, currentResultId)
            ~~context.sendBroadcast~~
            ~~(Intent(ACTION_SHOW_NOTIFICATION))~~
            val intent =
            Intent(ACTION_SHOW_NOTIFICATION).apply {
                setClass(context
                , NotificationReceiver::class.java)
            }
            context.sendBroadcast(intent, PERM_PRIVATE)
        }
        return Result.success()
    }
    ...
}
```

اکنون، برنامه شما تنها برنامه‌ای است که می‌تواند آن گیرنده را فعال کند. PhotoGallery را دوباره اجرا کنید. باید گزارش خود را از NotificationReceiver در Logcat ببینید (اگرچه اعلان‌های شما هنوز آداب مناسب را نمی‌دانند و تا زمانی که برنامه در پیش زمینه است – درست مانند قبل از این فصل – نمایش داده می‌شوند).

تصویر ۲۷.۲ اعلان دریافت تصویر جدید



اطلاعات بیشتر در مورد سطوح محافظت

هر مجوز سفارشی باید مقداری را برای android:protectionLevel مشخص کند. سطح محافظت به Android می‌گوید که چگونه باید از آن استفاده شود. ما از سطح حفاظت signature استفاده کرده‌ایم.

سطح محافظت signature به این معنی است که اگر برنامه دیگری بخواهد از مجوز شما استفاده کند، باید با همان کلید برنامه شما امضا شود. این معمولا انتخاب درستی برای مجوزهایی است که به صورت داخلی در برنامه خود استفاده می‌کنید. از آنجایی که سایر توسعه دهندگان کلید شما را ندارند، نمی‌توانند به چیزی که این مجوز محافظت می‌کند دسترسی داشته باشند. به علاوه، از آنجایی که کلید خود را دارید، می‌توانید از این مجوز در هر برنامه دیگری که تصمیم دارید بعدا بنویسید استفاده کنید.

چهار سطح حفاظت موجود در جدول ۲۸.۱ خلاصه شده است:

جدول ۲۸.۱ مقادیر سطح حفاظت

ارزش	توضیحات
normal	این برای محافظت از عملکرد برنامه است که هیچ کار خطرناکی انجام نمی‌دهد، مانند دسترسی به داده‌های شخصی امن یا پیدا کردن مکان روی نقشه. کاربر می‌تواند قبل از انتخاب نصب برنامه، مجوز را ببیند، اما هرگز به صراحت از او خواسته نمی‌شود که آن را اعطا کند. android.permission.INTERNET از این سطح مجوز استفاده می‌کند و همچنین مجوزی که به برنامه شما اجازه می‌دهد دستگاه کاربر را به لرزه درآورد.
dangerous	این برای بسیاری از چیزهایی است که شما برای استفاده عادی استفاده نمی‌کنید – دسترسی به داده‌های شخصی، دسترسی به سخت افزاری که ممکن است برای جاسوسی از کاربر استفاده شود، یا هر چیز دیگری که می‌تواند مشکلات واقعی ایجاد کند. مجوز دوربین، مجوز مکان و مجوز مخاطبین همگی در این دسته قرار می‌گیرند. با شروع درMarshmlow ، مجوزهای خطرناک مستلزم آن است که requestPermission(...) در زمان اجرا از کاربر بخواهید صریحا به برنامه شما اجازه دهد.
signature	اگر برنامه با همان گواهی برنامه اعلام کننده امضا شده باشد و در غیر این صورت آن را رد کند، سیستم این مجوز را اعطا می‌کند. اگر مجوز داده شود، به کاربر اطلاع داده نمی‌شود. این برای عملکردی است که داخلی یک برنامه است.
signatureOrSystem	به عنوان توسعه دهنده، از آنجایی که شما گواهی را دارید و فقط برنامه‌هایی که با یک گواهی امضا شده‌اند می‌توانند از مجوز استفاده کنند، کنترل دارید که چه کسی از مجوز استفاده می‌کند. شما از آن در اینجا استفاده کردید تا از دیدن دیگران استفاده کنید

ایجاد و رجیستر گیرنده پویا

در مرحله بعد، برای اینتنت ACTION_SHOW_NOTIFICATION مربوط به Broadcast خود به یک گیرنده نیاز داریم. وظیفه این گیرنده جلوگیری از ارسال اعلان در حین استفاده کاربر از برنامه ما خواهد بود.

این گیرنده فقط زمانی ثبت می‌شود که اکتیویتی شما در پیش زمینه باشد. اگر این گیرنده با طول حیات طولانی تر (مانند طول حیات فرآیند برنامه شما) اعلام شده باشد، به روش دیگری برای دانستن اینکه PhotoGalleryFragment در حال اجرا است نیاز داریم (که در وهله اول اینتنت داشتن این گیرنده پویا را شکست می‌دهد).

راه حل استفاده از گیرنده Broadcast پویا است.

ما گیرنده را با فراخوانی Context.registerReceiver(BroadcastReceiver, IntentFilter) ثبت می‌کنیم و با فراخوانی Context.unregisterReceiver(BroadcastReceiver) آن را حذف می‌کنیم. خود گیرنده معمولا به عنوان یک کلاس داخلی یا یک لامبدا تعریف می‌شود، مانند شنونده با کلیک دکمه. با این حال، از آنجایی که شما به همان نمونه در registerReceiver(…) نیاز داریم و unregisterReceiver(…)، باید گیرنده را به یک متغیر نمونه اختصاص دهیم.

یک کلاس انتزاعی جدید به نام VisibleFragment با سوپرکلاس Fragment ایجاد کنید. این کلاس یک فرگمنت عمومی خواهد بود که اعلان‌های پیش زمینه را پنهان می‌کند.

فهرست ۲۷.۸ گیرنده خود VisibleFragment (VisibleFragment.kt)

```
package com.codeb.photogallery
import android.content.BroadcastReceiver
import android.content.Context
import android.content.Intent
import android.content.IntentFilter
import android.util.Log
import androidx.core.content.ContextCompat
import androidx.fragment.app.Fragment
private const val TAG = "VisibleFragment"
abstract class VisibleFragment : Fragment() {
    private val onShowNotification = object
    : BroadcastReceiver() {
        override fun onReceive(context: Context, intent
        : Intent) {
            Log.i(TAG,
            "Foreground activity detected: intercepting ${in-
tent.action}")
```

```
        }
    }
    override fun onStart() {
        super.onStart()
        val filter = IntentFilter(
        PollWorker.ACTION_SHOW_NOTIFICATION)
        ContextCompat.registerReceiver(
            requireContext(),
            onShowNotification,
            filter,
            PollWorker.PERM_PRIVATE,
            null,
            ContextCompat.RECEIVER_NOT_EXPORTED
        )
    }
    override fun onStop() {
        super.onStop()
        requireContext().unregisterReceiver(
        onShowNotification)
    }
}
```

برای اینکه فرگمنت اصلی برنامه شما بتواند از این پس به صورت هوشمند نوتیفیکیشن‌های پیش‌زمینه را رهگیری کند، باید زنجیره ارث‌بری آن را تغییر دهیم. با متصل کردن این فرگمنت به کلاس انتزاعی جدیدی که ساختید، تمامی منطقِ مربوط به ثبت و لغو ثبت پویا (که در متدهای onStart و onStop لایه زیرین تعریف کردیم) به طور خودکار به این صفحه تزریق خواهد شد.

کلاس فرگمنت خود را باز کنید و طبق الگو و کدهای زیر، کلاس سوپر را از Fragment معمولی به VisibleFragment تغییر دهید.

لیست ۲۷.۹ ساخت فرگمنت خود را قابل مشاهده (PhotoGalleryFragment.kt)

```
class PhotoGalleryFragment : ~~Fragment()~~ VisibleFragment() {
    ...
}
```

انتقال و دریافت داده‌ها با Broadcastهای سفارش داده شده

وقت آن است که بالاخره این نوزاد را به خانه بیاوریم. آخرین کار این است که اطمینان حاصل کنید که گیرنده ثبت شده پویا شما همیشه PollWorker.ACTION_SHOW_NOTIFICATION مربط به Broadcast را قبل از هر گیرنده دیگری دریافت می‌کند و Broadcast را تغییر می‌دهد تا نشان دهد که اعلان نباید ارسال شود.

در حال حاضر شما Broadcast خصوصی شخصی خود را ارسال می‌کنید، اما تاکنون فقط ارتباط یک طرفه دارید.

به این دلیل است که یک اینتنت Broadcast منظم از نظر مفهومی‌توسط همه به طور همزمان دریافت می‌شود. زیرا onReceive(…) در نخ اصلی فراخوانی می‌شود، گیرنده‌های ما در واقع به طور همزمان اجرا نمی‌شوند. با این حال، نمی‌توان به اجرای آنها به ترتیب خاصی اعتماد کرد یا پی برد که چه زمانی همه آنها اجرا را به پایان رسانده‌اند. در نتیجه، برقراری ارتباط گیرنده‌های Broadcast با یکدیگر یا برای فرستنده قصد دریافت اطلاعات از گیرنده‌ها دردسرساز است.

ما می‌توانیم ارتباط منظم قابل پیش بینی را با استفاده از اینتنت ordered پیاده‌سازی کنیم. Broadcastهای مرتب شده به دنباله‌ای از گیرنده‌های Broadcast اجازه می‌دهد تا اینتنت Broadcast را به ترتیب پردازش کنند.

در سمت گیرنده، این بیشتر شبیه یک Broadcast معمولی است. اما ما یک ابزار اضافی دریافت می‌کنیم: مجموعه‌ای از توابع که برای تغییر اینتنت در طول زنجیره استفاده می‌شود. در اینجا، می‌خواهیم اعلان را بازنویسی کنیم. این را می‌توان با استفاده از یک کد نتیجه عدد صحیح ساده با تنظیم resultCode روی Activity.RESULT_CANCELED ارتباط برقرار کرد.

VisibleFragment را تغییر دهید تا به فرستنده SHOW_NOTIFICATION بگویید که آیا اعلان باید ارسال شود یا خیر. این اطلاعات همچنین به سایر گیرنده‌های Broadcast در طول زنجیره ارسال می‌شود.

لیست ۲۷.۱۰ ارسال یک نتیجه ساده (VisibleFragment.kt)

```
abstract class VisibleFragment : Fragment() {
    private val onShowNotification = object
    : BroadcastReceiver() {
        override fun onReceive(context: Context, intent
        : Intent) {
            Log.i(TAG, "Foreground activity detected
            : intercepting ${intent.action}")
```

```
                resultCode = Activity.RESULT_CANCELED
                abortBroadcast()
            }
        }
        ...
}
```

زیرا تنها کاری که باید انجام دهیم نشان دادن بله یا خیر است، فقط به کد نتیجه نیاز داریم. اگر نیاز به انتقال داده‌های پیچیده‌تر داشتیم، می‌توانیم resultData را تنظیم یا setResultExtras(Bundle?) را فراخوانی کنیم. و اگر می‌خواهیم هر سه مقدار را تنظیم کنیم، می‌توانیم setResult(Int, String?, Bundle?) را فراخوانی کنیم. هر گیرنده بعدی قادر خواهد بود این مقادیر را ببیند یا حتی تغییر دهد.

برای اینکه این توابع کار مفیدی انجام دهند، Broadcast شما باید ordered شود. یک تابع جدید برای ارسال Ordered Broadcast در PollWorker بنویسید. این تابع یک فراخوانی اعلان را پکیج بندی می‌کند و آن را به عنوان Broadcast ارسال می‌کند. ()doWork را به روز کنید تا تابع جدید خود را فراخوانی و به نوبه خود، به جای ارسال مستقیم اعلان به NotificationManager، یک Ordered Broadcast ارسال کنید.

لیست ۲۷.۱۱ ارسال Broadcast سفارش داده شده (PollWorker.kt)

```
class PollWorker(
    private val context: Context,
    workerParams: WorkerParameters
) : CoroutineWorker(context, workerParams) {
    companion object {
        const val ACTION_SHOW_NOTIFICATION =
        "com.codeb.photogallery.SHOW_NOTIFICATION"
        const val PERM_PRIVATE =
        "com.codeb.photogallery.PRIVATE_SHOW_NOTIFICATION"
        const val REQUEST_CODE = "REQUEST_CODE"
        const val NOTIFICATION = "NOTIFICATION"
    }
    override suspend fun doWork(): Result {
        val query = QueryPreferences.getStoredQuery(context)
        val lastResultId = QueryPreferences
            .getStoredLastResultId(context)
        val unsplashFetcher = UnsplashFetcher()
        val photos: List<UnsplashPhoto> = try {
            if (query.isBlank()) {
                unsplashFetcher.fetchPhotos().first()
            } else {
                unsplashFetcher.searchPhotos(query).first()
            }
        } catch (e: Exception) {
```

```
            Log.e(TAG, "Background network fetch failed", e)
            return Result.retry()
        }
        if (photos.isEmpty()) {
            Log.i(TAG, "Server returned 0 photos.")
            return Result.success()
        }
        val currentResultId = photos.first().id
        if (currentResultId == lastResultId) {
            Log.i(TAG,
            "No new photos: $currentResultId")
        } else {
            Log.i(TAG,
            "New photos: $currentResultId, Old ID: $lastRe-
sultId")
            QueryPreferences.setStoredLastResultId(context
            , currentResultId)
            val intent = PhotoGalleryActivity.newIntent(
            context)
            val pendingIntent = PendingIntent.getActivity(
                context,
                0,
                intent,
                PendingIntent.FLAG_IMMUTABLE
            )
            val notification = NotificationCompat.Builder(
            context, NOTIFICATION_CHANNEL_ID)
            .setSmallIcon(android.R.drawable
                .ic_menu_report_image)
                .setContentTitle("New Photos!")
                .setContentText(
                "Someone uploaded new photos to Unsplash!")
                .setContentIntent(pendingIntent)
                .setAutoCancel(true)
                .build()
            showBackgroundNotification(0, notification)

        }
        return Result.success()
    }
    private fun showBackgroundNotification(
        requestCode: Int,
        notification: Notification
    ) {
        val intent = Intent(ACTION_SHOW_NOTIFICATION).apply {
            putExtra(REQUEST_CODE, requestCode)
            putExtra(NOTIFICATION, notification)
        }
```

```
        intent.setPackage(context.packageName)
        context.sendOrderedBroadcast(intent, PERM_PRIVATE)
        Log.i(TAG, "Broadcast sent!")
    }
    …
}
```

Context.sendOrderedBroadcast(Intent, String?) بسیار شبیه به ()sendBroadcast رفتار می‌کند، اما تضمین می‌کند که Broadcast شما یک به یک به هر گیرنده تحویل داده می‌شود. کد نتیجه در ابتدا هنگام ارسال این Broadcast سفارش داده شده روی Activity.RESULT_OK تنظیم می‌شود.

NotificationReceiver را به روز کنید تا اعلان را برای کاربر ارسال کند.

لیست ۲۷.۱۲ پیاده‌سازی گیرنده نتیجه خود را (NotificationReceiver.kt)

```
class NotificationReceiver : BroadcastReceiver() {
    override fun onReceive(context: Context, intent: Intent)
{
        Log.i(TAG,
        "Received broadcast intent:  ${intent.action}")
        if (resultCode == Activity.RESULT_CANCELED) {
            Log.i(TAG,
            "Broadcast cancelled by foreground fragment.")
            return
        }
        val requestCode = intent.getIntExtra(
            PollWorker.REQUEST_CODE, 0)
        val notification: Notification? =
            if (Build.VERSION.SDK_INT >=
                Build.VERSION_CODES.TIRAMISU)
            {
                intent.getParcelableExtra(
                PollWorker.NOTIFICATION
                    , Notification::class.java)
            } else {
                @Suppress("DEPRECATION")
                intent.getParcelableExtra(
                PollWorker.NOTIFICATION)
            }
        Log.i(TAG, "onReceive call started")
        if (notification != null) {
            val notificationManager =
                NotificationManagerCompat.from(context)
            if (androidx.core.content.ContextCompat
                    .checkSelfPermission(
                        context,
                        android.Manifest.permission
```

```
                        .POST_NOTIFICATIONS
                        ) ==
                    android.content.pm
                    .PackageManager.PERMISSION_GRANTED
                ) {
                    notificationManager.notify(requestCode
                    , notification)
                }
            }
        }
}
```

برای اطمینان از اینکه NotificationReceiver Broadcast را پس از گیرنده ثبت شده پویا شما دریافت می‌کند (بنابراین می‌تواند بررسی کند که‌ایا باید اعلان را به NotificationManager ارسال کند یا خیر)، باید اولویت پایینی را برای NotificationReceiver در مانیفست تنظیم کنید. به آن اولویت -۹۹۹ بدهید تا آخرین اجرا شود. این کمترین اولویت ممکن است که توسط کاربر تعریف شده است (-۱۰۰۰ و کمتر رزرو شده است).

لیست ۲۷.۱۳ اولویت بندی گیرنده اعلان (manifests/AndroidManifest.xml)

```
<receiver android:name=".NotificationReceiver"
    android:exported="false">
    <intent-filter
        android:priority="-999">
        <action
            android:name=
            "com.codeb.photogallery.SHOW_NOTIFICATION" />
    </intent-filter>
</receiver>
```

برای اینکه فرگمنت اصلی برنامه (PhotoGalleryFragment) بتواند از این زنجیره هوشمند استفاده کند، باید آن را تغییر دهیم.

کد فایل PhotoGalleryFragment.kt را باز و تغییرات زیر را روی آن اعمال کنید:

لیست ۲۷.۱۴ استفاده از زنجیره هوشمند (PhotoGalleryFragment.kt)

```
class PhotoGalleryFragment : VisibleFragment() {
    private val requestPermissionLauncher =
    registerForActivityResult(
        ActivityResultContracts.RequestPermission()
    ) { isGranted: Boolean ->
        if (isGranted) {
            Log.d(TAG,
            "Notification permission granted by user.")
            setupBackgroundWork()
```

```
            startPollingProcess()
        } else {
            Log.w(TAG,
            "Notification permission denied by user.")
        }
    }

    private fun checkNotificationPermission() {
        if (Build.VERSION.SDK_INT >=
            Build.VERSION_CODES.TIRAMISU) {
            val permission =
            Manifest.permission.POST_NOTIFICATIONS
            when {
                ContextCompat.checkSelfPermission
                (requireContext(), permission) ==
                PackageManager.PERMISSION_GRANTED -> {
                    ~~setupBackgroundWork()~~
                    startPollingProcess()
                }
                else -> {
                    requestPermissionLauncher.launch
                       (permission)
                }
            }
        } else {
            ~~setupBackgroundWork()~~
            startPollingProcess()
        }
    }
    private fun startPollingProcess() {
        setupBackgroundWork()
        QueryPreferences.setPolling(requireContext(), true)
        activity?.invalidateOptionsMenu()
    }
    …
    override fun onViewCreated(view: View
        , savedInstanceState: Bundle?) {
        …
        requireActivity().addMenuProvider(object
        : MenuProvider {
            override fun onCreateMenu(menu: Menu,
            menuInflater: MenuInflater) {
                …
            }
            override fun onMenuItemSelected(menuItem:
            MenuItem): Boolean {
                return when (menuItem.itemId) {
                    R.id.menu_item_clear -> {
```

```
                        …
                    }
                    R.id.menu_item_toggle_polling -> {
                        val isPolling = QueryPreferences
                        .isPolling(requireContext())
                        if (isPolling) {
                            WorkManager.getInstance
                            (requireContext()
                            .applicationContext)
                                .cancelUniqueWork
                                (POLL_WORK_NAME)
                            QueryPreferences.setPolling
                            (requireContext(), false)
                        } else {
                            setupBackgroundWork()
                            QueryPreferences.setPolling
                            (requireContext(), true)
                            checkNotificationPermission()
                        }
                        activity?.invalidateOptionsMenu()
                        true
                    }
                    else -> false
                }
            }
        }, viewLifecycleOwner, Lifecycle.State.RESUMED)
        if (QueryPreferences.isPolling(requireContext())) {
            checkNotificationPermission()
        }
    }
    …
}
```

PhotoGallery را اجرا کنید. باید ببینید که وقتی برنامه را در پیش زمینه دارید، اعلان‌ها دیگر ظاهر نمی‌شوند.

گیرنده‌ها و کارهای طولانی مدت

اگر بخواهیم یک اینتنت Broadcast یک کار طولانی مدت را از محدودیت‌های حلقه اجرای اصلی شروع کند، چه باید کرد؟ شما دو گزینه دارید.

اولین مورد این است که به جای آن کار را در یک سرویس قرار دهیم و سرویس را در پنجره کوچک فرصت گیرنده Broadcast خود راه اندازی کنید. یک سرویس می‌تواند چندین درخواست را در صف قرار دهد و آنها را به ترتیب اجرا کند یا درخواست‌ها را به دلخواه خود مدیریت کند. این روشی است که ما توصیه می‌کنیم. یک سرویس پنجره بسیار طولانی تری دارد که می‌تواند از آن برای انجام کار استفاده کند، اما اگر برای مدت طولانی اجرا شود، ممکن است همچنان متوقف شود. (این آستانه بسته به نسخه سیستم عامل و دستگاه متفاوت است، اما به طور کلی در دستگاه‌های جدیدتر چند دقیقه است.) همچنین می‌توانیم انتخاب کنیم که سرویسی را در پیش زمینه اجرا کنیم تا تمام محدودیت‌های مربوط به مدت زمان اجرای کارمان را حذف کنیم، که برای کارهایی مانند پشتیبان گیری از عکس‌ها، Broadcast موسیقی یا پیمایش گام به گام‌ایده آل است.

گزینه دوم استفاده از عملکرد BroadcastReceiver.goAsync() است. این تابع یک شی BroadcastReceiver.PendingResult را برمی‌گرداند که می‌تواند برای ارائه نتیجه در زمان بعدی استفاده شود. بنابراین می‌توانیم آن PendingResult را به یک AsyncTask بدهید تا کارهای طولانی تری را انجام دهد و سپس با فراخوانی توابع در PendingResult به Broadcast پاسخ دهید.

استفاده از تابع goAsync() یک نقطه ضعف دارد: انعطاف پذیری کمتری دارد. شما هنوز باید Broadcast را در عرض ۱۰ ثانیه یا بیشتر سرویس دهید و گزینه‌های معماری کمتری نسبت به یک سرویس دارید. البته، goAsync() یک مزیت بزرگ دارد: می‌توانید نتایج را برای Broadcastهای مرتب شده با آن تنظیم کنید. اگر واقعا به آن نیاز دارید، هیچ چیز دیگری انجام نمی‌دهد. فقط مطمئن شوید که خیلی طول نمی‌کشد.

برای کنجکاوترین‌ها: رویدادهای محلی

اینتنت‌های Broadcast به شما امکان می‌دهد اطلاعات را در سراسر سیستم به صورت سراسری منتشر کنید. اگر بخواهیم وقوع یک رویداد را فقط در فرآیند برنامه خود Broadcast کنیم، چه؟ استفاده از event bus یک جایگزین عالی است.

یک event bus بر اساس‌ایده داشتن یک گذرگاه مشترک یا جریانی از داده‌ها عمل می‌کند که اجزای برنامه ما می‌توانند در آن مشترک شوند. هنگامی که یک رویداد در گذرگاه ارسال می‌شود، مولفه‌های مشترک فعال می‌شوند و کد callback آنها اجرا می‌شود.

EventBus توسط greenrobot یک کتابخانه event bus شخص ثالث است که ما در برخی از برنامه‌های اندرویدی خود استفاده کرده‌ایم. همچنین می‌توانیم از Otto Square، که یکی دیگر از پیاده‌سازی EventBus است، یا RxJava Subjects and Observable استفاده کنیم.

اندروید راهی برای ارسال اینتنت‌های Broadcast محلی به نام LocalBroadcastManager ارائه می‌دهد. اما متوجه شدیم که کتابخانه‌های شخص ثالث ذکر شده در اینجا یک API انعطاف پذیرتر و آسان‌تر برای Broadcast رویدادهای محلی ارائه می‌دهند.

برای کنجکاوترین‌ها: محدودیت‌های گیرنده‌های Broadcast

همانطور که قبلا در این فصل مشاهده کردید، محدودیت‌هایی برای گیرنده‌های Broadcast اعلام شده در مانیفست اندروید شما وجود دارد که ممکن است باعث عدم فراخوانی گیرنده شود. این رفتار در مورد گیرنده‌هایی که به صورت پویا با استفاده از registerReceiver(…) ثبت می‌کنیم اعمال نمی‌شود و فقط زمانی قابل اجرا است که برنامه ما روی Android Oreo (سطح API 26) و نسخه‌های جدیدتر اندروید اجرا شود.

محدودیت‌های گیرنده‌های Broadcast مستقل با اینتنت بهبود حیات باتری و عملکرد در دستگاه‌های کاربران معرفی شد. اگر یک گیرنده Broadcast در مانیفست شما ثبت شده است و برنامه شما اجرا نمی‌شود، سیستم باید هر زمان که نیاز به تحویل Broadcast به این گیرنده داشته باشد، فرآیند جدیدی را بچرخاند. اگر فقط یک یا دو برنامه نیاز به شروع فرآیند خود داشته باشند، این عمل چرخش فرآیندها نسبتا غیرقابل توجه است، اما اگر بسیاری از برنامه‌ها بخواهند Broadcast یکسانی را مدیریت کنند، عملکرد به سرعت بدتر می‌شود.

یکی از نمونه‌هایی که این امر می‌تواند به طور مستقیم به تجربه کاربر آسیب برساند این است که کاربر برنامه‌هایی داشته باشد که به طور خودکار از عکس‌های جدید گرفته شده توسط دوربین نسخه پشتیبان تهیه می‌کنند. این باعث می‌شود که هر زمان که کاربر دکمه شاتر را در برنامه دوربین فشار می‌دهد، چندین فرآیند در پس زمینه شروع شود، که باعث می‌شود دوربین به کاربر پاسخ ندهد.

برای کاهش این نگرانی عملکرد، نسخه‌های جدید اندروید که با Android Oreo شروع می‌شوند، دیگر Broadcast ضمنی را به گیرنده‌های اعلام شده در مانیفست شما ارائه نمی‌دهند – به استثنای برخی موارد. (Broadcast صریح تحت تأثیر این محدودیت‌ها قرار نمی‌گیرند، اما به یاد داشته باشید که این نوع Broadcastها فقط به یک گیرنده ارسال می‌شوند و به ندرت مورد استفاده قرار می‌گیرند.)

تعدادی از Broadcastهای سیستم از این قانون مستثنی هستند. گیرنده‌های Broadcast ثبت شده در مانیفست برای Broadcastهایی از جمله BOOT_COMPLETE، TIMEZONE_CHANGED و NEW_OUTGOING_CALL همچنان آن Broadcastها را دریافت خواهند کرد. این Broadcastها و سایر Broadcastها مستثنی هستند زیرا یا به ندرت ارسال می‌شوند یا راه جایگزینی برای انجام عملکرد مورد نیاز برنامه‌ها ندارند. می‌توانید لیست کامل معافیت‌ها را در مستندات broadcast-exceptions بیابید.

همانطور که هنگام کار بر روی این فصل دیدید، Broadcastهایی که با مجوز در سطح امضا ارسال می‌شوند نیز از این محدودیت معاف هستند. این به شما امکان می‌دهد از گیرنده‌های Broadcast مستقل که برای برنامه شما خصوصی هستند و هر برنامه دیگری که با استفاده از همان امضا می‌سازید استفاده کنید. از آنجایی که فقط برنامه‌های یک توسعه دهنده مجوز ارسال این Broadcastها را دارند، بعید است که مشکلات عملکردی در سایر قسمت‌های سیستم ناشی از این نوع Broadcastها وجود داشته باشد.

بیشتر بدانیم: تشخیص Visibility فرگمنت خود

هنگامی که در مورد اجرای PhotoGallery خود فکر می‌کنید، ممکن است متوجه شوید که از مکانیسم Broadcast سراسری برای Broadcast اینتنت SHOW_NOTIFICATION استفاده کرده‌ایم. با این حال، با استفاده از مجوزهای سفارشی، دریافت آن Broadcast را در موارد محلی فرآیند برنامه خود قفل کردیم. ممکن است متوجه شوید که بپرسید: "چرا من از یک مکانیسم سراسری استفاده می‌کنم اگر فقط با چیزهای موجود در برنامه خودم ارتباط برقرار می‌کنم؟ چرا به جای آن یک مکانیسم محلی نباشد؟"

به این دلیل است که ما به طور خاص سعی می‌کردیم مشکل دانستن اینکه‌ایا PhotoGalleryFragment قابل مشاهده است یا خیر را حل کنید. ترکیبی از Broadcastهای سفارش داده شده، گیرنده‌های مستقل و گیرنده‌های ثبت شده پویا که پیاده‌سازی کرده‌ایم، کار را انجام می‌دهد. راه ساده تری برای انجام این کار در اندروید وجود ندارد.

به طور خاص، LocalBroadcastManager به دو دلیل اصلی برای Broadcast اعلان PhotoGallery و تشخیص فرگمنت‌ها قابل مشاهده کار نمی‌کند.

اول، LocalBroadcastManager از Broadcastهای مرتب پشتیبانی نمی‌کند (اگرچه یک راه مسدود کننده برای Broadcast ارائه می‌دهد، یعنی (sendBroadcastSync(Intent). این برای PhotoGallery کار نخواهد کرد زیرا باید NotificationReceiver را مجبور کنید آخرین بار در زنجیره اجرا شود.

دوم، (sendBroadcastSync(Intent از ارسال و دریافت Broadcast در رشته‌های جداگانه پشتیبانی نمی‌کند. در PhotoGallery، باید Broadcast را از یک رشته پس زمینه (در (PollWorker.doWork) ارسال کنید و اینتنت موضوع اصلی را دریافت کنید (توسط گیرنده پویا که توسط PhotoGalleryFragment در موضوع اصلی در onStart (...) ثبت شده است.).

از زمان نگارش این مقاله، معناشناسی تحویل موضوع LocalBroadcastManager به خوبی مستند نشده یا در تجربه ما، شهودی نیست. به عنوان مثال، اگر (…)sendBroadcastSync را فراخوانی کنیداز یک موضوع پس زمینه، تمام Broadcastهای معلق در آن موضوع پس زمینه حذف می‌شوند، حتی اگر از موضوع اصلی ارسال شده باشند.

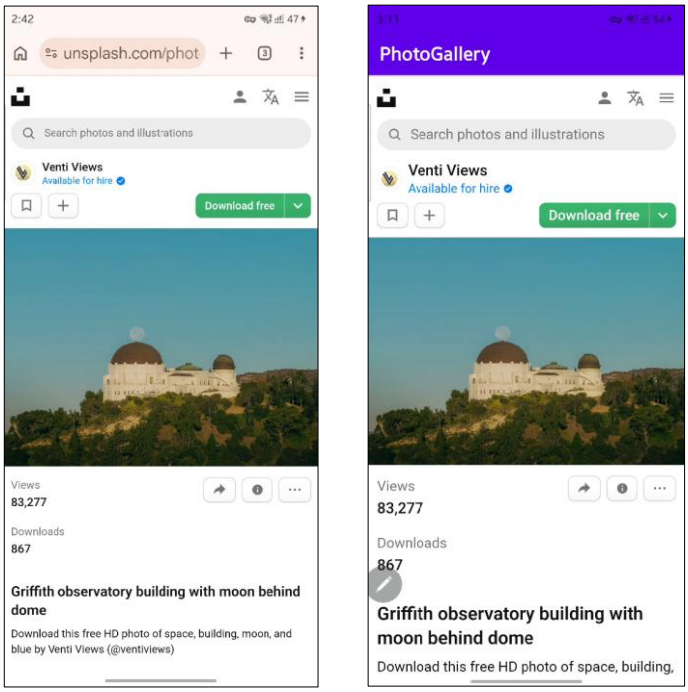
این بدان معنا نیست که LocalBroadcastManager مفید نیست. این به سادگی ابزار مناسبی برای مشکلاتی نیست که در این فصل حل کرده‌اید.

۲۸

مرور وب و WebView

هر عکسی که از Unsplash دریافت می‌کنیم یک صفحه مرتبط با آن دارد. در این فصل قصد دارید PhotoGallery را به روز کنیم تا کاربران بتوانند عکسی را فشار دهند تا صفحه Unsplash آن را ببینند. ما دو روش مختلف برای ادغام محتوای وب در برنامه‌های خود را یاد خواهیم گرفت که در شکل ۲۸.۱ نشان داده شده است. اولی با برنامه مرورگر دستگاه (سمت چپ) کار می‌کند و دومی‌از WebView برای نمایش محتوای وب در PhotoGallery (راست) استفاده می‌کند.

شکل ۲۸.۱ محتوای وب: دو رویکرد متفاوت



آخرین بیت داده Unsplash

صرف نظر از چگونگی باز کردن صفحه عکس Unsplash، ابتدا باید URL آن را دریافت کنیم. کد JSON نمونه برای هر عکس همانند لیست زیر است. بخش links حاوی لینک‌های مربوط به تعامل با سایت است. فیلد html دقیقاً همان آدرس صفحه واقعی عکس در سایت Unsplash است که کاربران در مرورگر می‌بینند.

```
{
  "id": "UKtuLKxjRfk",
  "description": "A beautiful sunset over the mountains",
  "urls": {
    "raw": "https://images.unsplash.com/photo-...",
    "full": "https://images.unsplash.com/photo-...",
    "regular": "https://images.unsplash.com/photo-...",
    "small": "https://images.unsplash.com/photo-...",
    "thumb": "https://images.unsplash.com/photo-..."
  },
  "links": {
    "self": "https://api.unsplash.com/photos/UKtuLKxjRfk",
    "html": "https://unsplash.com/photos/
        a-beautiful-sunset-UKtuLKxjRfk",
    "download": "https://unsplash.com/
        photos/UKtuLKxjRfk/download",
    "download_location": "https://api.unsplash.com/
         hotos/UKtuLKxjRfk/download"
  },
  "user": {
    ...
  }
}
```

(به یاد داشته باشید که url_s آدرس نسخه کوچک عکس است، نه عکس دراندازه کامل.)

پس باید در اولین مرحله مقدار تگ html را از JSON بیرون بکشیم. ابتدا باید به اندروید بگوییم که در JSON یک بخش به نام links وجود دارد که داخلش یک html هست. فایل UnsplashPhoto را برای عملیاتی کردن این روش به روز کنید و داده html را در مدل خود قرار دهید.

لیست ۲۸.۱ افزودن کد برای صفحه عکس (UnsplashPhoto.kt)

```
data class UnsplashPhoto(
    val id: String,
    @SerializedName("description") val title: String?,
    @SerializedName("urls") val url: Urls,
    val links: PhotoLinks
)
```

```
data class Urls(
    val raw: String,
    val full: String,
    val regular: String,
    val small: String,
    val thumb: String
)
```

```
data class PhotoLinks(
    val html: String
)
```

حالا هر جا که به URL عکس نیاز داشتیم از مقدار UnsplashPhoto.links.html استفاده خواهیم کرد.

راه آسان: اینتنت‌های ضمنی

ابتدا با استفاده از دوست قدیمی‌خود با اینتنت ضمنی به این URL مراجعه خواهیم کرد. این اینتنت مرورگر را با URL عکس راه اندازی می‌کند.

برای پیاده‌سازی این قابلیت به صورت کاملاً حرفه‌ای و تمیز، فایل PhotoGalleryFragment.kt را باز و تغییرات زیر را اعمال کنید.

لیست ۲۸.۲ شلیک یک اینتنت ضمنی هنگام فشار دادن یک آیتم (PhotoGalleryFragment.kt)

```
class PhotoGalleryFragment : VisibleFragment() {
    …
    private val photoGalleryViewModel:
        PhotoGalleryViewModel by viewModels()
    private val photoAdapter = PhotoAdapter()
    private val photoAdapter = PhotoAdapter { photoPageUri ->
        val intent = Intent(Intent.ACTION_VIEW, photoPageUri)
        startActivity(intent)
    }
    …
    private class PhotoHolder(
        private val imageView: ImageView,
        private val onItemClicked: (Uri) -> Unit
    ) : RecyclerView.ViewHolder(imageView) {
        fun bind(photo: UnsplashPhoto) {
            imageView.load(photo.url.small) {
                crossfade(true)
                placeholder
                (android.R.drawable.progress_horizontal)
                error(android.R.drawable.stat_notify_error)
                memoryCachePolicy(CachePolicy.ENABLED)
                diskCachePolicy(CachePolicy.ENABLED)
            }
            imageview.setOnClickListener {
                onItemClicked((photo.links.html).toUri())
            }
        }
    }
private class PhotoAdapter(
    private val onItemClicked: (Uri) -> Unit
) : ListAdapter<UnsplashPhoto,
PhotoHolder>(PhotoDiffCallback()) {
    override fun onCreateViewHolder(parent: ViewGroup
    , viewType: Int): PhotoHolder {
        val imageView = ImageView(parent.context).apply {
            layoutParams = ViewGroup.LayoutParams(
```

```
                    ViewGroup.LayoutParams.MATCH_PARENT,
                    300
                )
                scaleType = ImageView.ScaleType.CENTER_CROP
            }
            return PhotoHolder(imageView, onItemClicked)
            }
            …
    }
    …
}
```

حالا که کدها را پیاده کردیم، بیایید منطق این سه مرحله ویرایش را به زبان ساده و فنی مرور کنیم: مورد اول اصلاح تعریف PhotoAdapter (در فرگمنت) بود. آداپتور را با یک بلاک کد { … } ساختیم. اینجا به آداپتور «برنامه کاری» دادیم؛ یعنی به جای اینکه آداپتور بداند چطور مرورگر باز کند، فرگمنت دستورِ باز کردن اینتنت را به آداپتور تزریق کرد.

مورد دوم اصلاح کلاس PhotoAdapter (در آداپتور) است. پارامتر onItemClicked را به سازنده (Constructor) اضافه کردیم. آداپتور حالا مثل یک واسطه عمل می‌کند؛ او این «دستورِ کلیک» را از فرگمنت می‌گیرد و بدون اینکه خودش اجرایش کند، به تک‌تکِ خانه‌های لیست (PhotoHolder) تحویل می‌دهد.

تغییر آخر اصلاح کلاس PhotoHolder بود. در متد bind برای تصویر setOnClickListener گذاشتیم. اینجا نقطه شروع است. وقتی کاربر روی عکس می‌زند، هولدر متد onItemClicked را با آدرس وب همان عکس (از فیلد links.html) صدا می‌زند. پیام از اینجا به آداپتور و در نهایت به فرگمنت می‌رسد تا اجرا شود.

خودشه. گالری عکس را راه اندازی کنید و یک عکس را فشار دهید. برنامه مرورگر شما باید ظاهر شود و صفحه عکس موردی را که فشار داده‌اید بارگذاری کند (مشابه تصویر سمت چپ در شکل ۲۹.۱).

راه سخت تر: WebView

استفاده از یک اینتنت ضمنی برای نمایش صفحه عکس آسان و موثر است. اما اگر نمی‌خواهید برنامه شما مرورگر را باز کند چه؟

اغلب، می‌خواهیم به جای رفتن به مرورگر، محتوای وب را در اکتیویتی‌های خود نمایش دهیم. ممکن است بخواهیم HTML را که خودمان تولید می‌کنیم نمایش دهیم، یا ممکن است بخواهیم مرورگر را به نحوی قفل کنیم. برای برنامه‌هایی که شامل راهنمایی هستند، پیاده‌سازی آن به صورت یک صفحه وب برای به روز رسانی آسان است. باز کردن یک مرورگر وب در یک صفحه وب راهنمایی حرفه‌ای به نظر نمی‌رسد و مانع از سفارشی کردن رفتار یا ادغام آن صفحه وب در رابط کاربری خود می‌شود.

هنگامی که می‌خواهیم محتوای وب را در رابط کاربری خود ارائه دهیم، از کلاس WebView استفاده می‌کنیم. ما در اینجا این را راه "سخت تر" می‌نامیم، اما بسیار آسان است. (هر چیزی در مقایسه با استفاده از اینتنت‌های ضمنی سخت است.)

اولین قدم ایجاد یک اکتیویتی و فرگمنت جدید برای نمایش WebView است. طبق معمول، با تعریف یک فایل لیوت به نام res/layout/fragment_photo_page.xml شروع کنید. ConstraintLayout را به لیوت ریشه تبدیل و شناسه آن را main قرار دهید. در ویوی Design، یک WebView را به عنوان فرزند به روی ConstraintLayout بکشید. (WebView را در بخش Widgets خواهید یافت.)

پس از اضافه شدن WebView، یک محدودیت برای هر طرف به والد آن اضافه کنید. این محدودیت‌ها به شکل زیر هستند:

- از بالای WebView تا بالای والد آن
- از پایین WebView به پایین والد آن
- از سمت چپ WebView به سمت چپ والد آن
- از سمت راست WebView به سمت راست والد آن

در نهایت ارتفاع و عرض را به Match Constraint تغییر دهید و تمام حاشیه‌ها را به ۰ تغییر دهید. و به WebView خود یک شناسه بدهید: web_view.

ممکن است فکر کنید، "که ConstraintLayout مفید نیست." به‌اندازه کافی درست است – در حال حاضر. بعدا در فصل آن را با "chrome" اضافی پر خواهید کرد.

در مرحله بعد، اصول اولیه فرگمنت خود را تنظیم کنید. PhotoPageFragment را به عنوان زیر کلاس VisibleFragment ایجاد کنید. شما باید فایل لیوت خود را بارگذاری کنید، WebView خود را از آن استخراج و URL به عنوان آرگومان فرگمنت ارسال کنید.

لیست ۲۸.۳ راه اندازی فرگمنت مرورگر وب (PhotoPageFragment.kt)

```
package com.codeb.photogallery
import android.net.Uri
import android.os.Bundle
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import android.webkit.WebView
import androidx.core.os.BundleCompat
private const val ARG_URI = "photo_page_url"
class PhotoPageFragment : VisibleFragment() {
    private lateinit var uri: Uri
    private lateinit var webView: WebView
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        uri = arguments?.let {
            BundleCompat.getParcelable(
            it, ARG_URI, Uri::class.java)
        } ?: Uri.EMPTY
    }
    override fun onCreateView(
        inflater: LayoutInflater,
        container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View? {
        val view = inflater.inflate(
        R.layout.fragment_photo_page, container, false)
        webView = view.findViewById(R.id.web_view)
        return view
    }
    companion object {
        fun newInstance(uri: Uri): PhotoPageFragment {
            return PhotoPageFragment().apply {
                arguments = Bundle().apply {
                    putParcelable(ARG_URI, uri)
                }
            }
        }
    }
}
```

در حال حاضر اسکلت اصلی را طراحی کردیم. در یک لحظه همه کارها را مرتب خواهیم کرد. ابتدا PhotoPageActivity برای میزبانی Fragment جدید ایجاد کنید.

لیست ۲۸.۴ ایجاد یک اکتیویتی وب (PhotoPageActivity.kt)

```
package com.codeb.photogallery
import android.content.Context
import android.content.Intent
import android.net.Uri
import android.os.Bundle
import androidx.activity.enableEdgeToEdge
import androidx.appcompat.app.AppCompatActivity
import androidx.core.view.ViewCompat
import androidx.core.view.WindowInsetsCompat
class PhotoPageActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContentView(R.layout.activity_photo_page)
        ViewCompat.setOnApplyWindowInsetsListener(
        findViewById(R.id.fragment_container))
        { v, insets ->
            val systemBars = insets.getInsets(
            WindowInsetsCompat.Type.systemBars())
            v.setPadding(systemBars.left, systemBars.top,
            systemBars.right, systemBars.bottom)
            insets
        }
        val fm = supportFragmentManager
        val currentFragment = fm.findFragmentById
        (R.id.fragment_container)
        if (currentFragment == null) {
            val fragment = PhotoPageFragment.newInstance(
            intent.data ?: Uri.EMPTY)
            fm.beginTransaction()
                .add(R.id.fragment_container, fragment)
                .commit()
        }
    }
    companion object {
        fun newIntent(context: Context, photoPageUri: Uri)
        : Intent {
            return Intent(context
            , PhotoPageActivity::class.java).apply {
                data = photoPageUri
            }
        }
```

```
    }
}
```

فایل لیوت res/layout/activity_photo_page.xml گم شده را ایجاد و یک FrameLayout با شناسه fragment_container اضافه کنید.

لیست ۲۸.۵ افزودن لیوت اکتیویتی (res/layout/activity_photo_page.xml)

```
<?xml version="1.0" encoding="utf-8"?>
<FrameLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    android:id="@+id/fragment_container"
    android:layout_width="match_parent"
    android:layout_height="match_parent"/>
```

اکنون کد خود را در PhotoGalleryFragment تغییر دهید تا به جای اینتنت ضمنی، اکتیویتی جدید خود را راه اندازی کند.

لیست ۲۸.۶ سوئیچینگ برای راه اندازی اکتیویتی شما (PhotoGalleryFragment.kt)

```
private val photoAdapter = PhotoAdapter { photoPageUri ->
    val intent = Intent(Intent.ACTION_VIEW, photoPageUri)
    val intent = PhotoPageActivity.newIntent(
        requireContext(), photoPageUri)
    startActivity(intent)
}
```

PhotoGallery را اجرا و یک عکس را لمس نمایید. باید یک اکتیویتی خالی جدید را ببینید. البته باید اکتیویتی جدید را همانند زیر در Manifest اعلام نمایید.

```
<activity
    android:name=".PhotoPageActivity"
    android:exported="true">
</activity>
```

خوب، حالا باید سه کار انجام دهیم تا WebView با موفقیت صفحه عکس Unsplash را نمایش دهد. اولین مورد ساده است – باید به آن بگوییم که چه URL را بارگیری کند.

دومین کاری که باید انجام دهیم این است که جاوا اسکریپت را فعال کنیم. به طور پیش‌فرض، جاوا اسکریپت غیرفعال است. همیشه نیازی به فعال کردن آن نداریم، اما برای Unsplash، این کار را باید انجام داد. (اگر اندروید لینت را اجرا می‌کنید، برای انجام این کار به شما هشدار می‌دهد. نگران حملات اسکریپت نویسی بین سایتی است. شما می‌توانید این هشدار Lint را با حاشیه‌نویسی @SuppressLint("SetJavaScriptEnabled") سرکوب کنید.)

حالا که uri با موفقیت دریافت شده است، نوبت به تنظیمات نهایی WebView می‌رسد تا محتوا را نمایش دهد. در فایل PhotoPageFragment.kt، تغییرات را اعمال کنید.

لیست ۲۸.۷ بارگذاری URL در WebView (PhotoPageFragment.kt)

```
class PhotoPageFragment : VisibleFragment() {
    private lateinit var uri: Uri
    private lateinit var webView: WebView
    override fun onCreate(savedInstanceState: Bundle?) {
        ...
    }
    companion object {
        ...
    }
    override fun onViewCreated(view: View
        , savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState)
        webView.apply {
            settings.javaScriptEnabled = true
            webViewClient = WebViewClient()
            loadUrl(uri.toString())
        }
    }
}
```

بیایید کدی که در متد onViewCreated برای WebView نوشتیم را کالبدشکافی کنیم. هر خط یک وظیفه حیاتی دارد. در کاتلین، apply مثل این است که به اندروید بگویید: «روی این شیء (یعنی webView)، این لیست از کارها را به ترتیب انجام بده».

settings.javaScriptEnabled = true اجازه به اجرای کدهای جاوا اسکریپت در صفحه وب را می دهد. سایت Unsplash یک وب اپلیکیشن مدرن است. این سایت برای نمایش انیمیشن‌ها، لود کردن عکس‌ها به صورت هوشمند و حتی چیدمان صفحه از جاوا اسکریپت استفاده می‌کند. اگر این خط را ننویسید، احتمالاً یک صفحه سفید یا شکسته می‌بینید.

webViewClient = WebViewClient() یعنی تعیین «ناظر» برای مرورگر داخلی. این خط مهم‌ترین بخش است. اگر این کار را نکنید، به محض اینکه دستور loadUrl صادر شود، سیستم‌عامل اندروید می‌گوید: «آهان! یک لینک وب! بگذارید ببینم چه مرورگرهایی در گوشی نصب است؟» و سپس کروم یا مرورگر سامسونگ را باز می‌کند و کاربر را از برنامه شما بیرون می‌برد. با نوشتن این خط، شما یک «ناظر پیش‌فرض» می‌گذارید که به سیستم می‌گوید: «هر اتفاقی افتاد، در همین کادر بمان و اجازه خروج نده».

loadUrl(uri.toString()) استارت عملیات دریافت اطلاعات از اینترنت است. متغیر uri که در متد onCreate از ورودی‌ها گرفته بودیم، در اینجا به رشته (String) تبدیل شده و به موتور مرورگر داده می‌شود تا عملیات دانلود و نمایش صفحه وب را آغاز کند.

نتیجه نهایی این می شود که وقتی کاربر روی عکس کلیک می‌کند، در کمتر از یک ثانیه اکتیویتی جدید باز می‌شود و مرورگر داخلی برنامه شما شروع به لود کردن صفحه Unsplash می‌کند، بدون اینکه کاربر احساس کند از برنامه خارج شده است.

PhotoGallery را اجرا کنید، یک مورد را فشار دهید و باید صفحه عکس مورد را در WebView مشاهده کنید (مانند تصویر سمت راست در شکل ۲۸.۱).

استفاده از WebChromeClient برای زیباسازی

با افزودن یک نوار پیشرفت و به روز رسانی زیرنویس نوار ابزار با عنوان صفحه بارگذاری شده، آن را کمی‌زیباتر می‌کنیم. این تزئینات و رابط کاربری خارج از WebView به عنوان chrome نامیده می‌شوند (نباید با مرورگر وب Chrome اشتباه گرفته شود). یک بار دیگر fragment_photo_page.xml را باز کنید.

در ویوی Design یک نوار پیشرفت را به عنوان فرزند دوم برای ConstraintLayout خود اضافه کنید. از نسخه ProgressBar افقی استفاده کنید. محدودیت بالای WebView را حذف و سپس ارتفاع آن را روی Fixed تنظیم کنید تا بتوانیم به راحتی با هندل‌های محدودیت آن کار کنید. با انجام این کار، محدودیت‌های زیر را ایجاد کنید:

- از ProgressBar به بالا، راست و چپ والد آن
- از بالای WebView به پایین نوار پیشرفت

با انجام این کار، ارتفاع WebView را به Match Constraint تغییر دهید، ارتفاع نوار پیشرفت را به wrap_content و عرض نوار پیشرفت را به Match Constraint تغییر دهید.

در نهایت، نوار پیشرفت را انتخاب کنید و توجه خود را به کادر ویژگی‌ها معطوف نمایید. visibility را به gone تغییر و tool visibility را به visible تغییر دهید. نام شناسه آن را به progress_bar تغییر دهید.

برای اتصال ProgressBar به WebView، باید از کلاسی به نام WebChromeClient استفاده کنید. برخلاف WebViewClient که مسئول مدیریت لینک‌هاست، WebChromeClient مسئول اتفاقاتی است که به ظاهر و پردازش مرورگر مربوط می‌شود (مثل میزان پیشرفت لود صفحه). تغییرات زیر را در PhotoPageFragment اعمال کنید.

لیست ۲۸.۸ با استفاده از WebChromeClient (PhotoPageFragment.kt)

```
class PhotoPageFragment : VisibleFragment() {
    private lateinit var progressBar: ProgressBar
    private lateinit var uri: Uri
    private lateinit var webView: WebView
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        uri = arguments?.let {
            BundleCompat.getParcelable
            (it, ARG_URI, Uri::class.java)
        } ?: Uri.EMPTY
    }
```

```
    override fun onCreateView(
        inflater: LayoutInflater,
        container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View? {
        val view = inflater.inflate
        (R.layout.fragment_photo_page, container, false)
        progressBar = view.findViewById(R.id.progress_bar)
        progressBar.max = 100
        webView = view.findViewById(R.id.web_view)
        return view
    }
    ...
    override fun onViewCreated(view: View
    , savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState)
        webView.apply {
            settings.javaScriptEnabled = true
            webViewClient = WebViewClient()
            loadUrl(uri.toString())
        }
        webView.webChromeClient = object
        : WebChromeClient() {
            override fun onProgressChanged
            (view: WebView?, newProgress: Int) {
                if (newProgress == 100) {
                    progressBar.visibility = View.GONE
                } else {
                    progressBar.visibility = View.VISIBLE
                    progressBar.progress = newProgress
                }
            }
            override fun onReceivedTitle
            (view: WebView?, title: String?) {
                val parentActivity = activity as?
                    AppCompatActivity
                parentActivity?.supportActionBar?.subtitle =
                    title
            }
        }
    }
}
```

با اضافه کردن این کدها، ما عملاً یک مرورگر داخلی پیشرفته و هوشمند برای برنامه‌مان ساختیم. بیایید تغییرات نهایی را بررسی کنیم. ما با دستور webViewClient = WebViewClient()، افسارِ مرورگر را به دست برنامه خودمان دادیم. حالا تمام لینک‌ها و جابه‌جایی‌ها در دلِ اپلیکیشن خودمان اتفاق می‌افتد و

کاربر برای دیدن جزئیات عکس، به بیرون از برنامه (مثل کروم) پرتاب نمی‌شود. این کار «نرخ ماندگاری» کاربر در برنامه شما را بالا می‌برد.

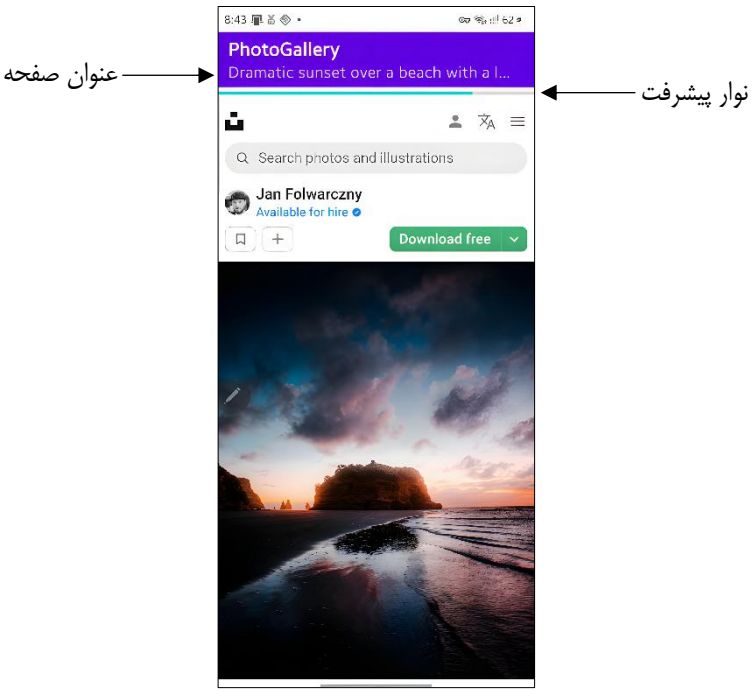
با استفاده از متد onProgressChanged در کلاس WebChromeClient، ارتباطی بین دنیای وب و دنیای اندروید برقرار کردیم. نوار پیشرفت (ProgressBar) دقیقاً با سرعت لود شدن سایت جلو می‌رود. این کار باعث می‌شود کاربر متوجه شود که برنامه در حال تلاش برای دریافت دیتای سنگین است و احساس «کُندی» یا «هنگ کردن» به او دست ندهد. وقتی لود تمام می‌شود (عدد ۱۰۰)، نوار به صورت هوشمند مخفی می‌شود.

پویایی نوار ابزار (onReceivedTitle) تیرِ آخر برای حرفه‌ای شدن ظاهر برنامه بود. به محض اینکه مرورگر متوجه شود نام عکاس یا عنوان عکس در سایت Unsplash چیست، آن را در Subtitle (زیرنویس) اکتیویتی نمایش می‌دهد.

استفاده از (activity as AppCompatActivity) به ما اجازه داد تا از داخلِ یک فرگمنت، به اجزای گرافیکیِ اکتیویتیِ مادر دسترسی پیدا کرده و محتوای آن را تغییر دهیم.

PhotoGallery را اجرا کنید تا تغییرات خود را آزمایش نمایید. باید شبیه شکل ۲۸.۳ باشد.

تصویر ۲۸.۳ ویوی وب فانتزی



وقتی عکسی را فشار می‌دهید، PhotoPageActivity ظاهر می‌شود. یک نوار پیشرفت با بارگذاری صفحه نمایش داده و یک زیرنویس منعکس کننده عنوان در نوار ابزار ظاهر می‌کند. پس از بارگیری صفحه، نوار پیشرفت ناپدید می‌شود.

چرخش مناسب با WebView

مطمئن شوید چرخش خودکار در شبیه‌ساز یا دستگاهتان فعال است. سپس صفحه نمایش را بچرخانید. برنامه به درستی کار می‌کند، اما متوجه خواهید شد که WebView باید صفحه وب را بار دیگر به طور کامل بارگذاری کند.

WebView داده‌های زیادی دارد که می‌تواند همه آن‌ها را در onSaveInstanceState(…) ذخیره کند. وچون اینکار انجام نشده، هر بار که در چرخش دوباره ایجاد می‌شود، باید از ابتدا شروع شود.

برای برخی از کلاس‌ها مانند این (VideoView یکی دیگر از کلاس‌ها است)، مستندات اندروید توصیه می‌کند که تغییرات پیکربندی خاصی مانند چرخش را خودتان انجام دهید. این بدان معنی است که به جای اینکه اکتیویتی در چرخش از بین برود، ویوهای آن فقط متناسب بااندازه صفحه نمایش جدید جابجا می‌شوند. در نتیجه، WebView مجبور نیست تمام داده‌های خود را دوباره بارگذاری شده کند.

برای اینکه به PhotoPageActivity بگویید که تغییرات پیکربندی را انجام خواهید داد، ترفند زیر را در انجام دهید.

لیست ۲۸.۹ مدیریت تغییرات پیکربندی (manifests/AndroidManifest.xml)

```
<manifest ...>
    ...
        <activity
            android:name=".PhotoPageActivity"
            android:configChanges=
                "keyboardHidden|orientation|screenSize"
            ...
        </activity>
    ...
</manifest>
```

این ویژگی می‌گوید که اگر پیکربندی به دلیل باز یا بسته شدن صفحه کلید، به دلیل تغییر جهت یا به دلیل تغییراندازه صفحه نمایش (که هنگام جابجایی بین عمودی و افقی پس از اندروید ۳.۲ نیز اتفاق می‌افتد) تغییر کند، تغییر داخل اکتیویتی مدیریت خواهد شد و نباید از بین برود. در واقع با این حال، هیچ کار خاصی برای رسیدگی به تغییر پیکربندی لازم نیست، زیرااندازه ویوها تغییر می‌کند و به طور خودکار متناسب با صفحه نمایش جدید منتقل می‌شود.

پس همین. سعی کنید دوباره بچرخید و تحسین کنید که چگونه تغییر به آرامی‌انجام می‌شود.

خطرات مدیریت تغییرات پیکربندی

این کار آنقدر آسان است و آنقدر خوب کار می‌کند که احتمالا تعجب می‌کنید که چرا همیشه این کار را انجام نمی‌دهید. به نظر می‌رسد زندگی را بسیار آسان‌تر می‌کند. با این حال، مدیریت تغییرات پیکربندی به تنهایی یک عادت خطرناک است.

اول، تغییرات پیکربندی مبتنی بر توصیفگر منابع دیگر به طور خودکار کار نمی‌کنند. در عوض باید هنگام تشخیص تغییر پیکربندی، ویوی خود را به صورت دستی دوباره بارگیری کنید. این می‌تواند پیچیده‌تر از آن چیزی باشد که به نظر می‌رسد.

دوم و مهمتر از آن، این رویکرد احتمالا باعث می‌شود که شما Activity.onSavedInstanceState(…) را برای ذخیره حالت‌های رابط کاربری بازنویسی نکنید. انجام این کار هنوز ضروری است، حتی اگر اکتیویتی در چرخش حفظ شود. زیرا هنوز باید نگران مرگ و ایجاد مجدد در موقعیت‌های کم حافظه باشید.

WebView در مقابل یک رابط کاربری سفارشی

یک رابط کاربری ساخته شده به صورت بومی (بدون WebView) به شما امکان کنترل کامل بر ظاهر و رفتار برنامه شما را می‌دهد. همچنین، اغلب برای کاربران پاسخگوتر و سازگارتر به نظر می‌رسد. اما نمایش محتوای وب به جای ارائه رابط کاربری سفارشی خود مزایای زیادی دارد.

نمایش سایت Unsplash در WebView به شما امکان می‌دهد یک ویژگی بزرگ را خیلی سریع تر ادغام کنید. برای ساخت این رابط کاربری نیازی به نگرانی در مورد واکشی توضیحات تصویر، نام حساب کاربری یا سایر ابرداده‌های عکس ندارید. شما به سادگی می‌توانید از آنچه Unsplash قبلا در دسترس قرار داده است استفاده کنید.

یکی دیگر از مزایای نمایش محتوای وب این است که محتوای وب می‌تواند بدون نیاز به به روز رسانی برنامه تغییر کند. به عنوان مثال، اگر نیاز به نمایش یک privacy policy یا شرایط خدمات در برنامه خود دارید، می‌توانید انتخاب کنید که به جای کدگذاری سند در برنامه خود، یک وب سایت را نشان دهید. به این ترتیب، هر تغییری را می‌توان به سادگی به جای به روز رسانی برنامه به یک وب سایت منتقل کرد.

بیشتر بدانیم: تزریق آبجکت‌های جاوا اسکریپت

در این فصل، نحوه استفاده از WebViewClient و WebChromeClient برای پاسخ به رویدادهای خاصی که در WebView شما اتفاق می‌افتد را مشاهده کردید. با این حال، می‌توان با تزریق آبجکت‌ها جاوا اسکریپت دلخواه به سند موجود در خود WebView کارهای بیشتری انجام داد. مستندات developer.android.com/reference/android/webkit/WebView.html را بررسی کنید و به تابع addJavascriptInterface(Object, String) بروید. مستندات از امضاهای متد جاوا استفاده می‌کنند، اما به یاد داشته باشید که Object معادل Any در Kotlin است. با استفاده از این می‌توانید یک شی دلخواه را با نامی‌که مشخص کرده‌اید به سند تزریق کنید:

```
webView.addJavascriptInterface
    (object : Any() {@JavascriptInterface
    fun send(message: String) {
        Log.i(TAG, "Received message: $message")
    }
}, "androidObject")
```

و سپس آن را به این صورت فراخوانی کنید:

```
<input type="button" value="In WebView!"
onClick="sendToAndroid('In Android land')" />
<script type="text/javascript">
function sendToAndroid(message) {
    androidObject.send(message);
}
</script>
```

چند قسمت پیچیده در این مورد وجود دارد. اولین مورد این است که وقتی send(String) را فراخوانی می‌کنید، تابع کاتلین در نخ اصلی فراخوانی نمی‌شود. به جای آن بر روی نخ متعلق به WebView فراخوانی می‌شود. بنابراین اگر می‌خواهید رابط کاربری اندروید را به روز کنید، باید از یک هندلر برای انتقال کنترل به نخ اصلی استفاده کنید.

بخش دیگر این است که انواع داده‌های زیادی پشتیبانی نمی‌شوند. شما String را دارید، انواع اصلی اولیه، و بس. هر چیز پیچیده‌تری باید از طریق یک رشته تنظیم شود، معمولا با تبدیل آن به JSON قبل از ارسال و سپس تجزیه آن هنگام دریافت.

با شروع API 17 (Jelly Bean 4.2) و بالاتر، فقط توابع عمومی‌حاشیه‌نویسی JavascriptInterface@ به جاوا اسکریپت صادر می‌شوند. قبل از آن، تمام توابع عمومی‌در سلسله مراتب object در دسترس بودند.

در هر صورت، این می‌تواند خطرناک باشد. شما اجازه می‌دهید برخی از صفحات وب بالقوه عجیب و غریب با برنامه شما بازی کنند. بنابراین، برای‌ایمن بودن،ایده خوبی است که مطمئن شوید که HTML مورد نظر را دارید – یا این، یا با رابط کاربری که در معرض آن قرار می‌دهید بسیار محافظه کار باشید.

بیشتر بدانیم: به روز رسانی WebView

WebView بر اساس پروژه منبع باز Chromium است. این همان موتور رندر استفاده شده توسط برنامه Chrome را برای Android به اشتراک می‌گذارد، به این معنی که صفحات باید در این دو یکسان به نظر برسند و رفتار کنند.

مبتنی بر Chromium به این معنی است که WebView در استانداردهای وب و جاوا اسکریپت به روز می‌ماند. از دیدگاه توسعه، یکی از هیجان انگیزترین ویژگی‌ها، پشتیبانی از دیباگ از راه دور WebView با استفاده از Chrome DevTools است.

از اندروید ۵.۰، لایه Chromium WebView به طور خودکار از فروشگاه Google Play به روز می‌شود. کاربران دیگر منتظر نسخه‌های جدید اندروید برای دریافت به روزرسانی‌های امنیتی (و ویژگی‌های جدید) نیستند. اخیرا، از Nougat (اندروید ۷.۰)، لایه Chromium برای WebView مستقیما از فایل APK Chrome می‌آید و مصرف حافظه و منابع را کاهش می‌دهد. بنابراین می‌توانید خیالتان راحت باشد، زیرا می‌دانید که Google برای به روز نگه داشتن اجزای WebView کار می‌کند.

بیشتر بدانیم: تب‌های سفارشی کروم (راه آسان دیگر)

شما قبلا دو راه برای نمایش محتوای وب را مشاهده کرده‌اید: می‌توانید مرورگر وب کاربر را راه اندازی کنید یا محتوا را در برنامه خود جاسازی کنید. گزینه سومی‌نیز وجود دارد که ترکیبی از آنچه تاکنون دیده‌اید است.

تب‌های سفارشی Chrome به شما امکان می‌دهد مرورگر وب Chrome را به گونه‌ای راه اندازی کنید که بومی‌برنامه شما باشد. می‌توانید ظاهر آن را طوری پیکربندی کنید که بخشی از برنامه شما به نظر برسد و احساس کنید کاربر هرگز برنامه شما را ترک نکرده است.

هنگامی که از یک تب سفارشی استفاده می‌کنید، بسیار شبیه به کروم رفتار می‌کند. نمونه مرورگر حتی به اطلاعاتی مانند رمزهای عبور ذخیره شده کاربر، حافظه پنهان مرورگر و کوکی‌ها از مرورگر کامل کروم دسترسی دارد. این بدان معناست که اگر کاربر در کروم وارد Unsplash شده باشد، در هر تب سفارشی نیز وارد Unsplash می‌شود. با WebView، کاربر باید هم در کروم و هم در گالری عکس وارد Unsplash شود.

نقطه ضعف استفاده از یک تب سفارشی به جای WebView این است که شما کنترل زیادی بر محتوایی که نمایش می‌دهید ندارید. به عنوان مثال، نمی‌توانید انتخاب کنید که از تب‌های سفارشی فقط در نیمه بالای صفحه خود استفاده کنید یا دکمه‌های پیمایش را به پایین یک تب سفارشی اضافه کنید.

چالش: استفاده از دکمه بازگشت برای تاریخچه مرورگر

ممکن است متوجه شده باشید که پس از راه اندازی PhotoPageActivity، می‌توانید پیوندهای دیگری را در WebView دنبال کنید. با این حال، مهم نیست که چند لینک را دنبال می‌کنید، دکمه Back همیشه شما را بلافاصله به PhotoGalleryActivity باز می‌گرداند. اگر در عوض بخواهید دکمه بازگشت کاربران را از طریق تاریخچه مرور خود در WebView بیاورد، چه؟

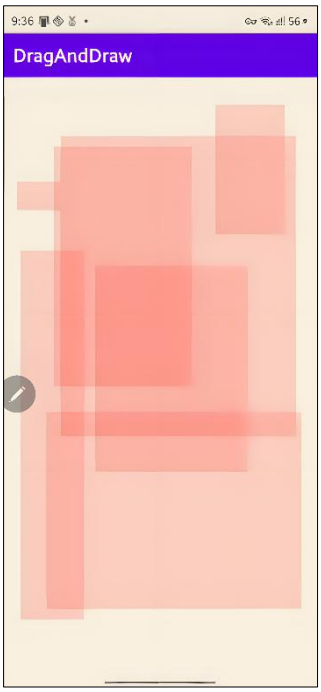
این رفتار را با بازنویسی تابع دکمه برگشت Activity.onBackPressed() پیاده‌سازی کنید. در تابع، از ترکیبی از توابع تاریخچه مرور WebView (WebView.canGoBack() و WebView.goBack()) برای انجام کار درست استفاده کنید. اگر مواردی در سابقه مرور WebView وجود دارد، به مورد قبلی برگردید. در غیر این صورت، اجازه دهید دکمه بازگشت با فراخوانی به super.onBackPressed() عادی رفتار کند.

۲۹

ویوهای سفارشی و رویدادهای لمسی

در این فصل، یاد خواهیم گرفت که چگونه رویدادهای لمسی را با نوشتن یک زیر کلاس سفارشی از View به نام BoxDrawingView مدیریت کنیم. کلاس BoxDrawingView ستاره پروژه جدیدی به نام DragAndDraw خواهد بود و در پاسخ به لمس صفحه نمایش و کشیدن توسط کاربر، جعبه‌هایی را ترسیم می‌کند. محصول نهایی مانند شکل ۲۹.۱ خواهد بود.

شکل ۲۹.۱ جعبه‌هایی که در اشکال واندازه‌های مختلف کشیده شده‌اند



راه اندازی پروژه DragAndDraw

یک پروژه جدید به نام DragAndDraw با API 24 به عنوان حداقل SDK ایجاد کنید. پروژه را بدون اکتیویتی شروع و سپس یک اکتیویتی لانچر خالی به نام DragAndDrawActivity ایجاد کنید.

DragAndDrawActivity زیر کلاس AppCompatActivity و میزبان یک BoxDrawingView خواهد بود، ویوی سفارشی که می‌خواهیم بنویسیم. مدیریت رویداد لمسی در BoxDrawingView پیاده‌سازی خواهد شد.

ایجاد یک Custom View

اندروید بسیاری از ویوها و ویجت‌های استاندارد عالی را ارائه می‌دهد، اما گاهی اوقات به یک ویوی سفارشی نیاز داریم که تصاویری منحصر به فرد برنامه شما را ارائه دهد.

در حالی که انواع ویوهای سفارشی وجود دارد، می‌توان آنها را به دو دسته گسترده تقسیم کرد:

- ساده (simple): یک ویوی ساده ممکن است در درون پیچیده باشد. چیزی که آن را "ساده" می‌کند این است که هیچ ویو فرزندی ندارد. یک ویوی ساده تقریبا همیشه رندر سفارشی را انجام می‌دهد.
- کامپوزیت (composite): این ویوهای ترکیبی از آبجکت‌ها ویو دیگر هستند. ویوهای کاپوزیت معمولا ویوهای فرزند را مدیریت می‌کنند اما رندر سفارشی را انجام نمی‌دهند. در عوض، رندر به هر ویوی فرزند تفویض می‌شود.

هنگام ایجاد یک ویوی سفارشی، سه مرحله را دنبال می‌کنیم:

۱. یک سوپرکلاس انتخاب کنید. برای یک ویوی سفارشی ساده، View یک بوم خالی است، بنابراین رایج ترین انتخاب است. برای یک ویوی سفارشی کامپوزیت، یک کلاس لیوت مناسب مانند FrameLayout را انتخاب می‌کنیم.

۲. این کلاس را زیر کلاس و سازندگان سوپرکلاس را بازنویسی می‌کنیم.

۳. سایر توابع کلیدی را برای سفارشی کردن رفتار ویو بازنویسی می‌کنیم.

ایجاد BoxDrawingView

BoxDrawingView یک ویوی ساده و یک زیر کلاس مستقیم از View خواهد بود.

یک کلاس جدید با نام BoxDrawingView ایجاد و View را به سوپرکلاس آن تبدیل کنید. در BoxDrawingView.kt، یک سازنده اضافه کنید که یک شی Context و یک AttributeSet غیر تهی را با پیش‌فرض null می‌گیرد.

لیست ۲۹.۱ پیاده‌سازی اولیه برای BoxDrawingView (BoxDrawingView.kt)

```
package com.codeb.draganddraw

import android.content.Context
import android.util.AttributeSet
import android.view.View

class BoxDrawingView(context: Context, attrs: AttributeSet?
    = null) :
    View(context, attrs) {
}
```

ارائه مقدار پیش‌فرض null برای AttributeSet به طور موثر دو سازنده برای ویوی شما فراهم می‌کند. دو سازنده مورد نیاز است، زیرا ویوی شما می‌تواند در کد یا از یک فایل لیوت نمونه سازی شود. ویوهای نمونه سازی شده از یک فایل لیوت نمونه‌ای از AttributeSet حاوی ویژگی‌های XML را دریافت می‌کنند. حتی اگر قصد استفاده از هر دو سازنده را ندارید، گنجاندن آنها تمرین خوبی است.

در مرحله بعد، فایل لیوت res/layout/activity_drag_and_draw.xml خود را برای استفاده از ویوی جدید خود به روز کنید.

لیست ۲۹.۲ افزودن BoxDrawingView به لیوت (res/layout/activity_drag_and_draw.xml)

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".DragAndDrawActivity">
</androidx.constraintlayout.widget.ConstraintLayout>
<com.codeb.draganddraw.BoxDrawingView
xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
```

```
android:layout_height="match_parent"
android:id="@+id/main"/>
```

باید از نام کلاس BoxDrawingView استفاده کنیم تا Layout inflater بتواند آن را پیدا کند. inflater از روی یک فایل لیوت کار می‌کند و نمونه‌های View را ایجاد می‌کند. اگر نام عنصر یک نام کلاس فاقد توصیفگر باشد، inflater به دنبال کلاسی با آن نام در پکیج‌های android.view و android.widget می‌گردد. اگر کلاس در جای دیگری قرار دارد، لیوت آن را پیدا نمی‌کند و برنامه شما از کار می‌افتد.

بنابراین برای کلاس‌های سفارشی و سایر کلاس‌هایی که خارج از android.view و android.widget قرار دارند، همیشه باید نام کلاس به طور کامل مشخص شود.

DragAndDraw را اجرا کنید تا مطمئن شوید که کارمان صحیح است. تنها چیزی که خواهید دید یک ویوی خالی است.

گام بعدی گوش دادن BoxDrawingView به رویدادهای لمسی و استفاده از اطلاعات آن برای ترسیم جعبه‌ها روی صفحه است.

مدیریت رویدادهای لمسی

یکی از راه‌های گوش دادن به رویدادهای لمسی، تنظیم شنونده رویداد لمسی با استفاده از توابع View زیر است:

```
fun setOnTouchListener(l: View.OnTouchListener)
```

این تابع مانند setOnClickListener(View.OnClickListener) کار می‌کند. ما پیاده‌سازی View.OnTouchListener را ارائه می‌دهیم و هر بار که یک رویداد لمسی اتفاق می‌افتد، شنونده فراخوانی می‌شود.

با این حال، از آنجایی که View را زیر کلاس کردیم، می‌توان میانبر زد و به جای آن این تابع View را بازنویس کرد:

```
override fun onTouchEvent(event: MotionEvent): Boolean
```

این تابع نمونه‌ای از MotionEvent را دریافت می‌کند، کلاسی که رویداد لمسی از جمله مکان و اکشن آن را توصیف می‌کند. این اکشن مرحله رویداد را توصیف می‌کند:

ثابت‌های عمل	توضیحات
ACTION_DOWN	کاربر انگشت خود را روی صفحه لمس می‌کند
ACTION_MOVE	کاربر انگشت خود را روی صفحه حرکت می‌دهد

ACTION_UP	کاربر انگشت خود را از روی صفحه بلند می‌کند
ACTION_CANCEL	ویوی والد رویداد لمسی را رهگیری کرده است

در پیاده‌سازی onTouchEvent(MotionEvent)، می‌توانیم مقدار اکشن را با فراخوانی تابع MotionEvent بررسی کنیم:

```
final fun getAction(): Int
```

بیایید به آن برسیم. در BoxDrawingView.kt، یک تگ Log اضافه و سپس یک پیاده‌سازی از onTouchEvent(MotionEvent) که برای هر یک از چهار اکشن پیامی‌را ثبت می‌کند، اضافه کنید.

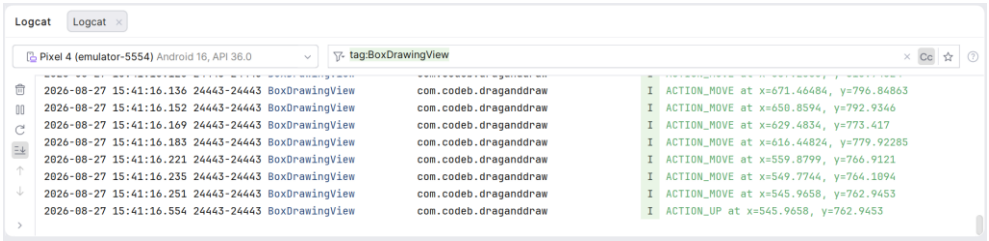
لیست ۲۹.۳ پیاده‌سازی BoxDrawingView (BoxDrawingView.kt)

```
private const val TAG = "BoxDrawingView"
class BoxDrawingView(context: Context, attrs: AttributeSet?
    = null) :
    View(context, attrs) {
    override fun onTouchEvent(event: MotionEvent): Boolean {
        val current = PointF(event.x, event.y)
        var action = ""
        when (event.action) {
            MotionEvent.ACTION_DOWN -> {
                action = "ACTION_DOWN"
            }
            MotionEvent.ACTION_MOVE -> {
                action = "ACTION_MOVE"
            }
            MotionEvent.ACTION_UP -> {
                action = "ACTION_UP"
            }
            MotionEvent.ACTION_CANCEL -> {
                action = "ACTION_CANCEL"
            }
        }
        Log.i(TAG,
        "$action at x=${current.x},y=${current.y}")
        return true
    }
}
```

توجه داشته باشید که مختصات X و Y خود را در یک شی PointF پکیج می‌کنیم. ما می‌خواهیم این دو مقدار را در ادامه فصل به کار بگیریم.

DragAndDraw را اجرا و Logcat را بالا بکشید. صفحه را لمس کنید و انگشتتان را بکشید. شما باید گزارشی از مختصات X و Y هر عمل لمسی که BoxDrawingView دریافت می‌کند را مشاهده کنید.

تصویر ۲۹.۲ تمام اکتیویتی‌های



ردیابی رویدادهای حرکت

BoxDrawingView برای ترسیم جعبه‌ها روی صفحه و نه گزارش مختصات، در نظر گرفته شده است. برای رسیدن به این منظور باید چند مشکل حل شود.

ابتدا برای تعریف یک جعبه به دو نقطه نیاز داریم: نقطه شروع (جایی که انگشت در ابتدا قرار گرفته است) و نقطه پایان (جایی که انگشت در حال حاضر قرار دارد).

بنابراین، برای تعریف یک جعبه، نیاز به پیگیری داده‌های بیش از یک MotionEvent است. ما این داده‌ها را در یک شی Box ذخیره خواهیم کرد.

یک کلاس به نام Box برای ذخیره داده‌هایی یک جعبه را تعریف می‌کنند، ایجاد نمایید.

لیست ۲۹.۴ افزودن جعبه (Box.kt)

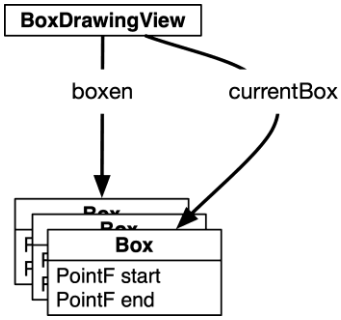
```
package com.codeb.draganddraw

import android.graphics.PointF

class Box(val start: PointF) {
    var end: PointF = start
    val left: Float
        get() = Math.min(start.x, end.x)
    val right: Float
        get() = Math.max(start.x, end.x)
    val top: Float
        get() = Math.min(start.y, end.y)
    val bottom: Float
        get() = Math.max(start.y, end.y)
}
```

هنگامی که کاربر BoxDrawingView را لمس می‌کند، یک جعبه جدید ایجاد و به لیست جعبه‌های موجود اضافه می‌شود (تصویر ۲۹.۳).

تصویر ۲۹.۳ آبجکت‌هاها در DragAndDraw



به BoxDrawingView برگردید، از آبجکت‌ها Box جدید خود برای ردیابی وضعیت استفاده نمایید.

لیست ۲۹.۵ افزودن توابع چرخه حیات درگ (BoxDrawingView.kt)

```
class BoxDrawingView(context: Context, attrs: AttributeSet?
    = null) :
    View(context, attrs) {
    private var currentBox: Box? = null
    private val boxen = mutableListOf<Box>()
    override fun onTouchEvent(event: MotionEvent): Boolean {
        val current = PointF(event.x, event.y)
        var action = ""
        when (event.action) {
            MotionEvent.ACTION_DOWN -> {
                action = "ACTION_DOWN"
                currentBox = Box(current).also {
                    boxen.add(it)
                }
            }
            MotionEvent.ACTION_MOVE -> {
                action = "ACTION_MOVE"
                updateCurrentBox(current)
            }
            MotionEvent.ACTION_UP -> {
                action = "ACTION_UP"
                updateCurrentBox(current)
                currentBox = null
            }
            MotionEvent.ACTION_CANCEL -> {
                action = "ACTION_CANCEL"
                currentBox = null
            }
```

```
        }
        Log.i(TAG, "$action at x=${current.x},
            y=${current.y}")
        return true
    }
    private fun updateCurrentBox(current: PointF) {
        currentBox?.let {
            it.end = current
            invalidate()
        }
    }
}
```

هر زمان که یک رویداد ACTION_DOWN دریافت می‌شود، currentBox را به عنوان یک جعبه جدید با مکان رویداد تنظیم می‌کنیم. این جعبه جدید به لیست جعبه‌ها اضافه می‌شود.

همانطور که انگشت کاربر در اطراف صفحه حرکت می‌کند، currentBox.end را به روز می‌کنیم. سپس، هنگامی که لمس قطع می‌شود یا زمانی که انگشت کاربر از صفحه خارج می‌شود، Box فعلی را با مکان گزارش شده نهایی به روز و currentBox را خالی می‌کنیم. جعبه کامل است؛ با خیال راحت در لیست ذخیره می‌شود.

به فراخوانی invalidate() در تابع updateCurrentBox() توجه کنید. این امر BoxDrawingView را مجبور می‌کند تا خود را دوباره ترسیم کند تا کاربر بتواند جعبه را در حین کشیدن روی صفحه ببیند. که ما را به مرحله بعدی می‌رساند: کشیدن جعبه‌ها روی صفحه.

رندرینگ در داخل onDraw

هنگامی که برنامه ما راه اندازی می‌شود، همه ویوهای آن نامعتبر (invalid) هستند. بدان معنا که آنها چیزی روی صفحه نمایش نکشیده‌اند. برای رفع این وضعیت، اندروید تابع draw() ویو را فراخوانی می‌کند که باعث می‌شود ویو و در ادامه فرزندانش نیز خودشان را ترسیم می‌کنند. سپس فرزندان آن فرزندان خود را ترسیم می‌کنند و به همین ترتیب سلسله مراتب را پایین می‌آورند. هنگامی که همه ویوها در سلسله مراتب خود را ترسیم کرده‌اند، ویوی سطح بالا دیگر نامعتبر نیست.

همچنین می‌توانیم به صورت دستی مشخص کنیم که یک ویو نامعتبر است، حتی اگر در حال حاضر روی صفحه باشد. این کار باعث می‌شود که سیستم با هر گونه به روز رسانی لازم، ویوی را دوباره ترسیم کند. هر زمان که کاربر کادر جدیدی ایجاد کند یا اندازه جعبه را با حرکت دادن انگشت خود تغییر دهد،

BoxDrawingView را به عنوان نامعتبر علامت گذاری می‌کنیم. این کار اطمینان حاصل می‌کند که کاربران می‌توانند هنگام ایجاد جعبه‌های خود ببینند که چگونه به نظر می‌رسند.

تابع View زیر را بازنویسی می‌کنیم:

```
protected fun onDraw(canvas: Canvas)
```

فراخوانی ()invalidate که در پاسخ به ACTION_MOVE در onTouchEvent(MotionEvent) انجام می‌دهیم، BoxDrawingView را دوباره نامعتبر می‌کند. این کار ترسیم دوباره و فراخوانی مجدد onDraw(Canvas) می‌شود.

حالا بیایید پارامتر Canvas را در نظر بگیریم. Canvas و Paint دو کلاس اصلی ترسیم در اندروید هستند:

- کلاس Canvas تمام عملیات ترسیمی را که انجام می‌دهید دارد. توابعی که در Canvas فراخوانی می‌کنیم تعیین می‌کند که کجا و چه چیزی ترسیم شود – یک خط، یک دایره، یک کلمه یا یک مستطیل.
- کلاس Paint نحوه انجام این عملیات را تعیین می‌کند. توابعی که در Paint فراخوانی می‌کنیم، ویژگی‌ها را مشخص می‌کنند – آیا اشکال پر شده باشند، متن فونت و رنگ خطوط چه باشد.

در BoxDrawingView.kt، هنگامی که BoxDrawingView مقداردهی اولیه شد، دو شی Paint ایجاد کنید.

لیست ۲۹.۶ ایجاد رنگ خود (BoxDrawingView.kt)

```
class BoxDrawingView(context: Context, attrs: AttributeSet?
    = null) :
    View(context, attrs) {
    private var currentBox: Box? = null
    private val boxen = mutableListOf<Box>()
    private val boxPaint = Paint().apply {
        color = 0x22ff0000.toInt()
    }
    private val backgroundPaint = Paint().apply {
        color = 0xfff8efe0.toInt()
    }
    …
  }
}
```

با ساخت رنگ‌ها، اکنون می‌توانیم جعبه‌های خود را روی صفحه بکشیم.

لیست ۲۹.۷ بازنویسی onDraw(Canvas) (BoxDrawingView.kt)

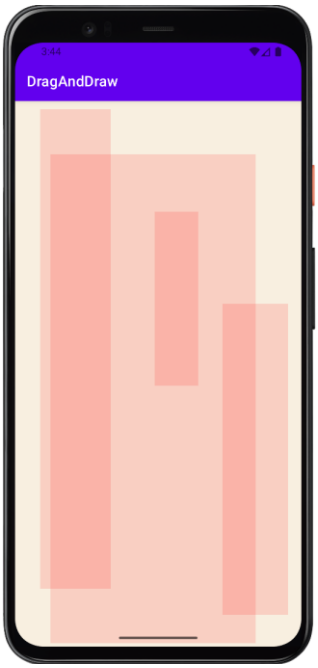
```
class BoxDrawingView(context: Context, attrs: AttributeSet?
    = null) :
    View(context, attrs) {
    …
    override fun onDraw(canvas: Canvas) {
        canvas.drawPaint(backgroundPaint)
        boxen.forEach { box ->
            canvas.drawRect(box.left, box.top, box.right
                , box.bottom, boxPaint)
        }
    }
}
```

بخش اول این کد ساده است: با استفاده از رنگ پس زمینه مایل به سفید خود، بوم (canvas) را با پس زمینه جعبه‌های خود پر می‌کنیم.

سپس، برای هر کادر در لیست جعبه‌های خود، با نگاه کردن به دو نقطه برای جعبه، تعیین می‌کنید که چپ، راست، بالا و پایین جعبه باید چه باشد. مقادیر چپ و بالا حداقل مقادیر و مقادیر پایین و راست حداکثر مقادیر خواهند بود.

پس از محاسبه این مقادیر، Canvas.drawRect(…) را فراخوانی می‌کنیم. برای کشیدن یک مستطیل قرمز روی صفحه DragAndDraw را اجرا و چند مستطیل قرمز بکشید (شکل ۲۹.۴).

شکل ۲۹.۴ بیان احساسات برنامه ریزی شده



و تمام. اکنون ویویی ایجاد کرده‌اید که رویدادهای لمسی خود را مدیریت و ترسیم خود را انجام می‌دهد.

بیشتر بدانیم: GestureDetector

گزینه دیگر برای مدیریت رویدادهای لمسی، استفاده از شی GestureDetector است. به جای افزودن منطقی برای تشخیص رویدادهایی مانند کشیدن انگشت، GestureDetector شنوندگانی دارد که کارهای سنگین را انجام و در صورت وقوع یک رویداد خاص به شما اطلاع می‌دهند. پیاده‌سازی GestureDetector.OnGestureListener دارای توابعی برای رویدادهایی مانند فشار دادن طولانی و پیمایش است. همچنین یک GestureDetector.OnDoubleTapListener وجود دارد که با تشخیص دوبار فشار فعال می‌شود. بسیاری از موارد به کنترل کامل ارائه شده با بازنویسی تابع onTouch نیاز ندارند، بنابراین استفاده از GestureDetector یک انتخاب عالی است.

چالش: حفظ وضعیت

چگونگی حفظ باکس‌های خود را در هنگام چرخش از داخل View کشف کنید. این کار را می‌توان با توابع View زیر انجام داد:

```
protected fun onSaveInstanceState(): Parcelable
protected fun onRestoreInstanceState(state: Parcelable)
```

این توابع مانند OnSaveInstanceState(Bundle) اکتیویتی و فرگمنت کار نمی‌کنند. ابتدا فقط در صورتی فراخوانی می‌شوند که ویوی شما شناسه داشته باشد. دوم، به جای گرفتن یک Bundle، آنها یک شی را برمی‌گردانند و پردازش می‌کنند که رابط Parcelable را پیاده‌سازی می‌کند.

توصیه می‌کنیم به جای پیاده‌سازی یک کلاس Parcelable خودتان، از یک Bundle به عنوان Parcelable استفاده کنید. کاتلین دارای یک حاشیه‌نویسی Parcelize@ است که ایجاد یک کلاس Parcelable را آسان‌تر می‌کند، اما Bundle یک کلاس رایج در اندروید است و اکثر توسعه دهندگان با نحوه عملکرد آن بسیار آشنا هستند.

در نهایت، باید saved state والد BoxDrawingView، کلاس View را نیز حفظ کنید. نتیجه super.onSaveInstanceState() را در Bundle جدید خود ذخیره و همان نتیجه را هنگام فراخوانی super.onRestoreInstanceState(Parcelable) به سوپرکلاس ارسال کنید.

چالش: جعبه‌های چرخان

برای یک چالش سخت تر، برنامه را طوری بسازید که بتوانید از انگشت دوم برای چرخاندن مستطیل‌های خود استفاده کنید. برای انجام این کار، باید چندین اشاره گر را در کد مدیریت MotionEvent خود مدیریت کنید. همچنین باید بوم خود را بچرخانید.

هنگام برخورد با لمس‌های متعدد، به این‌ایده‌های اضافی نیاز دارید:

- pointer index: این به شما می‌گوید که رویداد برای کدام اشاره گر در مجموعه اشاره گرهای فعلی است
- pointer ID: یک شناسه منحصر به فرد برای یک انگشت خاص در یک حرکت به شما می‌دهد. شاخص اشاره گر ممکن است تغییر کند، اما شناسه اشاره گر تغییر نمی‌کند.

برای جزئیات بیشتر، مستندات توابع MotionEvent زیر را بررسی کنید:

```
final fun getActionMasked(): Int
final fun getActionIndex(): Int
final fun getPointerId(pointerIndex: Int): Int
final fun getX(pointerIndex: Int): Float
final fun getY(pointerIndex: Int): Float
```

همچنین به مستندات ثابت‌های ACTION_POINTER_UP و ACTION_POINTER_DOWN نگاهی بیاندازید.

۳۰

انیمیشن

برای اینکه یک برنامه کاربردی باشد، تنها کاری که باید انجام شود نوشتن کد خود به درستی است. با این حال، برای اینکه استفاده از یک برنامه لذت بخش باشد، باید آن را مانند یک پدیده واقعی و فیزیکی که روی صفحه نمایش تلفن یا تبلت Broadcast می‌شود، ایجاد کنیم.

چیزهای واقعی حرکت می‌کنند. باید عناصر برنامه را متحرک کنیم. در واقع باید از انیمیشن‌ها استفاده کنیم.

در این فصل اپلیکیشنی به نام Sunset خواهیم نوشت که صحنه‌ای از خورشید را در آسمان به نمایش می‌گذارد. وقتی صحنه را لمس می‌کنیم، خورشید را در زیر افق متحرک می‌کند و آسمان مانند غروب خورشید تغییر رنگ می‌دهد.

ساختن صحنه

اولین قدم ساخت صحنه‌ای است که متحرک خواهد شد. یک پروژه جدید به نام Sunset ایجاد کنید.

غروب خورشید در کنار دریا باید رنگارنگ باشد، بنابراین ابتدا با نامگذاری چند رنگ شروع می‌کنیم. فایل colors.xml را در پوشه res/values را باز و مقادیر زیر را به اضافه کنید.

لیست ۳۰.۱ افزودن رنگ‌های غروب آفتاب (res/values/colors.xml)

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    ...
    <color name="bright_sun">#fcfcb7</color>
    <color name="blue_sky">#1e7ac7</color>
    <color name="sunset_sky">#ec8100</color>
    <color name="night_sky">#05192e</color>
    <color name="sea">#224869</color>
</resources>
```

ویوهای مستطیل شکل تصور خوبی از آسمان و دریا ایجاد می‌کند. اما خارج از Minecraft مردم یک خورشید مستطیل شکل را دوست ندارند، مهم نیست که چقدر به نفع سادگی فنی آن استدلال کنید.

بنابراین، در پوشه /res/drawable، یک شکل بیضی برای یک خورشید دایره‌ای به نام sun.xml اضافه کنید.

لیست ۳۰.۲ اضافه کردن Sun XML drawable (res/drawable/sun.xml)

```
<?xml version="1.0" encoding="utf-8"?>
<shape
xmlns:android="http://schemas.android.com/apk/res/android"
    android:shape="oval">
    <solid android:color="@color/bright_sun" />
</shape>
```

هنگام نمایش این بیضی را در ویوی مربع شکل، یک دایره به نمایش در می‌آید.

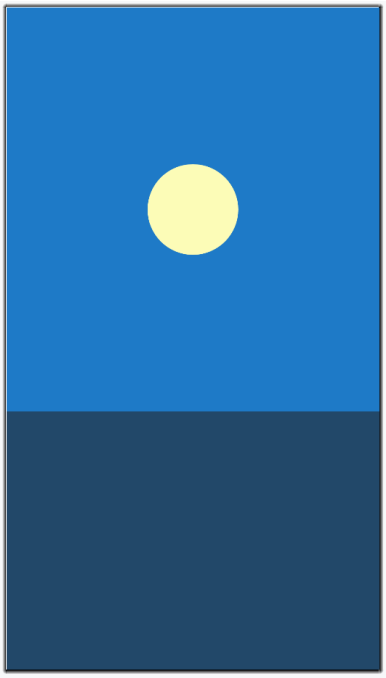
در مرحله بعد، کل صحنه را در یک فایل لیوت بسازید. res/layout/activity_main.xml را باز، محتویات فعلی را حذف و موارد زیر را اضافه کنید.

لیست ۳۰.۳ راه اندازی لیوت (res/layout/activity_main.xml)

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    android:id="@+id/scene"
    android:orientation="vertical"
    android:layout_width="match_parent"
    android:layout_height="match_parent">
    <FrameLayout
        android:id="@+id/sky"
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="0.61"
        android:background="@color/blue_sky">
        <ImageView
            android:id="@+id/sun"
            android:layout_width="100dp"
            android:layout_height="100dp"
            android:layout_gravity="center"
            android:src="@drawable/sun" />
    </FrameLayout>
    <View
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="0.39"
        android:background="@color/sea" />
</LinearLayout>
```

پیش‌نمایش را بررسی کنید. باید یک صحنه روز از خورشید را در آسمان آبی بر فراز دریای آبی تیره ببینید. قبل از حرکت، لحظه‌ای وقت بگذارید و Sunset را اجرا کنید تا مطمئن شوید که همه چیز به درستی انجام شده است. باید شبیه شکل ۳۰.۱ به نظر برسد.

شکل ۳۰.۱ قبل از غروب خورشید



property animator

اکنون که صحنه را تنظیم کردیم، وقت آن است که با حرکت دادن بخش‌هایی، آن را به کار بگیریم. قصد داریم خورشید را در زیر افق متحرک کنید.

اما قبل از شروع انیمیشن، اضافه کردن چند بیت اطلاعات در اکتیویتی لازم است. داخل onCreate(…) چند ویو را به عنوان متغییر در MainActivity تعریف کنید.

لیست ۳۰.۴ بیرون کشیدن مراجع view (MainActivity.kt)

```
class MainActivity : AppCompatActivity() {
    private lateinit var sceneView: View
    private lateinit var sunView: View
    private lateinit var skyView: View
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContentView(R.layout.activity_main)
        ViewCompat.setOnApplyWindowInsetsListener
        (findViewById(R.id.~~main~~scene)) { v, insets ->
            val systemBars = insets.getInsets
            (WindowInsetsCompat.Type.systemBars())
            v.setPadding(systemBars.left, systemBars.top
            , systemBars.right, systemBars.bottom)
            insets
        }
        sceneView = findViewById(R.id.scene)
        sunView = findViewById(R.id.sun)
        skyView = findViewById(R.id.sky)
    }
}
```

نقشه از این قرار است: sunView را به آرامی‌حرکت می‌دهیم تا بالای آن درست در لبه پایین آسمان باشد. از آنجایی که کف آسمان و بالای دریا یکسان است، خورشید در پشت افقی دریا پنهان خواهد شد. ما این کار را با ترجمه (translating) محل بالای sunView به پایین والد آن انجام خواهید داد.

دلیل حرکت افقی خورشید در پشت دریا بلافاصله مشخص نیست. این به ترتیب قرار گیری ویوها مربوط می‌شود. ویوها به ترتیبی که در لیوت اعلام شده‌اند ترسیم می‌شوند. ویوهایی که بعدا در چیدمان اعلام می‌شوند در بالای ویوهای بالاتر ترسیم می‌شوند. در این مورد از آنجایی که ویو خورشید قبل از ویو دریا تعریف می‌شود، ویو دریا در بالای ویو خورشید قرار دارد. هنگامی که خورشید از کنار دریا عبور می‌کند، به نظر می‌رسد که پشت آن می‌رود.

اولین قدم فهمیدن شروع و پایان انیمیشن است. این مرحله را در یک تابع جدید به نام startAnimation() می‌نویسیم.

لیست ۳۰.۵ دریافت برترین بازدیدها (MainActivity.kt)

```
class MainActivity : AppCompatActivity() {
    …
    private fun startAnimation() {
        val sunYStart = sunView.top.toFloat()
        val sunYEnd = skyView.height.toFloat()
    }
}
```

ویژگی top یکی از چهار ویژگی موجود در View است که local layout rect ویو را بر می‌گرداند. top، bottom، right و left یک مستطیل فرضی برای یک ویو ایجاد می‌کنند. این مستطیل موقعیت واندازه آن ویو را در رابطه با والد آن مشخص می‌کند.

با تغییر این مقادیر می‌توان مکان ویوی روی صفحه را تغییر داد، اما توصیه نمی‌شود. آنها هر بار که یک پاس لیوت رخ می‌دهد، بازنشانی می‌شوند، بنابراین تمایلی به حفظ ارزش خود ندارند.

در هر صورت، انیمیشن با top ویو در مکان فعلی شروع می‌شود. باید top در bottom والد sunView یعنی skyView به پایان برسد. برای رسیدن به این هدف height.toFloat() را فراخوانی می‌کنیم. مقدار ویژگی height همان مقدار bottom منهای است.

اکنون که می‌دانید انیمیشن باید از کجا شروع شود و به پایان برسد، یک ObjectAnimator برای اجرای آن ایجاد و اجرا کنید.

لیست ۳۰.۶ ایجاد انیماتور خورشید (MainActivity.kt)

```
class MainActivity : AppCompatActivity() {
    …
    private fun startAnimation() {
        val sunYStart = sunView.top.toFloat()
        val sunYEnd = skyView.height.toFloat()
        val heightAnimator = ObjectAnimator
            .ofFloat(sunView, "y", sunYStart, sunYEnd)
            .setDuration(3000)
        heightAnimator.start()
    }
}
```

نحوه عملکرد ObjectAnimator بعدا اشاره خواهیم کرد. ابتدا، startAnimation() را پیاده می‌کنیم تا هر بار که کاربر در هر نقطه از صحنه فشار می‌دهد، فراخوانی شود.

لیست ۳۰.۷ شروع انیمیشن در هر لمس (MainActivity.kt)

```
class MainActivity : AppCompatActivity() {
    …
    override fun onCreate(savedInstanceState: Bundle?) {
        …
        sceneView = findViewById(R.id.scene)
        sunView = findViewById(R.id.sun)
        skyView = findViewById(R.id.sky)
        sceneView.setOnClickListener {
            startAnimation()
        }
    }
    …
}
```

Sunset را اجرا و هر نقطه از صحنه را برای اجرای انیمیشن لمس کنید (تصویر ۳۰.۲).

شکل ۳۰.۲ غروب خورشید



حرکت خورشید در زیر افق را تماشا کنید.

نحوه عملکرد آن به این شرح است: ObjectAnimator یک property animator است. به جای داشتن دانش تخصصی در مورد نحوه حرکت یک ویو در صفحه، یک property animator به طور مکرر توابع تنظیم کننده ویژگی را با مقادیر مختلف فراخوانی می‌کند.

ما یک ObjectAnimator در فراخوانی ObjectAnimator.ofFloat(sunView, "y", 0, 1) ایجاد کردیم. هنگامی که ObjectAnimator راه اندازی می‌شود، به طور مکرر sunView.setY(Float) را با مقادیری که از ۰ شروع می‌شوند و به سمت بالا حرکت می‌کنند، فراخوانی می‌کند:

```
sunView.setY(0)
sunView.setY(0.02)
sunView.setY(0.04)
sunView.setY(0.06)
sunView.setY(0.08)
...
```

... و به همین ترتیب، تا زمانی که در نهایت sunView.setY(1) را فراخوانی کند. به این فرآیند یافتن مقادیر بین نقطه شروع و پایان، درون یابی (interpolation) گفته می‌شود. بین هر مقدار درون یابی شده، کمی‌زمان می‌گذرد، که باعث می‌شود به نظر برسد که ویو در حال حرکت است.

transformation propertyهای ویو

property animatorها عالی هستند، اما تنها با آنها نمی‌توان یک ویوی را به راحتی کاری که انجام دادید، متحرک کرد. property animatorهای مدرن اندروید هماهنگ با transformation propertyها کار می‌کند.

قبلا گفتیم که ویوی ما دارای یک local layout rect است، که موقعیت واندازه‌ای است که در فرآیند لیوت به آن اختصاص داده شده است. پس از آن می‌توان با تنظیم ویژگی‌های اضافی در ویو، به نام transformation propertiyهای، ویو را جابجا کرد. ما سه ویژگی برای چرخاندن ویو داریم (rotation، pivotX و pivotY)، دو ویژگی برای مقیاس بندی ویو به صورت عمودی و افقی (scaleX و scaleY)، و دو ویژگی برای حرکت ویو اطراف صفحه (translationX و translationY).

همه این ویژگی‌ها را می‌توان واکشی و اصلاح کرد. به عنوان مثال، اگر بخواهیم مقدار فعلی translationX را بدانیم، view.translationX را فراخوانی می‌کنیم. اگر بخواهیم آن را تنظیم کنیم، view.translationX = Float را فراخوانی می‌کنیم. پس ویژگی y چه می‌کند؟ ویژگی‌های x و y به ما اجازه می‌دهند کدی بنویسیم که به سادگی می‌گوید: "این ویو را در این مختصات X و این مختصات Y قرار دهید."در پشت صحنه این ویژگی‌ها translationX یا translationY را تغییر می‌دهند تا ویوی را در جایی که می‌خواهید قرار دهند. sunView.y = 50 واقعا به این معنی است:

```
sunView.translationY = 50 - sunView.top
```

استفاده از interpolatorهای مختلف

انیمیشن ما، اگرچه زیبا است، اما ناگهانی است. خورشید باید کم کم شتاب بگیرد. برای اضافه کردن این حس شتاب، تنها کاری که باید انجام دهیم این است که از TimeInterpolator استفاده کنیم. TimeInterpolator یک نقش دارد: تغییر نحوه حرکت انیمیشن شما از نقطه A به نقطه B.

از یک AccelerateInterpolator در ()startAnimation استفاده کنید تا سرعت خورشید خود را در ابتدا کمی‌افزایش دهید.

لیست ۳۰.۸ افزودن شتاب (MainActivity.kt)

```
private fun startAnimation() {
    val sunYStart = sunView.top.toFloat()
    val sunYEnd = skyView.height.toFloat()
    val heightAnimator = ObjectAnimator
        .ofFloat(sunView, "y", sunYStart, sunYEnd)
        .setDuration(3000)
    heightAnimator.interpolator = AccelerateInterpolator()
    heightAnimator.start()
}
```

Sunset را یک بار دیگر اجرا کنید و برای دیدن انیمیشن جایی از صفحه را لمس نمایید. اکنون خورشید باید به آرامی‌شروع به حرکت کند و با حرکت به سمت افق بیشتر شتاب بگیرد.

استایل‌های حرکتی زیادی وجود دارد که ممکن است بخواهیم در برنامه خود از آنها استفاده کنیم، بنابراین TimeInterpolators مختلف زیادی وجود دارد. به مستندات مرجع TimeInterpolator نگاهی بیاندازید.

evaluation رنگ

حالا که خورشید در حال حرکت است، بیایید آسمان را به رنگی متناسب با غروب خورشید متحرک کنیم. رنگ‌هایی را که در colors.xml تعریف کرده‌ایم با استفاده از یک lazy delegate به ویژگی‌ها بکشید.

لیست ۳۰.۹ بیرون کشیدن رنگ غروب آفتاب (MainActivity.kt)

```
class MainActivity : AppCompatActivity() {
    …
    private lateinit var skyView: View
    private val blueSkyColor: Int by lazy {
        ContextCompat.getColor(this, R.color.blue_sky)
    }
    private val sunsetSkyColor: Int by lazy {
        ContextCompat.getColor(this, R.color.sunset_sky)
    }
    private val nightSkyColor: Int by lazy {
        ContextCompat.getColor(this, R.color.night_sky)
    }
    …
}
```

حالا یک انیمیشن به startAnimation() اضافه کنید تا آسمان را از blueSkyColor به sunsetSkyColor ببرد.

لیست ۳۰.۱۰ متحرک سازی رنگ‌های آسمان (MainActivity.kt)

```
private fun startAnimation() {
    val sunYStart = sunView.top.toFloat()
    val sunYEnd = skyView.height.toFloat()
    val heightAnimator = ObjectAnimator
        .ofFloat(sunView, "y", sunYStart, sunYEnd)
        .setDuration(3000)
    heightAnimator.interpolator = AccelerateInterpolator()
    val sunsetSkyAnimator = ObjectAnimator
        .ofInt(skyView, "backgroundColor", blueSkyColor,
            sunsetSkyColor)
        .setDuration(3000)
    heightAnimator.start()
    sunsetSkyAnimator.start()
}
```

به نظر می‌رسد که در مسیر درستی حرکت می‌کند، اما اگر آن را اجرا کنید، خواهید دید که چیزی اشتباه است. به جای حرکت آرام از آبی به نارنجی، رنگ‌ها به شدت تغییر می‌کنند.

دلیل این اتفاق این است که یک عدد صحیح رنگ یک عدد ساده نیست. متشکل از چهار عدد است که در یک Int با هم جمع شده‌اند. بنابراین برای اینکه ObjectAnimator به درستی ارزیابی کند که کدام رنگ در نیمه راه بین آبی و نارنجی قرار دارد، باید بداند که چگونه کار می‌کند.

هنگامی که درک عادی ObjectAnimator از نحوه یافتن مقادیر بین شروع و پایان کافی نیست، می‌توانید یک زیر کلاس از TypeEvaluator برای رفع چیزها ارائه دهید. TypeEvaluator شیئی است که به ObjectAnimator می‌گوید مثلا یک چهارم فاصله بین مقدار شروع و مقدار پایانی چقدر است. اندروید یک زیر کلاس از TypeEvaluator به نام ArgbEvaluator ارائه می‌دهد که این کار را در اینجا انجام می‌دهد.

لیست ۳۰.۱۱ ارائه ArgbEvaluator (MainActivity.kt)

```
private fun startAnimation() {
    val sunYStart = sunView.top.toFloat()
    val sunYEnd = skyView.height.toFloat()
    val heightAnimator = ObjectAnimator
        .ofFloat(sunView, "y", sunYStart, sunYEnd)
        .setDuration(3000)
    heightAnimator.interpolator = AccelerateInterpolator()
    val sunsetSkyAnimator = ObjectAnimator
        .ofInt(skyView, "backgroundColor", blueSkyColor
            , sunsetSkyColor)
        .setDuration(3000)
    sunsetSkyAnimator.setEvaluator(ArgbEvaluator())
    heightAnimator.start()
    sunsetSkyAnimator.start()
}
```

(چندین نسخه از ArgbEvaluator وجود دارد؛ نسخه android.animation را ایمپورت کنید.)

انیمیشن خود را یک بار دیگر اجرا کنید و خواهید دید که آسمان به رنگ نارنجی زیبا محو می‌شود (تصویر ۳۰.۳).

تصویر ۳۰.۳ تغییر رنگ غروب خورشید



بازی انیماتورها با هم

اگر تنها کاری که باید انجام دهیم این است که چند انیمیشن را به طور همزمان شروع کنیم، کار ساده است: start() را به طور همزمان فراخوانی می‌کنیم. همه آنها به صورت هماهنگ با یکدیگر متحرک می‌شوند.

برای طراحی انیمیشن پیچیده‌تر، این کار جوابگو نیست. به عنوان مثال، برای تکمیل تصویر غروب خورشید، خوب است که نشان دهیم آسمان پس از غروب خورشید از نارنجی به آبی نیمه شب تبدیل می‌شود.

این کار را می‌توان با استفاده از AnimatorListener انجام داد که به ما می‌گوید چه زمانی یک انیمیشن کامل می‌شود. بنابراین می‌توانیم شنونده‌ای بنویسیم که تا پایان اولین انیمیشن منتظر بماند، در آن زمان می‌توانیم دومین انیمیشن آسمان شب را شروع کنیم. اما این یک دردسر بزرگ است و به شنوندگان زیادی نیاز دارد. استفاده از AnimatorSet بسیار آسان‌تر است.

ابتدا انیمیشن آسمان شب را بسازید و کد شروع انیمیشن قدیمی‌خود را حذف کنید.

لیست ۳۰.۱۲ ساخت انیمیشن شب (MainActivity.kt)

```
private fun startAnimation() {
    val sunYStart = sunView.top.toFloat()
    val sunYEnd = skyView.height.toFloat()
    val heightAnimator = ObjectAnimator
        .ofFloat(sunView, "y", sunYStart, sunYEnd)
        .setDuration(3000)
    heightAnimator.interpolator = AccelerateInterpolator()
    val sunsetSkyAnimator = ObjectAnimator
        .ofInt(skyView, "backgroundColor", blueSkyColor
            , sunsetSkyColor)
        .setDuration(3000)
    sunsetSkyAnimator.setEvaluator(ArgbEvaluator())
    val nightSkyAnimator = ObjectAnimator
        .ofInt(skyView, "backgroundColor", sunsetSkyColor
            , nightSkyColor)
        .setDuration(1500)
    nightSkyAnimator.setEvaluator(ArgbEvaluator())
    heightAnimator.start()
    sunsetSkyAnimator.start()
}
```

و سپس یک AnimatorSet ساخته و اجرا کنید.

لیست ۳۰.۱۳ ساخت مجموعه انیماتور (MainActivity.kt)

```
private fun startAnimation() {
    …
    nightSkyAnimator.setEvaluator(ArgbEvaluator())
    val animatorSet = AnimatorSet()
    animatorSet.play(heightAnimator)
        .with(sunsetSkyAnimator)
        .before(nightSkyAnimator)
    animatorSet.start()
}
```

AnimatorSet چیزی بیش از مجموعه‌ای از انیمیشن‌ها نیست که می‌توان آنها را با هم پلی کرد. چند راه برای ساخت آن وجود دارد، اما ساده ترین راه استفاده از عملکرد play(Animator) است که در بالا استفاده کردیم.

هنگامی که play (Animator) را فراخوانی می‌کنیم، یک AnimatorSet.Builder دریافت می‌کنیم که به ما امکان می‌دهد زنجیره‌ای از دستورالعمل‌ها بسازیم. Animator ارسالی به play(Animator) موضوع (subject) زنجیره است. بنابراین زنجیره فراخوانی‌هایی که در اینجا نوشتیم را می‌توان به این صورت توصیف کرد: "heightAnimator را با sunsetSkyAnimator پلی کن. همچنین، قبل از nightSkyAnimator، heightAnimator را پلی کن". برای AnimatorSetهای پیچیده، ممکن است لازم باشد چند بار play(Animator) را فراخوانی کنیم.

برنامه خود را یک بار دیگر اجرا کنید و از غروب آرامش بخشی که ایجاد کرده‌اید لذت ببرید.

چالش

برای چالش اول، قابلیت معکوس کردن غروب خورشید را پس از اتمام آن اضافه کنید. برای انجام این کار باید AnimatorSet دیگری بسازید – AnimatorSetها را نمی‌توان به صورت معکوس اجرا کرد.

برای چالش دوم، یک انیمیشن مداوم به خورشید اضافه کنید. آن را با گرما تپش دهید یا به آن‌هاله‌ای از پرتوها بدهید. (می‌توانید از متد setRepeatCount(Int) را در ObjectAnimator قرار دهید تا انیمیشن شما تکرار شود.)

چالش خوب دیگر این است که انعکاس خورشید در آب داشته باشید.

چالش نهایی شما این است که توانایی لمس را برای معکوس کردن صحنه غروب خورشید اضافه کنید.

۳۱

ورود به دنیای مدرن Android با Jetpack Compose

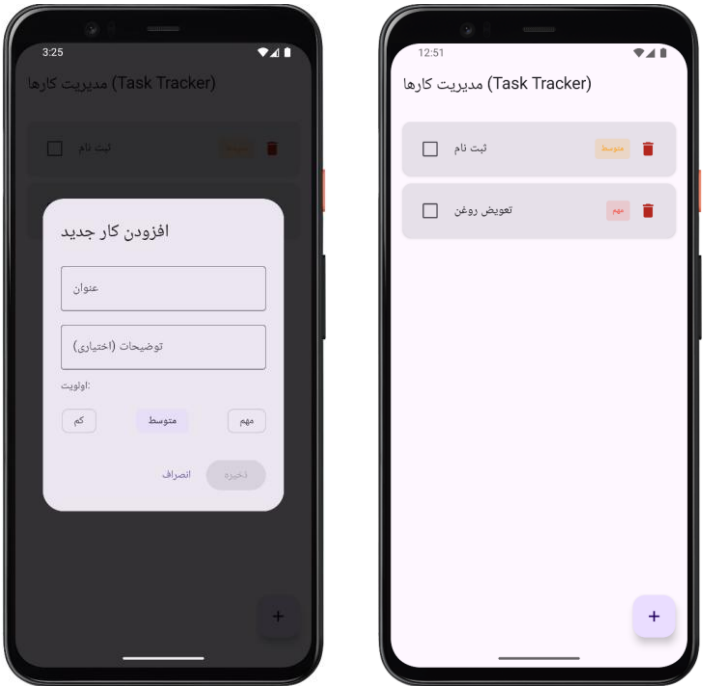
تا پیش از این فصل، رابط کاربری اندروید را با سیستم ویو سنتی ساختیم. فایل‌های XML برای چیدمان و کلاس‌هایی مانند Activity یا Fragment برای متصل کردن منطق برنامه به آن ویوها از طریق فراخوانی توابعی مانند findViewById اتفاق می‌افتاد. این رویکرد از ابتدای اندروید وجود داشته و همچنان در بسیاری از پروژه‌های موجود به‌کار می‌رود.

از سال ۲۰۲۱، گوگل Jetpack Compose را به‌عنوان تووِل‌کیت رسمی و توصیه شده برای ساخت UI معرفی کرد. Compose جایگزین XML و سیستم ویو نیست به این معنا که آن‌ها را غیرقابل استفاده کند؛ هر دو می‌توانند در کنار هم و حتی در یک پروژه‌ی واحد وجود داشته باشند. اما از نظر معماری، Compose یک تغییر پارادایم از UI دستوری (Imperative UI) به UI اعلانی (Declarative UI) است.

Jetpack Compose فریم‌ورک مدرن و رسمی گوگل برای ساخت رابط کاربری (UI) بومی در اندروید است. این فریم‌ورک با حذف فایل‌های XML و کدهای تکراری، فرآیند طراحی و پیاده‌سازی UI را کاملاً بر پایه زبان کاتلین قرار داده است.

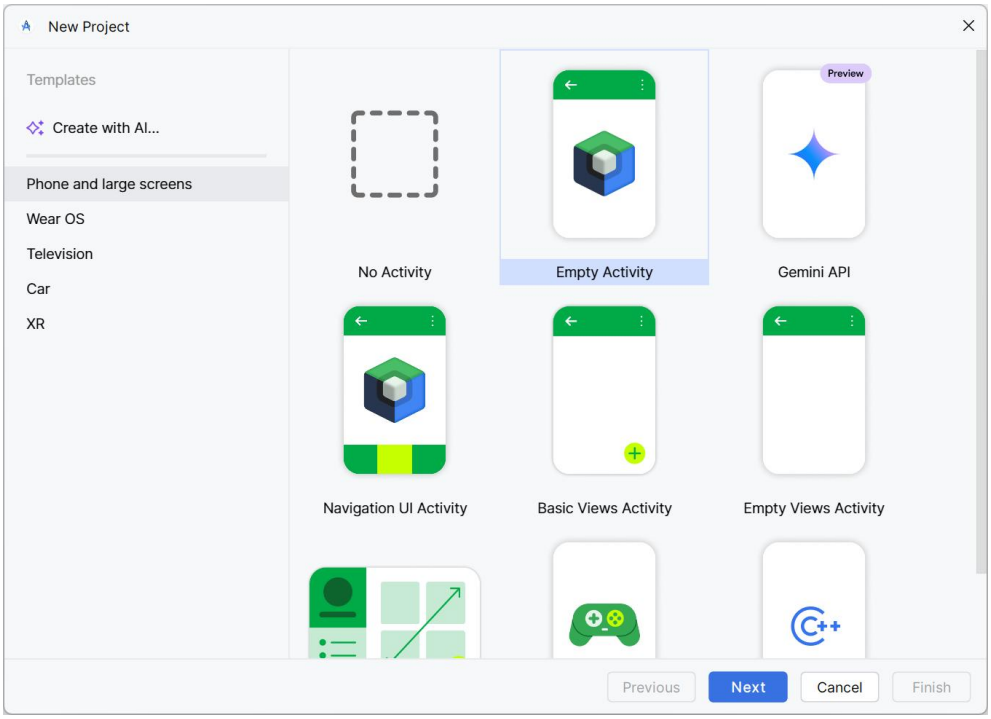
هدف این فصل، آموزش دقیق مبانی Compose در قالب یک پروژه‌ی کامل و کوچک است: اپلیکیشن «لیست وظایف». برنامه Task Tracker (مدیریت کارهای روزانه) یک ابزار ساده اما کاربردی برای مدیریت زمان و سازمان‌دهی کارهای شخصی یا شغلی است. هدف اصلی این برنامه، تبدیل ایده‌ها و مسئولیت‌های پراکنده ذهنی به یک لیست شفاف و قابل پیگیری است تا کاربر بتواند با تمرکز بالا کارهای روزمره خود را انجام دهد.

شکل ۳۱.۱ تصویر نهایی برنامه لیست وظایف



ساخت پروژه

در اندروید New Project را بزنید و در لیست تمپلیت‌ها، Empty Activity را انتخاب کنید. این تمپلیت از نسخه‌های اخیر اندروید استودیو به طور پیش‌فرض بر پایه‌ی Compose است؛ برای نام پروژه عنوان TaskTracker را در نظر بگیرید. روی دکمه Finish کلیک کنید و منتظر بمانید تا گردل (Gradle) پروژه را Sync کند.



بعد ساخته شدن پروژه، MainActivity شما باید چیزی شبیه به لیست پایین باشد.

```
package com.codeb.tasktracker

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.activity.enableEdgeToEdge
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.padding
import androidx.compose.material3.Scaffold
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
```

```
import androidx.compose.ui.Modifier
import androidx.compose.ui.tooling.preview.Preview
import com.codeb.tasktracker.ui.theme.TaskTrackerTheme

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            TaskTrackerTheme {
                Scaffold(modifier = Modifier.fillMaxSize())
                { innerPadding ->
                    Greeting(
                        name = "Android",
                        modifier = Modifier.padding(
                        innerPadding)
                    )
                }
            }
        }
    }
}

@Composable
fun Greeting(name: String, modifier: Modifier = Modifier) {
    Text(
        text = "Hello $name!",
        modifier = modifier
    )
}

@Preview(showBackground = true)
@Composable
fun GreetingPreview() {
    TaskTrackerTheme {
        Greeting("Android")
    }
}
```

بسیاری از کدهای بالا پیش‌فرض است. برای ادامه کار و ساخت ایده خودمان لازم است این فایل را تغییر داده و مقادیر پیش فرض را حذف کنیم. بنابراین تغییرات زیر را اعمال نمایید.

لیست ۳۱.۱ حذف توابع پیش‌فرض (MainActivity.kt)

```
package com.codeb.tasktracker

import android.os.Bundle
...
```

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            TaskTrackerTheme {
                Scaffold(modifier = Modifier.fillMaxSize()) {
                innerPadding ->
                    Greeting(
                        name = "Android",
                        modifier = Modifier.padding(
                        innerPadding)
                    )
                }
            }
        }
    }
}

@Composable
fun Greeting(name: String, modifier: Modifier = Modifier) {
    Text(
        text = "Hello $name!",
        modifier = modifier
    )
}

@Preview(showBackground = true)
@Composable
fun GreetingPreview() {
    TaskTrackerTheme {
        Greeting("Android")
    }
}
```

افزودن وابستگی‌ها

قدم بعدی ما افزودن وابستگی‌هاست. ما باید وابستگی ROOM، ویومدل، متریال، فعال سازی KSP و Hilt را انجام دهیم. این وابستگی‌ها و تنظیمات مربوط به آن ها را در سه فایل اعمال خواهیم کرد. ابتدا فایل lib.version.toml را باز کرده و تغییرات زیر را انجام دهید.

لیست ۳۱.۲ افزودن وابستگی‌ها (lib.version.toml)

```
[versions]
...
ksp = "2.3.6"
hilt = "2.60.1"
[libraries]
...
hilt-android = { group = "com.google.dagger", name = "hilt-
android", version.ref = "hilt" }
hilt-compiler = { group = "com.google.dagger", name = "hilt-
compiler", version.ref = "hilt" }
androidx-hilt-navigation-compose = { group = "androidx.hilt",
name = "hilt-navigation-compose", version = "1.2.0" }


[plugins]
…
ksp = { id = "com.google.devtools.ksp", version.ref = "ksp" }
hilt-android = { id = "com.google.dagger.hilt.android", ver-
sion.ref = "hilt" }
```

در قدم دوم فایل build.gradle.kts(Project:TaskTracker) را تغییر می دهیم.

لیست ۳۱.۳ افزودن وابستگی‌ها (build.gradle.kts(Project:TaskTracker))

```
plugins {
    alias(libs.plugins.android.application) apply false
    alias(libs.plugins.kotlin.compose) apply false
    alias(libs.plugins.ksp) apply false
    alias(libs.plugins.hilt.android) apply false
}
```

حالا در قدم نهایی نوبت فایل build.gradle.kts(Module:app) است. تغییرات زیر را اعمال کنید.

لیست ۳۱.۴ افزودن وابستگی‌ها (build.gradle.kts(Module:app))

```
plugins {
    ...
    alias(libs.plugins.ksp)
    alias(libs.plugins.hilt.android)
}

...

dependencies {
    ...
    implementation(libs.androidx.lifecycle.runtime.ktx)
```

```
    implementation(libs.hilt.android)
    ksp(libs.hilt.compiler)
    implementation(libs.androidx.hilt.navigation.compose)
    implementation ("androidx.room:room-runtime:2.8.4")
    ksp("androidx.room:room-compiler:2.8.4")
    implementation(
        "androidx.compose.material:material-icons-extended")
    ...
}
```

پروژه را sync کنید، نباید خطایی داشته باشید.

لایه داده

قبل از اینکه به سراغ UI بریم، باید مشخص کنیم یک کار یا Task در دنیای برنامه ما چه ویژگی‌هایی دارد. به سراغ لایه Data و ساخت Entity جدول کارها در دیتابیس Room می‌رویم. پکیج جدید ایجاد و آن را data نام گذاری و فایل TaskEntity را مطابق لیست ۳۱.۵ ایجاد نمایید.

لیست ۳۱.۵ ایجاد مدل داده (data/TaskEntity.kt)

```
package com.codeb.tasktracker.data

import androidx.room.Entity
import androidx.room.PrimaryKey

@Entity(tableName = "tasks")
data class TaskEntity(
    @PrimaryKey(autoGenerate = true)
    val id: Long = 0,
    val title: String,
    val description: String = "",
    val isCompleted: Boolean = false,
    val priority: Priority = Priority.MEDIUM,
    val createdAt: Long = System.currentTimeMillis()
)

enum class Priority {
    LOW,
    MEDIUM,
    HIGH
}
```

این کلاس یک جدول با نام tasks در دیتابیس SQLite تبدیل می‌کند و Enum اولویت هر کار را (کم، متوسط، زیاد) مشخص می‌کند.

حالا سراغ فایل DAO (Data Access Object) می‌رویم که دستورات SQL و عملیات اصلی دیتابیس (خواندن، اضافه کردن، ویرایش و حذف) را مشخص می‌کند. فایل TaskDao.kt را در پکیج data ایجاد و آن را همانند لیست ۳۱.۶ به روز کنید.

لیست ۳۱.۵ ایجاد دائو (data/TaskEntity.kt)

```
package com.codeb.tasktracker.data

import androidx.room.Dao
import androidx.room.Delete
import androidx.room.Insert
import androidx.room.OnConflictStrategy
import androidx.room.Query
import androidx.room.Update
import kotlinx.coroutines.flow.Flow

@Dao
interface TaskDao {
    @Query("SELECT * FROM tasks ORDER BY createdAt DESC")
    fun getAllTasks(): Flow<List<TaskEntity>>

    @Query("SELECT * FROM tasks WHERE id = :taskId")
    suspend fun getTaskById(taskId: Long): TaskEntity?

    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insertTask(task: TaskEntity)

    @Update
    suspend fun updateTask(task: TaskEntity)

    @Delete
    suspend fun deleteTask(task: TaskEntity)

    @Query("DELETE FROM tasks WHERE isCompleted = 1")
    suspend fun deleteCompletedTasks()
}
```

@Dao به Room اعلام می‌کند این یک رابط عملیاتی برای دیتابیس است. همچنین استفاده از Flow باعث می‌شود هر زمان تغییری در دیتابیس رخ داد، UI به صورت خودکار و زنده بروزرسانی شود.

حالا سراغ فایل AppDatabase.kt می‌رویم که ساختار اصلی دیتابیس Room را پیکربندی می‌کند. آن را در data ایجاد و همانند لیست ۳۱.۶ به روز کنید.

لیست ۳۱.۶ قلب اصلی دیتابیس (data/AppDatabase.kt)

```
package com.codeb.tasktracker.data

import androidx.room.Database
import androidx.room.RoomDatabase

@Database(
    entities = [TaskEntity::class],
    version = 1,
    exportSchema = false
)
abstract class AppDatabase : RoomDatabase() {

    abstract val taskDao: TaskDao
}
```

فایل AppDatabase.kt قلب اصلی دیتابیس Room است و به عنوان پل ارتباطی بین کدهای برنامه شما و دیتابیس SQLite عمل می‌کند. وظایف و دلایل وجود این فایل عبارتند از:

- معرفی جدول‌ها (Entities): با مشخص کردن کلاسی مثل TaskEntity در این فایل، به Room می‌گویید که چه جدول‌هایی باید درون دیتابیس ساخته شوند.
- تعریف نسخه دیتابیس (Version): شماره نسخه دیتابیس (version = 1) اینجا تعیین می‌شود. اگر در آینده کلاسی را تغییر دهید یا ستون جدیدی اضافه کنید، با تغییر این عدد و نوشتن Migration، ساختار دیتابیس بدون پاک شدن داده‌های کاربر بروزرسانی می‌شود.
- دسترسی به DAOها: این کلاس متدهای دسترسی به DAO (مثل TaskDao) را ارائه می‌دهد تا برنامه بتواند دستورات SELECT،INSERT یا DELETE را روی دیتابیس اجرا کند.
- مدیریت کدهای ساخت دیتابیس (Boilerplate): شما این کلاس را به صورت abstract تعریف می‌کنید و کتابخانه Room در زمان Compile، کدهای پیچیده SQLite و باز/بسته کردن دیتابیس را به صورت خودکار زیر پوشش این کلاس می‌سازد.

حالا سراغ لایه Repository می‌رویم. این لایه یک ساختار تمیز بین دیتابیس (Room) و لایه UI/ViewModel ایجاد می‌کند تا ViewModel مستقیم به دیتابیس متصل نباشد. ابتدا یک Interface تعریف می‌کنیم و سپس پیاده‌سازی آن را می‌نویسیم.

فایل TaskRepository.kt را ایجاد و آن را همانند لیست ۳۶.۷ به روز کنید.

لیست ۳۱.۷ Repository (data/TaskRepository.kt)

```
package com.codeb.tasktracker.data
import kotlinx.coroutines.flow.Flow

interface TaskRepository {
    fun getAllTasks(): Flow<List<TaskEntity>>
    suspend fun getTaskById(id: Long): TaskEntity?
    suspend fun insertTask(task: TaskEntity)
    suspend fun updateTask(task: TaskEntity)
    suspend fun deleteTask(task: TaskEntity)
    suspend fun deleteCompletedTasks()
}

class TaskRepositoryImpl(
    private val dao: TaskDao
) : TaskRepository {

    override fun getAllTasks(): Flow<List<TaskEntity>> {
        return dao.getAllTasks()
    }

    override suspend fun getTaskById(id: Long): TaskEntity? {
        return dao.getTaskById(id)
    }

    override suspend fun insertTask(task: TaskEntity) {
        dao.insertTask(task)
    }

    override suspend fun updateTask(task: TaskEntity) {
        dao.updateTask(task)
    }

    override suspend fun deleteTask(task: TaskEntity) {
        dao.deleteTask(task)
    }

    override suspend fun deleteCompletedTasks() {
        dao.deleteCompletedTasks()
    }
}
```

تزریق وابستگی‌ها (Dependency Injection)

تزریق وابستگی (DI) یک الگوی طراحی نرم‌افزار (Design Pattern) است که بر پایه اصل معکوس‌سازی کنترل (Inversion of Control - IoC) قرار دارد. در این الگو، فرایند ساخت و مدیریت اشیاء (Dependencies) از داخل کلاسی که به آن‌ها نیاز دارد گرفته شده و به یک بخش بیرونی (مانند یک فریم‌ورک یا کانتینر تزریق وابستگی) واگذار می‌شود.

این الگو به طور مستقیم پنجمین اصل از اصول SOLID، یعنی اصل معکوس‌سازی وابستگی (Dependency Inversion Principle) را پیاده‌سازی می‌کند؛ به این معنا که کلاس‌های سطح بالا نباید به کلاس‌های سطح پایین وابسته باشند، بلکه هر دو باید به انتزاع (Interface یا Abstract Class) وابسته باشند.

ارکان اصلی الگو در ساختار شیءگرا:

- سرویس (Service / Dependency): شیء یا کلاسی که خدمات خاصی را ارائه می‌دهد (مانند TaskRepository یا AppDatabase).
- کلاینت (Client): کلاسی که برای انجام وظایف خود به سرویس نیاز دارد (مانند TaskViewModel).
- تزریق‌کننده (Injector / Container): بخش بیرونی یا فریم‌ورکی (مانند Hilt یا Dagger) که شیء سرویس را نمونه‌سازی (Instantiate) کرده و آن را به کلاینت تحویل می‌دهد.

انواع روش‌های تزریق وابستگی:

- تزریق از طریق سازنده (Constructor Injection): وابستگی‌ها مستقیماً از طریق ورودی‌های متد سازنده (Constructor) به کلاس پاس داده می‌شوند (استانداردترین روش).
- تزریق از طریق ویژگی/فیلد (Field Injection): وابستگی‌ها بدون استفاده از سازنده و مستقیماً روی متغیرهای کلاس ست می‌شوند.
- تزریق از طریق متد (Setter / Method Injection): وابستگی‌ها از طریق متدهای تنظیم‌کننده (Setter) به کلاس تزریق می‌گردند.

اهداف و مزایای معماری:

- کاهش وابستگی شدید (Loose Coupling): کلاس‌ها مستقلاً توسعه می‌یابند و تغییر در یک کلاس باعث شکست کدهای سایر بخش‌ها نمی‌شود.
- تست‌پذیری بالا (Testability): امکان جایگزینی اشیاء واقعی با اشیاء جعلی (Mock/Fake) جهت اجرای تست‌های واحد (Unit Testing) فراهم می‌شود.
- ارتقای قابلیت نگهداری (Maintainability): مدیریت چرخه حیات (Lifecycle) اشیاء یکپارچه شده و کدهای تکراری ساخت اشیاء (Boilerplate Code) حذف می‌شوند.

برای درک بهتر تصور کنید می‌خواهید در خانه‌تان پیتزا بخورید. برای این کار دو راه دارید:

۱. روش بدون تزریق وابستگی: خودتان مواد اولیه (آرد، گوجه، پنیر) را می‌خرید، فر را روشن می‌کنید و پیتزا را از صفر می‌پزید. مشکل چیست؟ شما فقط می‌خواستید پیتزا بخورید، اما حالا مجبورید آشپزی یاد بگیرید، فر بخرید و ظرف بشویید! اگر فر خراب شود یا دستور پخت تغییر کند، کلاً به مشکل می‌خورید.

۲. روش با تزریق وابستگی – DI: پیتزا را به یک پیک سفارش می‌دهید و پیک پیتزای آماده را دم در به شما تحویل می‌دهد (تزریق می‌کند).

مزیت چیست؟ برای شما مهم نیست پیتزا چطور و در چه فری پخته شده؛ شما فقط پیتزا را تحویل می‌گیرید و مصرف می‌کنید!

همین مفهوم در دنیای برنامه‌نویسی به این صورت است که در برنامه، وقتی کلاس A برای انجام کارش به کلاس B نیاز دارد:

۱. روش سنتی (بدون DI): کلاس A خودش داخل کدهایش با دستور new کلاس B را می‌سازد. مشکل: کلاس‌ها شدیداً به هم قفل می‌شوند. اگر فردا بخواهید نحوه ساخت B را تغییر دهید، باید کدهای کلاس A را هم دستکاری کنید.

۲. روش تزریق وابستگی (با DI): کلاس A می‌گوید: «من به B نیاز دارم، اما کاری به ساختنش ندارم.» سپس یک ابزار بیرونی (مثل Hilt) کلاس B را می‌سازد و آماده به کلاس A تحویل (Inject) می‌دهد.

با این توضیحات سراغ فایل بعدی یعنی ماژول Hilt برای تزریق وابستگی‌ها (Dependency Injection) می‌رویم. در یک جمله Hilt چارچوب رسمی گوگل برای Dependency Injection در Android است

که ساخت، مدیریت و اشتراک‌گذاری وابستگی‌ها را به‌صورت خودکار و بهینه انجام می‌دهد. Hilt باعث می‌شود کلاس‌ها خودشان وابستگی نسازند، بلکه آن‌ها را از یک سیستم هوشمند دریافت کنند.

در این فایل، نحوه ساخته شدن AppDatabase، TaskDao و TaskRepository را به سیستم Hilt معرفی می‌کنیم تا بتواند آن‌ها را به صورت خودکار در ViewModel تزریق کند. AppModule.kt را در پکیج جدیدی به نام di ایجاد و همانند لیست ۳۱.۷ به روز کنید.

لیست ۳۱.۷ تزریق وابستگی‌ها با AppModule (data/AppModule.kt)

```
package com.codeb.tasktracker.di

import android.content.Context
import androidx.room.Room
import com.codeb.tasktracker.data.AppDatabase
import com.codeb.tasktracker.data.TaskDao
import com.codeb.tasktracker.data.TaskRepository
import dagger.Module
import dagger.Provides
import dagger.hilt.InstallIn
import dagger.hilt.android.qualifiers.ApplicationContext
import dagger.hilt.components.SingletonComponent
import javax.inject.Singleton

@Module
@InstallIn(SingletonComponent::class)
object AppModule {

    @Provides
    @Singleton
    fun provideAppDatabase(
        @ApplicationContext context: Context
    ): AppDatabase {
        return Room.databaseBuilder(
            context,
            AppDatabase::class.java,
            "task_tracker_db"
        ).build()
    }

    @Provides
    @Singleton
    fun provideTaskDao(db: AppDatabase): TaskDao {
        return db.taskDao
    }

    @Provides
```

```
    @Singleton
    fun provideTaskRepository(dao: TaskDao): TaskRepository {
        return TaskRepositoryImpl(dao)
    }
}
```

لایه ViewModel

برای داشتن یک معماری تمیز و یک‌طرفه (Unidirectional Data Flow)، رویدادها (Events) و حالت‌های UI (State) را کنار TaskViewModel در یک فایل قرار می‌دهیم.

فایل TaskViewModel.kt را در پکیج viewmodel داخل پکیج ui ساخته و همانند لیست ۳۱.۸ به روز کنید.

لیست ۳۱.۸ ویومدل (ui/viewmodel/TaskViewModel.kt)

```
package com.codeb.tasktracker.ui
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.setValue
import androidx.lifecycle.ViewModel
import androidx.lifecycle.viewModelScope
import com.codeb.tasktracker.data.Priority
import com.codeb.tasktracker.data.TaskEntity
import com.codeb.tasktracker.data.TaskRepository
import dagger.hilt.android.lifecycle.HiltViewModel
import kotlinx.coroutines.flow.SharingStarted
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.stateIn
import kotlinx.coroutines.launch
import javax.inject.Inject

sealed interface TaskEvent {
    data class OnTitleChange(val title: String) : TaskEvent
    data class OnDescriptionChange(val description: String) :
TaskEvent
    data class OnPriorityChange(val priority: Priority) :
TaskEvent
    object OnAddTask : TaskEvent
    data class OnToggleComplete(val task: TaskEntity) :
TaskEvent
    data class OnDeleteTask(val task: TaskEntity) : TaskEvent
}
```

```
@HiltViewModel
class TaskViewModel @Inject constructor(
    private val repository: TaskRepository
) : ViewModel() {
    val tasks: StateFlow<List<TaskEntity>> = repository.get-
AllTasks()
        .stateIn(
            scope = viewModelScope,
            started = SharingStarted.WhileSubscribed(5000),
            initialValue = emptyList()
        )

    var taskTitle by mutableStateOf("")
        private set

    var taskDescription by mutableStateOf("")
        private set

    var taskPriority by mutableStateOf(Priority.MEDIUM)
        private set

    fun onEvent(event: TaskEvent) {
        when (event) {
            is TaskEvent.OnTitleChange -> taskTitle =
event.title
            is TaskEvent.OnDescriptionChange -> taskDescrip-
tion = event.description
            is TaskEvent.OnPriorityChange -> taskPriority =
event.priority
            is TaskEvent.OnAddTask -> {
                if (taskTitle.isBlank()) return
                viewModelScope.launch {
                    repository.insertTask(
                        TaskEntity(
                            title = taskTitle.trim(),
                            description = taskDescrip-
tion.trim(),
                            priority = taskPriority
                        )
                    )
                    taskTitle = ""
                    taskDescription = ""
                    taskPriority = Priority.MEDIUM
                }
            }
            is TaskEvent.OnToggleComplete -> {
```

```
                viewModelScope.launch {
                    repository.updateTask(
                        event.task.copy(isCompleted =
!event.task.isCompleted)
                    )
                }
            }
            is TaskEvent.OnDeleteTask -> {
                viewModelScope.launch {
                    repository.deleteTask(event.task)
                }
            }
        }
    }
}
```

توضیحات این فایل:

TaskEvent: اکشن‌های کاربر (مثل نوشتن عنوان، تغییر اولویت، زدن تیک انجام کار، حذف یا ذخیره) را به کدهای تمیز و قابل مدیریت تبدیل می‌کند.

stateIn: جریان داده Flow دیتابیس را به یک StateFlow تبدیل می‌کند تا با تغییرات چرخه حیات UI قطع و وصل نشود و حافظه هدر نروید.

@HiltViewModel و @Inject: باعث می‌شود Hilt خودش TaskRepository را به این ViewModel تزریق کند.

قبل از اینکه سراغ کدهای صفحه اصلی (UI) برویم، باید سیستم Hilt را در سطح کل برنامه فعال کنیم تا دیتابیس و ViewModel به درستی initialize شوند.

فایل TaskApplication.kt را ساخته و همانند لیست ۳۱.۹ به روز کنید.

لیست ۳۱.۹ Hilt را در سطح کل برنامه فعال کنیم (TaskApplication.kt)

```
package com.codeb.tasktracker

import android.app.Application
import dagger.hilt.android.HiltAndroidApp

@HiltAndroidApp
class TaskApplication : Application()
```

باید این کلاس را به فایل AndroidManifest.xml معرفی کنید:

لیست ۳۱.۱۰ مانیفست (AndroidManifest.xml)

```
<application
    android:name=".TaskApplication"
    ...>
</application>
```

لایه نمایش (UI) با Jetpack Compose و Material 3

در فایل TaskTrackerScreen.kt تمام اجزای UI شامل لیست کارها، کارت‌های اختصاصی (Task Item)، بج اولویت (Priority Badge) و دیالوگ افزودن کار جدید را پیاده‌سازی می‌کنیم. فایل را در پکیج screens در داخل پکیج ui بسازید. اولین کار ما در این فایل ایجاد یک تابع compose است پس کد این فایل را همانند لیست زیر به روز کنید.

لیست ۳۱.۱۱ صفحه برنامه (TaskTrackerScreen.kt)

```
package com.codeb.tasktracker.ui.screens

import androidx.compose.material3.Text
import androidx.compose.runtime.Composable

class TaskTrackerScreen {}

@Composable
fun TaskTrackerScreen() {
    Text(
        text = "Hello TaskTracker!"
    )
}
```

حالا برای اینکه خروجی را ببینیم باید MainActivity.kt را تغییر دهیم. تغییرات زیر را در MainActivity اعمال کنید تا بعد از دیدن نتیجه به توضیح کدها بپردازیم.

```
package com.codeb.tasktracker

...
import com.codeb.tasktracker.ui.screens.TaskTrackerScreen
...
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            TaskTrackerTheme {
```

```
                TaskTrackerScreen()
            }
        }
    }
}
```

حالا برنامه را اجرا کنید. خروجی باید چاپ جمله !Hello TaskTracker روی صفحه خالی باشد.

اولین مفهوم مهم ما انوتیشن Composable@ است. در فایل TaskTrackerScreen.kt ما تابع TaskTrackerScreen() را با این انوتیشن تعریف کردیم. این انوتیشن می‌گوید "این تابع بخشی از رابط کاربری Compose را توصیف کند." و چون این تابع یک UI تولید می‌کند ما آن را در داخل setContent قرار دادیم تا آن را روی صفحه نمایش دهیم. ساده به نظر می‌رسد و واقعا هم همینطور است.

وقت آن است که پازل UI را کامل و کاملتر کنیم. قصد داریم TaskTrackerScreen را طوری تغییر دهیم که برنامه ما در هنگام اجرای دارای AppBar و عنوان باشد. همچنین فضایی را برای سایر عناصر در نظر خواهیم گرفت، پس تابع را همانند لیست زیر تغییر دهید.

```
package com.codeb.tasktracker.ui.screens

...
import androidx.compose.material3.ExperimentalMaterial3Api
import androidx.compose.material3.Scaffold
import androidx.compose.material3.Text
import androidx.compose.material3.TopAppBar
import androidx.compose.runtime.Composable
import androidx.compose.ui.Modifier
import androidx.compose.ui.unit.dp

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun TaskTrackerScreen() {
    Text(
        text = "Hello TaskTracker!"
    )
    Scaffold(
        topBar = {
            TopAppBar(
                title = { Text("Task Tracker") }
            )
        }
    ) { paddingValues ->
        Box(
            modifier = Modifier
```

```
                .fillMaxSize()
                .padding(paddingValues)
                .padding(16.dp)
        ) {
            //other items
        }
    }
}
```

حالا برنامه را دوباره اجاره کنید. تصویر زیر به نمایش در می‌آید.



در کد بالا ما عنصر Scaffold را تعریف کردیم. Scaffold را نقش داربست صفحه (Page Layout Framework) را دارد. یک چارچوب آماده برای ساخت صفحات استاندارد اپلیکیشن است. یعنی جای بخش‌های رایج UI را مشخص می‌کند. عناصری مانند TopAppBar (نوار بالا)، BottomBar (نوار پایین)، FloatingActionButton (دکمه شناور)، Drawer (منوی کناری)، Snackbar و محتوای اصلی صفحه به وسیله Scaffold مرتب می‌شوند. قطعه کد زیر ساختار کلی این مفهوم است.

```
Scaffold(
    topBar = { ... },
    bottomBar = { ... },
    floatingActionButton = { ... },
```

```
    drawerContent = { ... }
) { paddingValues ->
    // محتوای اصلی صفحه
}
```

به همین دلیل است که اسم این عنصر Scaffold = داربست / اسکلت‌بندی است.

paddingValues فاصله‌های لازم برای اینکه محتوای صفحه زیر TopAppBar یا BottomBar نرود را اعمال می‌کند. Scaffold خودش این مقدار را حساب می‌کند و به ما می‌دهد. عنصر داخل این قسمت یک Box است. Box یعنی یک ظرف (Container) که می‌توانیم عناصر دیگر را داخلش قرار بدهیم. هر چیزی که داخل { } قرار گیرد، داخل این Box نمایش داده می‌شود. عناصر متفاوت و دیگری برای ساخت UI در compose وجود دارد که با پیشروی در ساخت برنامه به آنها خواهیم رسید. اما یک انوتیشن جدید داشتیم: @OptIn(ExperimentalMaterial3Api::class). خیلی ساده این انوتیشن یعنی داریم از یک قابلیت آزمایشی در Material3 استفاده می‌کنیم. این انوتیشن برای استفاده از AppBar است.

اما مفهوم بعدی که باید به آن بپردازیم، مفهوم Modifier است. عنصر Box ما دارای یک Modifier است. Modifier یعنی این که چطور باید Box ما رفتار کند. چقدر بزرگ باشد؟ چقدر فاصله داشته باشد؟ چه ظاهری داشته باشد؟ و ... . Modifier مثل یک لیست دستور است که به Box می‌گوید چطور باشد.

fillMaxSize() یعنی این Box کل صفحه را پر کند. پس Modifier خصوصیات هر عنصر را مشخص می‌کند.

با این توضیحات UI را یک مرحله دیگر پیش می‌بریم. چه چیزی باید داخل باکس ما قرار گیرد. سناریو به این شکل است که اگر کاری برای انجام باشد، برنامه لیست کارها را از ویومدل دریافت کند و آنها را به نمایش بگذارد و در غیر این صورت پیامی مبنی بر اینکه هنوز کار یا وظیفه ای ایجاد نشده نمایش دهد. اما چون تعداد تسک‌های متفاوت در لیست خواهیم داشت ابتدا به تعریف ویو یک نمونه از تسک‌ها خواهیم پرداخت و با استفاده از عنصری شبیه به RecyclerView در کامپوز که معرفی خواهیم کرد، به صورت مکرر از آن استفاده خواهیم کرد. تصویر یک آیتم به صورت زیر است.

تصویر یک آیتم



با توجه به تصویر، ویو آیتم‌ها را با دو تابع پیاده خواهیم کرد:

۱. تابع TaskItem به عنوان کل

۲. تابع PriorityBadge به عنوان قسمت اولویت که با سه مقدار مهم، متوسط و کم نمایش می‌یابد.

اما قبل از نوشتن این دو تابع باید ویومدل را جهت گرفتن مقادیر به تابع اصلی اضافه نماییم. برای این کار کد تابع TaskTrackerScreen به صورت لیست زیر به روز کنید.

```
fun TaskTrackerScreen(
    viewModel: TaskViewModel = hiltViewModel()
) {
    val tasks by viewModel.tasks.collectAsState()
    Scaffold(
        topBar = {
            TopAppBar(
                title = { Text("Task Tracker") }
            )
        }
    ) { paddingValues ->
        Box(
            modifier = Modifier
                .fillMaxSize()
                .padding(paddingValues)
                .padding(16.dp)
        ) {
            //other Items
        }
    }
}
```

این دو خط دقیقاً چه کار می‌کند؟ تابع یک پارامتر به نام viewModel دارد، یعنی تابع TaskTrackerScreen انتظار دارد یک TaskViewModel دریافت کند. این مقدار را در MainActivity پاس خواهیم کرد.

خط دوم اضافه شده یعنی val tasks by viewModel.tasks.collectAsState() یکی از کلیدی‌ترین مفاهیم کامپوز است. یعنی از ViewModel یک جریان لیست کارها را بگیر، آن را تبدیل کن به State قابل استفاده در Compose و مقدارش را مستقیم در متغیر tasks قرار بده تا هر بار داده تغییر کرد، UI خودش آپدیت شود. برای فهم بیشتر باید مفهوم State را بیشتر توضیح دهیم. State یعنی مقدار قابل تغییر که UI به آن گوش می‌دهد. خیلی ساده اگر بگوییم State یک مقدار است که اگر تغییر کند، UI خودش آپدیت می‌شود. یعنی ما مقدار را عوض می‌کنیم، Compose خودش UI را دوباره می‌سازد (Recompose). ما هیچ کاری برای آپدیت UI نمی‌کنیم. در واقع این خط لیست کارها را برمی‌گرداند و اگر لیست کارها تغییر کند کامپوز خودش UI را تغییر می‌دهد.

با این توضیحات دو تابع به TaskTrackerScreen برای ساخت آیتم‌ها اضافه کنید.

```
@Composable
fun TaskItem(
    task: TaskEntity,
    onToggleComplete: () -> Unit,
    onDelete: () -> Unit
) {
    Card(
        modifier = Modifier.fillMaxWidth(),
        elevation = CardDefaults.cardElevation(
        defaultElevation = 2.dp)
    ) {
        Row(
            modifier = Modifier
                .fillMaxWidth()
                .padding(12.dp),
            verticalAlignment = Alignment.CenterVertically
        ) {
            Checkbox(
                checked = task.isCompleted,
                onCheckedChange = { onToggleComplete() }
            )
            Column(
                modifier = Modifier
                    .weight(1f)
                    .padding(horizontal = 8.dp)
            ) {
                Text(
```

```
                    text = task.title,
                    style = MaterialTheme.typography
                    .titleMedium.copy(
                        textDecoration = if (
                        task.isCompleted) TextDecoration
                        LineThrough else TextDecoration.None
                    ),
                    color = if (task.isCompleted) Color.Gray
                    else MaterialTheme.colorScheme.onSurface
                )
                if (task.description.isNotBlank()) {
                    Text(
                        text = task.description,
                        style = MaterialTheme
                        .typography.bodySmall,
                        color = Color.Gray
                    )
                }
            }
            PriorityBadge(priority = task.priority)
            IconButton(onClick = onDelete) {
                Icon(
                    imageVector = Icons.Default.Delete,
                    contentDescription = "کار حذف",
                    tint = MaterialTheme.colorScheme.error
                )
            }
        }
    }
}
@Composable
fun PriorityBadge(priority: Priority) {
    val backgroundColor = when (priority) {
        Priority.LOW -> Color(0xFF4CAF50)
        Priority.MEDIUM -> Color(0xFFFF9800)
        Priority.HIGH -> Color(0xFFF44336)
    }
    Box(
        modifier = Modifier
            .background(color = backgroundColor.copy(
             alpha = 0.15f)
                , shape = RoundedCornerShape(4.dp))
            .padding(horizontal = 8.dp, vertical = 4.dp)
    ) {
        Text(
            text = when (priority) {
                Priority.LOW -> "کم"
                Priority.MEDIUM -> "متوسط"
```

```
                Priority.HIGH -> "مهم"
            },
            style = MaterialTheme.typography.labelSmall,
            color = backgroundColor
        )
    }
}
```

هدف کلی این دو کامپوننت چیست؟ هدف TaskItem نمایش یک «آیتم کار» در لیست کارها است. این آیتم باید ظاهر استاندارد و کارت‌مانند داشته باشد (Card)، اطلاعات کار را نمایش دهد (+ title description)، وضعیت انجام‌شدن را نشان دهد (Checkbox)، اولویت را نشان دهد (PriorityBadge) و امکان حذف کردن کار را بدهد (IconButton).

تابع PriorityBadge یک نشان کوچک رنگی برای نمایش سطح اهمیت کار بوجود می‌آورد.

در توضیح بالا عنوان عناصر ویوها را نوشتیم. به عنوان مثال وضعیت انجام‌شدن کار را با Checkbox کردیم. حالا تک‌تک عناصر را با هدف و نقش رسمی‌شان توضیح می‌دهم. به کدها نگاهی بیاندازید و توضیحات زیر را دنبال کنید.

- Card: این عنصر یک سطح (Surface) با سایه و برجستگی از پس‌زمینه ایجاد می‌کند که به عنوان بدنه اصلی ویو آیتم به کار گرفته شده است.
- Row: این عنصر چیدمان افقی عناصر را برعهده دارد.Row برای قرار دادن عناصر در یک خط استفاده می‌شود. در کد ما Row چک‌باکس، متن‌ها، نشان اولویت و دکمه حذف را در یک خط کنار هم قرار می‌دهد.
- Checkbox: نمایش وضعیت انجام‌شدن کار را بر عهده دارد. Checkbox برای انتخاب دوحالته (On/Off) استفاده می‌شود. در کد ما وضعیت انجام کار با استفاده از مقدار بولی task.isCompleted بدست می‌آید.
- Column: این عنصر چیدمان عمودی عناصر دیگر را انجام می‌دهد. برای قرار دادن عناصر زیر هم استفاده می‌شود.در کد ما عنوان کار و توضیحات کار زیر هم قرار می‌گیرند.
- Text: برای نمایش اطلاعات کاربرد دارد.
- PriorityBadge: نمایش اهمیت کار با رنگ و متن را انجام می‌دهد. رنگ سبز ⟵ کم، نارنجی ⟵ متوسط و قرمز ⟵ مهم
- IconButton + Delete Icon برای اکشن سریع و واضح کاربرد دارند.

حالا که ویو آیتم آمادست بخش بعدی کار را انجام خواهیم داد. یعنی سراغ نمایش کارها در صورت وجود یا نمایش پیام مناسب در صورت عدم وجود خواهیم رفت. TaskTrackerScreen را به صورت لیست زیر به روز کنید.

```
fun TaskTrackerScreen(
    viewModel: TaskViewModel = hiltViewModel()
) {
    val tasks by viewModel.tasks.collectAsState()
    Scaffold(
        topBar = {
            TopAppBar(
                title = { Text("Task Tracker") }
            )
        }
    ) { paddingValues ->
        Box(
            modifier = Modifier
                .fillMaxSize()
                .padding(paddingValues)
                .padding(16.dp)
        ) {
            //other Items
            if (tasks.isEmpty()) {
                Text(
                    text = "هیچ کاری ثبت نشده است!",
                    modifier = Modifier.align(
                    Alignment.Center),
                    style = MaterialTheme.
                    typography.bodyLarge,
                    color = Color.Gray
                )
            } else {
                LazyColumn(
                    verticalArrangement =
                    Arrangement.spacedBy(10.dp),
                    modifier = Modifier.fillMaxSize()
                ) {
                    items(tasks, key = { it.id }) { task ->
                        TaskItem(
                            task = task,
                            onToggleComplete = {
                            viewModel.onEvent(
                            TaskEvent.OnToggleComplete(
                            task)) },
                            onDelete = { viewModel.
                            onEvent(TaskEvent.
```

```
                        OnDeleteTask(task)) }
                        )
                    }
                }
            }
        }
    }
}
```

در این بخش در If چک می‌کنیم که آیا داده‌ای برای نمایش وجود دارد یا خیر. در صورت وجود داده آن را با عنصر LazyColumn به صورتی که قبلا با RecyclerView نمایش می‌دادیم، نمایش می‌دهیم. در واقع این عنصر لیست استانداردی را بوجود می‌آورد و در صورت حذف عنصری رویداد را به ویومدل ارسال می‌کند.

اگر برنامه را اجرا کنیم خطایی دریافت خواهیم کرد، زیرا ویومدل را در MainActivity پاس نکردیم. پس باید آن را انجام دهیم. این کار از آن چیزی که فکر می‌کنید بسیار آسان تر است. MainActivity را مانند لیست زیر به روز کنید.

```
...
import dagger.hilt.android.AndroidEntryPoint

@AndroidEntryPoint
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        ...
    }
}
```

@AndroidEntryPoint یعنی چی؟ خیلی ساده یعنی این کلاس اجازه دارد وابستگی‌ها (Dependencyها) را از Hilt دریافت کند. یعنی اگر یک Activity، Fragment، ViewModel، یا Composable بخواهد چیزی را از Hilt بگیرد، باید با این Annotation علامت‌گذاری شود. با این کار همه چیز به‌صورت خودکار تزریق می‌شود و برنامه اجرا می‌شود.



همانطور که می‌بینید برنامه داده‌ای برای نمایش ندارد. وظیفه بعدی ما ایجاد دکمه‌ای برای نمایش یک دیالوگ و اضافه کردن داده است.

```
fun TaskTrackerScreen(
    viewModel: TaskViewModel = hiltViewModel()
) {
    val tasks by viewModel.tasks.collectAsState()
    var showAddDialog by remember { mutableStateOf(false) }
    Scaffold(
        topBar = {
            ...
        },
        floatingActionButton = {
            FloatingActionButton(
                onClick = { showAddDialog = true }
            ) {
                Icon(Icons.Default.Add,
                contentDescription = "کار افزودن")
            }
        }
    ) { paddingValues ->
```

```
        …
            if (showAddDialog) {
                AddTaskDialog(
                    title = viewModel.taskTitle,
                    description = viewModel.taskDescription,
                    priority = viewModel.taskPriority,
                    onTitleChange = { viewModel
                    .onEvent(TaskEvent.OnTitleChange(it)) },
                    onDescriptionChange = { viewModel
                    .onEvent(TaskEvent
                    .OnDescriptionChange(it)) },
                    onPriorityChange = { viewModel
                    .onEvent(TaskEvent
                    .OnPriorityChange(it)) },
                    onDismiss = { showAddDialog = false },
                    onConfirm = {
                        viewModel
                        .onEvent(TaskEvent.OnAddTask)
                        showAddDialog = false
                    }
                )
            }
        }
    }
}


@Composable
fun AddTaskDialog(
    title: String,
    description: String,
    priority: Priority,
    onTitleChange: (String) -> Unit,
    onDescriptionChange: (String) -> Unit,
    onPriorityChange: (Priority) -> Unit,
    onDismiss: () -> Unit,
    onConfirm: () -> Unit
) {
    AlertDialog(
        onDismissRequest = onDismiss,
        title = { Text("جدید کار افزودن") },
        text = {
            Column(verticalArrangement =
            Arrangement.spacedBy(10.dp)) {
                OutlinedTextField(
                    value = title,
                    onValueChange = onTitleChange,
                    label = { Text("عنوان") },
```

```
                singleLine = true,
                modifier = Modifier.fillMaxWidth()
            )
            OutlinedTextField(
                value = description,
                onValueChange = onDescriptionChange,
                label = { Text("توضیحات (اختیاری)") },
                modifier = Modifier.fillMaxWidth()
            )
            Text("اولویت:", style =
            MaterialTheme.typography.bodyMedium)
            Row(
                modifier = Modifier.fillMaxWidth(),
                horizontalArrangement =
                Arrangement.SpaceBetween
            ) {
                Priority.entries.forEach { p ->
                    FilterChip(
                        selected = priority == p,
                        onClick = {
                        onPriorityChange(p) },
                        label = {
                            Text(
                            when (p) {
                            Priority.LOW -> "کم"
                            Priority.MEDIUM -> "متوسط"
                            Priority.HIGH -> "مهم"
                                }
                            )
                        }
                    )
                }
            }
        }
    },
    confirmButton = {
        Button(
            onClick = onConfirm,
            enabled = title.isNotBlank()
        ) {
            Text("ذخیره")
        }
    },
    dismissButton = {
        TextButton(onClick = onDismiss) {
            Text("انصراف")
        }
    }
```

```
        )
}
...
```

هدف کلی AddTaskDialog چیست؟ این کامپوننت یک دیالوگ افزودن کار جدید است. یعنی کاربر یک فرم کوچک می‌بیند، عنوان، توضیحات و اولویت را وارد می‌کند. می‌تواند ذخیره یا انصراف بزند. دیالوگ روی صفحه ظاهر می‌شود و توجه کاربر را جلب می‌کند.

برنامه را اجرا کنید. کار تمام است.



تم در جت پک کامپوز

قبلا با تم آشنا و روش کار با آن را فرا گرفتید. اعمال تم در کامپوز کمی متفاوت است. همانطور که می‌دانید تم ظاهر کلی و یکپارچه‌ی اپلیکیشن را مشخص می‌کند. در واقع کار تم تعریف موارد زیر است می‌کند:

- رنگ اصلی اپ چه باشد؟

- رنگ پس‌زمینه چیست؟
- متن‌ها چه رنگ و اندازه‌ای داشته باشند؟
- فونت برنامه چیست؟
- اپ در حالت Dark / Light چگونه دیده شود؟
- شکل ظاهری بعضی کامپوننت‌ها چگونه باشد؟

در کامپوز پس از ساخت پروژه معمولاً فایل تم در مسیر ui/theme/Theme.kt قرار دارد. آن را باز کنید. فایل شما چیزی شبیه به لیست زیر است.

```
package com.codeb.tasktracker.ui.theme

import ...

private val DarkColorScheme = darkColorScheme(
    primary = Purple80,
    secondary = PurpleGrey80,
    tertiary = Pink80
)

private val LightColorScheme = lightColorScheme(
    primary = Purple40,
    secondary = PurpleGrey40,
    tertiary = Pink40
)

@Composable
fun TaskTrackerTheme(
    darkTheme: Boolean = isSystemInDarkTheme(),
    // Dynamic color is available on Android 12+
    dynamicColor: Boolean = true,
    content: @Composable () -> Unit
) {
    val colorScheme = when {
        dynamicColor && Build.VERSION.SDK_INT >=
            Build.VERSION_CODES.S -> {
            val context = LocalContext.current
            if (darkTheme) dynamicDarkColorScheme(context)
                else dynamicLightColorScheme(context)
        }
        darkTheme -> DarkColorScheme
        else -> LightColorScheme
    }
```

```
    MaterialTheme(
        colorScheme = colorScheme,
        typography = Typography,
        content = content
    )
}
```

این قطعه کد، ساختار اصلی تم و سیستم طراحی (Design System) برنامه را در Jetpack Compose بر اساس استاندارد متریال دیزاین ۳ تنظیم می‌کند. هدف این کد، مدیریت هوشمند حالت‌های تیره (Dark Mode)، روشن (Light Mode) و رنگ‌های پویای اندروید است.

بخش‌های تشکیل‌دهنده آن عبارتند از:

۱. پالت‌های رنگی ثابت (DarkColorScheme و LightColorScheme): این دو متغیر، رنگ‌های برنامه‌ریزی‌شده برای حالت شب و روز را مشخص می‌کنند. DarkColorScheme رنگ‌های روشن‌تر (مثل Purple80) را برای تم تیره اختصاص می‌دهد تا روی پس‌زمینه تاریک، کنتراست و خوانایی کافی داشته باشند و LightColorScheme رنگ‌های تیره‌تر (مثل Purple40) را برای تم روشن انتخاب می‌کند تا روی پس‌زمینه سفید یا روشن کاملاً مشخص باشند.

۲. تابع پوسته اصلی (TaskTrackerTheme): این یک کامپوز سفارشی است که تمام صفحات برنامه درون آن قرار می‌گیرند تا ظاهر برنامه‌ریزی‌شده به آن‌ها اعمال شود. این تابع سه ورودی می‌گیرد:

- darkTheme: وضعیت تم تیره را بررسی می‌کند. مقدار پیش‌فرض آن isSystemInDarkTheme() است که تم را به‌طور خودکار با تنظیمات گوشی کاربر هماهنگ می‌سازد.
- dynamicColor: فعال‌سازی قابلیت «رنگ پویا» (اعمال‌شده از اندروید ۱۲ به بعد).
- content: تمام المان‌های گرافیکی و رابط کاربری برنامه که قرار است زیرمجموعه این تم باشند.

۳. منطق انتخاب رنگ (colorScheme) :بر اساس یک شرط سه مرحله‌ای (when)، ترکیب رنگی نهایی انتخاب می‌شود. شرط اول (Dynamic Color) که اگر قابلیت رنگ پویا فعال باشد و نسخه اندروید دستگاه ۱۲ به بالا (Build_VERSION_CODES.S) باشد، رنگ‌های برنامه به صورت هوشمند از تصویر پس‌زمینه (Wallpaper) گوشی کاربر استخراج می‌شوند. شرط دوم (Dark Theme) که اگر دستگاه از رنگ پویا پشتیبانی نکند اما تم شب فعال باشد، از پالت

DarkColorScheme استفاده می‌شود. شرط سوم (Light Theme) که در غیر این صورت، پالت پیش‌فرض روز یا LightColorScheme اعمال می‌گردد.

۴. اعمال نهایی (MaterialTheme): در نهایت، پالت رنگی انتخاب‌شده همراه با فونت‌ها و تایپوگرافی برنامه (typography = Typography) به کامپوننت پایه MaterialTheme داده می‌شود. با این کار، تمامی دکمه‌ها، متن‌ها و اجزای برنامه به‌طور یکپارچه تغییر ظاهر می‌دهند.

اگر به MainActivity برگردید، خواهید دید که در قسمت setContent تابع UI یعنی TaskTrackerScreen داخل تابعی دیگری با نام TaskTrackerTheme فراخوانی می‌شود.

```
setContent {
    TaskTrackerTheme {
        TaskTrackerScreen()
    }
}
```

این تابع دقیقا همان تابع داخل فایل theme.kt است. با این روش است که تم اعمال می‌شود.

ساختار تم

در سیستم Jetpack Compose، سه فایل Color.kt، Type.kt و Theme.kt پایه‌های اصلی طراحی بصری (Design System) برنامه را تشکیل می‌دهند. این سه فایل بر اساس اصول Material Design 3 تنظیم می‌شوند تا ظاهر برنامه کاملاً یکدست و منسجم باشد. هر سه این فایل در داخل پکیج theme قرار دارد.

فایل Color.kt (فایل تعریف رنگ‌ها) مخزن خام رنگ‌های برنامه است. در این فایل، تمام کد رنگ‌هایی که قرار است در برنامه استفاده شوند، به عنوان متغیر تعریف می‌شوند.

```
val Purple80 = Color(0xFFD0BCFF)
val PurpleGrey80 = Color(0xFFCCC2DC)
val Pink80 = Color(0xFFEFB8C8)

val Purple40 = Color(0xFF6650a4)
val PurpleGrey40 = Color(0xFF625b71)
val Pink40 = Color(0xFF7D5260)
```

رنگ‌ها در این فایل معنای ساختاری ندارند و فقط نامگذاری می‌شوند؛ یعنی مشخص نیست که کدام رنگ قرار است برای پس‌زمینه باشد و کدام برای متن.

فایل Type.kt (فایل تنظیم فونت‌ها و تایپوگرافی) مسئول مدیریت سبک متن‌ها (Typography) در سراسر برنامه است. اندازه متن‌ها، ضخامت قلم (Font Weight)، فاصله حروف و نوع فونت در این بخش تعیین می‌شوند.

خلاصه ارتباط این سه فایل

فایل	نقش اصلی	مثال ملموس
Color.kt	مواد اولیه (مداد رنگی‌ها)	این رنگ، بنفش روشن با کد0xFFD0BCFF است
Type.kt	قالب قلم و ساختار متن	متن‌های اصلی باید اندازه ۱۶ sp و فونت معمولی داشته باشند
Theme.kt	تصمیم‌گیرنده و اجراکننده	در حالت روز از مداد بنفش تیره و در حالت شب از مداد بنفش روشن استفاده کن

۳۲

سخن پایانی

تبریک می‌گوییم! شما به انتهای این آموزش رسیدید. همه افراد نظم و انضباط لازم برای انجام کاری را که شما انجام داده‌اید ندارند. لحظه‌ای وقت بگذارید و به خودتان افتخار کنید.

سخت کوشی شما نتیجه داده است: شما اکنون یک توسعه دهنده اندروید هستید.

چالش نهایی

ما یک چالش نهایی برای شما داریم: تبدیل شدن به یک توسعه دهنده اندروید خوب. توسعه دهندگان خوب هر کدام به روش خود خوب هستند، بنابراین شما باید مسیر خود را از اینجا به بعد پیدا کنید.

از کجا شروع کنید؟ در اینجا چند مکان وجود دارد که ما توصیه می‌کنیم:

کد بنویسید؛ از همین حالا. اگر آن را به کار نگیرید، به سرعت آنچه را که در اینجا آموخته‌اید فراموش خواهید کرد. در یک پروژه مشارکت کنید یا یک برنامه ساده برای خود بنویسید. هر کاری که انجام می‌دهید، وقت را تلف نکنید: کد بنویسید.

یاد بگیرید. شما در این کتاب کمی‌در مورد چیزهای زیادی یاد گرفته‌اید.ایا هیچ یک از آنها تخیل شما را برانگیخت؟ چند کد بنویسید تا با چیز مورد علاقه خود بازی کنید. پیدا کردن و خواندن بیشتر مستندات مربوط به آن – یا یک کتاب کامل، اگر وجود داشته باشد. همچنین، کانال YouTube Android Developers (youtube.com/user/androiddevelopers) را بررسی کنید و به پادکست Android Developers Backstage (androidbackstage.blogspot.com) برای به روزرسانی‌های اندروید از Google گوش دهید.

با مردم ملاقات کنید. جلسات محلی مکان خوبی برای ملاقات با توسعه دهندگان همفکر است. بسیاری از توسعه دهندگان درجه یک اندروید در توییتر فعال هستند. در کنفرانس‌های اندروید شرکت کنید تا با سایر توسعه دهندگان اندروید (مانند ما!) ملاقات کنید.

جامعه منبع باز را کاوش کنید. توسعه اندروید در github.com در حال انفجار است. وقتی یک کتابخانه جالب پیدا کردید، ببینید مشارکت کنندگان آن به چه پروژه‌های دیگری متعهد هستند. کد خود را نیز به

اشتراک بگذارید – شما هرگز نمی‌دانید چه کسی آن را مفید یا جالب می‌داند. ما لیست پستی هفتگی اندروید (androidweekly.net) را راهی عالی برای دیدن آنچه در جامعه اندروید اتفاق می‌افتد می‌دانیم.